Data Structures and its Applications

V R BADRI PRASAD
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

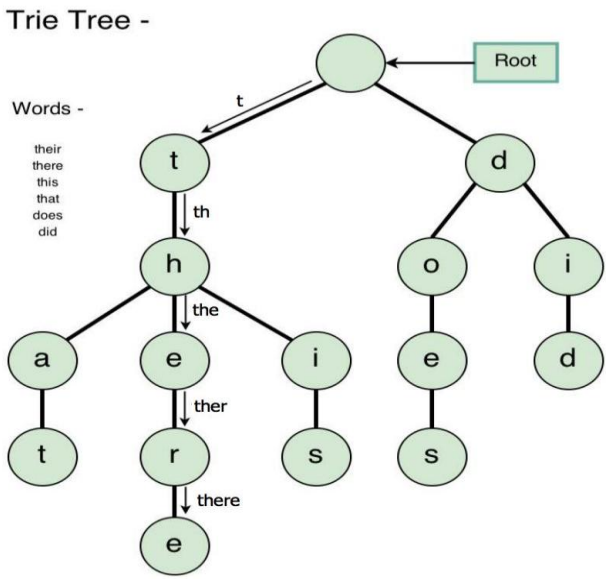
Introduction to TRIE Trees

V R BADRI PRASAD
Department of Computer Science & Engineering

TRIE Trees – An Introduction

- TRIE tree is a digital search tree, need not be implemented as a binary tree.
- Each node in the tree can contain ‘m’ pointers – corresponding to ’m‘ possible symbols in each position of the key.
- Generally used to store strings.

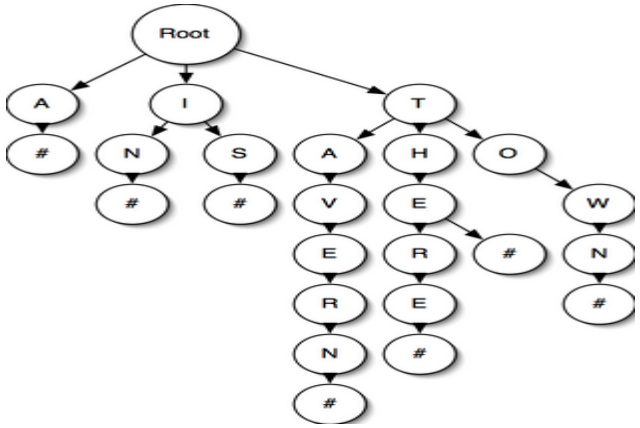
Examples:



Data Structures and its Applications

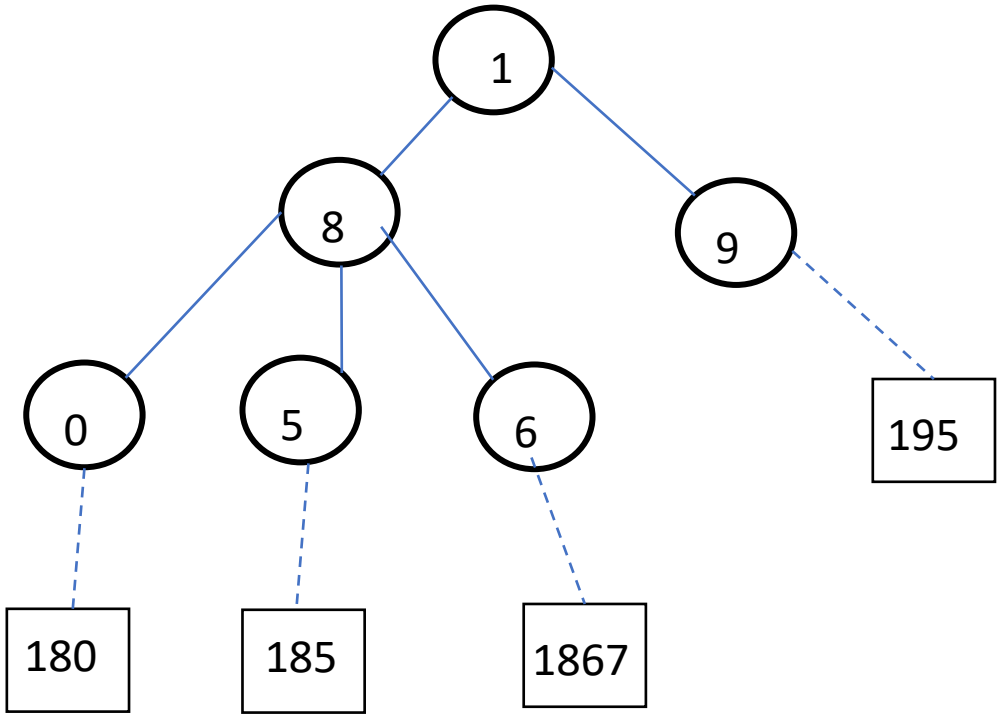
TRIE Trees – An Introduction

- A trie, pronounced “try”, is a tree that exploits some structure in the keys
  - e.g. if the keys are strings, a binary search tree would compare the entire strings but a trie would look at their individual characters
  - A trie is a tree where each node stores a bit indicating whether the string spelled out to this point is in the set
  - **Examples:**



TRIE Trees – Numeric Keys : Example2

- If the keys are numeric, there would be 10 pointers in a node.

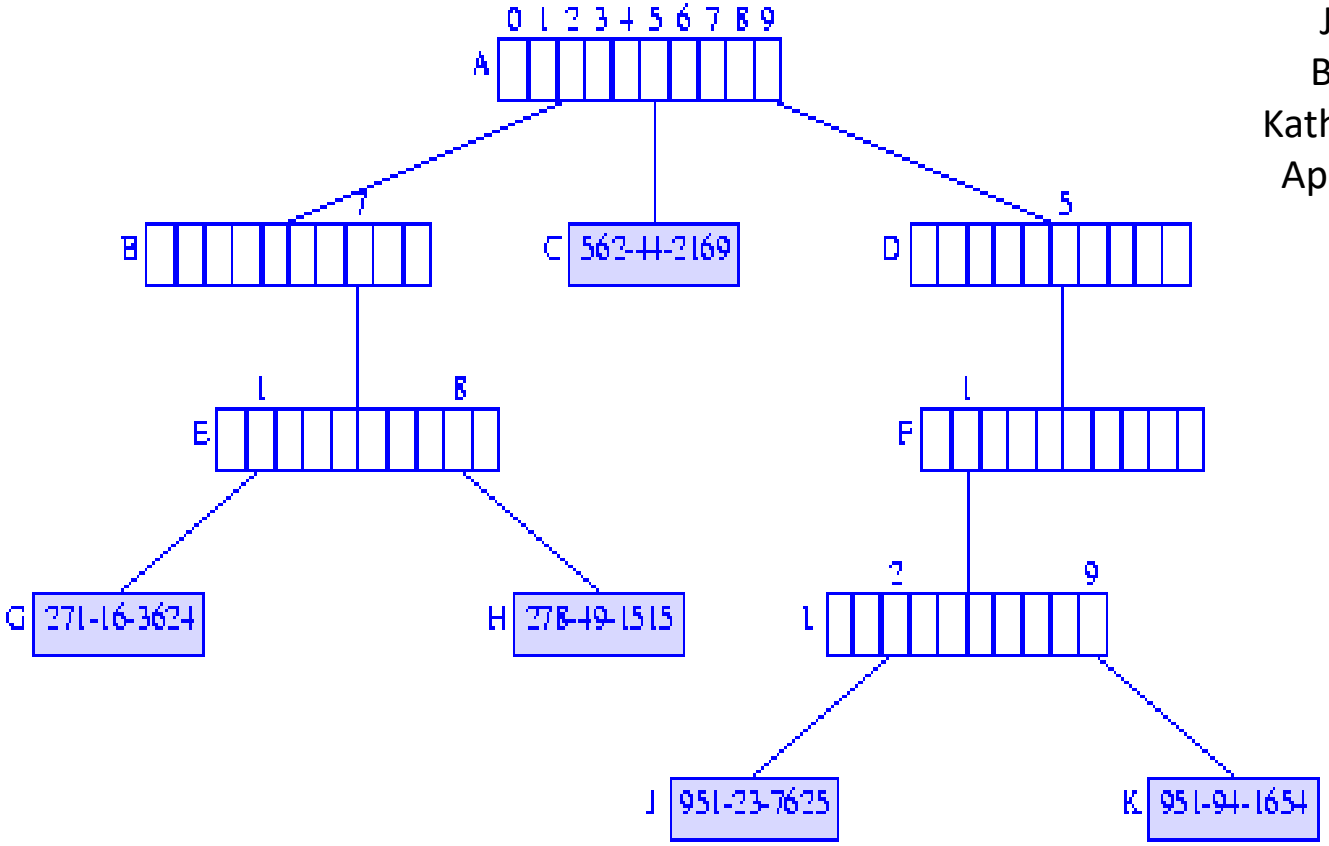


Data Structures and its Applications

TRIE Trees – Numeric Keys : Example1

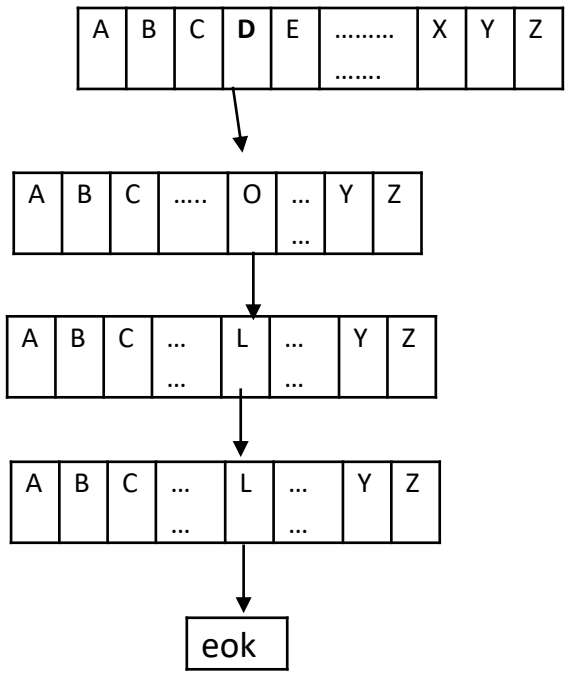
- If the keys are numeric, there would be 10 pointers in a node.
- Consider the SSN number as shown.

Name	**Social Security Number (SS#)**
Jack	951-94-1654
Jill	562-44-2169
Bill	271-16-3624
Kathy	278-49-1515
April	951-23-7625



TRIE Trees – An Introduction

- If the keys are Alphabetic, there would be 26 pointers.



Ex: The word DOLL has been stored as shown in the figure.

TRIE Trees – An Introduction

- An extra pointer corresponding to eok (end of key) or a flag with each pointer indicating that it point to a record rather than to a tree node. (normally $ symbol is used).
- A pointer in the node is associated with a particular symbol value based on its position in the node.
  - First pointer corresponds to the lowest value.
  - Second pointer to the second lowest and so forth.
- This way of implementation of a digital search tree is called a **TRIE** tree.
- The word ***TRIE*** is extracted from re***trie***val word.

TRIE Trees – Structure

- Tries are extremely special and useful data-structure that are based on the *prefix of a string*.
- Strings are stored in a top to bottom manner on the basis of their prefix in a TRIE.
- All prefixes of length 1 are stored at until level 1, all prefixes of length 2 are sorted at until level 2 and so on.

Suffix Trie:

- Suffix Trie is a space-efficient data structure to store a string that allows many kinds of queries to be answered quickly.
- Example:
  Text is "**banana\$**" where '\$' is the string terminating character.

Suffix Trie – Building

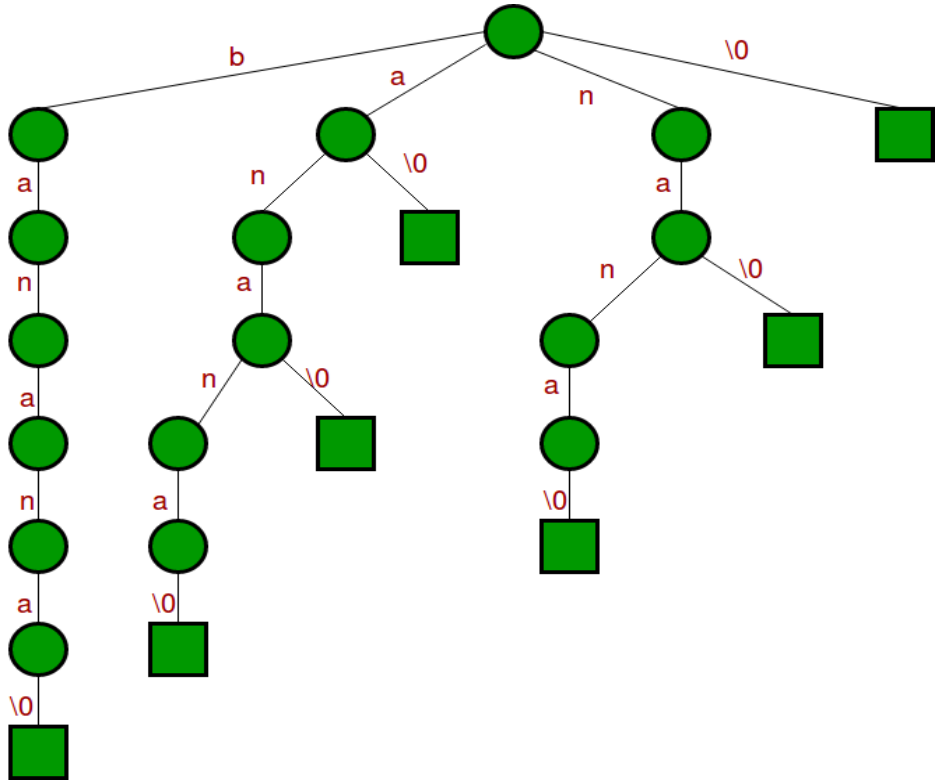
- A Trivial Algorithm for building a suffix tree.
  - Step1 : Generate all suffixes of a given text
  - Step2: Consider all suffixes as individual words and build a compressed trie.

- Example1:
  Text is "**banana\$**" where '\$' is the string terminating character.

Following are the suffixes of Text

"banana/$"
"anana/$"
"nana/$"
"ana/$"
"na/$"
"a/$"
"/$"

Suffix trie



Suffix Trie – Building

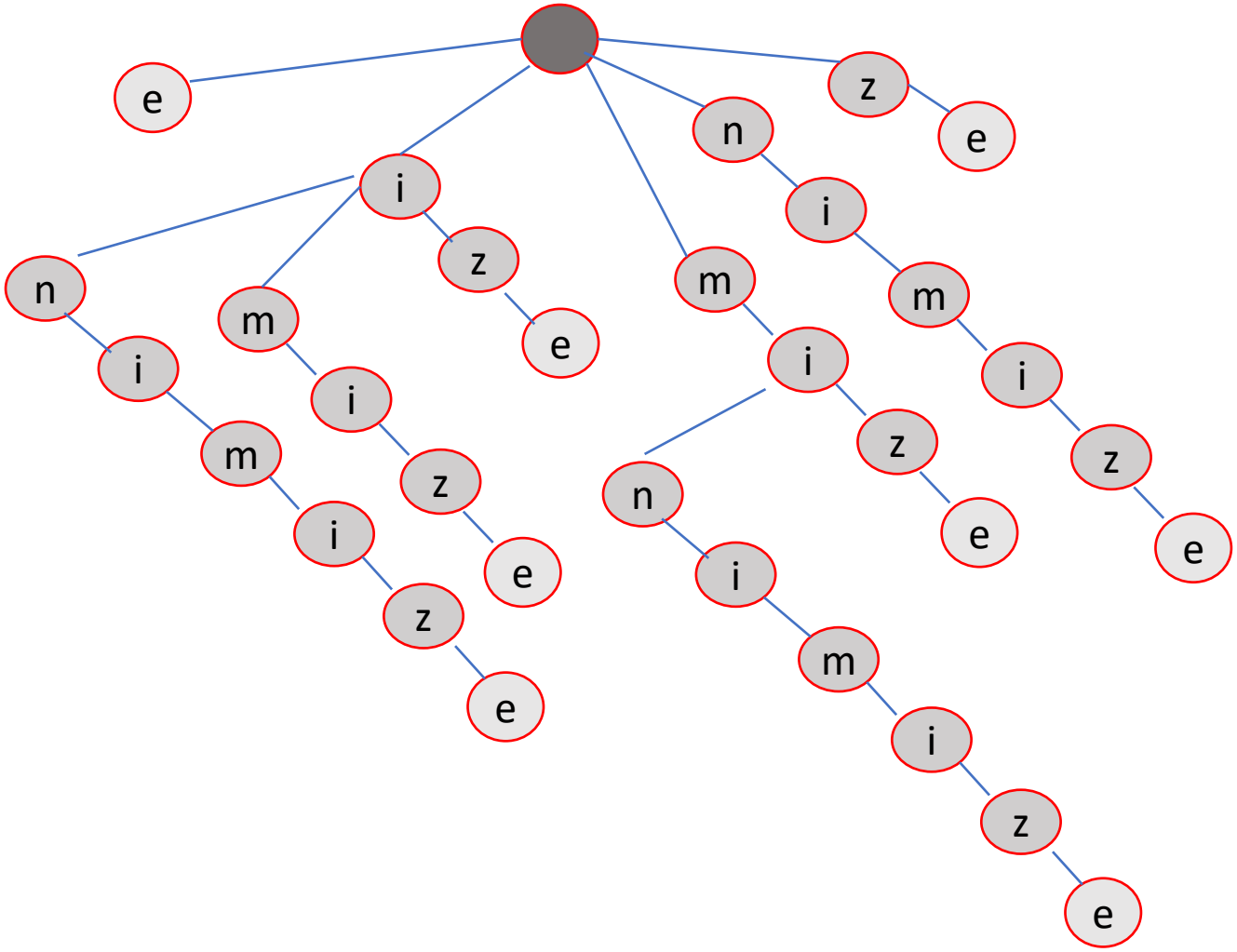
Example 2 : Generate the suffix trie for the word **minimize**

Step1. Generate all the suffixes of the word **minimize**.

e
ze
ize
mize
imize
nimize
inimize
minimize

S - set of strings to include in the suffix trie.

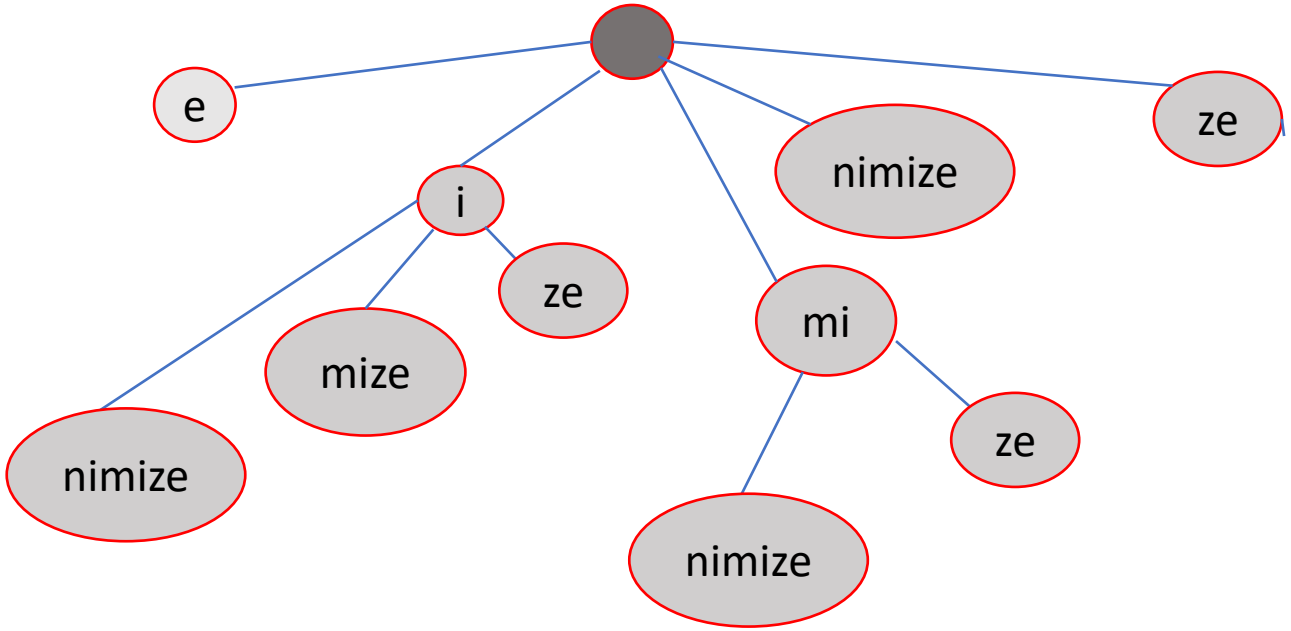
Suffix Trie – Building - for the word **minimize**



e
ze
ize
mize
imize
nimize
inimize
minimize

S - set of strings to include in the suffix trie

Suffix Trie – Compressed Trie

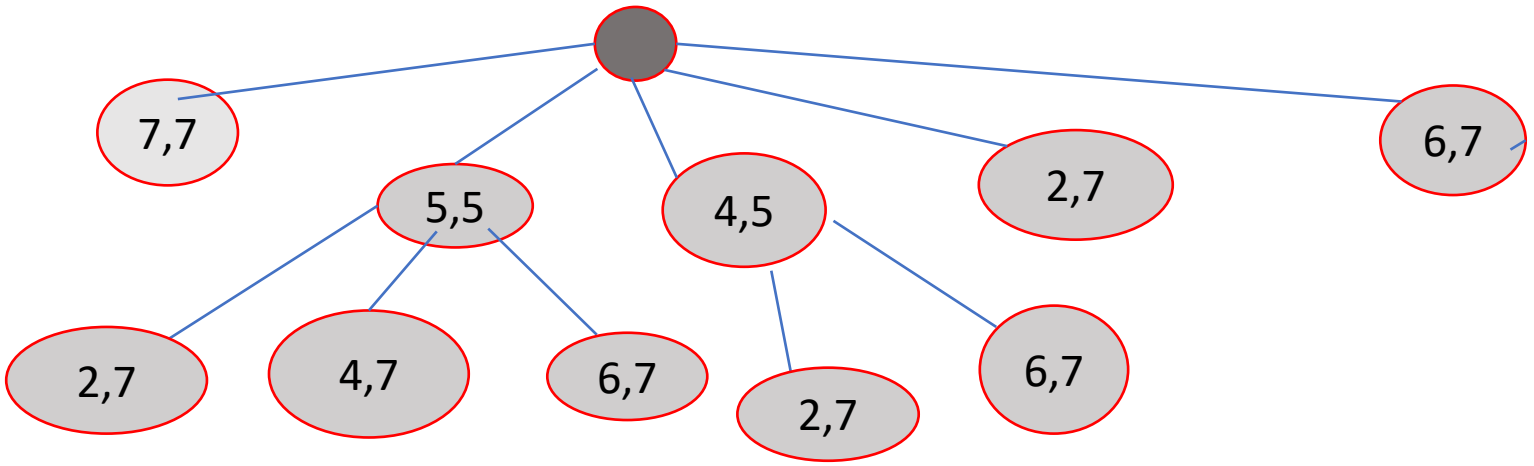


Data Structures and its Applications

Suffix Trie – Compressed Trie – using numbers

- Representation of Compressed trie using numbers - (**Indexes**)
- The indexes of the word is ...

0	1	2	3	4	5	6	7
m	**i**	**n**	**i**	**m**	**i**	**z**	**e**



Suffix Trees – Introduction

- A simple data structure for string searching
- It's a compressed Trie Tree
- Allow many fast implementations of many important string operations

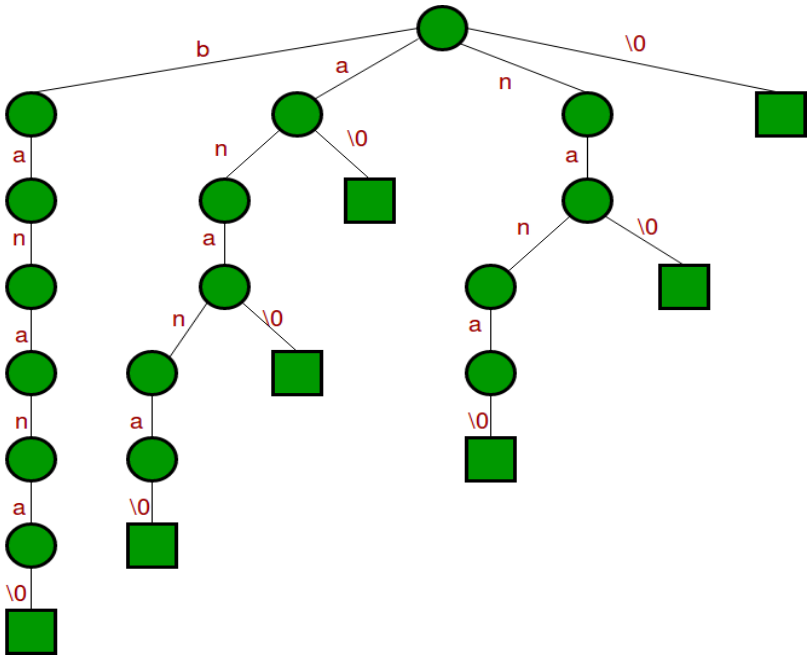
- Properties of a suffix trees:
  - ✓ A suffix tree for a text X of size n from an alphabet of size d.
  - ✓ Stores all the n(n-1) suffixes of X.
  - ✓ Supports arbitrary pattern matching and prefix matching queries

Example – Banana$

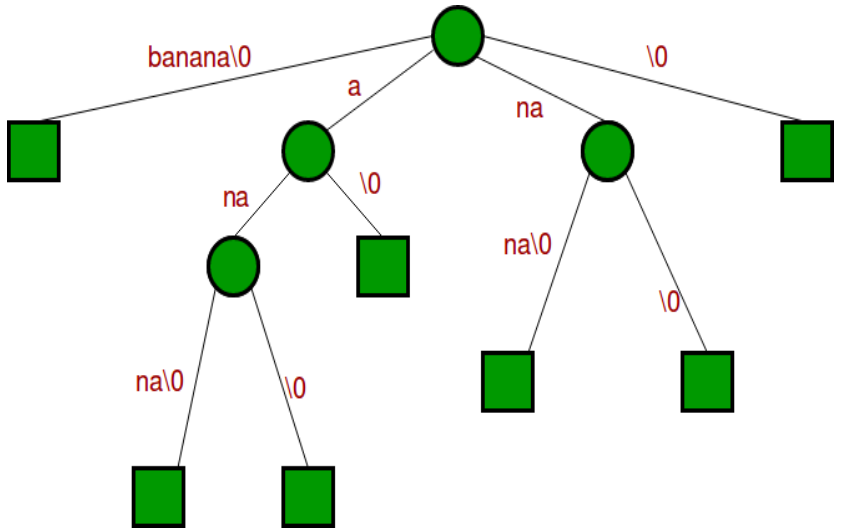
Data Structures and its Applications

Suffix Trees – Introduction

- Join chains of single nodes,to get the following compressed trie, which is the Suffix tree for given text “banana\0”



Suffix TREE



Data Structures and its Applications

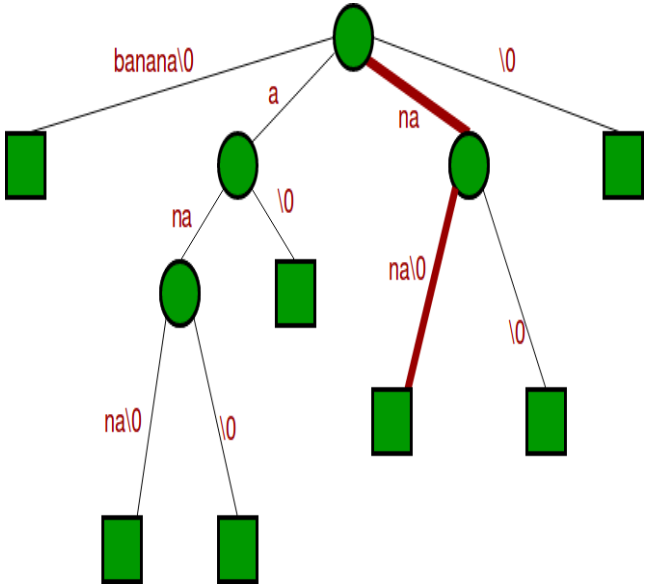
Search for a substring in a Suffix Tree

1) Starting from the first character of the pattern and root of Suffix Tree, do following for every character.

i) For the current character of pattern, if there is an edge from the current node of suffix tree, follow the edge.

ii) If there is no edge, print “pattern doesn’t exist in text” and return.

2) If all characters of pattern have been processed, i.e., there is a path from root for characters of the given pattern, then print “Pattern found”.



TRIE Trees – Applications, advantages and disadvantages

PES UNIVERSITY ONLINE

Applications:

- English dictionary
- Predictive text
- Auto-complete dictionary found on Mobile phones and other gadgets.

Advantages:

- Faster than BST
- Printing of all the strings in the alphabetical order easily.
- Prefix search can be done (Auto complete).

Disadvantages:

- Need for a lot of memory to store the strings,
- Storing of too many node pointers.

THANK YOU

V R BADRI PRASAD

Department of Computer Science & Engineering

badriprasad@pes.edu

DATA STRUCTURES & ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science & Engineering

DATA STRUCTURES & ITS APPLICATIONS

Trie, Suffix Tree

Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES & ITS APPLICATIONS

Trie

- In computer science, a trie, also called digital tree or prefix tree, is a type of search tree, a tree data structure used for locating specific keys from within a set
- These keys are most often strings, with links between nodes defined not by the entire key, but by individual characters

- Common **applications** of tries include storing a predictive text or autocomplete dictionary and implementing approximate matching algorithms, such as those used in spell checking
- Such applications take advantage of a trie's ability to quickly search for, insert, and delete entries

https://en.wikipedia.org/wiki/Trie

Trie

Construct a Trie for the words: algorithm, data, datum, all, self, sea, search

PES UNIVERSITY
CELEBRATING 50 YEARS

Trie

Applications:
- Dictionary
- Auto-complete feature found on Search Engine, Mobile Phone

Advantages:
- Strings can be printed in the alphabetical order easily
- Prefix search can be done (Auto complete)

Disadvantages:
- Need for a lot of memory to store the strings because of storing many node pointers

Suffix Tree

Suffix Trie: A trie containing all the suffixes of the given data

Suffix Tree: A compressed suffix Trie

Suffix Tree

Construct a suffix trie for the word banana and show its suffix tree.

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE21CS252A

Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Collision Resolution
Quadratic Probing, Double Hashing

Kusuma K V
Department of Computer Science & Engineering

Hashing: Quadratic Probing

Quadratic Probing (Open addressing, closed hashing) resolves collision by using the below formula:

h(key) = (h(key) + i^2) % tableSize

where i = 1, 2, 3, …

Hashing: Quadratic Probing

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7. Use key%tableSize as the hash function & quadratic probing to resolve collision.

h(7) = 7 % 7 = 0

h(20) = 20 % 7 = 6

h(41) = 41 % 7 = 6 collision

Resolving collision: h(41)=((6+1^2)%7)=0, h(41)=((6+2^2)%7)=3

h(31) = 31 % 7 = 3 collision

Resolving collision: h(31)=((3+1^2)%7)=4

h(18) = 18 % 7 = 4 collision

Resolving collision: h(18)=((4+1^2)%7)=5

h(8) = 8 % 7 = 1

h(9) = 9 % 7 = 2

0	1	2	3	4	5	6
7	8	9	41	31	18	20

Hashing: Double Hashing

Double Hashing (Open addressing, closed hashing) resolves collision by using a second hash function whenever there results a collision. The below formula is used:

hash(key) = (hash1(key) + i * hash2(key)) % tableSize

Here hash1() and hash2() are hash functions and tableSize is the size of hash table, i = 1, 2, …

Hashing: Double Hashing

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use **key%tableSize** as the **first hash function** & double hashing (closed hashing) to resolve collision. Use **key%5** as the **second hash function**.

0	1	2	3	4	5	6
7	41		31	18		20

h(7) = 7 % 7 = 0

h(20) = 20 % 7 = 6

h(41) = 41 % 7 = 6 collision

Resolving collision: 2nd hash function: h(41) = 41 % 5 = 1

Therefore, h(41) = (6 + 1*1) % 7 = 0, h(41) = (6 + 2*1) % 7 = 1

h(31) = 31 % 7 = 3

h(18) = 18 % 7 = 4

Hashing: Double Hashing

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use **key%tableSize** as the **first hash function** & double hashing (closed hashing)to resolve collision. Use **key%5** as the **second hash function**.

contd..,

0	1	2	3	4	5	6
7	41	8	31	18	9	20

h(8) = 8 % 7 = 1 collision

Resolving collision: 2nd hash function: h(8) = 8 % 5 = 3

Therefore, h(8) = (1 + 1*3) % 7 = 4, h(8) = (1 + 2*3) % 7 = 0, h(8) = (1 + 3*3) % 7 = 3

h(8) = (1 + 4*3) % 7 = 6, h(8) = (1 + 5*3) % 7 = 2

h(9) = 9 % 7 = 2 collision

Resolving collision: 2nd hash function: h(9) = 9 % 5 = 4

Therefore, h(9) = (2 + 1*4) % 7 = 6, h(9) = (2 + 2*4) % 7 = 3, h(9) = (2 + 3*4) % 7 = 0

h(9) = (2 + 4*4) % 7 = 4, h(9) = (2 + 5*4) % 7 = 1, h(9) = (2 + 6*4) % 7 = 5

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE21CS252A

Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Collision Resolution
Linear Probing

Kusuma K V
Department of Computer Science & Engineering

Hashing: Linear Probing

Linear Probing (open addressing, closed hashing) resolves collision by finding the next vacant spot in the hash table, in other words, it uses the below formula to resolve collision:

h(key) = (h(key) + i) % tableSize

where i = 1, 2, 3, …

Hashing: Linear Probing

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use key%tableSize as the hash function & linear probing(closed hashing)to resolve collision.

h(7) = 7 % 7 = 0

h(20) = 20 % 7 = 6

h(41) = 41 % 7 = 6 collision,

0	1	2	3	4	5	6
7	41	8	31	18	9	20

Resolving collision: h(41)=((6+1)%7)=0, h(41)=((6+2)%7)=1

h(31) = 31 % 7 = 3

h(18) = 18 % 7 = 4

h(8) = 8 % 7 = 1 collision

Resolving collision: h(8)=((1+1)%7)=2

h(9) = 9 % 7 = 2 collision

Resolving collision: h(9)=((2+1)%7)=3, h(9)=((2+2)%7)=4, h(9)=((2+3)%7)=5

Hashing: Linear Probing

```
void initTable(NODE ht[],int *n)
{
        for(int i=0;i<SIZE;i++)
                ht[i].mark=-1;

        *n=0;
}
```

Hashing: Linear Probing

```
int insertRecord(NODE ht[],int rNo,char name[],int *n) {
        if(SIZE==*n)
                return 0;
        int index=rNo%SIZE;             //hash function
        while(ht[index].mark!=-1)
                index=(index+1)%SIZE;
        ht[index].rNo=rNo;
        strcpy(ht[index].name,name);
        ht[index].mark=1;
        (*n)++;
        return 1;
}
```

Hashing: Linear Probing

```
int deleteRecord(NODE ht[],int rNo,int *n){
   if(*n==0)
         return 0;

   int index=rNo%SIZE;       //hash function
   int i=0;
   while(ht[index].rNo!=rNo && i<*n)
   {
         index=(index+1)%SIZE;
         if(ht[index].mark==1)
                  i++;
   }
   if(ht[index].rNo==rNo && ht[index].mark==1)
   {
         ht[index].mark=-1;
         (*n)--;
         return 1;
   }
   return 0;
}
```

Hashing: Linear Probing

```
int searchRecord(NODE ht[],int rNo,int n){
    if(n==0)
            return 0;

    int index=rNo%SIZE;        //hash function
    int i=0;
    while(ht[index].rNo!=rNo && i<n)
    {
            index=(index+1)%SIZE;
            if(ht[index].mark==1)
                    i++;
    }
    if(ht[index].rNo==rNo && ht[index].mark==1)
    {
            strcpy(name,ht[index].name);
            return 1;
    }
    return 0;
}
```

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE21CS252A

Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Collision Resolution
Separate Chaining

Kusuma K V
Department of Computer Science & Engineering

Hashing: Separate Chaining

Separate Chaining (Open hashing) handles collision by making every hash table cell point to linked lists of records with identical hash function values.

Hashing: Separate Chaining

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use key%tableSize as the hash function and separate chaining (open hashing) to resolve collision.

h(7) = 7 % 7 = 0

h(20) = 20 % 7 = 6

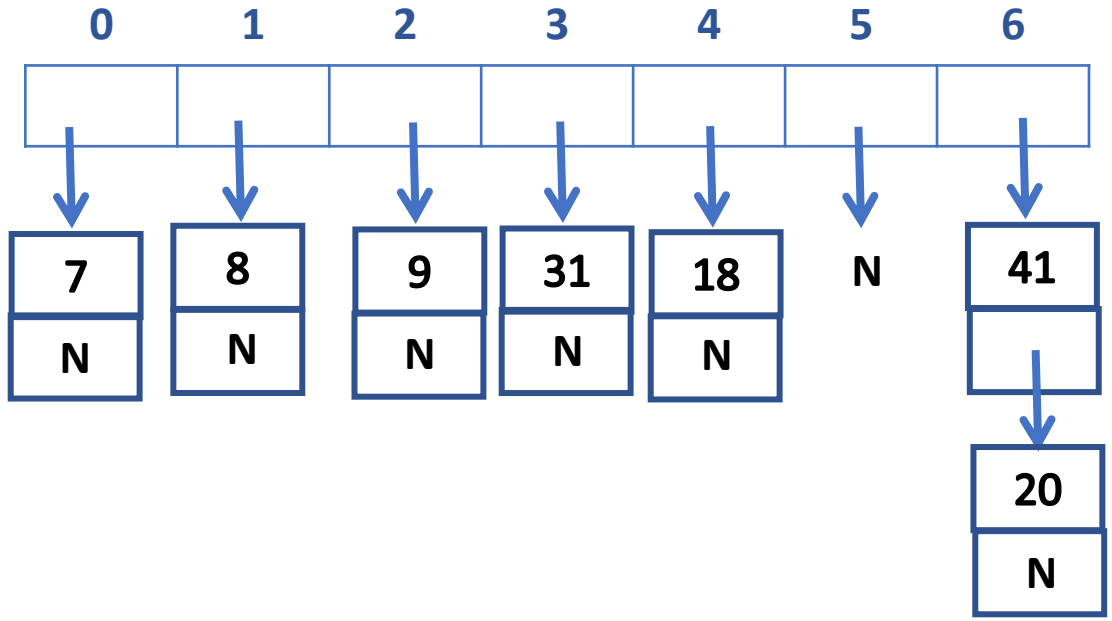
h(41) = 41 % 7 = 6 collision

h(31) = 31 % 7 = 3

h(18) = 18 % 7 = 4

h(8) = 8 % 7 = 1

h(9) = 9 % 7 = 2



Hashing: Separate Chaining

```
typedef struct element
{
        int rNo;
        char name[30];
        struct element *next;
}NODE;
```

Hashing: Separate Chaining

```
void initTable(NODE* ht[SIZE])
{
        for(int i=0;i<SIZE;i++)
                ht[i]=NULL;
}
```

Hashing: Separate Chaining

```
void insert(NODE* ht[SIZE],int rNo,char name[30])
{
        int index=rNo%SIZE;     //hash function

        NODE *newNode=malloc(sizeof(NODE));
        newNode->rNo=rNo;
        strcpy(newNode->name,name);
        newNode->next=ht[index];

        ht[index]=newNode;
}
```

Hashing: Separate Chaining

```
int delete(NODE* ht[SIZE],int rNo)
{
        int index=rNo%SIZE;  //hash function

        NODE *p=ht[index];
        NODE *q=NULL;

        while(p!=NULL && p->rNo!=rNo)
        {
                q=p;
                p=p->next;
        }
        if(p!=NULL)
        {
                if(q==NULL)
                        ht[index]=p->next;
                else
                        q->next=p->next;

                free(p);
                return 1;
        }
        return 0;
}
```

Hashing: Separate Chaining

```
int search(NODE* ht[SIZE],int rNo,char name[30]) {
        int index=rNo%SIZE;     //hash function
        NODE *p=ht[index];
        while(p!=NULL) {
                if(p->rNo==rNo) {
                        strcpy(name,p->name);
                        return 1;
                }
                p=p->next;
        }
        return 0;
}
```

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE21CS252A

Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing

Kusuma K V
Department of Computer Science & Engineering

Hashing

Time taken for search

- Linear DS: Array List, Linked List --- O(n)
- Non linear DS: Balanced Binary Search Tree --- O(log n)
- Can we search in O(1) time ??
    - Hashing

Hashing

Hashing is the process of transforming any given key into another value.

This is usually represented by a shorter, fixed-length value or key that makes it easier to find the original key.

The most popular use for hashing is the implementation of hash tables.

A hash table stores key and value pairs in a list that is accessible through its index.

Hashing

Direct Mapping

Eg: Employee Records

Hashing

Hash function: A function which maps key value into a hash table range

- Folding
- Truncation
- Modulo

Hashing

Collision: The phenomenon when two or more keys generate the same hash.

Collision resolution:

- Open addressing (Closed Hashing)
- Separate Chaining (Open Hashing)

Hashing

Collision resolution:

- Open addressing (Closed Hashing)
- Open addressing handles collisions by storing all data in the hash table itself and then seeking out availability in the next spot created by the algorithm.
    - Linear Probing, Quadratic Probing, Double Hashing

- Separate Chaining (Open Hashing)
- Separate chaining handles collision by making every hash table cell point to linked lists of records with identical hash function values.

Hashing

Collision resolution:

- Rehashing (Hash again)
  - Increase the table size when the load factor becomes more than a predefined value (Say 0.75) and hash all the keys present in the table again and store in the new table of increased size.

  Load factor=No. of filled records/Total capacity

References

- Live lecture Videos on Hashing for the course UE19CS202, Data Structures, by Dr. Shylaja S S
- What is hashing and how does it work? (techtarget.com)

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Saritha
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Finding all the paths from a given source to destination

Saritha
Department of Computer Science & Engineering

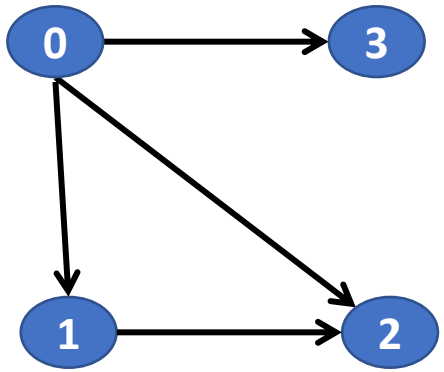
Applications of BFS and DFS

Application of DFS

- Detecting whether a cycle exist in graph.
- Finding a path in a network
- Topological Sorting: Used for job scheduling
- To check whether a graph is strongly connected or not

Path

- Sequence of edges that allows the user to go from vertex A to vertex B



- The paths from vertex 0 to vertex 2:
- 1.0->1->2
- 2.0->2

Finding all the paths from the given source to destination

- **Methodology**
- 1.Start from any traversal Method from a given source node
- 2.Store all the visited vertices in an array
- 3.Once the destination vertex is reached, print all the contents of Array.

Function to print all the paths using DFS traversal method

```
//Function to print all the paths from a given source to destination
void path_find(int source,int destination)
{
    int visited[10]  //An array to store the vertices as visited or not
    int path[10]    //An array to store the path
    int count=0;
    for(int i=0;i<n;i++)
        visited[i]=0   //initilize all the vertices as not visited.
    printallpaths(source,destination,visited,path);
}
```

Function to print all the paths using DFS traversal method

```
void printallpaths(int u,int d,int visited[10],int path[10])
{
   visited[u]=1;//Mark the current node and and store it in the array path
   path[count]=u;
   count++;
   if(u==d) //if the current vertex is same as the destination then print the array
which has stored the path
    {
      for(i=0;i<count;i++)
            printf("%d",path[i]);
     }
   else // if the current vertex is not the destination
   {
     for(NODE temp=a[u];temp!=NULL;temp=temp->link)
```

Function to print all the paths using DFS traversal method

```
if(!visited[temp->data])
{
   printallpaths(temp->data,d,visited,path);
}
}
count--  //Remove the current vertex from the path[] and mark it as
unvisited
Visited[u]=0;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Function to read adjacency list

```
void read_adjacency_list(NODE a[],int n)
{
   int ele,m;
   for(int i=0;i<n;i++)
   {
      printf("enter the number of nodes adjacent to %d:",i);
       scanf("%d",&m);
       if(m==0)
        continue;
       printf("enter the nodes adjacent to %d:",i);
       for(int j=0;j<m;j++)
      {
        scanf("%d",&ele);
       a[i]=insert_rear(ele,a[i]); // insert at rear end
       }   } }
```

Function to insert a node at rear end of the list

Function to insert a node at rear end

```
NODE insert_rear(int v,NODE head)
 {
    NODE temp;
    NODE cur;
    temp=getnode(); // create a node
    temp->info=v;
     temp->link=NULL;
    if(head==NULL)
          return temp;
     cur=head;
   while(cur->link!=NULL)
   {
     cur=cur->link;
    }
   cur->link=temp;
 return(head); }
```

Connectivity of the graph using Adjacency list

Function to create a node

```
NODE getnode()
 {
  NODE temp;
  temp=(NODE)malloc(sizeof(struct node)); //Dynamic allocation
  if(temp==NULL)
  {
     printf("out of memory");
      return;
   }
  return temp;
 }
```

THANK YOU

Saritha

Department of Computer Science & Engineering

Saritha.k@pes.edu

9844668963

Data Structures and its Applications

V R BADRI PRASAD
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of TRIE Trees

V R BADRI PRASAD
Department of Computer Science & Engineering

TRIE Trees – Implementation

Structure of a node in a TRIE Tree :

- A node of a TRIE tree is represented as shown below.
- One field for each alphabet(A – Z), 26 columns.
- Each column is a pointer to another TRIE node or carries NULL and
- One field for end of word (key).

A	B	C	D	E	F	.....	W	X	Y	Z
F1	F2	F3	F4	F5	F6	.....	F23	F24	F25	F26
End of Word / (eok)										

Address of the next node (reference for us)
Field number – for user's reference no field is created , No memory is allocated
End of word / key field

```
struct trienode
{
  struct trienode* child[26];
  int endofword;
};
```

TRIE Trees – Implementation

```
struct trienode
{
   struct trienode* child[26];
   int endofword;
};
```

A	B	C	D	E	F	.....	W	X	Y	Z
F1	F2	F3	F4	F5	F6	.....	F23	F24	F25	F26
End of Word / (eok - $)										

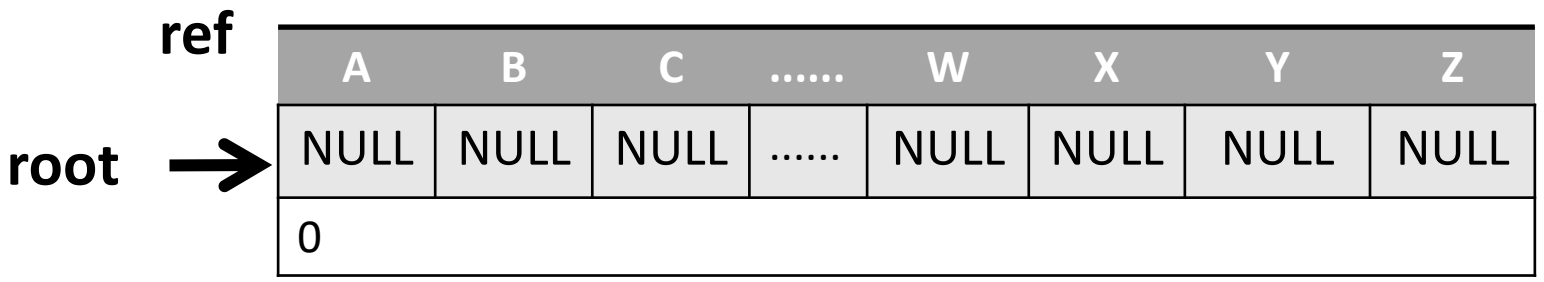
Address of the next node (reference for us)
Field number
End of word / key field

Data Structures and its Applications

TRIE Trees – Implementation

Creation of a node in a TRIE Tree using malloc() function.

```
struct trienode *getnode()
 {
   int i;
   struct trienode *temp;
    temp=(struct trienode*)(malloc(sizeof(struct trienode)));
    for(i=0;i<26;i++)
        temp->child[i]=NULL;      // Initialize all the fields to NULL.
    temp->endofword=0;     // Initialize endofword to 0.
   return temp;
 }
```



root=getnode();

TRIE Trees – Implementation

Insertion of a node in a TRIE Tree

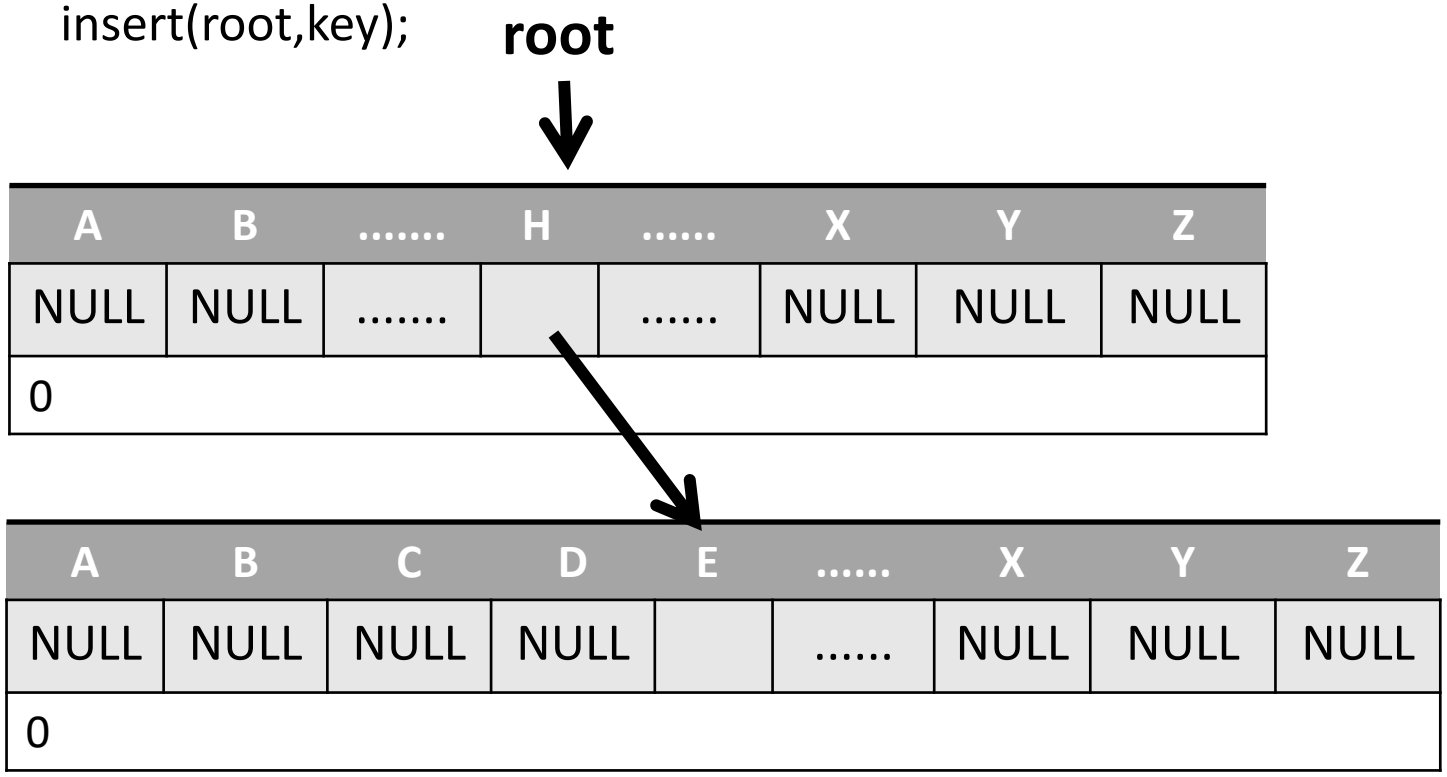
```
void insert(struct trienode *root, char *key)
 {
   struct trienode *curr;
   int i,index;

   curr=root;
   for(i=0;key[i]!='\0';i++)
   {
     index=key[i]-'a';
     if(curr->child[index]==NULL)
        curr->child[index]=getnode();
     curr=curr->child[index];
   }
   curr->endofword=1;
 }
```

TRIE Trees – Implementation

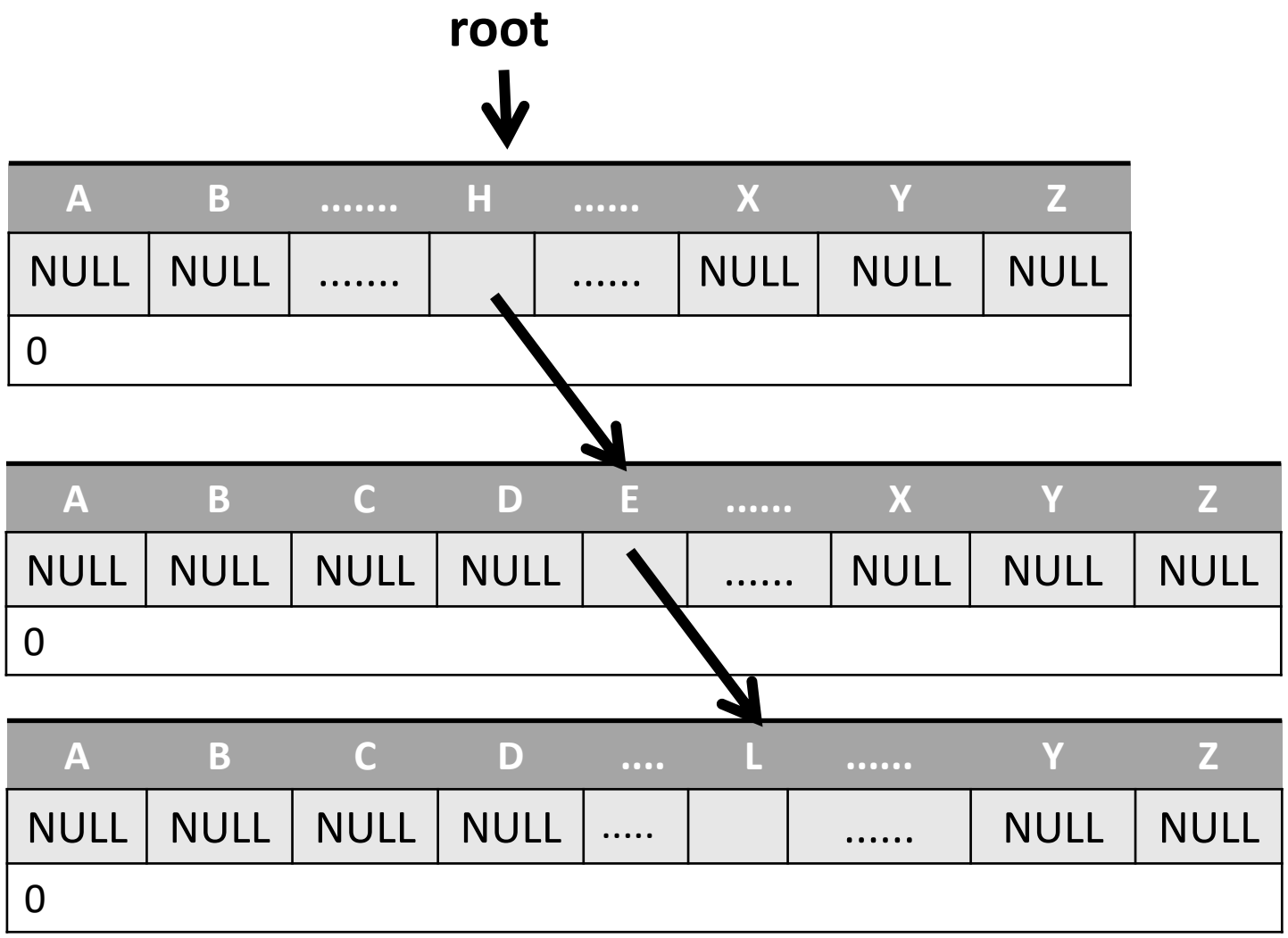
Insertion of a node in a TRIE Tree

On function call insert, the given string “HELLO” is inserted into the TRIE tree as shown below.

insert(root,key);



TRIE Trees – Implementation



THANK YOU

V R BADRI PRASAD

Department of Computer Science & Engineering

badriprasad@pes.edu

Data Structures and its Applications

V R BADRI PRASAD
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of TRIE Trees :

- Display Operation
- Deletion Operation
- Search Operation

V R BADRI PRASAD

Department of Computer Science & Engineering

TRIE TREES – Display function

```
void display(struct trienode *curr)
 {
  int i,j;
  for(i=0;i<255;i++)
  {
   if(curr->child[i]!=NULL)
   {
     word[length++]=i;
     if(curr->child[i]->endofword==1)//if end of word
     {
      printf("\n");
      for(j=0;j<length;j++)
        printf("%c",word[j]);
     }
    display(curr->child[i]);
     }
   }
  length--;
  return ;
}
```

Data Structures and its Applications

TRIE Trees – Deletion Operation

```
void delete_trie(struct trienode *root, char *key)
 {
   int i,index,k;
   struct trienode *curr;
   struct stack x;
   curr=root;

   for(i=0;key[i]!='\0';i++)
   {
     index=key[i];
     if(curr->child[index]==NULL)
     {
        printf("The word not found..\n");
        return;
      }
     push(curr,index);
     curr=curr->child[index];
   }
   curr->endofword=0;
   push(curr,-1);
```

TRIE Trees – Deletion Operation

```
while(1)
  {
    x=pop();
    if(x.index!=-1)
      x.m->child[x.index]=NULL;
    if(x.m==root)//if root
     break;
    k=check(x.m);
    if((k>=1)||(x.m->endofword==1))
      break;
    else
      free(x.m);
   }
  return;
 }
```

Data Structures and its Applications

TRIE Trees – Search Operation

```
int search(struct trienode * root,char *key)
 {

 int i,index;
  struct trienode *curr;
  curr=root;

  for(i=0;key[i]!='\0';i++)
  {
    index=key[i];

    if(curr->child[index]==NULL)
       return 0;
     curr=curr->child[index];
   }
   if(curr->endofword==1)
      return 1;
  return 0;
}
```

THANK YOU

V R BADRI PRASAD

Department of Computer Science & Engineering

badriprasad@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Saritha
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Connectivity of the graph

Saritha

Department of Computer Science & Engineering

Applications of BFS

Application of BFS

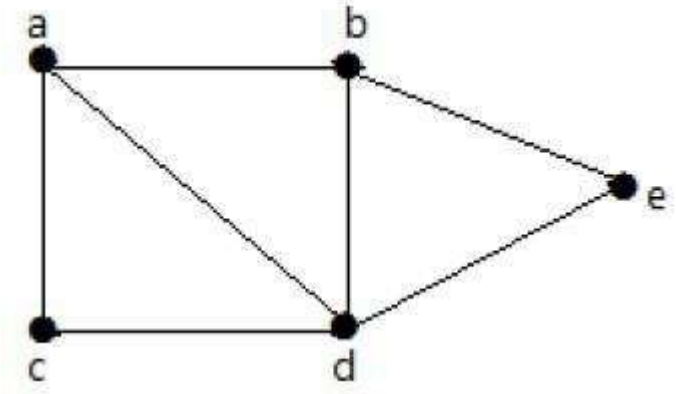
- Finding the shortest path
- Social Networking websites like twitter, Facebook etc.
- GPS Navigation system
- Web crawlers
- Finding a path in network
- In Networking to broadcast the packets

Connectivity of graph

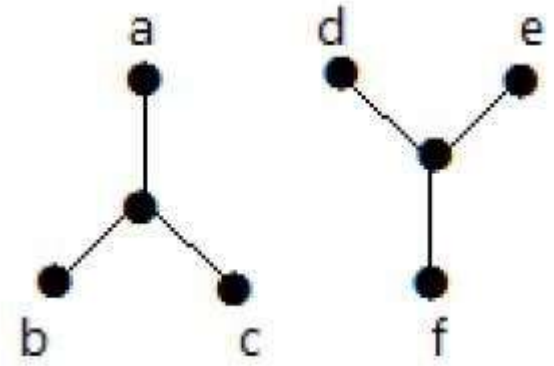
Connectivity of graph

- Connectivity refers to connection between two or more nodes or things

Connected graph



Disconnected graph

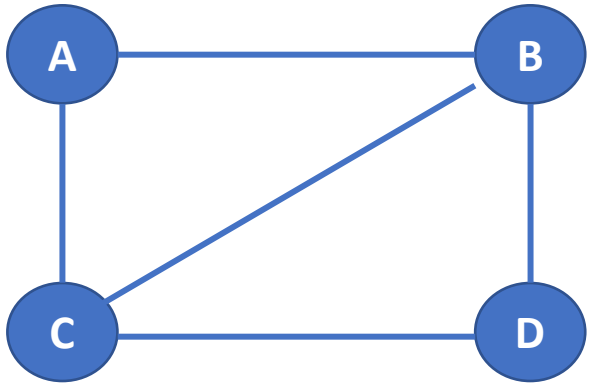


Connectivity of the graph

- A graph is connected if there is a path between every pair of vertices.
- The graph that is not connected is called disconnected graph.
- In connected graph there is no unreachable vertex.
- In the area of Information Technology the connectivity refers to internet connectivity through which various devices are connected to global network

Connectedness in the Direct graph using adjacency matrix

- To check whether a graph is connected or not, we traverse the graph using either bfs traversal or dfs traversal method
- After the traversal if there is at-least one node which is not marked as visited then that graph is disconnected graph
- For example: Below graph is connected



	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	1
D	0	1	1	0

Connectedness in the Direct graph using adjacency matrix

Procedure to check the whether a graph is connected or not using adjacency matrix

- Read the adjacency matrix .
- Create a visited [] array. Start DFS/BFS traversal method from any arbitrary vertex and mark the visited vertices in the visited[] array.
- Once DFS/BFS is completed check the visited [] array. if there is at-least one vertex which is marked as unvisited then the graph is disconnected otherwise it is connected.

Connectivity of the graph using Adjacency Matrix

Function to read adjacency matrix:

```
void read_ad_matr(int graph[][],int n)
{
   int i,j;
   for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
        {
          scanf("%d", &graph[i][j]);//reading the values into two dimensional array
         }
    }
}
```

Connectivity of the graph using Adjacency Matrix

Function to traverse the graph using DFS

```
void traverse(int u, int visited[])
{
   visited[u] = 1; //mark v as visited
   for(int v = 0; v<n; v++)
  {
     if(graph[u][v])
     {
       if(!visited[v])
       traverse(v, visited);
     }
  }
}
```

Connectivity of the graph using Adjacency Matrix

Function to traverse the graph using bfs:

```
void traverse(int s, int visited[],int n,int graph[][])
 {
   int f,r,v,q[10];
   f=0,r=-1;
   q[++r]=s;
   visited[s]=1;// mark the nodes as visited
   while(f<=r)
   {
      s=q[f++];
      for(v=0;v<n;v++)
     {
      if(graph[s][v]==1) //adjacent nodes
       {
```

Connectivity of the graph using Adjacency Matrix

```
    if(visited[v]==0) //nodes not visited
     {
       visited[v]=1; //mark as visited
       q[++r]=v;
     }
}
 }
   }
}
```

Connectivity of the graph using Adjacency Matrix

Function to check whether the graph is connected or not

```
int isConnected()
{
   int vis[NODE];   //for all vertex u as start point, check whether all nodes are visible or not
 for(int u=0; u < NODE; u++)
    {
      for(int i = 0; i<NODE; i++)
          vis[i] = 0; //initialize as no node is visited
      traverse(u, vis); // any traversal method either bfs or dfs
       for(int i = 0; i<NODE; i++)
       {
          if(!vis[i]) //if there is a node, not visited by traversal, graph is not connected
          return 0;
        }
    }
 return 1;}
```

THANK YOU

Saritha

Department of Computer Science & Engineering

Saritha.k@pes.edu

9844668963

DATA STRUCTURES AND ITS APPLICATIONS

Balanced Trees

Sandesh B. J
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Balanced Trees

Sandesh B. J
Department of Computer Science & Engineering

Balanced Trees

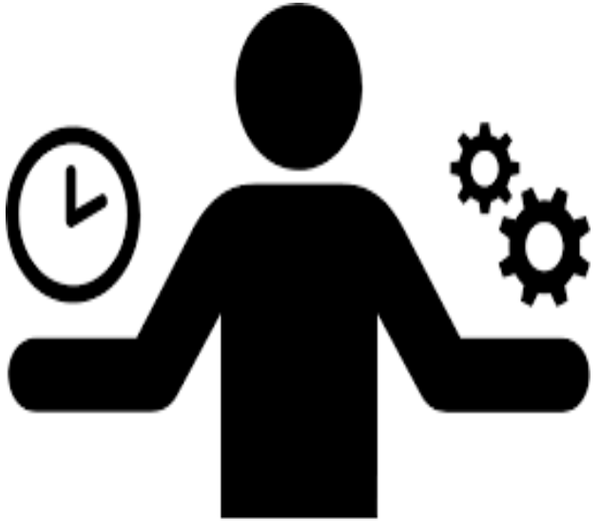
In this lecture you will be able to learn:

- **Why trees becomes unbalanced?**
- **Why we need to balance the tree?**
- **AVL Tree**
- **How do we balance the unbalanced trees using tree rotation techniques**
- **Different tree Rotation techniques**
  - ✓ **Left rotation, right rotation**
  - ✓ **Left-right rotation and right-left rotations**

Why Balanced Trees?

- Binary Search Tree(BST) – Data Structure used to implement the dictionary.

- What do we gain by implementing dictionary using BST instead of array ?

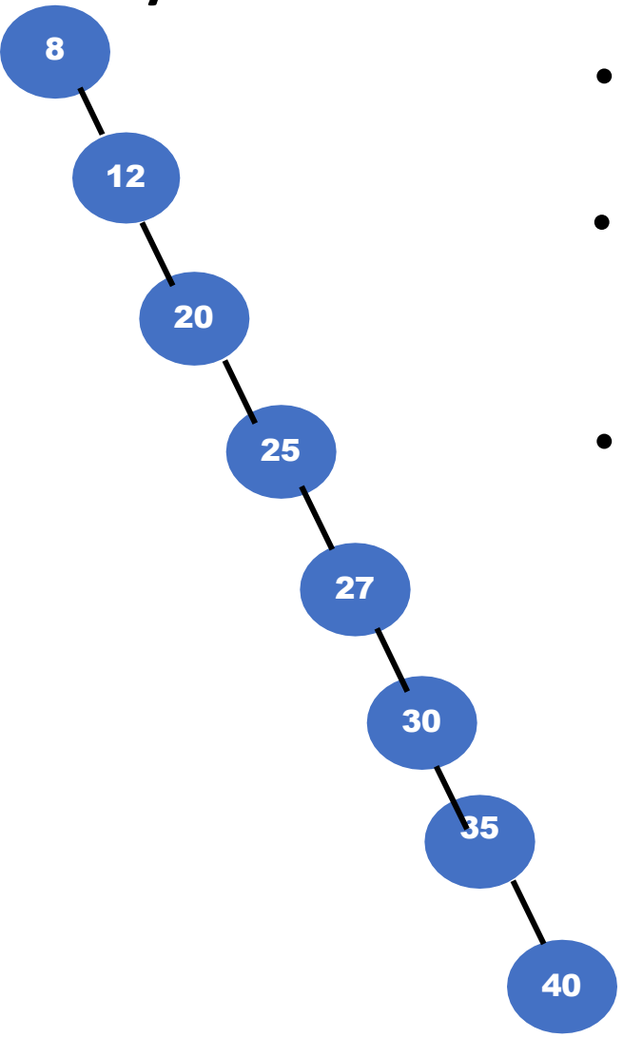


Complexity of Binary Search Tree

Operations	Average	Worst
Insert	O(log(n))	O(n)
Delete	O(log(n))	O(n)
Search	O(log(n))	O(n)

Balanced Trees

Why Balanced trees?

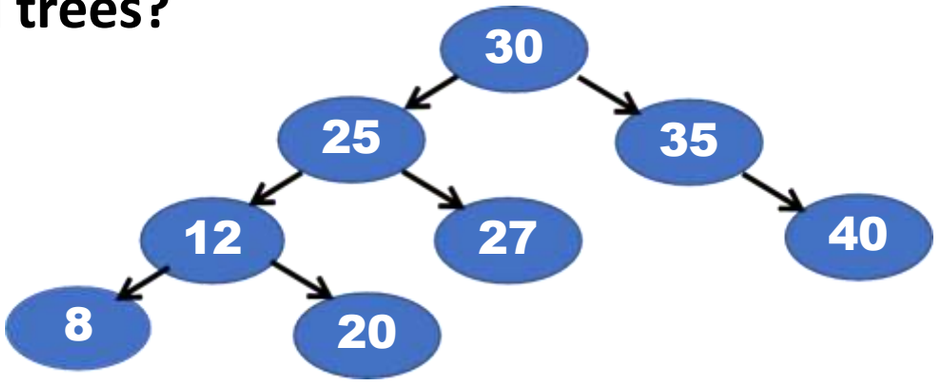


- Height of the BST depends on the order of insertion of elements into tree
- What happens if we insert elements given in increasing order?
  - Insert 8, 12, 20, 25, 27, 30, 35, 40
- Severely unbalanced

Disadvantages of Binary search trees

- The search and insertion algorithm does not ensure that the tree remains balanced
- The degree of balance dependent on the order in which the keys are inserted
- Tree can attain a height which can be as large as n-1
- Time taken for most of the operations worst case O(n)

Why Balanced trees?



- we can construct the binary search tree in which the height of the tree is always log(n)

Complexity of Balanced Binary Search Tree

Operations	Average	Worst
Insert	O(log(n))	O(log(n))
Delete	O(log(n))	O(log(n))
Search	O(log(n))	O(log()n)

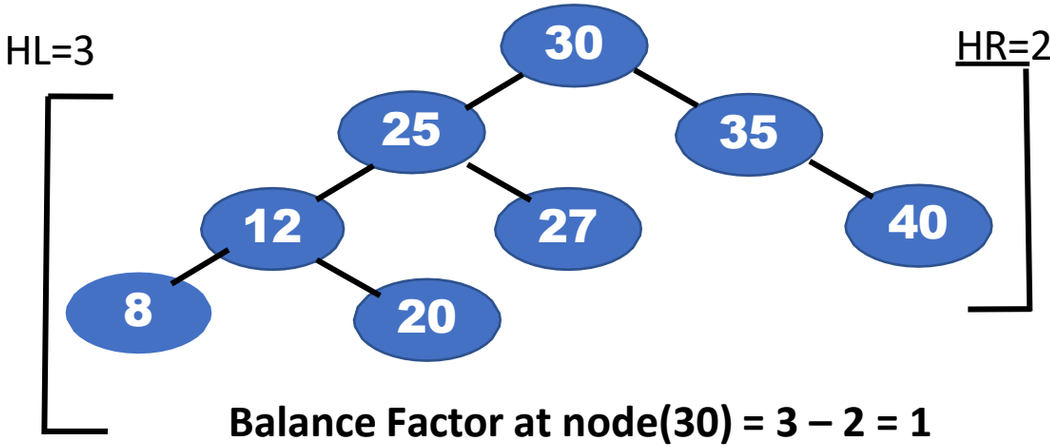
AVL Tree-Balanced Binary Search Trees

- AVL tree was invented in 1962 by two Russian mathematicians G.M. Adel'son-Vel'skii and E.M. Landis(AVL)

- An AVL tree is a binary search tree in which, for every node, the difference between the heights of the left and right subtrees, called the balance factor is either 0 or +1 or -1

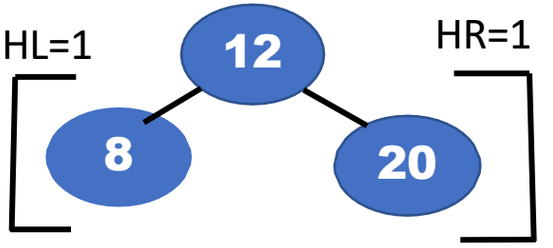
The Balance factor of any node:
Balance Factor = Height(left subtree) – Height(right subtree)

Balanced Binary Search Trees(AVL)

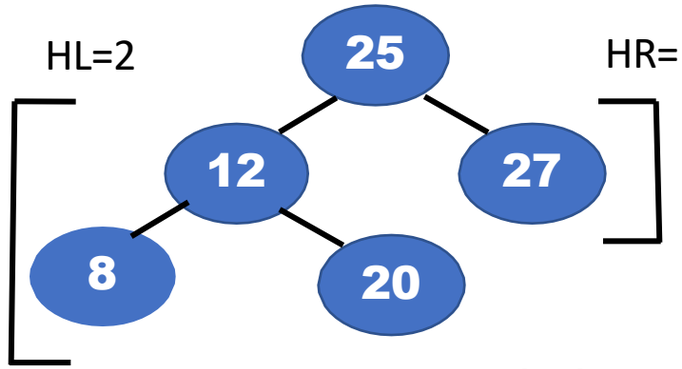


Balance Factor = HL - HR

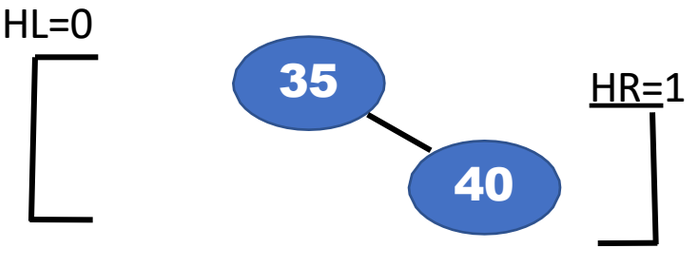
Balance Factor at node(30) = 3 – 2 = 1



Balance Factor at node(12) = 1– 1 = 0

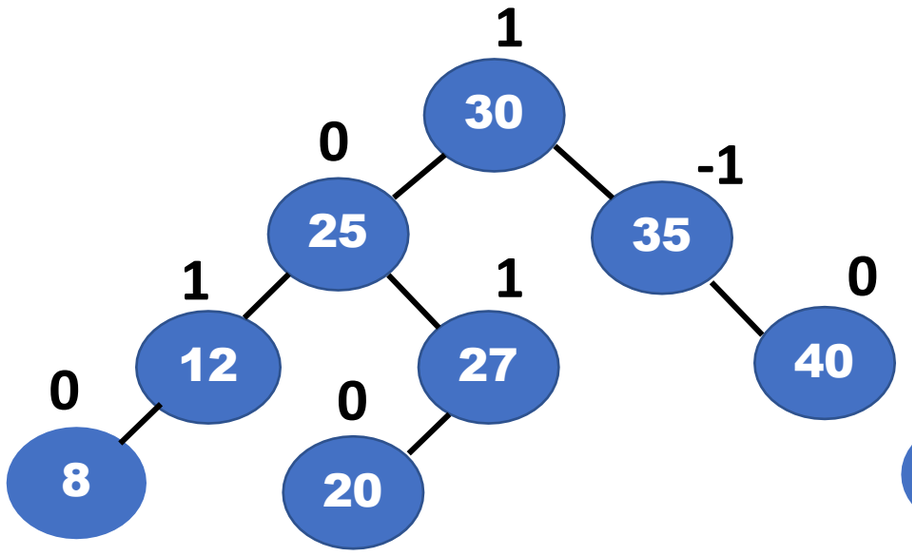


Balance Factor at node(25) = 2 – 1 = 1

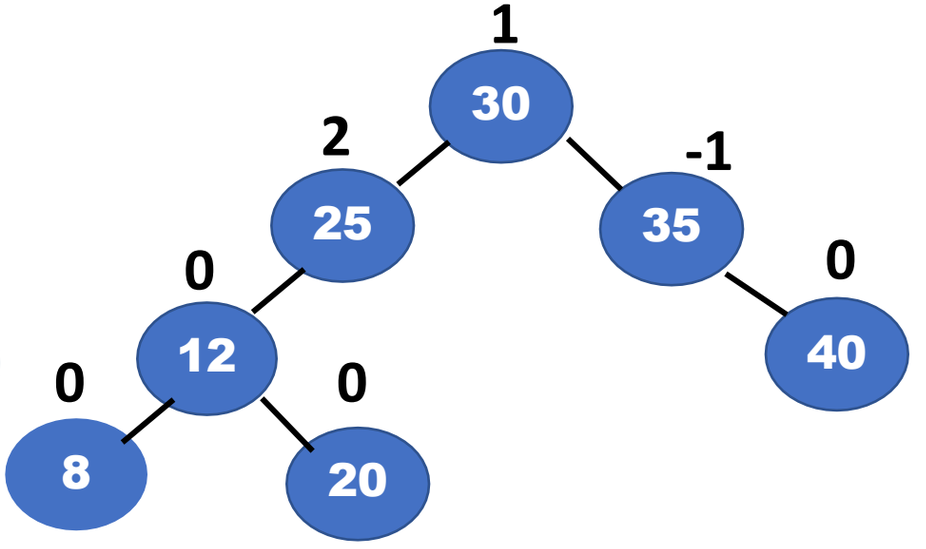


Balance Factor at node(35) = 0 – 1 = -1

Balanced Binary Search Trees(AVL)



AVL Tree



Not an AVL tree

- Node in balanced binary tree has balance of
  - 1 – Height(left subtree) > Height(right subtree)
  - 0 – Height(left subtree) = Height(right subtree)
  - -1 – Height(left subtree) < Height(right subtree)

Rotations – AVL Tree

- The AVL tree may become unbalanced after insert and delete operations

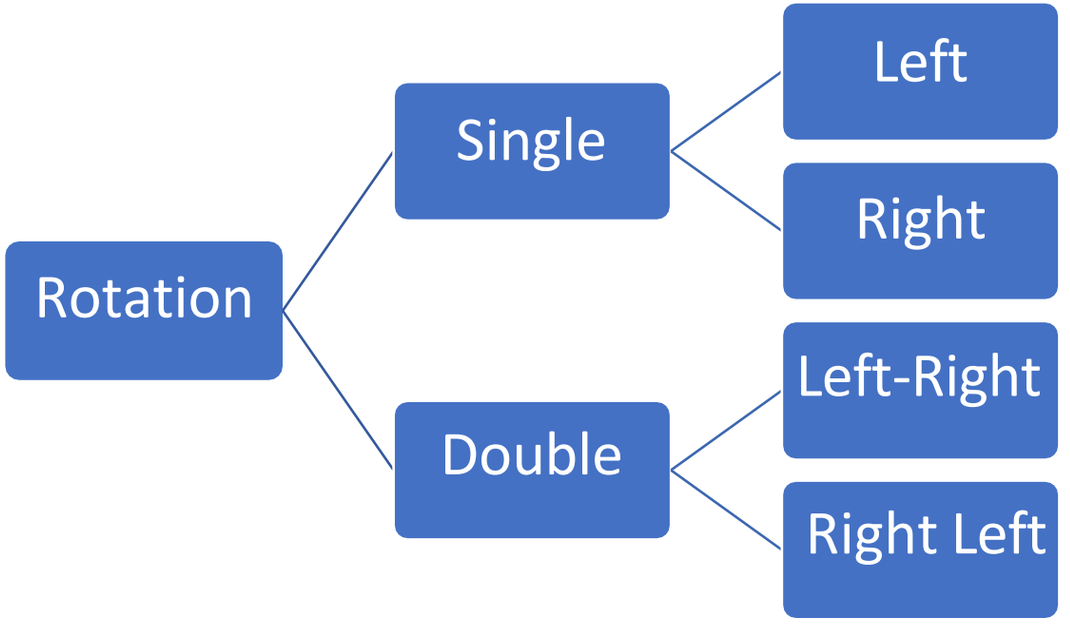
- If a key insertion violates the balance requirement at some node, the subtree rooted at that node is transformed via one of the four *rotations.*

- **Rotation** in a AVL tree is a local transformation of its subtree rooted at a node whose balance has become either +2 or -2

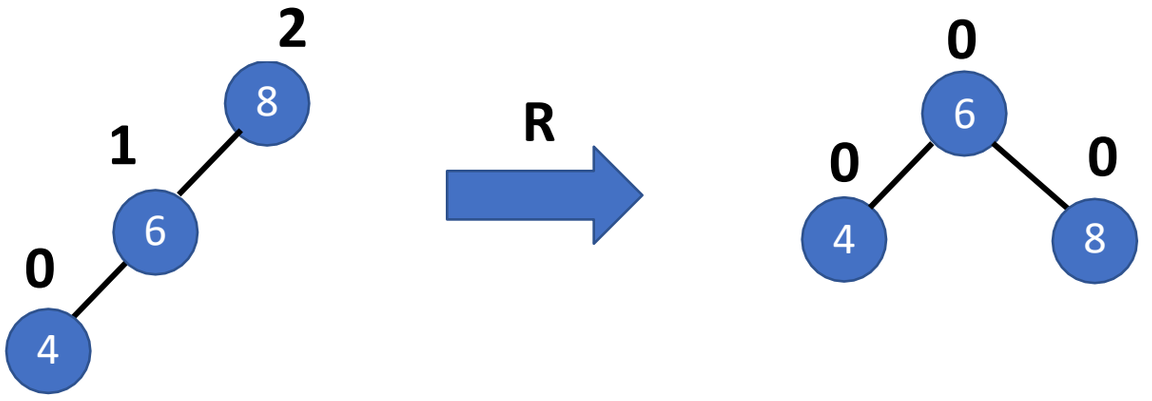
- In case there are several such nodes, The rotation is always performed for a subtree rooted at “unbalanced" node closest to the newly inserted leaf node.

Rotations – AVL Tree

- **Different types of Rotation:**



Single Right Rotation (R-Rotation)

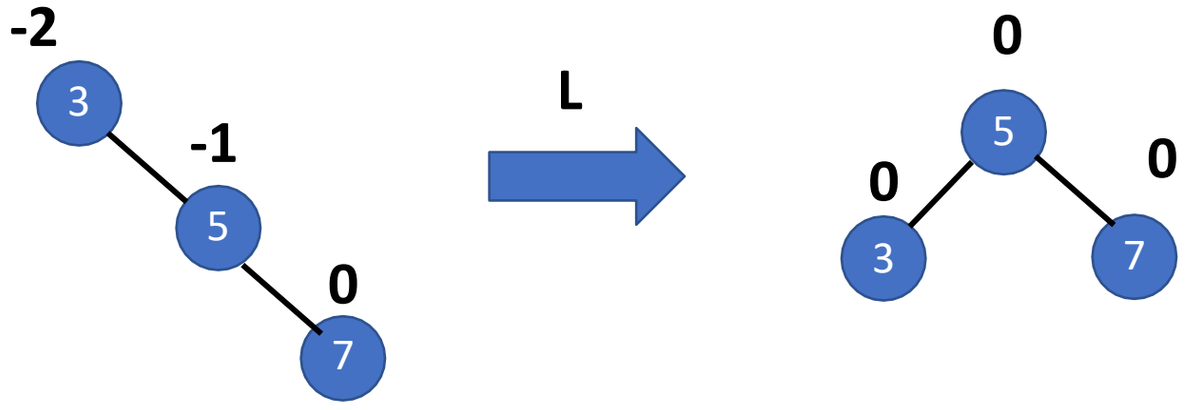


- Root of the tree has balance of +1 before the insertion
- New key is inserted to the left of left child (Left-Left-case)
- To Balance the tree we need to perform the right rotation

Courtesy: “Introduction to Design and Analysis of Algorithms” By Anany Levitin

Rotations - AVL Trees

Single Left Rotation:

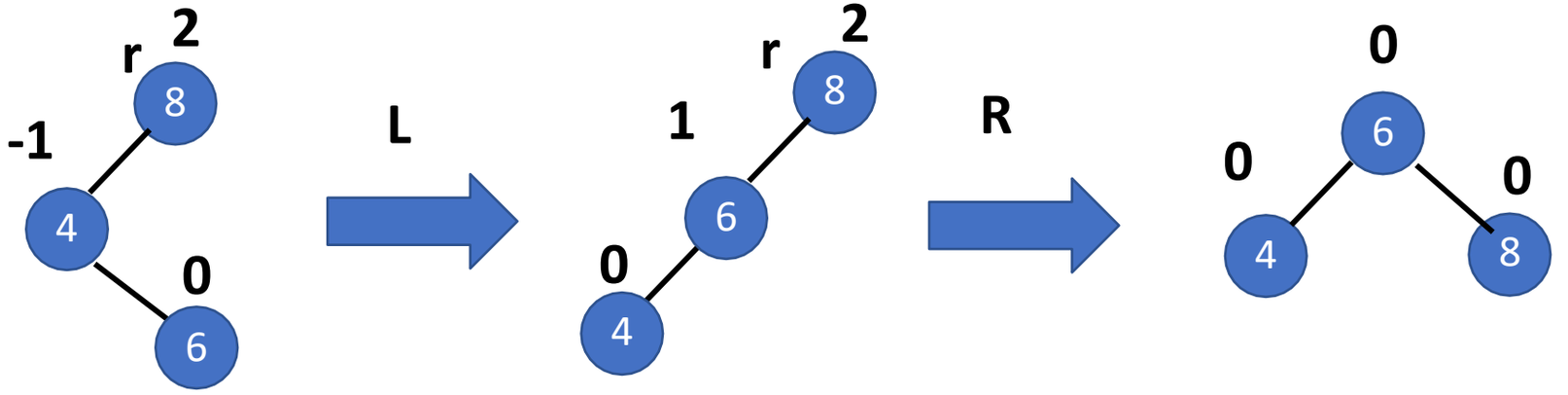


- Root of the tree had balance of -1 before the insertion
- New key is inserted to the right of right child (Right-Right-case)
- To Balance the tree we need to perform Left Rotation

Courtesy: “Introduction to Design and Analysis of Algorithms” By Anany Levitin

Rotations - AVL Trees

Double Left Right Rotation:

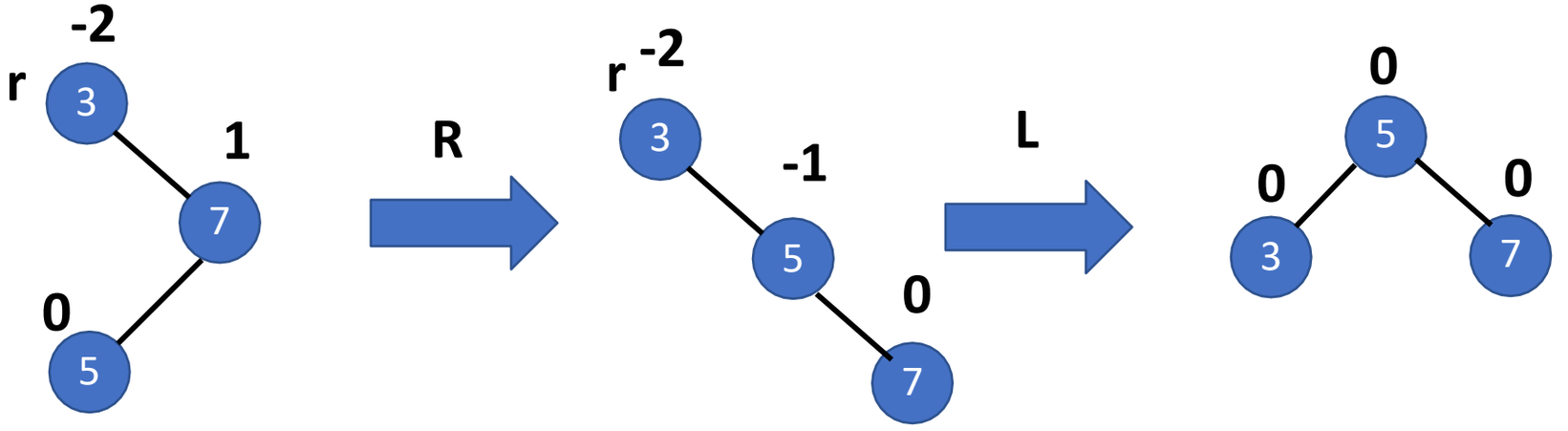


- Root r had the balance of +1 before the insertion
- New key is inserted to the right of left child (Left-Right-case)
- We perform the left rotation of left subtree of root r
- Right rotation of the new tree rooted at r

Courtesy: “Introduction to Design and Analysis of Algorithms” By Anany Levitin

Rotations - AVL Trees

Double Right Left Rotation:

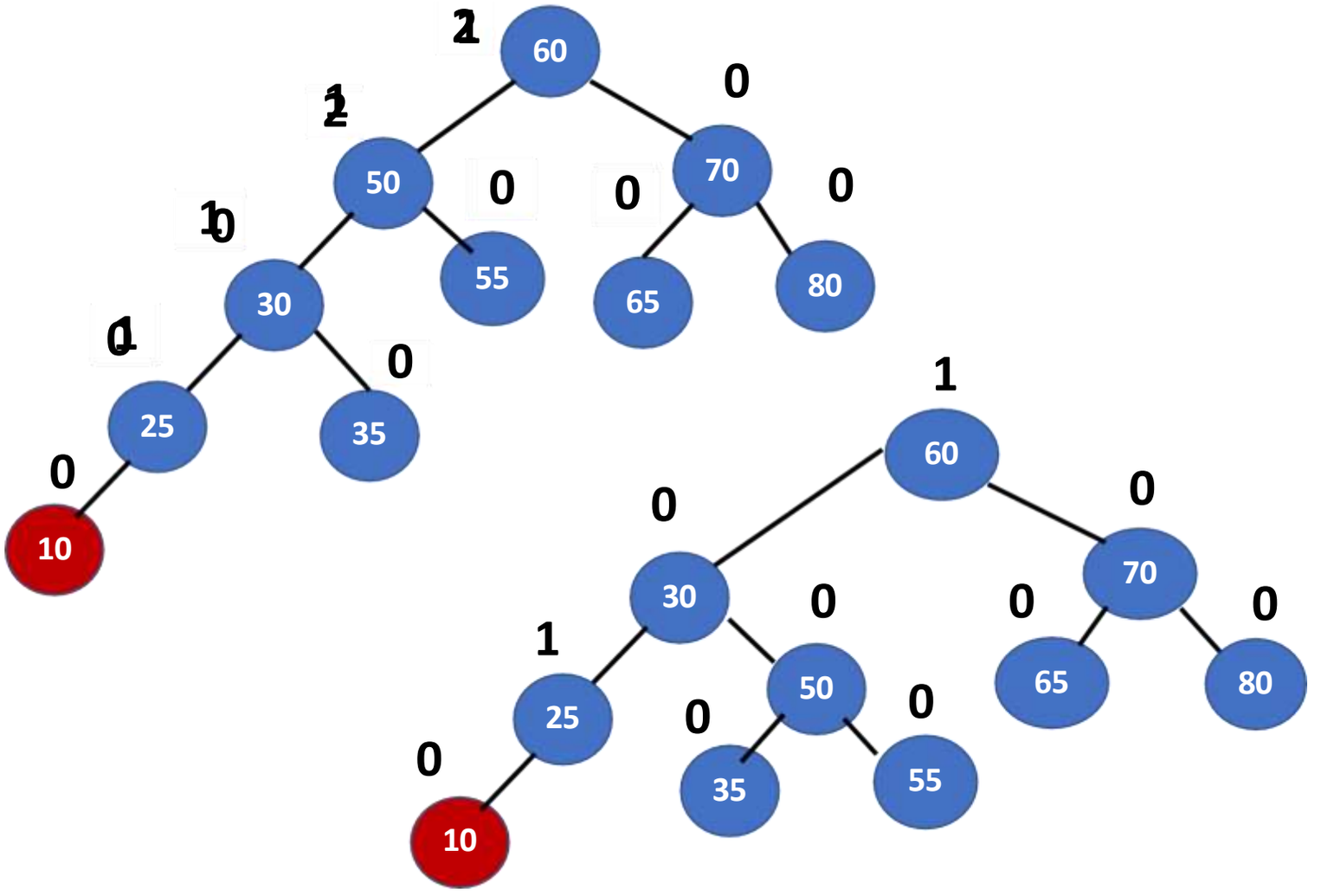


- Root r had the balance of -1 before the insertion
- New key is inserted to the left of right child (Right-Left-case)
- We perform the right rotation of right subtree of root r
- Left rotation of the new tree rooted at r

Courtesy: “Introduction to Design and Analysis of Algorithms” By Anany Levitin

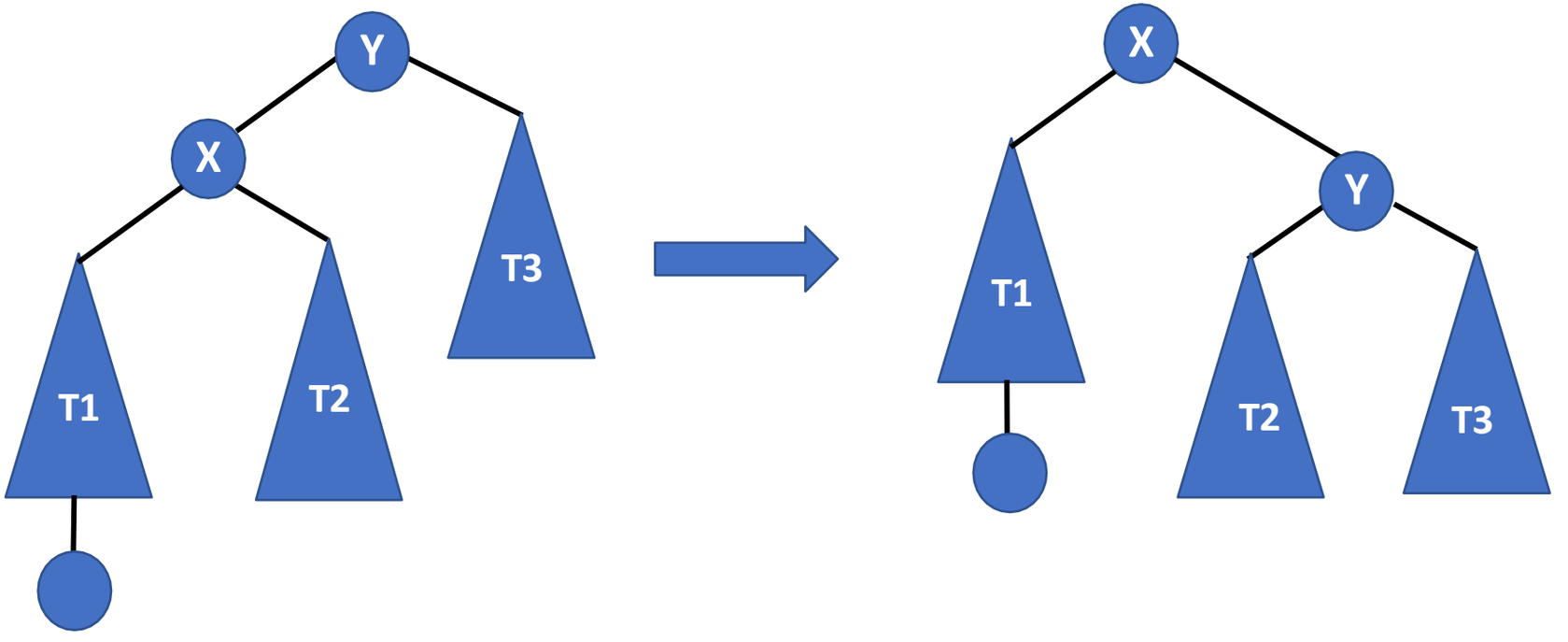
DATA STRUCTURES AND ITS APPLICATIONS

Example – Rotations in AVL Trees



Courtesy: “Introduction to Design and Analysis of Algorithms” By Anany Levitin

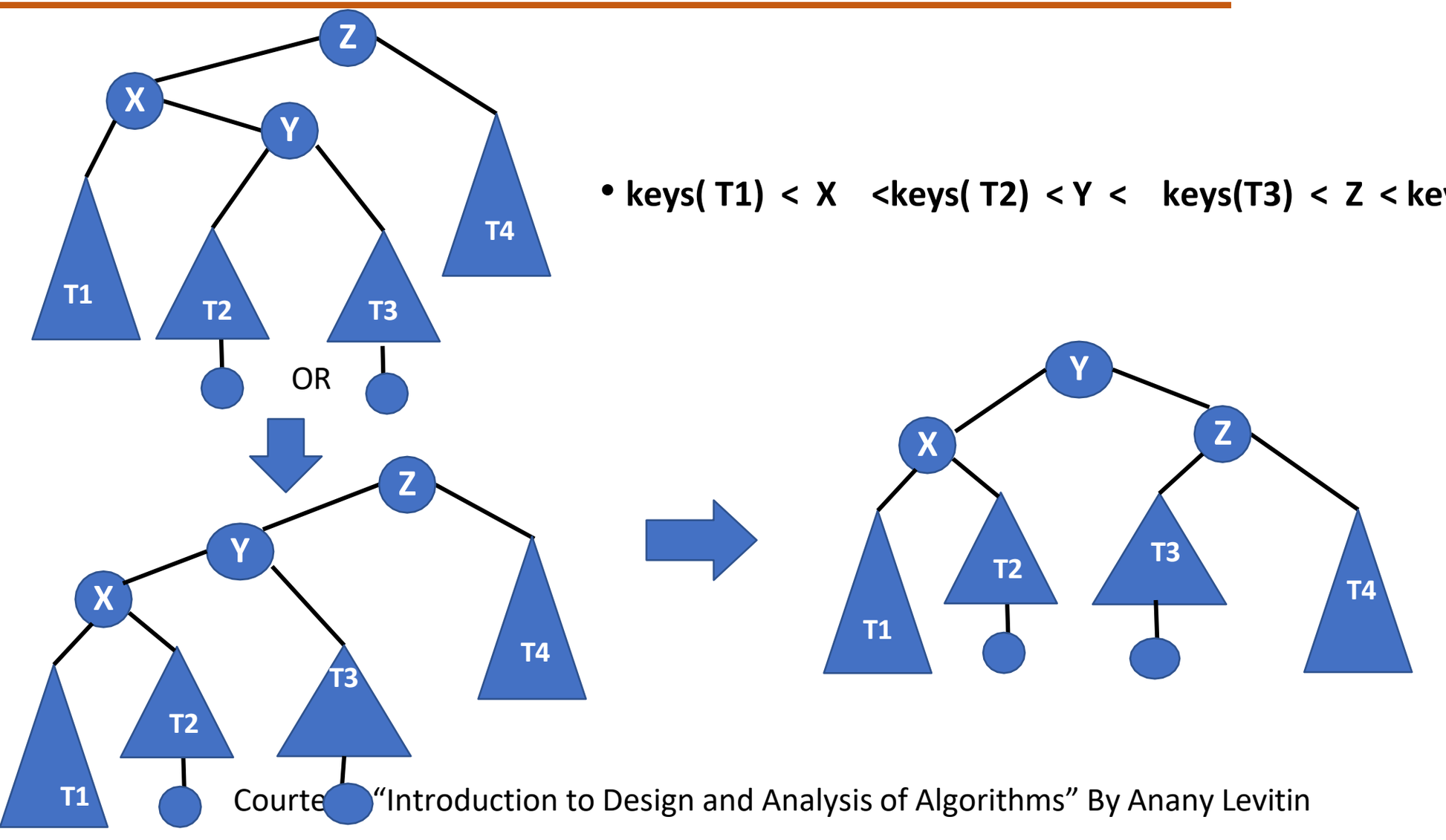
General form of Right-Rotation



- keys(T1) < X < keys(T2) < Y < keys(T3)

Courtesy: “Introduction to Design and Analysis of Algorithms” By Anany Levitin

General form of Left – Right Rotation



- keys(T1) < X <keys(T2) < Y < keys(T3) < Z < keys(T4)

Courte “Introduction to Design and Analysis of Algorithms” By Anany Levitin

THANK YOU

Sandesh B. J
Department of Computer Science & Engineering

sandesh_bj@pes.edu

+91 80 6618 6623

DATA STRUCTURES AND ITS APPLICATIONS

Graph Representation

Sandesh B. J
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Graph Representation

Sandesh B. J
Department of Computer Science & Engineering

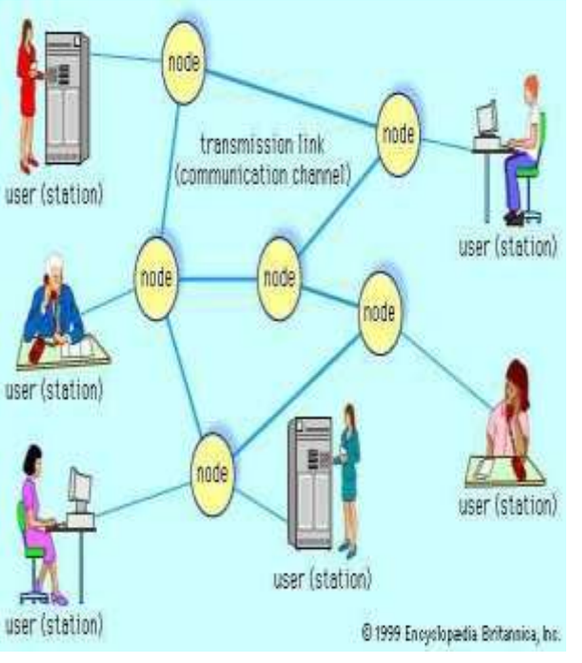
Graph Representation

Learning objectives:

- Why we need to represent the structure of the graph in computer memory?
- How do we represent the graph ?
- Data structure used for the representation:
  - ✓Adjacency Matrix and Adjacency List
- Representation : Directed, Undirected and Weighted graphs
- Implementation details of representation using ‘c’
- Multilinked Representation

Graph Representation

Why Graph Representation?



Graph Representation

- How do we store mathematical structure of a graph in the computer memory?

- What types of data structures are used to represent the graph?

- Information Required to represent the graph
  - set of vertices of the graph
  - for each vertex neighbours of the vertex(edge information)

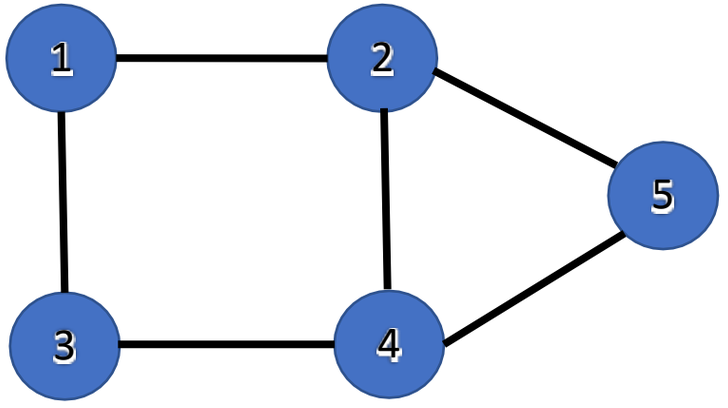
Graph Representation

- Depending on the density of edges, ease of use and types of operation performed , Graphs can be represented by

  - ✓ Adjacency Matrix
    - Two Dimensional Array
  - ✓ Adjacency List
    - Linked List

Adjacency Matrix Representation – Undirected graph

- Adjacency matrix = n x n matrix M, graph of n vertices/nodes
- **M[i][j] = 1** if (i, j) is an edge
- **M[i][j] = 0 ,** no edge between the pair of vertices i and j



Undirected Graph

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	1	1
3	1	0	0	1	0
4	0	1	1	0	1
5	0	1	0	1	0

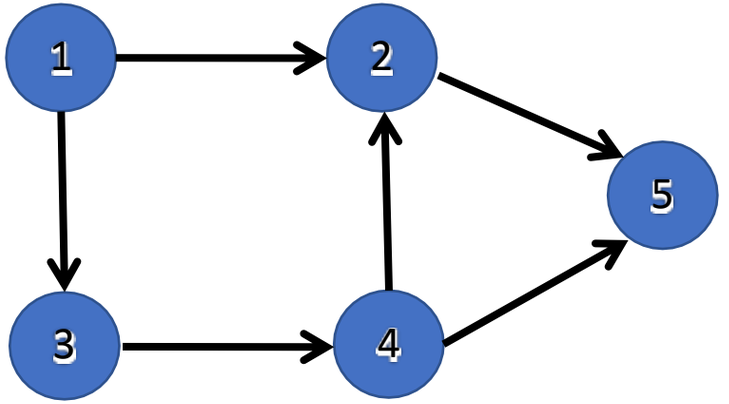
Main diagonal

Note:

- For undirected graph, M is symmetric i.e M[i][j] = M[j][i]
- Assume no edge from node to itself. So diagonal elements has value 0

Adjacency Matrix Representation – Directed graph

- In directed graph edge is directed
- In directed graph edge(i,j) is not equal to edge(j,i)
- Adjacency matrix is asymmetric



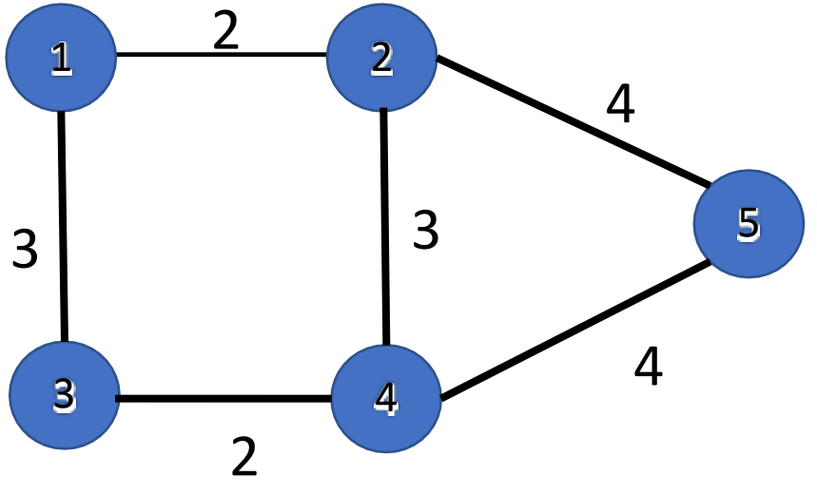
Directed Graph

	1	2	3	4	5
1	0	1	1	0	0
2	0	0	0	0	1
3	0	0	0	1	0
4	0	1	0	0	1
5	0	0	0	0	0

Main diagonal

Adjacency Matrix Representation – weighted graph

- In the weighted graph distance or cost between the nodes are represented on the edge
- cost/distance value specified on the edge between adjacent nodes are stored in the adjacency matrix



Weighted Graph

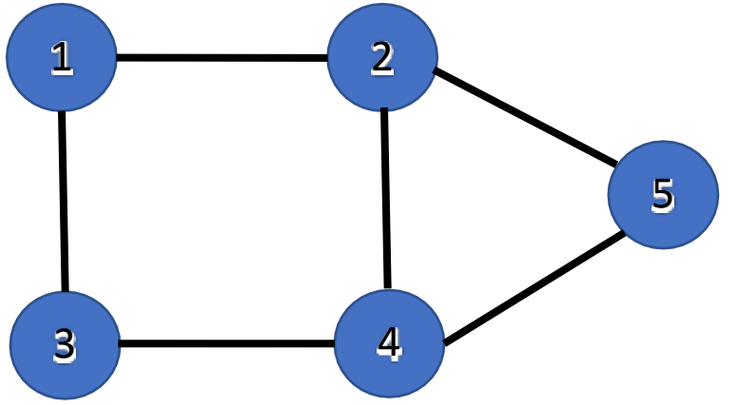
	1	2	3	4	5
1	0	2	3	0	0
2	2	0	0	3	4
3	3	0	0	2	0
4	0	3	2	0	4
5	0	4	0	4	0

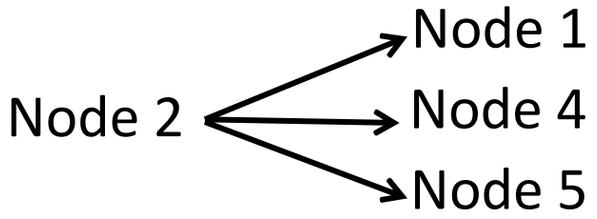
Drawbacks of Adjacency Matrix Representation

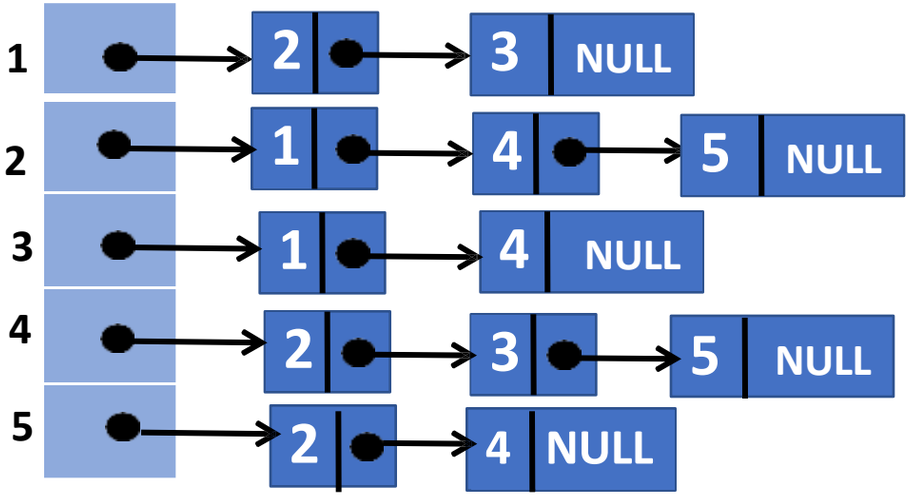
- Number of nodes in the graph needs to be known in prior
- In case graph needs to be updated dynamically as the program proceeds new matrix must be created for each addition or deletion
- To detect presence of edge between pair of nodes takes constant time O(1) but it takes $O(v^2)$ to visit all the neighbouring nodes of each node.
- Adjacency matrix becomes sparse in case graph has very few edges.
- space complexity is $O(v^2)$. v^2 locations are required for graph with v nodes

Adjacency list representation – Undirected graph

- Each node maintains
  - linked list of its neighbours

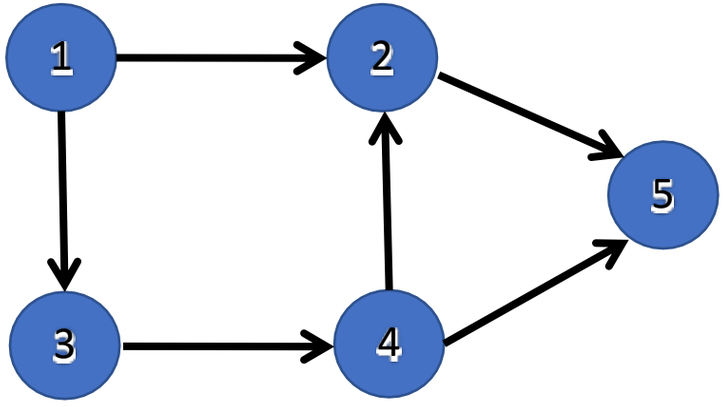


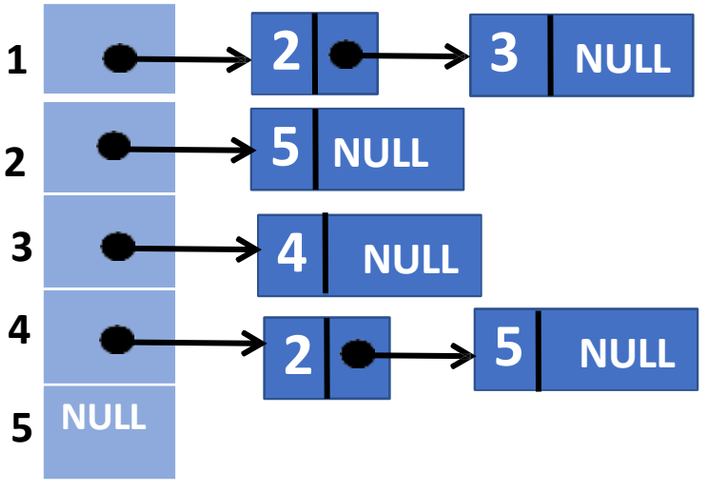




DATA STRUCTURES AND ITS APPLICATIONS

Adjacency list representation – Directed graph



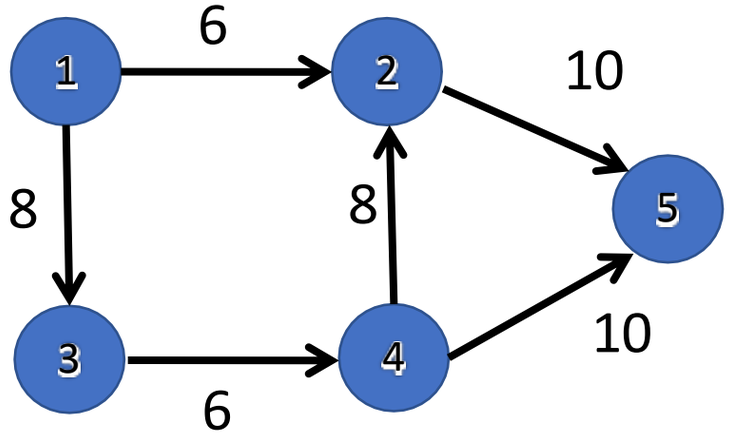


Directed Graph

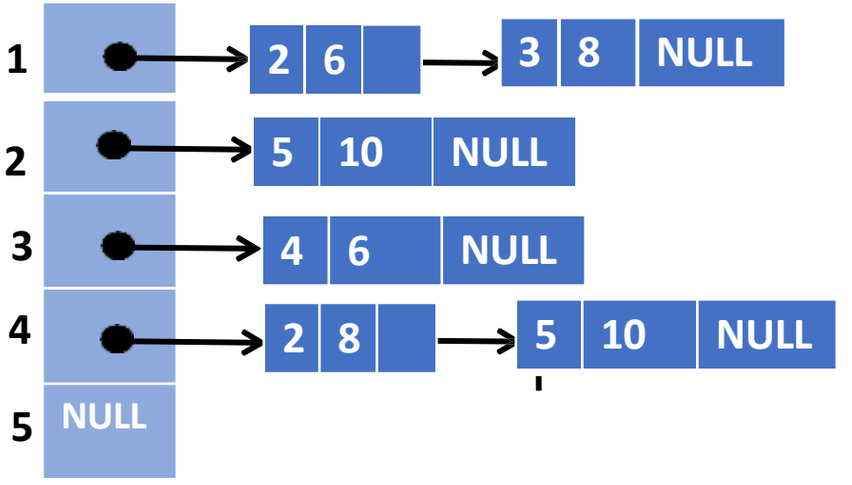
Node 2 ⟶ Node 5

Adjacency matrix representation – weighted graph

- In weighted graph the distance or cost between the nodes are represented on the edge
- Along with the information of adjacent node , cost/distance value is stored in each node of the linked list.



Weighted Graph



Pros and Cons of Adjacency List Representation

- Space Complexity of Adjacency list is O(V+E), because it stores the information of edges that actually exists in the graph
- In case of low density edges, the adjacency matrix becomes sparse using adjacency list is better for representation
- To detect the presence of edge between two nodes, we need to traverse the linked list of the node.

Implementation of graph using adjacency matrix

Data structure to represent the adjacency matrix:

- To represent only the existence of edge between nodes

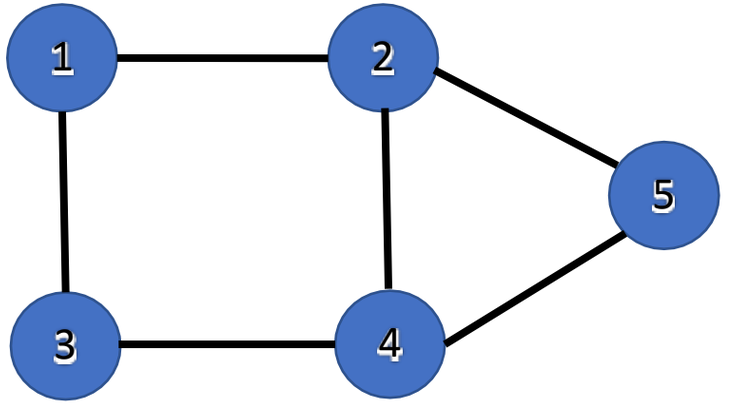
```
#define maxnodes 50
adj[maxnodes][maxnodes];
```

```
Typedef Boolean AdjecencyMatrix[maxnodes][maxnodes]

Typedef struct graph
{
    int n;      /* number of vertices in the graph */
    AdjacencyMatrix adj;
}Graph;
```

Implementation of Graph using Adjacency Matrix

To check the presence of edge between pair of nodes:

```
if ( adj[i][j] == 1)   */ i and j represents the vertex in a graph */
        then "edge exists between the pair of nodes"
else
          " there is  no edge between the pair of nodes"
```



	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	1	1
3	1	0	0	1	0
4	0	1	1	0	1
5	0	1	0	1	0

Implementation of graph using adjacency matrix:

To add an edge from node1 to node2:

```
void join(int adj[][MAXNODES],int node1,int node2)
{
        adj[node1][node2]=TRUE;
}
```

To delete an edge from node1 to node2 if it exists:

```
void remv(int adj[][MAXNODES],int node1,int node2)
{
        adj[node1][node2]=FALSE;
}
```

Implementation of adjacency matrix:

To check whether arc exists between node1 and node2:

```
int adjacent(int adj[][MAX],int node1,int node2)
{
        return((if(adj[node1][node2])==TRUE)?TRUE:FALSE);
}
```

C-Representation of graphs- Adjacency Matrix

```
#define MAXNODES 50
struct node
{
        //information associated with each node
};
struct arc
{
        int adj;// information associated with each edge
};
struct graph
{
        struct node nodes[MAXNODES];
        struct arc arcs[MAXNODES][MAXNODES];
}
struct graph g;
```

C representation of weighted graph

- Weighted graph with fixed number of nodes is declared as

```
struct arc{
      int adj;
      int weight;
  };
  struct arc g[maxnodes][maxnodes];
```

To add an edge from node1 to node2:

```
void  joinwt(struct arc g[][maxnodes],int node1,int node2, int wt)
{
  g[node1][node2].adj = TRUE;
  g[node1][node2].weight= wt;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

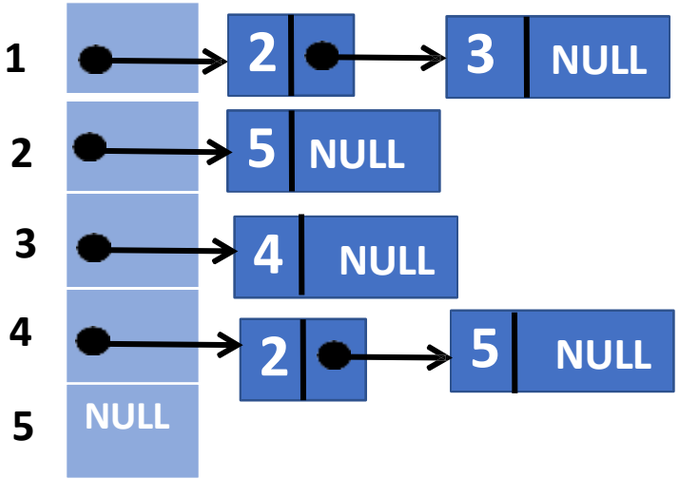
Implementation of adjacency List Representation:

```
struct node
{
   int node;
   struct node *nextedge;
};
```

NODE info	Nextedge

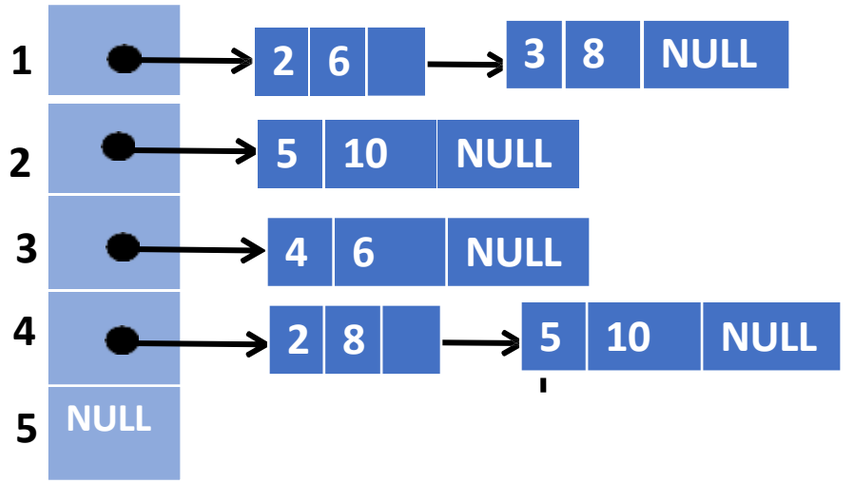
To create a node list:

```
 struct node  *nodelist;
 for(i=0; i < no_of_nodes; i++)
     nodelist[i]= NULL;
```



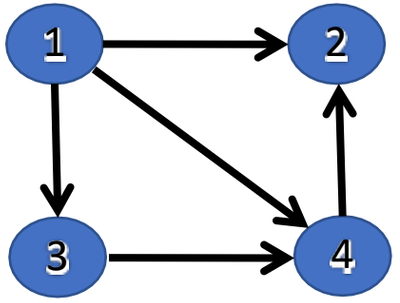

```
struct node
{
   int node;
   int weight;
   struct node *nextedge;
};
```

nodeinfo	weight	nextedge



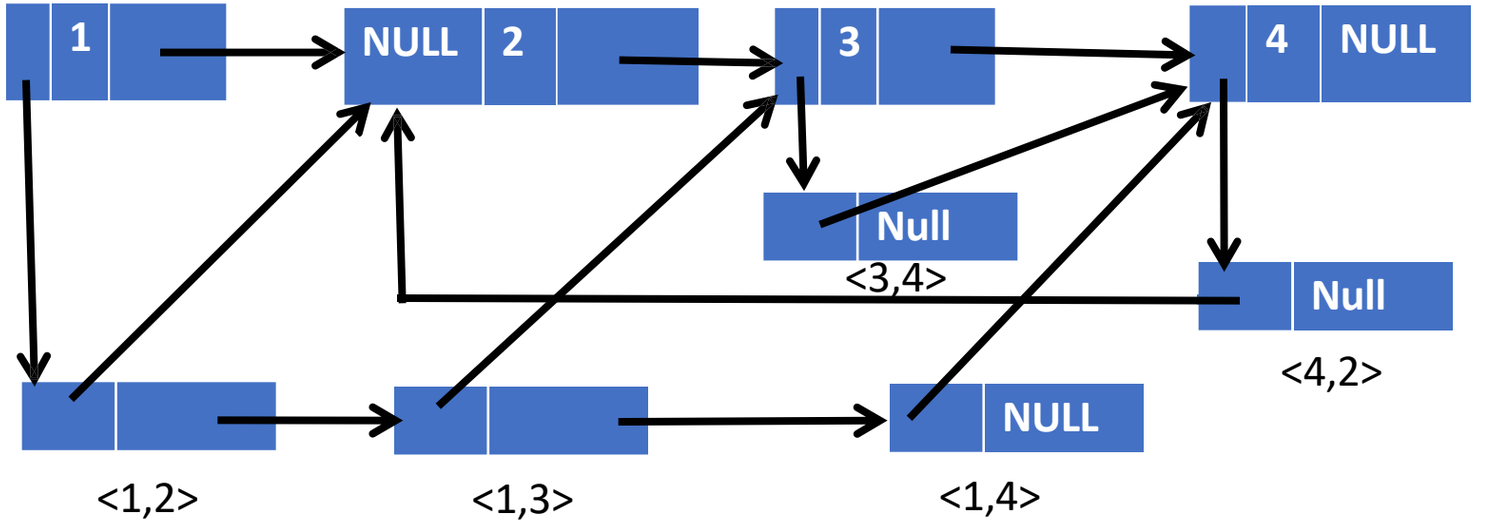
DATA STRUCTURES AND ITS APPLICATIONS

Multilinked structure representation



Sample header node representing the graph node

A sample list node representing edge



Multilinked structure representation

Dynamic Implementation:

```
struct nodetype
{
  int info;
  struct nodetype  *ptr;
  struct node type *next;
};
struct nodetype *nodeptr;
```

Sample header node representing the graph node

A sample list node representing edge

- Note that header node and list node have different formats and must be represented by different structures.
- In case of weighted graph in which each list node contains info field to store the weight of the edge the same dynamic implementation could be used for both types of nodes.

THANK YOU

Sandesh B. J
Department of Computer Science & Engineering

sandesh_bj@pes.edu

+91 80 6618 6623

DATA STRUCTURES AND ITS APPLICATIONS

Balanced Trees

Sandesh B. J
Department of Computer Science & Engineering

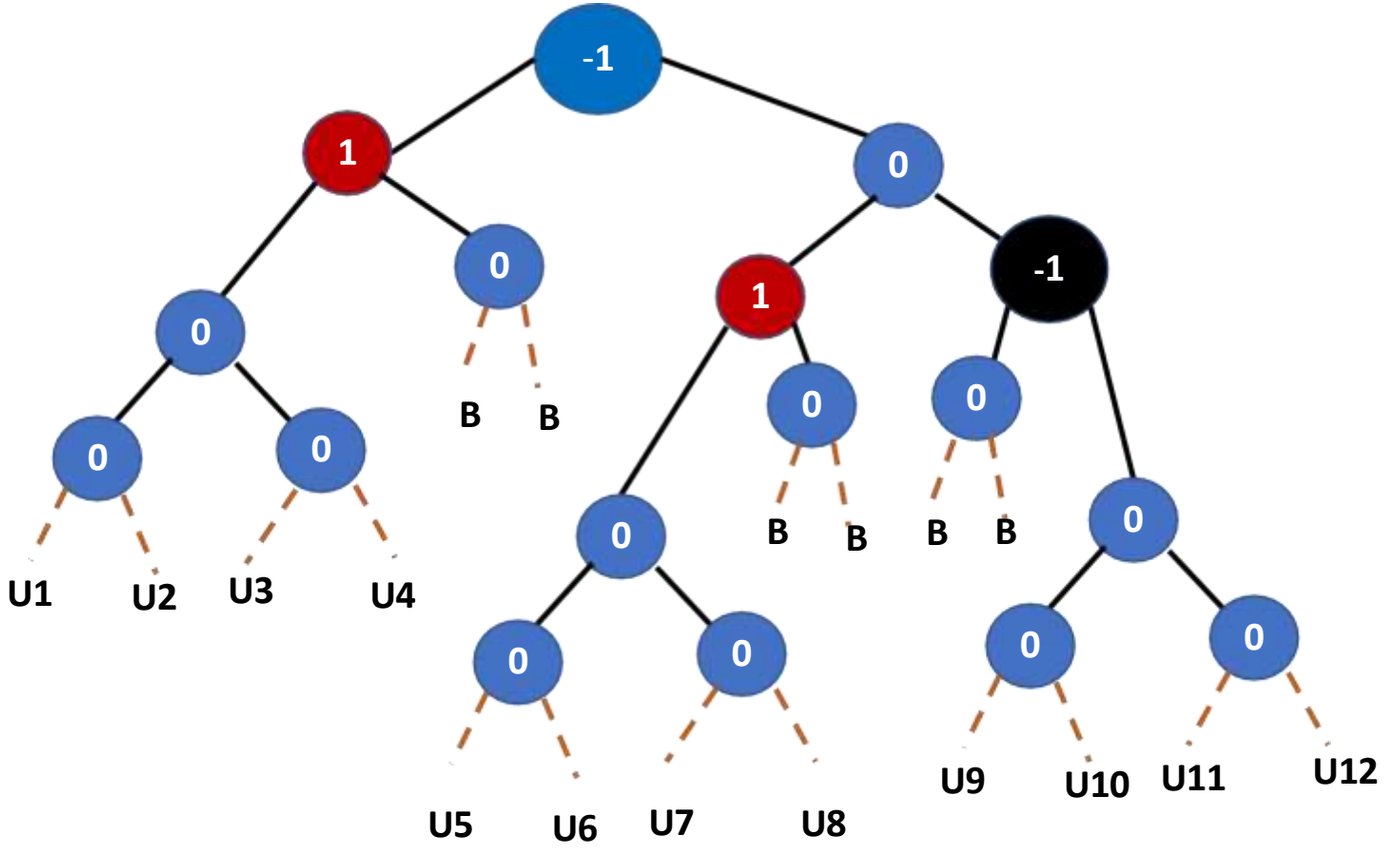
DATA STRUCTURES AND ITS APPLICATIONS

Balanced Trees

Sandesh B. J
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

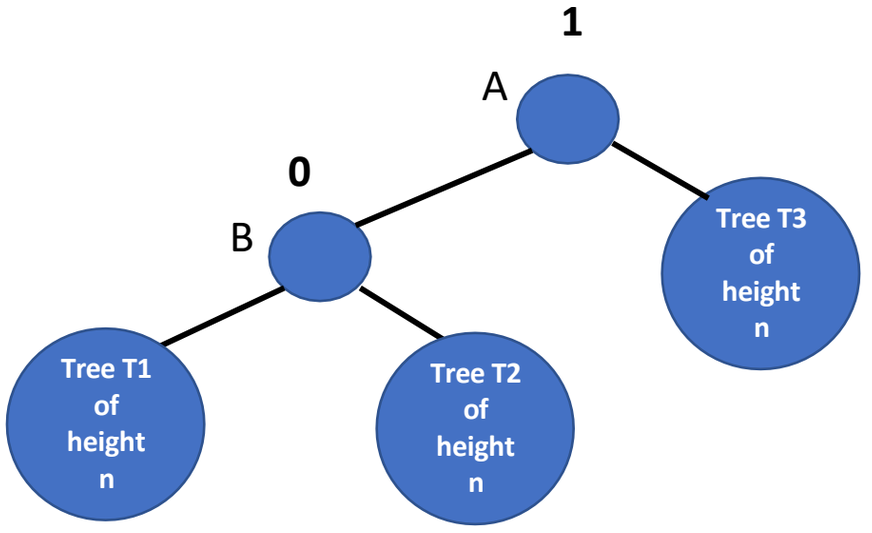
Possible insertion into AVL tree



- Unbalanced insertions are indicated by **U**
- Balanced insertions are indicated by **B**

Courtesy: “Data Structures using c and c++ ” By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

AVL tree Insertions



Balance factor(A) = (n+1) – n =1

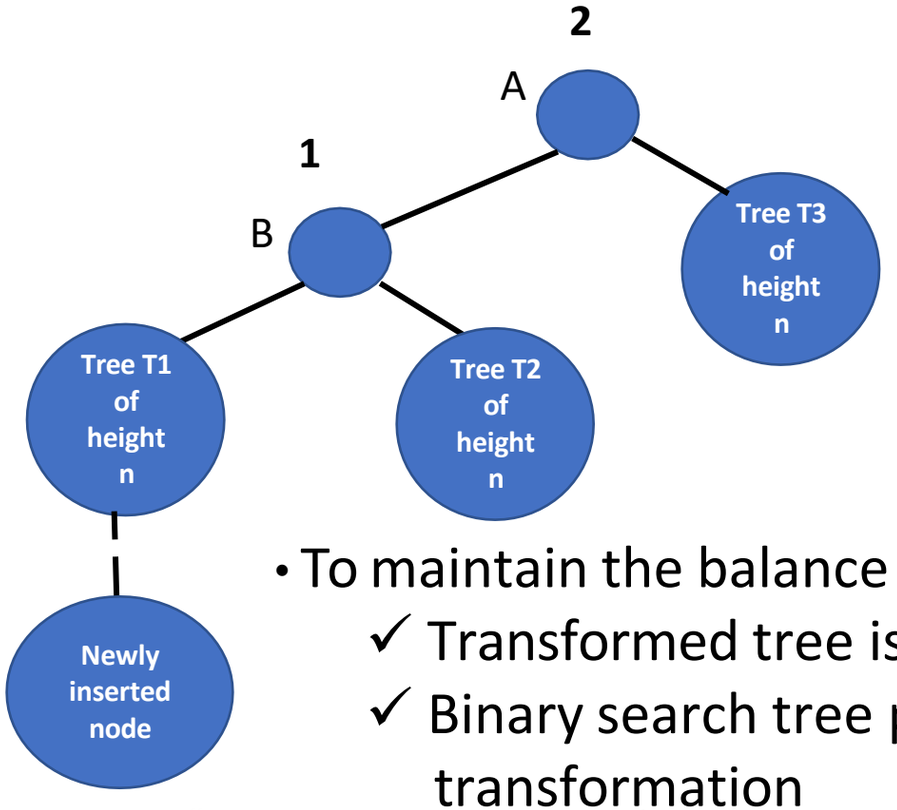
Balance factor(B) = n – n = 0

- Let us consider A is the youngest ancestor which becomes unbalanced
- Balance factor of A should be 1 before insertion
- A should have a left child B with the balance factor of 0

Courtesy: “Data Structures using c and c++ ” By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

Unbalanced Tree after inserting a node to left subtree

- Newly inserted node is left descendent of node A
- Changing the balance **B to 1 and A to 2**
- A is the youngest ancestor of the new node to become unbalanced

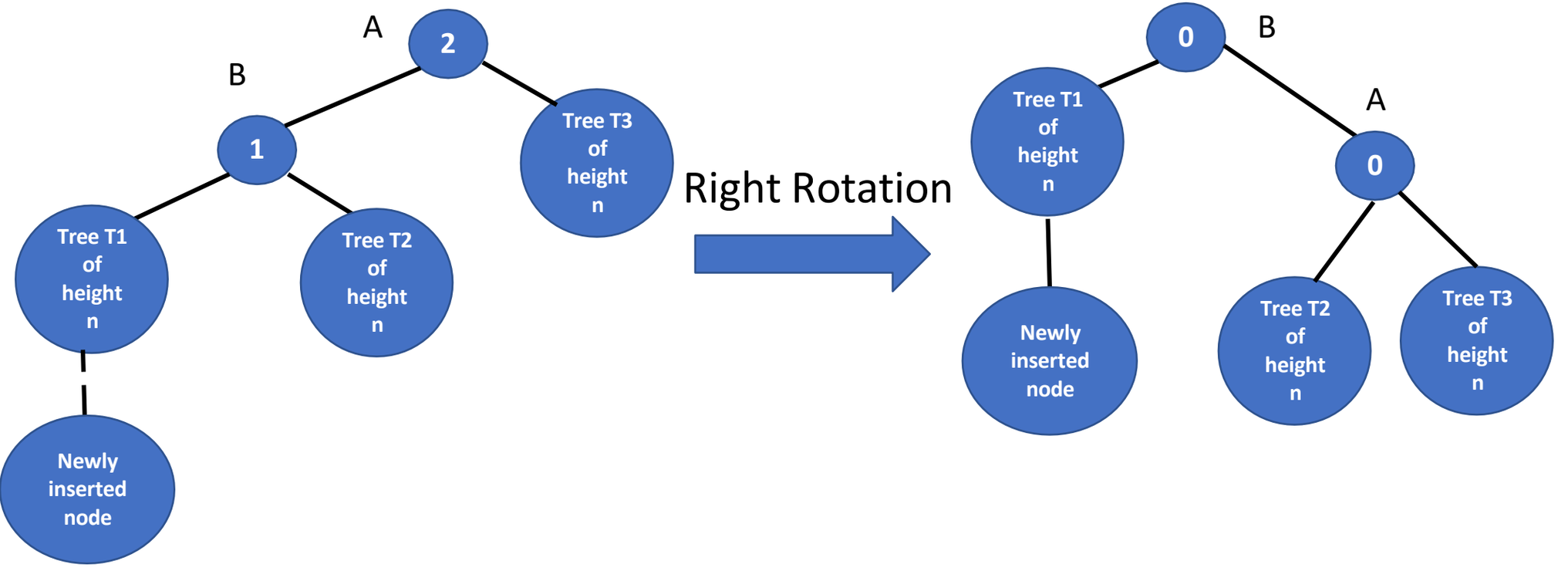


- To maintain the balance : Tree needs to be transformed
  - ✓ Transformed tree is balanced
  - ✓ Binary search tree property is maintained after transformation

Courtesy: "Data Structures using c and c++ " By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

Transformed Balanced Tree after Rotations

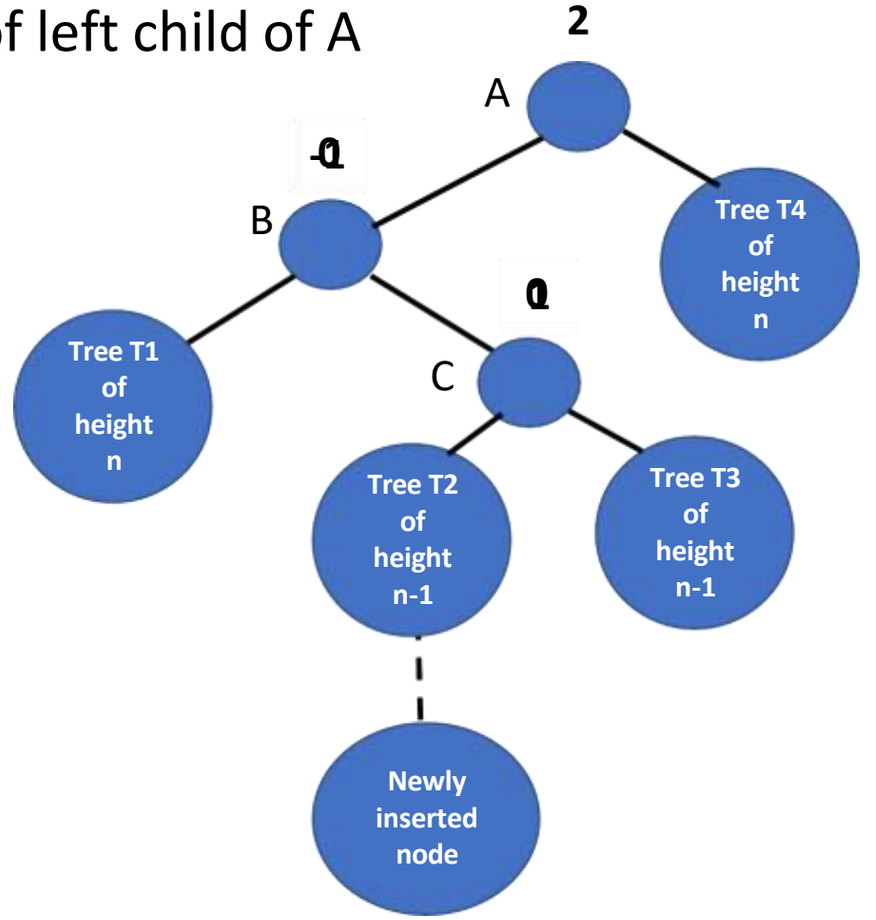
- To maintain a balance we need to rotate sub tree B rooted at node A



Courtesy: “Data Structures using c and c++ ” By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

Unbalanced Tree after inserting a node to right subtree

- Newly inserted node is left descendent of the node A
- New node is inserted into right subtree of left child of A

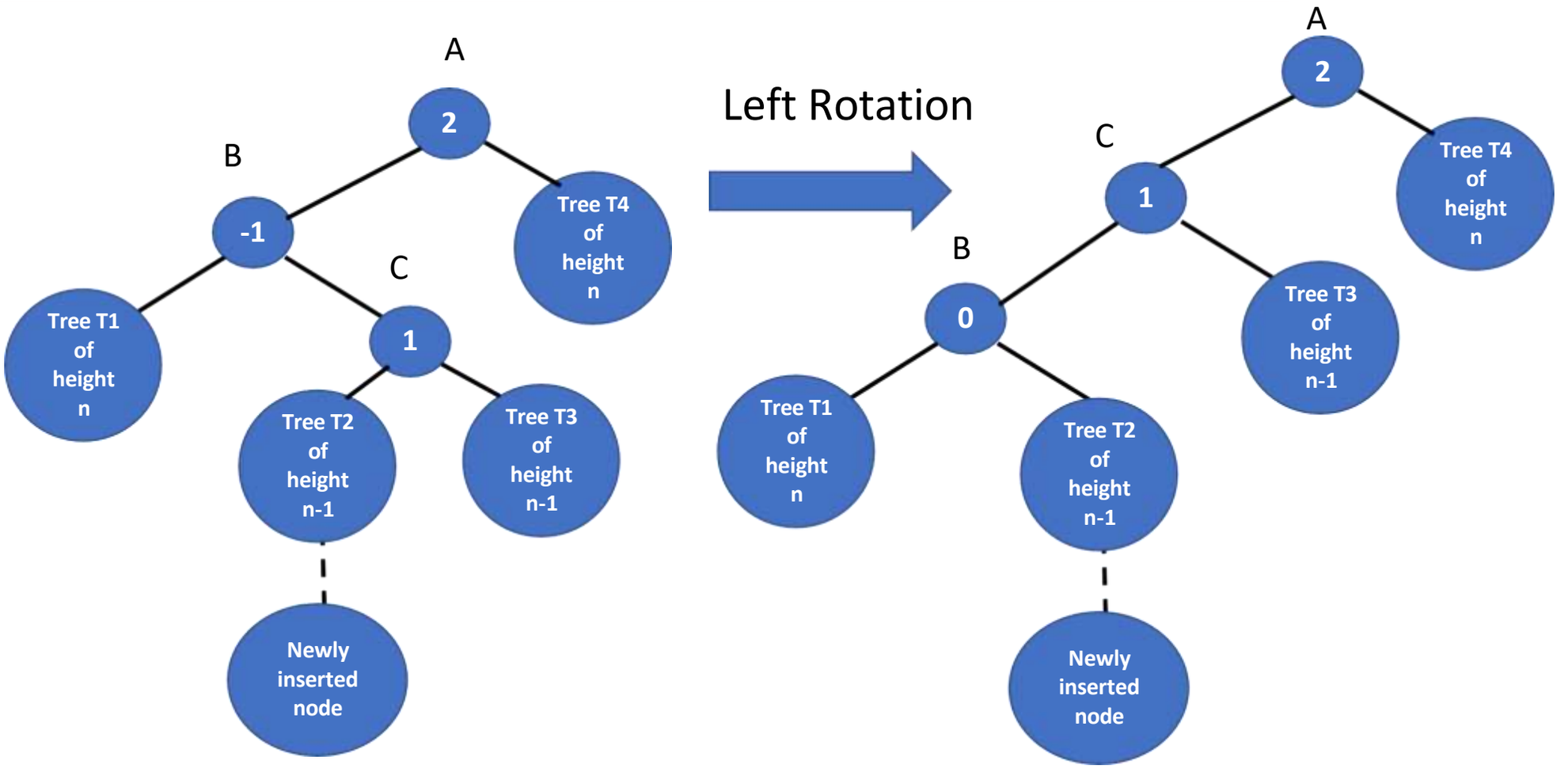


Balance factor(C)= n-(n-1) = 1
Balance factor(B)= n-(n+1) = -1
Balance factor(A)= n+2-n= 2

Courtesy: “Data Structures using c and c++ ” By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

DATA STRUCTURES AND ITS APPLICATIONS

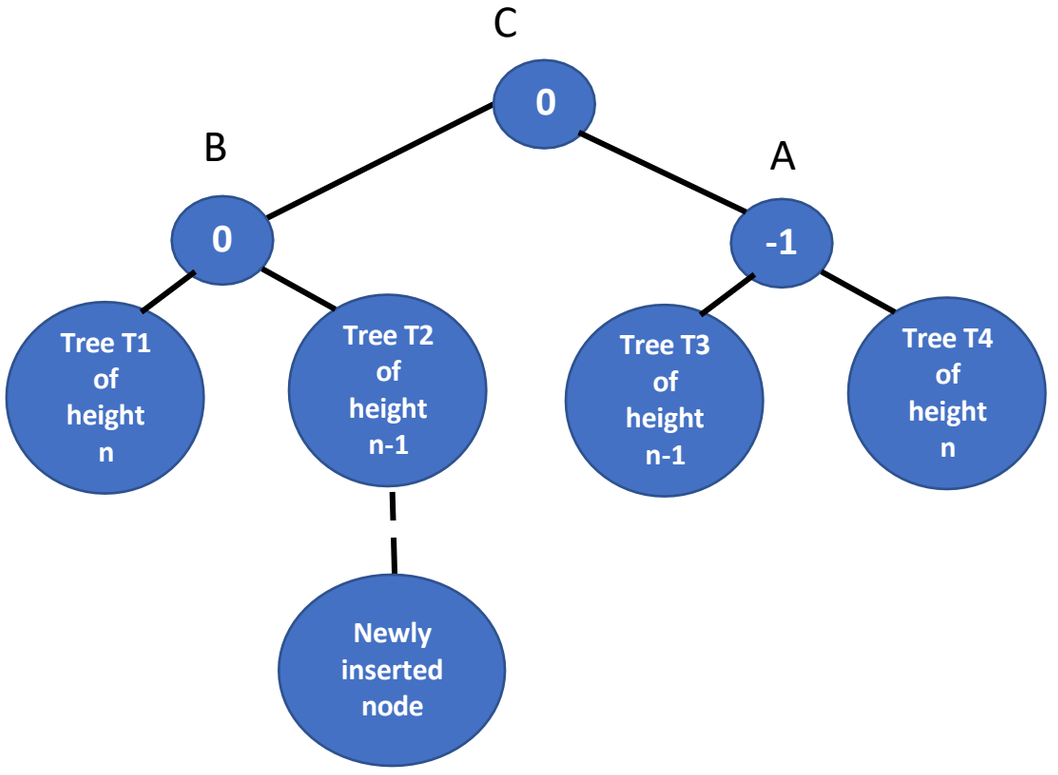
Transformed Balanced tree after Rotations



Courtesy: “Data Structures using c and c++ ” By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

Transformed Balanced tree after Rotations

Right Rotation



Courtesy: “Data Structures using c and c++ ” By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

AVL tree Insertion

- Insertion in AVL tree is performed using standard BST Insertion
- If tree becomes unbalanced, we rebalance the tree using left or right rotation
- If node X is inserted into balanced BST
- we need to find the youngest ancestor which becomes unbalanced

Four cases:

- IF(Balance factor of node) == 2 – unbalanced node(U)
  - ✓ case-1 : Left-Left case
    - IF((newly inserted key) < (key in the left subtree' root))
  - ✓ case-2: Left-Right case
    - IF((newly inserted key) > (key in the left subtree' root))

AVL tree Insertion

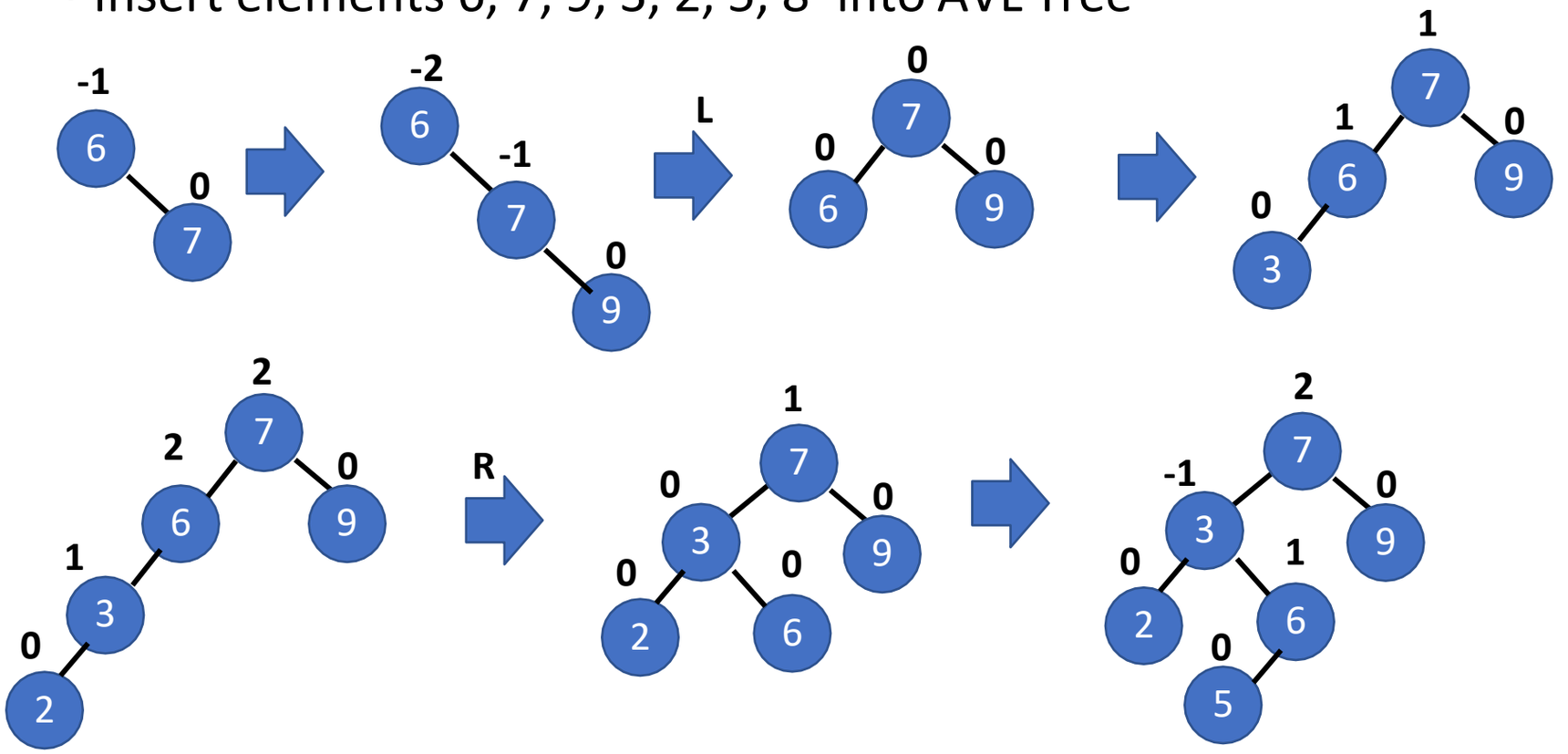
Four cases:

- IF((Balance factor of node)) == -2 – unbalanced node(U)
  - ✓ Case 3: Right-Right case
    - IF((newly inserted key) > (key in the right subtree' root))
  - ✓ Case 4:Right-Left case
    - IF((newly inserted key) < (key in the right subtree' root))

DATA STRUCTURES AND ITS APPLICATIONS

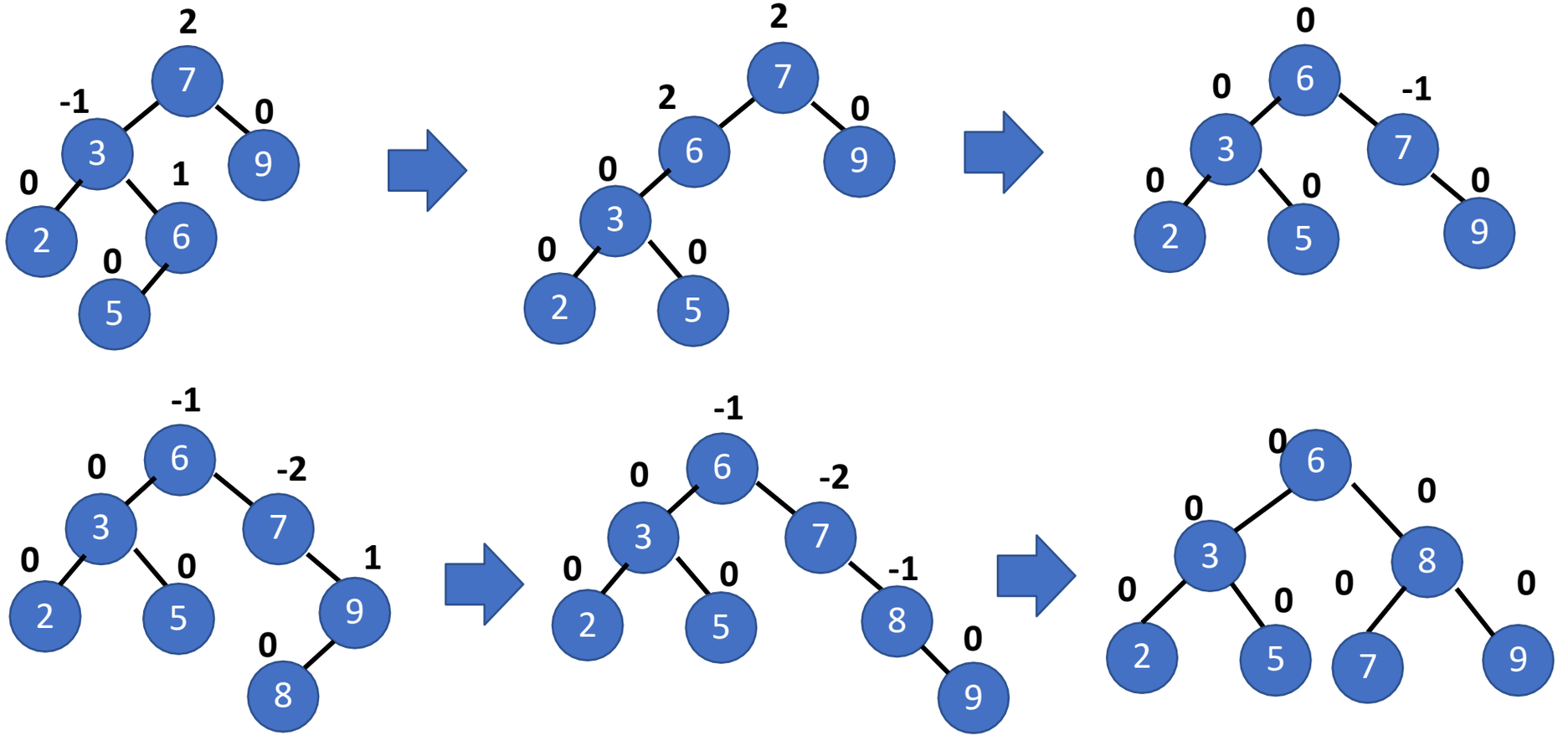
Examples – AVL Tree Insertions

- Insert elements 6, 7, 9, 3, 2, 5, 8 into AVL Tree



Example – AVL Tree Insertions

- Insert elements 6, 7, 9, 3, 2, 5, 8 into AVL Tree



Deletions in AVL tree

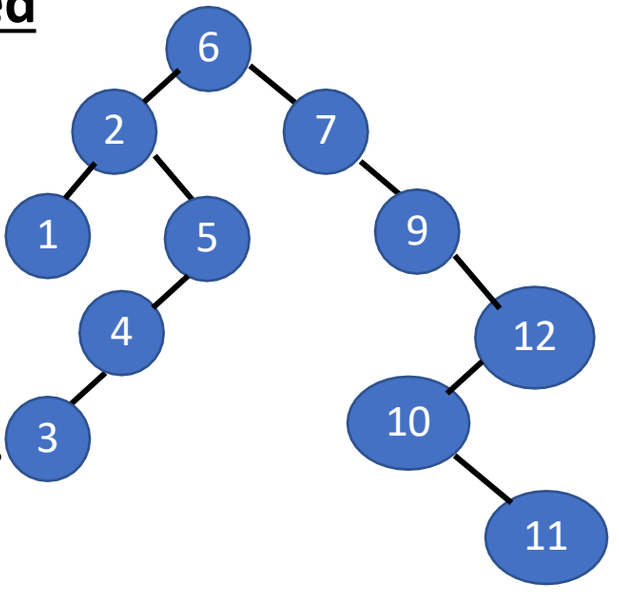
- Deletion in AVL tree is performed using standard BST Deletion
- If tree becomes unbalanced, we rebalance the tree using left or right rotation

BST Deletion: 3- case: Node to be delted

Case 1: Does not have any children

Case 2: has either left or right subtrees

Case 3: has both left and right subtrees.

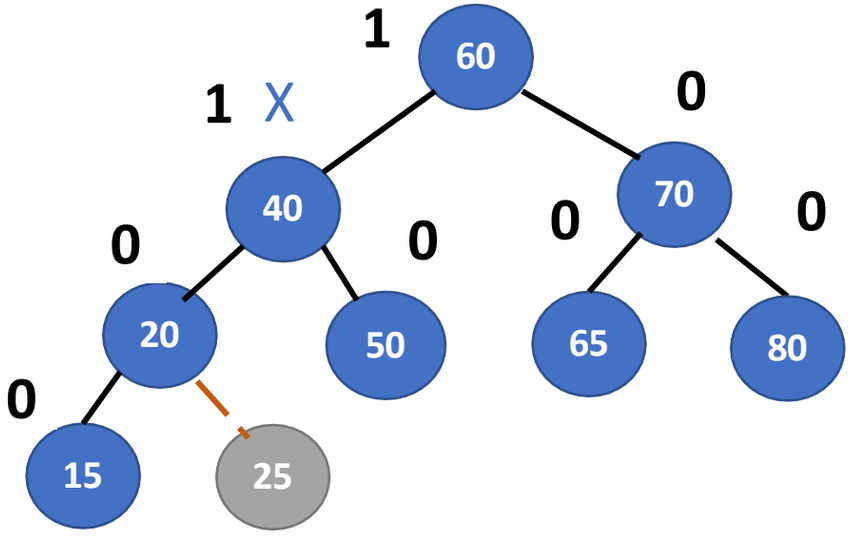


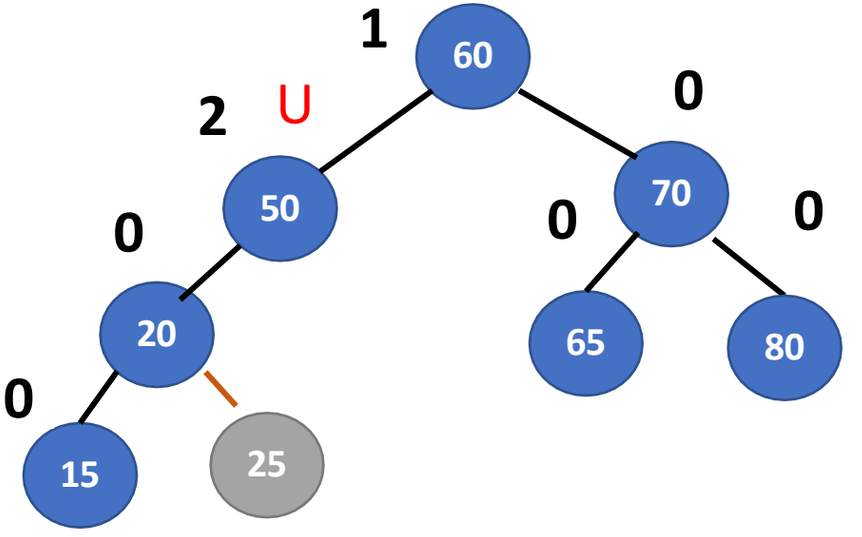
Deletions in AVL tree

- If node X is deleted from the BST
- we need to find the youngest ancestor which becomes unbalanced

Four cases: Case-1

- IF((Balance factor of a node) == 2) - unbalanced node(U)
  - ✓ Left-Left case:
    - IF(Balance factor of left subtree's root) >= 0

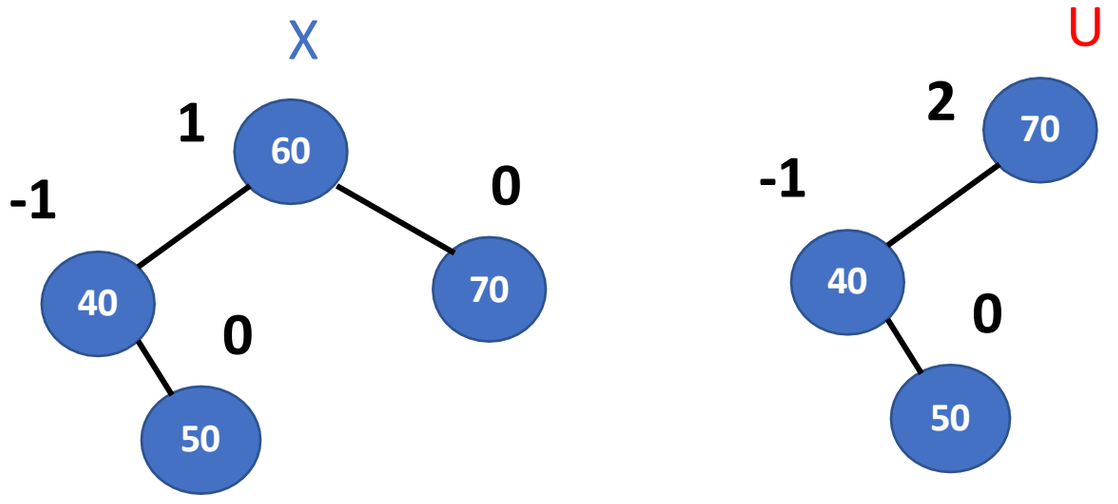




Deletions in AVL tree

Case-2:

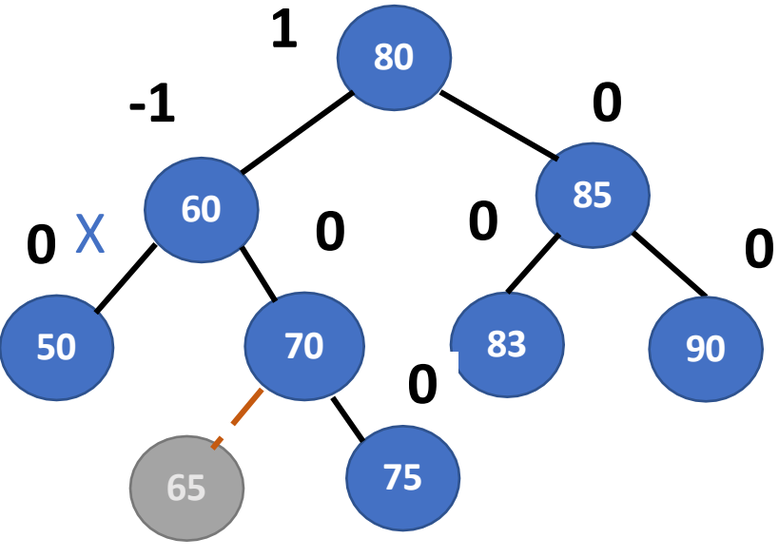
- IF(Balance factor of a node == 2) – unbalanced node(U)
  - ✓Left-Right case
    - IF(Balance factor of left subtree's root) < 0

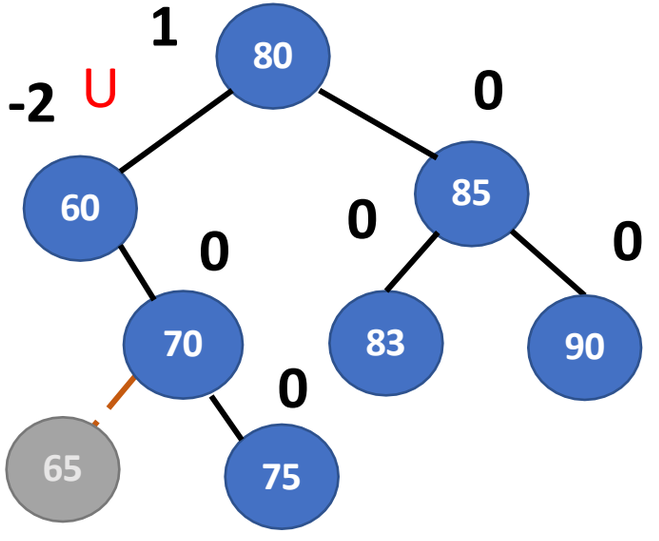


Deletions in AVL tree

Case-3:

- If ((Balance factor of a node) == -2) unbalanced node(U)
  - ✓Right-Right case
    - IF(Balance factor of Right subtree's root) <= 0



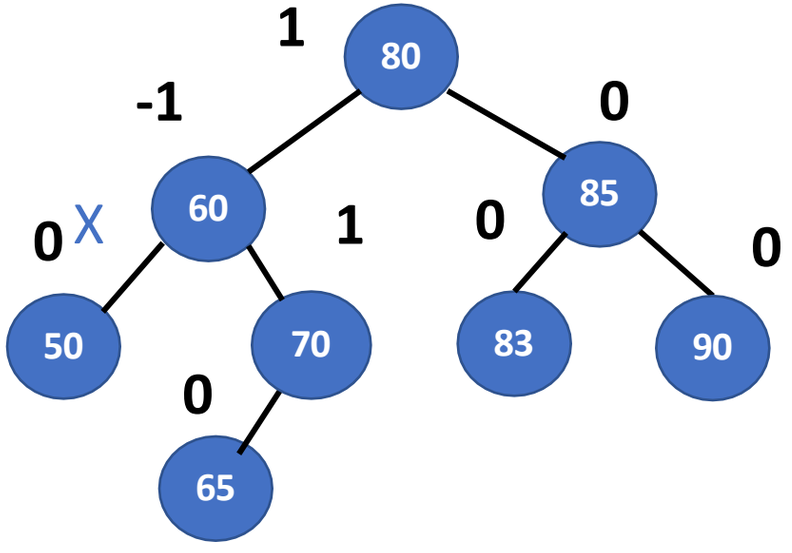


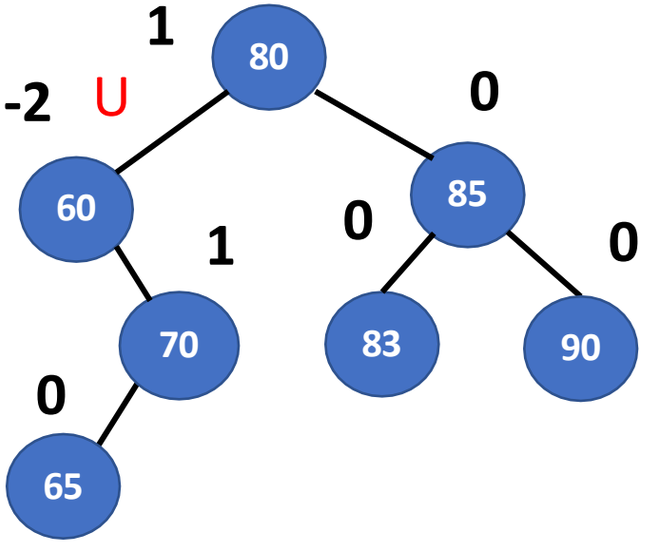
Deletions in AVL tree

Case-4:

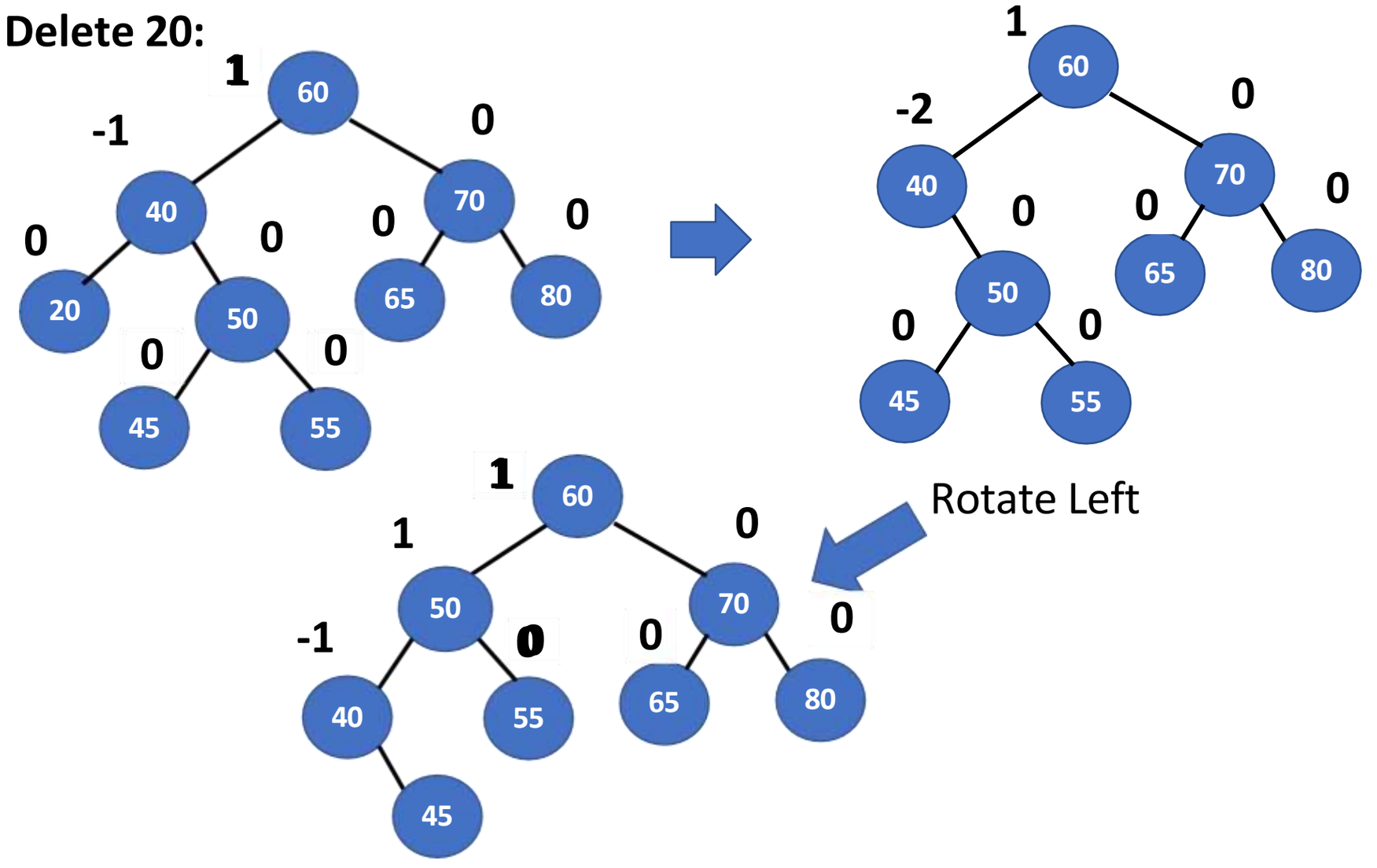
IF((Balance factor of unbalanced node) == -2) unbalanced node(U)

- ✓Right-Left case
  - IF(Balance factor of Right sub tree's root) > 0



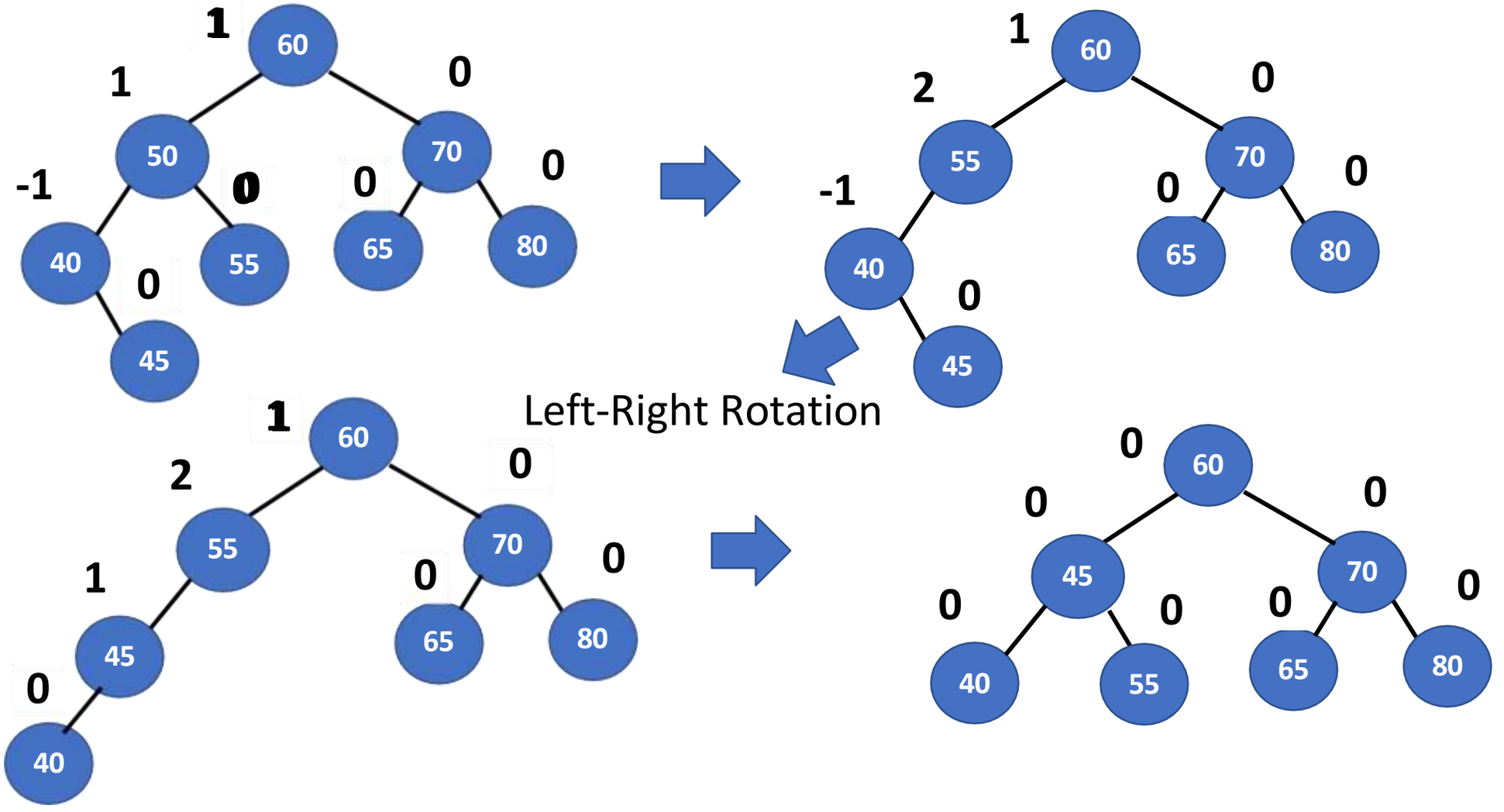


Example – Deletions in AVL tree



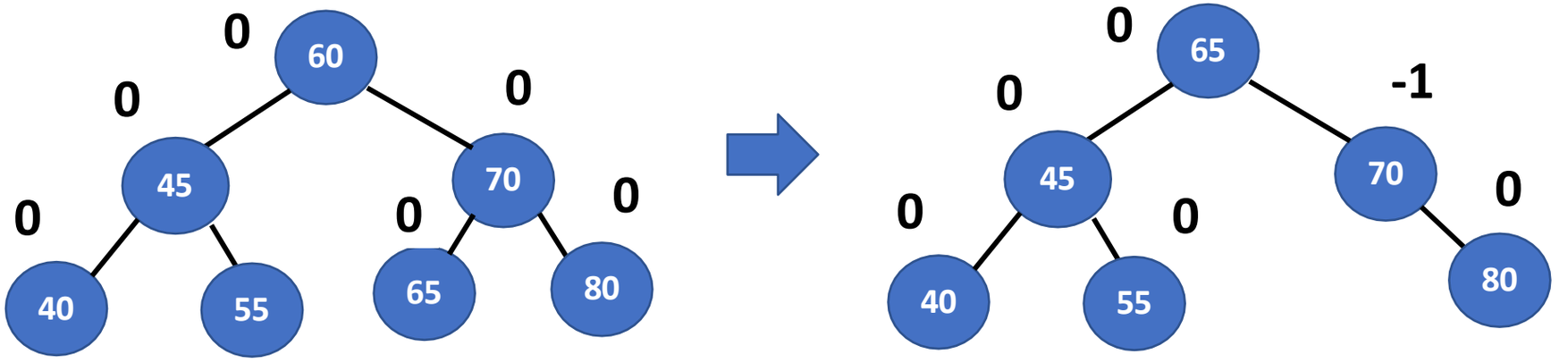
Example – Deletions in AVL tree

Delete 50:

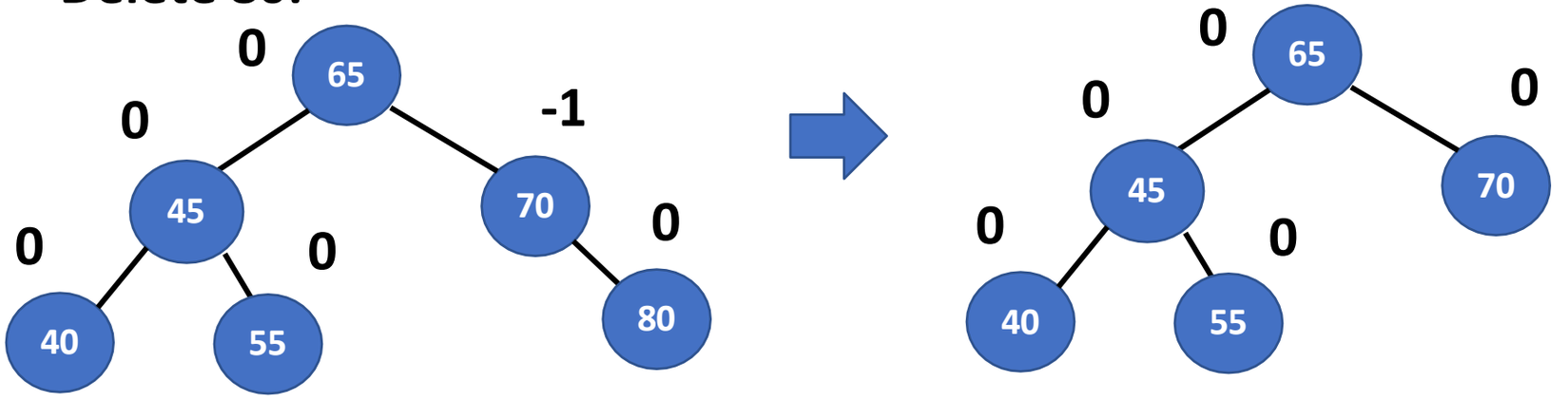


Example – Deletions in AVL tree

Delete 60:

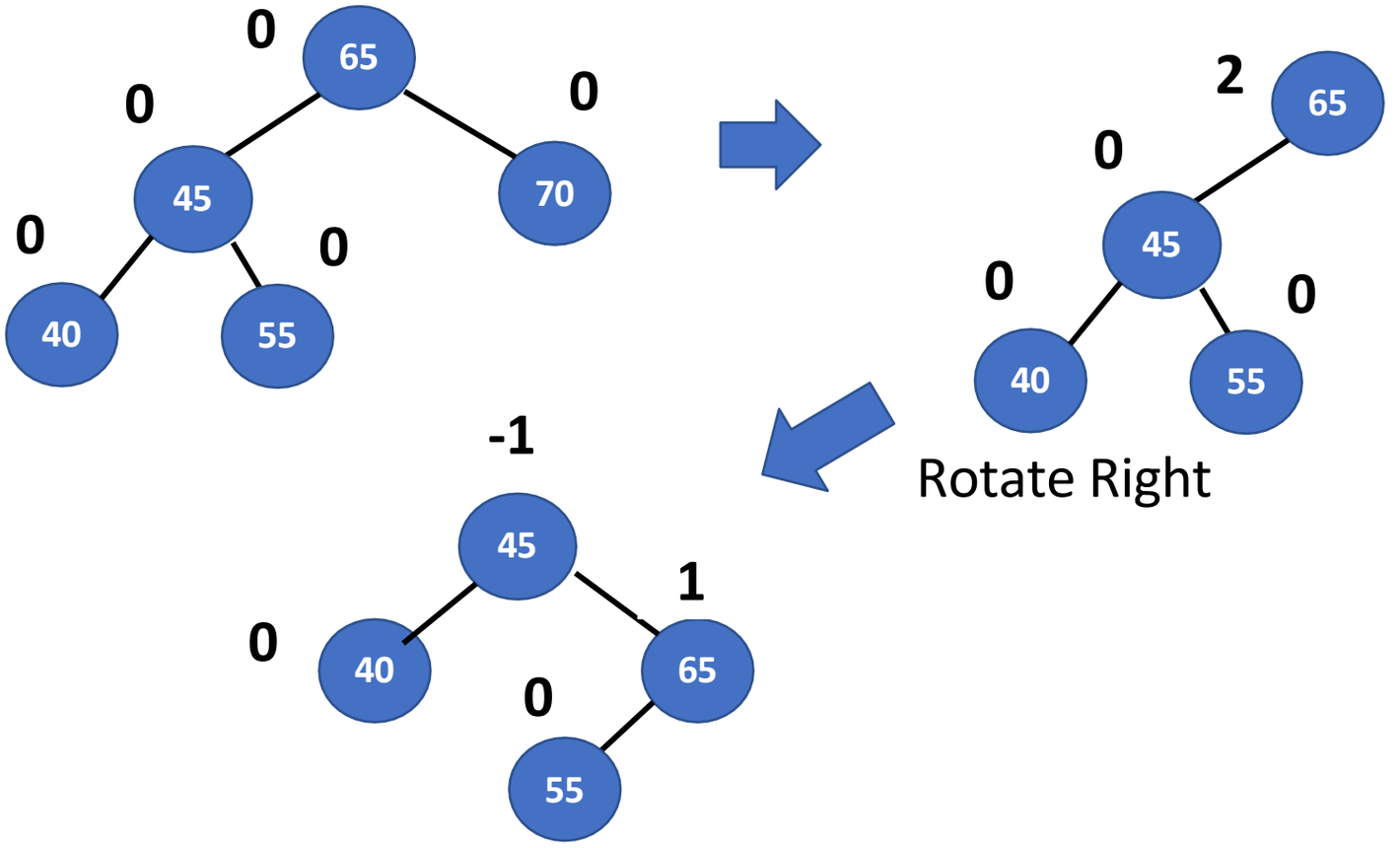


Delete 80:



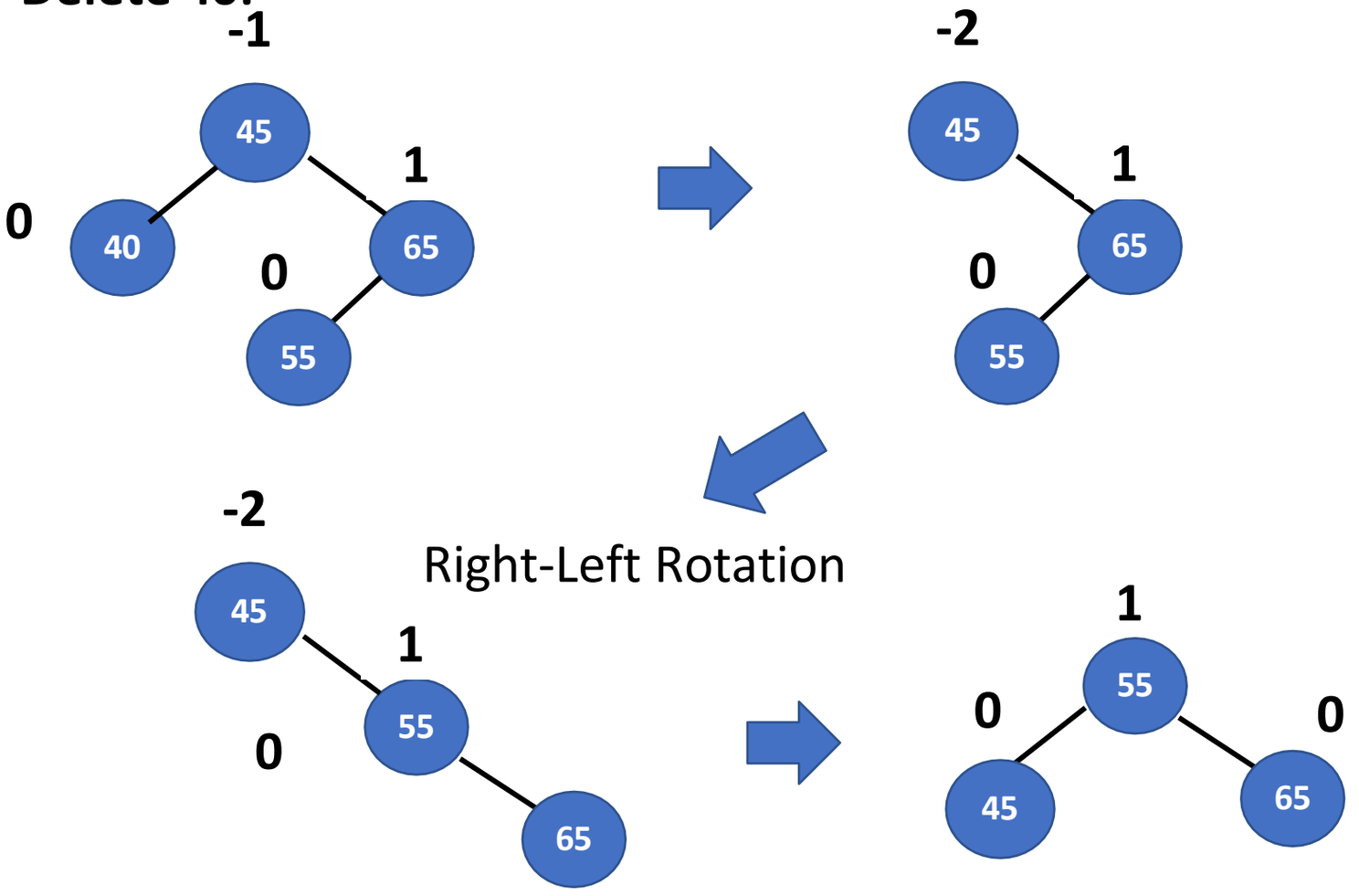
Example – Deletions in AVL tree

Delete 70:



Example – Deletions in AVL tree

Delete 40:



THANK YOU

Sandesh B. J
Department of Computer Science & Engineering

Sandesh_bj@pes.edu

+91 80 6618 6649

DATA STRUCTURES AND ITS APPLICATIONS
UE19CS202

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

BST Implementation using Dynamic Allocation: Insertion

Shylaja S S
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree – An Application of Binary Tree

Background

Problem : find a target key in a list of elements

Sequential: Potentially enumerate every key

Ordered List: Searching can be done on logn

Frequent insertions and deletions : Ordered List is much slower

Solution: Binary Trees provide an excellent solution to this by organizing every element in the list as a node in the tree

Binary Search Tree: Definition

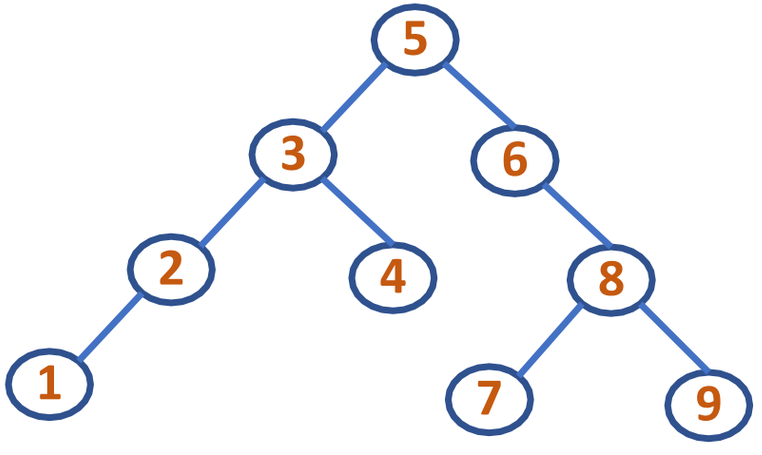
A Binary Search Tree is a binary tree which has the following properties:

- all the elements in the left subtree of a node **n** are less than the contents of node **n**
- all the elements in the right subtree of a node **n** are greater than or equal to the contents of node **n**

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree – An Application of Binary Tree

A Binary Search Tree with the nodes inserted in the order: 5, 3, 6, 4, 2, 8, 1,7, 9



DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation

Linked implementation

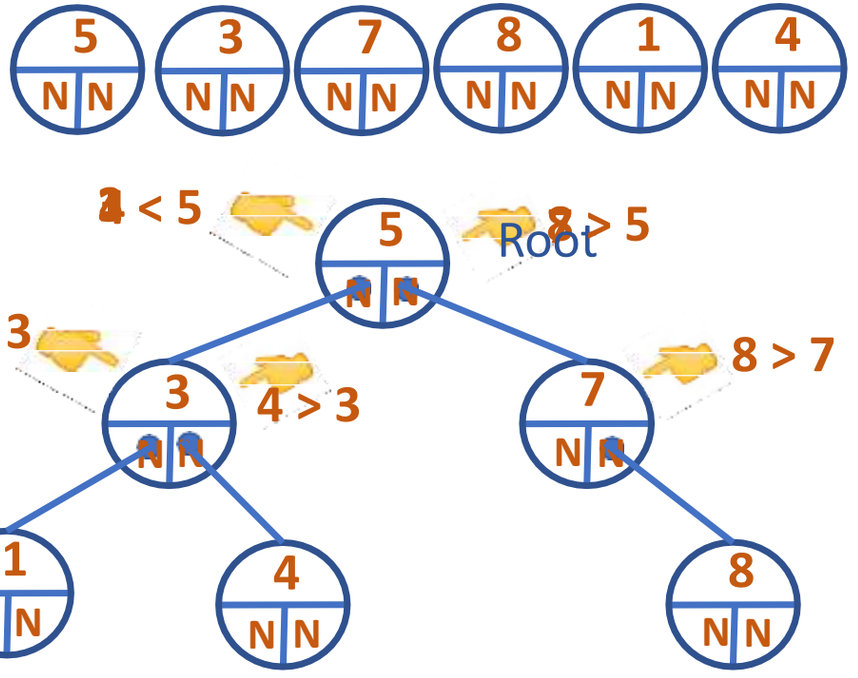
Here every node will have its own **info** along with the **links to left child** and **right child**

```
typedef struct tree_linked
{
  int info;
  struct tree_linked *left,*right;
}NODE;

NODE *root=NULL; //root points to Root of the tree and initially it is null
```

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation

Linked implementation: 5, 3, 7, 8, 1, 4



THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

+91 9449867804

DATA STRUCTURES AND ITS APPLICATIONS
UE19CS202

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

BST: Deletion Operations

Shylaja S S
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion

Deletion of a Node in Binary Search Tree

case1: Node with no child (leaf node)

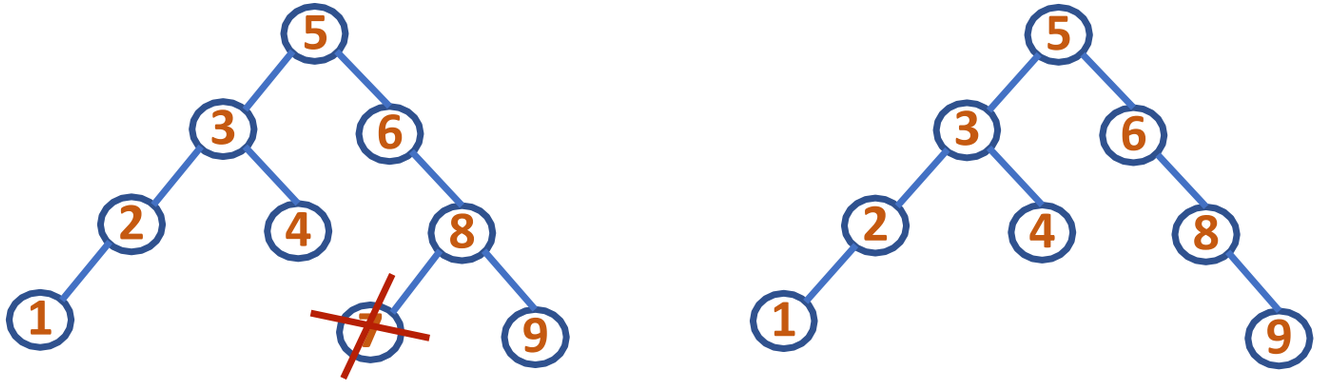
case2: Node with 1 child

case3: Node with 2 children

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion

case1: Node with no child (leaf node)



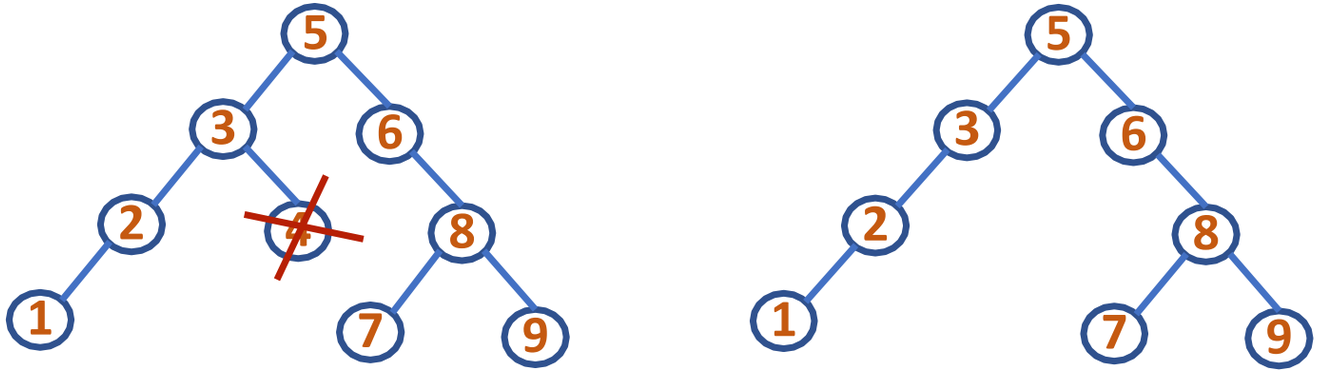
To delete the node with info 7:

- Set its parent's left child field to point to NULL
- Free memory allocated to node with info 7

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion

case1: Node with no child (leaf node)

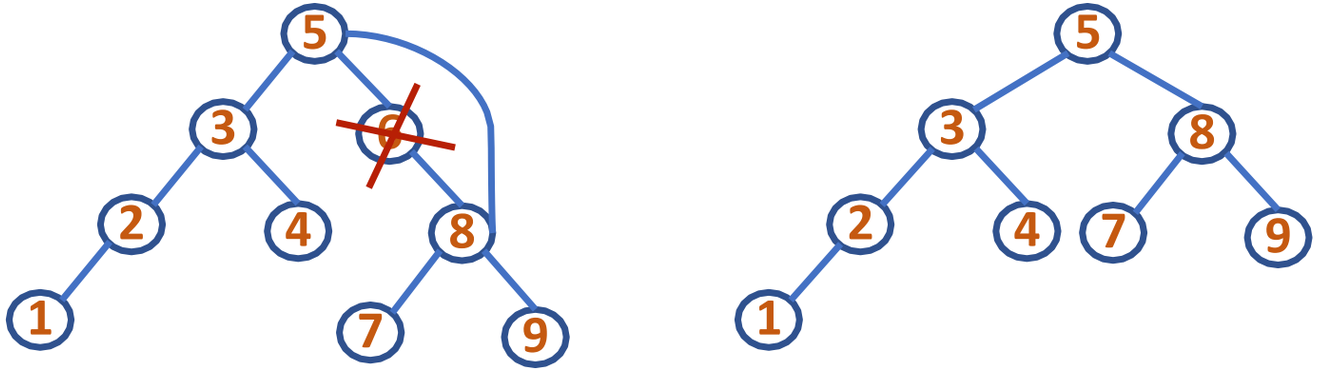


To delete the node with info 4:

- Set its parent's right child field to point to NULL
- Free memory allocated to node with info 4

Binary Search Tree - Deletion

case2: Node with 1 child



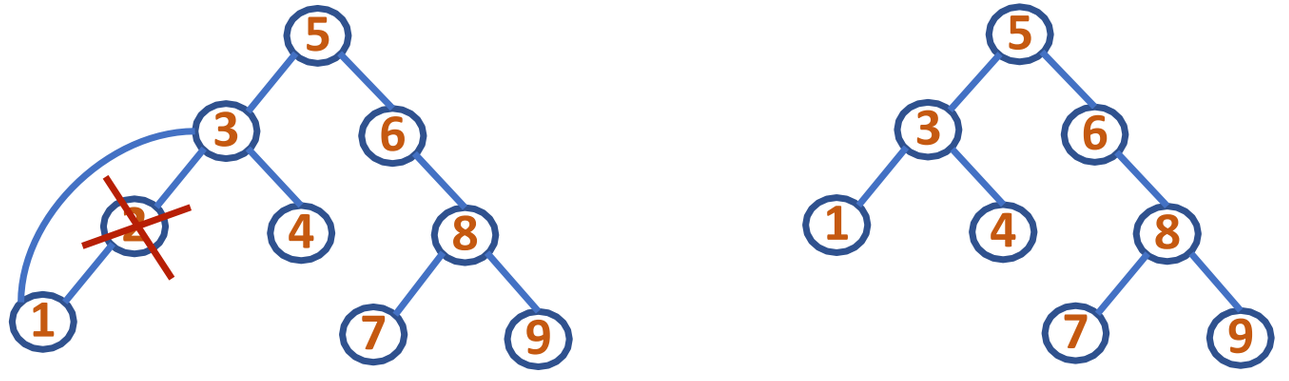
To delete the node with info 6:

- Set its parent's right child field to point to its only child
- Free memory allocated to node with info 6

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion

case2: Node with 1 child



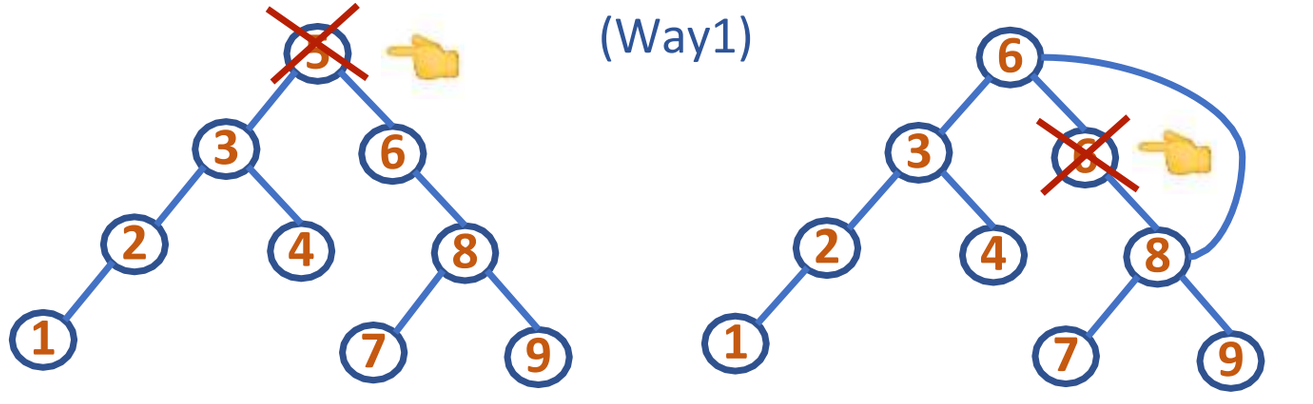
To delete the node with info 2:

- Set its parent's left child field to point to its only child
- Free memory allocated to node with info 2

DATA STRUCTURES AND ITS APPLICATIONS

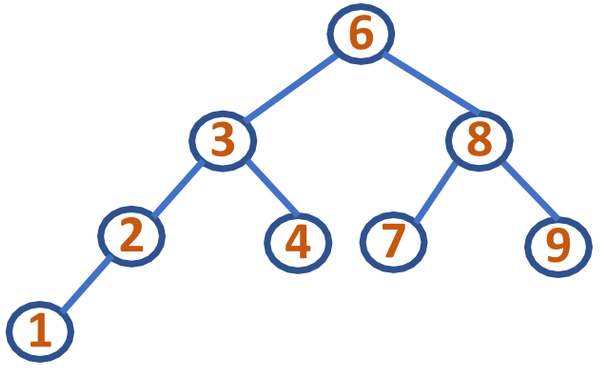
Binary Search Tree - Deletion

case3: Node with 2 children (Replace with inorder successor) (Way1)



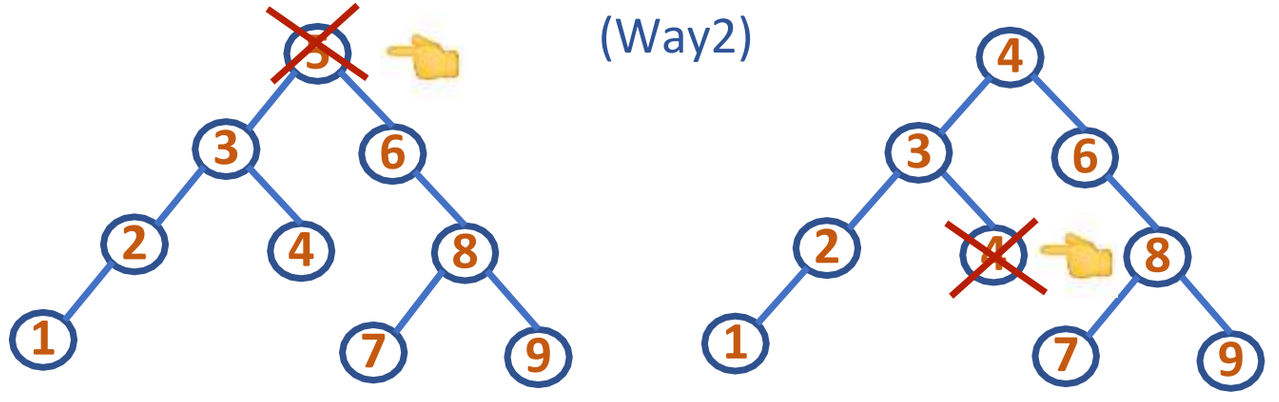
To delete the node with info 5:

- Replace 5 with its **inorder successor** and delete that inorder successor
- Now case3 has got changed to case2 (In general may change to case2 or case1)



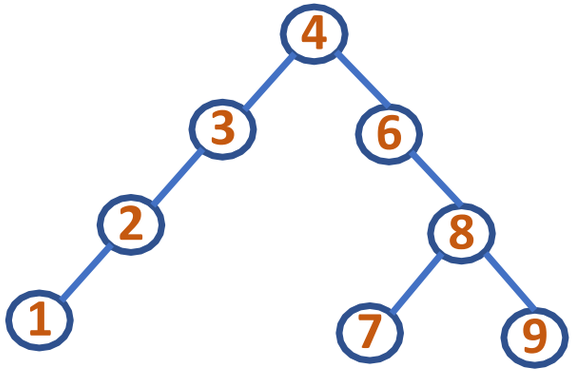
Binary Search Tree - Deletion

case3: Node with 2 children (Replace with inorder predecessor) (Way2)



To delete the node with info 5:

- Replace 5 with its **inorder predecessor** and delete that inorder predecessor
- Here case3 has got changed to case1 (In general may change to case2 or case1)



THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

+91 9449867804

DATA STRUCTURES AND ITS APPLICATIONS
UE19CS202

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

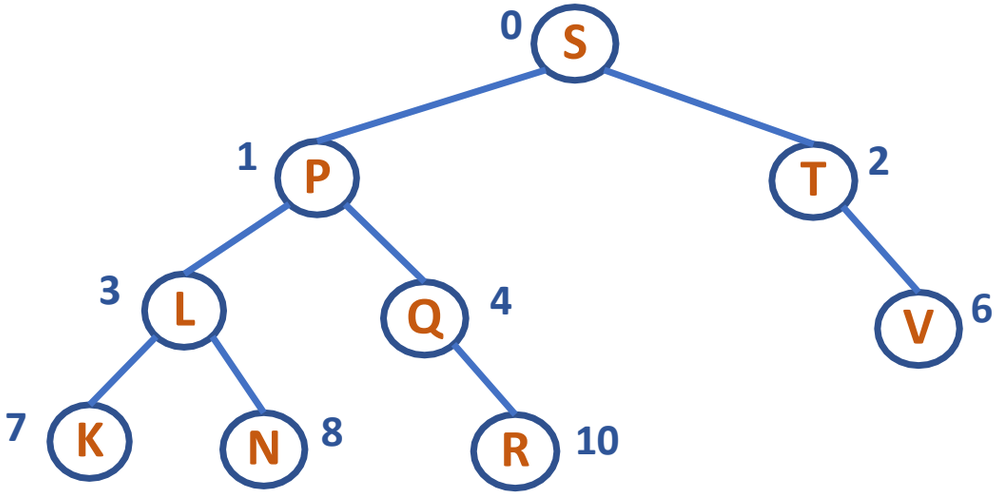
BST: Implementation using Arrays

Shylaja S S
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation

Array Implementation (Implicit implementation)



S	P	T	L	Q	...	V	K	N	...	R
0	1	2	3	4	5	6	7	8	9	10

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation

Array Implementation (Implicit implementation)

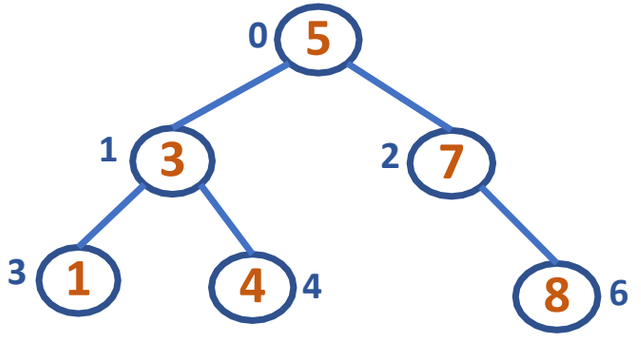
```
typedef struct tree_array
{
        int info;
        int used;
}NODE;
```

- NODE bst[MAX]; //here bst is an array of nodes
- each node has its data and another field by name used to contain whether it is a valid node or not
- used = 1 or 0

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation

Array Implementation: 5, 3, 7, 8, 1, 4

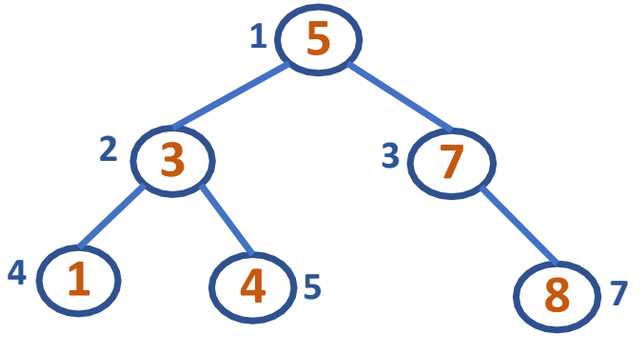


Root Position: i = 0

Left Child Position: 2i + 1

Right Child Position: 2i + 2

OR



Root Position: i = 1

Left Child Position: 2i

Right Child Position: 2i + 1

info	5	3	7	1	4		8
used	1	1	1	1	1	0	1
Position: i	0	1	2	3	4	5	6

info		5	3	7	1	4		8
used	0	1	1	1	1	1	0	1
Position: i	0	1	2	3	4	5	6	7

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation

Array Implementation: 5, 3, 7, 8, 1, 4

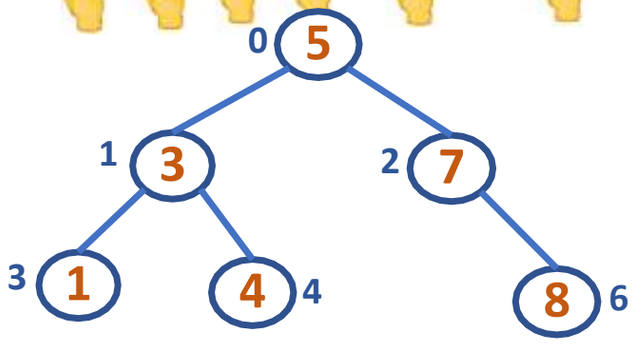
Root Position: i = 0
Left Child Position: 2i + 1
Right Child Position: 2i + 2

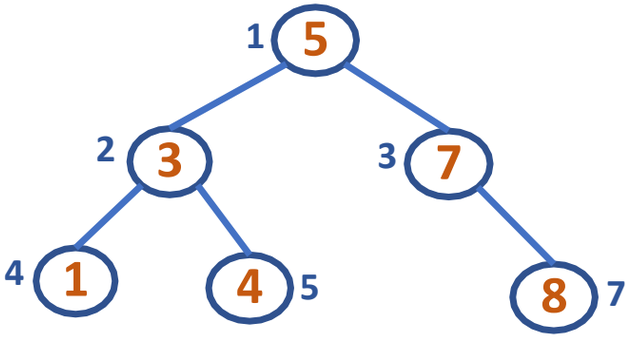
OR

Root Position: i = 1
Left Child Position: 2i
Right Child Position: 2i + 1

info	5	3	7	1	4		8
used	1	1	1	1	1	0	1
Position: i	0	1	2	3	4	5	6

info		5	3	7	1	4		8
used	0	1	1	1	1	1	0	1
Position: i	0	1	2	3	4	5	6	7





THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

+91 9449867804

DATA STRUCTURES AND ITS APPLICATIONS
UE19CS202

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Threaded BST and its Implementation

Shylaja S S
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree

Motivation

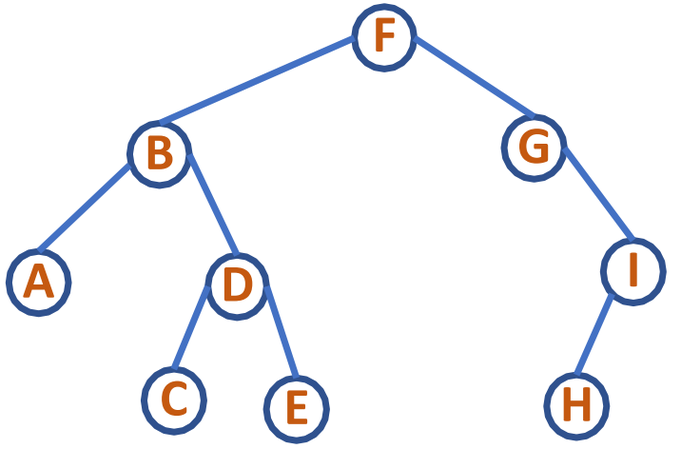
- Iterative Inorder Traversal requires Explicit stack
- Costly
- Since we loose track of address as and when we navigate, Node addresses were stacked
- If this can be achieved through some other less expensive mechanism, we can eliminate the use of explicit stack
- Small structural modification carried on Binary tree will solve the above problem

Threaded Binary Search Tree

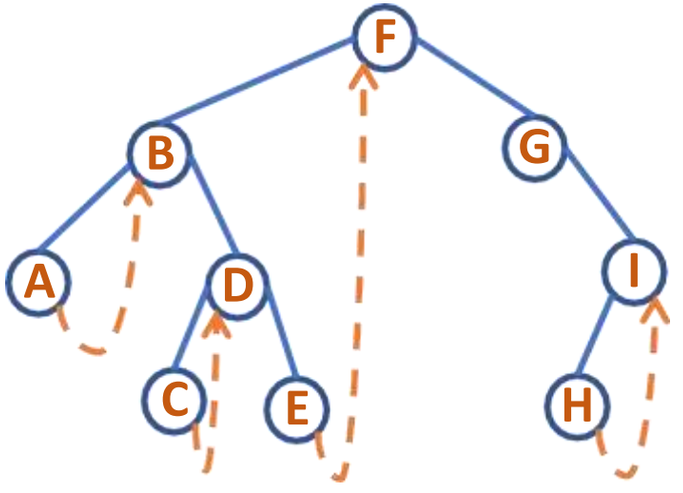
- We can use the right pointer of a node to point to the inorder successor if in case it is not pointing to the child. Such a tree is called **Right-In Threaded** Binary Tree
- If we use the left pointer to store the inorder predecessor, the tree is called **Left-In Threaded** Binary Tree
- If we use both the pointers, the tree is called **In Threaded** Binary Tree

Threaded Binary Search Tree

Right-In Threaded Binary Tree



Binary Tree



Right-In Threaded Binary Tree

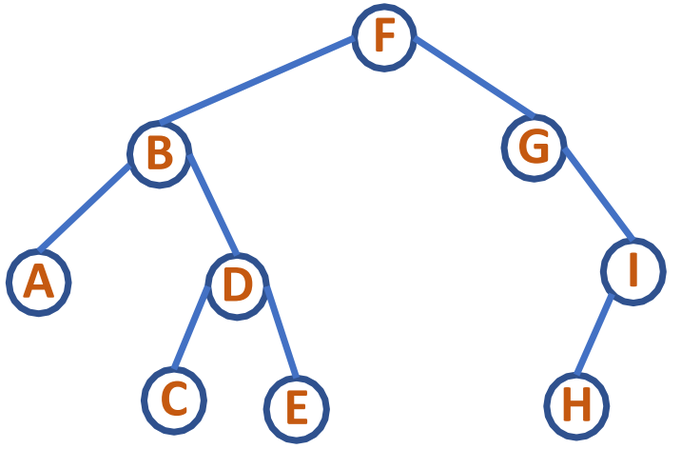
Inorder Traversal:

A B C D E F G H I

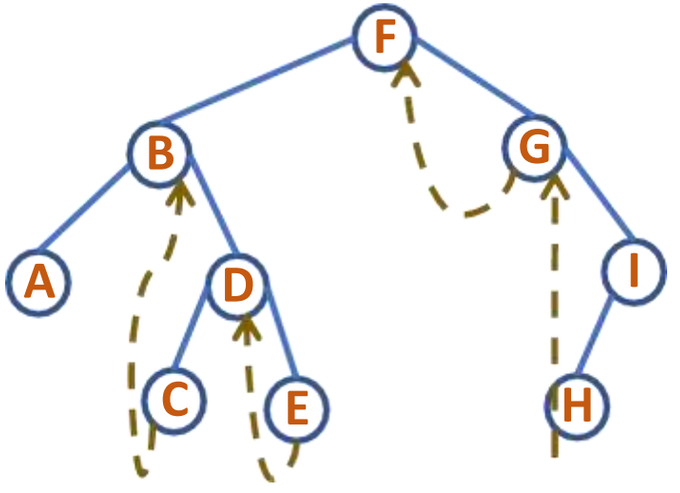
Nodes with Right Pointer NULL	A	C	E	H	I
Inorder Successor	B	D	F	I	-

Threaded Binary Search Tree

Left-In Threaded Binary Tree



Binary Tree



Left-In Threaded Binary Tree

Inorder Traversal:

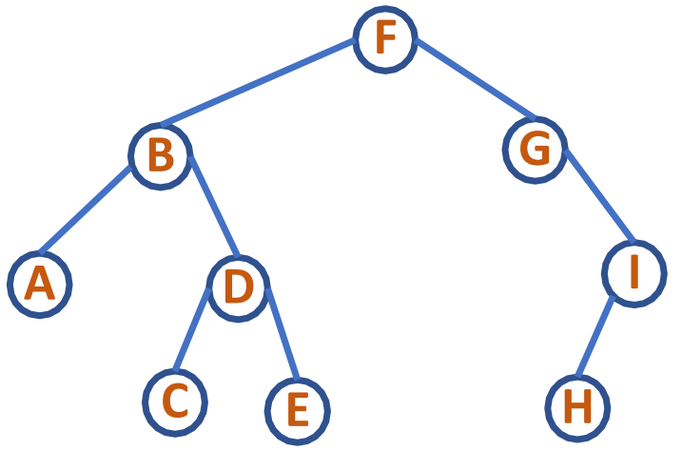
A B C D E F G H I

Nodes with Left Pointer NULL	A	C	E	G	H
Inorder Predecessor	-	B	D	F	G

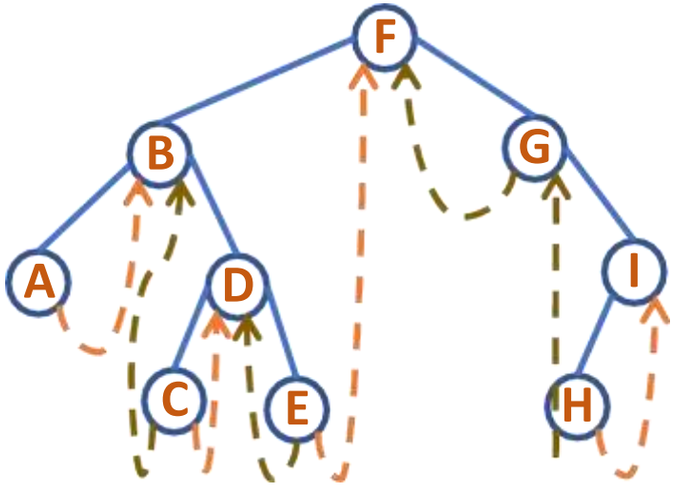
DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree

In Threaded Binary Tree



Binary Tree



In Threaded Binary Tree

Threaded Binary Search Tree: Implementation

Right In Threaded Binary Tree

```
typedef struct node
{
        int info;
        struct node *left;      // pointer to left child
        struct node *right;     // pointer to right child
        int rthread;            // rthread is TRUE if right is NULL
                                // or a non-NULL thread
}NODE;
```

Node Structure

DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree: Implementation

```
NODE* createNode(int e)
{
        NODE* temp=malloc(sizeof(NODE));
        temp->info=e;
        temp->left=NULL;
        temp->right=NULL;
        temp->rthread=1;
        return temp;          // Returns: 2000
}
```

createNode(57)

temp ->

Let Address of this node on Heap: 2000

Threaded Binary Search Tree: Implementation

Right In Threaded Binary Tree: 57, 25, 28

- A node is created with rthread set to TRUE
- **insert 57**

Address: 800

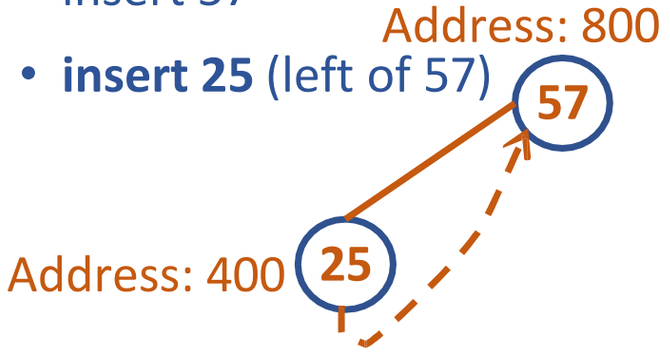
Node Structure

info	left	right	rthread
57	NULL	NULL	1

Threaded Binary Search Tree: Implementation

Right In Threaded Binary Tree: 57, 25, 28

- A node is created with rthread set to TRUE
- insert 57
- **insert 25** (left of 57)



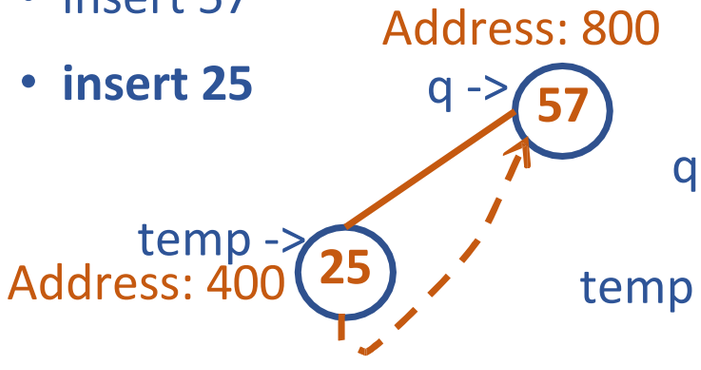
Node Structure

info	left	right	rthread
57	400	NULL	1
25	NULL	800	1

Threaded Binary Search Tree: Implementation

Right In Threaded Binary Tree: 57, 25, 28

- A node is created with rthread set to TRUE
- insert 57
- **insert 25**



Node Structure

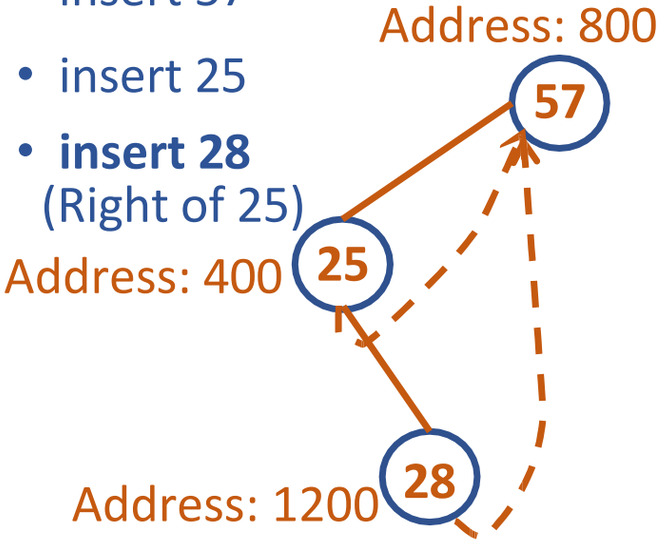
	info	left	right	rthread
q ->	57	NULL	NULL	1
temp ->	25	NULL	800	1

```
void setLeft(NODE* q, int e) {      //set node with info e to left of q
        NODE* temp=createNode(e);           //e=25
        q->left=temp;
        temp->right=q;
}
```

Threaded Binary Search Tree: Implementation

Right In Threaded Binary Tree: 57, 25, 28

- A node is created with rthread set to TRUE
- insert 57
- insert 25
- **insert 28**
  (Right of 25)



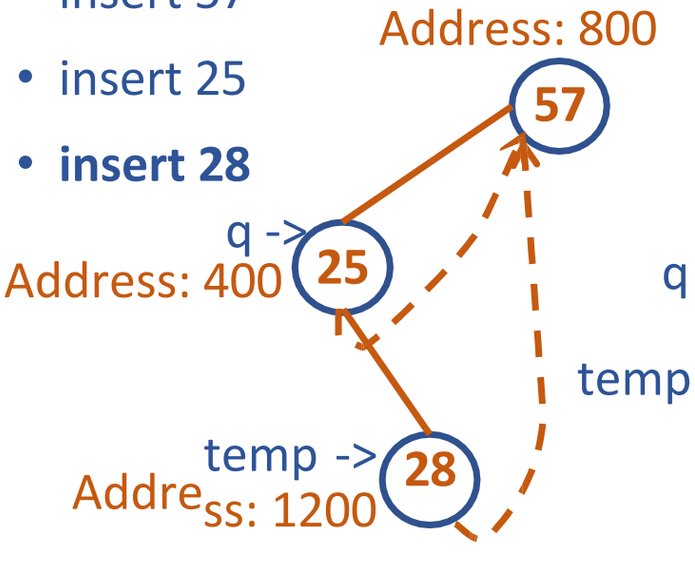
Node Structure

info	left	right	rthread
57	400	NULL	1
25	NULL	1200	0
28	NULL	800	1

Threaded Binary Search Tree: Implementation

Right In Threaded Binary Tree: 57, 25, 28

- A node is created with rthread set to TRUE
- insert 57
- insert 25
- **insert 28**



Node Structure

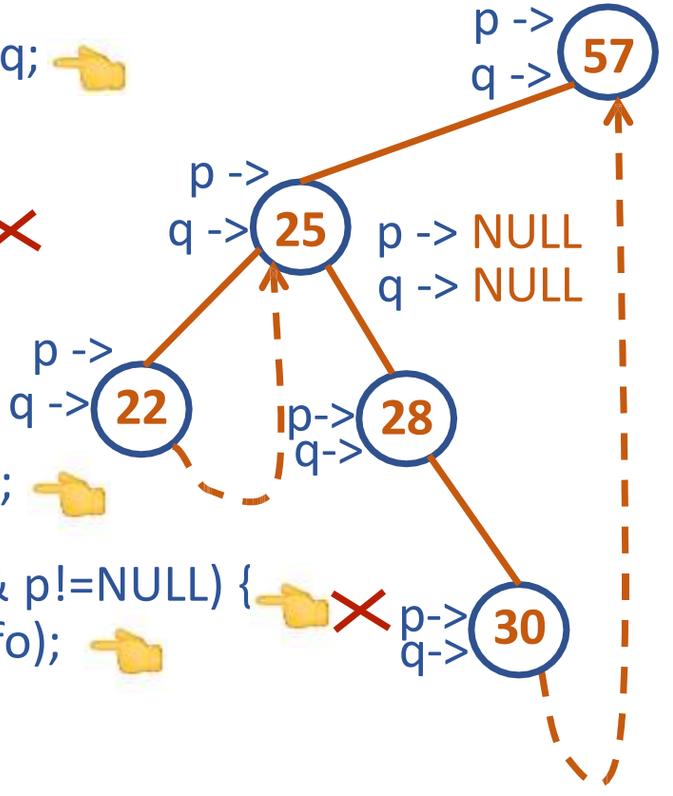
	info	left	right	rthread
	57	400	NULL	1
q ->	25	NULL	1200	0
temp ->	28	NULL	800	1

```
void setRight(NODE* q,int e) {
   NODE* temp=createNode(e);
   temp->right=q->right;
   q->right=temp;
   q->rthread=0;
   }
```

DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree: Inorder Traversal

```
void inOrder(NODE *root) {
   NODE *p=root;   NODE *q;
   do{
         q=NULL;
         while(p!=NULL) {
            q=p;
            p=p->lett;
         }
         if(q!=NULL) {
            printf("%d ",q->info);
            p=q->right;
            while(q->rthread && p!=NULL) {
               printf("%d ",p->info);
               q=p;
               p=p->right;
            }
         }
   }while(q!=NULL);
}
```



q ->NULL

rthread is TRUE for nodes with info: 22, 30, 57

Inorder Traversal:

22 25 28 30 57

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

+91 9449867804

DATA STRUCTURES AND ITS APPLICATIONS
UE19CS202

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of Binary Expression Tree

Shylaja S S
Department of Computer Science & Engineering

Expression Tree

- An expression can be represented using the **Expression Tree** data structure
- Such a tree is built normally for translating the code as data and then analysing and evaluating expressions
- **Immutable**: To change the expression another tree has to be constructed

DATA STRUCTURES AND ITS APPLICATIONS

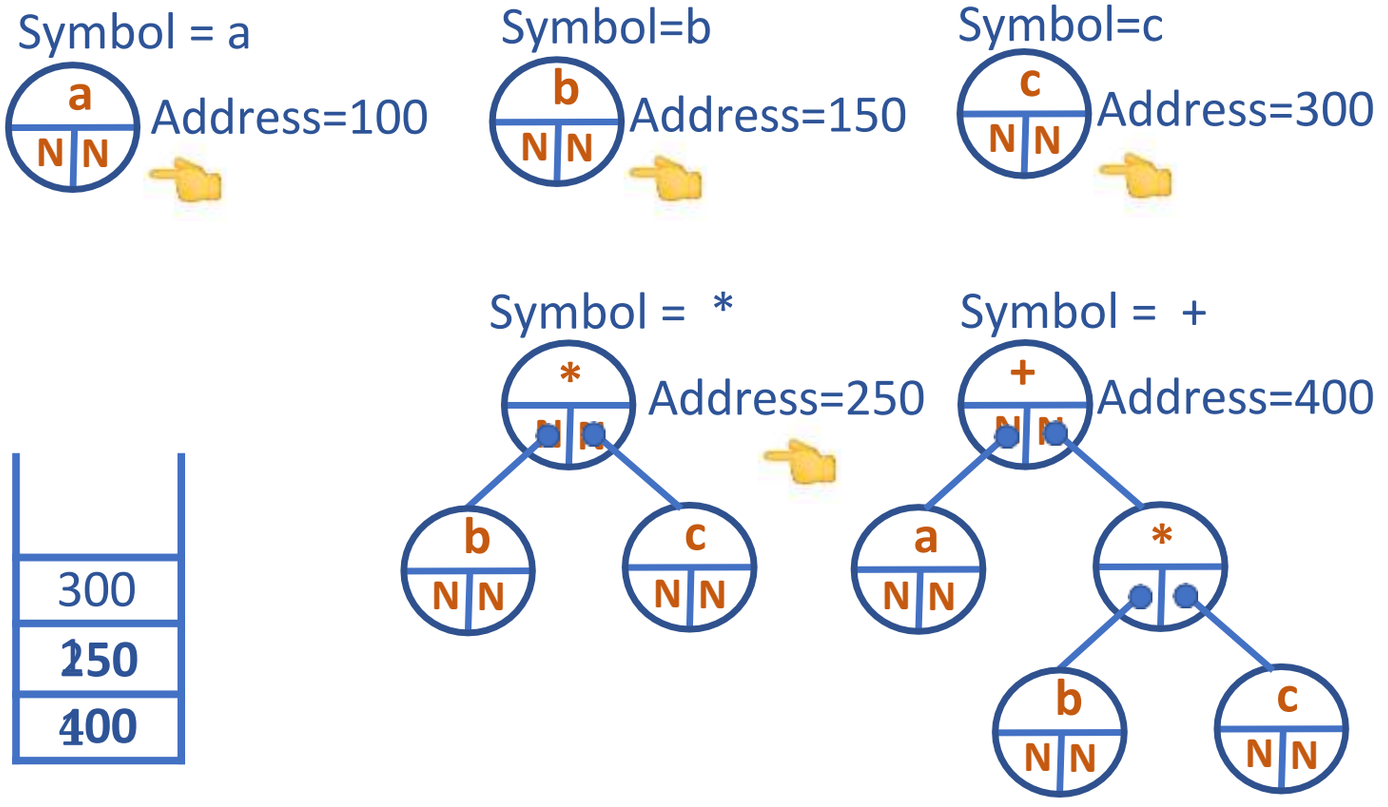
Expression Tree Construction

- Normally a postfix expression is used in constructing the Expression tree
- When an operand is received, a new node is created which will be a leaf in the expression tree
- If an operator, it connects to two leaves
- Stack DS is used as intermediary storing place of node's address

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction

Postfix Expression: abc*+



Expression Tree Construction

- Scan the postfix expression till the end, one symbol at a time
  - Create a new node, with symbol as info and left and right link as NULL
  - If symbol is an operand, push address of node to stack
  - If symbol is an operator
    - Pop address from stack and make it right child of new node
    - Pop address from stack and make it left child of new node
    - Now push address of new node to stack
- Finally, stack has only element which is the address of the root of expression tree

DATA STRUCTURES AND ITS APPLICATIONS

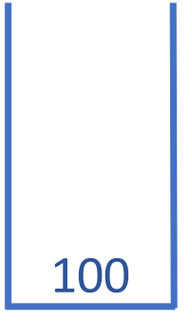
Expression Tree Construction

Postfix Expression: **a bc * +**

- Scan the postfix expression till the end, one symbol at a time
  - Create a new node, with symbol as info and left and right link as NULL
  - If symbol is an operand, push address of node to stack
  - If symbol is an operator
    - Pop the address from stack and make it right child of new node
    - Pop the address from stack and make it left child of new node
    - Now push address of new node to stack
- Finally, stack has only element which is the address of the root of expression tree

Symbol = a

Address=100



DATA STRUCTURES AND ITS APPLICATIONS

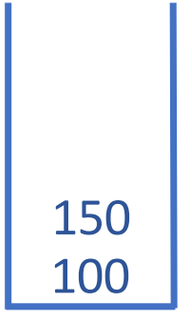
Expression Tree Construction

Postfix Expression: **a bc * +**

- Scan the postfix expression till the end, one symbol at a time
  - Create a new node, with symbol as info and left and right link as NULL
  - If symbol is an operand, push address of node to stack
  - If symbol is an operator
    - Pop the address from stack and make it right child of new node
    - Pop the address from stack and make it left child of new node
    - Now push address of new node to stack
- Finally, stack has only element which is the address of the root of expression tree

Symbol = b

Address=150



DATA STRUCTURES AND ITS APPLICATIONS

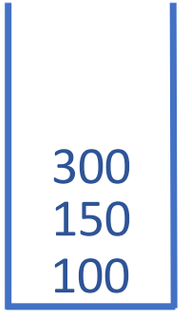
Expression Tree Construction

Postfix Expression: **a b c * +**

- Scan the postfix expression till the end, one symbol at a time
  - Create a new node, with symbol as info and left and right link as NULL
  - If symbol is an operand, push address of node to stack
  - If symbol is an operator
    - Pop the address from stack and make it right child of new node
    - Pop the address from stack and make it left child of new node
    - Now push address of new node to stack
- Finally, stack has only element which is the address of the root of expression tree

Symbol = c

Address=300

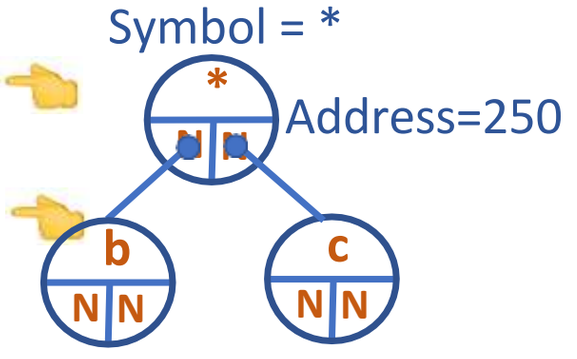


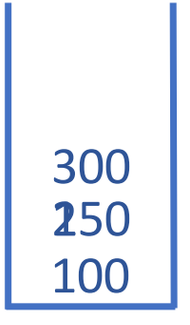
DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction

Postfix Expression: **a bc * +**

- Scan the postfix expression till the end, one symbol at a time
  - Create a new node, with symbol as info and left and right link as NULL
  - If symbol is an operand, push address of node to stack
  - If symbol is an operator
    - Pop the address from stack and make it right child of new node
    - Pop the address from stack and make it left child of new node
    - Now push address of new node to stack
- Finally, stack has only element which is the address of the root of expression tree



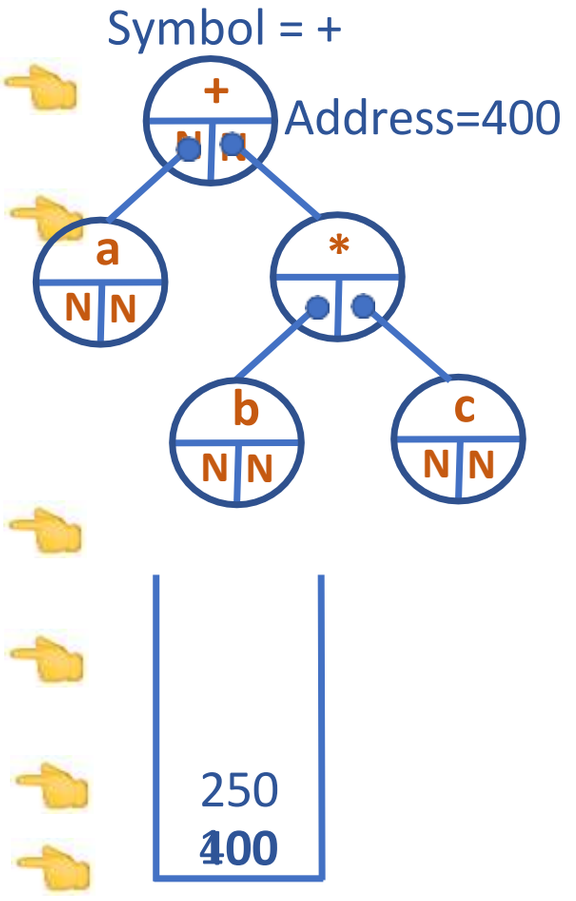


DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction

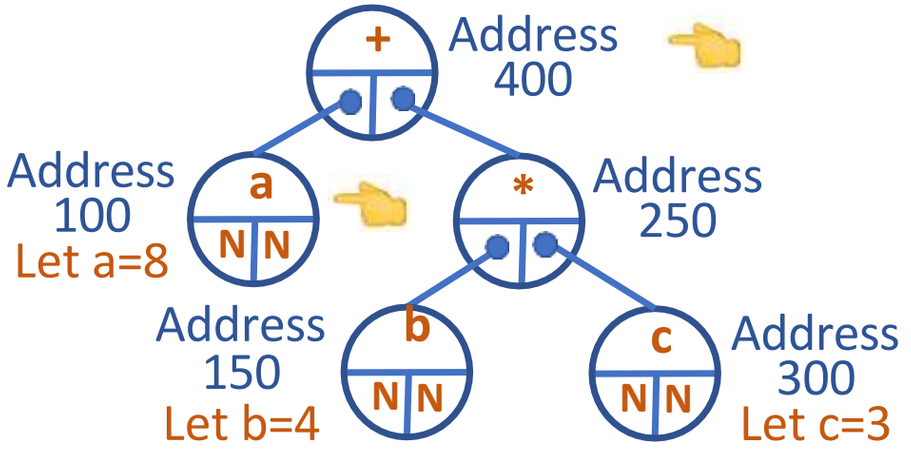
Postfix Expression: **a bc * +**

- Scan the postfix expression till the end, one symbol at a time
  - Create a new node, with symbol as info and left and right link as NULL
  - If symbol is an operand, push address of node to stack
  - If symbol is an operator
    - Pop the address from stack and make it right child of new node
    - Pop the address from stack and make it left child of new node
    - Now push address of new node to stack
- Finally, stack has only element which is the address of the root of expression tree



DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Evaluation



eval(400)
return eval(100) + eval(250)

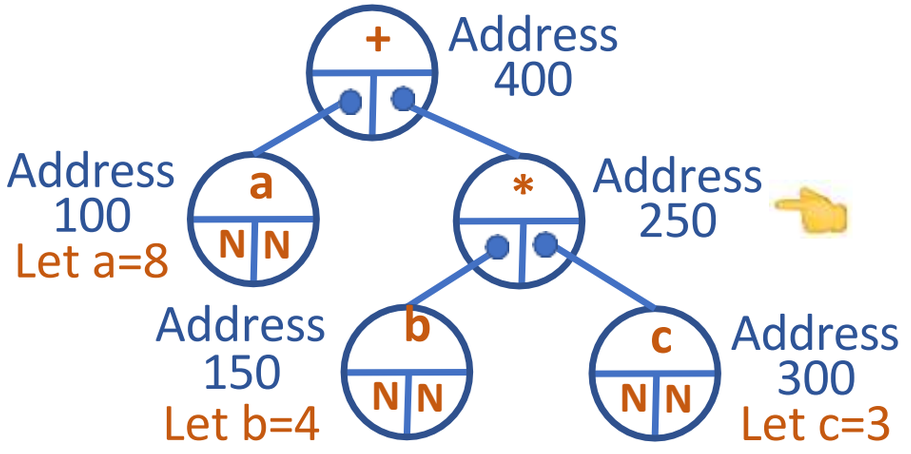
eval(100)
return 8

- Think in terms of recursion

```
eval(t)  // 't' has the address of the root node of expression tree
  if t->data is an operator
    return eval (t->left) t->data eval(t->right)
  return t->data
```

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Evaluation



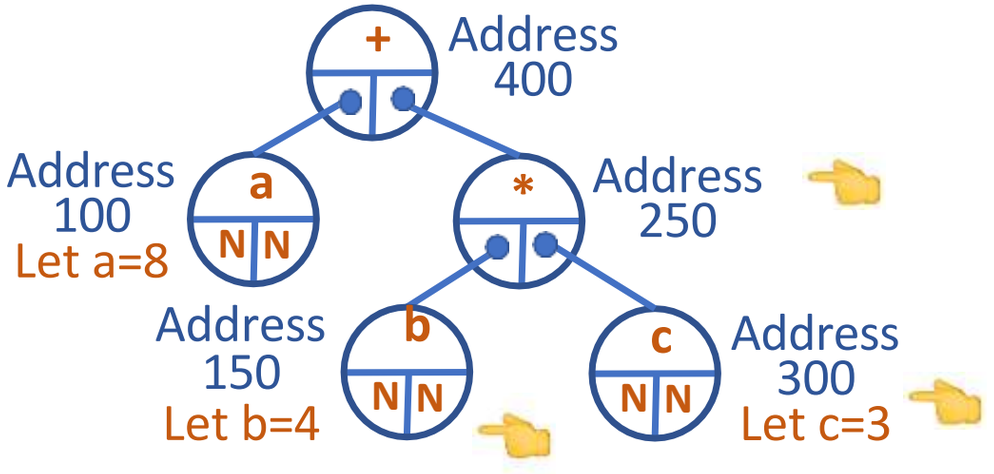
eval(400)
return 8 + eval(250)
eval(250)
return eval(150) * eval(300)

- Think in terms of recursion

```
eval(t) // 't' has the address of the root node of expression tree
  if t->data is an operator
    return eval (t->left) t->data eval(t->right)
  return t->data
```

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Evaluation



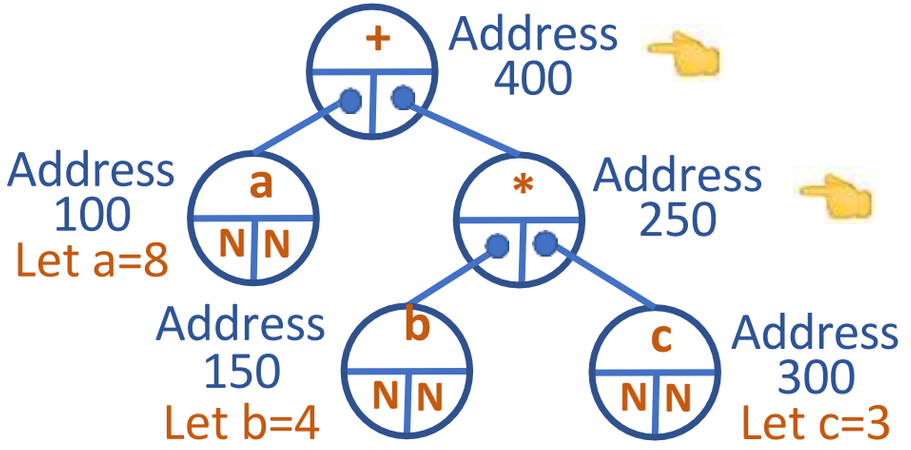
eval(400)
return 8 + eval(250)
eval(250)
return eval(150) * eval(300)
eval(150)
return 4
eval(300)
return 3

- Think in terms of recursion

```
eval(t) // 't' has the address of the root node of expression tree
  if t->data is an operator
    return eval (t->left) t->data eval(t->right)
  return t->data
```

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Evaluation



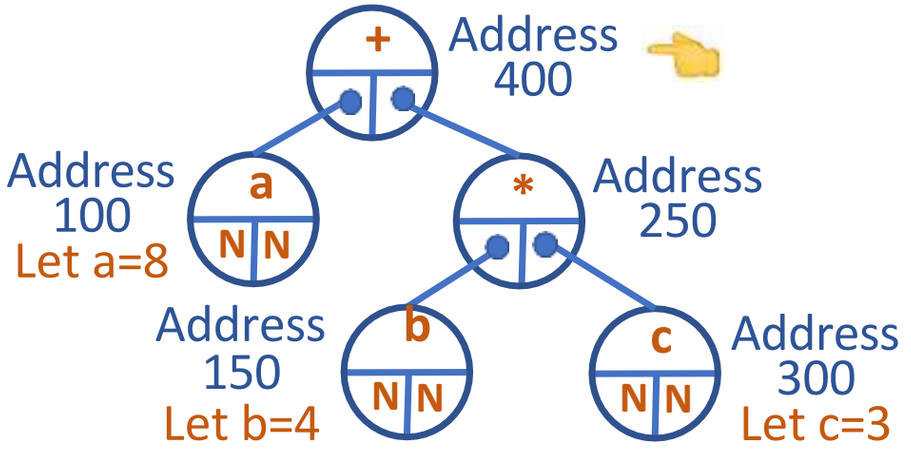
eval(400)
return 8 + eval(250)
eval(250)
return 12

- Think in terms of recursion

```
eval(t)     // 't' has the address of the root node of expression tree
  if t->data is an operator
    return eval (t->left) t->data eval(t->right)
  return t->data
```

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Evaluation



eval(400)

return 208 + 12

Postfix abc*+ : 20

- Think in terms of recursion

```
eval(t)     // 't' has the address of the root node of expression tree
   if t->data is an operator
     return eval (t->left) t->data eval(t->right)
   return t->data
```

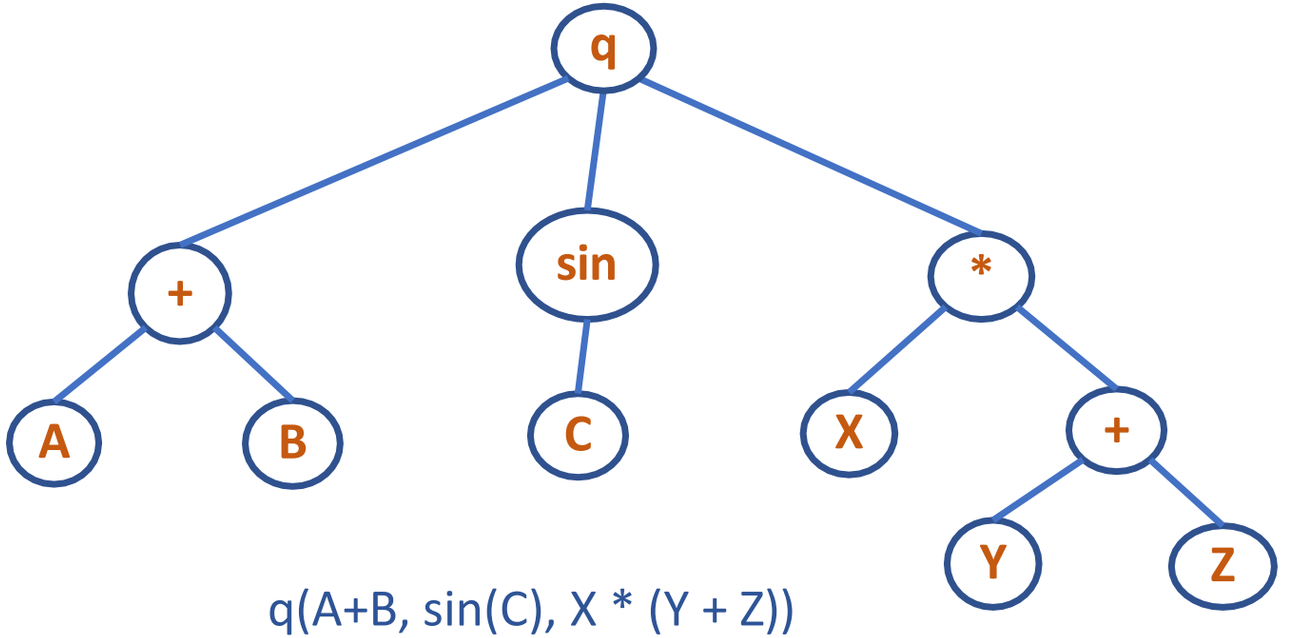
General Expression Tree Evaluation

```
struct treenode
{
  short int utype;
   union{
      char operator[MAX];
       float val;
     }info;
   struct treenode *child;
   struct treenode *sibling;
};
typedef struct treenode TREENODE;
```

DATA STRUCTURES AND ITS APPLICATIONS

General Expression Tree Evaluation

Here node can be either an operand or an operator



q(A+B, sin(C), X * (Y + Z))

Tree representation of an arithmetic expression

General Expression Tree Evaluation

```
void replace(TREENODE *p)
{
  float val;
  TREENODE *q,*r;
  if(p->utype == operator)
  {
   q = p->child;
   while(q != NULL)
    {
      replace(q);
      q = q->next;}
    }
```

General Expression Tree Evaluation

```
    value = apply(p);
    p->utype = OPERAND;
    p->val = value;
    q = p->child;
    p->child = NULL;
    while(q != NULL)
    {
        r = q;
        q = q->next;
        free(r);
    }
  }
}
```

General Expression Tree Evaluation

```
float eval(TREENODE *p)
{
  replace(p);
  return(p->val);
  free(p);
 }
```

Constructing a Tree

```
void setchildren(TREENODE *p,TREENODE *list)
{
  if(p == NULL) {
    printf("invalid insertion");
    exit(1);
  }
 if(p->child != NULL) {
   printf("invalid insertion");
   exit(1);
 }
 p->child = list;
}
```

Constructing a Tree

```
void addchild(TREENODE *p,int x)
{
  TREENODE *q;
  if(p==NULL)
  {
    printf("void insertion");
    exit(1);
  }
```

Constructing a Tree

```
r = NULL;
q = p->child;
while(q != NULL)
{
  r = q;
  q = q->next;
}
q = getnode();
q->info = x;
q->next = NULL;
```

Constructing a Tree

```
if(r==NULL)
   p->child=q;
  else
   r->next=q;
}
```

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

+91 9449867804

DATA STRUCTURES AND ITS APPLICATIONS
UE19CS202

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of Priority Queue using min heap/max heap

Shylaja S S
Department of Computer Science & Engineering

Ascending and Descending Heap

- Ascending Heap: Root will have the lowest element. Each node's data is greater than or equal to its parent's data. It is also called min heap.

- Descending Heap: Root will have the highest element. Each node's data is lesser than or equal to its parent's data. It is also called max heap.

Priority Queue using Heap

- Priority Queue is a Data Structure in which intrinsic ordering of the elements does determine the results of its basic operations

- Ascending Priority Queue: is a collection of items into which items can be inserted arbitrarily and from which only the smallest item can be removed

- Descending Priority Queue: is a collection of items into which items can be inserted arbitrarily and from which only the largest item can be removed

Priority Queue using Heap

Consider the properties of a heap

- The entry with largest key is on the top(Descending heap) and can be removed immediately. But O(logn) time is required to readjust the heap with remaining keys

- If another entry need to be done, it requires O(logn)

- Therefore, heap is advantageous to implement a Priority Queue

Priority Queue using Heap: Implementation

- dpq: Array that implements descending heap of size k (position from 0 to k-1)
- pqinsert(dpq,k,elt): insert element into the heap dpq of size k. Size increases to k+1
- This insertion is done using siftup operation

Priority Queue using Heap: Implementation

Algorithm for siftup

```
c = k;
p = (c-1)/2;
while(c>0 && dpq[p]<elt) {
   dpq[c]=dpq[p];
   c=p;
   p=(c-1)/2;
}
dpq[c]=elt;
```

Priority Queue using Heap: Implementation

```
pqmaxdelete(dpq,k)  //for a descending heap of size k
p = dpq[0];
adjustheap(0,k-1)
return p;
```

Priority Queue using Heap: Implementation

Algorithm largechild(p,m)

```
c = 2*p+1;
if(c+1 <= m && x[c] < x[c+1])
  c=c+1;
if(c > m)
  return -1;
else
  return (c);
```

Priority Queue using Heap: Implementation

Algorithm adjustheap(root,k) //recursive

```
p = root;
c = largechild(p,k-1);
if(c >= 0 && dpq[k] < dpq[c]){
  dpq[p] = dpq[c];
  adjustheap(c,k);
}
else
  dpq[p] = dpq[k];
```

Priority Queue using Heap: Implementation

Iterative version

```
p = root;
kvalue = dpq[k];
c = largechild(p,k-1);
while(c >= 0 && kvalue < dpq[c]){
    dpq[p] = dpq[c];
    p = c;
    c = largechild(p,k-1);
}
dpq[p] = kvalue;
```

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

+91 9449867804

DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Sandesh B. J & Saritha

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

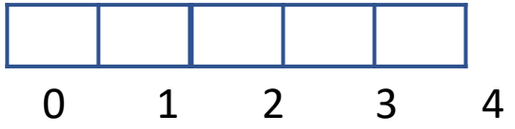
Graphs

Saritha

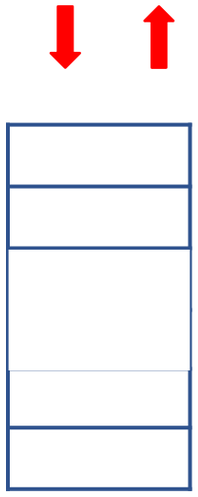
Department of Computer Science & Engineering

Linear data structures

Array



Stack



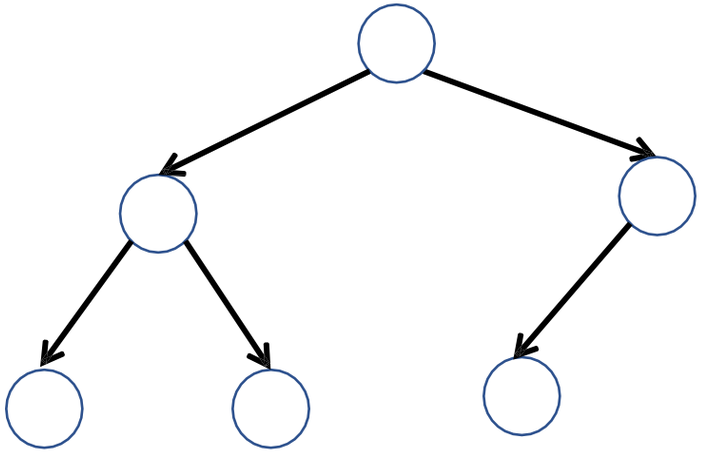
Queue

Linked List

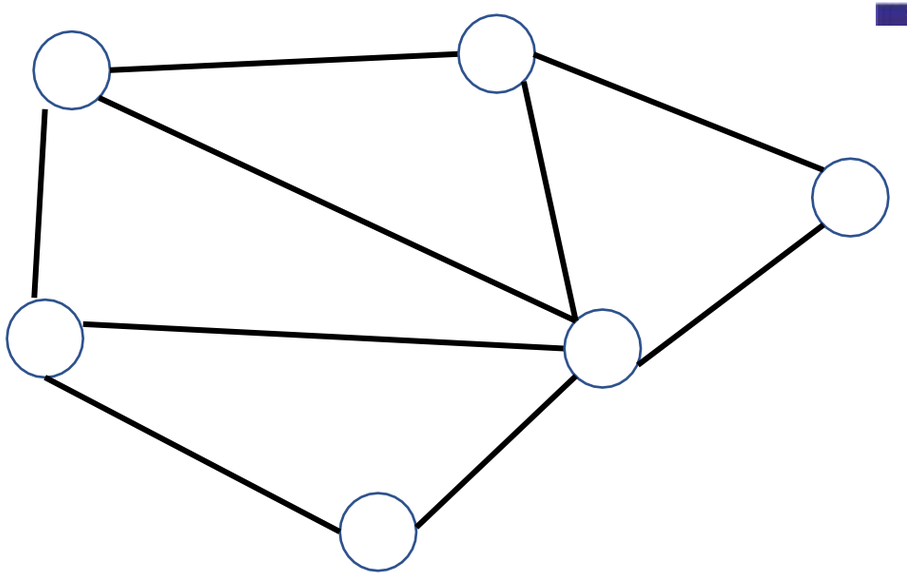
DATA STRUCTURES AND ITS APPLICATIONS

Introduction to Graphs

Non-Linear Data Structure



Tree



Graph

In a tree with N nodes there are N-1 edges

Introduction to graph

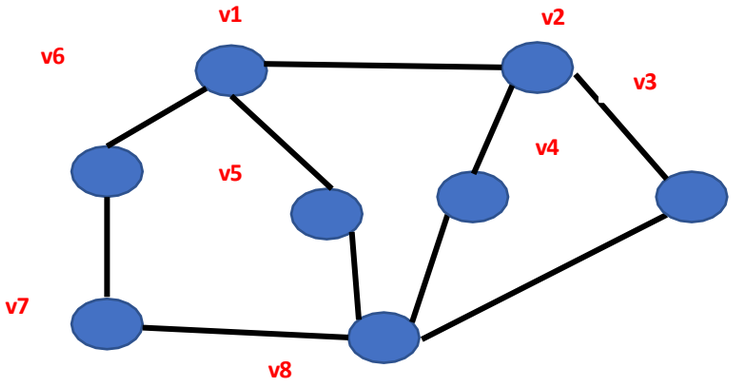
Graphs

- A Graph is a data structure that consists of set of vertices and a set of edges that relate the node to each other.
- The set of edges represents the relationship among the vertices.
- A graph G is defined as

G=(V,E)

V: finite nonempty set of vertices

E: a set of edges



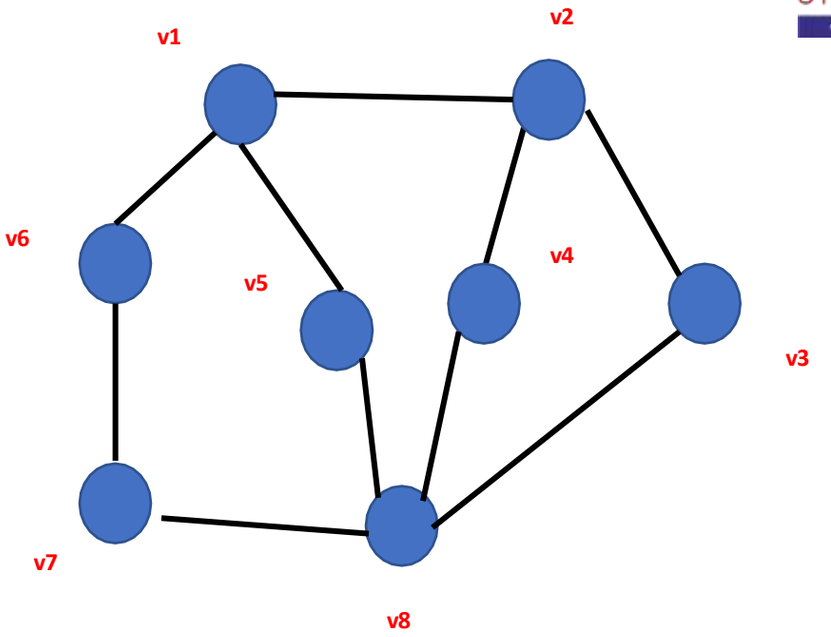
V = { v1,v2,v3,v4,v5,v6,v7,v8}

DATA STRUCTURES AND ITS APPLICATIONS

Representation of Edge

V = { v1,v2,v3,v4,v5,v6,v7,v8}

E={(v1,v2),(v2,v3),(v2,v4),(v4,v8),(v1,v5),(v1,v6),
(v6,v7),(v5,v8)}



Types of Graphs

Undirected Graph:

- A graph is undirected, when the pair of vertices representing any edge is unordered.

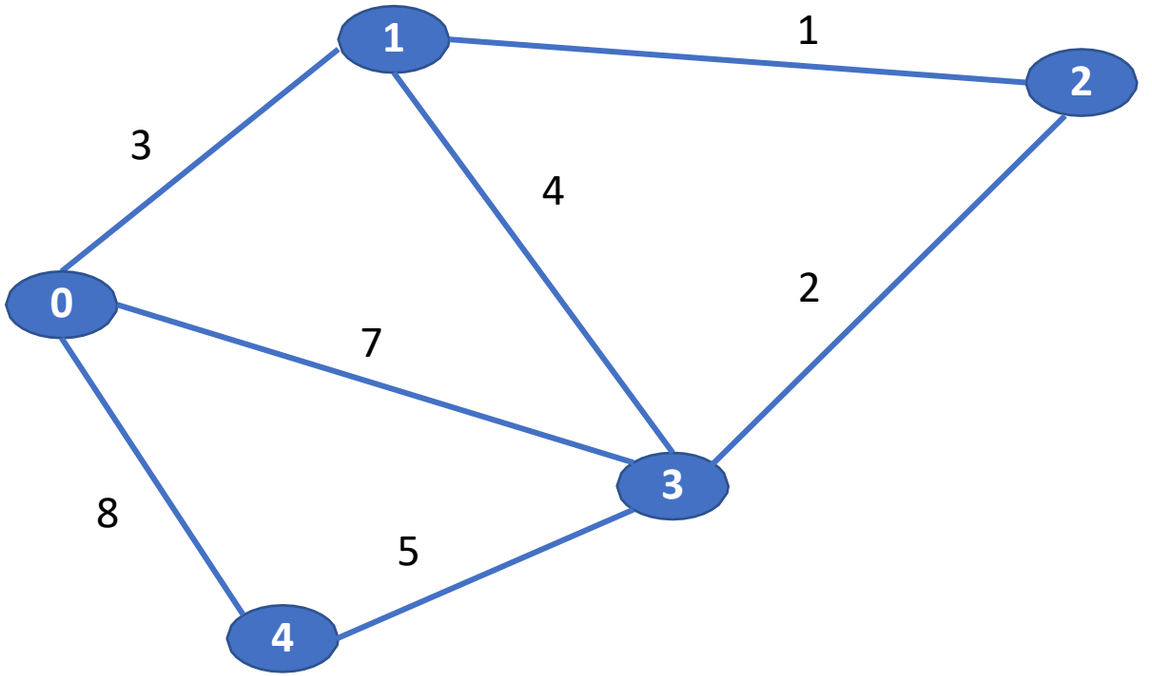
Directed Graph:

- A graph with all directed edges is called diagraph or directed graph.

Types of Graphs

Weighted Graph:

• A weighted graph is a graph where each edge has a numerical value called weight.



Graph terminologies

Adjacent Nodes :

- A node n is adjacent to node m if there is an edge from m to n.
- if n is adjacent to m, then n is called the **successor** of m and m is called the **predecessor** of n.

For example : a is adjacent to b

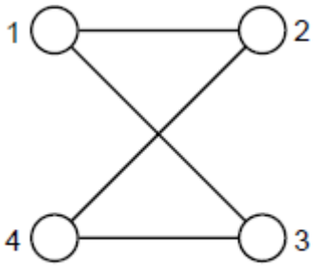
Path:

Path is a sequence of vertices that connect two nodes in a graph.

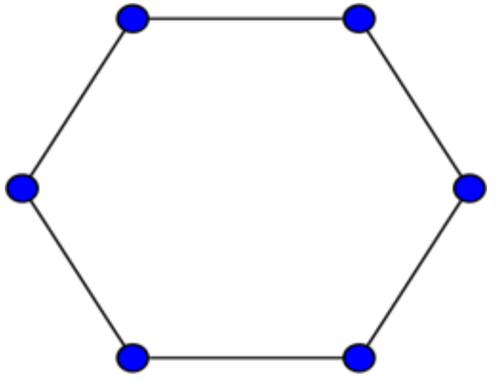
Graph terminologies

Cycle :

- A path from node to itself is called a cycle or cycle is path in which first and last vertices are same. A graph with at-least one cycle is called cyclic graph. For example the below graph are cyclic graphs



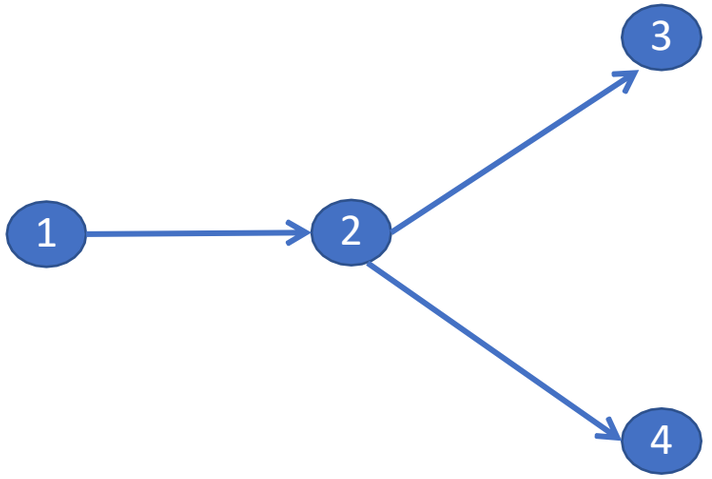
Cycle



Graph terminologies

Acylic :

- A graph with no cycles is called acyclic graph. A directed acyclic graph is called dag. For example below graph is a directed acylic graph



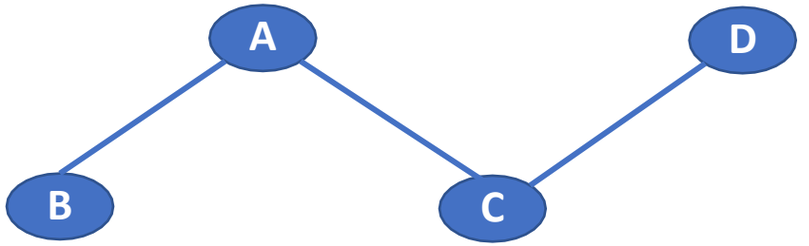
Graph terminologies

Incident:

A node n is incident to an edge x, if node is one of the two nodes the edge connects.

Degree:

The degree of vertex i is the number of edges incident on vertex i.



degree(A)=2,degree of(D)=1

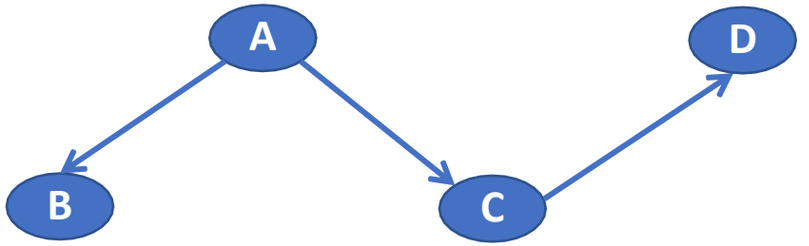
Graph terminologies

In-degree:

- In-degree of vertex i is the number of edges incident to i.

Out-degree:

- Out-degree of vertex i is the number of edges incident from i.



Out-degree(A)=2,in-degree of(A)=0

Out-degree(c)=1,in-degree of (c)=1

Properties of Graph

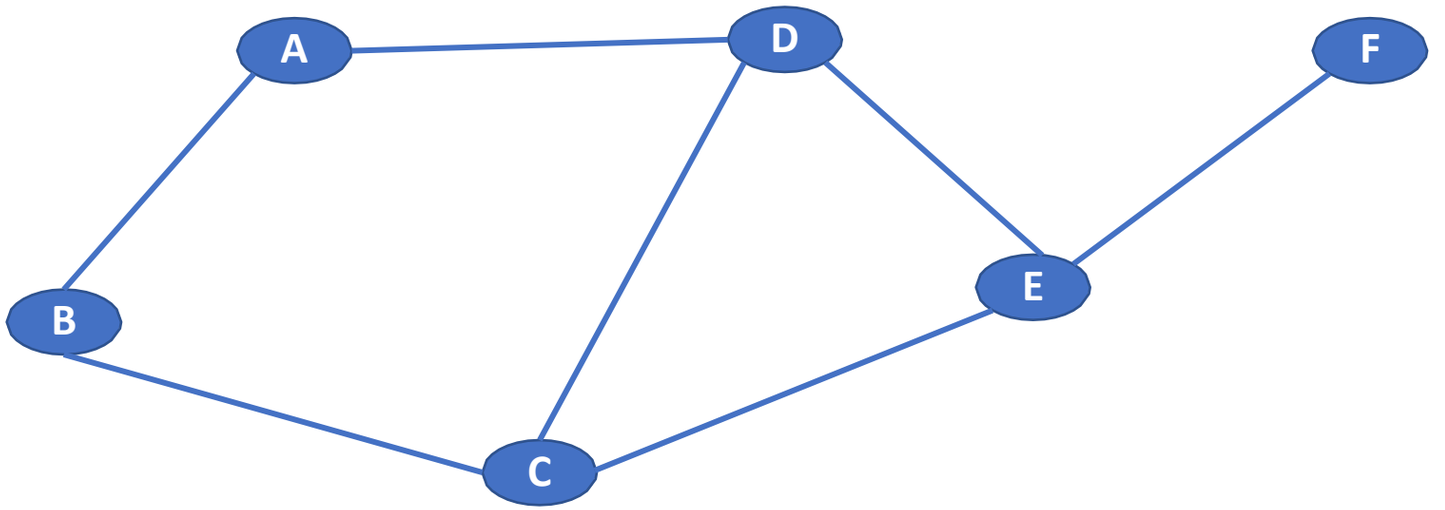
Directed graph:

- The number of possible pairs in an m vertex graph is m*(m-1)
- The number of edges in an directed graph is m*(m-1) since the edge(u, v) is not the same as the edge(v, u)
- The number of edges in an directed graph is<=m*(m-1)

Properties of Graph

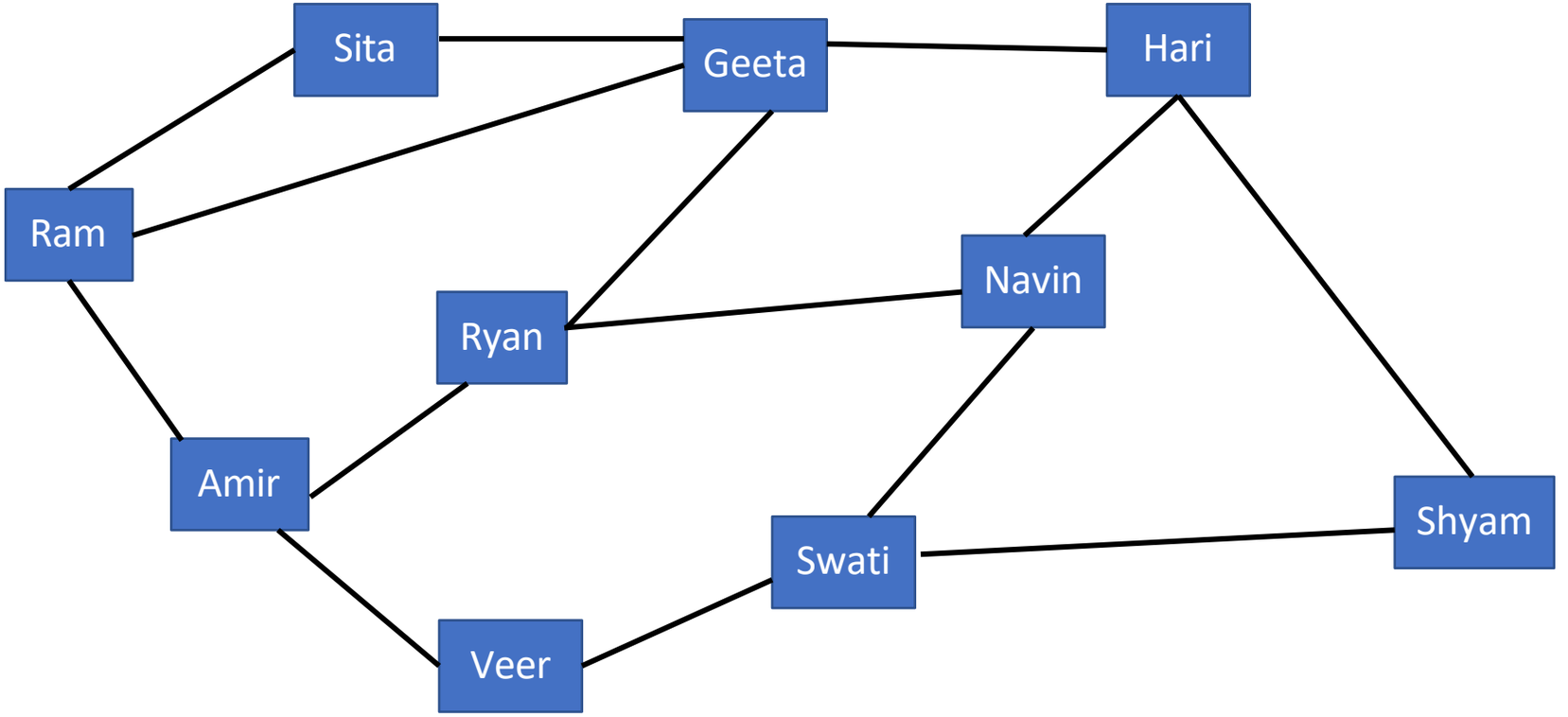
Undirected graph:

- The number of possible pairs in an m vertex graph is m*(m-1)
- The number of edges in an undirected graph is m*(m-1)/2 since the edge(u, v) is same as the edge(v, u)



Applications of graph

Social Networking sites



THANK YOU

Saritha

Department of Computer Science & Engineering

Saritha.k@pes.edu

9844668963

DATA STRUCTURES AND ITS APPLICATIONS

Graph Traversal Methods

Sandesh B. J
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Graph Traversals Methods

Sandesh B. J
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Graph Traversal Methods - Topics

- Why we need to traverse the graph?
- Traversing the graph using different techniques
  - ✓ Depth First Search and Breadth First Search Traversal
- Algorithms for Depth First and Breadth First Search Traversal

Graph Traversals

- **Graph traversal**(also known as graph search) refers to the process of visiting/investigating (checking or updating) each vertex of the graph in some systematic order -Wiki

- Traversal can start in any arbitrary vertex

- Traversals are classified by the order in which the vertices are visited

Methods to Traverse the Graph

- Depth First Search
- Breadth First Search

Depth first search (DFS)

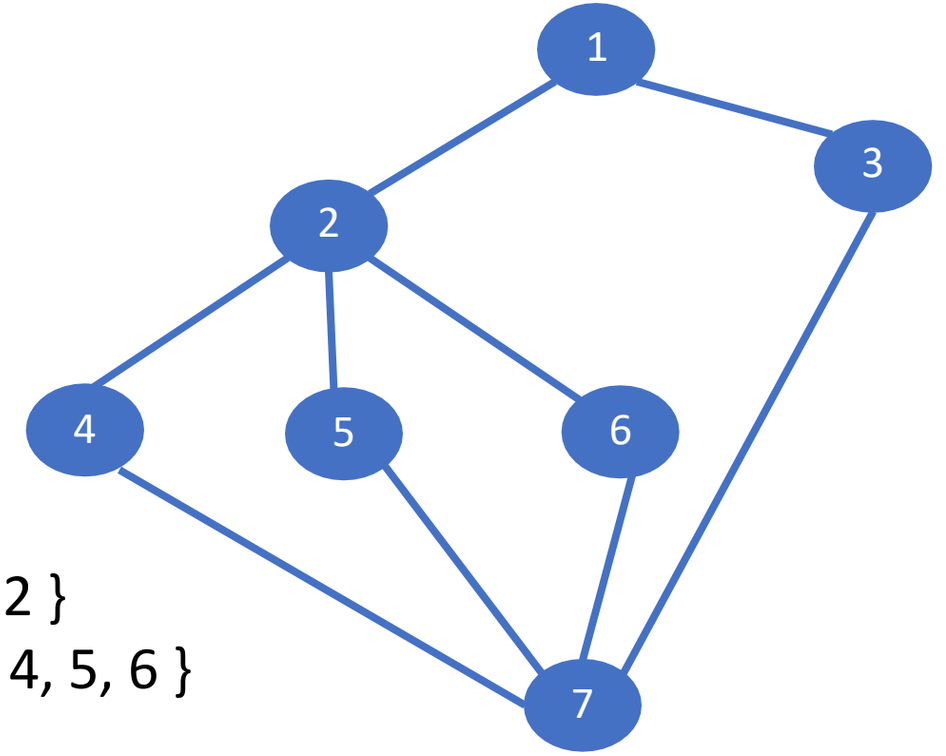
- Visits all the nodes related to one neighbour before visiting the other neighbours and its related nodes. Once it reaches dead end, it backtracks along previously visited node and continue traversing from there.

- Analogues to pre-order traversal of an ordered tree

- Uses stack behaviour, hence implemented using recursive algorithm

Depth First Search Traversal

- DFS traverse the depth of the graph, until it cannot go any further at which point it backtracks and continues

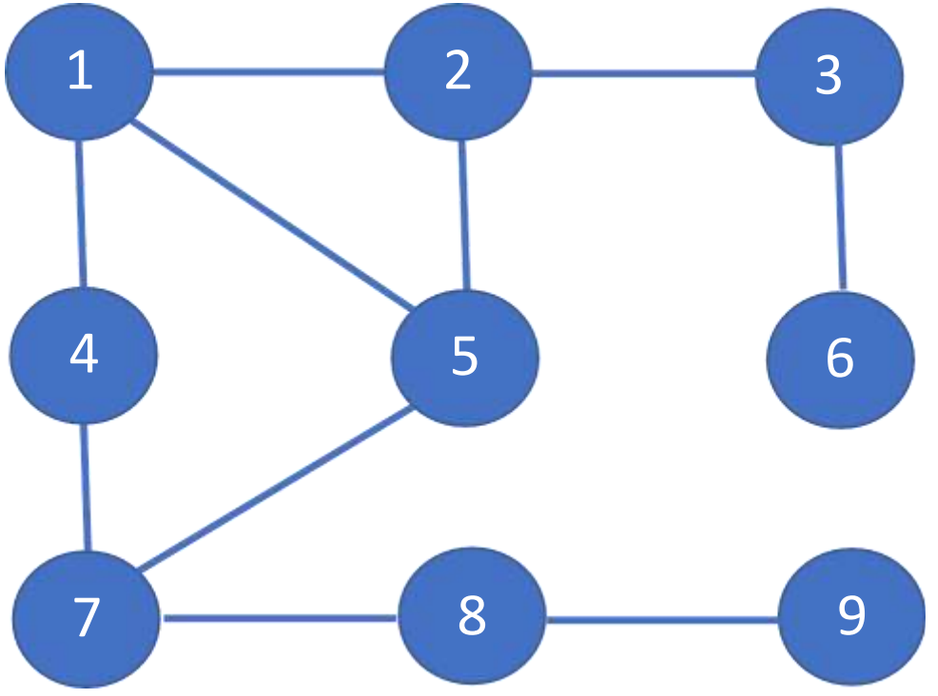


V = { 2 }
W= { 4, 5, 6 }

Traversal order : 1 , 2, 4 , 7, 3, 5, 6

DATA STRUCTURES AND ITS APPLICATIONS

Depth First Search Traversal

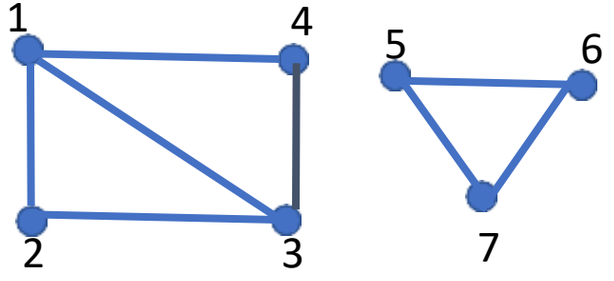


1 2 3 6 5 7 4 8 9

Difficulties in the graph traversals

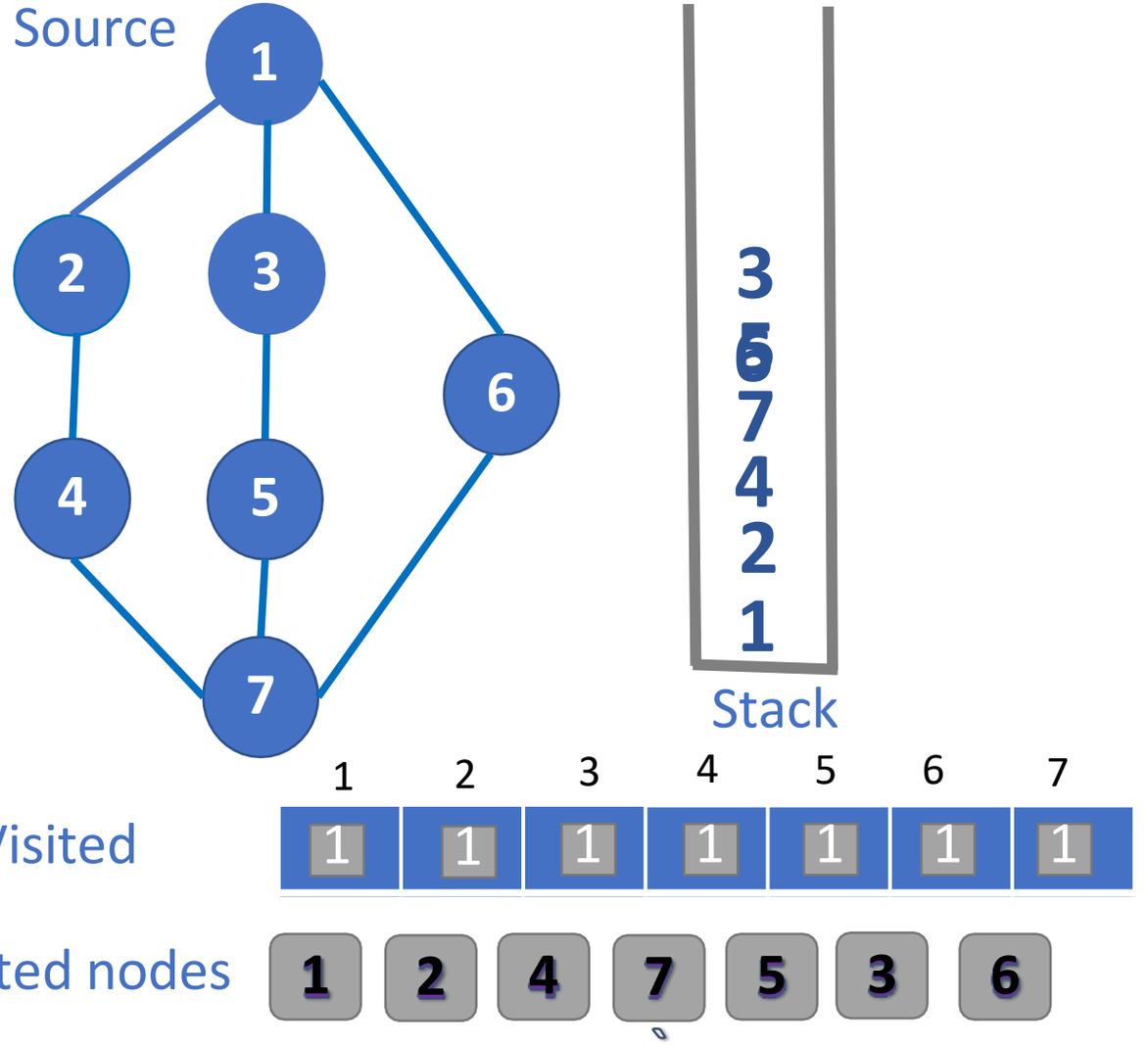
- **Graph may contain cycles**
  - Traversal algorithm may reach the same vertex second time.
  - To prevent the infinite recursion , we introduce Boolean valued array **visited**
  - Set the **visited[v] to true** once node v is visited
  - Check the **visited[w]** before processing any node w

- **Graph may not be connected:**
  - Traversal algorithm may fail to reach all the nodes from a single starting point



DATA STRUCTURES AND ITS APPLICATIONS

DFS Traversal – Using Stack



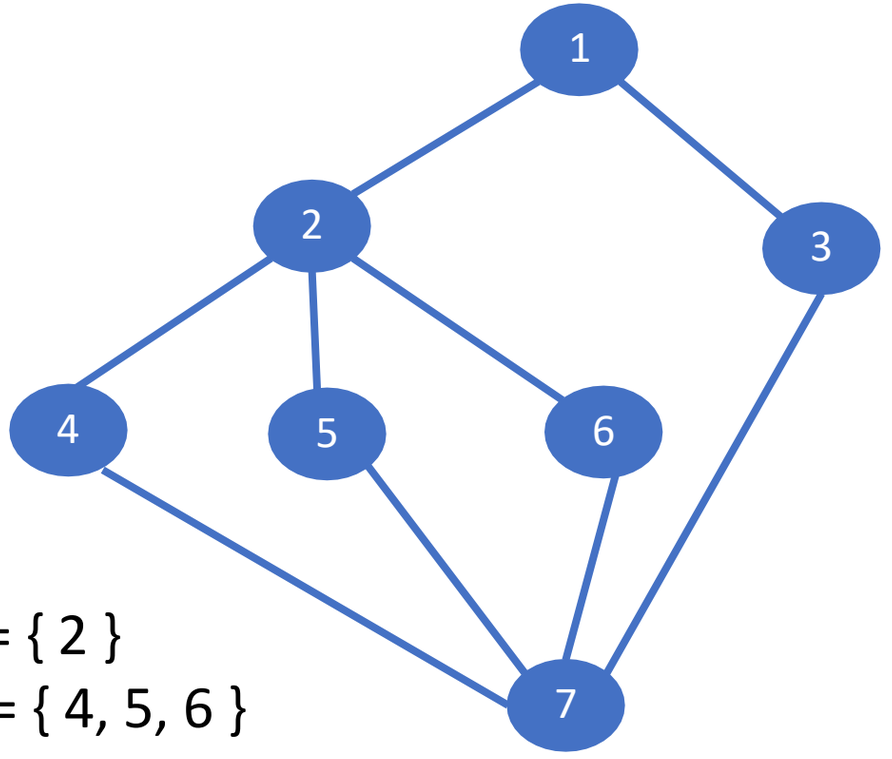
Breadth first search (BFS)

- Explores all the neighbour nodes in first level from an arbitrary node, before moving to next level of neighbouring nodes.

- Uses Queue behaviour

Breadth First Search Traversal

- Analogous to level-by-level traversal of ordered tree

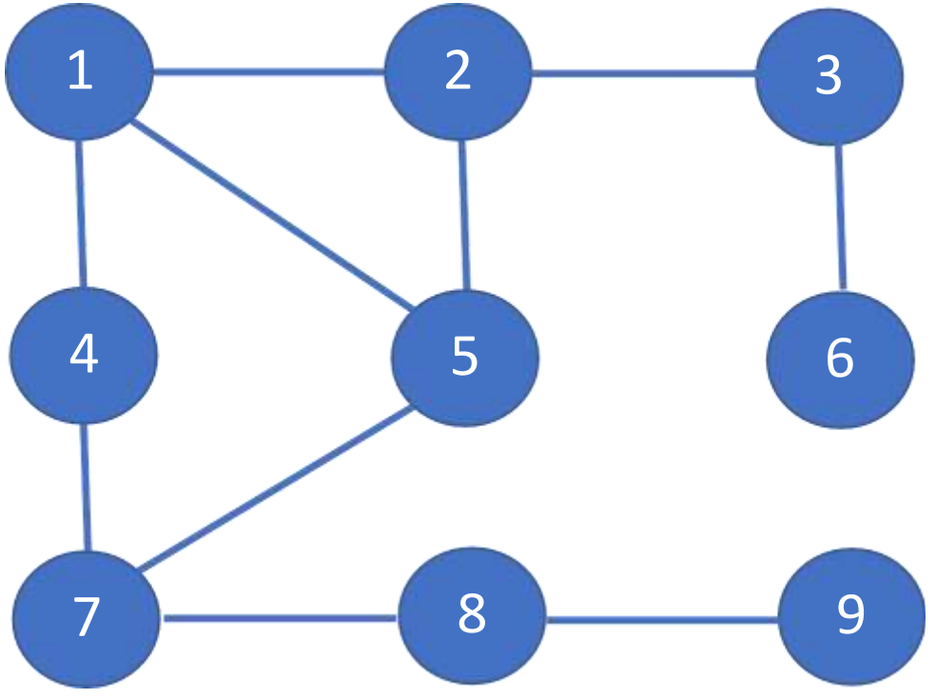


V = { 2 }
W= { 4, 5, 6 }

Traversal order : 1 , 2, 3, 4, 5, 6 , 7

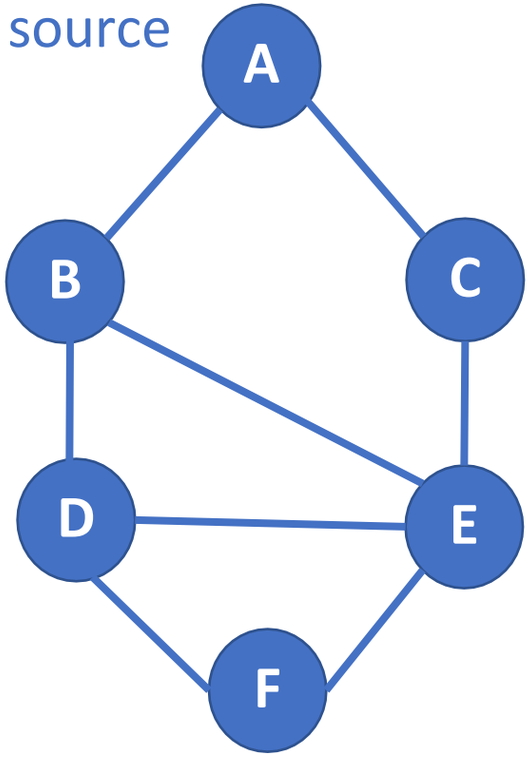
DATA STRUCTURES AND ITS APPLICATIONS

Breadth first search Traversal



1 2 4 5 3 7 6 8 9

BFS Traversal – Using Queue

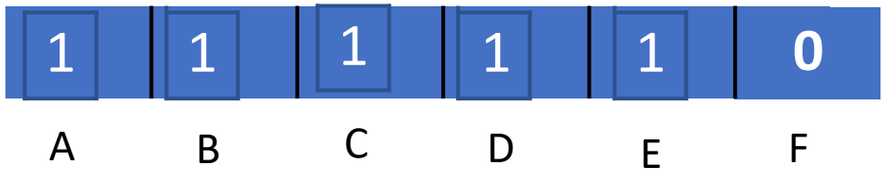


front

Queue

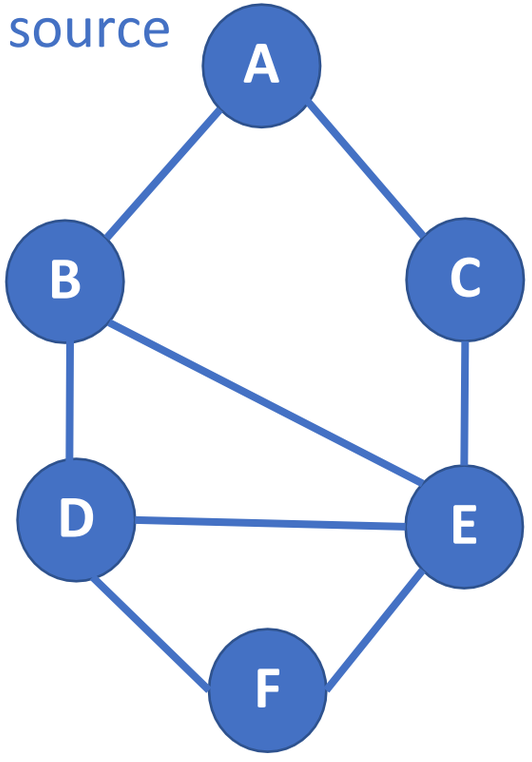
Remove_Q

Visited



Visited nodes A B C D E

BFS – Traversal – Using Queue



front

Queue

Remove_Q

Visited

A B C D E F

Visited nodes A B C D E F

DFS - Algorithm

```
Algorithm DFS (Graph G )
//Implements DFS traversal for given graph
// Input Graph G = (V, E)
//Output Graph G with vertices marked as visited
 mark each vertex in V with 0  as a mark of being “unvisited”
     for each vertex v in V do
          if v is marked with o
               dfs(v)
```

```
dfs(v)
// visits recursively all the unvisited  vertices connected to
    vertex v by a path
For each vertex w in V adjacent to v do
    if w is marked with o
        mark w as visited
       dfs(w)
```

Courtesy: “Introduction to Design and Analysis of Algorithms” By Anany Levitin

BFS - Algorithm

```
Algorithm BFS (Graph G )
//Implements BFS traversal for given graph
// Input Graph G = (V, E)
//Output Graph G with vertices marked as visited
 mark each vertex in V with 0  as a mark of being “unvisited”
     for each vertex v in V do
          if v is marked with 0
               bfs(v)
```

Courtesy: “Introduction to Design and Analysis of Algorithms” By Anany Levitin

BFS - Algorithm

```
bfs(v)
// visits recursively all the unvisited  vertices connected to
    vertex v by a path
While the queue is not empty
   for each vertex w in V adjacent to front vertex do
      if w is marked with 0
         mark w as visited
         add w to queue
         remove the front vertex from the queue
```

Courtesy: “Introduction to Design and Analysis of Algorithms” By Anany Levitin

THANK YOU

Sandesh B. J
Department of Computer Science & Engineering

sandesh_bj@pes.edu

+91 80 6618 6623

DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Saritha

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Representation of Network Topology

Saritha

Department of Computer Science & Engineering

Applications: Network Topology

Graph data structure is mainly in Computer Networks, Telecommunication, Electronic Circuits and Transport Networks.

Networking uses the Notation G(N,L) instead of G(V,E) for a graph where N is the set of nodes and L is the set of links.

Applications: Network Topology

- Topology is the order in which nodes and edges are arranged in the network.
- How the computers are connected or related to one another in a computer.
- There are 2 types of Topology
    1. Physical
    2. Logical

Applications: Network Topology

- 1.Ring Topology
- 2.Star Topology
- 3.Mesh Topology
- 4.BusTopology

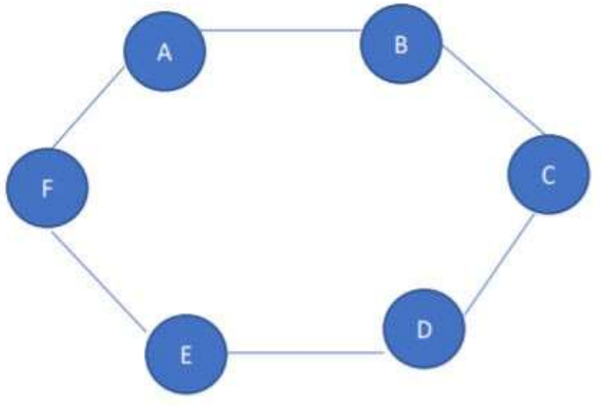
Representation of Graph

1.Adjacency Matrix

2.Adjacency List

Applications: Network Topology

1. Ring topology (cycle): A cycle graph is a simple graph which has two degrees of vertices.

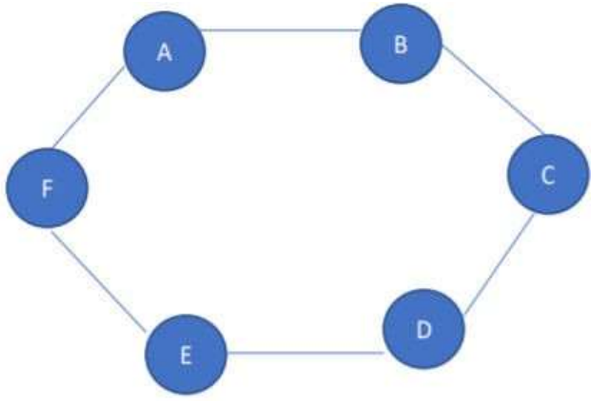


Ring topology

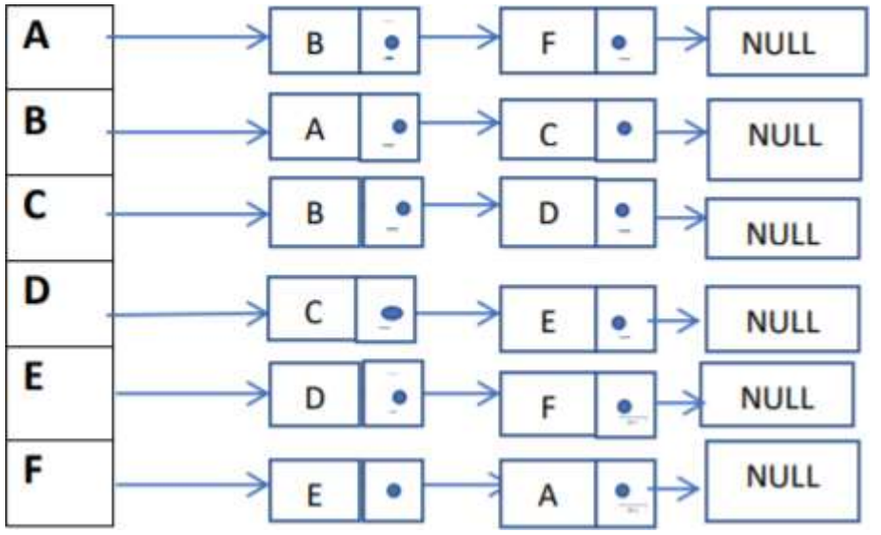
	A	B	C	D	E	F
A	0	1	0	0	0	1
B	1	0	1	0	0	0
C	0	1	0	1	0	0
D	0	0	1	0	1	0
E	0	0	0	1	0	1
F	1	0	0	0	1	0

DATA STRUCTURES AND ITS APPLICATIONS

Applications: Network Topology

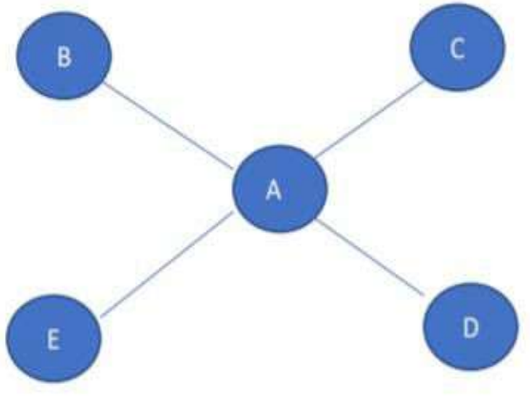


Ring topology



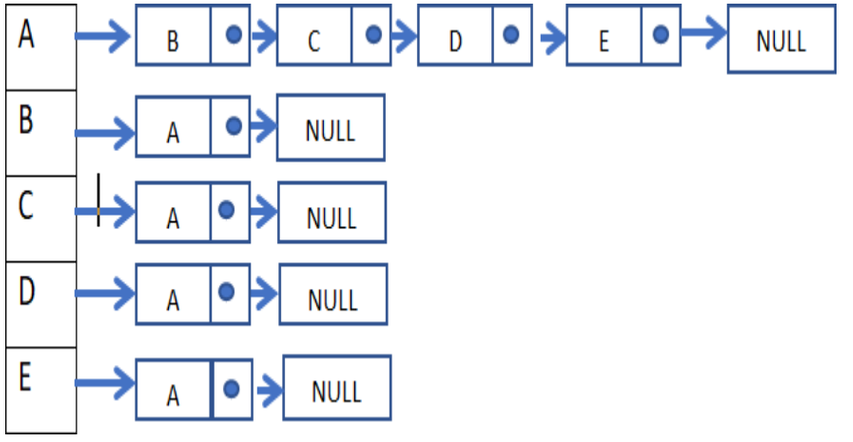
Applications: Network Topology

2.Star topology: Star topology is a network topology in the form of merging from the central vertex to each vertex .



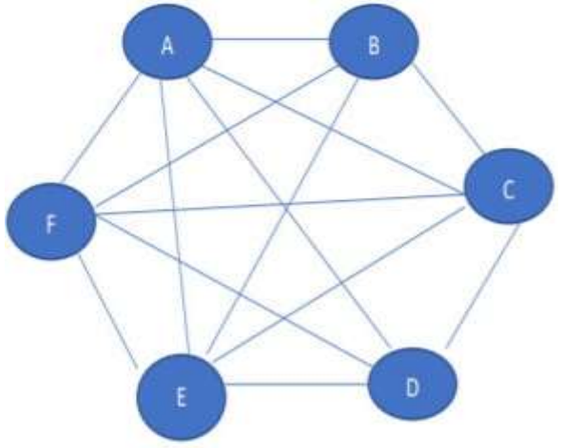
Star topology

	A	B	C	D	E
A	0	1	1	1	1
B	1	0	0	0	0
C	1	0	0	0	0
D	1	0	0	0	0
E	1	0	0	0	0



Applications: Network Topology

3.**Mesh topology**: Mesh Topology is a complete graph in which all the vertex is connected to all other vertices

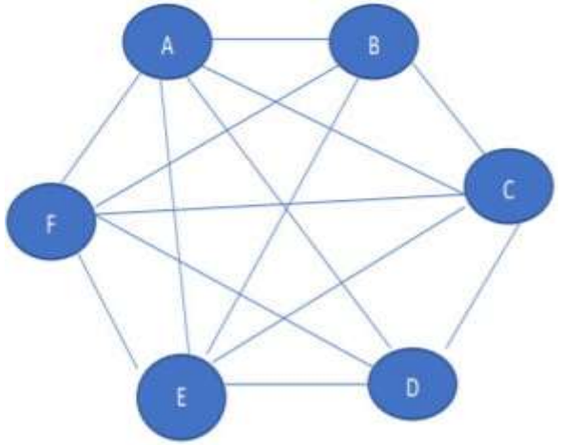


Mesh Topology

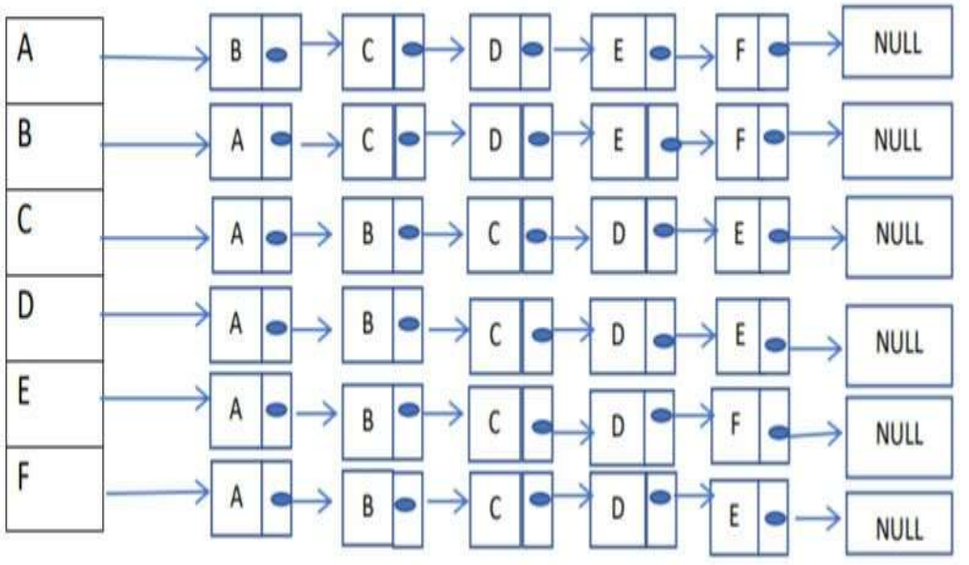
	A	B	C	D	E	F
A	0	1	1	1	1	1
B	1	0	1	1	1	1
C	1	1	0	1	1	1
D	1	1	1	0	1	1
E	1	1	1	1	0	1
F	1	1	1	1	1	0

DATA STRUCTURES AND ITS APPLICATIONS

Applications: Network Topology



Mesh Topology

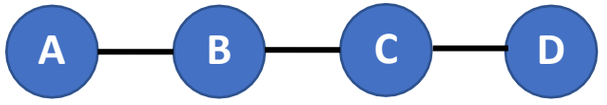


Bus Topology

A Graph G with V vertices is said to represent a bus topology if

1.Every node except the starting and ending node has degree 2 and starting and ending node have degree 1.

2. Number of edges=Number of vertices -1

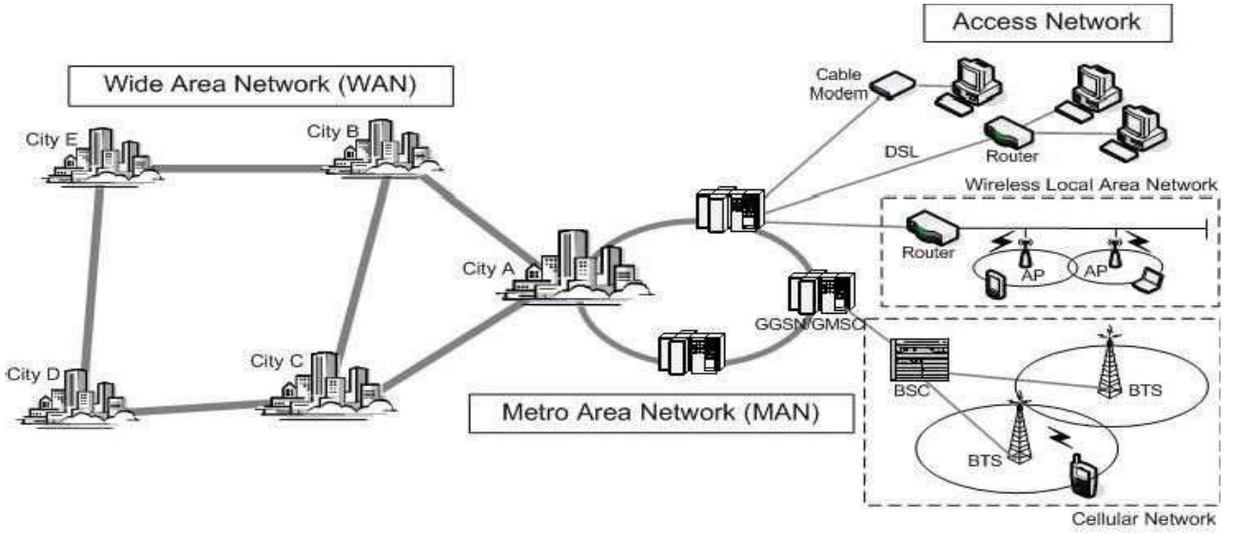


	A	B	C	D
A	0	1	0	0
B	1	0	1	0
C	0	1	0	1
D	0	0	1	0

DATA STRUCTURES AND ITS APPLICATIONS

Applications: Network Topology

Most networks a mix of rings, mesh – depending on network type, cost/traffic/reliability



Picture Courtesy: ComputerNetworkTopology.com

THANK YOU

Saritha

Department of Computer Science & Engineering

Saritha.k@pes.edu

9844668963

DATA STRUCTURES AND ITS APPLICATIONS UE22CS252A

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Programs on Binary Trees

Shylaja S S
Department of Computer Science & Engineering

Programs on Binary Tree

```
//Returns the smallest element in Binary Search Tree
 int minimum(struct tnode *t)
 {
   while(t->left!=NULL)
    t=t->left;
   return(t->data);
 }
```

Programs on Binary Tree

```
//Returns the largest element in Binary Search Tree
 int maximum(struct tnode *t)
 {
   while(t->right!=NULL)
    t=t->right;
   return(t->data);
 }
```

Programs on Binary Tree

```
//Computes the height of a Binary Tree
int height(struct tnode *t)
 {
  if(t==NULL)
    return -1;
  if((t->left==NULL)&&(t->right==NULL))
    return 0;
  return (1+max(height(r->left),height(r->right)));
 }
```

Programs on Binary Tree

```
//Count the number of leaf nodes in a Binary Tree
int leafcount(struct tnode *t)
{
   if(t==NULL)
    return 0;
  if((t->left==NULL)&&(t->right==NULL))
    return 1;
  int l=leafcount(t->left);
  int r=leafcount(t->right);
  return(l+r);
}
```

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE22CS252A

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversal

Shylaja S S
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversals

Important operation: Traversal

Traversal: Moving through all the nodes in a binary tree and visiting each one in turn

Trees: There are many orders possible since it is a nonlinear DS

Tasks: 1. Visiting a node denoted by V

2. Traversing the left subtree denoted by L

3. Traversing the right subtree denoted by R

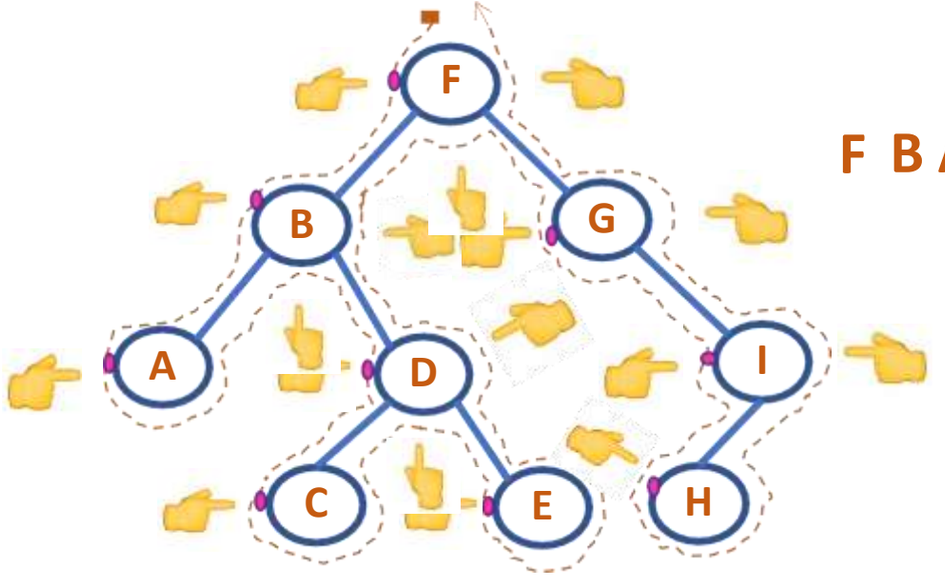
Six ways to arrange them: VLR, LVR, LRV, VRL, RVL, RLV

Standard Traversals include: VLR-Preorder, LVR-Inorder, LRV-Postorder

Binary Tree Traversal: Preorder

Steps:

- Root Node is visited before the subtrees
- Left subtree is traversed in preorder
- Right subtree is traversed in preorder

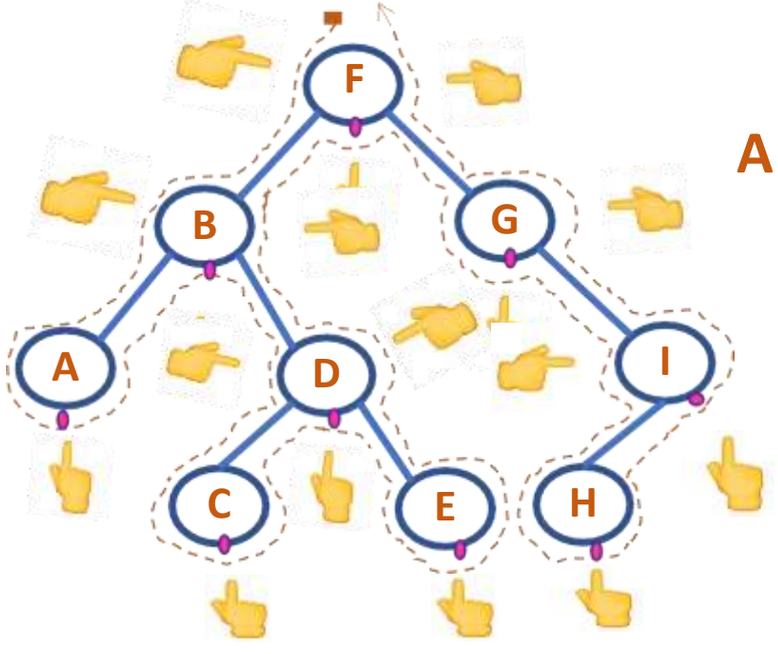


F B A D C E G I H

Binary Tree Traversal: Inorder

Steps:

- Left subtree is traversed in Inorder
- Root Node is visited
- Right subtree is traversed in Inorder

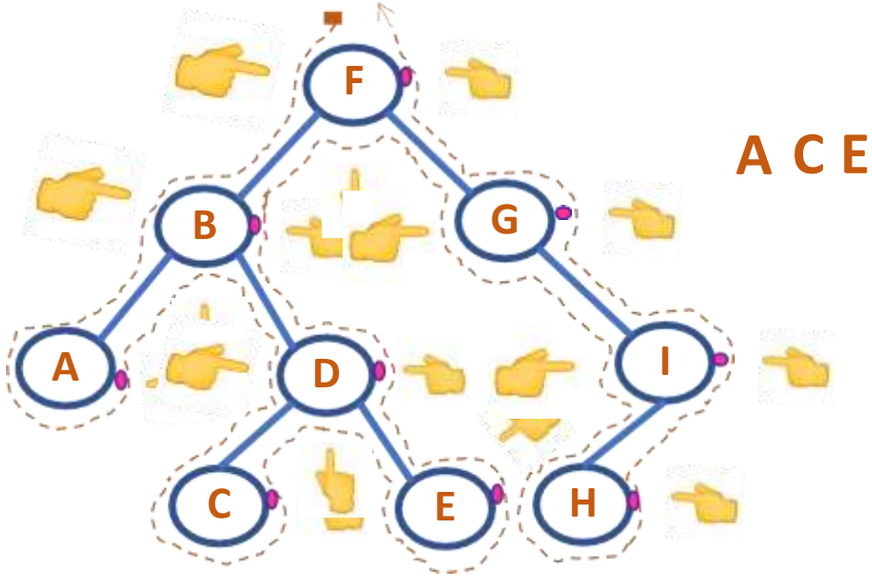


A B C D E F G H I

Binary Tree Traversal: Postorder

Steps:

- Left subtree is traversed in postorder
- Right subtree is traversed in postorder
- Root Node is visited



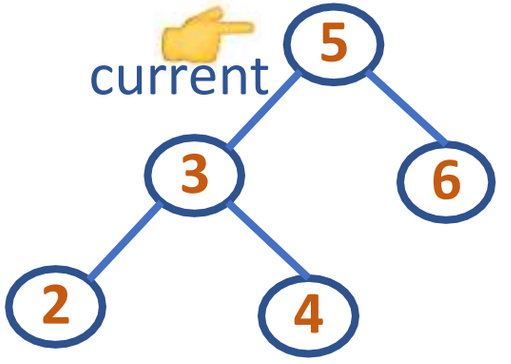
A C E D B H I G F

Iterative Inorder Traversal

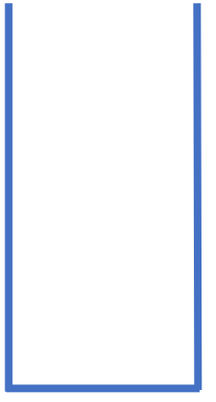
```
iterativeInorder(root)
s = emptyStack
current = root
do {
        while(current != null)
        {       /* Travel down left branches as far as possible
                saving pointers to nodes passed in the stack*/
                push(s, current)
                current = current->left
        } //At this point, the left subtree is empty
        poppedNode = pop(s)
        print poppedNode ->info         //visit the node
        current = poppedNode ->right //traverse right subtree
} while(!isEmpty(s) or current != null)
```

Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
      while(current != null)
     {
          push(s, current)
          current = current->left
      }
      poppedNode = pop(s)
      print poppedNode ->info
      current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



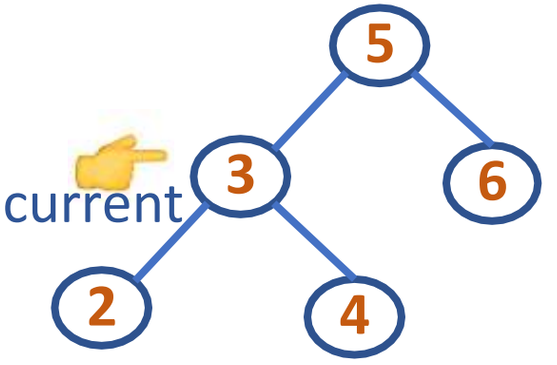
current = **5**



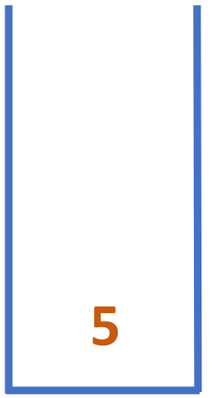
Note: Stack has Address of Nodes Pushed In

Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
       while(current != null)
      {
           push(s, current)
           current = current->left
       }
       poppedNode = pop(s)
       print poppedNode ->info
       current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```

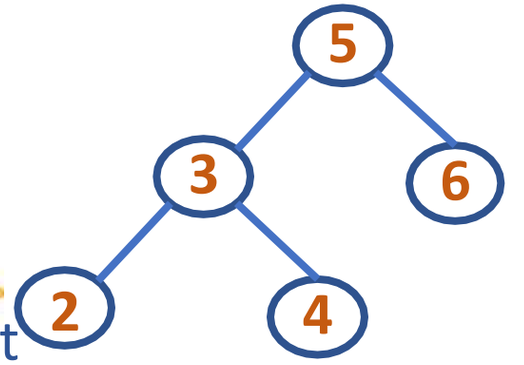


current = **3**

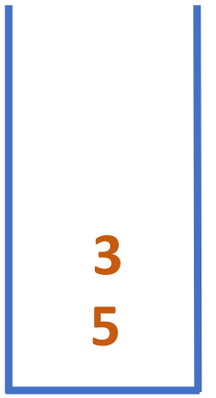


Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
       while(current != null)
      {
           push(s, current)
           current = current->left
       }
       poppedNode = pop(s)
       print poppedNode ->info
       current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```

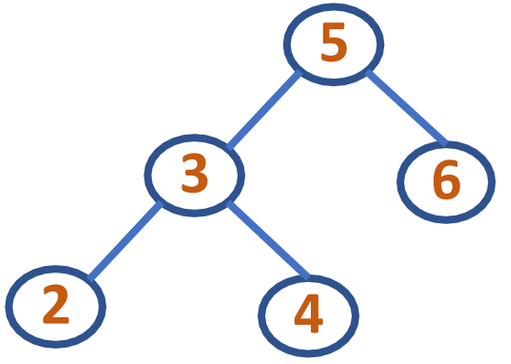


current = **2**

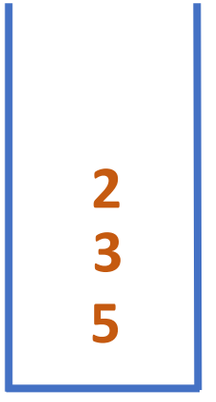


Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
       while(current != null)
      {
           push(s, current)
           current = current->left
       }
       poppedNode = pop(s)
       print poppedNode ->info
       current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```

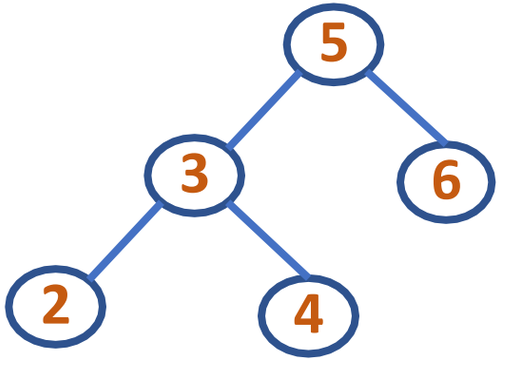


current = **N**



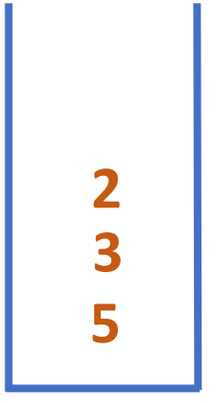
Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
       while(current != null)
     {
          push(s, current)
          current = current->left
       }
       poppedNode = pop(s)
       print poppedNode ->into
       current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



current = **N**

poppedNode =

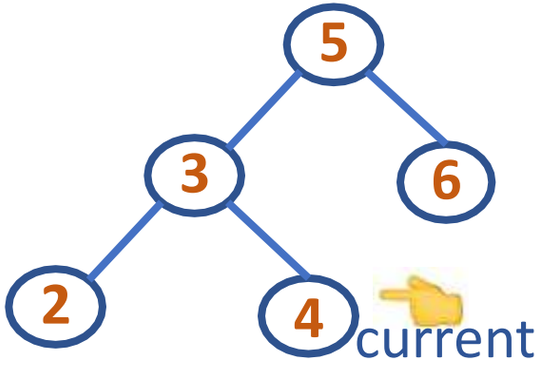


Inorder Traversal:

2

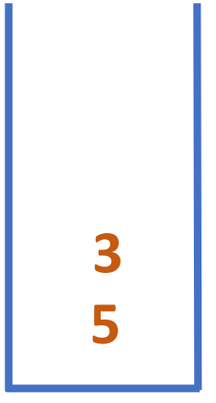
Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
      while(current != null)
     {
          push(s, current)
          current = current->left
      }
      poppedNode = pop(s)
      print poppedNode ->info
      current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



current = Null

poppedNode = 2

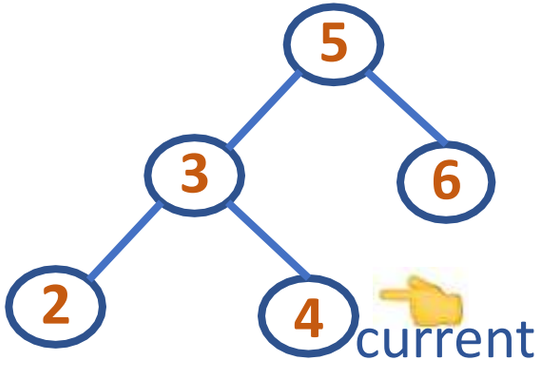


Inorder Traversal:

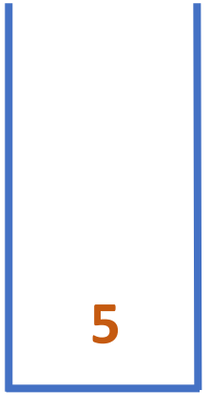
2 3

Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
      while(current != null)
     {
          push(s, current)
          current = current->left
      }
      poppedNode = pop(s)
      print poppedNode ->info
      current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



current = **4**

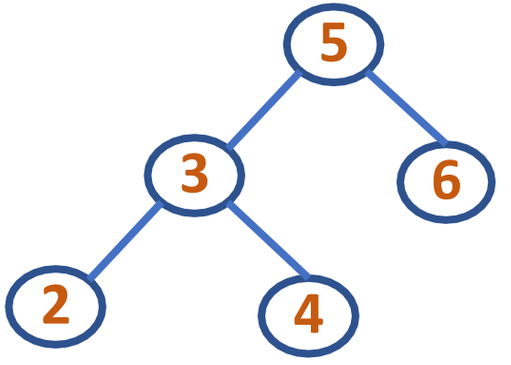


Inorder Traversal:

2 3

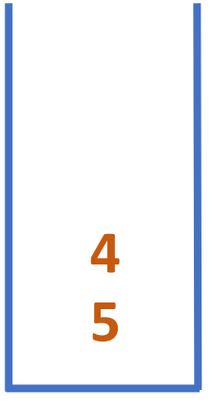
Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
       while(current != null)
      {
           push(s, current)
           current = current->left
       }
       poppedNode = pop(s)
       print poppedNode ->info
       current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



current = **N**

poppedNode = **3**

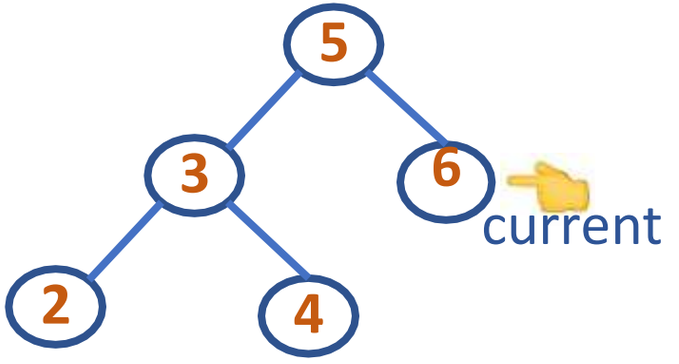


Inorder Traversal:

2 3 4

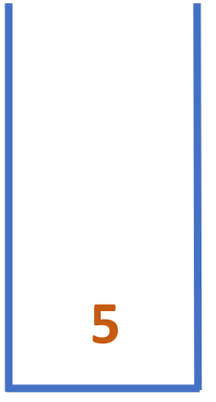
Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
      while(current != null)
     {
          push(s, current)
          current = current->left
      }
      poppedNode = pop(s)
      print poppedNode ->info
      current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



current = NULL

poppedNode = 4

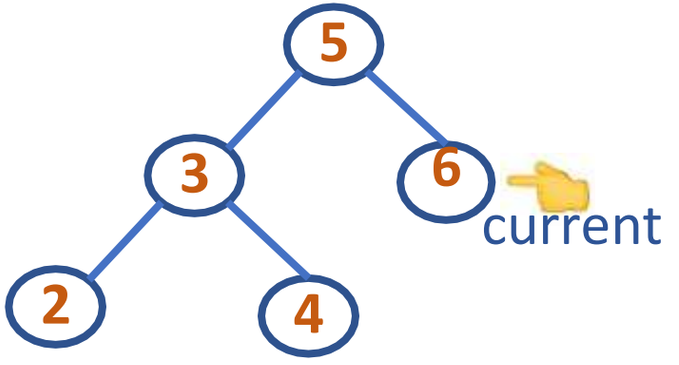


Inorder Traversal:

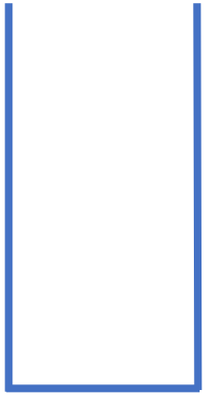
2 3 4 5

Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
      while(current != null)
     {
          push(s, current)
          current = current->left
      }
      poppedNode = pop(s)
      print poppedNode ->info
      current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



current = **6**

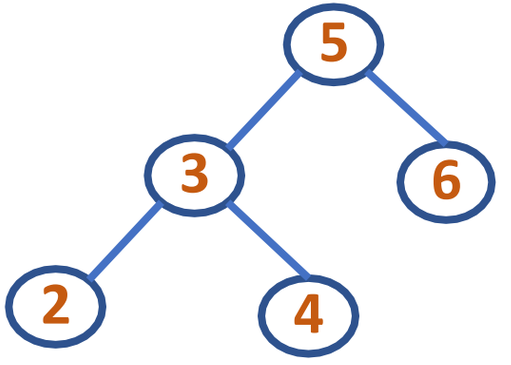


Inorder Traversal:

2 3 4 5

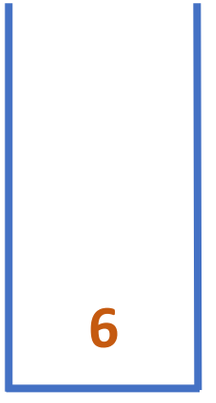
Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
       while(current != null)
      {
           push(s, current)
           current = current->left
       }
       poppedNode = pop(s)
       print poppedNode ->into
       current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



current = **N**

poppedNode =



Inorder Traversal:

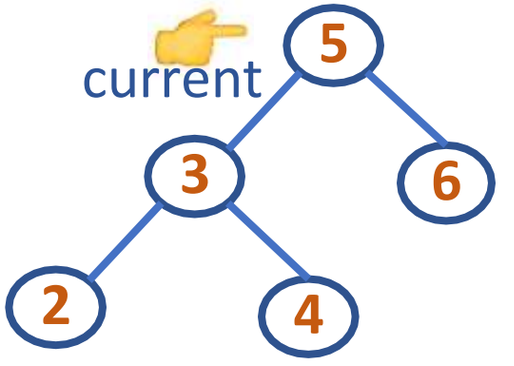
2 3 4 5 6

Iterative Preorder Traversal

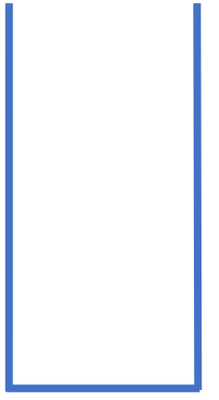
```
iterativePreorder(root)
current=root
if (current == null)
    return
s = emptyStack
push(s, current)
while(!isEmpty(s)) {
      current = pop(s)
      print current->info
      //right child is pushed first so that left is processed first
      if(current->right !=NULL)
            push(s, current->right)
      if(current->left !=NULL)
            push(s, current->left)
}
```

Iterative Preorder Traversal

```
iterativePreorder(root)
current=root
if (current == null)
    return
s = emptyStack
push(s, current)
    ...
```



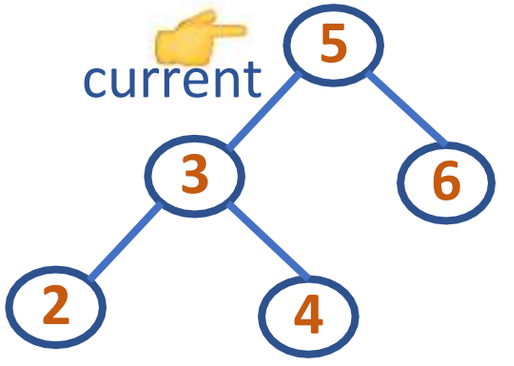
current = **5**



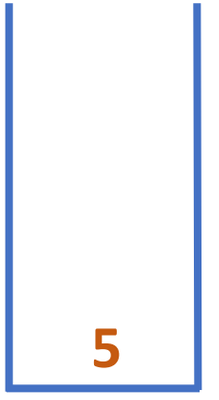
Note: Stack has Address of Nodes Pushed In

Iterative Preorder Traversal

```
iterativePreorder(root)
      ::::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```



current = 5

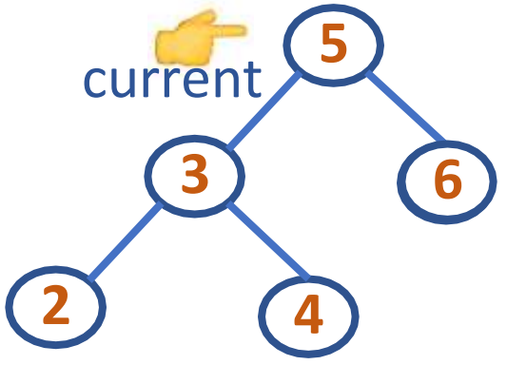


Preorder Traversal:

5

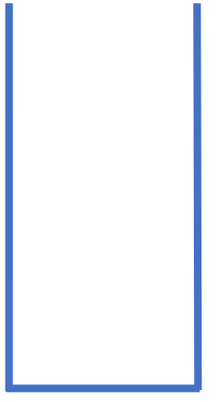
Iterative Preorder Traversal

```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```



current = 5

current->right = 6

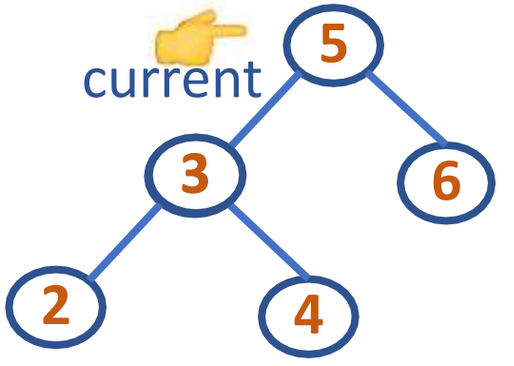


Preorder Traversal:

5

Iterative Preorder Traversal

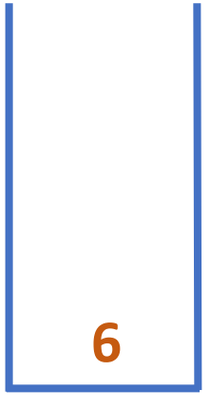
```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```



Preorder Traversal:
5

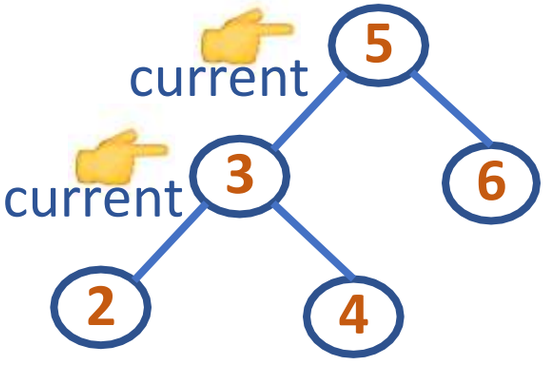
current = 5

current->left = 3

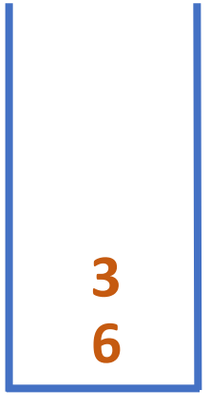


Iterative Preorder Traversal

```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```



current = 3

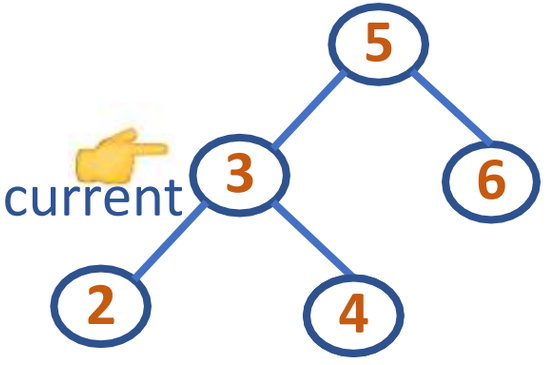


Preorder Traversal:

5 3

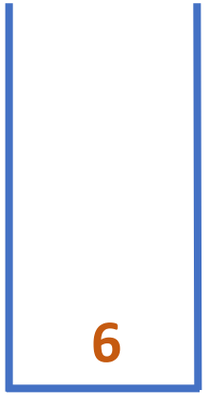
Iterative Preorder Traversal

```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```



current = 3

current->right = 4

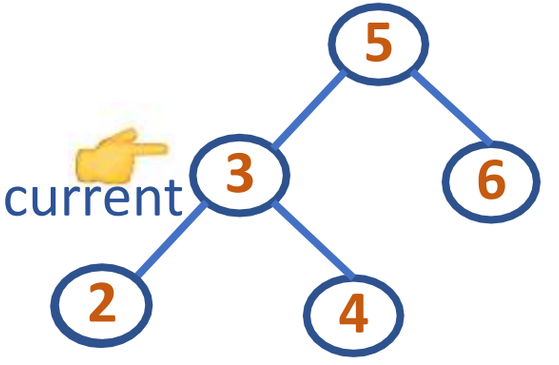


Preorder Traversal:

5 3

Iterative Preorder Traversal

```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```



current = 3

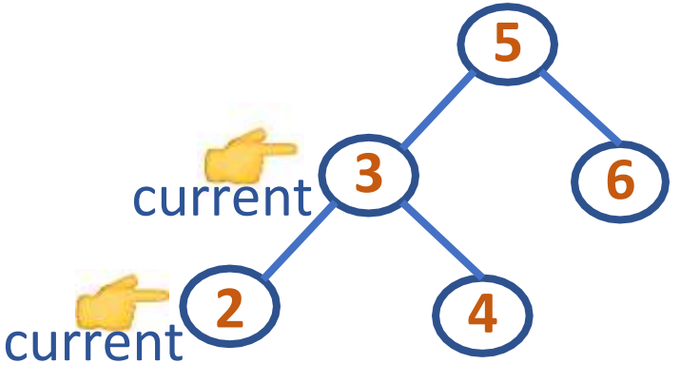
current->left = 2

Preorder Traversal:

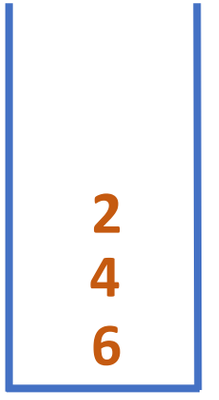
5 3

Iterative Preorder Traversal

```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```



current = **3**

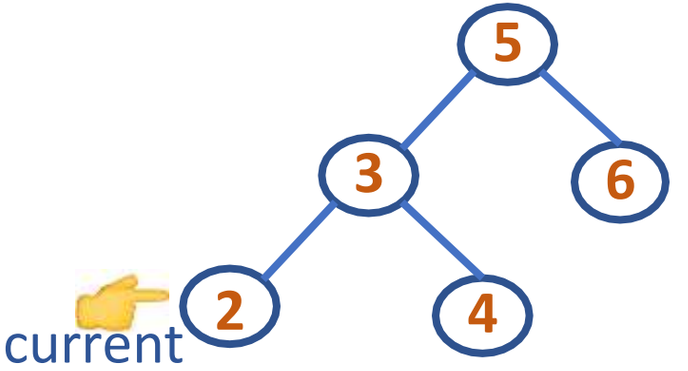


Preorder Traversal:

5 3 2

Iterative Preorder Traversal

```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```



current = **2**

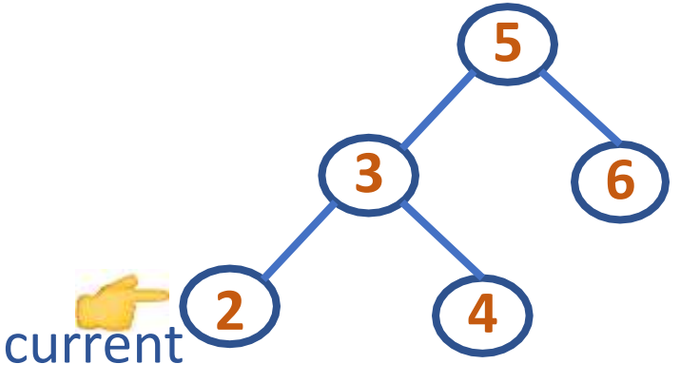
current->right = **N**

Preorder Traversal:

5 3 2

Iterative Preorder Traversal

```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```



current = **2**

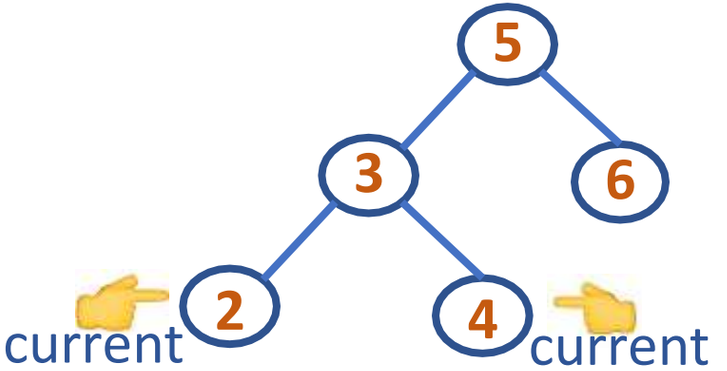
current->left = **N**

Preorder Traversal:

5 3 2

Iterative Preorder Traversal

```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```



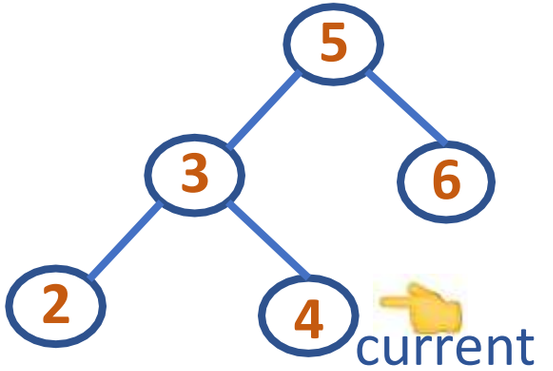
current = **2**

Preorder Traversal:

5 3 2 4

Iterative Preorder Traversal

```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```

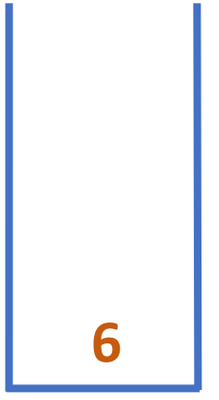


Preorder Traversal:
5 3 2 4

current = 4

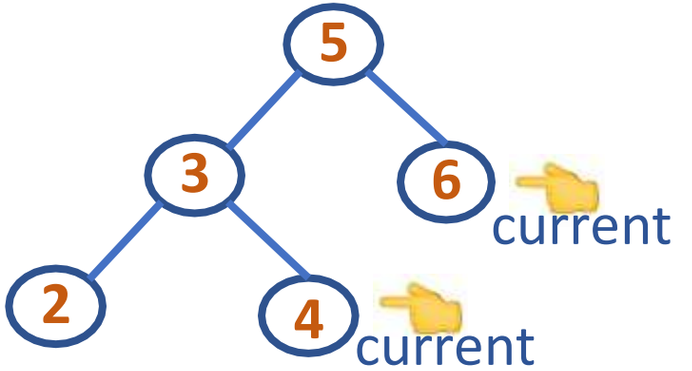
current->right = N

current->left = N



Iterative Preorder Traversal

```
iterativePreorder(root)
      :::
while (!isEmpty(s))
{
   current = pop(s)
   print current ->info
   if(current->right != null)
      push(s, current->right)
   if(current->left != null)
      push(s, current->left)
}
```

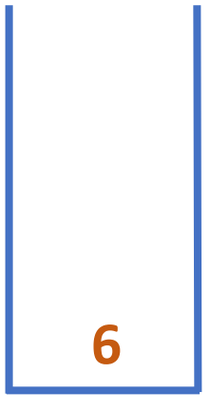


Preorder Traversal:
5 3 2 4 6

current = **4**

current->right = **N**

current->left = **N**

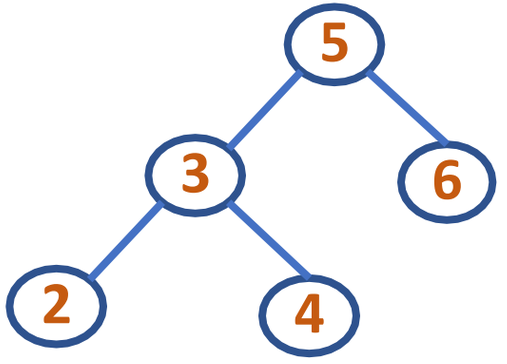


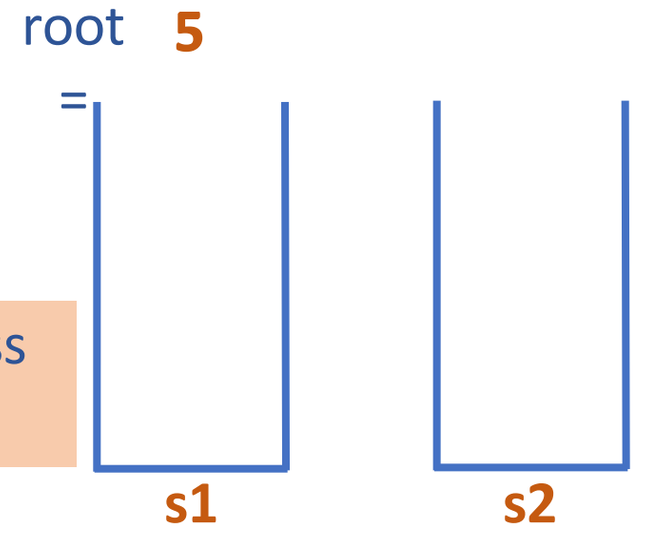
Iterative Postorder Traversal

```
iterativePostorder(root)
s1 = emptyStack ; s2 = emptyStack ; push(s1, root)
while(!isEmpty(s1)) {
      current = pop(s1)
      push(s2,current)
      if(current->left !=NULL)
            push(s1, current->left)
      if(current->right !=NULL)
            push(s1, current->right)
}
while(!isEmpty(s2)) {   //Print all the elements of stack2
      current = pop(s2)
      print current->info
}
```

Iterative Postorder Traversal

```
iterativePostorder(root)
s1 = emptyStack
s2 = emptyStack
push(s1, root)
    :::
```

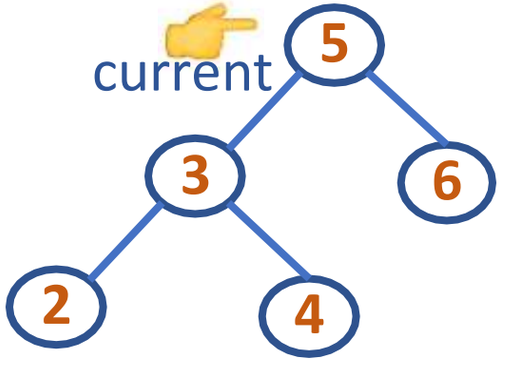




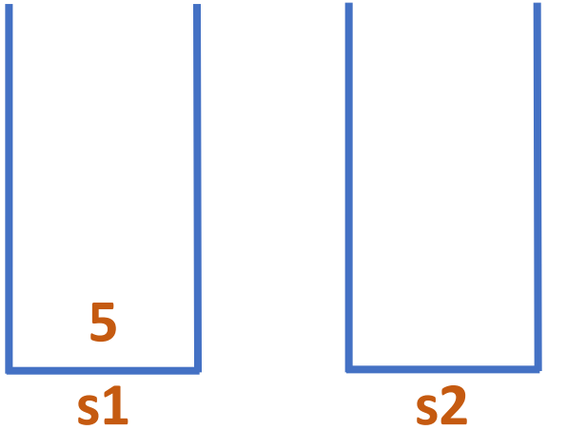
Note: Stacks have Address of Nodes Pushed In

Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```

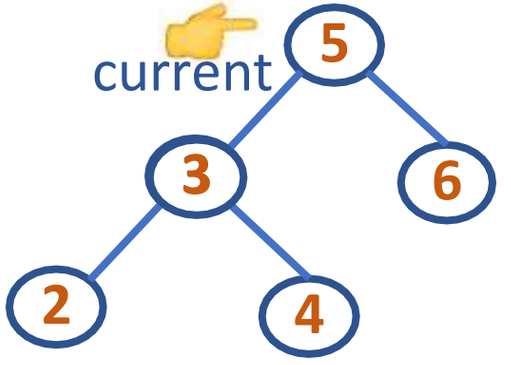


current =



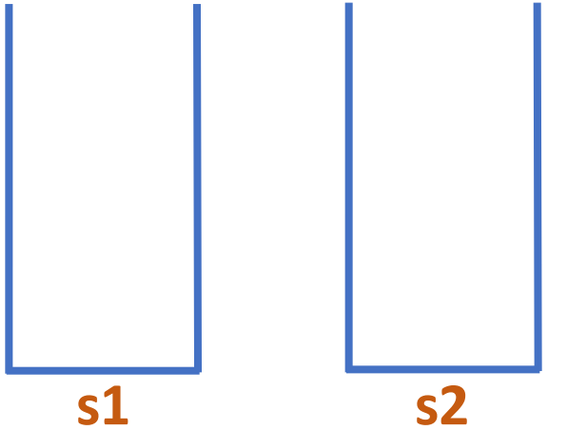
Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```



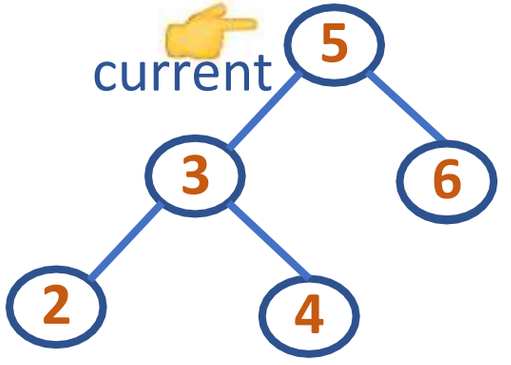
current = 5

current->left = 3



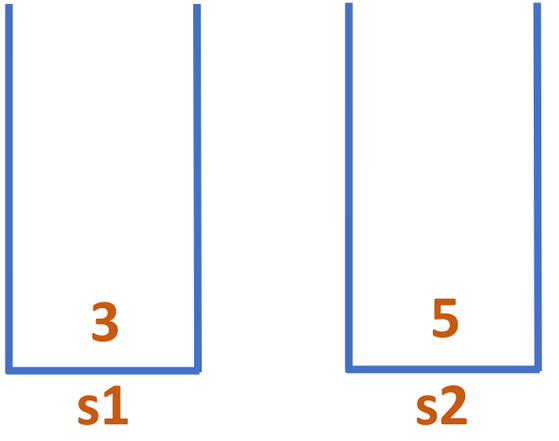
Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```



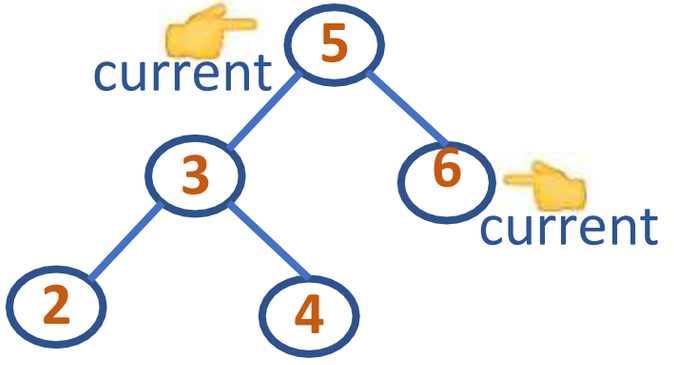
current = 5

current->right = 6

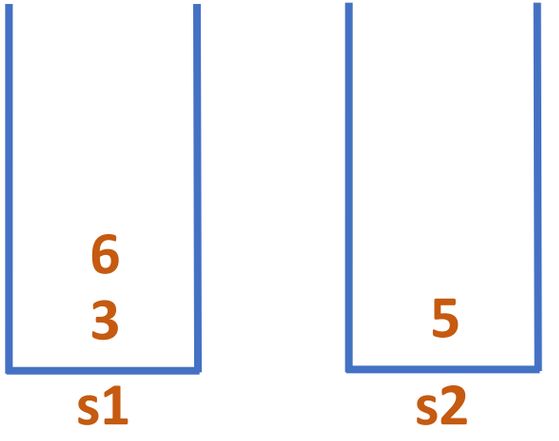


Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```

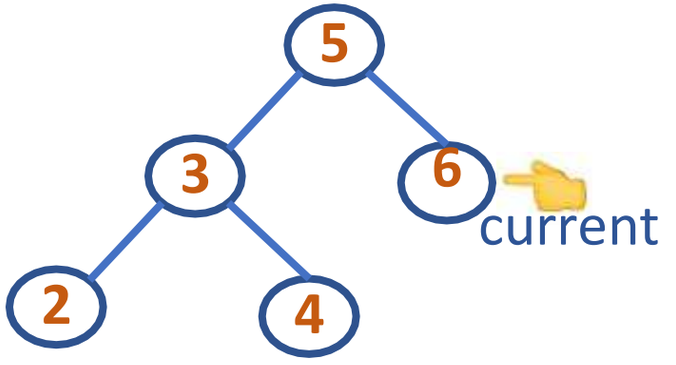


current = 5



Iterative Postorder Traversal

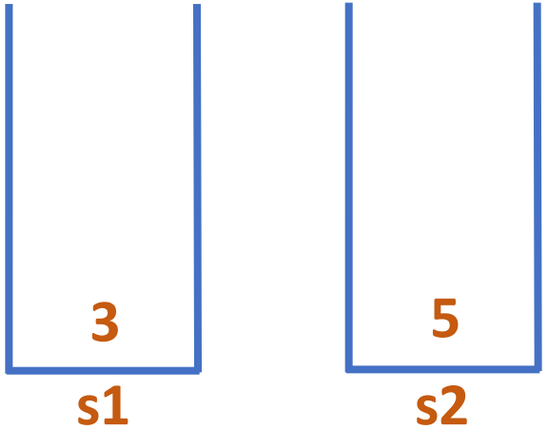
```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```



current = 6

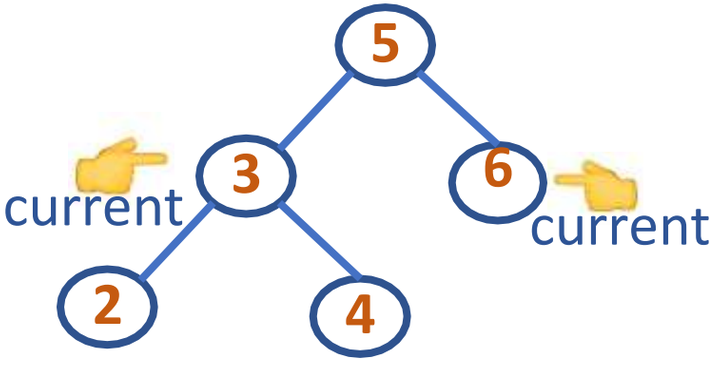
current->left = N

current->right = N

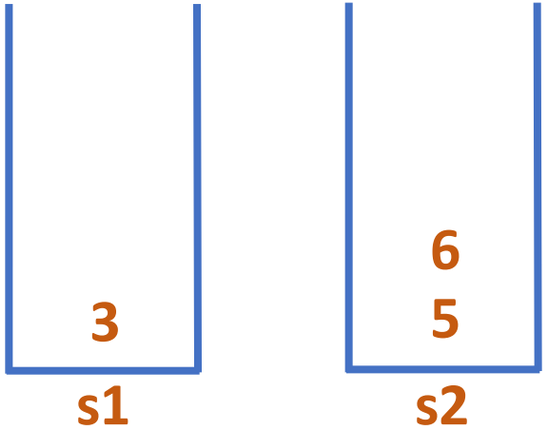


Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```

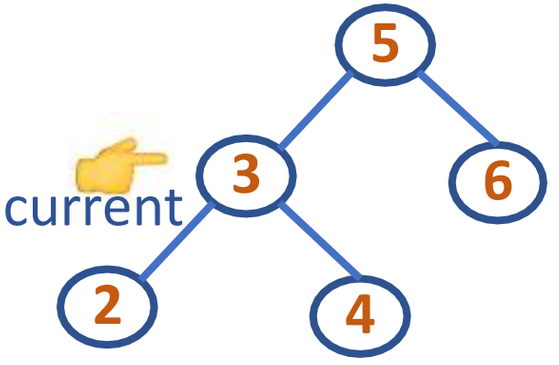


current = 6

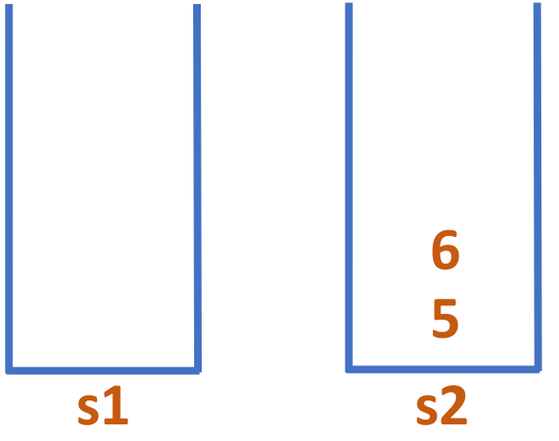


Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```

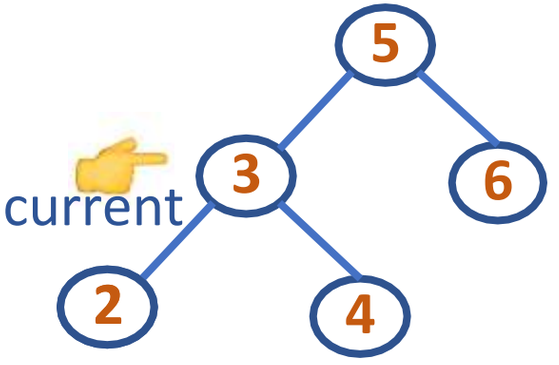


current = 3



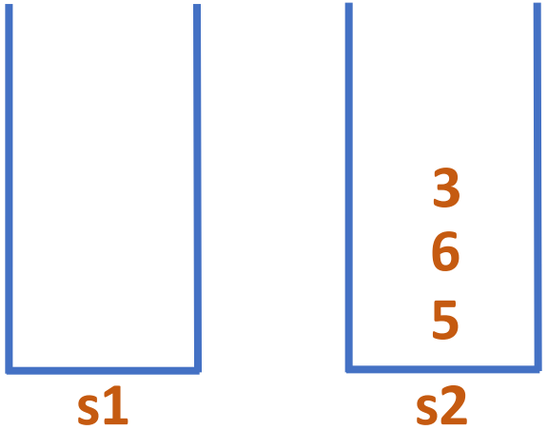
Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```



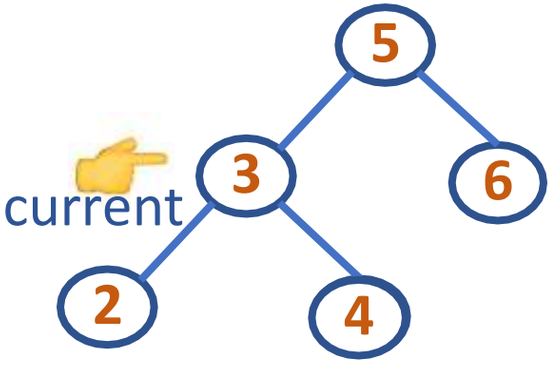
current = 3

current->left = 2



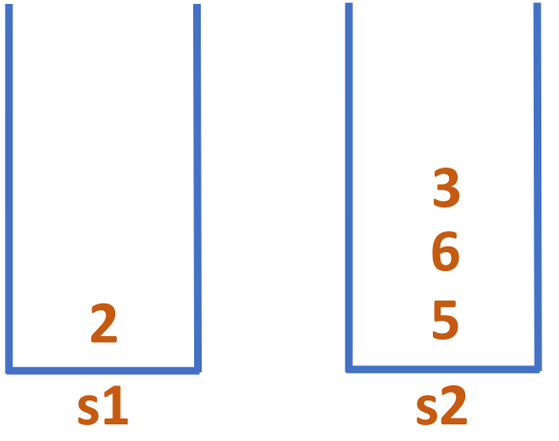
Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```



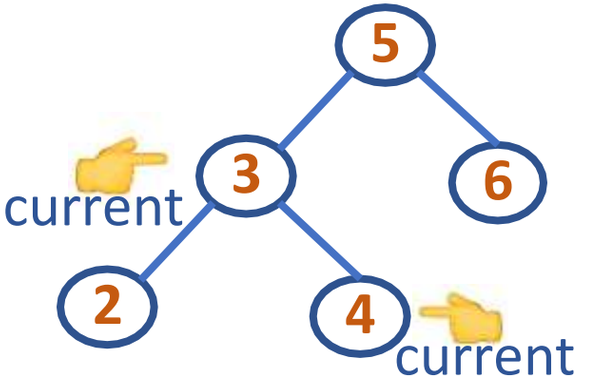
current = 3

current->right = 4

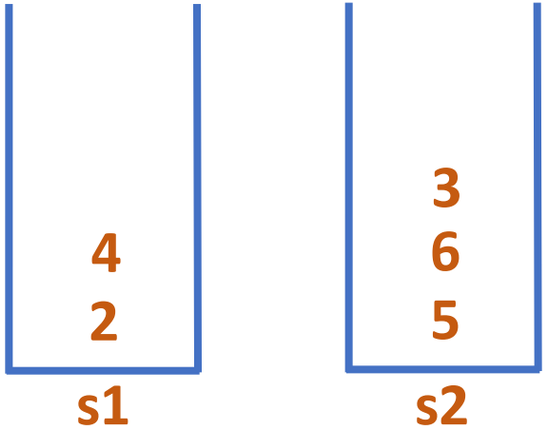


Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```

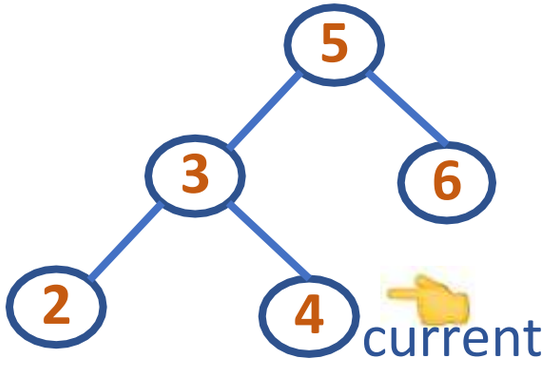


current = 3



Iterative Postorder Traversal

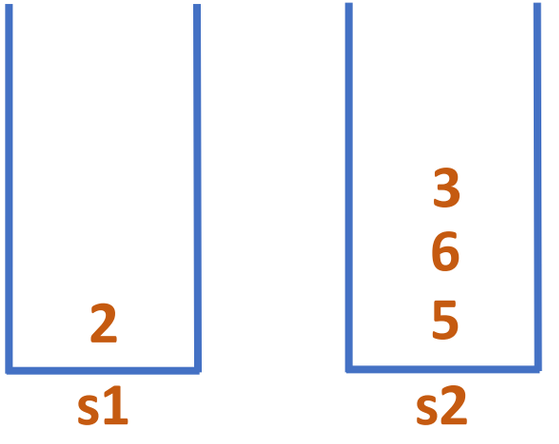
```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```



current = 4

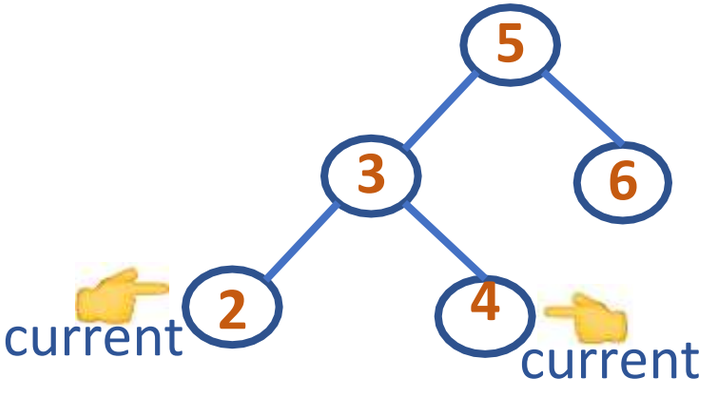
current->left = N

current->right = N

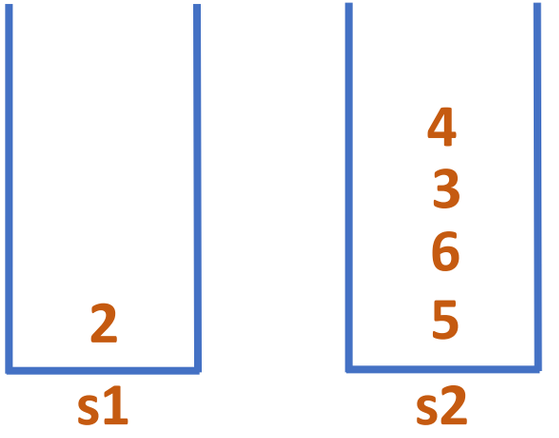


Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```

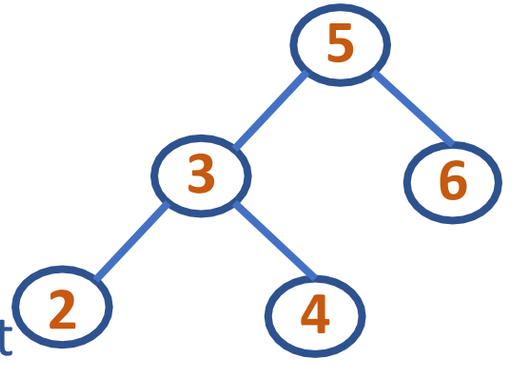


current = 4



Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
    ...
```

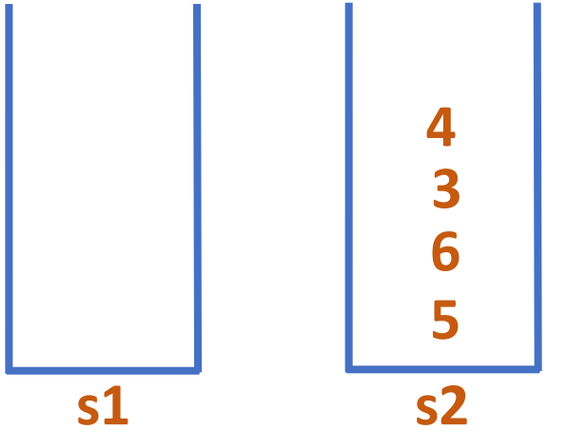


current

current = 2

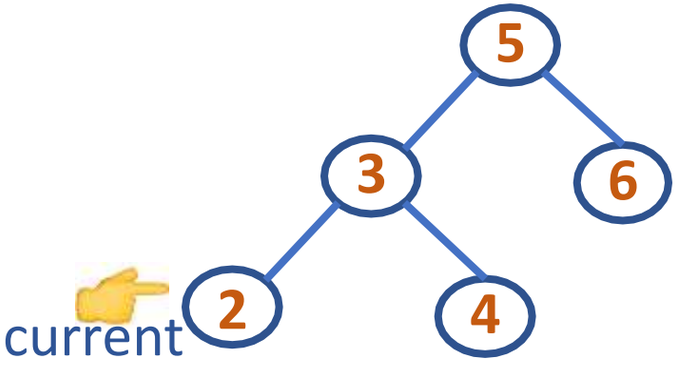
current->left = N

current->right = N

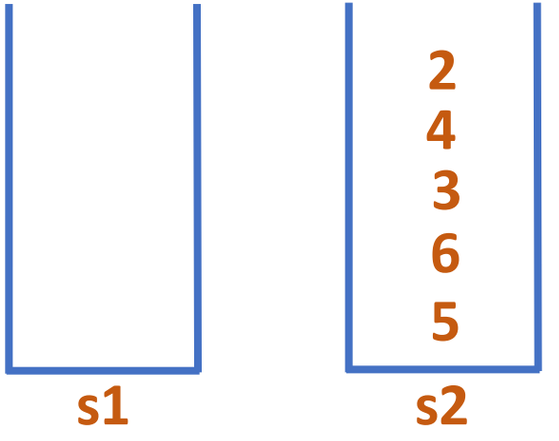


Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
while(!isEmpty(s2))
     current  =  pop(s2)
     print current->info
}
```

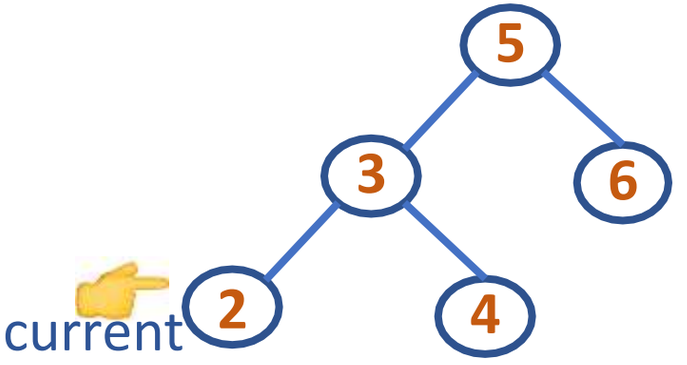


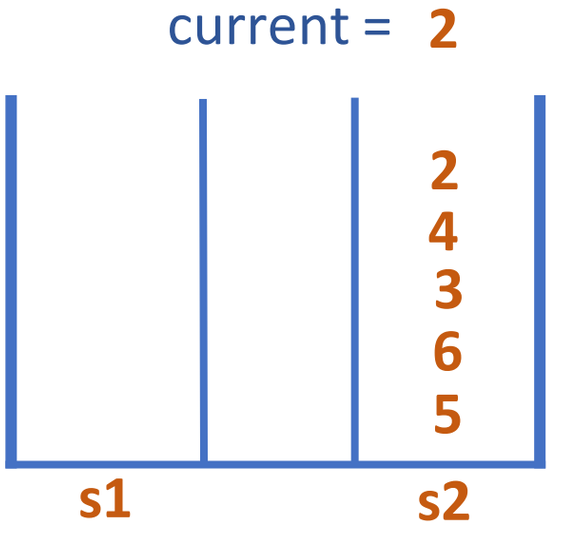
current = 2



Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
while(!isEmpty(s2))   {
     current  =  pop(s2)
     print current->info
}
```





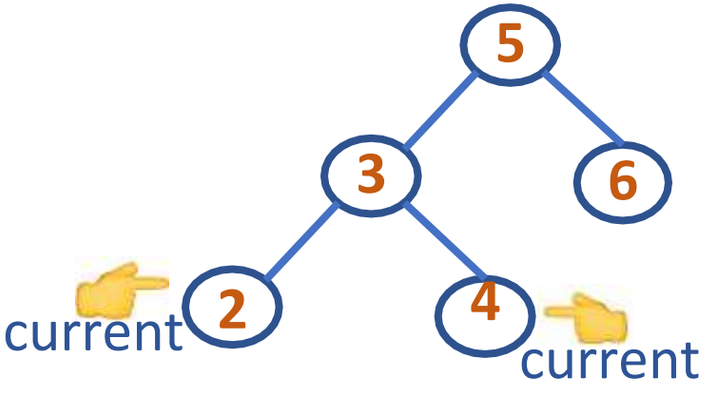
Postorder Traversal:

2

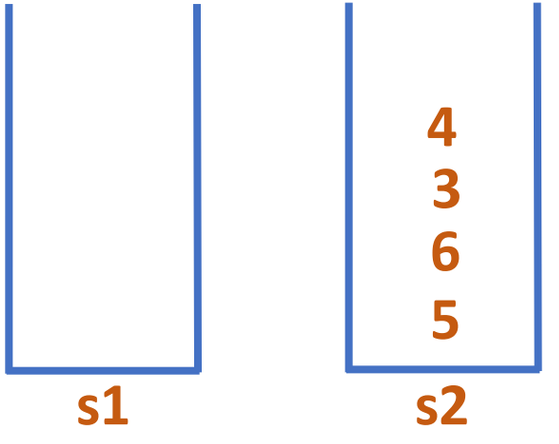
DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
while(!isEmpty(s2))
     current  =  pop(s2)
     print current->info
}
```



current = 2

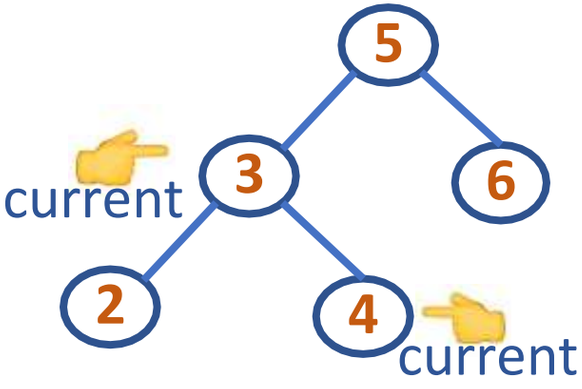


Postorder Traversal:

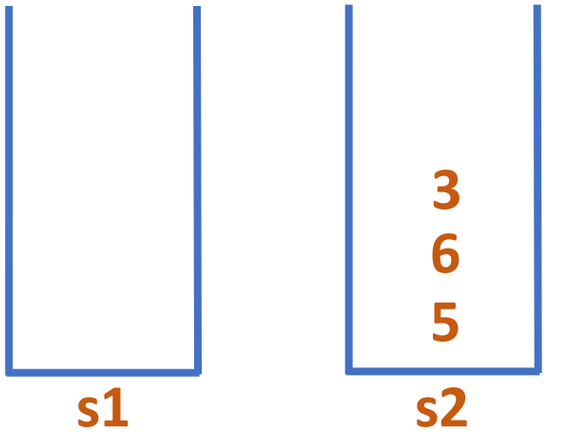
2 4

Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
while(!isEmpty(s2))
     current  =  pop(s2)
     print current->info
}
```



current = 4

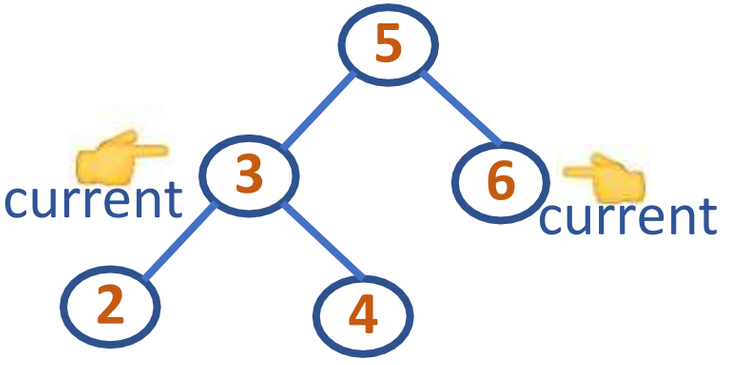


Postorder Traversal:

2 4 3

Iterative Postorder Traversal

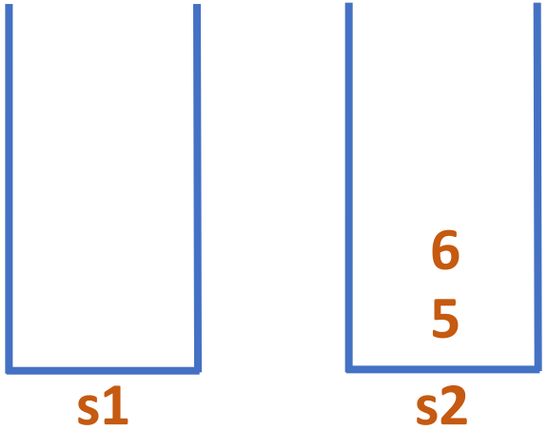
```
iterativePostorder(root)
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
while(!isEmpty(s2))
     current  =  pop(s2)
     print current->info
}
```



Postorder Traversal:

2 4 3 6

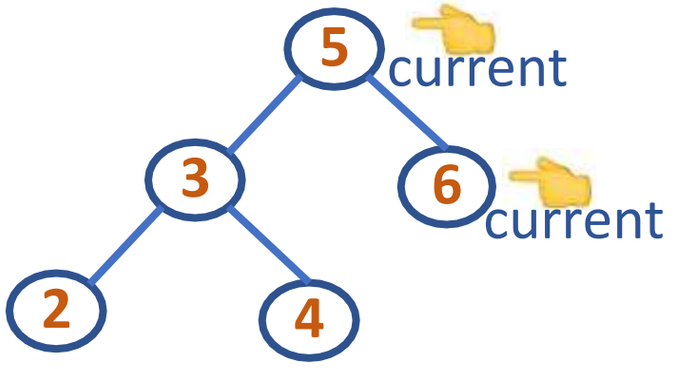
current = 3



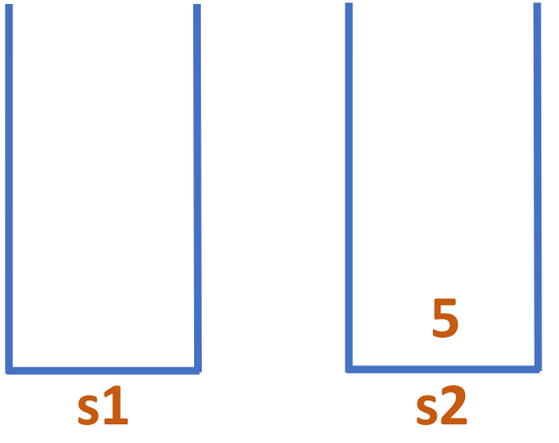
DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
    ...
    ...
while(!isEmpty(s1))
{
   current = pop(s1)
   push(s2,current)
   if(current->left !=NULL)
     push(s1, current->left)
   if(current->right !=NULL)
     push(s1, current->right)
}
while(!isEmpty(s2))
     current  =  pop(s2)
     print current->info
}
```



current = 6



Postorder Traversal:

2 4 3 6 5

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE22CS252A

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

n-ary Tree Traversal

Shylaja S S
Department of Computer Science & Engineering

Tree Traversal

Structure of a treenode revisited

```
struct treenode{
        int info;
        struct treenode *child;
        struct treenode *sibling;
};
```

Tree Traversal

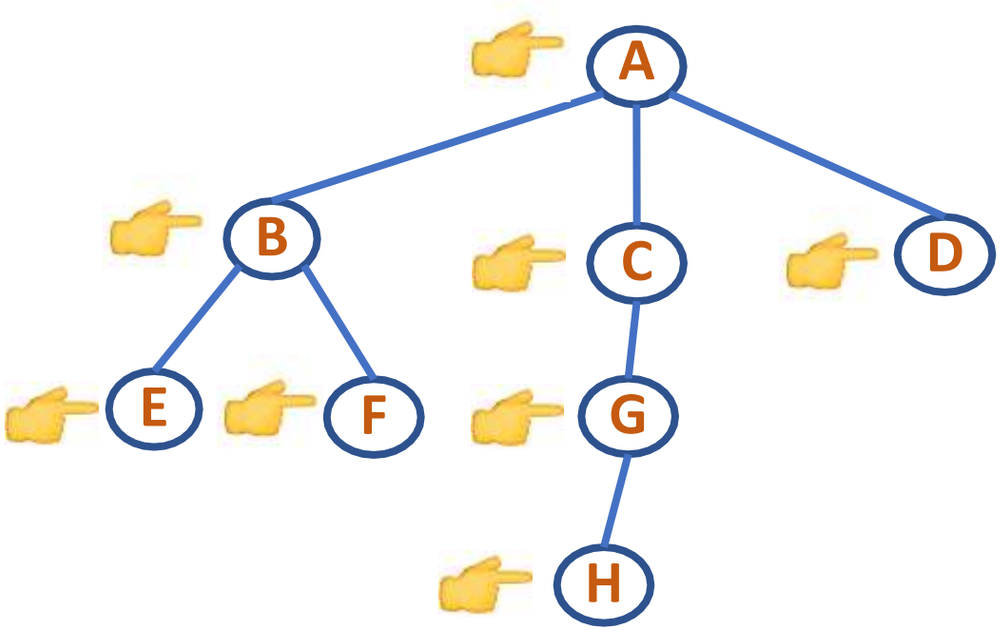
With the treenode implemented as having pointers to first child and immediate sibling, the traversal preorder, inorder and postorder for a tree are defined as follows:

Preorder:

1. Visit the root of the first tree in the forest
2. Traverse in preorder the forest formed by the subtrees of the first tree, if any
3. Traverse in preorder the forest formed by the remaining trees in the forest, if any

Tree Traversal

Preorder Tree Traversal



A BE F C GHD

Tree Traversal

```
void preorder(TREE *root)
{
    if(root!=NULL)
    {
      printf(" %d ",root->info);
      preorder(root->child);
      preorder(root->sibling);
    }
}
```

Tree Traversal

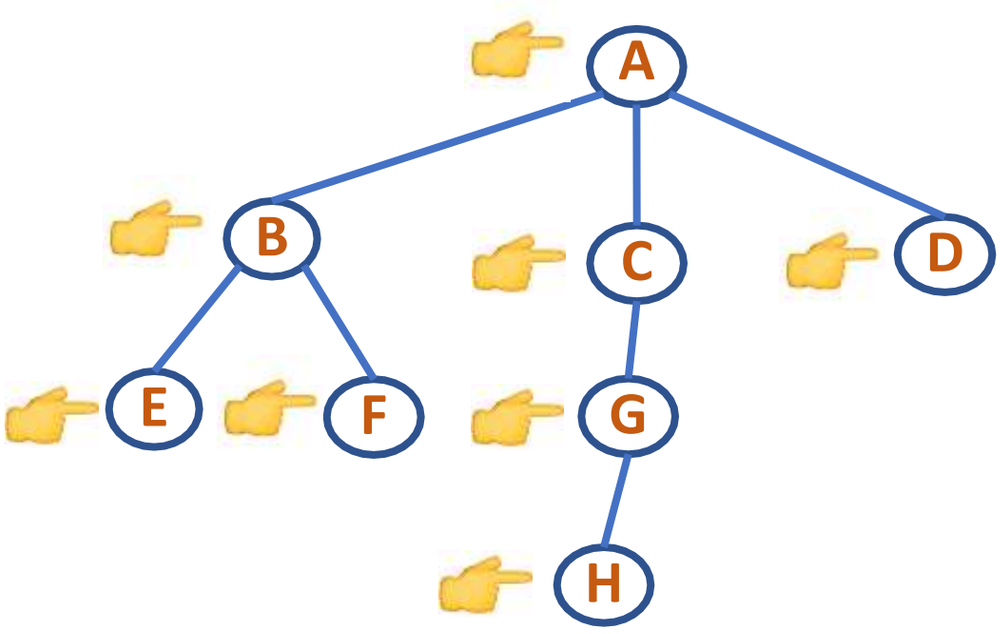
Inorder

1. Traverse in inorder the forest formed by the subtrees of the first tree, if any
2. Visit the root of the first tree in the forest
3. Traverse in inorder the forest formed by the remaining trees in the forest, if any

Tree Traversal

PES UNIVERSITY ONLINE

Inorder Tree Traversal



E F B H G C D A

Tree Traversal

```
void inorder(TREE *root)
{
  if(root!=NULL)
   {
     inorder(root->child);
     printf(" %d ",root->info);
     inorder(root->sibling);
   }
}
```

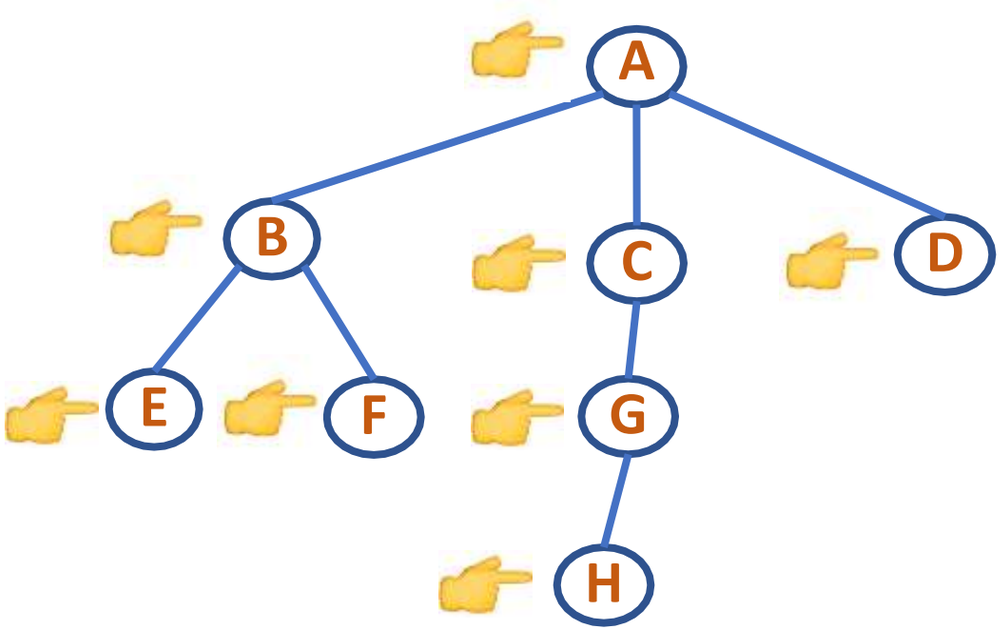
Postorder

1. Traverse in postorder the forest formed by the subtrees of the first tree, if any
2. Traverse in postorder the forest formed by the remaining trees in the forest, if any
3. Visit the root of the first tree in the forest

Tree Traversal

PES UNIVERSITY ONLINE

Postorder Tree Traversal



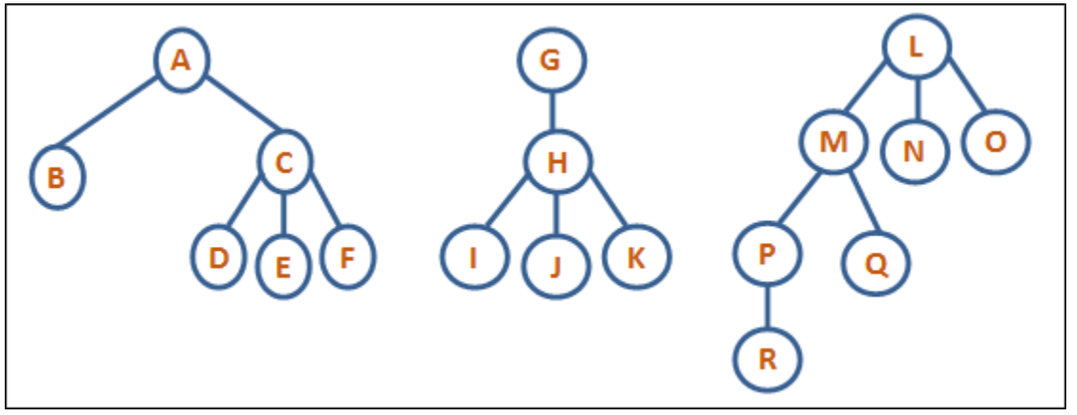
F E H G D C B A

Tree Traversal

```
void postorder(TREE *root)
{
  if(root!=NULL)
   {
     postorder(root->child);
     postorder(root->sibling);
     printf(" %d ", root->info);
   }
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Forest traversal



Preorder: ABCDEFGHIJKLMPRQNO
Inorder: BDEFCAIJKHGRPQMNOL
Postorder: FEDCBKJIHRQPONMLGA

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS
UE21CS252A

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Heap: Definition and Implementation

Shylaja S S
Department of Computer Science & Engineering

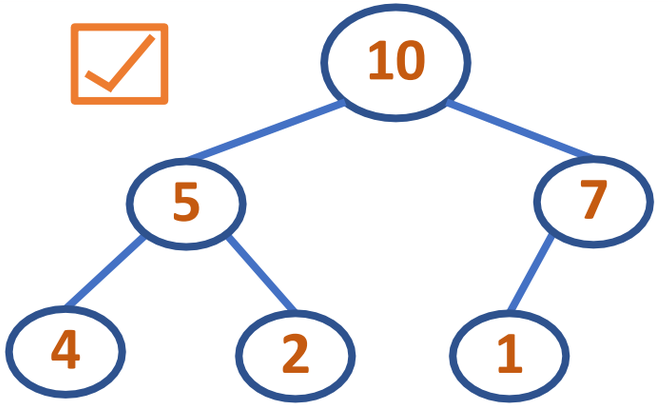
Heap Tree

Definition: A heap can be defined as a binary tree with keys assigned to its nodes (one key per node) provided the following two conditions are met:

1.The tree's shape requirement - The binary tree is essentially complete, that is, all its levels are full except possibly the last level, where only some rightmost leaves may be missing

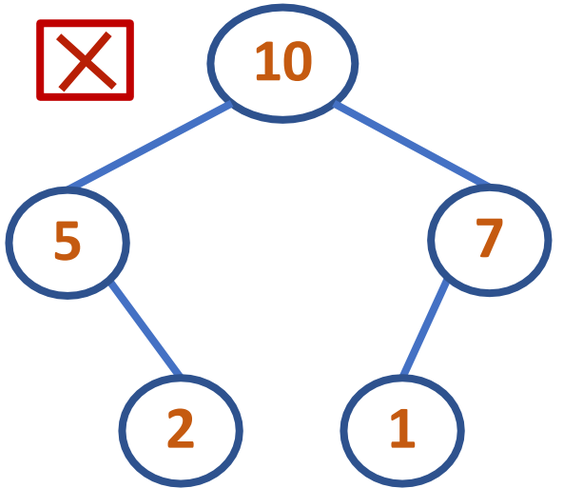
2. The parental dominance requirement - The key at each node is greater than or equal to the keys at its children. (This condition is considered automatically satisfied for all leaves.)

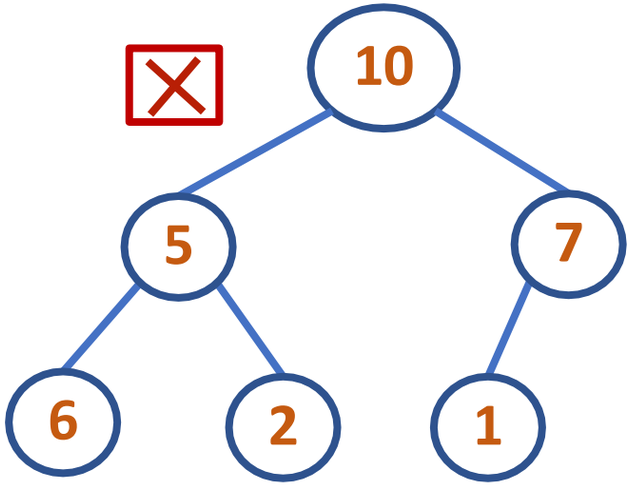
Heap Tree



Shape Requirement not satisfied

Parental dominance not satisfied





Only the topmost Binary Tree is a heap. Why?

Properties of Heap

1. There exists exactly one essentially complete binary tree with n nodes. Its height is equal to $\lfloor \log_2 n \rfloor$

2. The root of a heap always contains its largest element

3. A node of a heap considered with all its descendants is also a heap

4. A heap can be implemented as an array by recording its elements in the top-down, left-to-right fashion. It is convenient to store the heap's elements in positions 1 through n of such an array, leaving H[0] either unused or putting there a sentinel whose value is greater than every element in the heap.

Properties of Heap

...

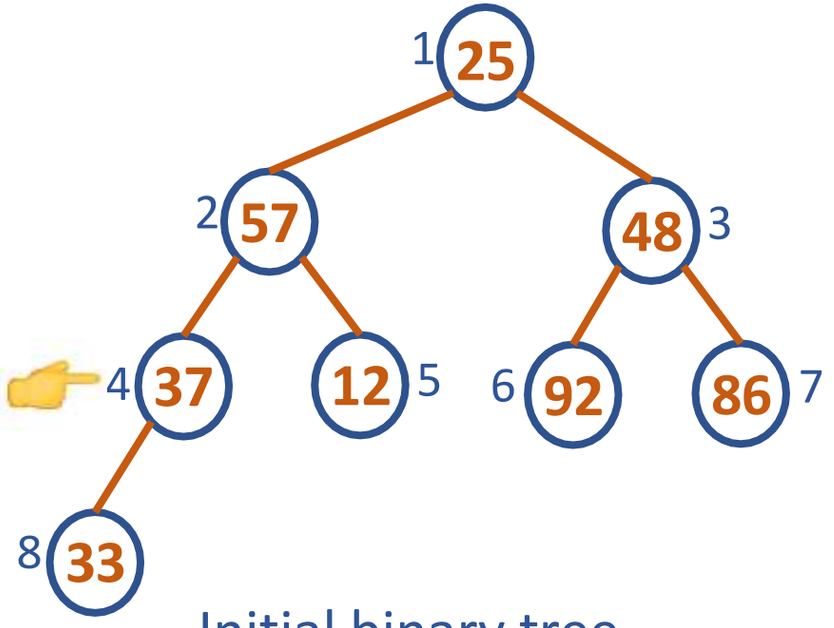
In such a representation,

a) The parental node keys will be in the first ⌊n/2⌋ positions of the array, while the leaf keys will occupy the last ⌈n/2⌉ positions

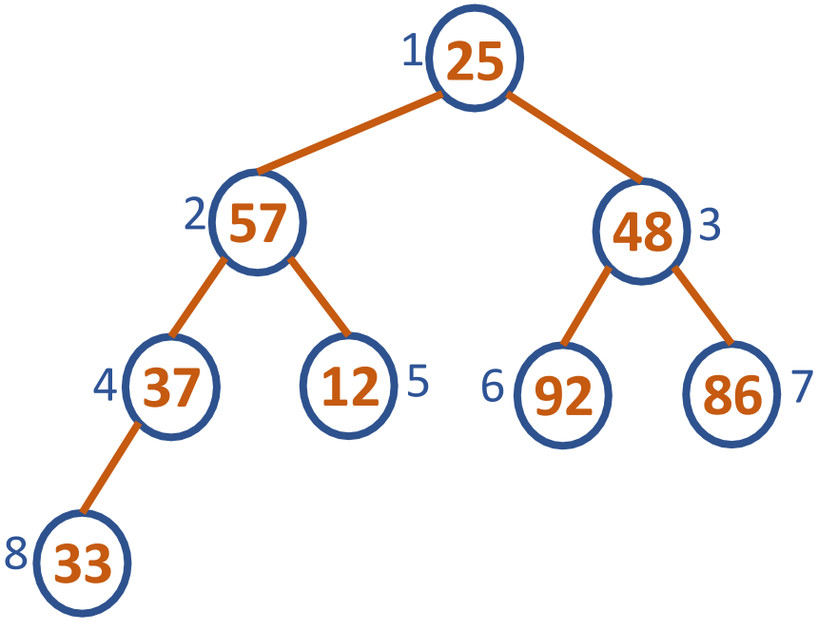
b) The children of a key in the array's parental position i (1 <= i <= ⌊n/2⌋) will be in positions 2i and 2i + 1, and, correspondingly, the parent of a key in position i (2 <= i <= n) will be in position ⌊i/2⌋

Heap Construction – Bottom Up

Bottom Up Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33
Here, n=8



Initial binary tree

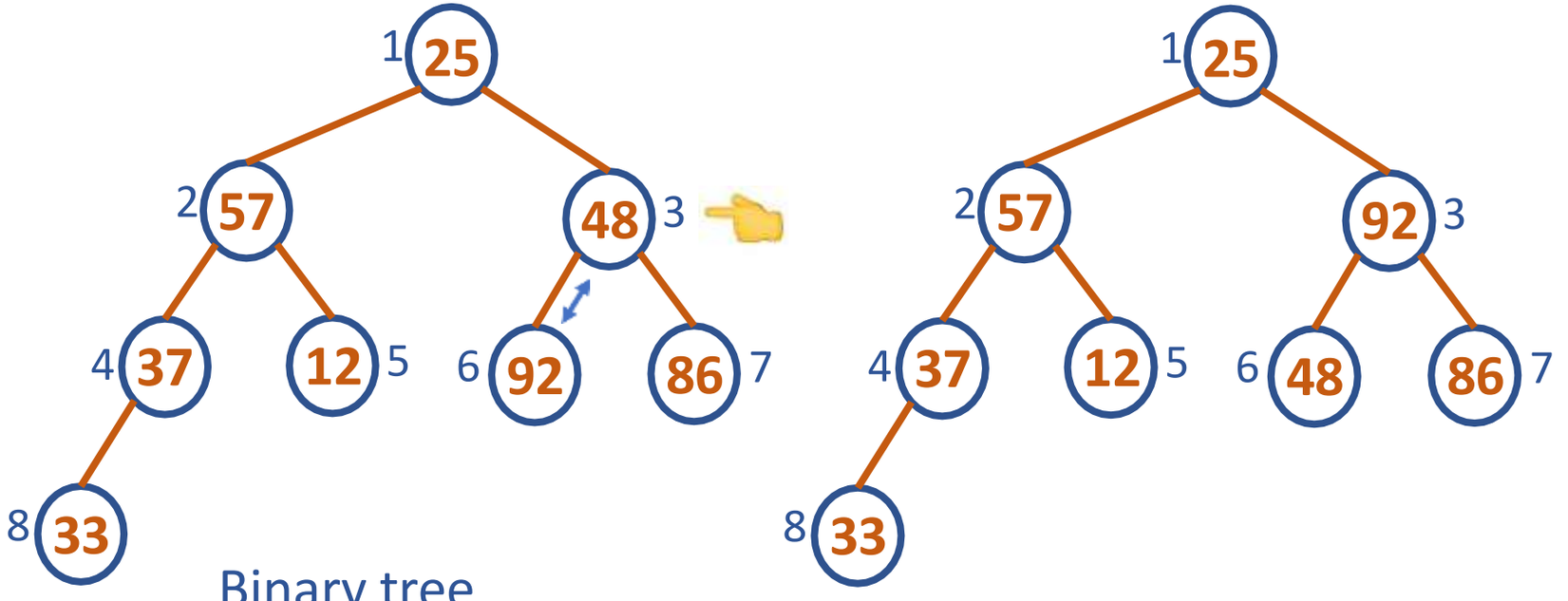


At k = 4, v = 37
Compare 37 with its only child 33
37 > 33, it's a heap at k=4

Heap Construction – Bottom Up

Bottom Up Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, n=8



Binary tree after one iteration at k=4

At k = 3, v = 48

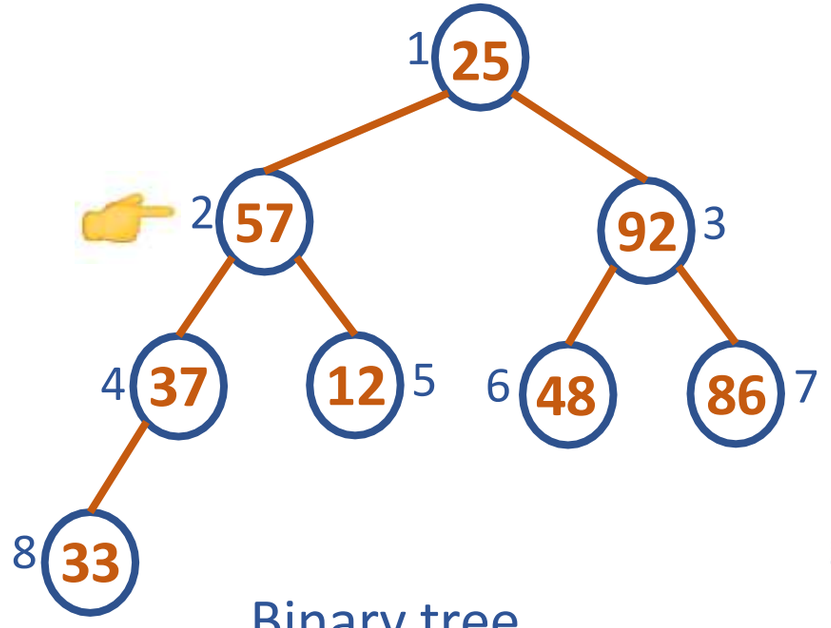
Largest child: 92

Compare 48 with its largest child

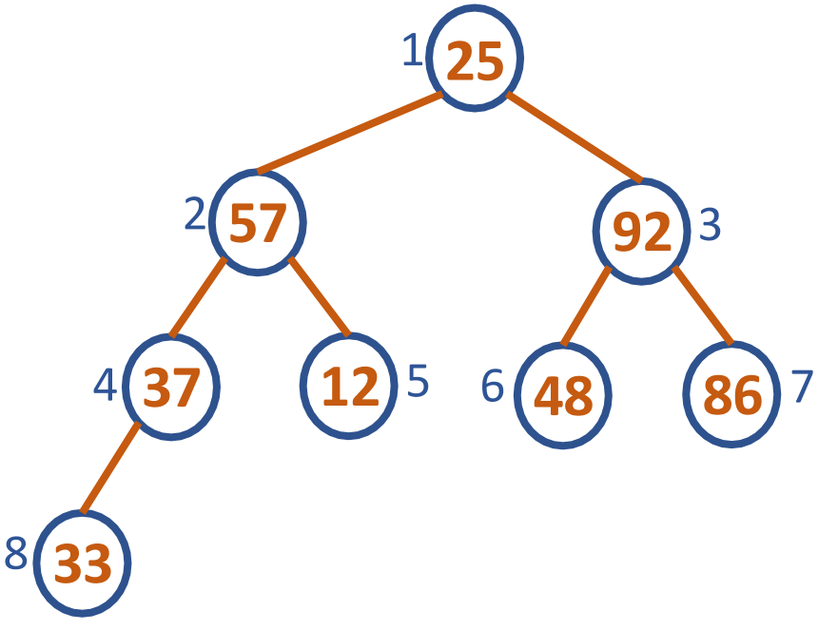
48 < 92, Heapify

Heap Construction – Bottom Up

Bottom Up Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33
Here, n=8



Binary tree
after two iterations at k=4, k=3

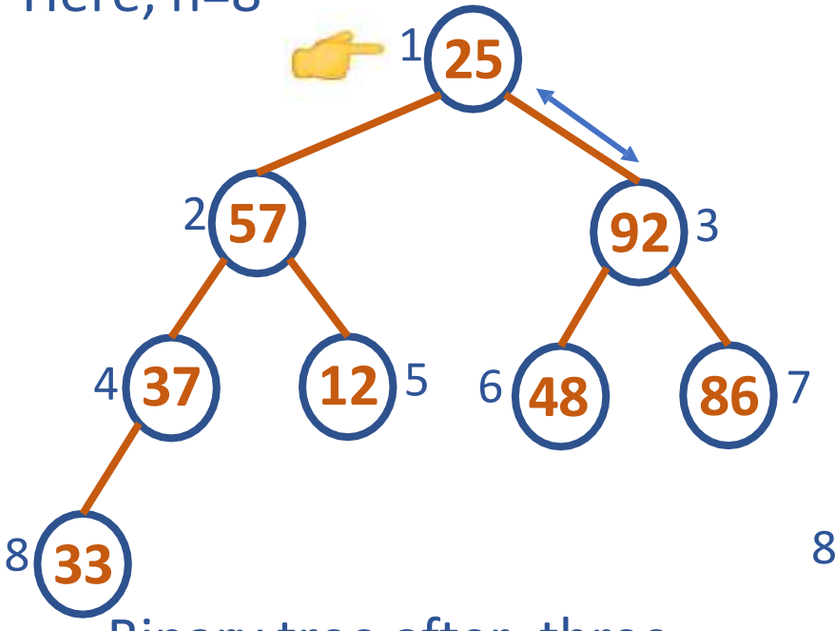


At k = 2, v = 57
Largest child: 37
Compare 57 with its largest child
57 > 37, it's a heap at k=2

Heap Construction – Bottom Up

Bottom Up Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, n=8



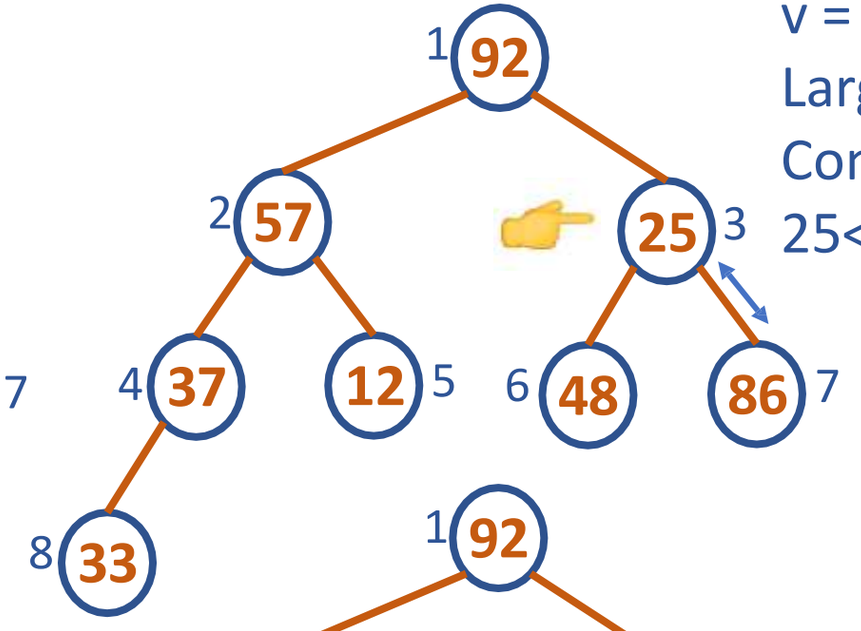
Binary tree after three iterations at k=4, k=3, k=2

At k = 1, v = 25

Largest child: 92

Compare 25 with its largest child

25 < 92, Heapify

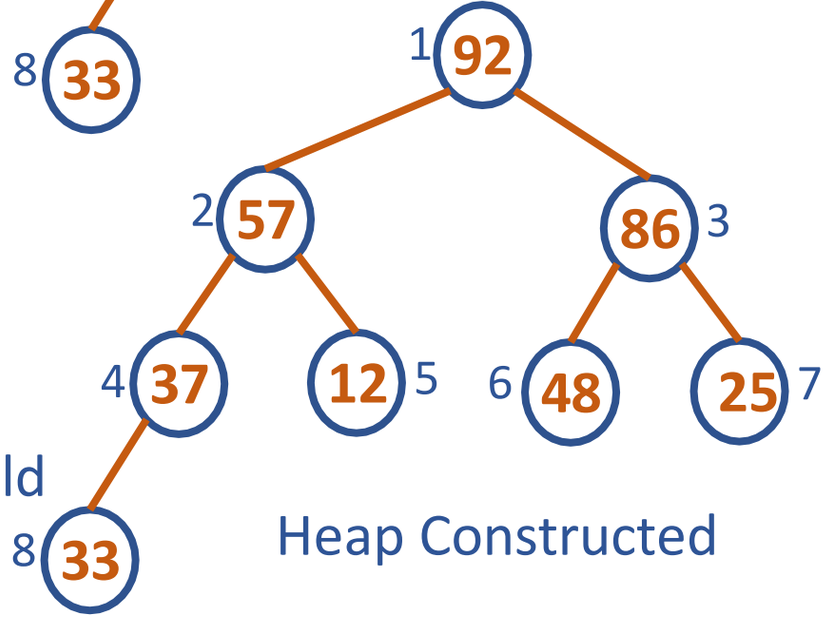


v = 25, Now at k = 3,

Largest child: 86

Compare 25 with its largest child

25<86, Heapify

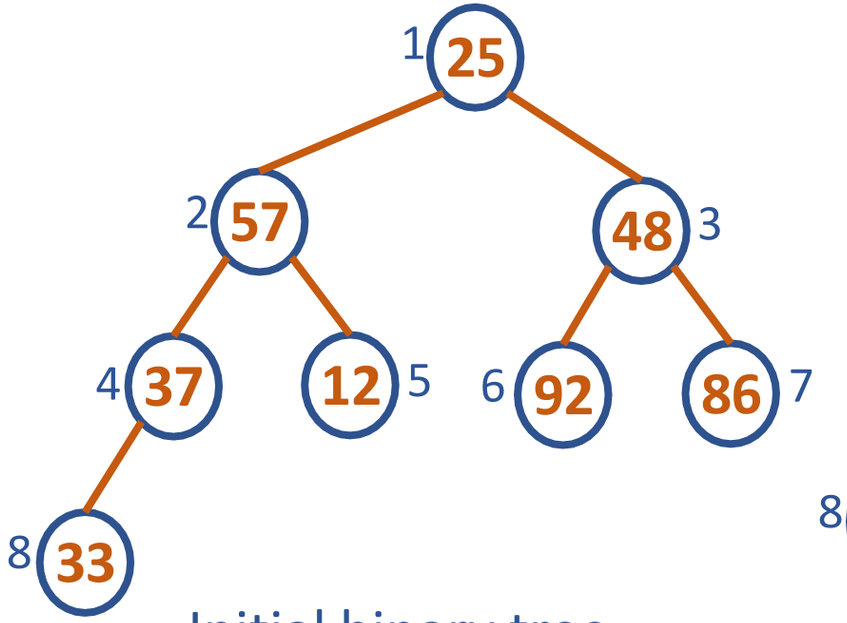


Heap Constructed

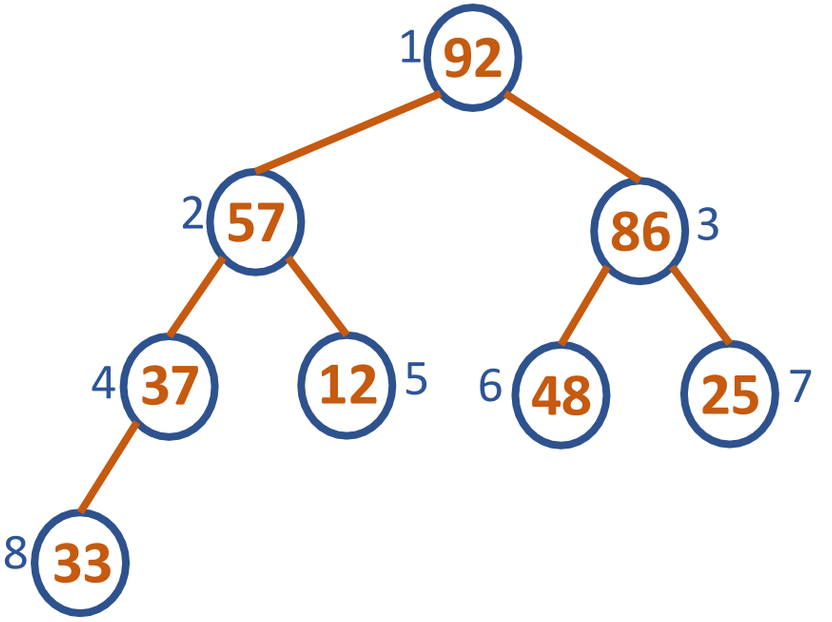
Heap Construction – Bottom Up

Bottom Up Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, n=8



Initial binary tree



Bottom Up Heap Constructed

Heap Construction – Bottom Up

```
ALGORITHM HeapBottomUp(H[1…n])
//Constructs a heap from the elements of a given array by bottom-up algorithm
//Input: An array H[1…n] of orderable items
//Output: A heap H[1…n]
for i ← ⌊n/2⌋ downto 1 {
      k ← i
      v ← H[k]
      heap ← false
      while not heap and 2*k ≤ n {
            j ← 2*k
            if j < n                        //if there are two children
               if H[j] < H[j+1]
                     j ← j+1                //find position of largest child
            if v ≥ H[j]          //if key of parent node ≥ key of largest child
               heap ← true                  //it's a heap
            else {                          //heapify
                     H[k] ← H[j]
                     k ← j
            }          //end of else
      }    //end of while
      H[k] ← v
}    //end of for
```

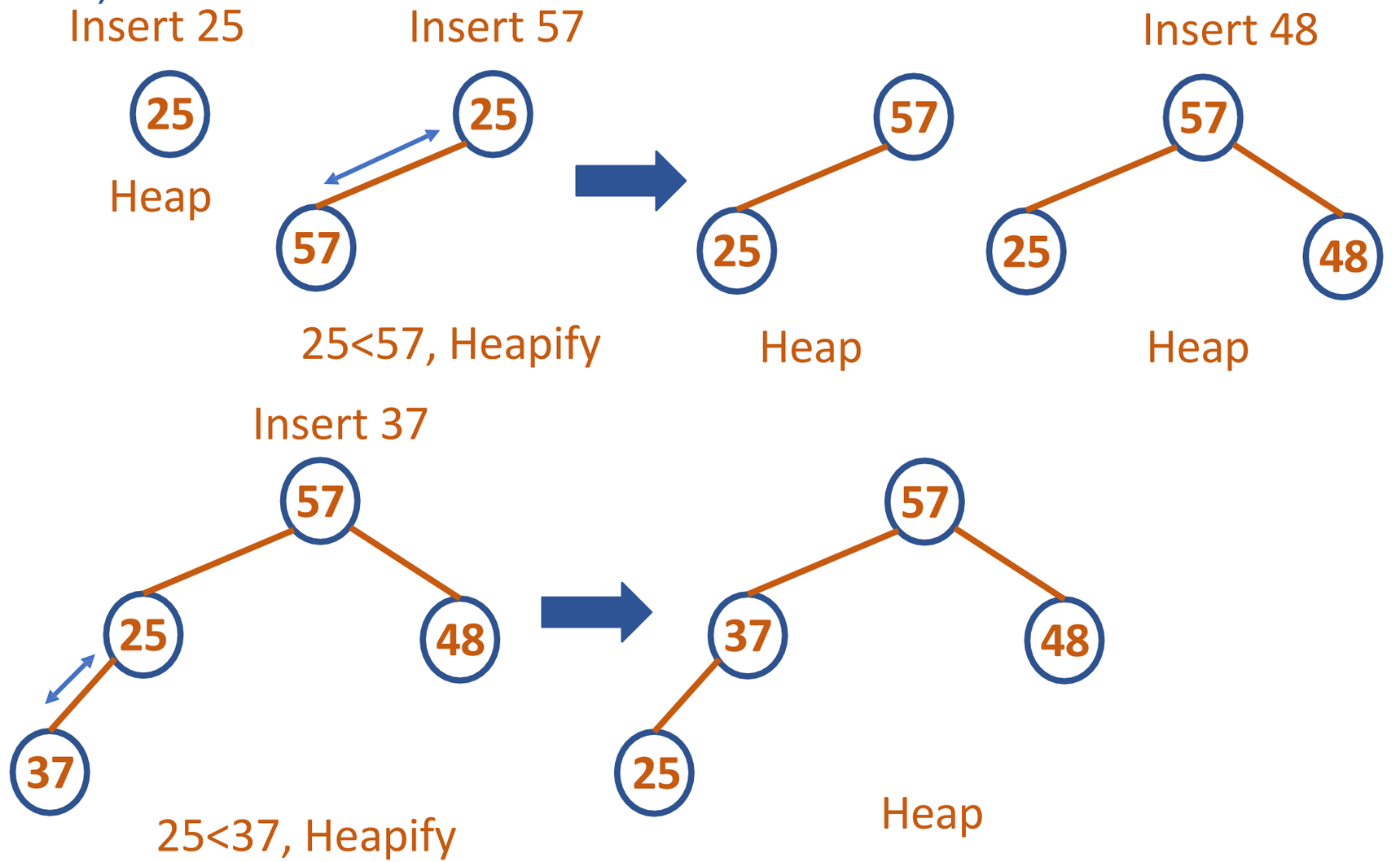
Heap Construction – Bottom Up

```
for(i=n/2-1;i>=0;i--)
 {
  k=i;
  v=h[k];
  heap=0;
  while(!heap && 2*k+1<=n-1)
   {
     j=2*k+1;
     if(j+1<=n-1)
      if(h[j+1]>h[j])  j=j+1;
    if(v>h[j])
      heap=1;
    else
     {
      h[k]=h[j];
      k=j;
      }
   }
    h[k]=v;
  }
```

Heap Construction – Top Down

Top Down Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

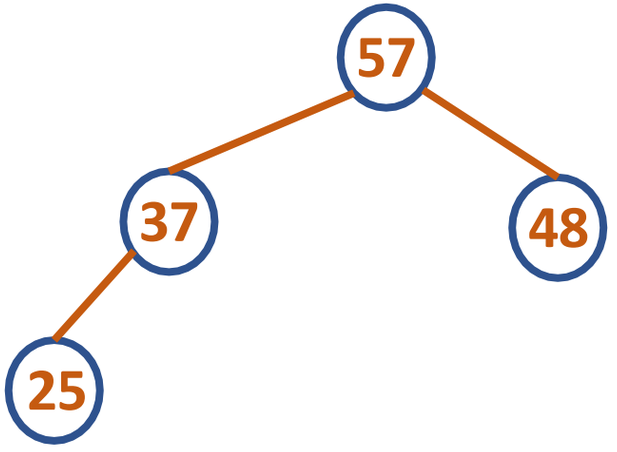
Here, n=8



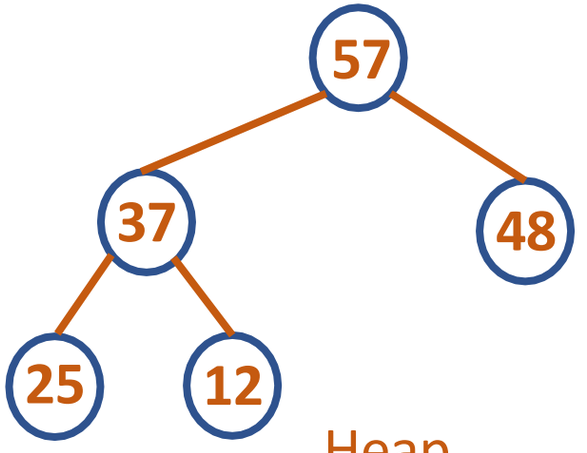
Heap Construction – Top Down

Top Down Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, n=8

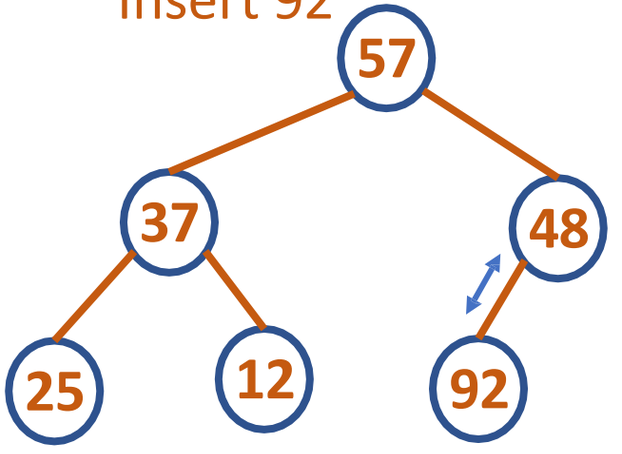


Insert 12

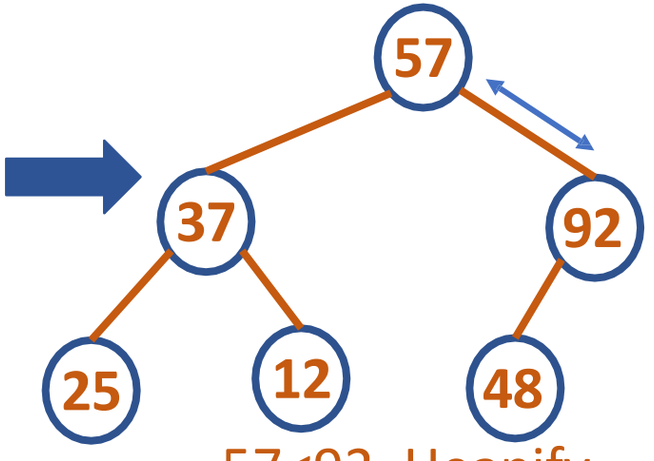


Heap

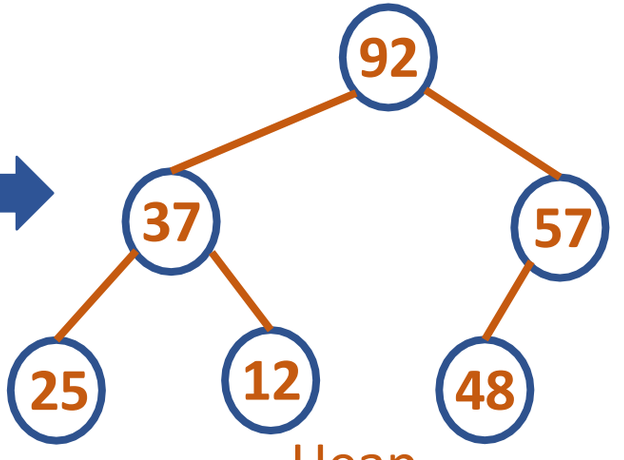
Insert 92



48<92, Heapify



57<92, Heapify

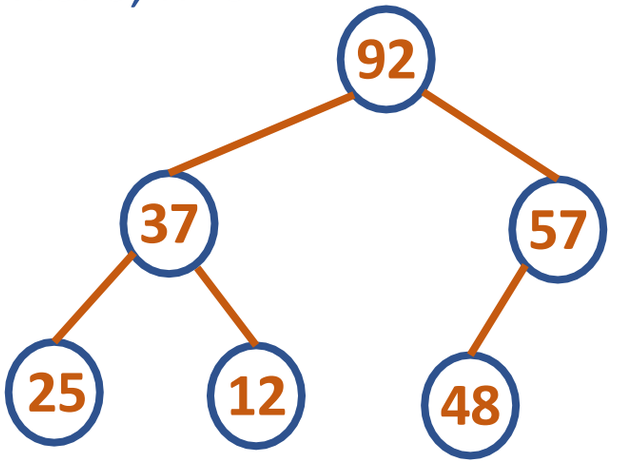


Heap

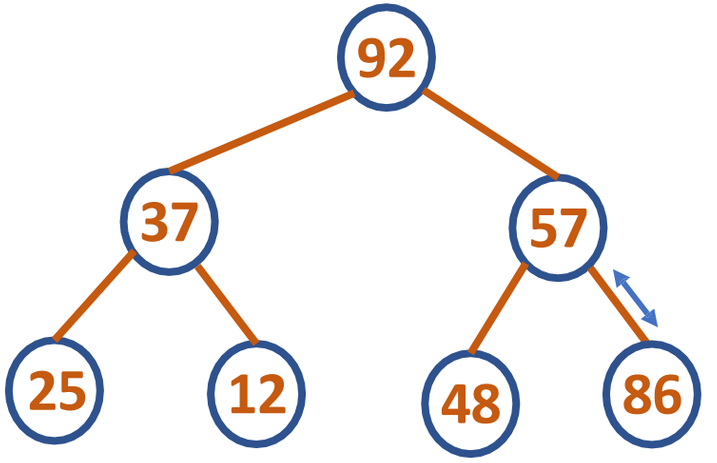
Heap Construction – Top Down

Top Down Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

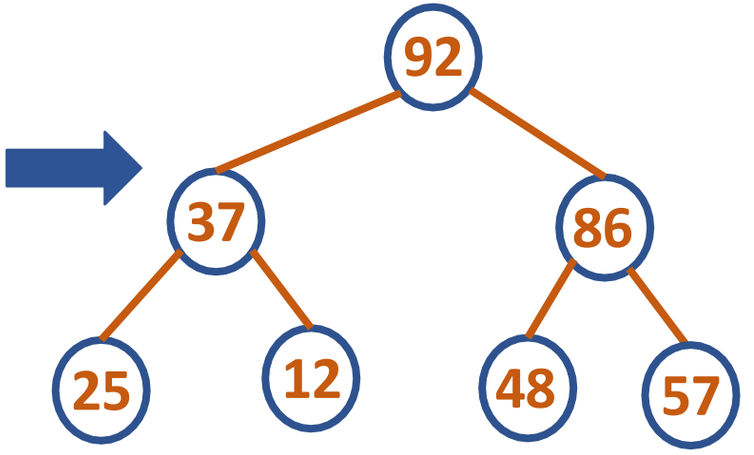
Here, n=8



Insert 86



57<86, Heapify

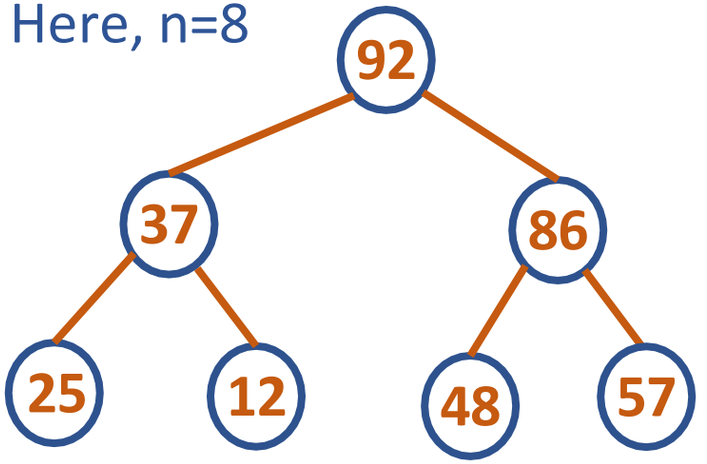


Heap

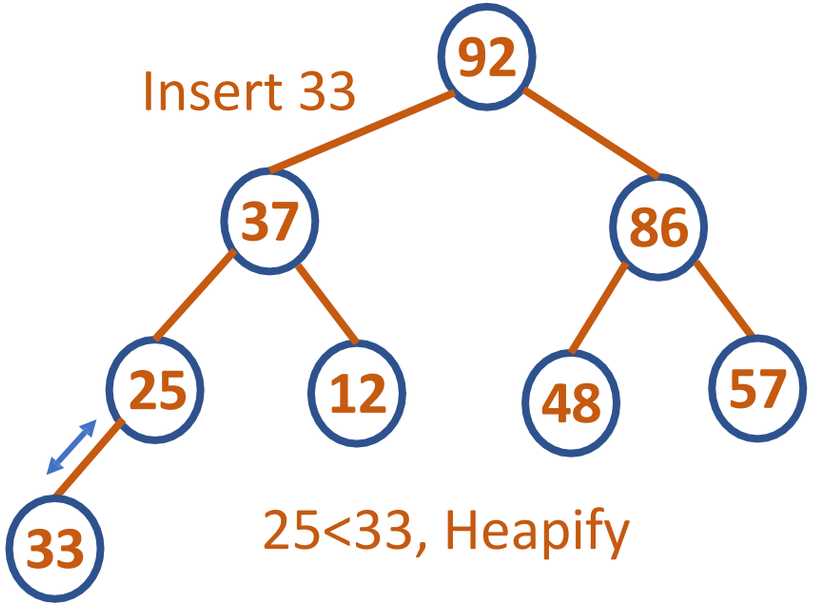
Heap Construction – Top Down

Top Down Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

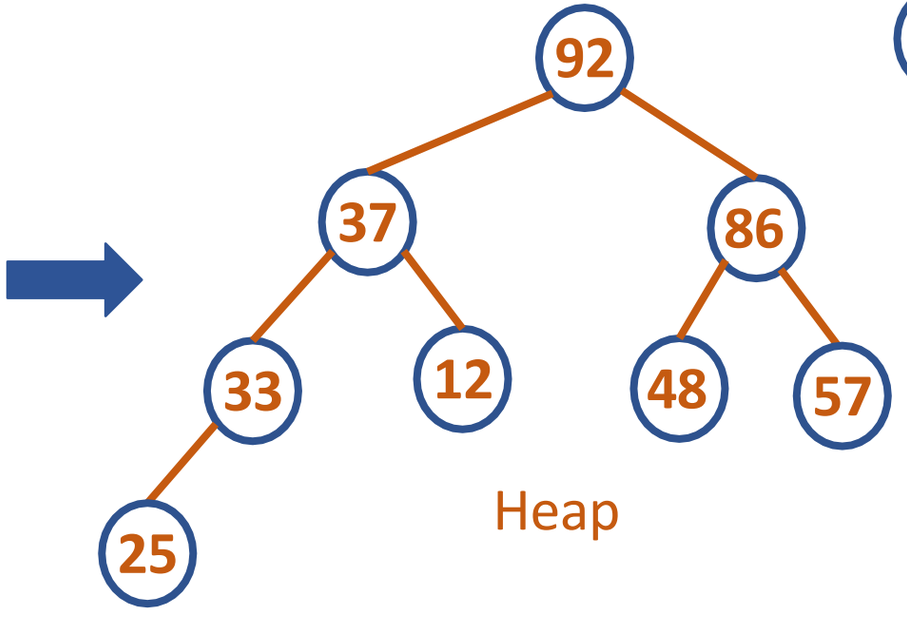
Here, n=8



Insert 33



25<33, Heapify



Heap

Heap Construction – Top Down

1. First, attach a new node with key *K* in it after the last leaf of the existing heap
2. Then sift *K* up to its appropriate place in the new heap as follows
3. Compare *K* with its parent's key: if the latter is greater than or equal to *K,* stop (the structure is a heap);
4. otherwise, swap these two keys and compare *K* with its new parent
5. This swapping continues until *K* is not greater than its last parent or it reaches the root

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

Department of Computer science and Engineering

PES UNIVERSITY

UE19CS202: Data Structures and its Applications (4-0-0-4-4)

GRAPHS

Abstract

Implementation of Graph using Adjacency List.

Dr.Sandesh and Saritha

Sandesh_bj@pes.edu

Saritha.k@pes.edu

Implementation of adjacency List:

C representation of adjacency list

1.using array implementation

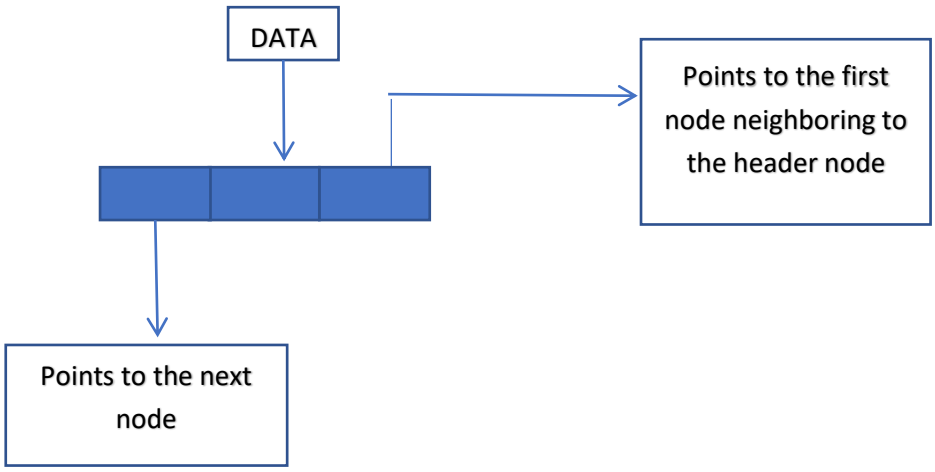
```
#define MAX 50
struct node
{
  int info;
  int point;
  int next;
};
struct node NODE[MAX];
```

2.Using dynamic implementation

```
struct node
{
   int info;
  strut node *pointer;
   struct node *next;
};
struct node *nodeptr;
```

Two lists are maintained for adjacency of list. The first list is used to store information of all the nodes. The second list is used to store the information of adjacent nodes for each node of a graph.

Header node is used to represent each list, which is assumed to be the first node of a list as shown below.



The different operations performed on adjacency list are

1. To create an arc between two nodes

```
void jointwt(int p,int q,int wt)
    {
        int r,r2;
        r2=-1;
       r=node[p].point;
       While(r>=0 && node[r].point!=q)
        {
            r2=r;
            r=node[r].next;
         }
       If(r>=0)
        {
            node[r].info=wt;
```

```
                    return;
                }
            R=getnode();
            node[r].point=q;
            node[r].next=-1;
             node[r].info=wt;
             (R2<0)?(node[p].point=r):(node[r2].next=r);
}
```

2. To remove an arc between two nodes if it exists:

```
Void remv(int p,int q)
{
    int r,r2;
   r2=-1;
   r=node[p].point;
  while(r>=0 ** node[r].point !=q)
 {
     r2=r;
    r=node[r].next;
}
If(r>=0)
{
   (r2<0)?(node[p].point=node[r].next):(node[r2].next=node[r].next);
   freenode(r);
   return;
}
```

```
}
```

3. Checks whether a node is adjacent to another node

```
int adjacent(int p,int q)
{
   int r;
    r=node[p].point;
   while(r>=0)
        If(node[r].point==q)
            return(TRUE)
         else
            r=node[r].next;
              return(FALSE);
}
```

4. Find a node with a specific information in a graph:

```
int findnode(int graph,int x)
{
        int p;
        p=graph;
        while(p>=0)
                if (node[p].info==x)
                        return(p);
                else
                        p=node[p].next;
```

```
        return(-1);
}
```

5. Add a node with specific information to a graph.

```
int addnode(int *graph,int x)
{
    int p;
    p=getnode();
    node[p].info=x;
    node[p].point=-1;
    node[p].next=*graph;
    *graph=p;
    return(p);
}
```

DATA STRUCTURES AND ITS APPLICATIONS UE21CS252A

Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree

Kusuma K V
Department of Computer Science & Engineering

Splay Tree

- A splay tree is a binary search tree with the additional property that recently accessed elements are quick to access again.
- Example: Hospital records
- Like self-balancing binary search trees, a splay tree performs basic operations such as insertion, look-up and removal in O(log n) amortized time.
- All normal operations on a binary search tree are combined with one basic operation, called splaying.

Splay Tree

- Splaying the tree for a certain element rearranges the tree so that the element is placed at the root of the tree.
- One way to do this with the basic search operation is to first perform a standard binary tree search for the element in question, and then use tree rotations in a specific fashion to bring the element to the top.

Splaying

- Each splay step depends on three factors:
  - whether x is the left or right child of its parent node, p,
  - whether p is the root or not, and if not
  - whether p is the left or right child of its parent, g (the grandparent of x).

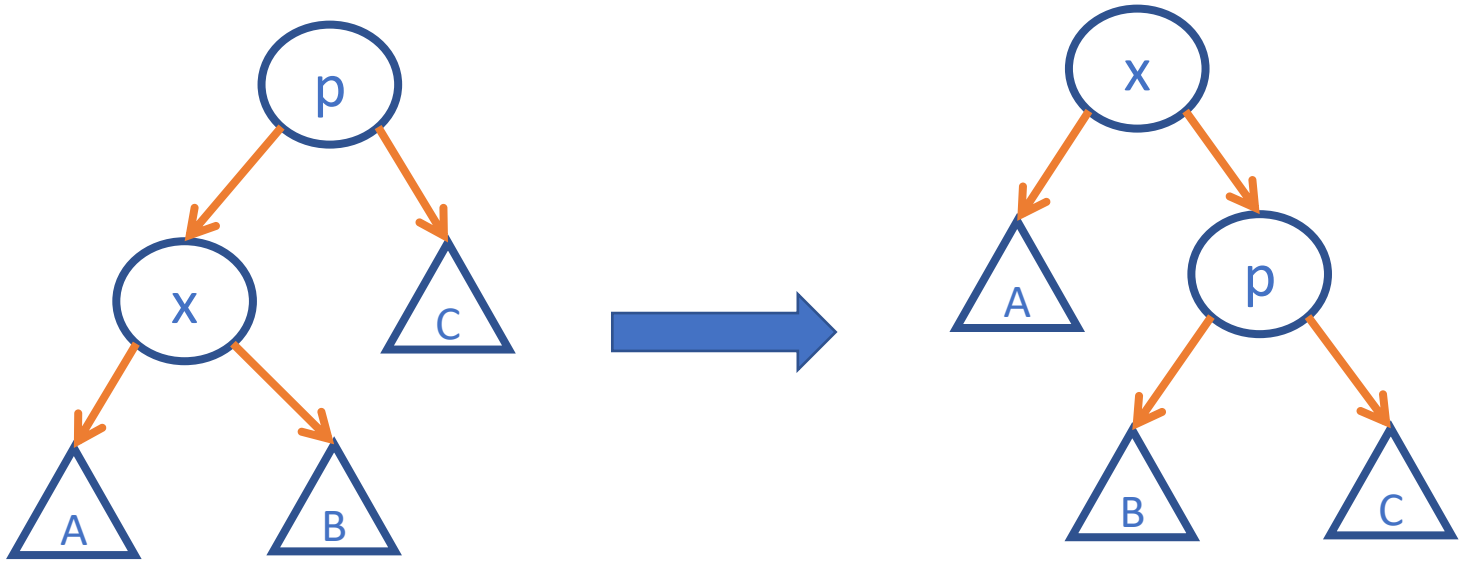
Splay Tree

Splaying

- There are three types of splay steps, each of which has two symmetric variants: left- and right-handed.
- Zig step: this step is done when p is the root.
- Zig - Zig step: this step is done when p is not the root and x and p are either both right children or are both left children.
- Zig - Zag step: this step is done when p is not the root and x is a right child and p is a left child or vice versa (x is left, p is right).

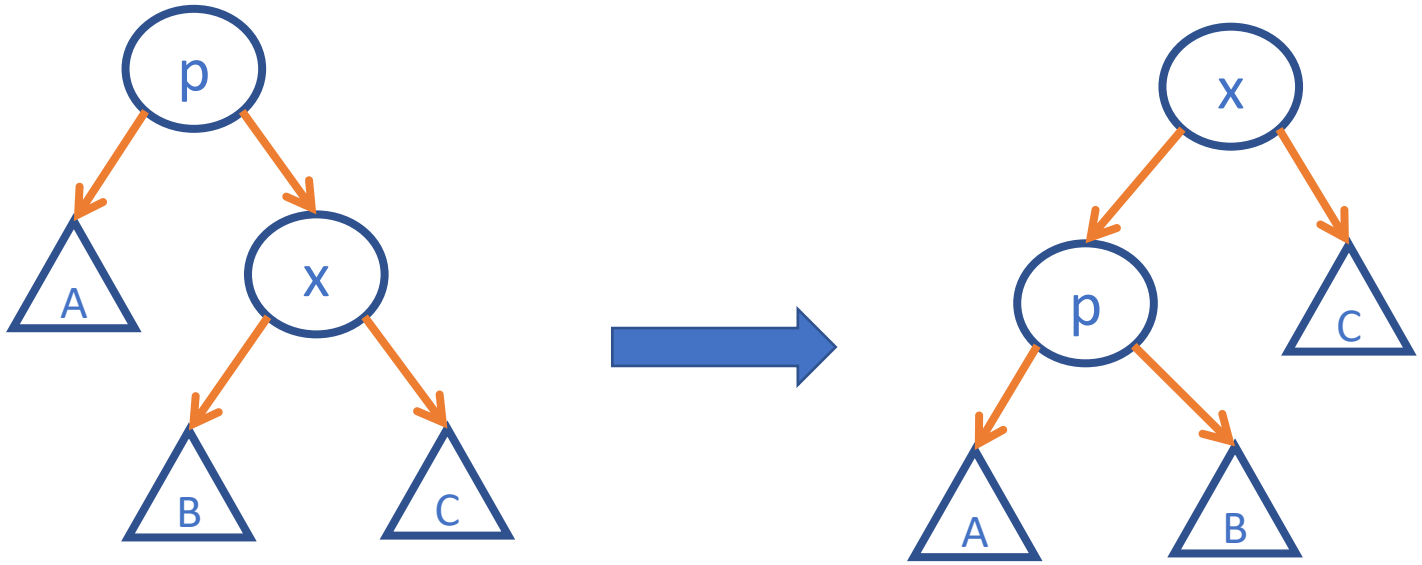
Splay Tree

Zig step : this step is done when *p* is the root. The tree is rotated on the edge between *x* and *p*.



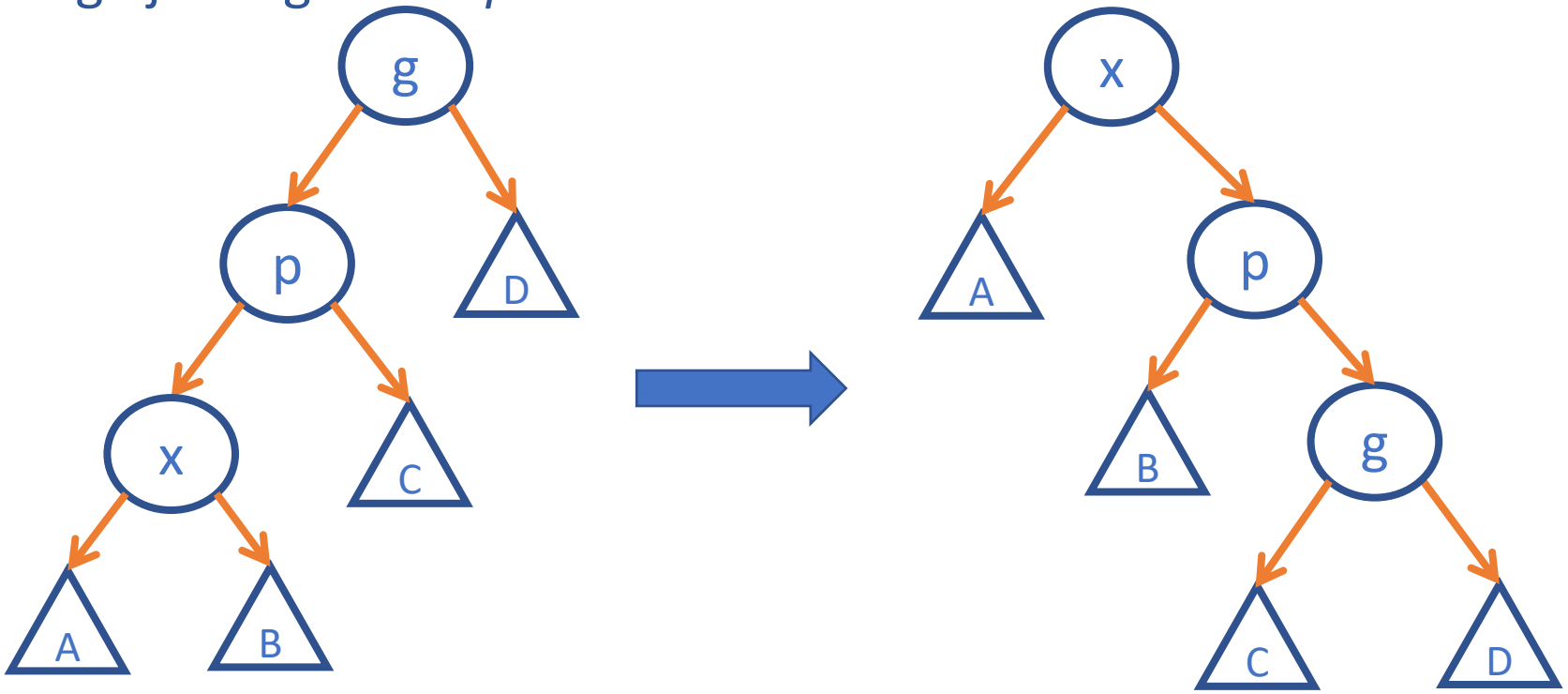
Splay Tree

Zig step : this step is done when *p* is the root. The tree is rotated on the edge between *x* and *p*.



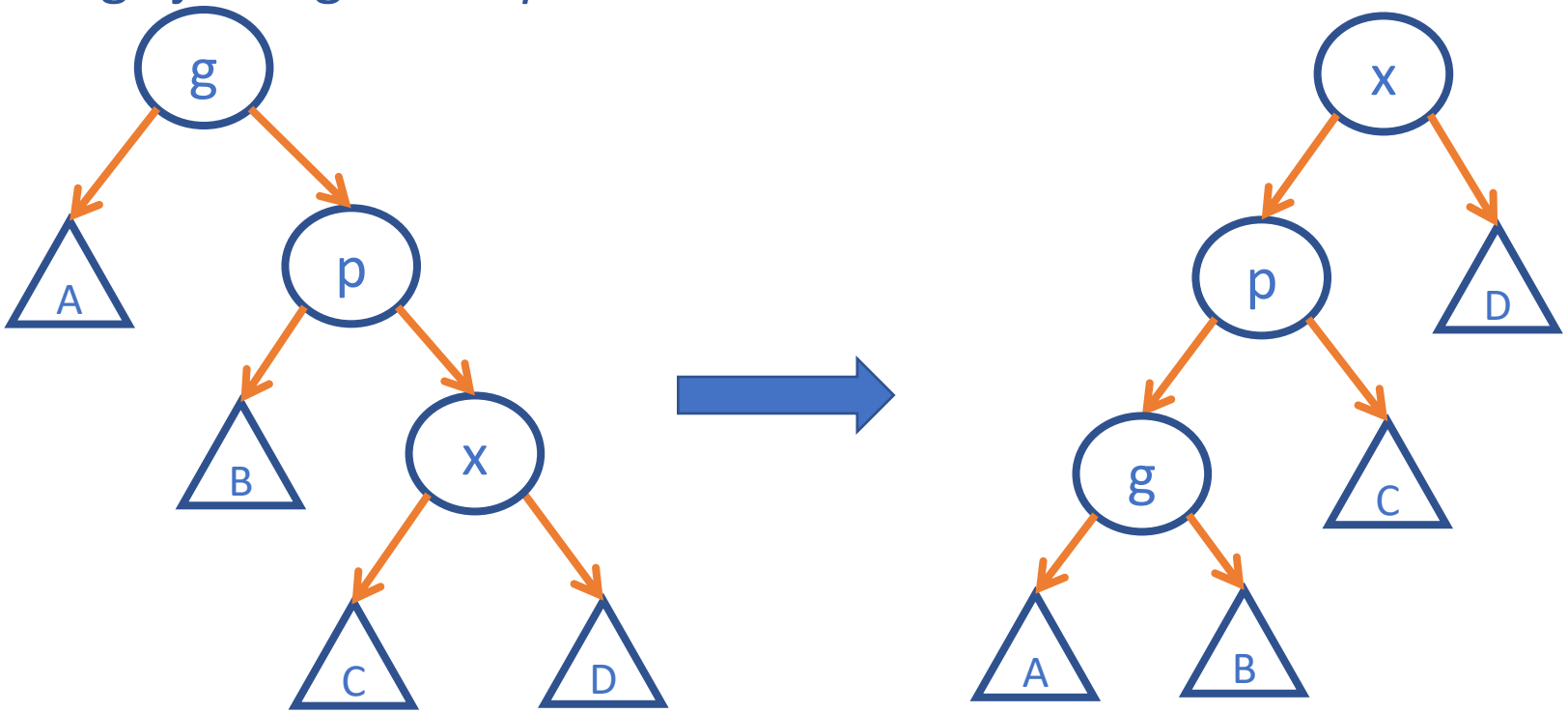
Splay Tree

Zig-Zig step : this step is done when *p* is not the root and *x* and *p* are either both right children or are both left children. The picture below shows the case where *x* and *p* are both **left** children. The tree is rotated on the edge joining *p* with *its* parent *g*, then rotated on the edge joining *x* with *p*.



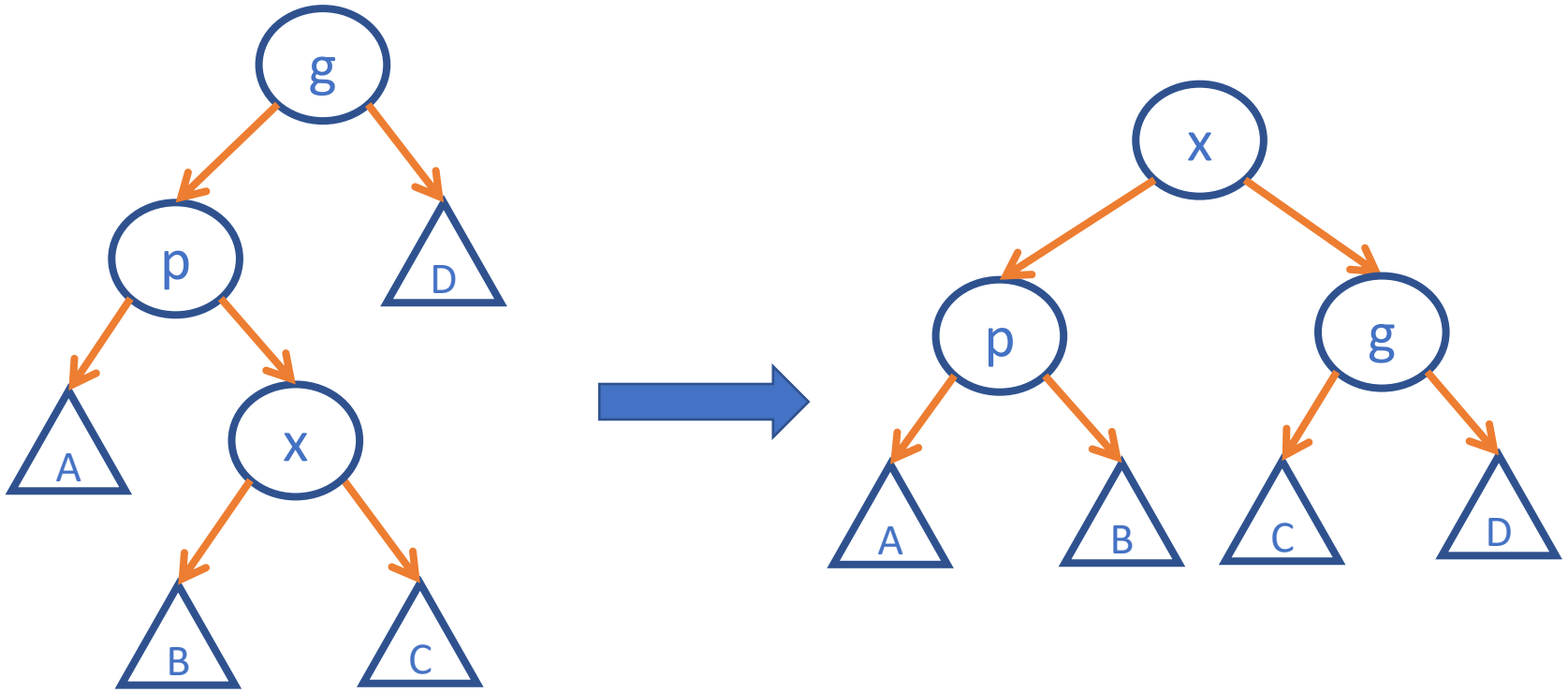
Splay Tree

Zig-Zig step : this step is done when *p* is not the root and *x* and *p* are either both right children or are both left children. The picture below shows the case where *x* and *p* are both **right** children. The tree is rotated on the edge joining *p* with *its* parent *g*, then rotated on the edge joining *x* with *p*.



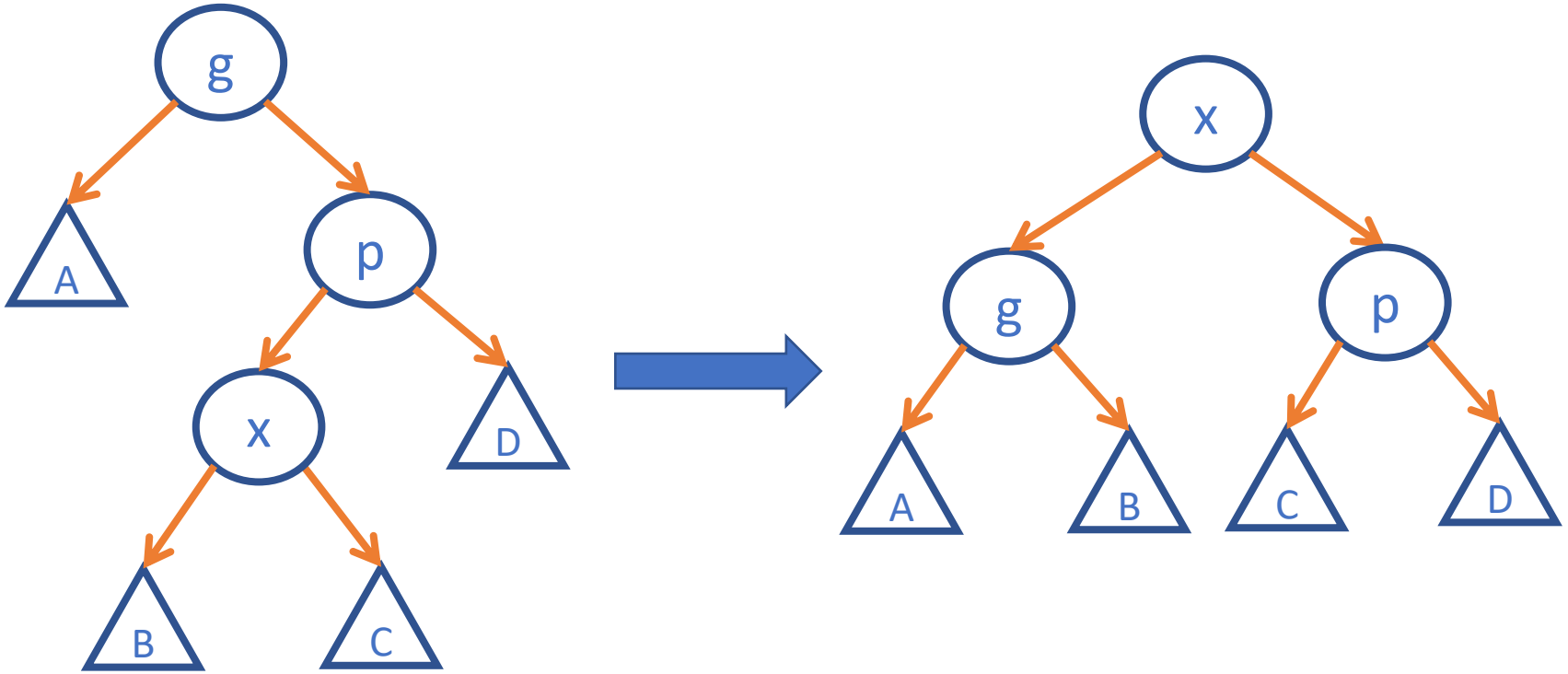
Splay Tree

Zig-Zag step : this step is done when *p* is not the root and *x* is a right child and *p* is a left child or vice versa (*x* is left, *p* is right). The tree is rotated on the edge between *p* and *x*, and then rotated on the resulting edge between *x* and *g*.



Splay Tree

Zig-Zag step : this step is done when *p* is not the root and *x* is a left child and *p* is a right child or vice versa (*x* is right, *p* is left). The tree is rotated on the edge between *p* and *x*, and then rotated on the resulting edge between *x* and *g*.



Splay Tree

Insertion

To insert a value x into a splay tree:

Insert x as with a normal binary search tree.

when an item is inserted, a splay is performed.

As a result, the newly inserted node x becomes the root of the tree.

Splay Tree

Deletion

To delete a node *x*, use the same method as with a binary search tree:
If *x* has two children:

- Swap its value with that of either the rightmost node of its left sub tree (its in-order predecessor) or the leftmost node of its right subtree (its in-order successor).
- Remove that node instead.

In this way, deletion is reduced to the problem of removing a node with 0 or 1 children.
Unlike a binary search tree, in a splay tree after deletion, we splay the parent of the removed node to the top of the tree.

References

- https://en.wikipedia.org/wiki/Splay_tree
- "Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019.

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE21CS252A

Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree

Kusuma K V
Department of Computer Science & Engineering

Splay Tree

Splaying

- There are three types of splay steps, each of which has two symmetric variants: left- and right-handed.
- Zig step: this step is done when p is the root.
- Zig - Zig step: this step is done when p is not the root and x and p are either both right children or are both left children.
- Zig - Zag step: this step is done when p is not the root and x is a right child and p is a left child or vice versa (x is left, p is right).

Splay Tree

Insertion

To insert a value x into a splay tree:

Insert x as with a normal binary search tree.

when an item is inserted, a splay is performed.

As a result, the newly inserted node x becomes the root of the tree.

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree Insertion

Insert 40, 30, 60, 10, 35, 50

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree Insertion

Insert 40, 30, 60, 10, 35, 50

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree Insertion

Insert 40, 30, 60, 10, 35, 50

References

- https://en.wikipedia.org/wiki/Splay_tree
- "Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019.

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE21CS252A

Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree

Kusuma K V
Department of Computer Science & Engineering

Splay Tree

Splaying

- There are three types of splay steps, each of which has two symmetric variants: left- and right-handed.
- Zig step: this step is done when p is the root.
- Zig - Zig step: this step is done when p is not the root and x and p are either both right children or are both left children.
- Zig - Zag step: this step is done when p is not the root and x is a right child and p is a left child or vice versa (x is left, p is right).

Splay Tree

Deletion

To delete a node *x*, use the same method as with a binary search tree:
If *x* has two children:

- Swap its value with that of either the rightmost node of its left sub tree (its in-order predecessor) or the leftmost node of its right subtree (its in-order successor).
- Remove that node instead.

In this way, deletion is reduced to the problem of removing a node with 0 or 1 children.
Unlike a binary search tree, in a splay tree after deletion, we splay the parent of the removed node to the top of the tree.

Splay Tree Deletion

Delete 40, 10, 35, 70

PES UNIVERSITY
CELEBRATING 50 YEARS

Splay Tree Search

Search 50, 20

References

- https://en.wikipedia.org/wiki/Splay_tree
- "Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019.

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS
UE19CS202

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Programs on Binary Trees

Shylaja S S
Department of Computer Science & Engineering

Programs on Binary Tree

```
//Returns the smallest element in Binary Search Tree
 int minimum(struct tnode *t)
 {
   while(t->left!=NULL)
    t=t->left;
   return(t->data);
 }
```

Programs on Binary Tree

```
//Returns the largest element in Binary Search Tree
 int maximum(struct tnode *t)
 {
   while(t->right!=NULL)
    t=t->right;
   return(t->data);
 }
```

Programs on Binary Tree

```
//Computes the height of a Binary Tree
int height(struct tnode *t)
 {
  if(t==NULL)
    return -1;
  if((t->left==NULL)&&(t->right==NULL))
    return 0;
  return (1+max(height(r->left),height(r->right)));
 }
```

Programs on Binary Tree

```
//Count the number of leaf nodes in a Binary Tree
int leafcount(struct tnode *t)
{
  if(t==NULL)
   return 0;
 if((t->left==NULL)&&(t->right==NULL))
   return 1;
 int l=leafcount(t->left);
 int r=leafcount(t->right);
 return(l+r);
}
```

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

+91 9449867804

Data Structures and its Applications

Dinesh Singh
Department of Computer Science & Engineering

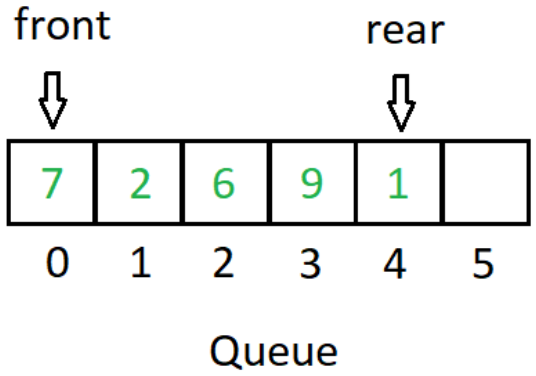
DATA STRUCTURES AND ITS APPLICATIONS

Basic Structure of a Queue , Implementation using an Array

Dinesh Singh
Department of Computer Science & Engineering

Queue Data Structure - definition

- A Queue is an ordered collection of items from which items may be deleted at one end (called the front of the queue)and into which items may be inserted at the other end (called the rear of the queue).



Data Structures and its Applications

Types Queue

- Different Types of Queues
  - Simple Queue
  - Circular Queue
  - Priority Queue
  - Dequeue

- Implementation
  - Sequential Representation (Arrays)
  - Linked Representation (Linked Lists)

Queue Operations

- Three primitive operations can be applied to the queue
- Insert (q,x) : inserts x at the rear of the queue q
- x = remove (q) : deletes front element from the queue and set x to its contents
- empty(q) : returns true or false depending on whether the queue contains any elements.
- Insert operation cannot be performed if the queue has reached the maximum size.
- The result of an illegal attempt to insert an element into a queue which has reached its maximum size is called overflow.
- The remove operation can be applied only if the queue is non empty.
- The result of an illegal attempt to remove an element from an empty queue is called underflow.
- The empty operation is always applicable.

Structure of a Simple Queue – Sequential Representation

```
#define MAXQUEUE 100
struct queue
{
  int items [MAXQUEUE];
  int front, rear;
} ;

struct queue q;
q.rear = q.front = -1;
```

Functions to implement the operations

- insert (x, &q)
- remove (&q)
- empty(&q)

Simple Queue – Insert Operation

Inserting into a queue

1. Check the queue for overflow condition
2. If the queue is not in overflow condition, increment the rear pointer and insert the element at a location indicated by the rear pointer.
3. If this is the first element in the queue, initialise front to 0.
4. Return 1

Simple Queue – Implementation of Insert

```
int insert(int x,struct queue *q)
 {
  //check queue overflow
 if(q->rear==MAXQUEUE - 1)
 {
  printf("Queue overflow..\n");
  return -1;
 }
 (q->rear)++;
 q->items[q->rear]=x;
 if(q->front==-1)//if first element
   q->front=0;
 return 1;
}
```

Simple Queue – Remove Operation

Removing element from the queue

1. Check the queue for underflow condition
2. If the queue is not in underflow condition, remove the element pointed by the front pointer into x.
3. If this was the only element in the queue, initialise front and rear to -1.
4. Return x

Simple Queue – Implementation of remove

```
int remove(struct queue *q)
 {
  int x;

  if(q->front==-1)
  {
    printf("Queue empty..\n");//underflow
    return -1;
  }
   x=q->items[q->front];
   if(q->front==q->rear)//only one element in queue
    q->front=q->rear=-1;
   else
    (q->front)++;
   return x;
 }
```

Data Structures and its Applications

Simple Queue – Implementation of empty and display

```
int empty ( struct queue * q)
{
  if (q->front ==-1)
    return 1;
  return -1;
}
display (struct queue * q)
{
 int i;
 if(q->front==-1)
   printf(”Empty queue..\n”)
 else
 {
   for ( i = q->front; i<=q->rear;i++)
     printf(”%d”,q->items[i]);
 }
}
```

Simple Queue – Another Implementation

PES UNIVERSITY
CELEBRATING 50 YEARS

Implementation where queue represented as an array , front and rear are separate variables

```
#define MAXQUEUE 100

int q[MAXQUEUE]
int front, rear
front = rear = -1
```

Function calls to implement the operations

- insert(x, q, &front, &rear) ;
- remove (q , &front, &rear);
- empty(q, &front)

Simple Queue – Another Implementation

Implementation where queue represented as an array , front and rear are separate variables

```
int insert(int x,int *q,int *f,int *r)
 {
  //check queue overflow
 if(*r==MAXQUEUE - 1)
 {
  printf("Queue overflow..\n");
  return -1;
 }
 (*r)++;
 q[*r]=x;
 if(*f==-1)//if first element
   *f=0;
 return 1;
}
```

Simple Queue – Another Implementation

Implementation where queue represented as an array , front and rear are separate variables

```
int remove(int *q,int *f,int *r)
 {
  int x;
  if(*f==-1)
  {
    printf("Queue empty..\n");
    return -1;
  }
   x=q[*f];
   if(*f==*r)//only one element in queue
     *f=*r=-1;
   else
     (*f)++;
   return x;
 }
```

Simple Queue – Another Implementation

Implementation where queue represented as an array , front and rear are separate variables

```
display (int *q, int f, int r)
{
  int i;
  if(f==-1)
    printf(”Empty queue..\n”)
  else
  {
    for ( i = f; i<= r; i++)
      printf(”%d”,q[i]);
  }
}
```

THANK YOU

Dinesh Singh
Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

Data Structures and its Applications

Dinesh Singh
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

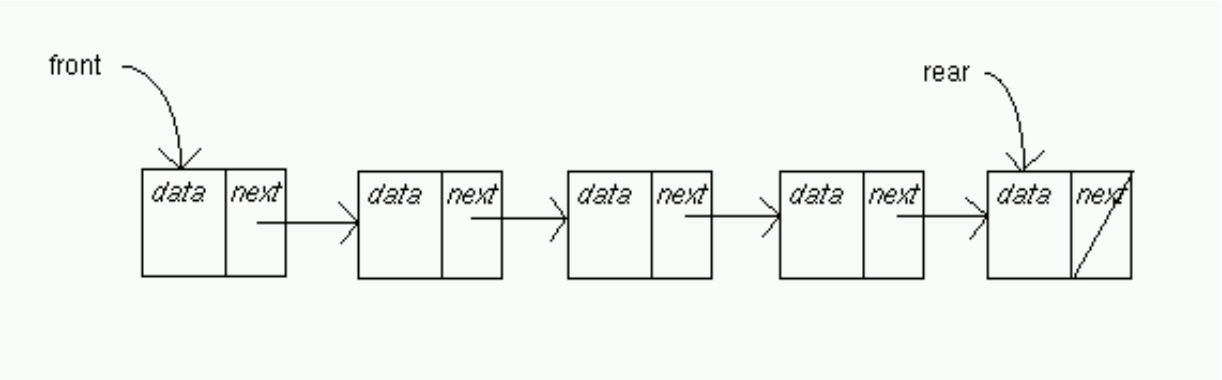
Queues – Linked List Implementation

Dinesh Singh
Department of Computer Science & Engineering

Queues - Linked list Implementation

In a linked list implementation two pointers are maintained : front and rear .

- **front points to the first item of the queue**
- **rear points to the last item of the queue**



Queues - Linked list Implementation

PES UNIVERSITY
CELEBRATING 50 YEARS

Operations :

- Insert() : adds a new node after the rear and moves rear to the next node
- Remove() : removes the first node and moves front to the next node
- Empty() : Checks if the queue is empty

Queues - Linked list Implementation

Structure of queue

```
struct node
{
  int data;
  struct node *next;
};
struct queue
{
   struct node * front;
   struct node *rear;
};

Struct queue q;
q.front=q.rear = NULL;
```

Queues - Linked list Implementation – Operations

Insert operation

```
Insert(q,x)

p=getnode();
initialise the node
if(q.rear=NULL)
    q.front=p;
else
   next(q.rear) =p;
q.rear = p;
```

remove operation

```
remove(q)

If(empty(q)
   print empty queue
else
  p=q.front;
  x=info(p);
  q.front = next(p);
  if(q.front =NULL)
     q,rear=NULL
  freenode(p);
  return x;
```

Data Structures and its Applications

Queues - Linked list Implementation – Operations

Insert operation of queue implemented by a linked list

```
void qinsert(struct node * q, int x)
  {
    struct node *temp;

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->next=NULL;

     //if this is the first node
    if(q->front==NULL)
      q->front=q->rear=temp;
    else //insert at the end
      {
         q->rear->next=temp;
        q->rear=temp;
       }
  }
```

Queues - Linked list Implementation – Operations

remove operation of a queue implemented by a linked list

```
int qremove(struct queue * q)
  {
      struct node *p;
      int x;
      p=q->front;
      if(p==NULL)
       {
        printf("Empty queue\n");
        return -1;
       }
```

Queues - Linked list Implementation – Operations

```
else
        {
         x=q->data;
         if(q->front==q->rear) //only one node
           q->front=q->rear=NULL;
         else
         {
         q->front=q->next; // move front to next node
         return x;
         }
        free(q);
       }
  }
```

Queues - Linked list Implementation – Operations

```
void qdisplay(struct queue q)
    {
      struct node * f, *r;
      if(q.front==NULL)
        printf("Queue Empty\n");
     else
      {
      f=q.front; r=q.rear;
      while(f!=r)
      {
        printf("%d-> ",f->data);
        f=f->next;
      }
     printf("%d-> ",f->data); // print the last node
     }
   }
```

Queues - Linked list Implementation – Operations

PES UNIVERSITY
CELEBRATING 50 YEARS

Insert operation in an alternate way

```
void qinsert(int x, struct node **f, struct node **r)
//f and r are pointers to variables front and rear of a queue
   {
    struct node *temp;

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->next=NULL;

     //if this is the first node
    if(*f==NULL)
      *f=*r=temp;
    else //insert at the end
      {
       (*r)->next=temp;
        *r=temp;
      }
  }
```

Queues - Linked list Implementation – Operations

Remove operation in an alternate way

```
int qdelete(struct node **f, struct node **r)
  {
      struct node *q;
      int x;
      q=*f;
      if(q==NULL)
       {
        printf("Empty queue\n");
        return -1;
       }
else
      {
       x=q->data;
       if(*f==*r) //only one node
         *f=*r=NULL;
       else
       {
       *f=q->next;
       return x;
       }
      free(q);
     }
}
```

Queues - Linked list Implementation – Operations

Disadvantages of representing queue by a linked list

- A node in linked list occupies more storage than the corresponding element in an array.
- Two pieces of information per element is necessary in a list node, where as only one piece of information is needed in an array implementation

THANK YOU

Dinesh Singh
Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

Data Structures and its Applications

Dinesh Singh
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Circular Queue - Implementation

Dinesh Singh
Department of Computer Science & Engineering

Circular Queues - definition

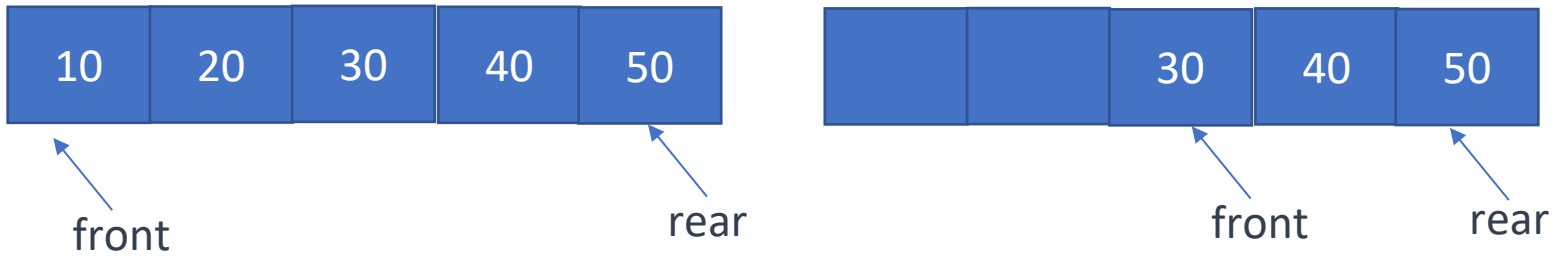
- Circular Queue is a linear data structure, which follows the principle of FIFO(First In First Out), but instead of ending the queue at the last position, it again starts from the first position after the last, hence making the queue behave like a circular data structure.

- In a simple queue, once the queue is completely full, it's not possible to insert more elements. Even if we perform remove operation on the queue to remove some of the elements, until the queue is reset, no new elements can be inserted

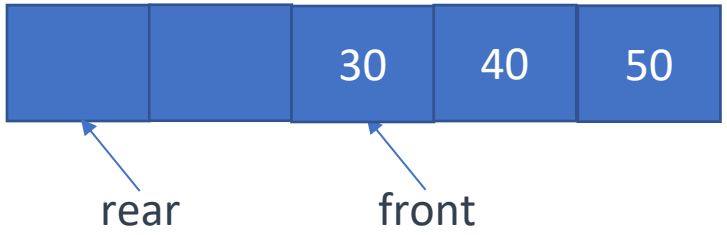
Data Structures and its Applications

Drawback of a simple Queue

Structure of the simple queue



Cannot insert even after two elements are removed and Space available in the front.



It is possible to insert in a circular queue by moving the rear To the beginning of the queue

Data Structures and its Applications

Implementation of operations

- To insert into the queue : Finding the rear index

```
rear = (rear + 1) % size
If(rear=front)
   cannot insert
else
   insert at rear index
```

- For eg. If size = 5, front = 2 and rear = 4
- rear = (4 + 1) % 5 = 0,
- The new element gets inserted at index 0

- For eg. If size = 5, front = 0 and rear = 4
- rear = (4 + 1) % 5 = 0,
- rear = front , therefore cannot insert

Implementation of operations

- To remove from the queue :

  remove the element pointed by front , move the front

  front = (front + 1) % size

For eg. If size = 5, front = 2 and rear = 4

- front = (2 + 1) % 5 = 3,
- front moves to index 3 after removal of the element,

- For eg. If size = 5, front = 4 and rear = 2
- front = (4 + 1) % 5 = 0,
- Front moves to 0 after removal of the element

Structure of a circular Queue – Sequential Representation

```
#define MAXQUEUE 100
struct queue
{
  int items [MAXQUEUE];
  int front, rear;
} ;

struct queue q;
q.rear = q.front = -1;
```

Functions to implement the operations

- insert (&q,x)
- remove (&q)

Implementation of operations - insert

```
int qinsert(struct queue *q, int x)
{

 //check for queue overflow
      if((q->r+1)%MAXQUEUE==q->f)
      {
           printf("Queue Overflow..\n");
            return -1;
       }
      else
       {
            q->rear=(q->rear+1)%size;     //get the rear index
            q->item[q->rear]=x;    //insert at rear index
            if(q->front==-1)  //if first element
              q->front=0;     // make front point to 0
            return 1;
             }
    }
```

Implementation of operations - insert

```
//ANOTHER WAY TO IMPLEMENT INSERT
int qinsert(int *q,int *f, int *r, int size, int x)
{
 // f ,r are pointers to front and rear of the queue, size is the max size of queue
 //check for queue overflow
      if((*r+1)%size==*f)
      {
           printf("Queue Overflow..\n");
            return -1;
       }
      else
       {
            *r=(*r+1)%size;     //get the rear index
            q[*r]=x;    //insert at rear index
            if(*f==-1)  //if first element
              *f=0;     // make front point to 0
            return 1;
             }
   }
```

Data Structures and its Applications

Implementation of operations - remove

```
int remove(struct queue *q)
 {
   int x;
  if(q->front==-1)  //check for empty queue
  {
   printf("Queue empty..\n");
   return -1;
  }
else
  {
    x=q->items[q->front];
    if(q->front==q->rear)//only one element
      q->front=q->rear=-1;
 else
      q->frontf=(q->front+1)%MAXQUEUE;   //increment the front
    return x;
   }
}
```

Data Structures and its Applications

Implementation of operations - remove

```
//ANOTHER WAY TO IMPLEMENT REMOVE
int remove(int *q, int *f, int *r,int size)
 {
   int x;
  if(*f==-1)  //check for empty queue
  {
   printf("Queue empty..\n");
   return -1;
  }
else
  {
    x=q[*f];
    if(*f==*r)//only one element
      *f=*r=-1;
 else
      *f=(*f+1)%size;   //increment the front
    return x;
   }
}
```

Implementation of operations - display

```
void display(struct queue q)
 {
  if(q.front==-1)
    printf("\nQueue empty..\n");
  else
  {
    while(q.front!=q.rear) //increment front till it reaches rear
    {
      printf("%d ",q.items[q.front]);
      q.front=(q.front+1)%MAXQUEUE;
    }
   printf("%d ",q->items[q->front]); // display the last element
  }
}
```

Implementation of operations - display

```
void display(int *q, int f, int r, int size)
 {
  if(f==-1)
    printf("\nQueue empty..\n");
  else
  {
    while(f!=r) //increment front till it reaches rear
    {
      printf("%d ",q[f]);
      f=(f+1)%size;
    }
   printf("%d ",q[f]); // display the last element
  }
}
```

THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Priority Queue - Implementation

Dinesh Singh
Department of Computer Science & Engineering

Priority Queues - definition

- Priority Queue is an extension of queue where every item has a priority associated with it.
- There are two types of priority queue
  - Ascending priority queue
  - Descending Priority queue
- In Ascending Priority queue ,the item with lowest priority is removed
- In descending priority queue, the item with the highest priority is removed.

Priority queues - Operations

- Operations in Ascending priority queue
  - pqinsert(apq,x) : inserts element x into the priority queue apq.
  - pqmindelete(apq) : removes the element with the minimum priority.
- Operations in Descending priority queue
  - pqinsert(dpq,x) : inserts element x into the priority queue dpq.
  - pqmaxdelete(dpq) : removes the element with the maximum priority.
- The operation empty(pq) determines whether the queue is empty or not.

Priority Queues- alternate definition

- In alternate definition of a priority queue, the items do not have a priority but the intrinsic ordering of elements determine results of its basic operation.
- In ascending priority queue, items can be inserted arbitrarily, but only the smallest element can be removed.
- In descending priority queue, items can be inserted arbitrarily, but only the largest element can be removed.

Priority Queues- Implementation

- Priority queues can be implemented by
  - Arrays
  - Linked Lists
  - Heap

Priority Queues- Array Implementation

```
struct pqueue
{
  int  data;
  int pty;
}
struct pqueue pq[10];
```

implementation of descending priority :

Items inserted based on their priority

- Insert : The item is inserted based on its priority. The queue is ordered in the descending priority of the items. The item with the highest priority will be at the first location in the array.

- Remove : delete always the first element, Shift the other elements to the left after the deletion

Priority Queues- Array Implementation

implementation of descending priority :

Items inserted arbitrarily

- Insert : The item is inserted at the rear of the queue. The queue is therefore unordered.
- Delete : The item with the highest priority is found and removed. The items following the position of the removed element are shifted to the left.

Priority Queues- Array Implementation

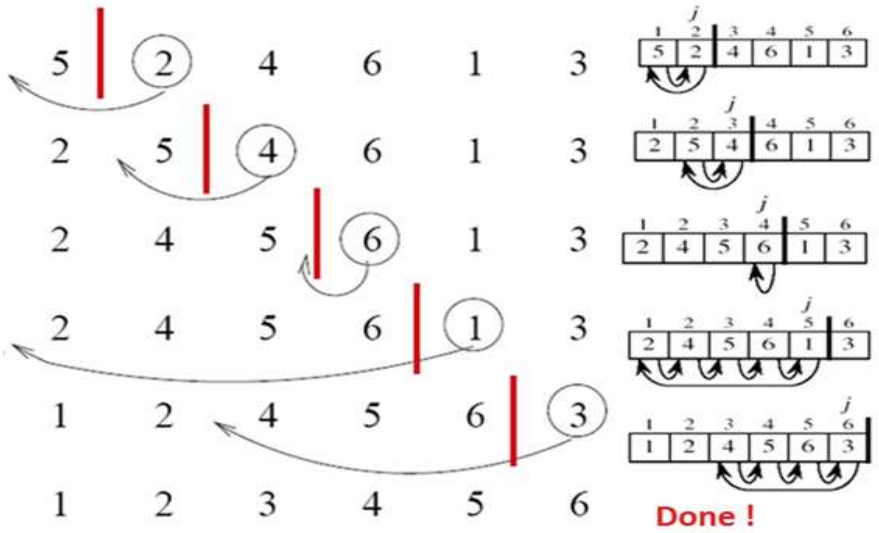
implementation of descending priority where items are inserted according to the priority

```
void pqinsert(int x,int pty,struct pqueue *pq,int *count)
{
 // x is item to be inserted
 // pty is the priority of the item
// pq is the pointer to the priority queue
// count is the number of items in the queue

    int j;
    struct pqueue key;
    key. data=x;
    key.pty=pty;
```

Priority Queues- Array Implementation

implementation of descending priority where items are inserted according to the priority

```
j=*count-1; // index of the initial position of the element

//compare the priority of the item being inserted with the
//priority of the items in the queue
// shift the items down while the priority of the item being
//inserted is greater than priority of the item in the queue

while((j>=0)&&(pq[j].pty>key.pty))
     {
          pq[j+1]=pq[j];
           j--;
      }
      pq[j+1]=key; // insert the element at its correct location
      (*count)++;
 }
```



Priority Queues- Array Implementation

PES UNIVERSITY
CELEBRATING 50 YEARS

implementation of descending priority where items are inserted according to the priority

```
struct pqueue pqdelete(struct pqueue *pq,int *count)
{
    // pq is a pointer to the priority queue
    // count is the number of elements in the queue

     int i;
     struct pqueue key;
     // if queue is empty, return a structure with priority -1

     if(*count==0)
       {
           key.data=0;
           key.pty=-1;
       }
```

Priority Queues- Array Implementation

PES UNIVERSITY
CELEBRATING 50 YEARS

implementation of ascending priority where items are inserted according to the priority

```
    //delete the first item
    //shift the other items to the left
     else
     {
          key=pq[0];
          for(i=1;i<=*count-1;i++)
               pq[i-1]=pq[i];
          (*count)--;
     }
     return key;  //return the key with the lowest priority
}
```

Priority Queues- Array Implementation

PES UNIVERSITY
CELEBRATING 50 YEARS

Implement the following

- Ascending Priority queue where item is inserted at the end and item with the lowest priority is deleted.
- Descending priority queue
- Ascending and Descending Priority queue using a linked list

THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

Data Structures and its Applications

Dinesh Singh
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Dequeue - Implementation

Dinesh Singh
Department of Computer Science & Engineering

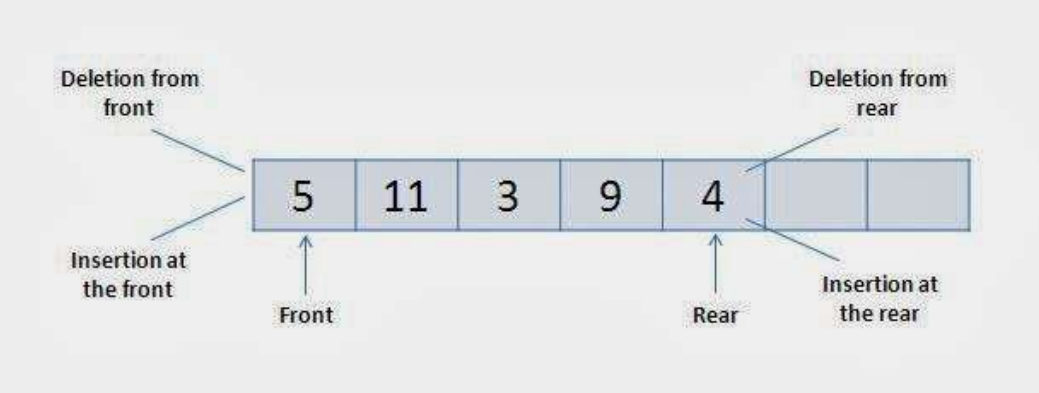
Dequeue(Double ended Queue) - definition

Double ended queue is a queue that allows insertion and deletion at both ends.

Dequeue(Double ended Queue) - definition

The following four basic operations are performed on dequeue:

- *insertFront()*: Adds an item at the front of Deque.
- *insertRear()*: Adds an item at the rear of Deque.
- *deleteFront()*: Deletes an item from front of Deque.
- *deleteRear()*: Deletes an item from rear of Deque.

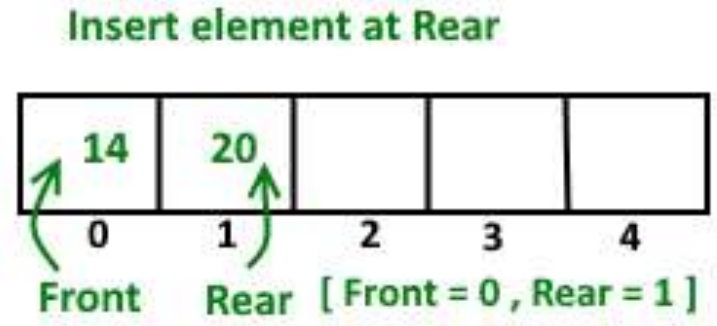


Data Structures and its Applications

Dequeue(Double ended Queue) - Array Implementation

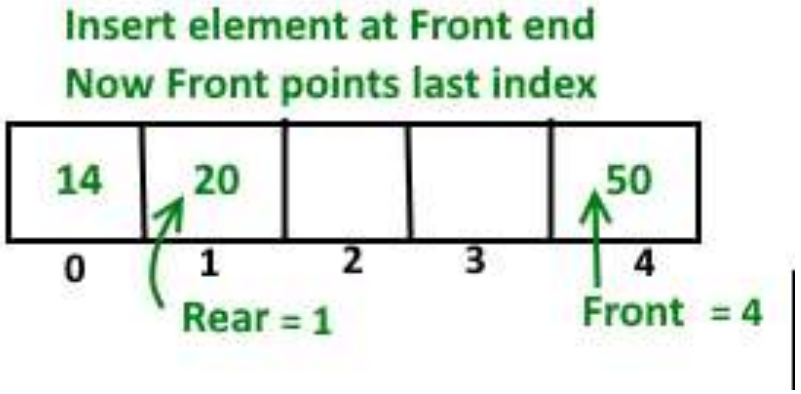
Insert Elements at Rear end :

Check whether the queue is full
If rear = size-1
 initialise rear to 0.
else
  increment rear by 1
 insert element at location rear



Insert element front end

Check if the queue is full
If Front =0
 move front to last location (size -1)
else
 decrement front by 1
 insert at location front



Data Structures and its Applications

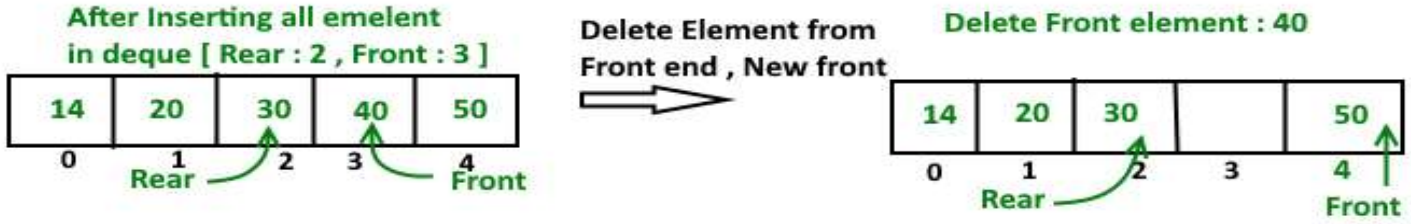
Dequeue(Double ended Queue) - Array Implementation

Delete element at Rear end

check if the queue is empty
delete the element pointed by rear
If dequeue has one element
  front=-1 rear=-1;
If rear is at first index
  make rear = size-1
else
  decrease rear by 1

Delete element at front end

check if the queue is empty
delete the element pointed by front
If dequeue has one element
  front=-1 rear=-1;
If front is at last index
  make front = 0
else
  increase front by 1



Dequeue(Double ended Queue) - Doubly Linked list Implementation

Structure of Dequeue

```
struct dequeue
{
   struct node * front;
   struct node * rear;
};
struct node
{
    int data;
    struct node * prev, *next;
};
struct   dequeue dq;

dq.front=dq.rear = NULL
```

Data Structures and its Applications

Dequeue(Double ended Queue) - - Doubly Linked list Implementation

```
//insert in front of the queue
void qinsert_head(int x,struct dequeue *dq)
  {
    struct node *temp;

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->prev=temp->next=NULL;

    if(dq->front==NULL) // first element
      dq->front=dq->rear=temp;
     else
      {
        temp->next=dq->front;  // insert in front
        dq->front->prev=temp;
        temp->prev=NULL;
        dq->front=temp;
      }
  }
```

Data Structures and its Applications

Dequeue(Double ended Queue) - - Doubly Linked list Implementation

```
//insert at the rear of the queue
void qinsert_tail(int x,struct dequeue* dq)
 {
  struct node *temp;

   temp=(struct node*)malloc(sizeof(struct node));
   temp->data=x;
   temp->prev=temp->next=NULL;

   if(dq->front==NULL)
     dq->front=dq->rear=temp;
    else
     {
       dq->rear->next=temp;
       temp->prev=dq->rear;
       dq->rear=temp;
     }
 }
```

Data Structures and its Applications

Dequeue(Double ended Queue) - - Doubly Linked list Implementation

```
//delete at the front of the queue
int qdelete_head(struct dequeue* dq)
 {
   struct node *q;
   int x;
   if(dq->front==NULL)
     return -1;

  q=dq->front;
   x=q->data;
  if(dq->front==dq->rear)//only one node
    dq->front=dq->rear=NULL;
  else
   {
     dq->front=dq->front->next;
     dq->front->prev=NULL;
   }
   free(q);
  return x;
 }
```

Data Structures and its Applications

Dequeue(Double ended Queue) - - Doubly Linked list Implementation

```
//delete at the rear of the queue
int  qdelete_tail(struct dequeue* dq)
 {
  struct node *q;
   int x;
   if(dq->front==NULL)
     return -1;
   q=dq->rear;
   x=q->data;
  if(dq->front==dq->rear)//only one node
    dq->front=dq->rear=NULL;
  else
   {
     dq->rear=dq->rear->prev;
     dq->rear->next=NULL;
    }
   free(q);
    return x;
 }
```

THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

Data Structures and its Applications

Kundhavai K R
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Queues – Implementation of Josephus Problem

Kundhavai K R
Department of Computer Science & Engineering

Queues – Implementation of Josephus Problem

- Josephus Problem : Postulates a group of soldiers surrounded by an overwhelming enemy force. There is no hope of victory without reinforcements. There is one horse available for escape

- The soldiers agree to a pact to determine which of them is to escape and seek help. The soldiers form a circle and a number n is picked from a hat. One of the names is also picked from the hat.

- Beginning with the soldier whose name is picked , they begin to count clockwise around the circle. when the count reaches n, that soldier is removed from the circle and the count begins with the next soldier.

- The process continues so that each time the count reaches n, another soldier is removed from the circle. Any soldier removed from the circle is no longer counted. The last soldier remaining is to take the horse and escape.

Queues – Implementation of Josephus Problem

- The input to the program is the number n and list of names, which is the clockwise ordering of the circle, beginning with the soldier from whom the count is to start.
- The program should print the names in the order that they are eliminated and the name of soldier who esapes.
- For example if n=3 and that there are five solders named A,B,C,D and E. We count three soldiers starting at A, so that C is eliminated first.
- We then begin at D and count D E and back to A. A is eliminated. Then we count B D and E, E is eliminated. And finally B D and B is eliminated.
- D is the one who escapes

Queues – Implementation of Josephus Problem

- Data structure used is a circular list where each node represents one soldier
- To represent the removal of a soldier form the circle, a node is deleted from the circular list.
- Finally one node remains on the list and the result is determined

Queues – Implementation of Josephus Problem

Pseudo code of implementation using circular list

```
read(n)
read(name)
while(all the names are read)
{
        insert name on the circular list
        read(name)
}
while(there is more than one node on the list)
{
        count through n-1 nodes on the list
        print name in the nth node
        delete the nth node
}
print the name of the only node on the list
```

Queues – Implementation of Josephus Problem

Code of implementation using circular list

```
int survivor(struct node **head, int n)
{
 // head is pointer to first node

   struct node *p, *q;
   int i;
   q = p = *head;
   while (p->next != p)
      {
         for (i = 0; i < n - 1; i++)
            {
               q = p;
               p = p->next;
            }
               q->next = p->next;
                printf("%d has been killed.\n", p->num);
                free(p);
                p = q->next;
      }
   *head = p;
   return (p->num);
}
```

Queues – Implementation of Josephus Problem

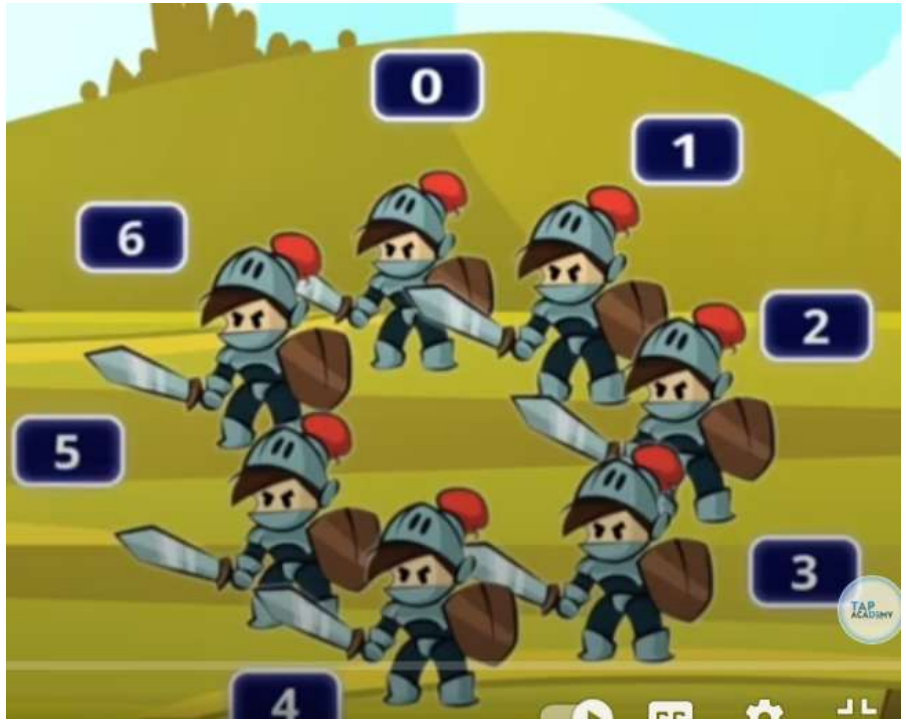
PES UNIVERSITY
CELEBRATING 50 YEARS

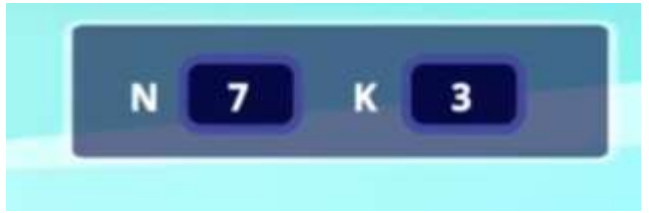
Pseudo code of implementation using circular queue

```
Enter n
while(all the names are read)
{
  insert name into the queue
  read(name)
}
while( q has one element)
{
   dequeue n-1 names from the queue and enqueue it.
   dequeue the n^th name
   print the n^th name
 }
 dequeue the only name of the queue
 print the name
```

Data Structures and its Applications

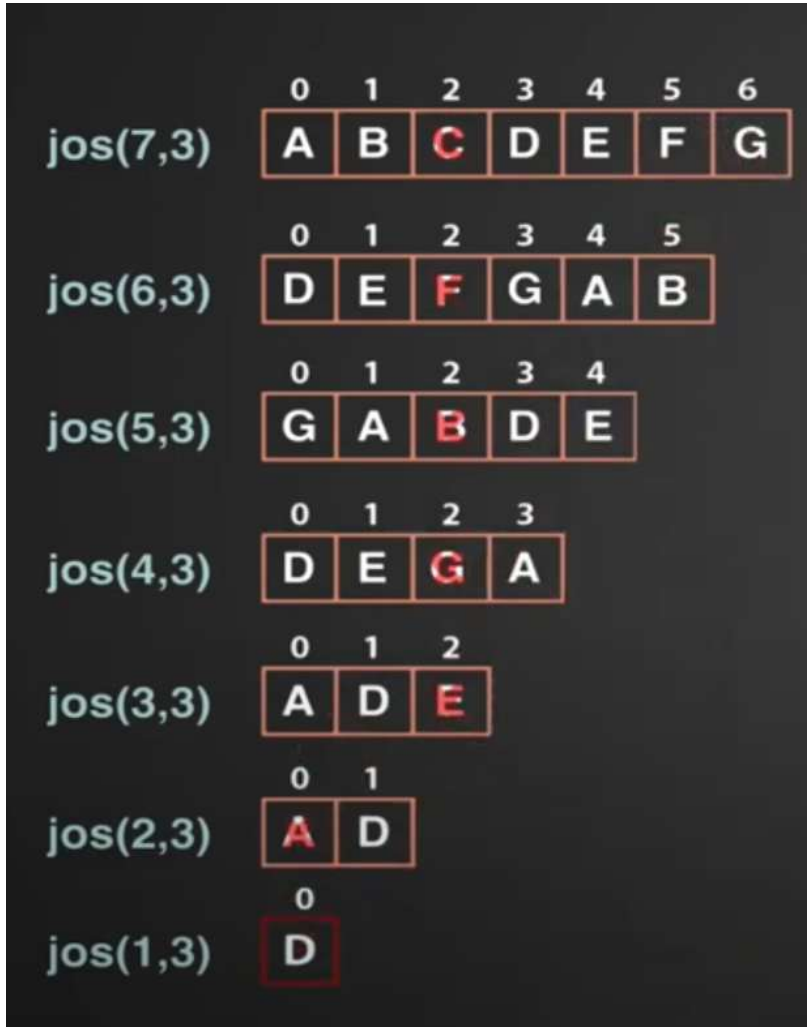
Queues – Implementation of Josephus Problem Recursively

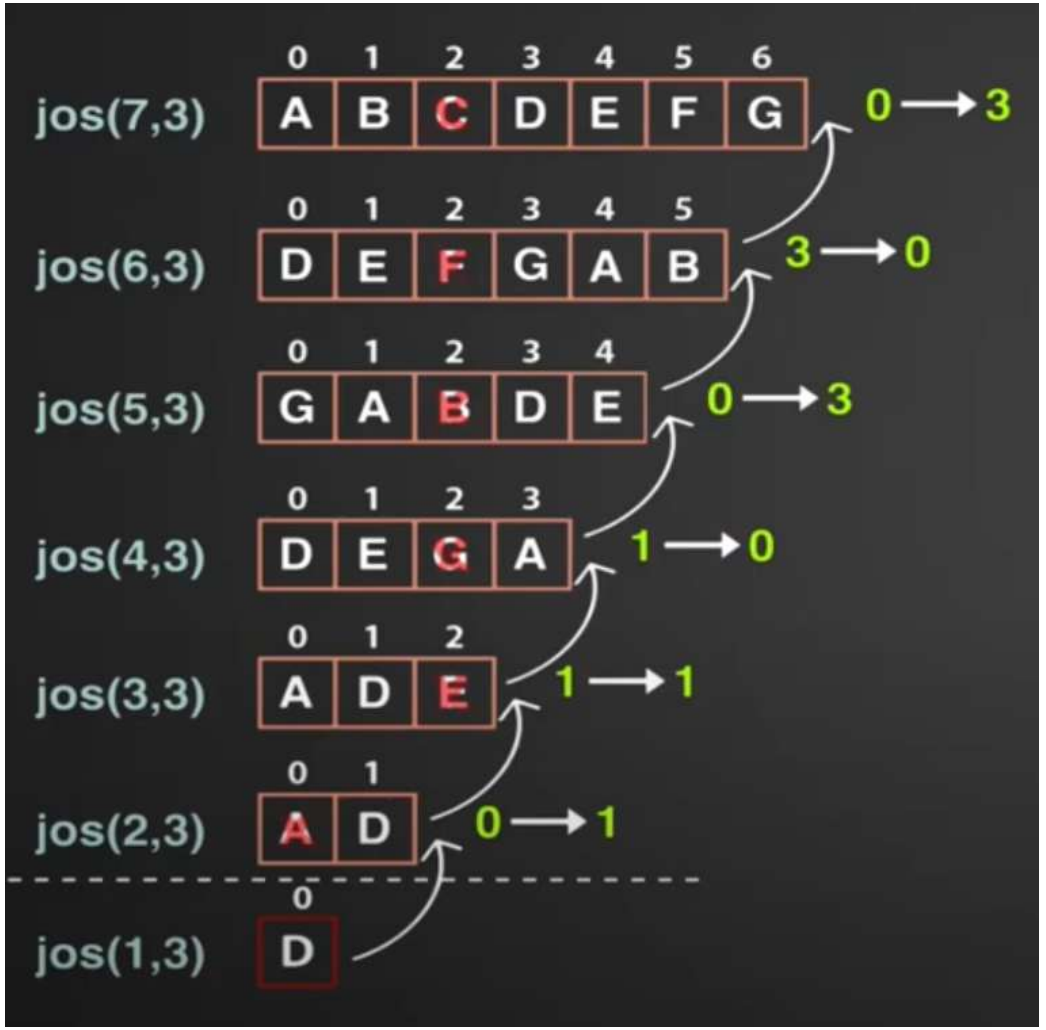


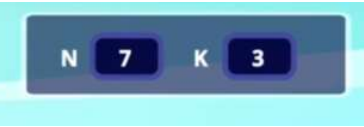


Data Structures and its Applications

Queues – Implementation of Josephus Problem

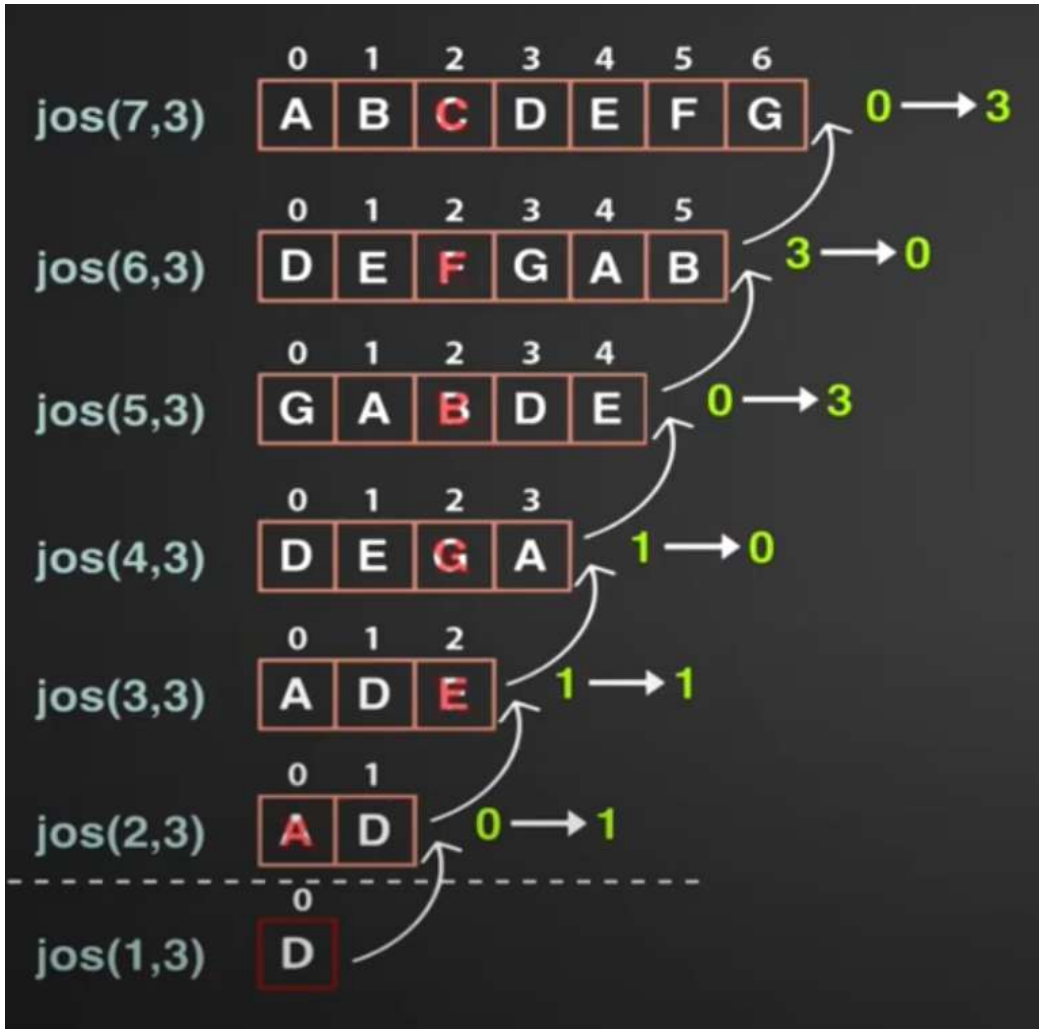


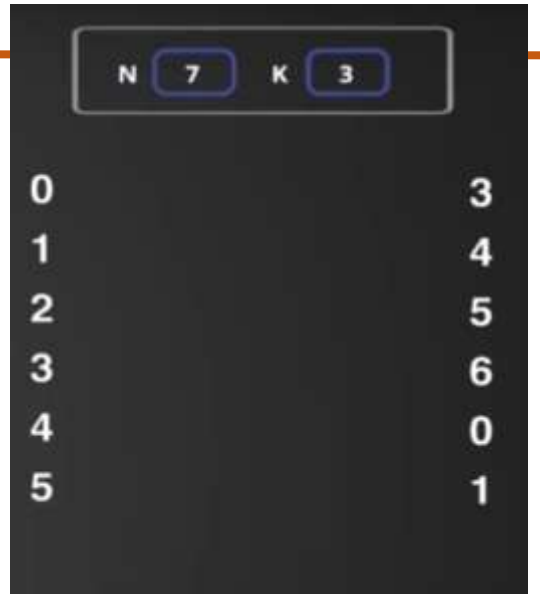


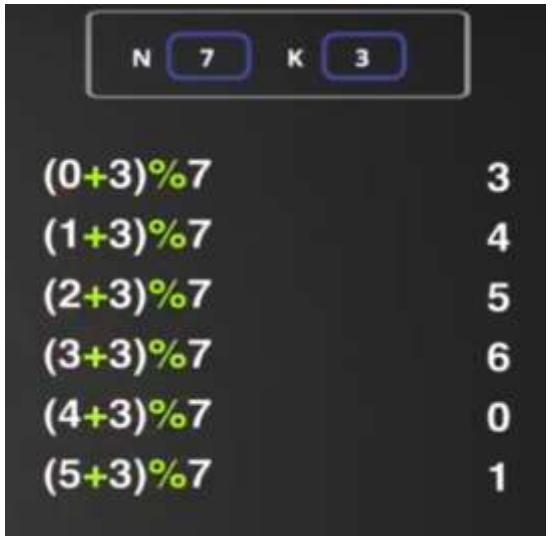


Data Structures and its Applications

Queues – Implementation of Josephus Problem







$$(jos(6,3)+k) \% n$$

$$jos(n,k) = (jos(n-1,k) + k) \% n$$

Data Structures and its Applications

Queues – Implementation of Josephus Problem

PES UNIVERSITY
CELEBRATING 50 YEARS

Assignment :

Implement the Josephus by using circular queue

Implement Josephus Problem by using recursion

THANK YOU

Kundhavai K R

Department of Computer Science & Engineering

Data Structures and its Applications

Dinesh Singh
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

CPU scheduling using Queue

Dinesh Singh
Department of Computer Science & Engineering

CPU scheduling using queue

Different Scheduling Algorithms:

First Come First Serve CPU Scheduling:

- Simplest scheduling algorithm that schedules according to arrival times of processes.
- First come first serve scheduling algorithm states that the process that requests the CPU first is allocated the CPU.
- It is implemented by using the simple queue. When a process enters the ready queue, its PCB is linked onto the rear of the queue.
- When the CPU is free, it is allocated to the process at the front of the queue.
- The running process is then removed from the queue.

CPU scheduling using queue

Shortest Job First(Preemptive):

- In Preemptive Shortest Job First Scheduling, jobs are put into the ready queue as they arrive
- As a process with short burst time arrives, the existing process is preempted or removed from execution, and the shorter job is executed first

Shortest Job First(Non-Preemptive):

- In Non-Preemptive Shortest Job First, a process which has the shortest burst time is scheduled first.
- If two processes have the same bust time then FCFS is used to break the tie

CPU scheduling using queue

Longest Job First(Preemptive):

It is similar to an Shortest Job First scheduling(SJF) algorithm.

In this scheduling algorithm, priority is given to the process having the largest burst time remaining.

Longest Job First(Non-Preemptive):

- It is similar to an SJF scheduling algorithm. But, in this scheduling algorithm, priority is given to the process having the longest burst time.
- This is non-preemptive in nature i.e., when any process starts executing, can’t be interrupted before complete execution.

Data Structures and its Applications

CPU scheduling using queue

PES UNIVERSITY
CELEBRATING 50 YEARS

Round Robin Scheduling:

- To implement Round Robin scheduling, The processes are kept in the queue of processes.
- New processes are added to the rear of the simple queue. The CPU scheduler picks the first process from the ready queue, sets a timer to interrupt after 1-time quantum, and dispatches the process.
- The process may have a CPU burst of less than 1-time quantum. In this case, the process itself will release the CPU voluntarily. The scheduler will then proceed to the next process in the ready queue.
- Otherwise, if the CPU burst of the currently running process is longer than 1-time quantum, the timer will go off and will cause an interrupt to the operating system.
- A context switch will be executed, and the process is put at the rear of the ready queue. The CPU scheduler will then select the next process in the ready

Data Structures and its Applications

CPU scheduling using queue

PES UNIVERSITY
CELEBRATING 50 YEARS

Premptive Priority Based Scheduling:

- In Preemptive Priority Scheduling, at the time of arrival of a process in the ready queue, its priority is compared with the priority of the other processes present in the ready queue as well as with the one which is being executed by the CPU at that point of time.

- The One with the highest priority among all the available processes will be given the CPU next.

Priority Based(Non-Preemptive) Scheduling:

- In the Non Preemptive Priority scheduling, The Processes are scheduled according to the priority number assigned to them.

- Once the process gets scheduled, it will run till the completion

THANK YOU

Dinesh Singh
Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

DATA STRUCTURES AND ITS APPLICATIONS UE23CS252A

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

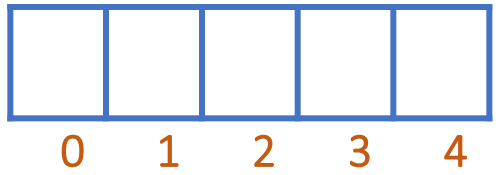
Basic Concept and Definitions: Trees

Shylaja S S
Department of Computer Science & Engineering

Introduction to Trees

PES UNIVERSITY

Linear Data Structures



List as an Array

Disadvantage:

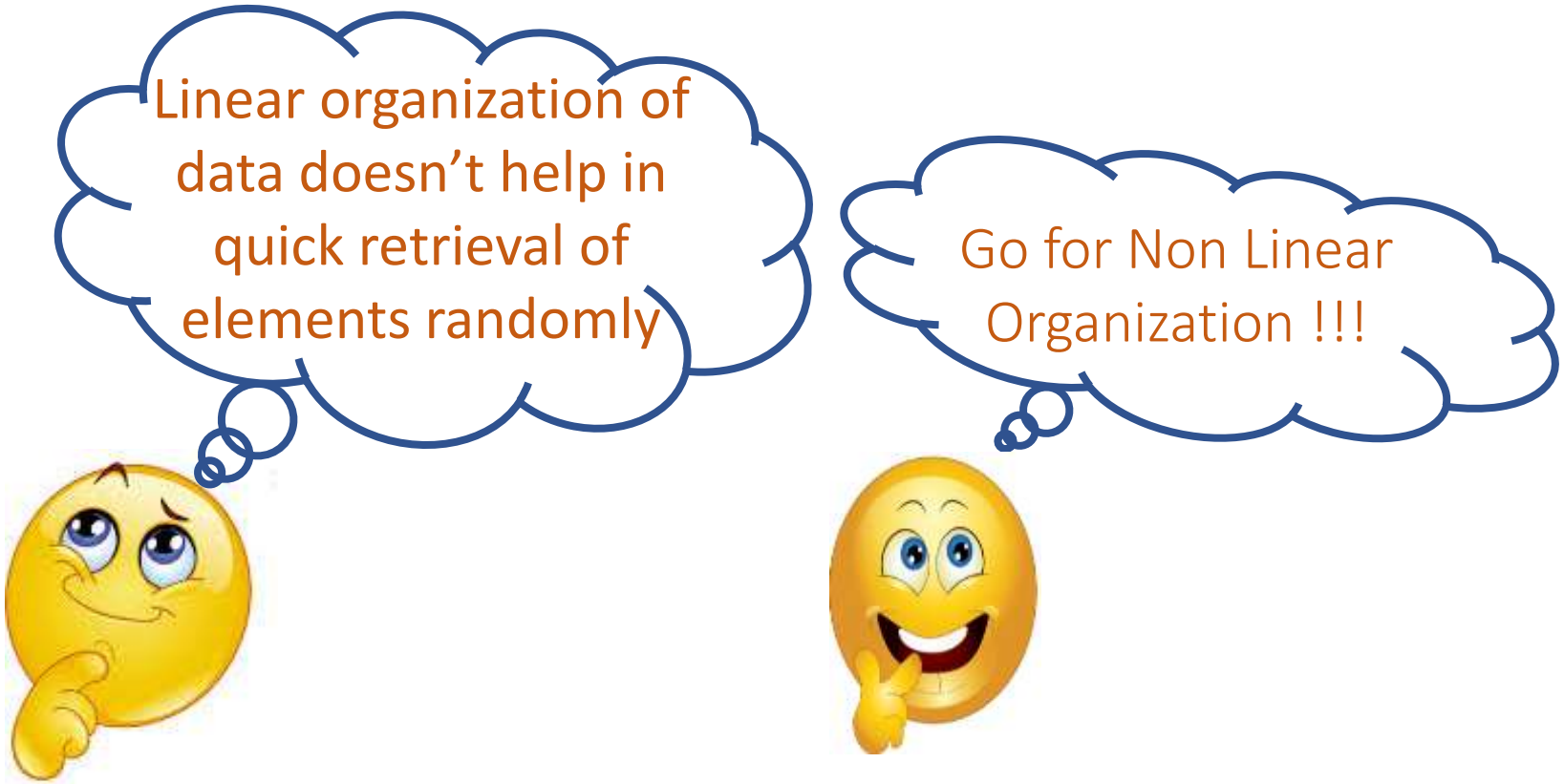
- Fixed Size
  - Expansion ✗
  - Shrink ✗
- Random Insertion & Deletion is Time Consuming

List as a Linked List

Disadvantage:

- Random Access is Time consuming

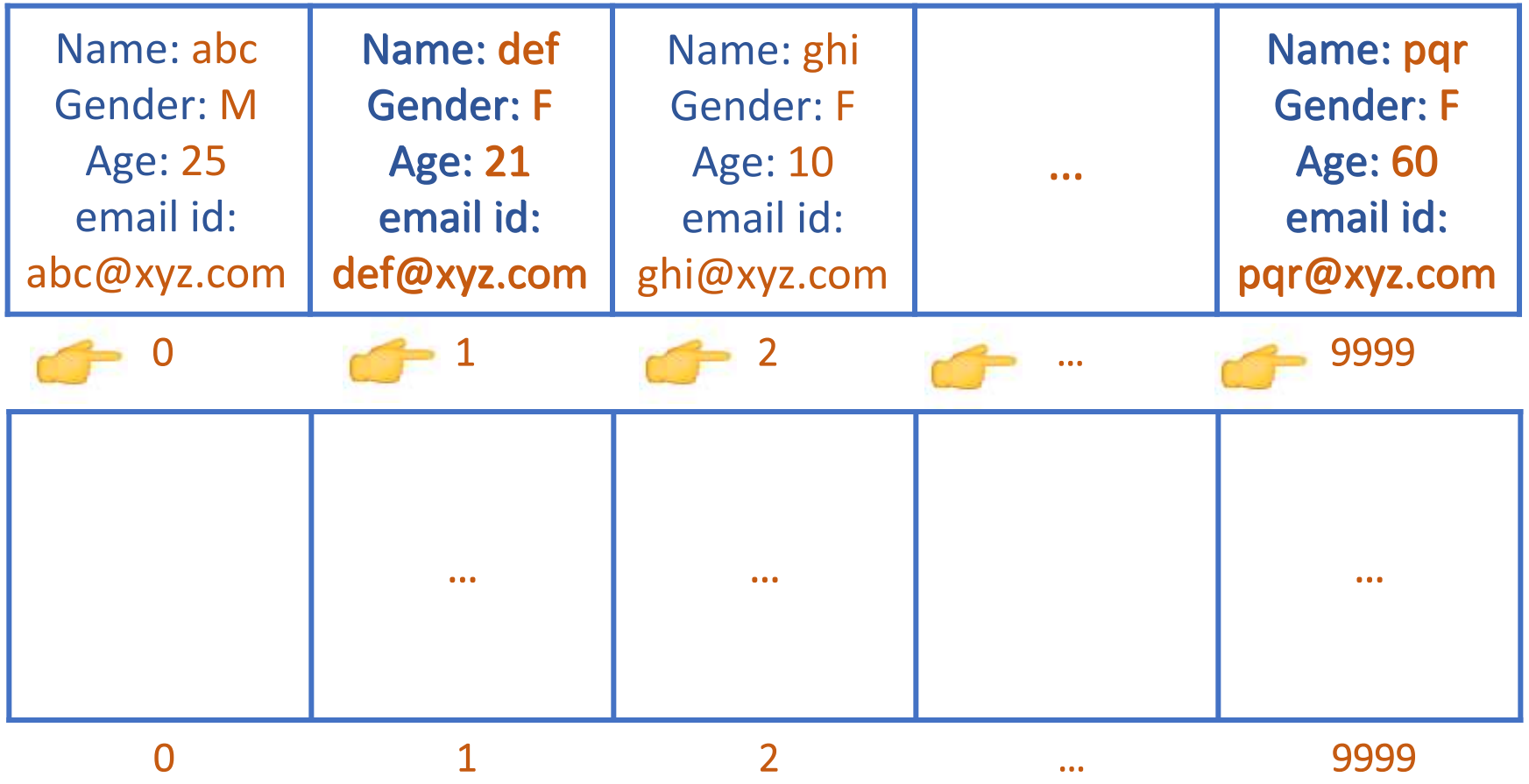
Introduction to Trees



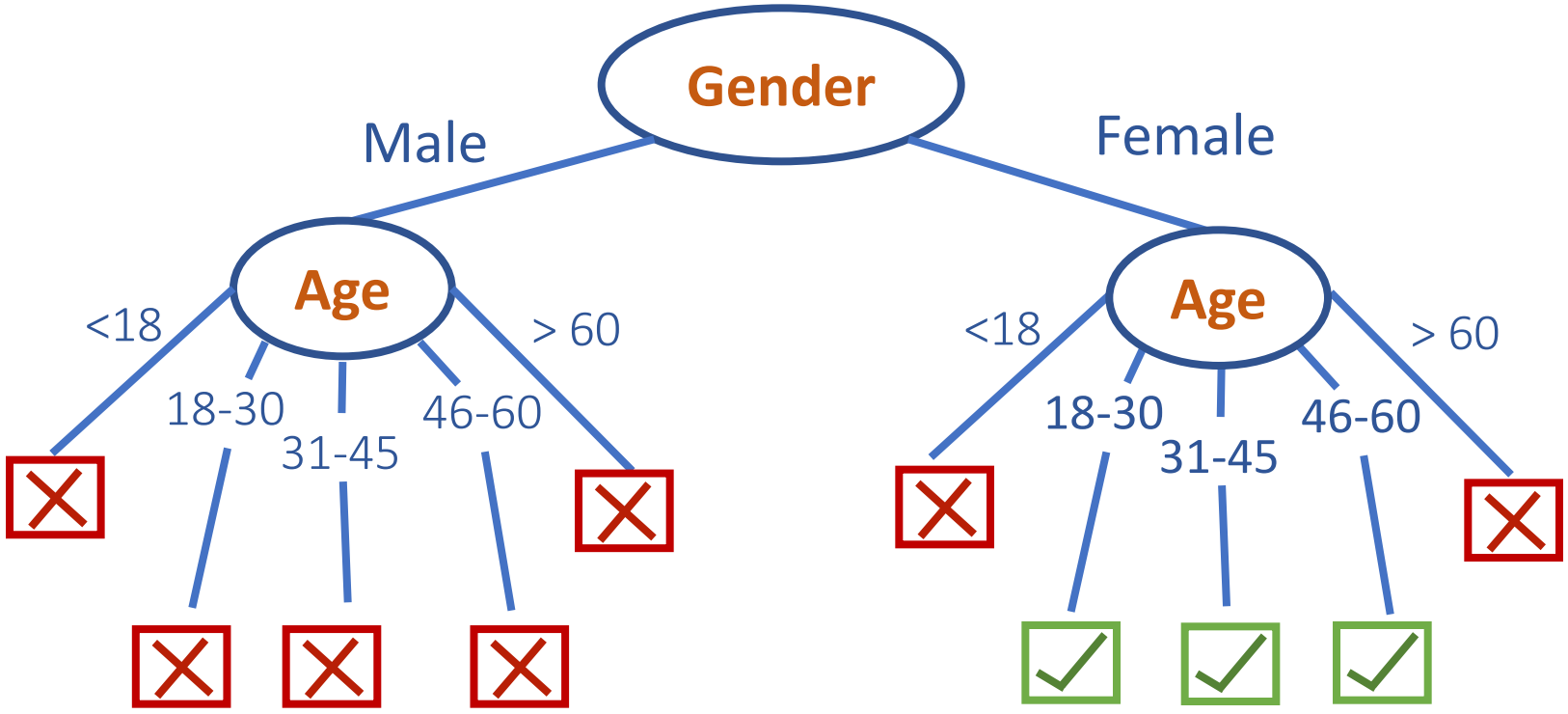
Introduction to Trees

PES UNIVERSITY

Example: To improve the probability of purchase of Women's Formal Wear in Less Time



Example: To improve the probability of purchase of Women's Formal Wear in Less Time

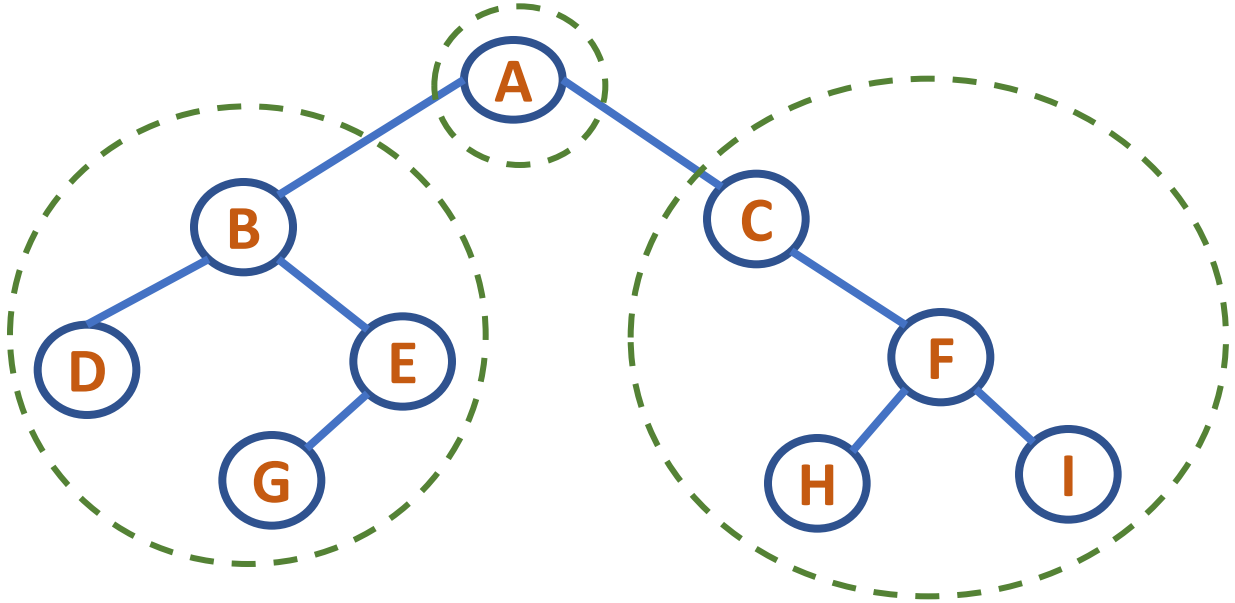


Search Not Matched

Search Matched

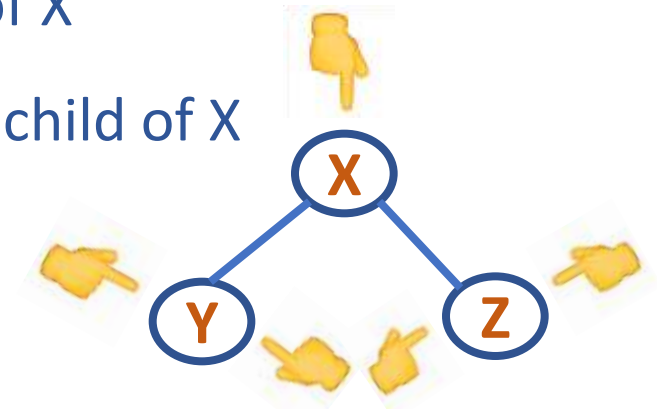
Binary Trees

- Non Linear Data Structure
- Finite set of elements that is either empty or is partitioned into three subsets
- First subset: is a single element, called the root
- Second subset: is a binary tree, called the left binary tree
- Third subset: is a binary tree, called the right binary tree



Binary Trees: Terminologies

- Each element of a binary tree is called a node of the tree
- Left node Y of X is called left child of X
- Right node Z of X is called the right child of X
- X is called the parent of Y and Z
- Y and Z are called siblings

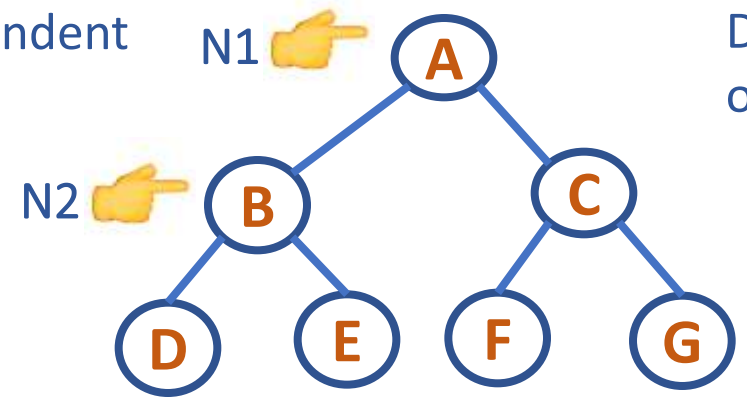


- A node which has no children is called leaf node/external node
- A node which has a child is called the non leaf node/internal node

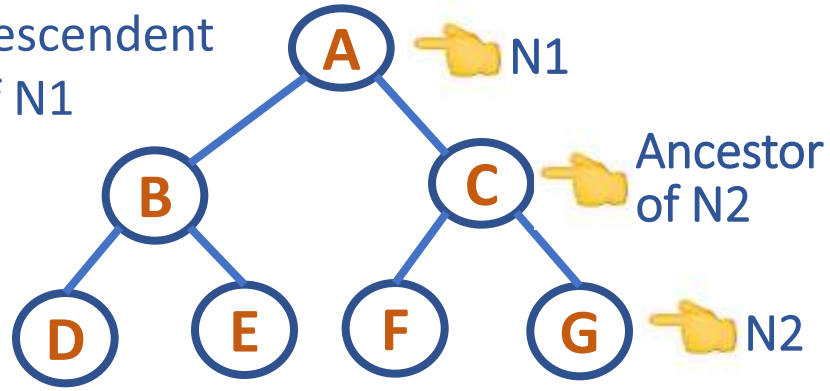
Binary Trees: Terminologies

- A node N1 is called the ancestor of a node N2 if
  - N1 is either the parent of N2 or
  - N1 is the parent of some ancestor of N2
- A node N2 becomes the descendent of node N1
- Descendent can be either the left descendent or the right descendent

N2 is the Left Descendent of N1



N2 is the Right Descendent of N1



Binary Trees: Terminologies

- Level of a node

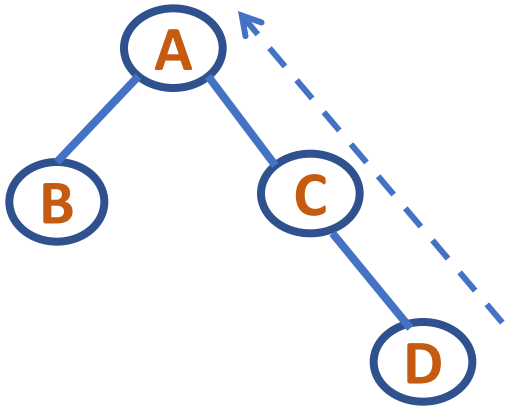
  Root has level 0; level of any other node is one more than its parent

- Depth of a tree

  Maximum level of any leaf in the tree (path length from the deepest leaf to the root)

- Depth of a node

  Path length from the node to the root

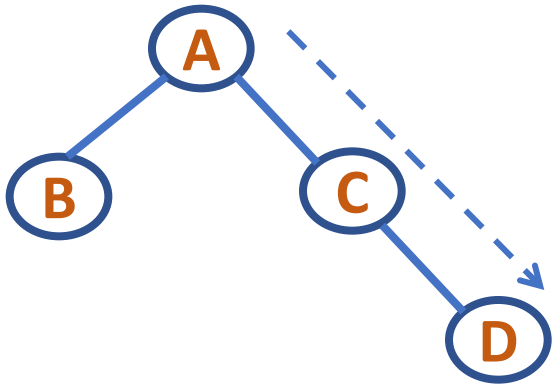


Level of node A – 0
Level of node B – 1
Level of node C – 1
Level of node D – 2

Depth of tree: 2
Depth of node A: 0
Depth of node B: 1
Depth of node C: 1
Depth of node D: 2

Binary Trees: Terminologies

- Height of a tree: Path length from the root node to the deepest leaf
- Height of a node: Path length from the node to the deepest leaf



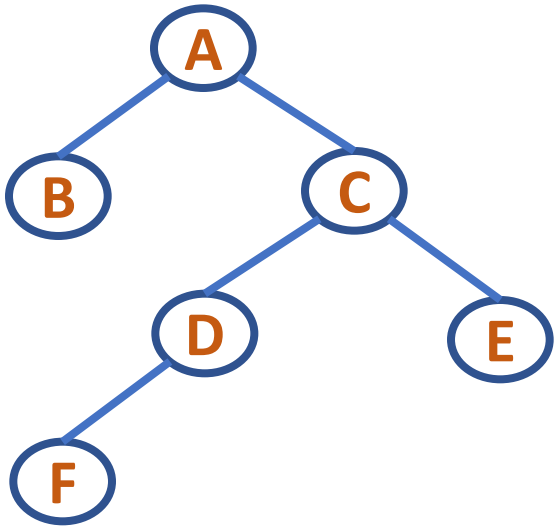
Height of Tree: 2

Height of Node A : 2
Height of Node B : 0
Height of Node C : 1
Height of Node D : 0

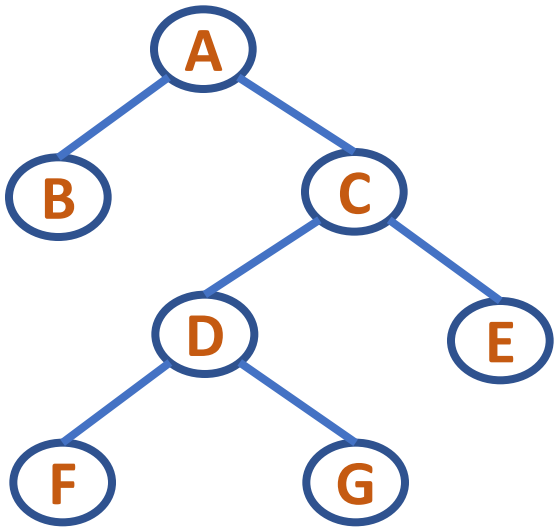
Binary Trees: Terminologies

Strictly Binary Tree

A Binary tree where every node has either zero/two children



Not a Strictly Binary Tree



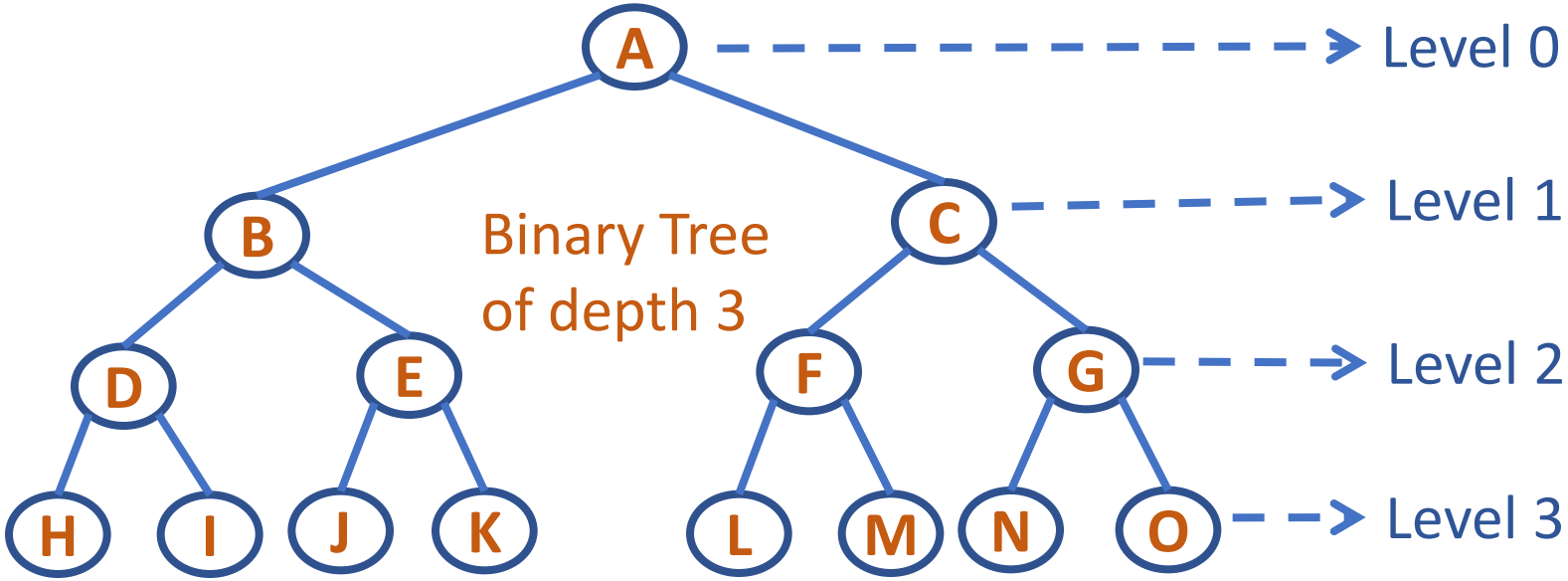
Strictly Binary Tree

Binary Trees: Terminologies

PES UNIVERSITY

Fully Binary Tree

- A binary tree with all the leaves at the same level
- If the binary tree has depth d, then there are 0 to d levels
- Total no. of nodes = $2^0 + 2^1 + \ldots + 2^d = 2^{(d+1)} - 1$



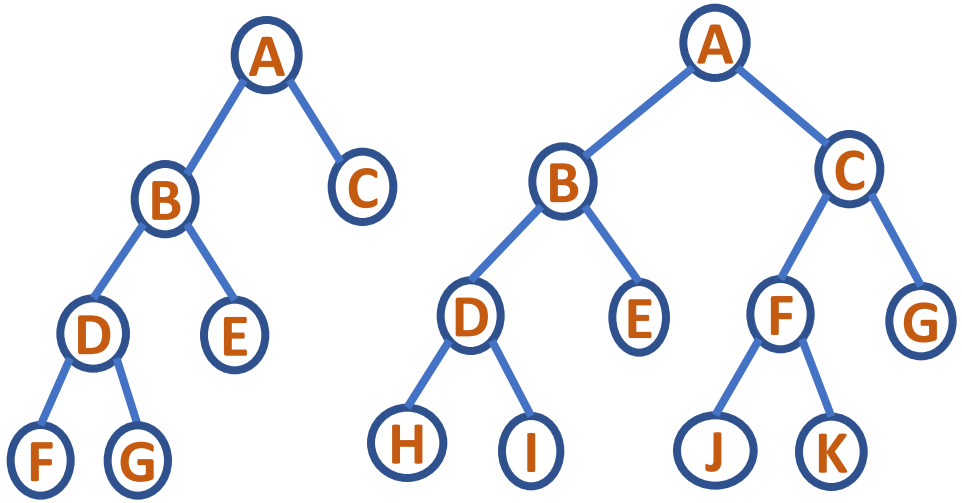
Binary Trees: Terminologies

PES UNIVERSITY

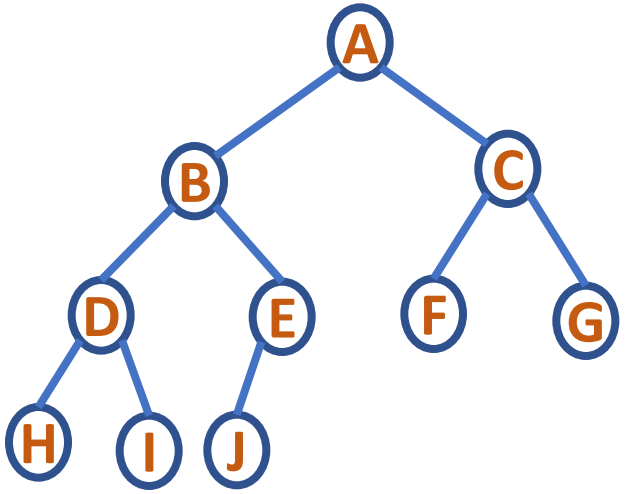
Complete Binary Tree

For a Complete Binary Tree with n nodes and depth d:

- Any node nd at level less than d-1 has two children
- For any node nd of the tree with a right descendent at level d, nd must have a left child and every left descendent of nd is either a leaf at level d or has two children



Not Complete Binary Trees



Complete Binary Tree

Binary Tree Properties

PES UNIVERSITY

Binary Tree Properties

- Every node except the root has exactly one parent
- A tree with n nodes has n-1 edges (every node except the root has an edge to its parent)
- A tree consisting of only root node has height of zero
- The total number of nodes in a full binary tree of depth d is $2^{(d+1)} – 1$, $d \geq 0$
- For any non-empty binary tree, if n_0 is the number of leaf nodes and n_2 the nodes of degree 2, then $n_0 = n_2 + 1$

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE23CS252A

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversal

Shylaja S S
Department of Computer Science & Engineering

Binary Tree Traversals

Important operation: Traversal

Traversal: Moving through all the nodes in a binary tree and visiting each one in turn

Trees: There are many orders possible since it is a nonlinear DS

Tasks: 1. Visiting a node denoted by V
2. Traversing the left subtree denoted by L
3. Traversing the right subtree denoted by R

Six ways to arrange them: VLR, LVR, LRV, VRL, RVL, RLV

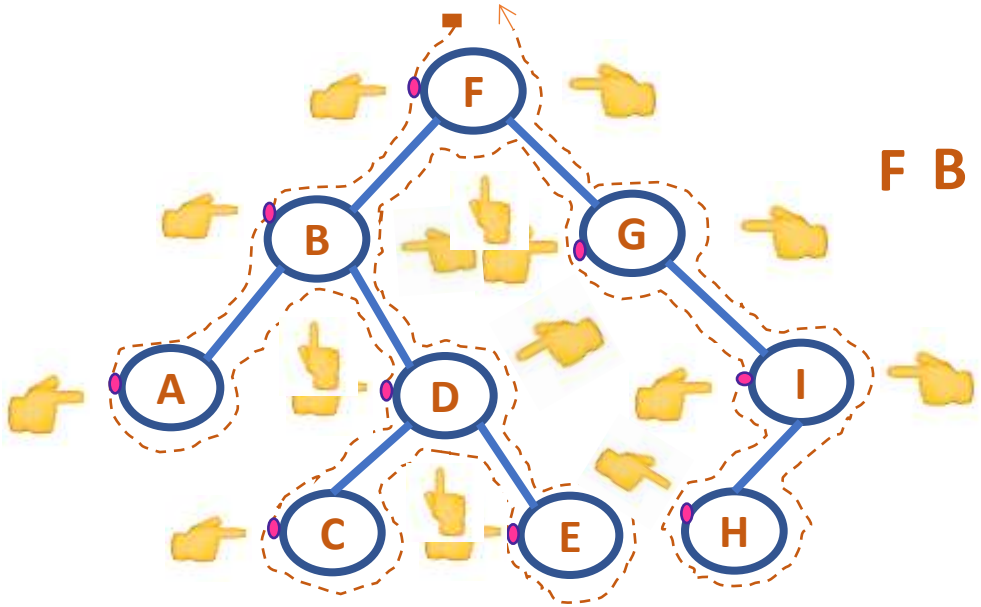
Standard Traversals include: VLR-Preorder, LVR-Inorder, LRV-Postorder

DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversal: Preorder

Steps:

- Root Node is visited before the subtrees
- Left subtree is traversed in preorder
- Right subtree is traversed in preorder



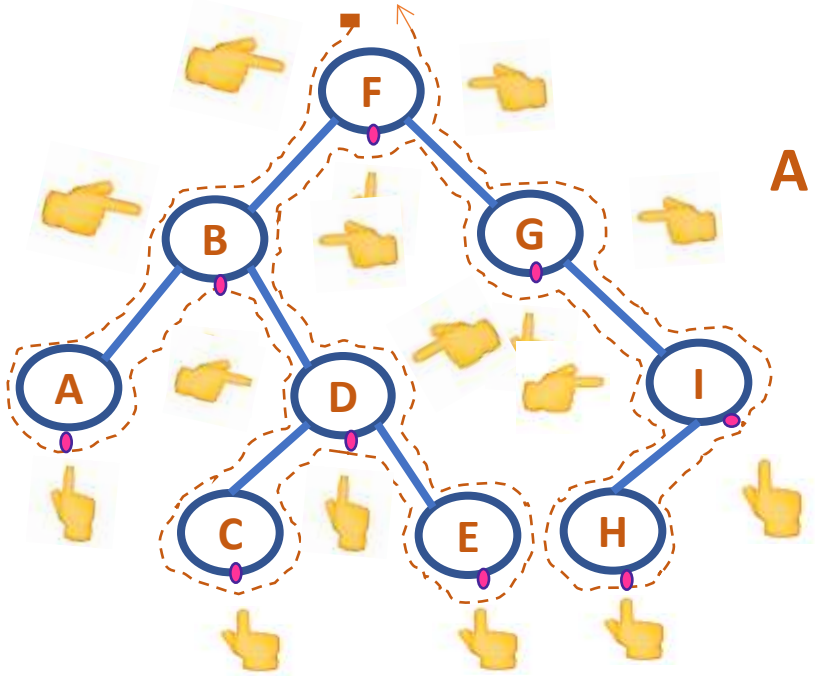
F B A D C E G I H

DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversal: Inorder

Steps:

- Left subtree is traversed in Inorder
- Root Node is visited
- Right subtree is traversed in Inorder



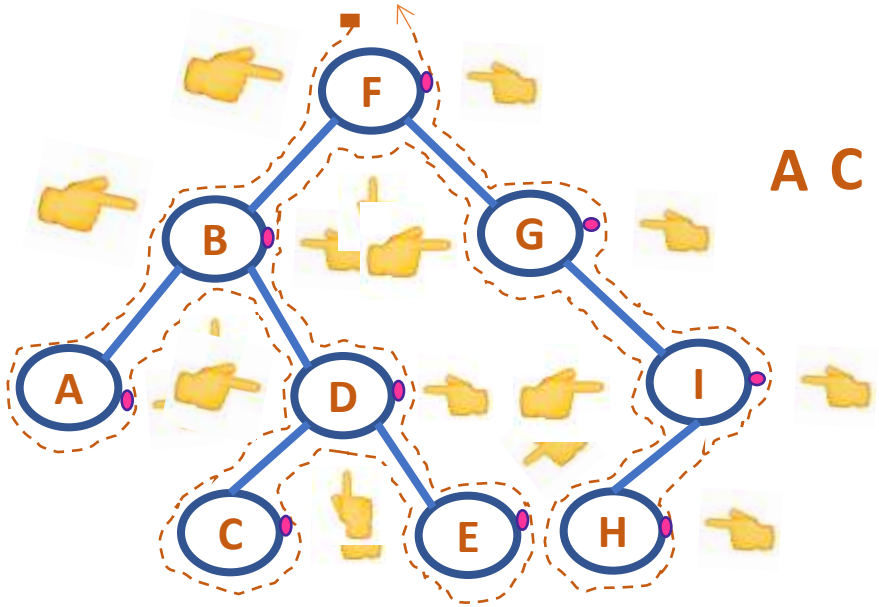
A B C D E F G H I

DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversal: Postorder

Steps:

- Left subtree is traversed in postorder
- Right subtree is traversed in postorder
- Root Node is visited



A C E D B H I G F

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE23CS252A

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

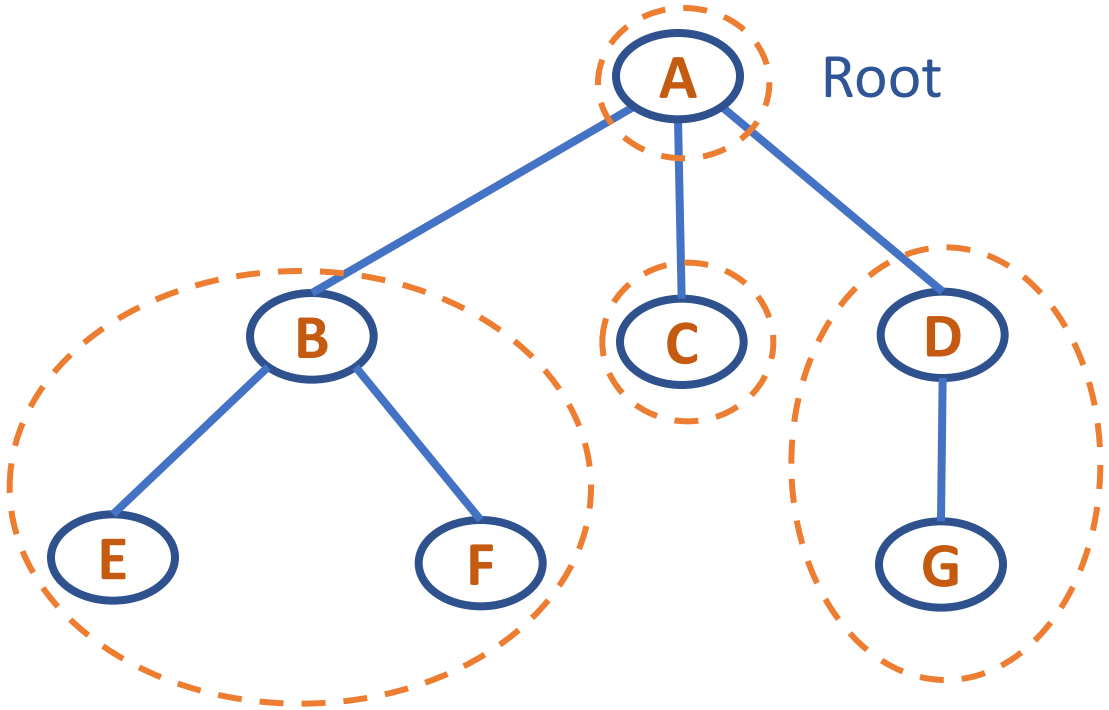
DATA STRUCTURES AND ITS APPLICATIONS

Basic Concept and Definitions: Trees

Shylaja S S
Department of Computer Science & Engineering

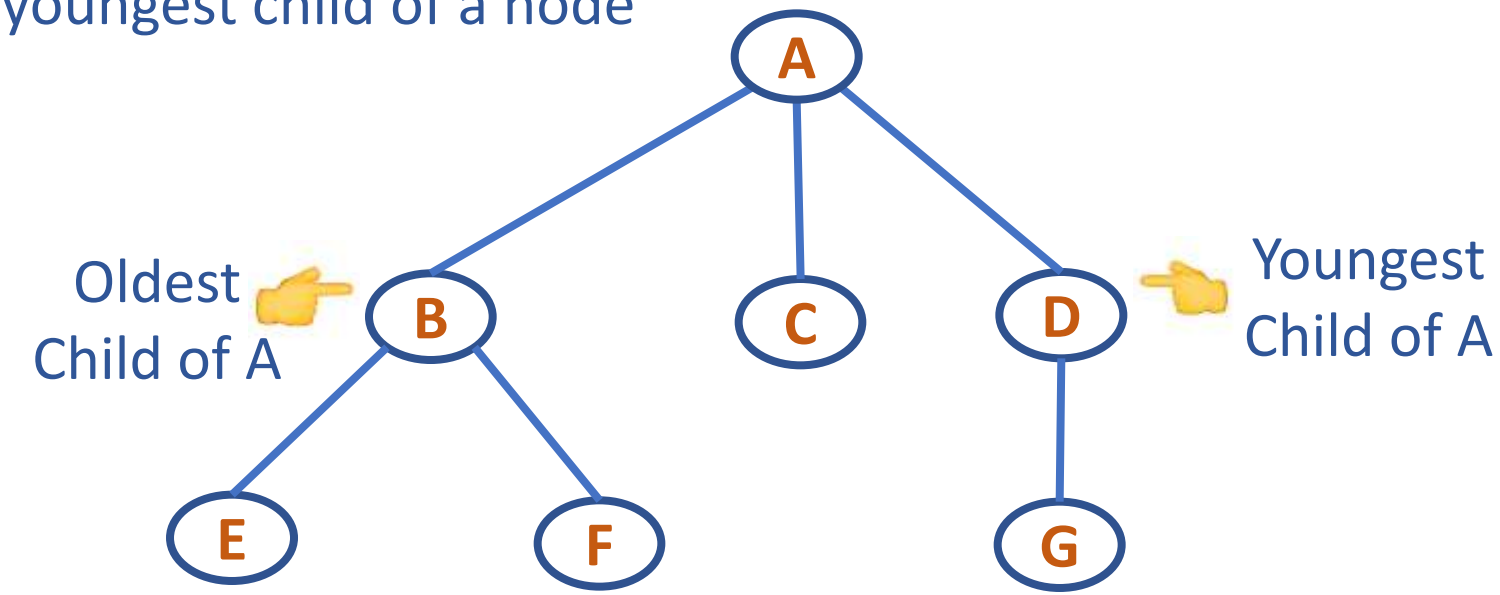
Trees

- Non Linear Data Structure
- Finite nonempty set of elements
  - One element is the root
  - Remaining elements are partitioned into m≥0 disjoint subsets each of which is itself a tree



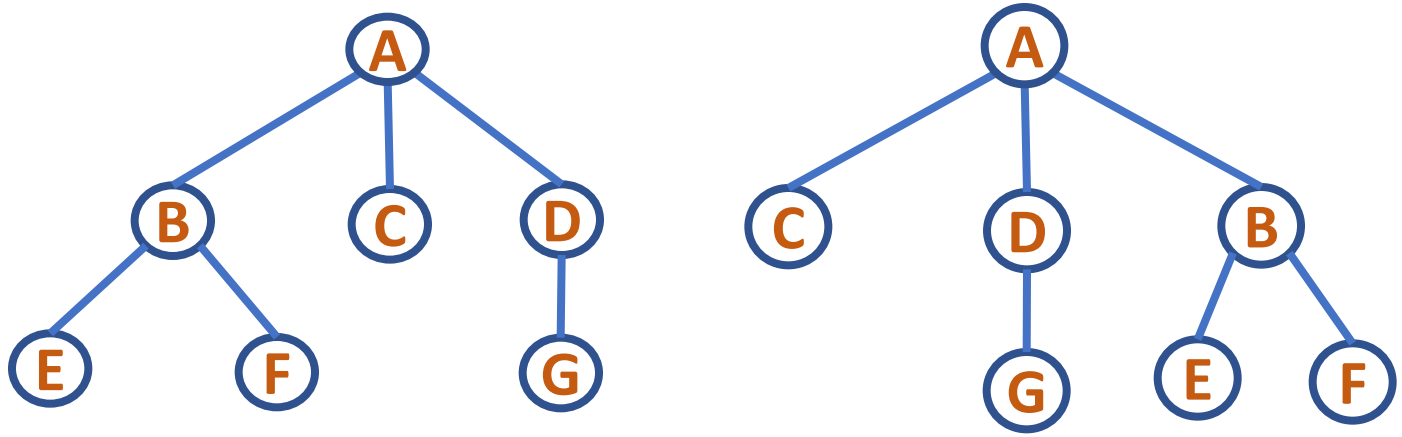
Trees

- Ordered Tree: a tree in which subtrees of each node forms an ordered set
- In such a tree we define first, second, ..., Last child of a particular node
- First child is called the oldest child and the last child the youngest child of a node



Trees

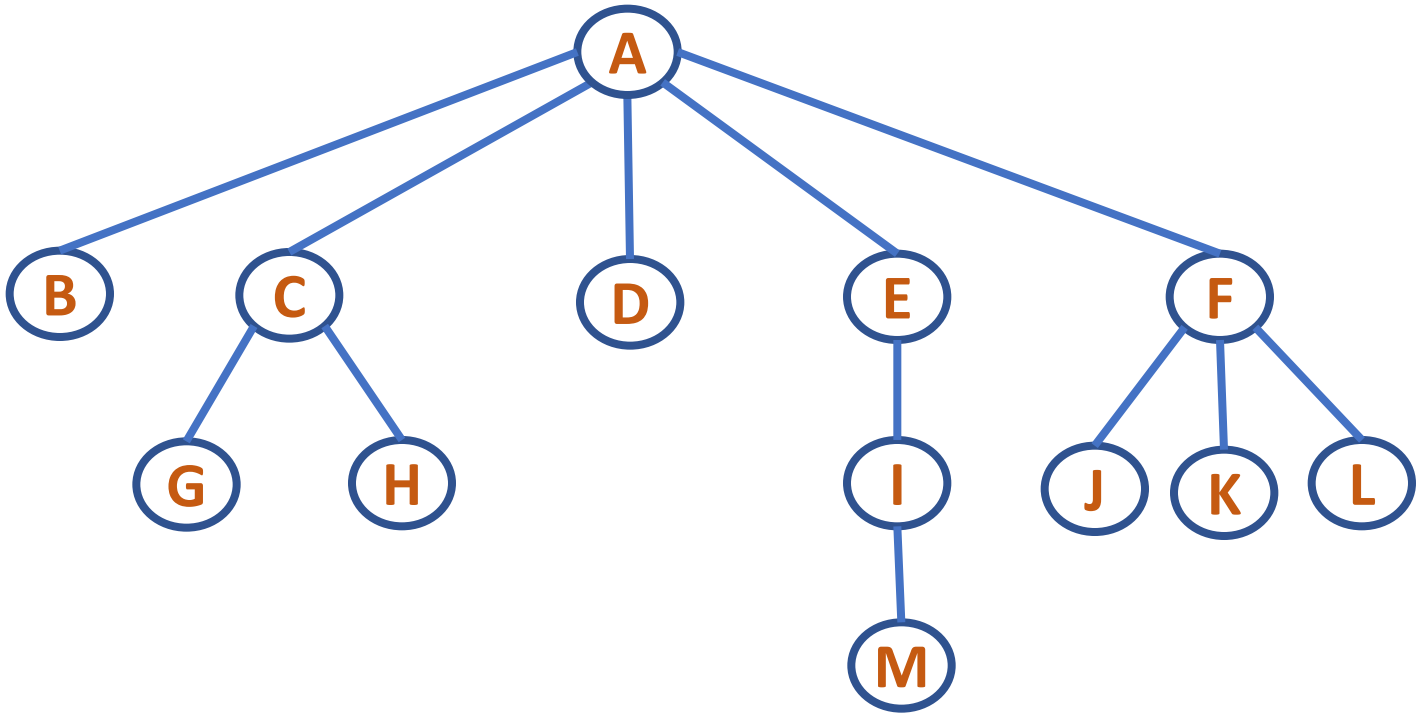
As unordered trees the below figures are equivalent but as ordered trees, they are different



N-ary Tree & Forest

n-ary tree: A rooted tree in which each node has no more than **n** children

- A binary tree is an n-ary tree with n=2
- n-ary tree with n=5



- Forest: is an ordered set of ordered trees

Tree

Representation of trees:

Tree node options:

```
struct treenode{
        int info;
        struct treenode *child[MAX];
};
```

where MAX is a constant

Restrictions with the above implementation:

- A node cannot have more than MAX children. Therefore cannot expand the tree

Tree

2nd implementation:

- All the children of a given node are linked and only the oldest child is linked to the parent
- A node has link to first child and a link to immediate sibling

```
struct treenode{
        int info;
        struct treenode *child;
        struct treenode *sibling;
};
```

```c
struct treenode {
    int info;                   // Stores the data (integer) in the node
    struct treenode *child;     // Pointer to the first (leftmost) child node
    struct treenode *sibling;   // Pointer to the next sibling node (right sibling)
};
```

DATA STRUCTURES AND ITS APPLICATIONS

Tree

```c
struct treenode {
    int info;                   // Stores the data (integer) in the node
    struct treenode *child;     // Pointer to the first (leftmost) child node
    struct treenode *sibling;   // Pointer to the next sibling node (right sibling)
};
```

Key Parts

1. `info`: This holds the data of the current node (in this case, an integer).
2. `child`: This points to the **first (leftmost) child** of the current node. If the node has no children, this pointer will be `NULL`.
3. `sibling`: This points to the **next sibling** of the current node (i.e., the next node at the same level). If there are no more siblings, this pointer will be `NULL`.

How This Structure Works

Instead of using a fixed-size array or a linked list for children, this approach models each node as having:

- A **single child** (the first or leftmost child).
- A **single sibling** (the next sibling of the current node).

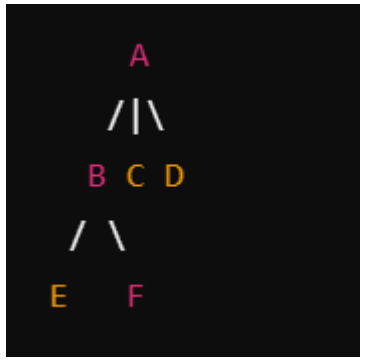
This effectively turns the general tree into a binary-like structure:

- The **child pointer** connects downward (to the children).
- The **sibling pointer** connects horizontally (to the siblings).

DATA STRUCTURES AND ITS APPLICATIONS

Tree

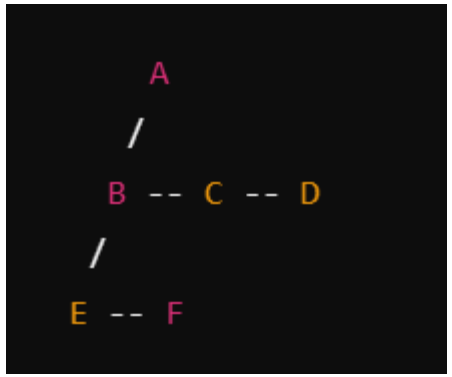
Consider this structure



In this tree:

- Node `A` has three children: `B`, `C`, and `D`.
- Node `B` has two children: `E` and `F`.
- Nodes `C`, `D`, `E`, and `F` have no children.

In the memory the above tree would look like:

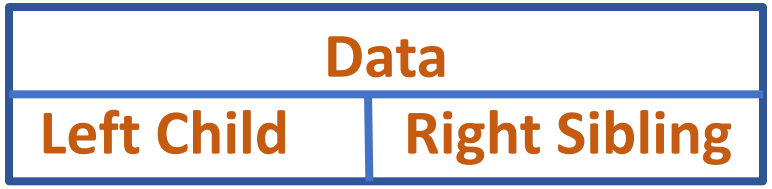


- Node `A`:
  - `child` → `B` (first child)
  - `sibling` → `NULL` (no sibling, since `A` is the root)
- Node `B`:
  - `child` → `E` (first child)
  - `sibling` → `C` (sibling)
- Node `C`:
  - `child` → `NULL` (no child)
  - `sibling` → `D` (sibling)
- Node `D`:
  - `child` → `NULL` (no child)
  - `sibling` → `NULL` (no sibling)
- Node `E`:
  - `child` → `NULL` (no child)
  - `sibling` → `F` (sibling)
- Node `F`:
  - `child` → `NULL` (no child)
  - `sibling` → `NULL` (no sibling)

Conversion of an n-ary Tree to a Binary Tree

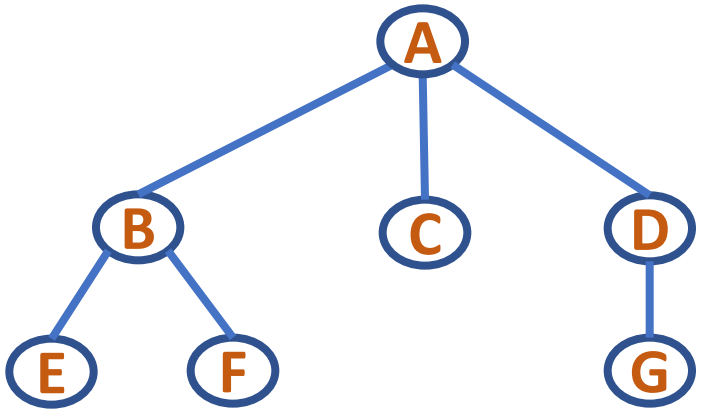
Left Child – Right Sibling Representation

- Link all the siblings of a node
- Delete all links from a node to its children except for the link to its leftmost child
- The left child in binary tree is the node which is the oldest child of the given node in an n-ary tree, and the right child is the node to the immediate right of the given node on the same horizontal line. Such a binary tree will not have a right sub tree
- The node structure corresponds to that of

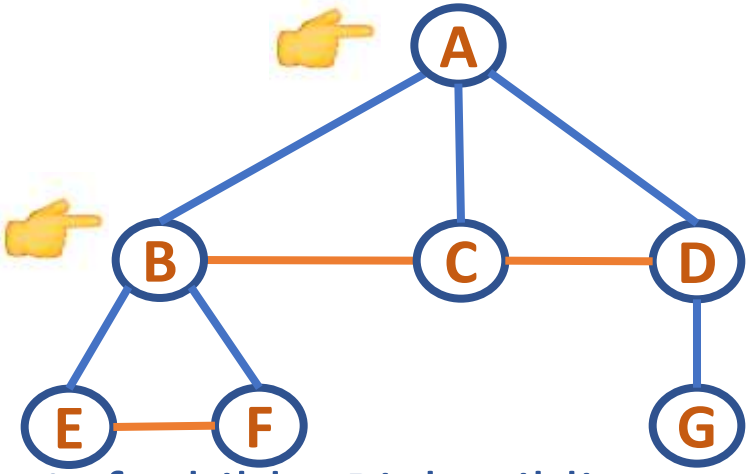


DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree



3-ary tree



Left child – Right sibling

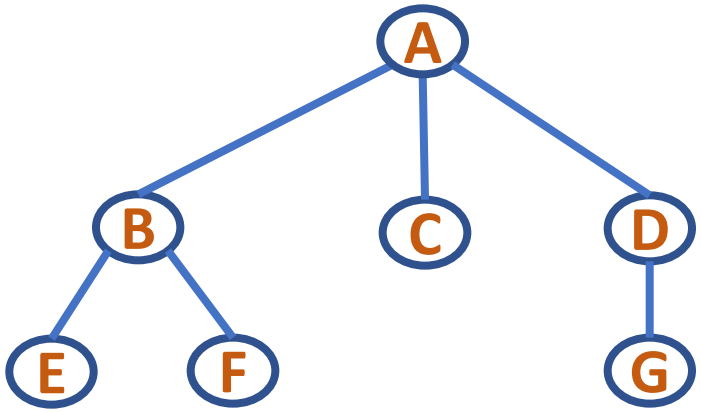
Representation of the above 3-ary tree

Link all siblings of a Node

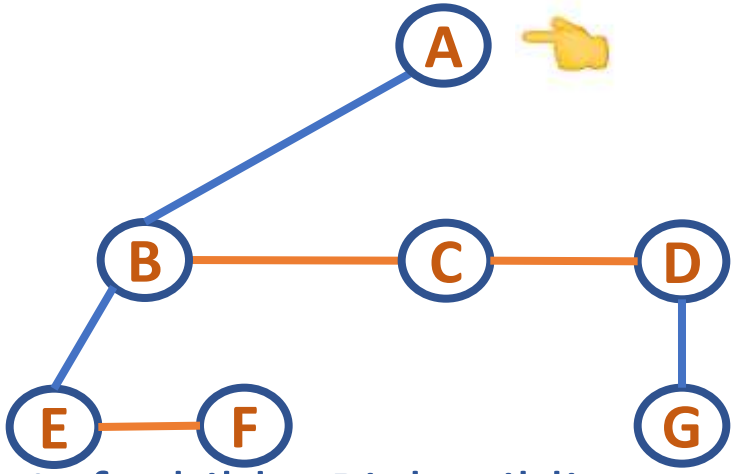
Delete all links from a Node to its children except for the link to its leftmost child

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree

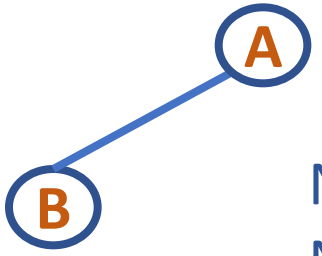


3-ary tree



Left child – Right sibling

Representation of the above 3-ary tree



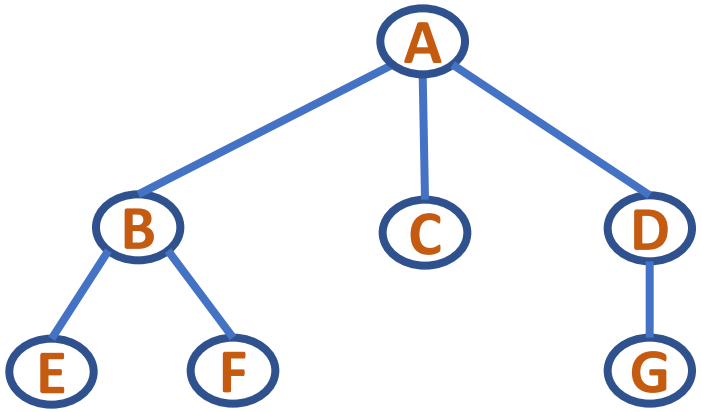
Node Below is B – Left child

No Right sibling so – no Right child

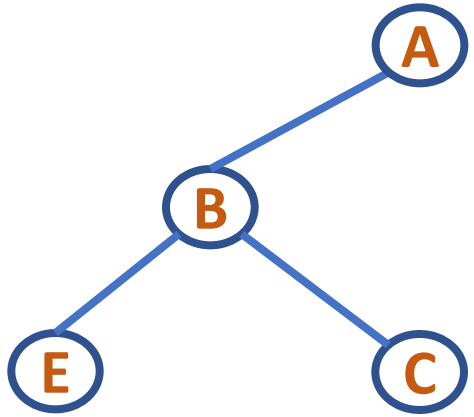
Corresponding binary tree

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree

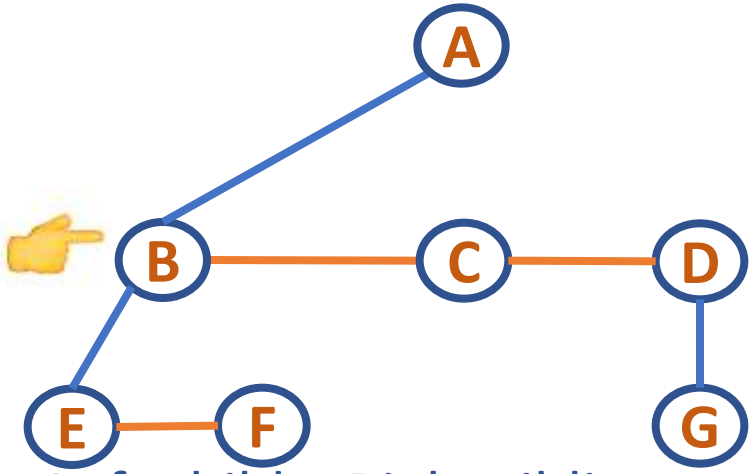


3-ary tree



Node Below is E – Left child

Next Right sibling is C – Right child



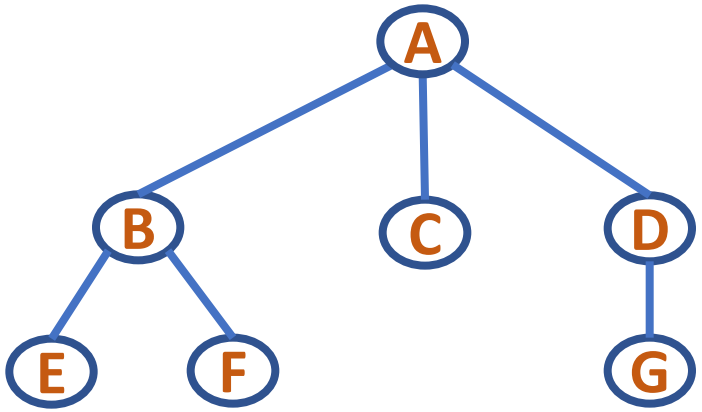
Left child – Right sibling

Representation of the above 3-ary tree

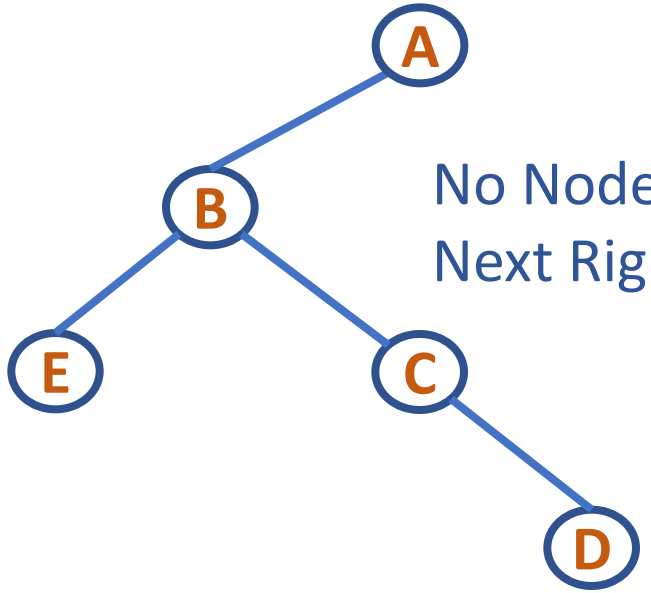
Corresponding binary tree

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree

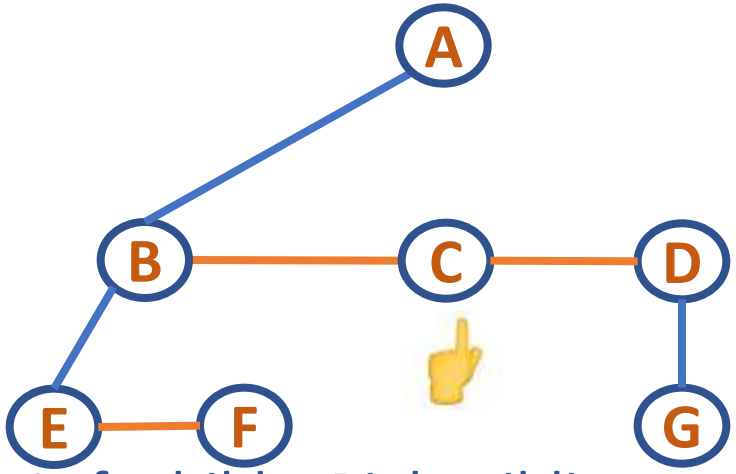


3-ary tree



No Node Below so – no Left child

Next Right sibling is D – Right child



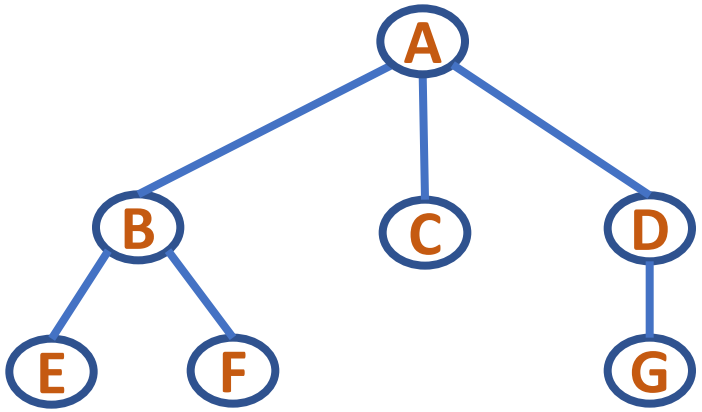
Left child – Right sibling

Representation of the above 3-ary tree

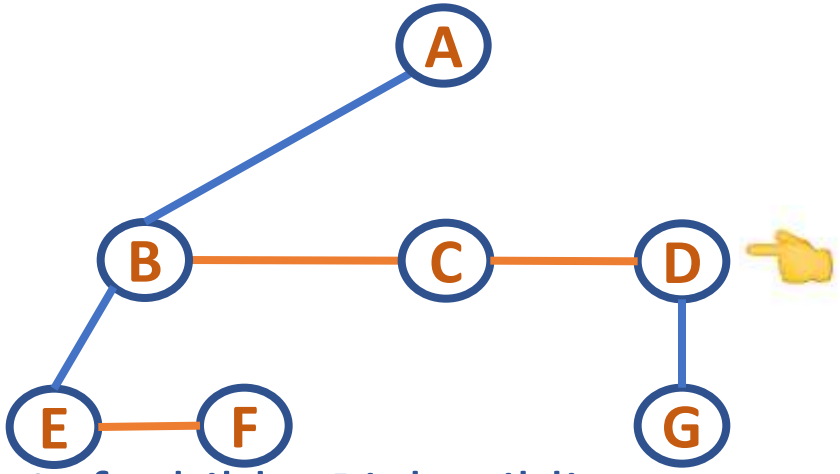
Corresponding binary tree

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree

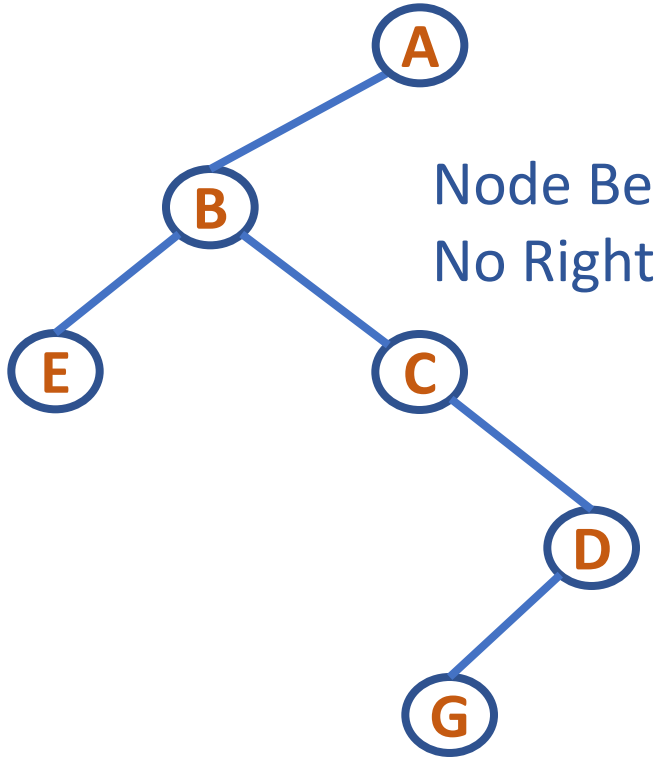


3-ary tree



Left child – Right sibling

Representation of the above 3-ary tree

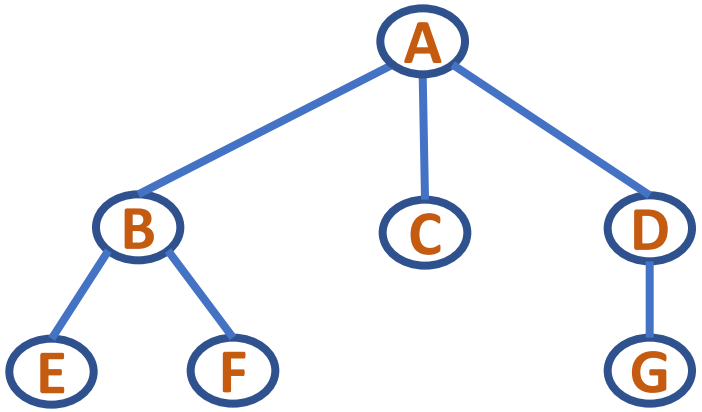


Node Below is G – Left child

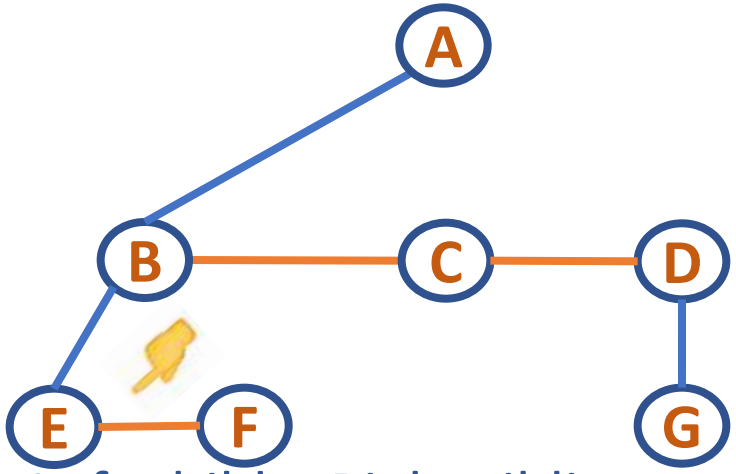
No Right sibling so – no Right child

Corresponding binary tree

Conversion of an n-ary Tree to a Binary Tree

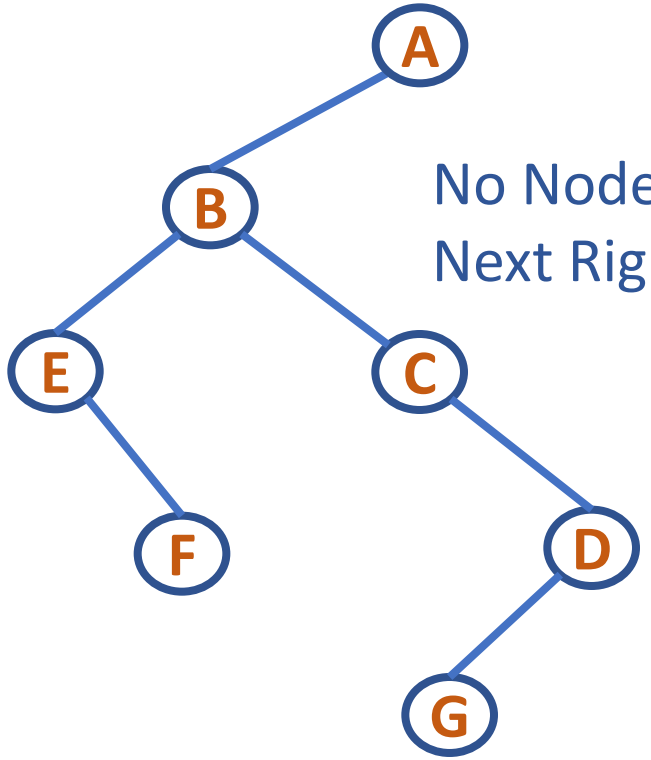


3-ary tree



Left child – Right sibling

Representation of the above 3-ary tree



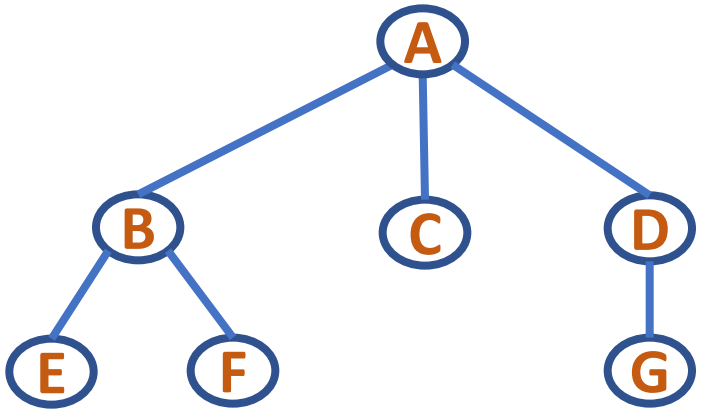
No Node Below so – no Left child

Next Right sibling is F – Right child

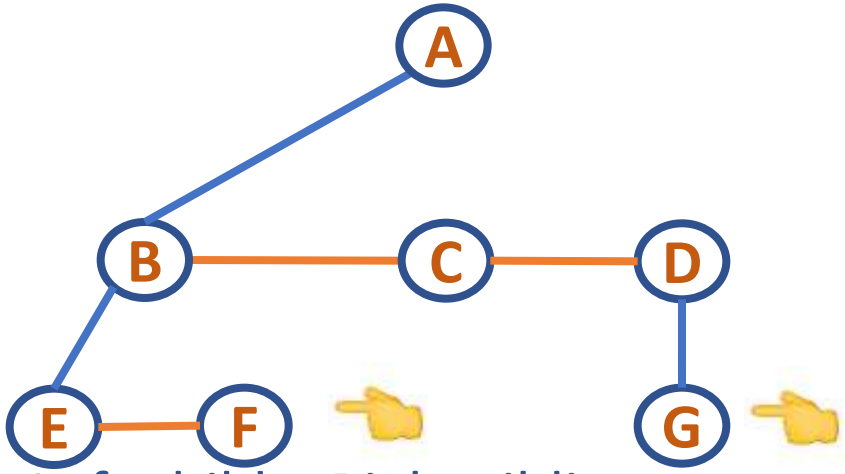
Corresponding binary tree

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree

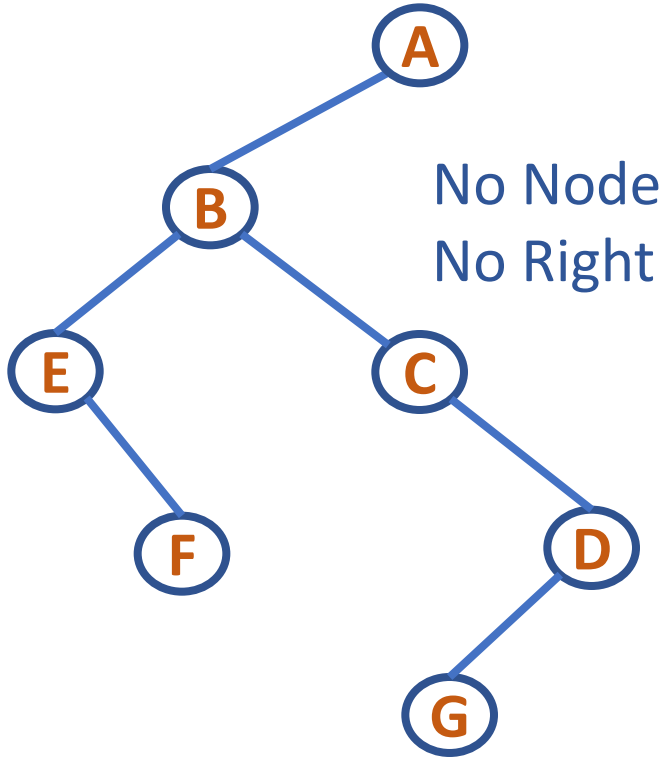


3-ary tree



Left child – Right sibling

Representation of the above 3-ary tree



No Node Below so – no Left child

No Right sibling so – no Right child

Corresponding binary tree

Conversion of a Forest to a Binary Tree

PES UNIVERSITY

- Right Child of the root node of every resulting binary tree will be empty. This is because the root of the tree we are transforming has no siblings.

- On the other hand, if we have a forest then these can all be transformed into a single binary tree as follows:
  - First obtain the binary tree representation of each of the trees in the forest
  - Link all the binary trees together through the right sibling field of the root nodes

Conversion of a Forest to a Binary Tree

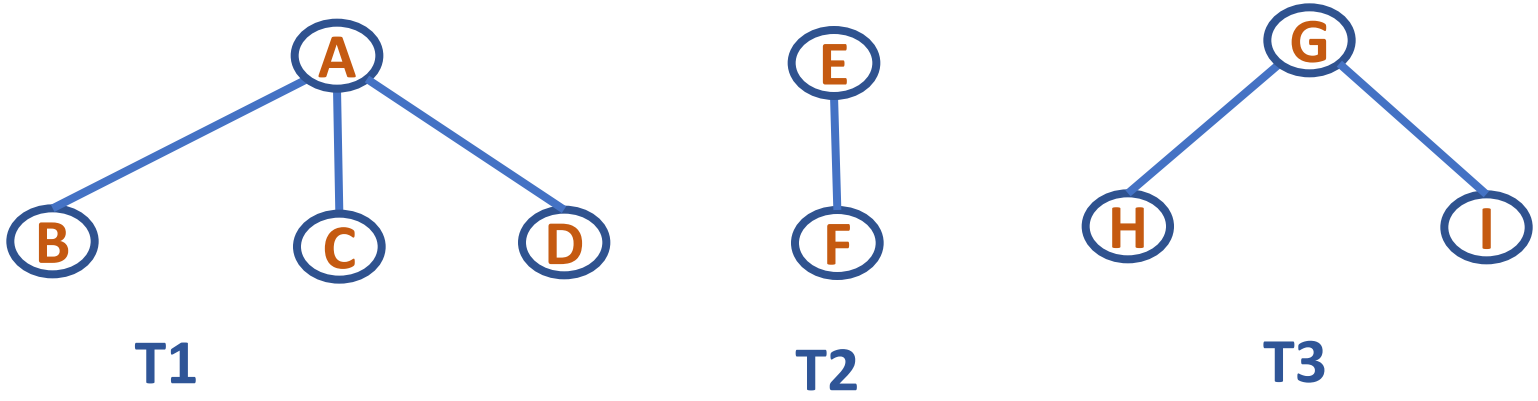
Conversion of a Forest to a Binary Tree can be formally defined as follows:

- If $T_1,...,T_n$ is a forest of n trees, then the binary tree corresponding to this forest, denoted by $B(T_1, ..., T_n)$:
  - is empty if $n = 0$
  - has root equal to root (T_1)
  - has left subtree equal to $B(T_{11}, T_{12}, ...,T_{1m})$ where T_{11} ,..., T_{1m} are the subtrees of root(T_1)
  - has right subtree $B(T_2, ..., T_n)$

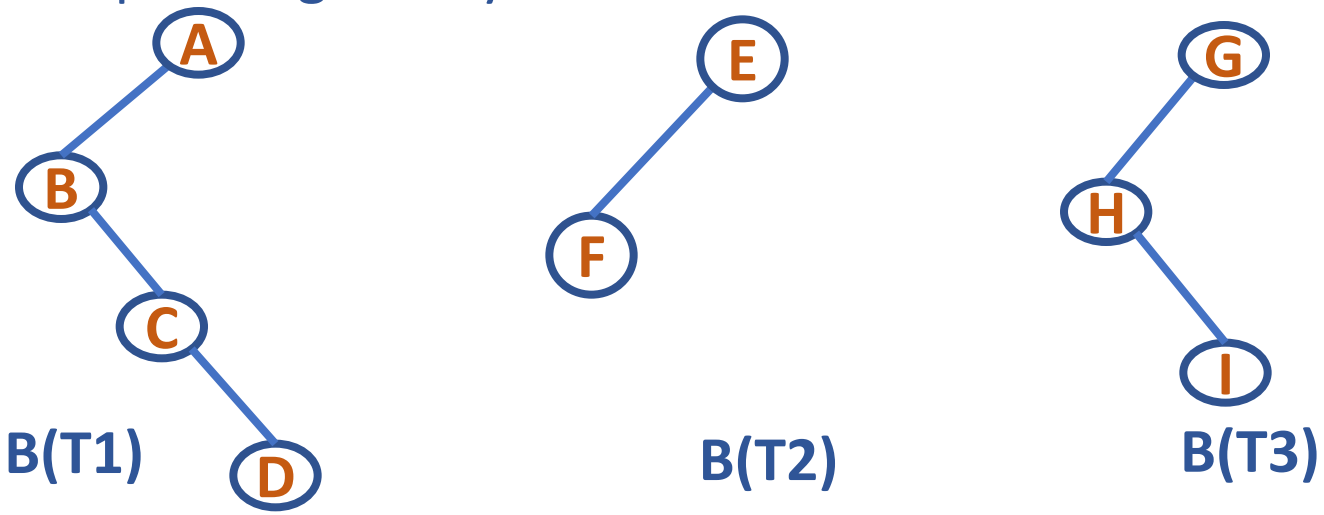
DATA STRUCTURES AND ITS APPLICATIONS

Conversion of a Forest to a Binary Tree

Consider the following Forest with three Trees

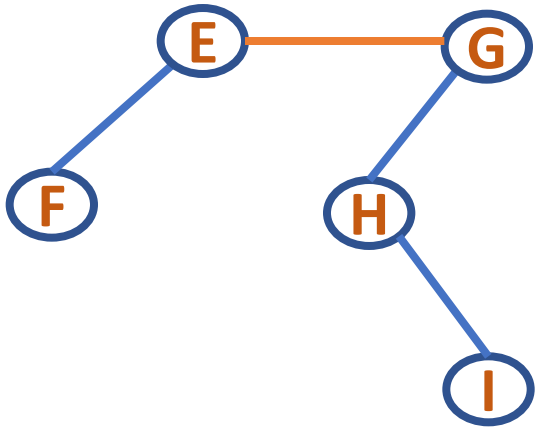


Corresponding Binary Trees

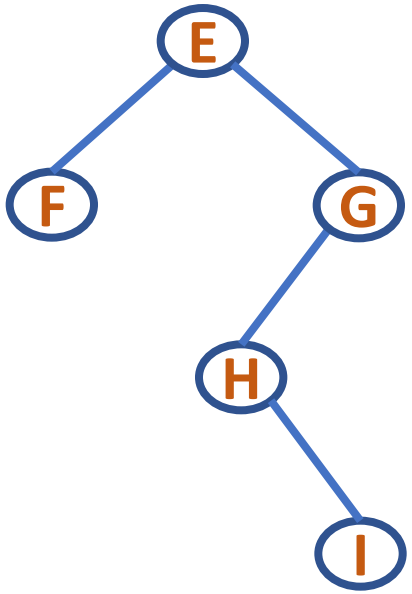


Conversion of a Forest to a Binary Tree

Link B(T2) and B(T3)



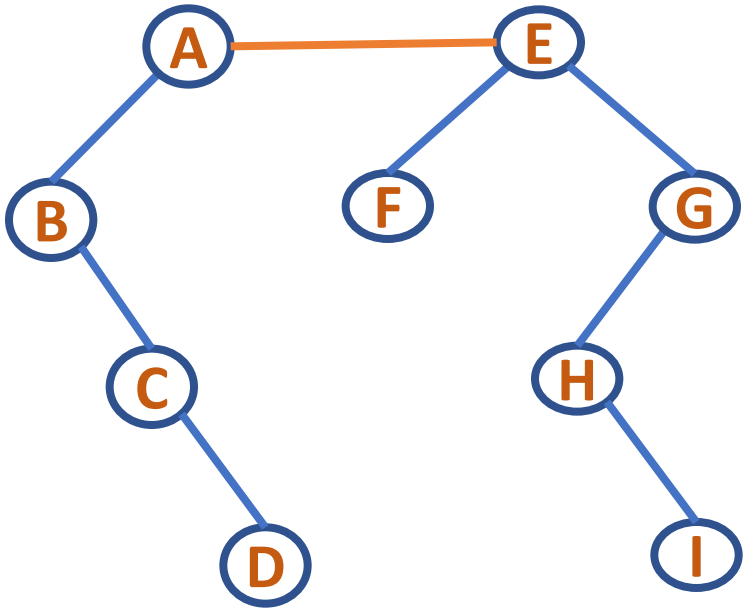
G becomes Right Child of E



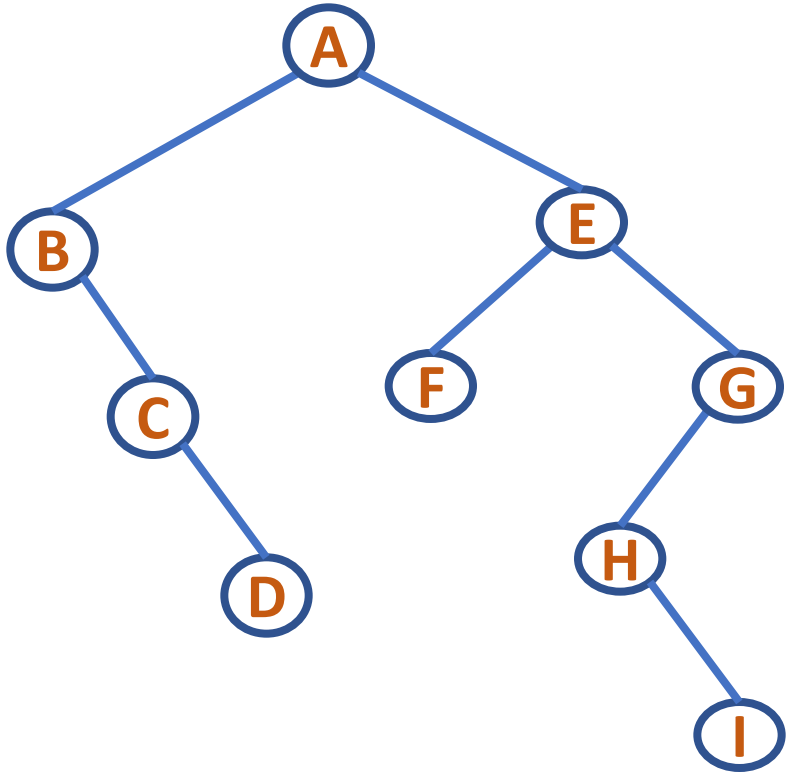
Corresponding Binary Tree
B(T2, T3)

Conversion of a Forest to a Binary Tree

Link B(T1) and B(T2, T3)



E becomes Right Child of A



Corresponding Binary Tree B(T1, T2, T3)

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS UE23CS252A

Shylaja S S & Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

n-ary Tree Traversal

Shylaja S S
Department of Computer Science & Engineering

Tree Traversal

Structure of a treenode revisited

```
struct treenode{
        int info;
        struct treenode *child;
        struct treenode *sibling;
};
```

Tree Traversal

With the treenode implemented as having pointers to first child and immediate sibling, the traversal preorder, inorder and postorder for a tree are defined as follows:

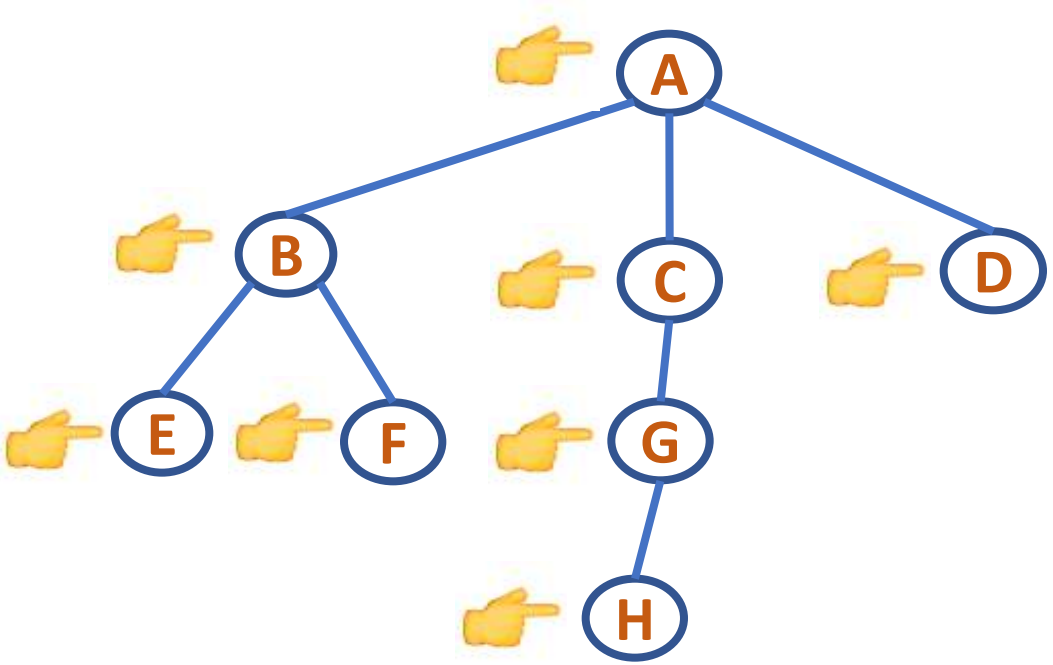
Preorder:

1. Visit the root of the first tree in the forest
2. Traverse in preorder the forest formed by the subtrees of the first tree, if any
3. Traverse in preorder the forest formed by the remaining trees in the forest, if any

Tree Traversal

PES UNIVERSITY

Preorder Tree Traversal



A B E F C G H D

Tree Traversal

```
void preorder(TREE *root)
{
    if(root!=NULL)
    {
      printf(" %d ",root->info);
      preorder(root->child);
      preorder(root->sibling);
    }
}
```

Tree Traversal

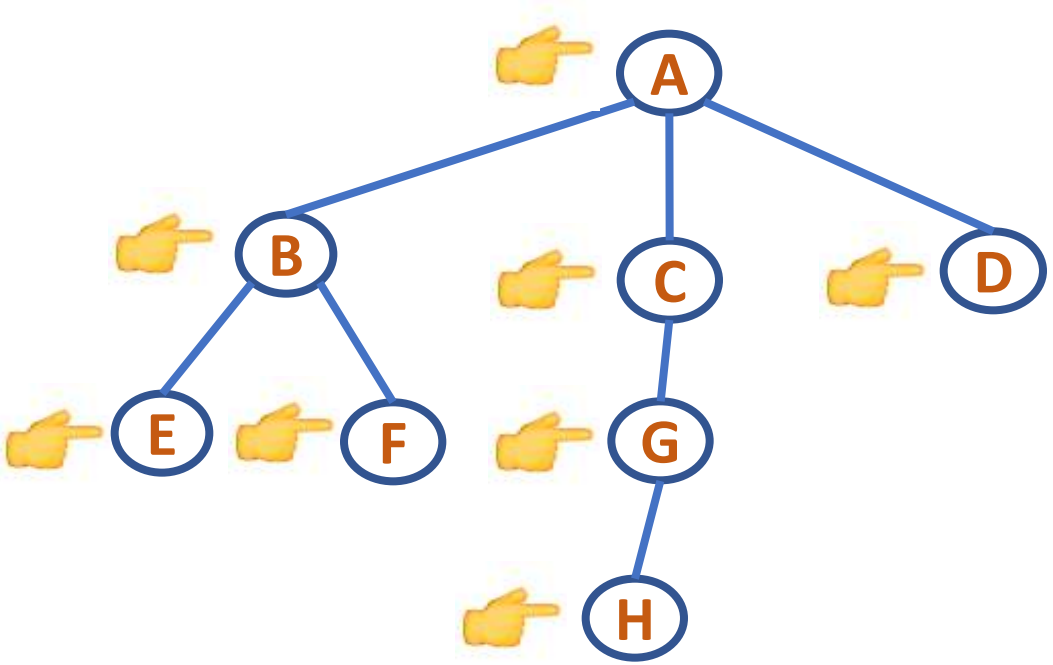
Inorder

1. Traverse in inorder the forest formed by the subtrees of the first tree, if any
2. Visit the root of the first tree in the forest
3. Traverse in inorder the forest formed by the remaining trees in the forest, if any

Tree Traversal

PES UNIVERSITY

Inorder Tree Traversal



E F B H G C D A

Tree Traversal

```
void inorder(TREE *root)
{
  if(root!=NULL)
   {
     inorder(root->child);
     printf(" %d ",root->info);
     inorder(root->sibling);
   }
}
```

Tree Traversal

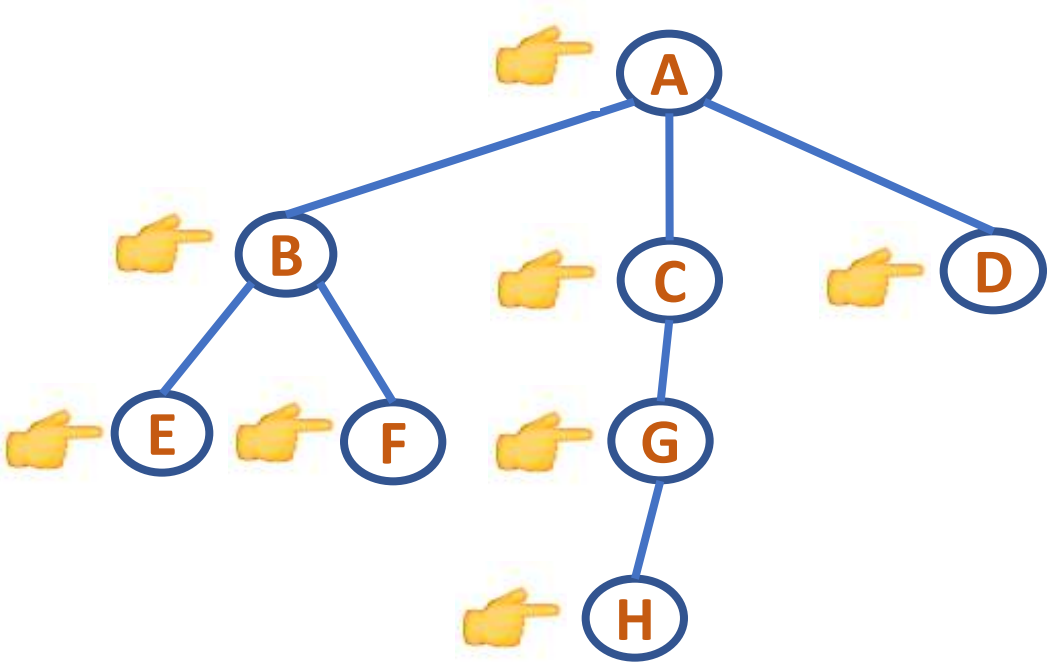
Postorder

1. Traverse in postorder the forest formed by the subtrees of the first tree, if any
2. Traverse in postorder the forest formed by the remaining trees in the forest, if any
3. Visit the root of the first tree in the forest

Tree Traversal

PES UNIVERSITY

Postorder Tree Traversal



F E H G D C B A

Tree Traversal

```
void postorder(TREE *root)
{
  if(root!=NULL)
   {
     postorder(root->child);
     postorder(root->sibling);
     printf(" %d ", root->info);
   }
}
```

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DATA STRUCTURES & ITS APPLICATIONS
UE21CS252A

Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES & ITS APPLICATIONS

UNIT 1: Skip List

Kusuma K V
Department of Computer Science & Engineering

Performance considerations: lists

- Size():
  - O(1) if size is stored explicitly, else O(n)
- IsEmpty():
  - O(1)
- FindElement(k):
  - O(n)
- InsertItem(k, e):
  - O(1) (assumes item already found)
- Remove(k):
  - O(1) (assumes item already found)

Skip List

- Skip Lists support O(log n)
  - Insertion
  - Deletion
  - Search
- A relatively recent data structure
  - W. Pugh in 1989
- “A probabilistic alternative to balanced trees”

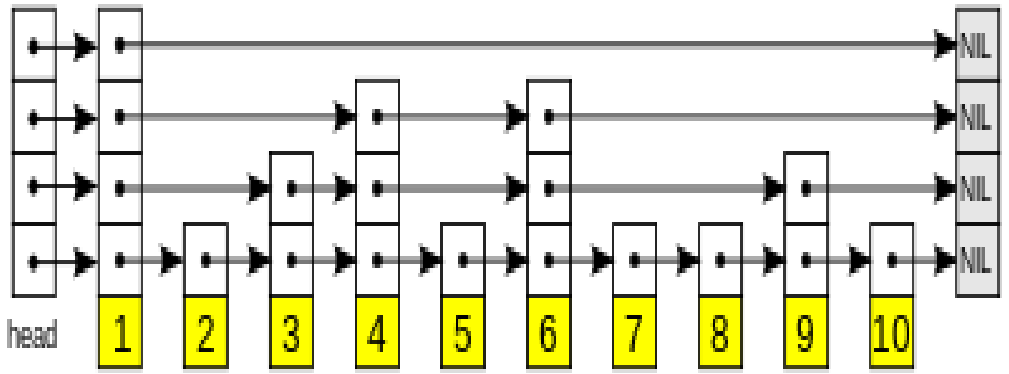
Skip List

- A skip list is a probabilistic data structure that allows O(logn) search complexity as well as O(logn) insertion complexity within an ordered sequence of n elements
- Thus it can get the best features of a sorted array (for searching) while maintaining a linked list-like structure that allows insertion, which is not possible in an array

https://en.wikipedia.org/wiki/Skip_list

Skip List

- Fast search is made possible by maintaining a linked hierarchy of sub sequences, with each successive subsequence skipping over fewer elements than the previous one
- Searching starts in the sparsest subsequence until two consecutive elements have been found, one smaller and one larger than or equal to the element searched for
- Via the linked hierarchy, these two elements link to elements of the next sparsest subsequence, where searching is continued until finally we are searching in the full sequence



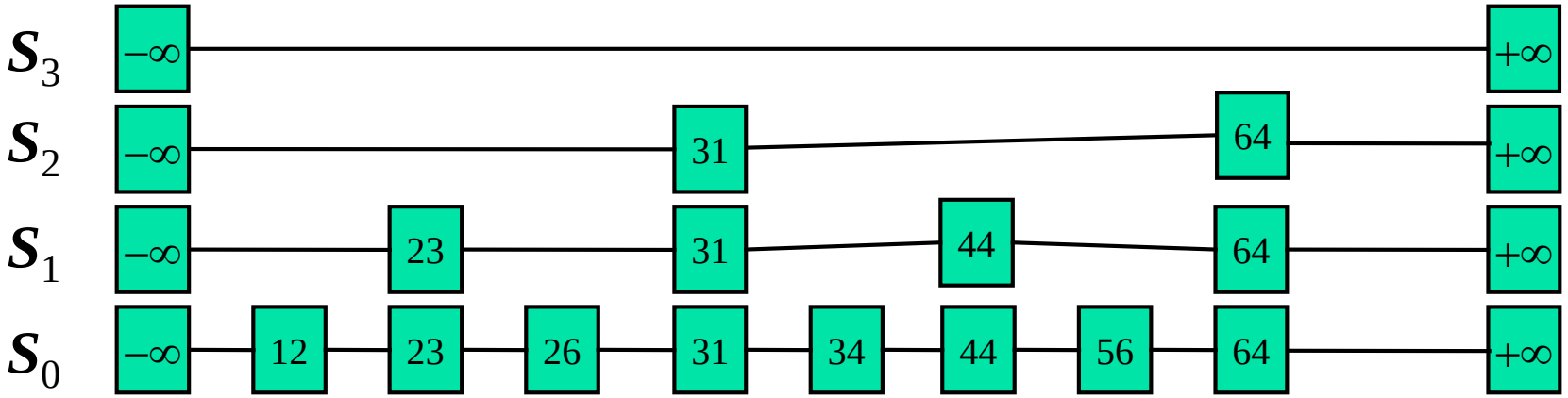
https://en.wikipedia.org/wiki/Skip_list

Perfect Skip List

- A skip list is a collection of lists at different levels
- The lowest level (0) is a sorted, singly linked list of all nodes
- The first level (1) links alternate nodes
- The second level (2) links every fourth node
- In general, level i links every 2^ith node
- In total, $\lceil \log_2 n \rceil$ levels (i.e. $O(\log_2 n)$ levels)
- Each level has half the nodes of the one below it

Perfect Skip List

- Example of a Perfect Skip List



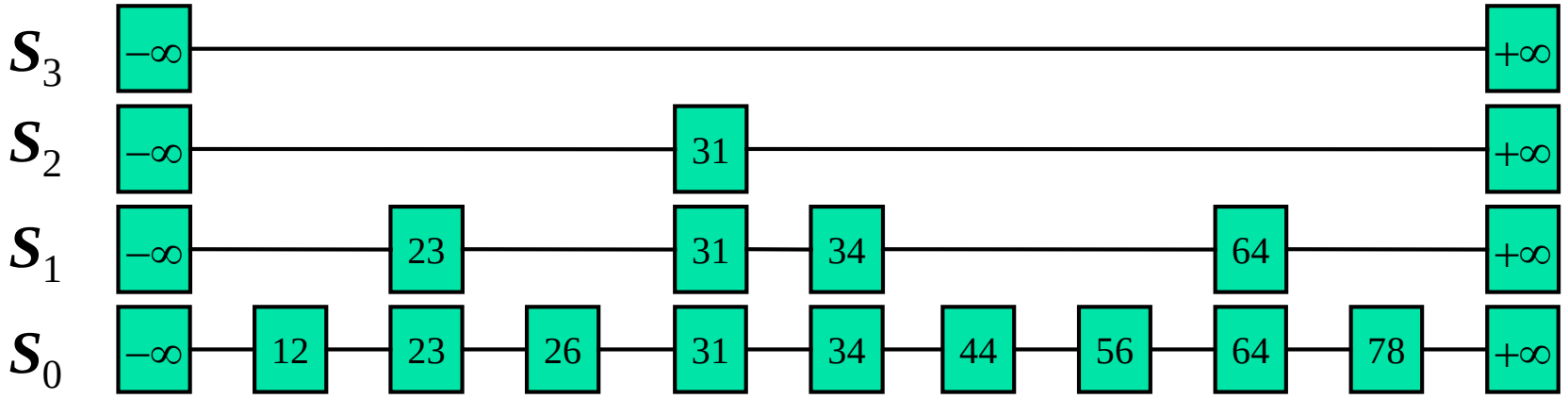
Insertions/Deletions

When we add a new node, our beautifully precise structure might become invalid

- We may have to change the level of every node
- One option is to move all the elements around
- But it takes O(n) time, which is back where we began
- Is it possible to achieve a net gain?

Randomized Skip List

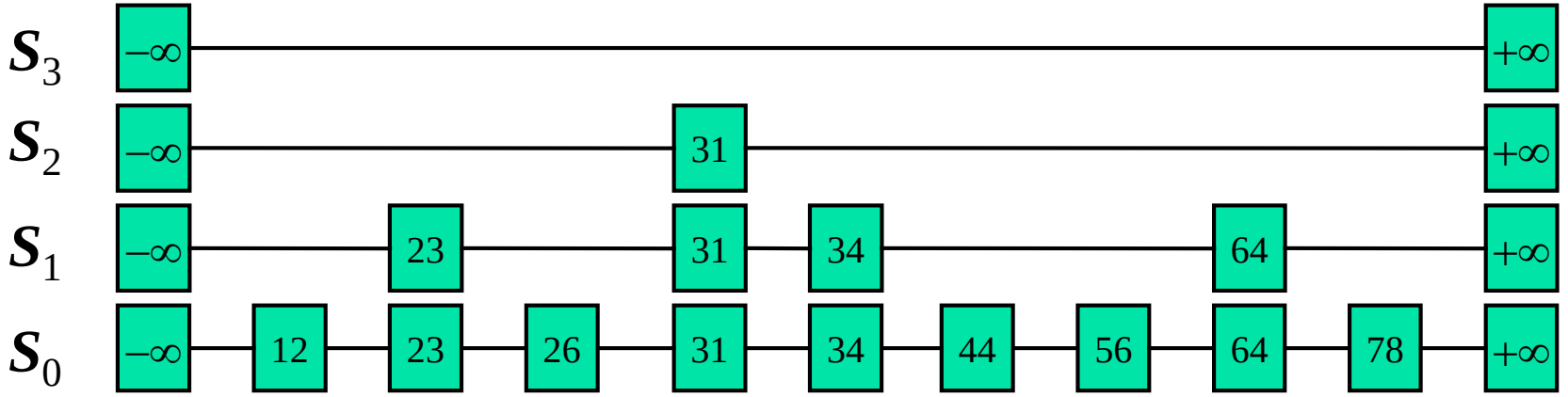
Example of a Randomized Skip List



Skip List

Skip List definition

- A skip list for a set S of distinct (key, element) items is a series of lists $S_0, S_1, \ldots, S_h$ such that:
  - Each list S_i contains the special keys $+\infty$ and $-\infty$
  - List S_0 contains the keys of S in non decreasing order
  - Each list is a subsequence of the previous one, i.e.,
    $$S_0 \supseteq S_1 \supseteq \ldots \supseteq S_h$$
  - List S_h contains only the two special keys

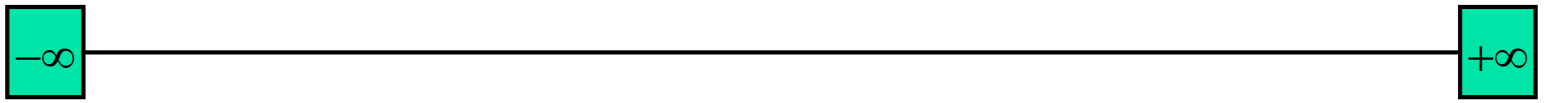


Skip List

Initialization

A new list is initialized as follows:

- A node NIL ($+\infty$) is created and its key is set to a value greater than the greatest key that could possibly used in the list
- Another node NIL ($-\infty$) is created, value set to lowest key that could be used



References

- Presentation Slides: Advanced Data Structures
  Dr. RamaMoorthy Srinath, Professor, CSE, PESU

Note: Apart from the links already mentioned on Slides

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES & ITS APPLICATIONS
UE21CS252A

Kusuma K V
Department of Computer Science & Engineering

DATA STRUCTURES & ITS APPLICATIONS

UNIT 1: Skip List

Kusuma K V
Department of Computer Science & Engineering

Skip List

Search

We search for a key x in a skip list as follows:

- We start at the first position of the top list
- At the current position p, we compare x with y i.e., key(after(p))

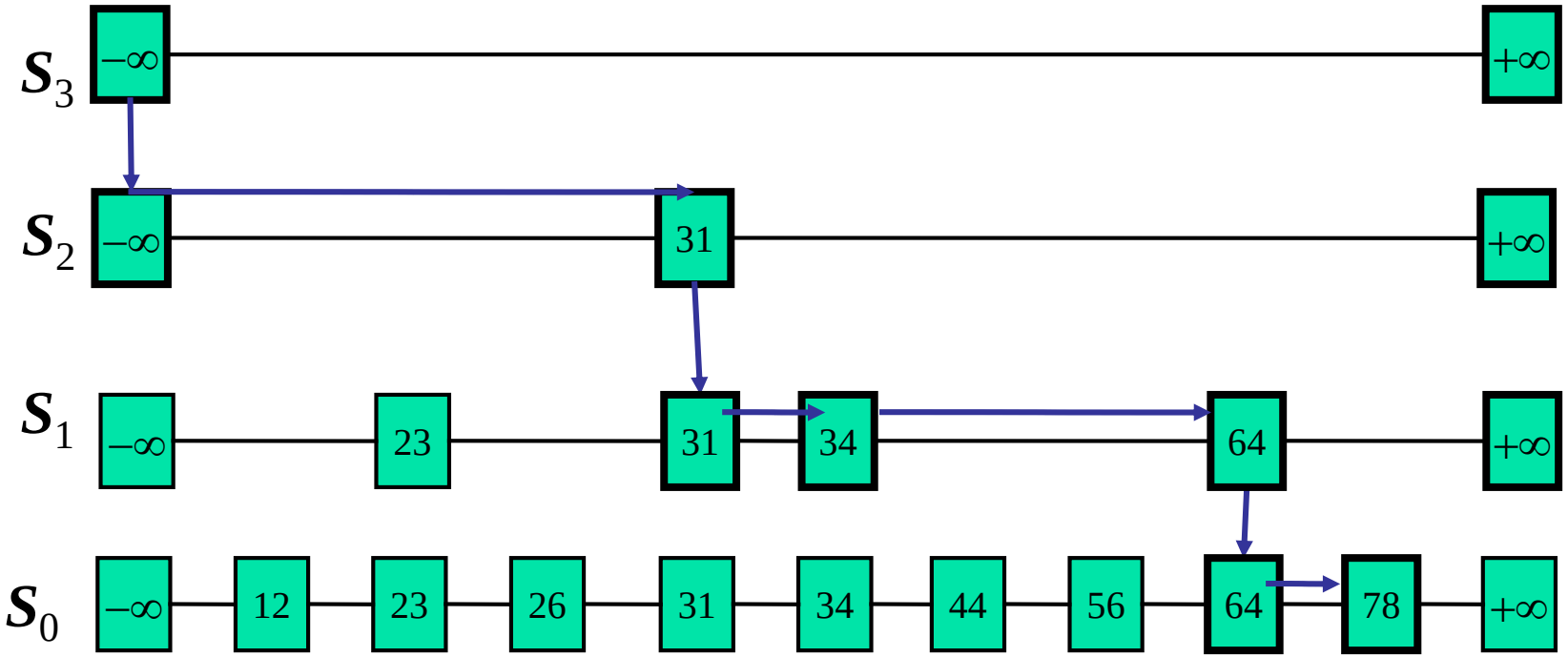
  x = y: we return element(after(p))

  x > y: we “scan forward”

  x < y: we “drop down”
- If we try to drop down past the bottom list, we return NO_SUCH_KEY
- Example: search for 78

Skip List

Searching in Skip List Example



Skip List

Insertion

- The insertion algorithm for skip lists uses randomization to decide how many references to the new item should be added to the skip list
- We then insert the new item in this bottom-level list at its appropriate position. After inserting the new item at this level we “flip a coin”
  - If the flip comes up tails, then we stop right there.
  - If the flip comes up heads, we move to next higher level and insert in this level at the appropriate position

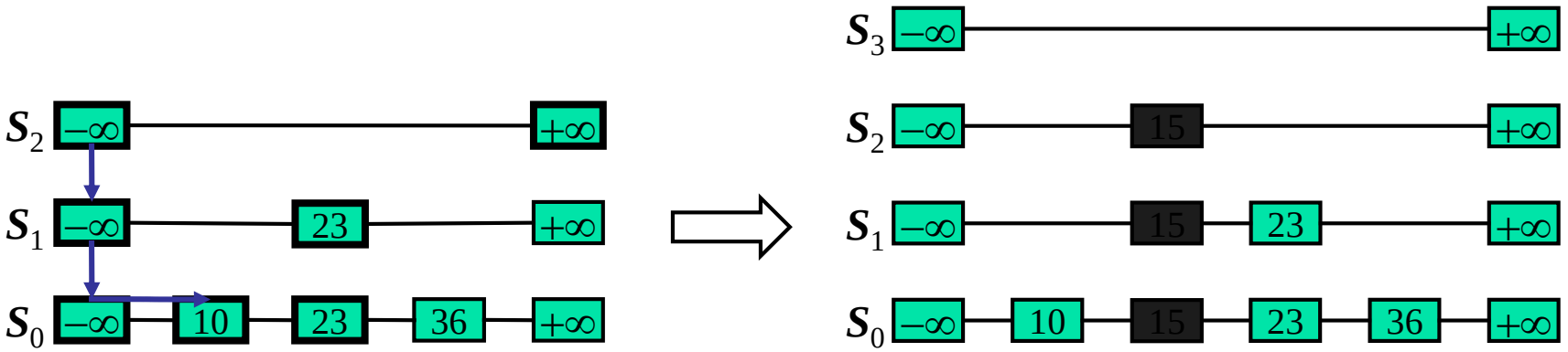
Randomized Algorithms

- A randomized algorithm performs coin tosses (i.e., uses random bits) to control its execution
- Its running time depends on the outcomes of the coin tosses
- We analyze the expected running time of a randomized algorithm under the following assumptions
  - the coins are unbiased, and
  - the coin tosses are independent
- The worst-case running time of a randomized algorithm is large but has very low probability (e.g., it occurs when all the coin tosses give “heads”)

Skip List

Insertion in Skip List Example

- Suppose we want to insert 15
- Do a search, and find the spot between 10 and 23
- Suppose the coin comes up “head” two times



Skip List

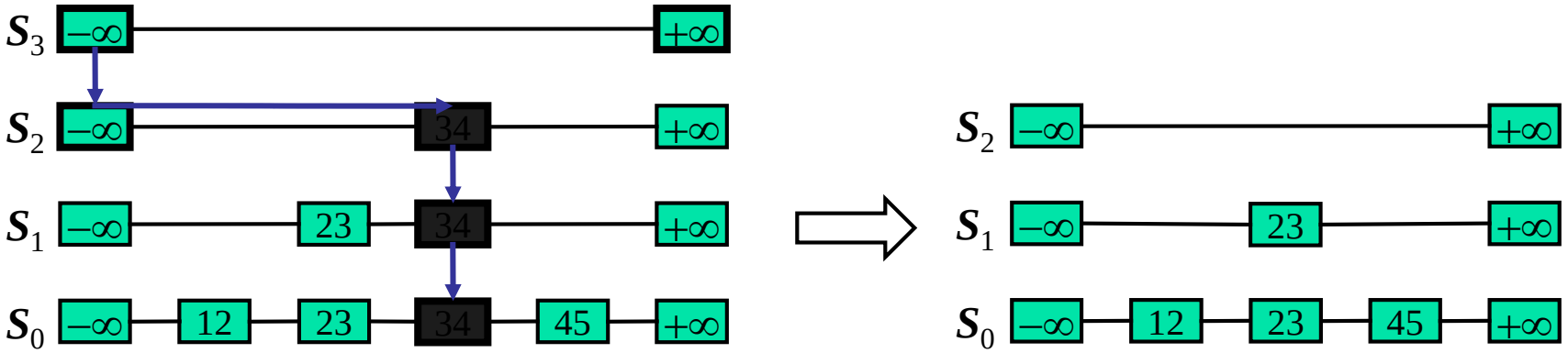
Deletion

- We begin by performing a search for the given key k.
- If a position p with key k is not found, then we return the NO_SUCH_KEY element
- Otherwise, if a position p with key k is found (it would be found on the bottom level), then we remove all the position above p
- If more than one upper level is empty, remove it

Skip List

Deletion in Skip List Example

- Suppose we want to delete 34
- Do a search, find the spot between 23 and 45
- Remove all the position above p



Skip List

Starting with an empty skip list, insert these keys, with these "randomly generated" levels

- 5, with level 1
- 26, with level 1
- 25, with level 4
- 6, with level 3
- 21, with level 0
- 3, with level 2
- 22, with level 2

Note that the ordering of the keys in the skip list is determined by the value of the keys only; the levels of the nodes are determined by the random number generator only

Skip List

5 with level 1, 26 with level 1, 25 with level 4, 6 with level 3
21 with level 0, 3 with level 2, 22 with level 2

PES UNIVERSITY ONLINE

References

- Presentation Slides: Advanced Data Structures
  Dr. RamaMoorthy Srinath, Professor, CSE, PESU

Note: Apart from the links already mentioned on Slides

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

Data Structures and its Applications

Dinesh Singh
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

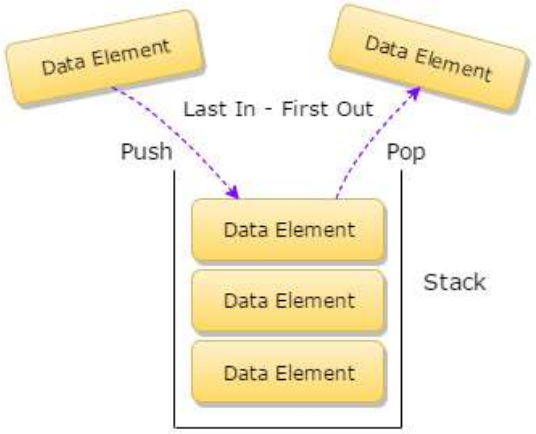
Stacks

Dinesh Singh
Department of Computer Science & Engineering

Data Structures and its Applications

Stacks - Definition

- A Stack is a data Structure in which all the insertions and deletions of entries are at one end. This end is called the TOP of the stack.
- When an item is added to a stack it is called push into the stack
- When an item is removed it is called pop from the stack.
- The Last item pushed onto a stack is always the first that will be popped from the stack.
- This property is called the *last in, first out* or LIFO for short



Data Structures and its Applications

Stacks – Representation in C

A stack in C is declared as a structure containing two objects :

- An array to hold the elements of the stack
- An Integer to indicate the position of the current stack top within the array
- Stack of integers can be done by the following declaration

```
#define STACKSIZE 100
struct stack
{
  int top;
  int items[STACKSIZE]
};
```

Once this is done, actual stack can be declared by

```
struct stack s;
```

Data Structures and its Applications

Stacks – Representation in C

Items need not be restricted to integers, items can be of any type.

A stack can contain items of different types by using C unions.

```
#define STACKSIZE 100
#define INT 1
#define FLOAT 2
#define STRING 3
struct stackelement {
    int etype;
    union{
        int ival;
        float fval;
        char *pavl; //pointer to string
    } element;
};
```

Stacks – Representation in C

```
struct stack
{
  int top;
  struct stackelement items[STACKSIZE];
};
```

- The above declaration defines a stack whose items can either be integers, floating point numbers or string depending on the value of etype (previous slide).

Data Structures and its Applications

Stacks – Implementation of operations of stack

Operations on stack

- Inserting an element on to the stack : push
- Deleting an element from the stack : pop
- Checking the top element : peep
- Checking if the stack is empty : empty
- Checking if the stack is full : overflow

Representation of stack will be as follows

```
#define STACKSIZE 100
struct stack
{
  int top;
  int items[STACKSIZE]
};
```

Stacks – Implementation of operations of stack

```
void push(struct stack *ps, int x)
/*ps is pointer to the structure representing stack, x is integer to be inserted
top is integer that indicates the position of the current stack top within the
      array, items is an integer array that represents stack, STACK_SIZE is the
      maximum size of the stack */
{
   if (ps->top == STACKSIZE -1) //check if the stack is full
         printf("STACK FULL Cannot insert..");
  else
   {
      ++(ps->top); //increment top
      ps->items[ps->top]=x; //insert the element at a location top
    }
}
```

Stacks – Implementation of operations of stack

```
int pop(struct stack *ps )
/*ps is pointer to the structure representing stack, top is integer that indicates
      the position of the current stack top within the array , items is an integer
      array that represents stack, STACK_SIZE is the maximum size of the stack */
  {
   if (ps->top == -1) // check if the stack is the empty
          printf("STACK EMPTY Cannot DELETE..");
  else
   {
      x=ps->items[ps->top]; //delete the element
      --(ps->top); //decrement top
     return x;
    }
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack

```
int display(struct stack *ps )
/*ps is pointer to the structure representing stack, top is integer that indicates
      the position of the current stack top within the array , items is an integer
      array that represents stack, STACK_SIZE is the maximum size of the stack */
  {
   if (ps->top == -1) // check if the stack is the empty
          printf("STACK EMPTY ");
  else
   {
      for (i=ps->top;i>=0;i--) // displays the elements from top
         printf("%d",ps->items[i]);
 }
}
```

Stacks – Implementation of operations of stack

```
int peep(struct stack *ps )
{
   if (ps->top == -1)
          printf("STACK EMPTY ..");
  else
   {
      x=ps->items[ps->top]; //get the element
      return x;
    }
}
```

Stacks – Implementation of operations of stack

```
int empty(struct stack *ps )
{
    if (ps->top == -1)
            return 1;
   return 0;
}
int overflow(struct stack *ps)
{
    if (ps->top==STACKSIZE-1)
      return 1;
  return 0;
}
```

Stacks – Implementation of operations of stack

implementation of stack operations where the items array and top are separate variables (Not part of structure)

```
void push(int *s, int *top, int x)
{
  if(*top==STACKSIZE-1)//check if the stack is full
  {
    printf("stack overflow..cannot insert");
   return 0;
 }
 else
  {
    ++*top; //increment the top
    s[*top]=x; //insert the element
  }
 return 1;
}
```

Stacks – Implementation of operations of stack

- Implementation of stack operations where the items and the top are separate variables (Not part of structure)

```
int pop(int *s, int *top)
{
  if(*top==-1)//check if the stack is empty
  {
    printf("Stack empty .. Cannot delete");
   return -1;
 }
 else
  {
    x=s[*top]; //insert the element
    --*top; //decrement top
   return x; // return the deleted element
  }
}
```

Stacks – Implementation of operations of stack

- Implementation of stack operations where the items and the top are separate variables (Not part of structure)

```
display(int *s, int *top)
{
   if(*top==-1)
    printf("Empty stack");
 else
  {
   for(i=*top;i>=0;i--) // display the elements from the top
     printf("%d",s[i]);
  }
}
```

Stacks – Application of Stack

- Write an algorithm to determine if an input character string is of the form x C y where x is a string consisting of the letters 'A' and 'B' and where y is the reverse of x. At each point you may read only the character of the string

```
 int check(t)
//the function returns 1 if string t is of the form x C y, else returns 0
//uses stack s and its operations push and pop
 {
    i=0;
    while(t[i]!='C') //push all the characters of the string into the stack
until C  is encountered
    {
      push(&s, t[i]);
      i=i+1;
    }
```

```
      //pop the contents of the stack and compare with t[i] until the end of  the string
while(t[i]!='\0')
{
   x=pop(&s);
   if(t[i]!=x) //if the character popped out is not equal to the character read from the string
      return 0; // not of the form xCy
   i=i+1;
 }
 return 1; // string of the form
 }
```

THANK YOU

Dinesh Singh
Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

Data Structures and its Applications

Dinesh Singh
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Stacks – Linked List Implementation

Dinesh Singh
Department of Computer Science & Engineering

Data Structures and its Applications

Stacks – linked list implementation

- A stack can be easily implemented through the linked list. In the Implementation by a linked list, the stack contains a top pointer. which is “head” of the stack. The pushing and popping of items happens at the head of the list.

Structure of the stack

```
struct node
{
   int data;
   struct node *next;
 }
struct node *top
top=NULL;
```

Data Structures and its Applications

Stacks – linked list implementation

Note :

- Items of the stack represented as the linked list.
- Each item is a node
- Top is a pointer that points to the first node (top of the stack)
- Top is initially NULL (Empty stack)
- Insertion and deletion happens at the front of the list
- stack size is not limited.

Operations on the stack

- Push : Inserting an element at the front of the list
- Pop : delete an element from the front of the list
- Display : displaying the list

Stacks – linked list implementation

```
//implements the push operation
void push(int x, struct node **top)
   {
        struct node *temp;
        temp=(struct node*)malloc(sizeof(struct node));
        temp->data=x;
        temp->next=*top; // insert in front of the list
        *top=temp; // make top points to the new top node
    }
```

Stacks – linked list implementation

```
//implements the pop operation
//returns top element, -1 if stack is empty
int pop(struct node **top)
    {
        int x;
       struct node *q;

       if(*top==NULL)
        {
          printf("Empty Stack\n");
           return -1;
         }
```

```
//implements the pop operation

else
        {
          q=*top;
           x=q->data; // get the top element
          *top=q->next; // make top point to the next top
          free(q); // free the memory of the node
           return(x);
         }
    }
```

Stacks – linked list implementation

```
//implements the display  operation
void display(struct node *top)
    {
       if(top==NULL)
         printf("Empty Stack\n");
       else
        {
         while(top!=NULL)
         {
           printf("%d->“,top->data);
           top=top->next;
         }
        }
    }
```

Stacks – linked list implementation

Another representation of structure of stack

```
struct node
{
  int data;
  struct node *next;
};
struct stack
{
   struct node *top;
}

struct stack s;
s.top=NULL;
```

Stacks – linked list implementation

```
//implements the push operation
void push(int x, struct stack * s)
//s is pointer to structure stack
   {
        struct node *temp;
        temp=(struct node*)malloc(sizeof(struct node));
        temp->data=x;
        temp->next=s->top; // insert in front of the list
        s->top=temp; // make top points to the new top node
    }
```

Stacks – linked list implementation

```
//implements the pop operation
//returns top element, -1 if stack is empty
int pop(struct stack *s)
    {
        int x;
       struct node *q;

       if(s->top==NULL)
        {
          printf("Empty Stack\n");
           return -1;
         }
```

Stacks – linked list implementation

```
//implements the pop operation

else
        {
          q=s->top;
           x=q->data; // get the top element
          s->top=q->next; // make top point to the next top
          free(q); // free the memory of the node
           return(x);
         }
    }
```

Stacks – linked list implementation

```
//implements the display  operation
void display(struct stack *s)
   {
      struct node *q;
      if(s->top==NULL)
        printf("Empty Stack\n");
      else
       {
        q=s->top;
        while(q!=NULL)
        {
          printf("%d->“,q->data);
          q=q->next;
         }
       }
   }
```

Stacks – Application

Write an Algorithm to print a string in the reverse order

```
//prints the text in a reverse order
reverse(t)
{
 i=0;
 //push all the characters on to the stack
 while(t[i]!='\0')
 {
    push(s,t[i]);
    i=i+1;
 }
```

Stacks – Application

pop all the characters from the stack until the stack is empty

```
while(!empty(s))
 {
    x= pop(s);
    print(x)
   }
}
```

THANK YOU

Dinesh Singh
Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

Data Structures and its Applications

Dinesh Singh
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Application of stacks– Functions, nested functions and Recursion

Dinesh Singh
Department of Computer Science & Engineering

Application of stacks – Functions, nested functions

Activation record :

- When functions are called, The system (or the program) must remember the place where the call was made, so that it can return there after the function is complete.
- It must also remember all the local variables, processor registers, and the like, so that information will not be lost while the function is working.
- This information is considered as large data structure . This structure is sometimes called the invocation record or the activation record for the function call.

Data Structures and its Applications

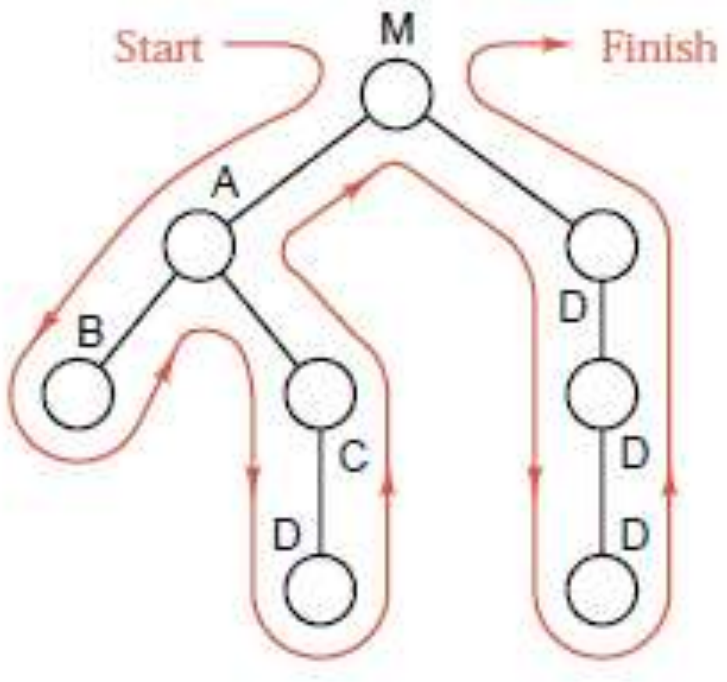
Application of stacks – Functions, nested functions and Recursion

- Suppose that there are three functions called A,B and C. M is the main function.
- Suppose that A invokes B and B invokes C. Then B will not have finished its work until C has finished and returned. Similarly, A is the first to start work, but it is the last to be finished, not until sometime after B has finished and returned.
- Thus the sequence by which function activity proceeds is summed up as the property last in, first out.
- The machine's task of assigning temporary storage area (activation records) used by functions would be in same order (LIFO).
- Since LIFO property is used, the machine allocates these records in the stack
- Hence a stack plays a key role in invoking functions in a computer system.

Application of stacks – Functions, nested functions

Example :

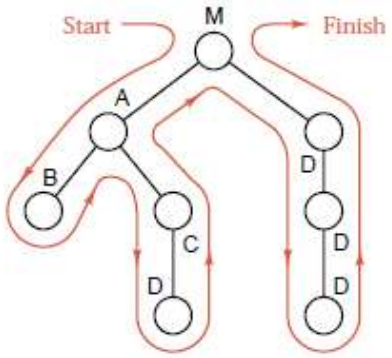
Consider the following showing the order in which the functions are invoked

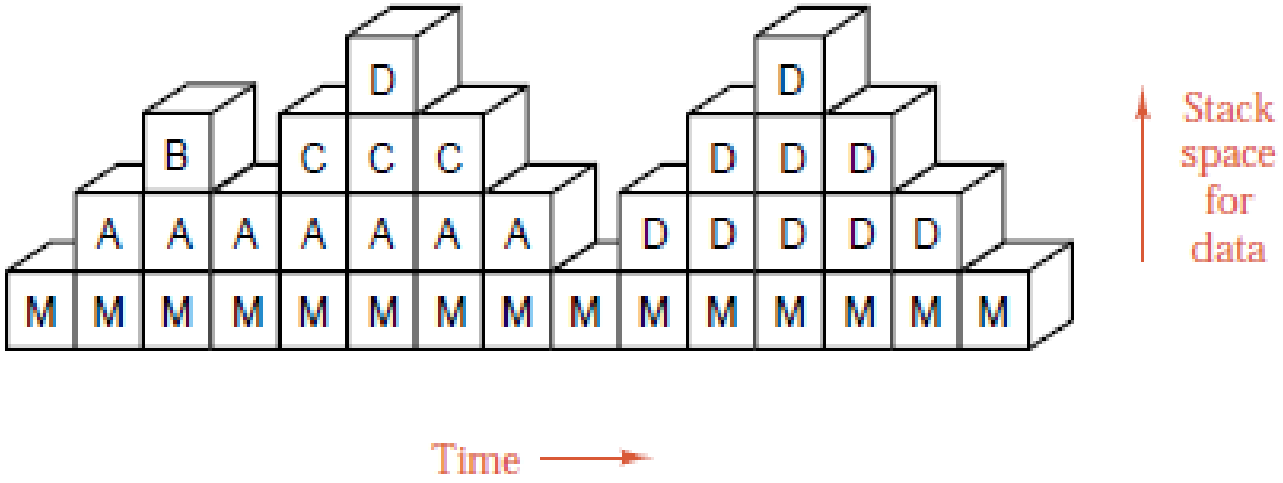


Data Structures and its Applications

Application of stacks – Functions, nested functions

The Sequence of stack frames of the activation records of the function calls is given below. Each vertical column shows the contents of the stack at a given time, and changes to the stack are portrayed by reading through the frames from left to right.





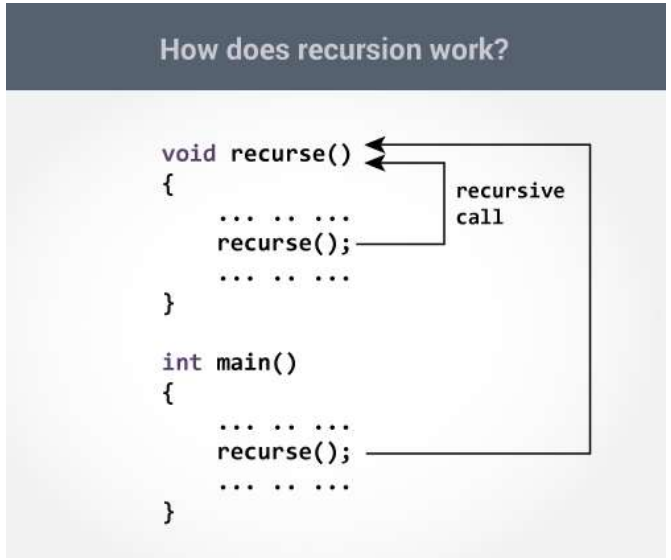
Data Structures and its Applications

Application of stacks – Recursion

Recursion :

- Recursion is a computer programming technique involving the use of a procedure, subroutine or a function.

- The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called as recursive function.



Application of stacks – Recursion

How a particular problem is solved using recursion?

- The idea is to represent a problem in terms of one or more smaller problems.

- A way is found to solve these smaller problems and then build up a solution to the entire problem.

- The sub problems are in turn broken down into smaller sub problems until the solution to the smallest sub problem is known.

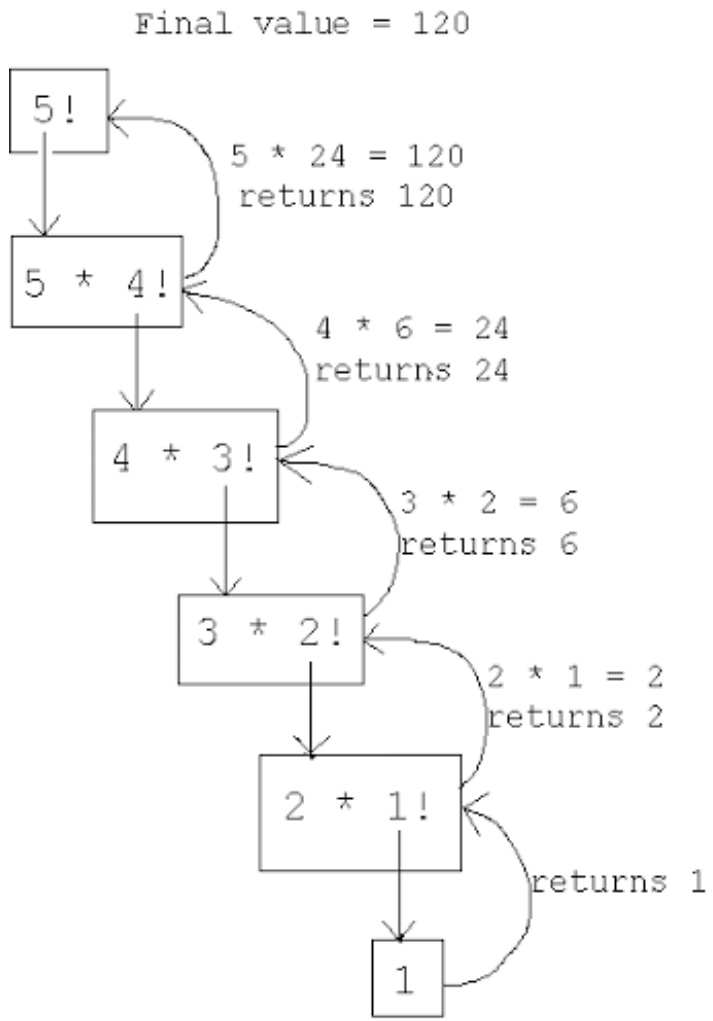
- The solution to the smallest sub problem is called the base case.

- In the recursive program, the solution to thebase case is provided

Data Structures and its Applications

Application of stacks – Recursion

Example 1: Factorial of a Number n



Recursive definition :

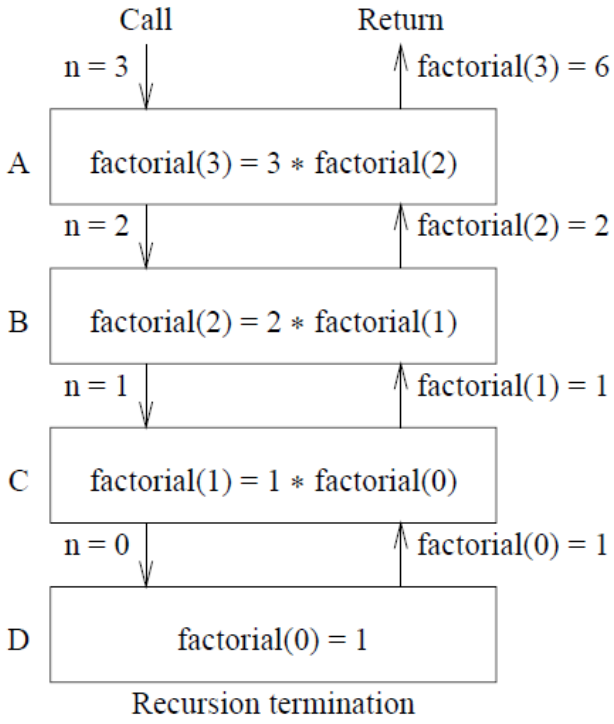
n! = 1 if n=1

n!= n * (n-1)! If n>1

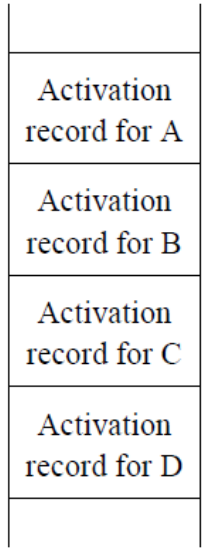
Application of stacks – Recursion

Recursive function to compute factorial of n

```
factorial(n)
{
  int f;
  if(n==0)
       return 1;
   f=n*factorial(n-1);
return f;
}
```



(a)



(b)

Application of stacks – Recursion

Recursive function to find the product of a*b

a*b= a if b=1;

a*b = a*(b-1)+a if b >1;

To evaluate 6 * 3

6*3 = 6*2 + 6 = 6*1 + 6 + 6 = 6 + 6 + 6 = 18

```
multiply(int a, int b)
{
   int p;
    if (b==1)
     return a
   p= multiply(a,b-1) + a;
  return p;
}
```

Application of stacks – Recursion

Fibonacci Sequence

The fibonacci sequence is the sequence of integers
1, 1, 2, 3, 5, 8, 13, 21, 34...

Each element is the sum of two preceding elements

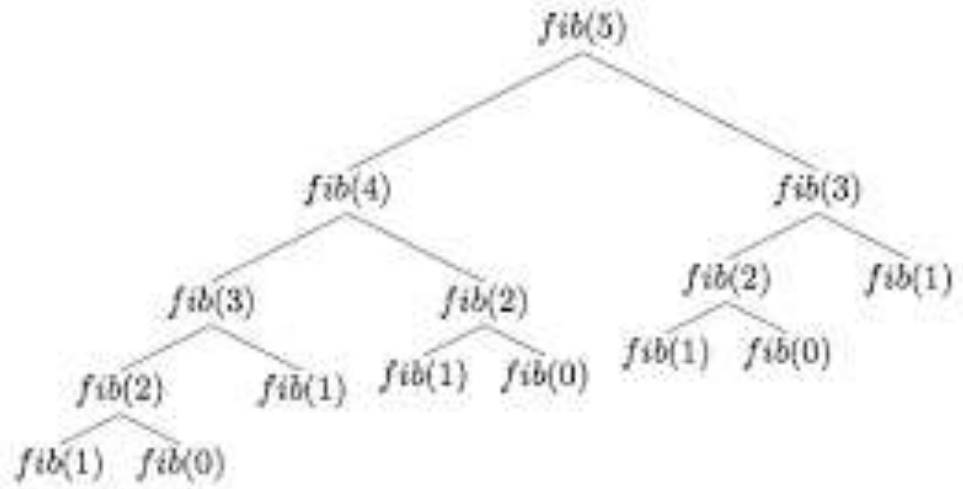
If we consider fib(0) = 1 and fib(1)=1
then recursive definition to compute the nth element in the sequence is

fib(n) = n if n=0 or n=1
fib(n) = fib(n-1) + fib(n-2) for n >=2

Application of stacks – Recursion

Recursive function to compute the nth element in the Fibonacci Sequence

```
 fib(int n)
{
   int x,y;
   if ( (n==0) || (n==1
     return n;
  x= fib(n-1) ;
  y=fib(n-2);
 return x+y;
}
```



Application of stacks – Recursion

Recursive function to find the sum of elements of an array

```
int sum(int *a, int n)
//a is pointer to the array, n is the index of the last element of the array
{
  int s;
  if(n==0) // base condition
   return a[0];
  s= sum(a,n-1) + a[n]; // compute sum of n-1 elements and add the nth element
  return s;
}
```

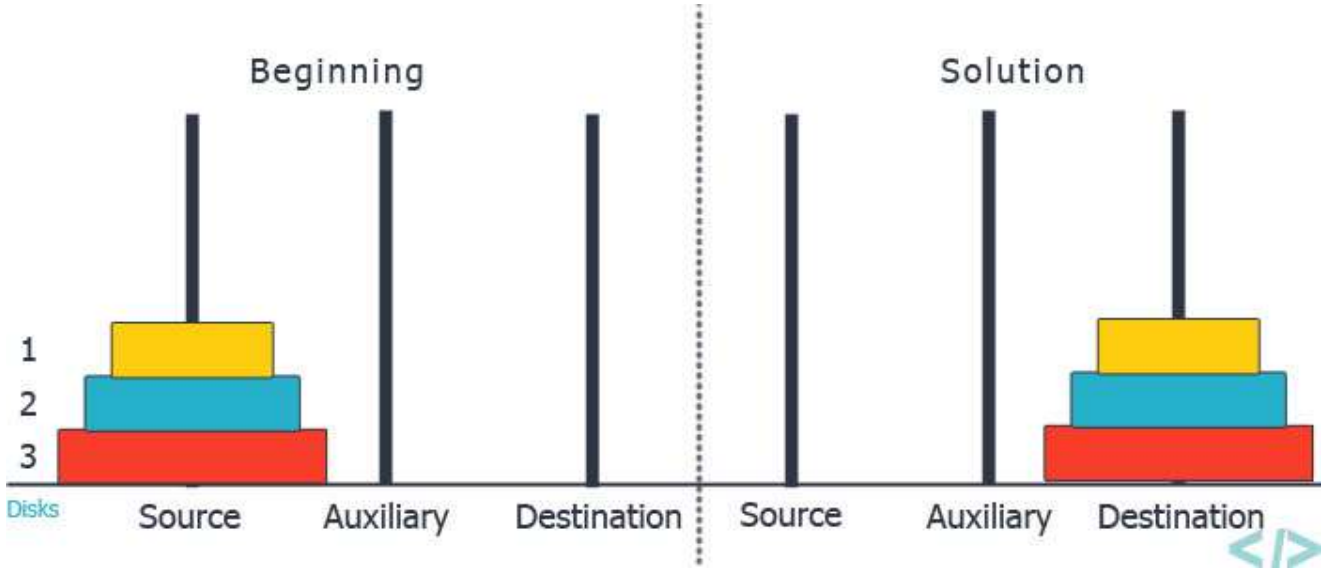
Application of stacks – Recursion

Recursive function to display the elements of the linked list in the reverse order

```
int display(struct node *p)
{
  if(p->next!=NULL)
     display(p->next)
  printf("%d ",p->data);
}
```

Tower of Hanoi

Three Pegs A, B and C exists. N disks of differing diameters are placed on peg A. The Larger disk is always below a smaller disk. The objective is to move the N disks from Peg A to Peg C using Peg B as the auxillary peg



Application of stacks – Recursion

Tower of Hanoi – recursive solution

If a solution to n-1 disks is found, then the problem would be solved. Because in the trivial case of one disk, the solution would be to move the single disk from Peg A to Peg C.

To move n disks from A to C , the recursive solution would be as follows

1. If n=1 move the single disk from A to C and stop
2. Move the top n-1 disks from A to B using C as auxillary
3. Move the remaining disk from A to C
4. Move n-1 disks from B to C using A as the auxillary

Application of stacks – Recursion

Recursive function for Tower of Hanoi

```
void tower(int n,char src,char tmp,char dst)
 {
  if(n==1)
  {
   printf("\nMove disk %d from %c to %c",n,src,dst);
   return;
  }
  tower(n-1,src,dst,tmp);
  printf("\nMove disk %d form %c to %c",n,src,dst);
  tower(n-1,tmp,src,dst);
  return;
 }
```

Application of stacks – Recursion

Recursive function for Tower of Hanoi

Solution for 4 disks

Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C
Move disk 3 from peg A to peg B
Move disk 1 from peg C to peg A
Move disk 2 from peg C to peg B
Move disk 1 from peg A to peg B

Solution for moving 3 disks from A to B

Move disk 4 from peg A to peg C

Move 4^{th} disk from A to C

Move disk 1 from peg B to peg C
Move disk 2 from peg B to peg A
Move disk 1 from peg C to peg A
Move disk 3 from peg B to peg C
Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C

Solution for moving 3 disks from B to C

THANK YOU

Dinesh Singh
Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

Data Structures and its Applications

Dinesh Singh
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Infix to Postfix and Prefix Expressions – Implementation

Dinesh Singh
Department of Computer Science & Engineering

Infix , Postfix and Prefix Expressions

- Consider the sum of A and B expressed as A + B
- This representation is called infix.
- There are two alternate notations for expressing sum of A and B using the symbols A , B and + , these are
  - Prefix : + A B
  - Postfix : A B +
- The prefixes “Pre” , “post” and “in” refers to the relative position of the operator with respect to the two operands.
- In prefix, operators precedes the two operands
- In postfix, the operator follows the two operands
- In infix, the operator is in between the two operands

Infix , Postfix and Prefix Expressions

Conversion of Infix to Postfix

- For Example : consider the expression A + B * C
- The evaluation of the above expression requires the knowledge of precedence of operators
- A + B * C can be expressed as A + (B * C) as multiplication takes precedence over addition
- Applying the rules of precedence the above infix expression can be converted to postfix as follows

A + B * C = A + (B * C)

= A + (BC *) convert the multiplication

= A (B C *) + convert the addition

= A B C * +

Infix , Postfix and Prefix Expressions

Conversion of Infix to Prefix

- For Example : consider the expression A + B * C
- Applying the rules of precedence the above infix expression can be converted to prefix as follows

A + B * C = A + (B * C)

= A + (* B C) convert the multiplication

= + A (* B C) + convert the addition

= + A * B C

Infix , Postfix and Prefix Expressions

Applying the rules of precedence the table shows the conversion of Infix to Postfix and Prefix Expression

Infix	Postfix	Prefix
A + B * C + D	A B C * + D +	+ + A * B C D
(A + B) * (C + D)	A B + C D + *	* + A B + C D
A * B + C * D	A B * C D * +	+ * A B * C D
A + B + C + D	A B + C + D +	+ + + A B C D
A $ B * C – D + E / F / (G+H)	A B $ C * D – E F / G H + / +	+ - * $ A B C D / /E F + G H
((A + B)*C – (D – E)) $ (F + G)	A B + C * D E – - F G + $	$ - * + A B C – D E + F G
A – B / (C * D $ E)	A B C D E $ * / -	- A / B * C $ D E

$ is the exponentiation operator and its precedence is from right to left

For example A $ B $ C = A $ (B $ C)

Data Structures and its Applications

Infix to postfix conversion - algorithm

```
opstk is the empty stack
while(not end of input)
{
  symb = next input character
// if the input symbol is an operand , add it to the postfix string
 if(symb is an operand)
    add symb to postfix string
else
   {
     // pop the contents of the stack while the precedence of
    // top of the stack is greater than the precedence of the symbol scanned
    //prcd function returns true in this case
      while(!empty(opstk ) and ( prcd(stacktop(opstk),symb))
       {
          topsymb=pop(opstk)
         add topsymb to postfix string
       }
```

Infix to postfix conversion - algorithm

```
   /* if prcd function returns false, then
if  the stack is empty and symbol is not ‘)’, push symb on to the stack*/
        if( empty(opstk) || symb!=‘)’)
            push(opstk,symb)
      else
          topsymb=pop(opstk); // if symb is ‘)’ ,  pop ‘ ( ‘ from the stack
}
while(!empty(opstk)
 {
   topsymb=pop(opstk)
   add topsymb to postfix string
 }
}
```

Infix to postfix conversion - algorithm

- prcd(OP1,OP2) is a function that compares the precedence of the top of the stack(OP1) and the input symbol (OP2) and returns TRUE if the precedence is greater else returns FALSE.
- Some of the return values of prcd function
  - prcd(‘ * ‘ , ‘ + ‘) returns TRUE : prcd(‘ * ’, ’ / ’) returns TRUE
  - prcd(‘ + ‘, ‘ * ‘) returns FALSE : prcd(‘ / ’, ’ * ’) returns TRUE
  - prcd(‘ + ‘ , ‘ + ‘) returns TRUE :
  - prcd(‘ + ‘ , ‘ – ‘) returns TRUE
  - prcd(‘ – ‘ , ‘ – ‘) returns TRUE
  - prcd(‘ $ ‘ , ‘$’) returns FALSE : exponentiation operator, associatively from right to left
  - prcd(‘ (‘, op) , prcd(op ‘ (‘) returns FALSE for any operator
  - prcd (op , ‘)‘) returns TRUE for any operator
  - Prcd(‘ (‘ , ‘) ‘) returns FALSE

Data Structures and its Applications

Infix to postfix conversion - algorithm

- The motivation behind the conversion algorithm is the desire to output the operators in the order in which they are to be executed.
- If an incoming operator is of greater precedence than the one on top of the stack, this new operator is pushed on to the stack.
- If on the other hand , the precedence of the new operator is less than the one on the top of the stack, the operator on the top of the stack should be executed first.
- Therefore the top of the stack is popped out and added to the postfix and the incoming symbol is compared with the new top and so on.
- Parenthesis in the input string override the order of operations
- When a left parenthesis is scanned, it is pushed on to the stack
- When its associated right parenthesis is found, all the operators between the two parenthesis are placed on the output string. Because they are to be executed before any operators appearing after the parenthesis

Infix to postfix conversion – Trace of the algorithm

PES UNIVERSITY ONLINE

Trace of the algorithm for A + B * C

symb	postfix string	opstk	Remarks
A	A	Empty	symb is operand , add 'A'on to the postfix string
+	A	+	symb is operator, and stack empty, push + on to the stack
B	AB	+	symb is operand , Add 'B' to the postfix string
*	AB	+*	symb is operator, precedence of + (stack top) is < than precedence of *, therefore push * on to the stack

Data Structures and its Applications

Infix to postfix conversion – Trace of the algorithm

Trace of the algorithm for A + B * C

symb	postfix string	opstk	Remarks
C	ABC	+ *	symb is operand , add C on to the postfix string
End of input	ABC*	+	Pop * from the stack and add to the postfix string
-	ABC*+	Empty	Pop + from the stack and add to the postfix string

Data Structures and its Applications

Infix to postfix conversion – Trace of the algorithm

PES UNIVERSITY ONLINE

Trace of the algorithm for (A + B)* C

symb	postfix string	opstk	Remarks
(	-	Empty	Stack empty, push '(' on to the stack
A	A	(	Symb is an operand, add 'A' to the post fix string
+	A	(+	Precedence of top of the stack '(' is less than '+' , push '+' on to the stack
B	AB	(+	Symb is an operand, add B to the postfix string
)	AB+	(	Precedence of stack top '+' is > than ')' Pop '+' from the stack and add to the postfix string
	AB+	empty	Precedence of stack top stack top '(' is not greater than ')' and symb is ')', therefore pop '(' from the stack

Infix to postfix conversion – Trace of the algorithm

Trace of the algorithm for (A + B)* C

symb	postfix string	opstk	Remarks
*	AB+	*	Stack empty, push '*' on to the stack
C	AB+C	*	Symb is an operand, add 'C' to the postfix string
End of string	AB+C*	Empty	End of input, pop * from the stack and add to the postfix.

Infix to posfix conversion – another algorithm

```
void convert_postfix(char *infix,char*postfix)
 {
    i=0;
   char ch;
   j=0;
   push(s,&top,'#');
   while(infix[i]!='\0')
   {
     ch=infix[i];
   //while the prcedence of top of stack is greater than the
//precedence of the input symbol, pop and add to the postfix
    while(stack_prec(peep(s,top))>input_prec(ch))
        postfix[j++]=pop(s,&top);
```

Infix to posfix conversion – another algorithm

```
if(input_prec(ch)!=stack_prec(peep(s,top)))
        push(s,&top,ch);
      else
        pop(s,&top);
    i++;
   }
   while(peep(s,top)!='#')
     postfix[j++]=pop(s,&top);
   postfix[j]='\0';
 }
```

Infix to posfix conversion – another algorithm

Precedence table

Operator	Input Precedence	Stack Precedence
+ , -	1	2
* , /	3	4
$	6	5
Operands	7	8
)	0	- : Never pushed on to stack
(	9	0
#	-	-1

Data Structures and its Applications

Infix to prefix conversion – algorithm

Example 1:
Input : A * B + C / D
Output : + * A B/ C D

Example 2 :
Input : (A - B/C) * (D/K-L)
Output : *-A/BC-/DKL

1: Reverse the infix expression i.e A+B*C will become C*B+A.
Note while reversing each '(' will become ')' and each ')' becomes '('.

2: Obtain the postfix expression of the modified expression i.e CB*A+.

3: Reverse the postfix expression. Hence in our example prefix is +A*BC.

THANK YOU

Dinesh Singh
Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

Data Structures and its Applications

Dinesh Singh
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Evaluation of Postfix expression and Parenthesis matching

Dinesh Singh
Department of Computer Science & Engineering

Evaluation of Postfix Expression - Algorithm

- Each operator in a postfix string refers to the previous two operands.
- Each time an operand is read , it is pushed on to the stack
- When an operator is reached, its operands will be the top two elements on to the stack.
- The two elements are popped out, the indicated operation is performed on them and result is pushed on the stack so that it will be available for use as an operand of the next operator.

Evaluation of Postfix Expression - Algorithm

```
opndstk is the empty stack
while(not end of the input) // scan the input string
{
 symb=next input character;
 if (symb is an operand)
   push(opndstk,symb)
else
   {
    opnd2=pop(opndstk);
    opnd1=pop(opndstk);
    value = result of applying symb to opnd1 and opn2;
   push(opndstk, value);
  }
 return(pop(opndstk));
}
```

Evaluation of Postfix Expression – Trace of the Algorithm

Infix : 3 + 5 * 4

Postfix expression : 3 5 4 * +

Symb	Opnd1	Opnd2	Value	opndstk
3	-			3
5				3, 5
4				3 , 5 , 4
*	5	4	20	3 , 20
+	3	20	60	60

Evaluation of Postfix Expression - implementation

```
int postfix_eval(char* postfix)
 {
  int i,top,r;
  int s[10];//stack
  top=-1;
  i=0;

  while(postfix[i]!='\0')
  {
    char ch=postfix[i];
    if(isoper(ch))
    {
      int op1=pop(s,&top);
      int op2=pop(s,&top);
```

Evaluation of Postfix Expression - implementation

```
switch(ch)
      {
        case '+':r=op1+op2;
                 push(s,&top,r);
                 break;
        case '-':r=op2-op1;
                  push(s,&top,r);
                  break;
         case '*':r=op1*op2;
                 push(s,&top,r);
                  break;
        case '/':r=op2/op1;
                  push(s,&top,r);
                 break;
       }//end switch
    }//end if
```

Evaluation of Postfix Expression - implementation

```
else
      push(s,&top,ch-'0');//convert charcter to
integer and push
     i++;
    }  //end while
  return(pop(s,&top));
 }
```

Parenthesis Matching – overview of the Algorithm

PES UNIVERSITY ONLINE

Examples

1. (( )) : Valid Expression
2. ((( )) : Invalid Expression (Extra opening parenthesis)
3. (( ))) : Invalid Expression (Extra closing parenthesis)
4. ((}) : Invalid Expression (Parenthesis mismatch)
5. (( )] : Invalid Expression (Parenthesis mismatch)

Parenthesis Matching – overview of the Algorithm

1.Read the input symbol from the input expression
2. If the input symbol is one of the open parenthesis (‘(‘ , ‘ { ‘ or ‘ [‘), it is pushed on to the stack
3. If the input symbol is of closing parenthesis, stack is popped and the popped parenthesis is compared with the input symbol, if there is a mismatch in the type of the parenthesis, return 0 (Mismatch of parenthesis)
4. If there is a match in the parenthesis , the next input symbol is read.
5. If during this process, if the stack becomes empty, return 0 (Extra closing parenthesis)
6. If at the end of the expression, if the stack is not empty, return 0 (Extra opening parenthesis)
7. If at the end of the input expression, if the stack is empty, return 1. (Parenthesis are matching)

Data Structures and its Applications

Parenthesis Matching - Implementation

```
int match(char *expr)
 {
   int i,top;
   char s[10],ch,x;//stack
   i=0;
    top=-1;

   while(expr[i]!='\0')
   {
     ch=expr[i];
     switch(ch)
     {
        case '(':
        case '{':
        case '[':push(s,&top,ch);
                 break;
```

Parenthesis Matching

```
case ')':if(!isempty(top))
                {
                  x=pop(s,&top);
                  if(x=='(')
                    break;
                  else
                    return 0;//mismatch of parenthesis
                }
                else
                  return 0;//extra closing parenthesis
```

Parenthesis Matching

```
case '}':if(!isempty(top))
         {
          x=pop(s,&top);
          if(x=='{')
            break;
          else
            return 0;//mismatch of parenthesis
         }
         else
          return 0;//extra closing parenthesis
```

Data Structures and its Applications

Parenthesis Matching

```
case ']':if(!isempty(top))
                {
                  x=pop(s,&top);
                  if(x=='[')
                    break;
                  else
                    return 0;//mismatch of parenthesis
                }
                else
                  return 0;//extra closing parenthesis

          }//end switch
          i++;
      }//end while
     if(isempty(top))
        return 1;
      return 0;//extra opening parenthesis
  }
```

THANK YOU

Dinesh Singh
Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Introduction to Algorithms, Design Techniques and Analysis

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Design and Analysis of Algorithms

Syllabus

- **UNIT I (12 Hours)**
  - Introduction
  - Analysis of Algorithm Efficiency,
  - Algebric structures
- **UNIT II (12 Hours)**
  - Brute Force,
  - Divide-and-Conquer
- **UNIT III (10 Hours)**
  - Decrease-and-Conquer
  - Transform-and-Conquer
- **UNIT IV (10 Hours)**
  - Space and Time Tradeoffs
  - Greedy Technique
- **UNIT V (12 Hours)**
  - Limitations of Algorithm Power
  - Coping with the Limitations of Algorithm Power
  - Dynamic Programming

Design and Analysis of Algorithms

Text Books

Book Type	Code	Title & Author	Publication Information		
			Edition	Publisher	Year
Text Book	T1	Introduction to The Design and Analysis of Algorithms Anany Levitin	2	Pearson	2012
Reference Book	R1	Introduction to Algorithms Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein	3	Prentice-Hall India	2009
Reference Book	R2	Fundamentals of Computer Algorithms Horowitz, Sahni, Rajasekaran,	2	Universities Press	2007
Reference Book	R3	Algorithm Design Jon Kleinberg, Eva Tardos,	1	Pearson Education	2006

Algorithm

Unambiguous – Each step must be precise and not open to multiple interpretations

What is an algorithm?

An algorithm is a sequence of unambiguous instructions for solving a problem,

i.e., for obtaining a required output for any legitimate input in a finite amount of time

Important Points about Algorithms

- The non-ambiguity requirement for each step of an algorithm cannot be compromised
- The range of inputs for which an algorithm works has to be specified carefully.
- The same algorithm can be implemented in several different ways
- There may exist several algorithms for solving the same problem.

Characteristics of Algorithm

Input: Zero or more quantities are externally supplied

Definiteness: Each instruction is clear and unambiguous

Finiteness: The algorithm terminates in a finite number of steps.

Effectiveness: Each instruction must be primitive and feasible

Output: At least one quantity is produced

Algorithm

Why do we need Algorithms?

- It is a tool for solving well-specified Computational Problem.
- Problem statement specifies in general terms relation between input and output
- Algorithm describes computational procedure for achieving input/output relationship This Procedure is irrespective of implementation details

Why do we need to study algorithms?

Exposure to different algorithms for solving various problems helps develop skills to design algorithms for the problems for which there are no published algorithms to solve it

Design and Analysis of Algorithms

Basic Issues related to Algorithms

- How to design algorithms
- How to express algorithms
- Proving correctness of designed algorithm
- Efficiency
  - Theoretical analysis
  - Empirical analysis

Design and Analysis of Algorithms

Basic Issues related to Algorithms

- How to design algorithms
- How to express algorithms
- Proving correctness of designed algorithm
- Efficiency
  - Theoretical analysis
  - Empirical analysis

Algorithm Design Technique

What do you mean by Algorithm Design Techniques?

General Approach to solving problems algorithmically .
Applicable to a variety of problems from different areas of computing

Various Algorithm Design Techniques

- Brute Force
- Divide and Conquer
- Decrease and Conquer
- Transform and Conquer
- Dynamic Programming
- Greedy Technique
- Branch and Bound
- Backtracking

Importance Framework for designing and analyzing algorithms for new problems

Design and Analysis of Algorithms

Basic Issues related to Algorithms

- How to design algorithms
- How to express algorithms
- Proving correctness of designed algorithm
- Efficiency
  - Theoretical analysis
  - Empirical analysis

Methods of Specifying an Algorithm

- **Natural language**
  - Ambiguous
- **Pseudocode**
  - A mixture of a natural language and programming language-like structures
  - Precise and succinct.
  - Pseudocode in this course
    - omits declarations of variables
    - use indentation to show the scope of such statements as for, if, and while.
    - use ← for assignment
- **Flowchart**
  - Method of expressing algorithm by collection of connected geometric shapes

Methods of Specifying an Algorithm

➢ Euclid's Algorithm

Problem: Find gcd(m,n), the greatest common divisor of two nonnegative, not both zero integers m and n

Examples: gcd(60,24) = 12, gcd(60,0) = 60, gcd(0,0) = ?

Euclid's algorithm is based on repeated application of equality

gcd(m,n) = gcd(n, m mod n)

until the second number becomes 0, which makes the problem trivial.

Example: gcd(60,24) = gcd(24,12) = gcd(12,0) = 12

Methods of Specifying an Algorithm

Two descriptions of Euclid's algorithm

Euclid's algorithm for computing gcd(m,n)

Step 1 If n = 0, return m and stop; otherwise go to Step 2

Step 2 Divide m by n and assign the value of the remainder to r

Step 3 Assign the value of n to m and the value of r to n. Go to step 1.

ALGORITHM Euclid(m,n)

//computes gcd(m,n) by Euclid's method

//Input: Two nonnegative,not both zero integers

//Output:Greatest common divisor of m and n

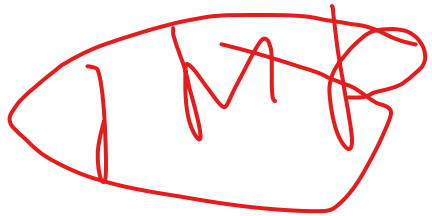
while n ≠ 0 do

    r ← m mod n

    m← n

    n ← r

return m



Algorithm for Sequential Search

ALGORITHM *SequentialSearch*$(A[0..n-1], K)$

```
//Searches for a given value in a given array by sequential search
//Input: An array A[0..n − 1] and a search key K
//Output: The index of the first element of A that matches K
//        or −1 if there are no matching elements
i ← 0
while i < n and A[i] ≠ K do
    i ← i + 1
if i < n return i
else return −1
```

Design and Analysis of Algorithms

Basic Issues related to Algorithms

- How to design algorithms
- How to express algorithms
- Proving correctness of designed algorithm
- Efficiency
  - Theoretical analysis
  - Empirical analysis

Design and Analysis of Algorithms

Basic Issues related to Algorithms

- How to design algorithms
- How to express algorithms
- Proving correctness of designed algorithm
- Efficiency
  - Theoretical analysis
  - Empirical analysis

Need of Analysis

time taken to map input to output

memory required to map input to output

- To determine resource consumption
  - CPU time
  - Memory space
- Compare different methods for solving the same problem before actually implementing them and running the programs.
- To find an efficient algorithm for solving the problem

Design and Analysis of Algorithms

Complexity of an algorithm

- A measure of the performance of an algorithm

- An algorithm's performance is characterized by

  - Time complexity

    How fast an algorithm maps input to output as a function of input

  - Space complexity

    amount of memory units required by the algorithm in addition to the memory needed for its input and output

PES UNIVERSITY ONLINE

How to determine complexity of an algorithm?

- Experimental study(Performance Measurement)
- Theoretical Analysis (Performance Analysis)

Limitations of Performance Measurement

- It is necessary to implement the algorithm, which may be difficult
- Results may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same hardware and software environments must be used
- Experimental data though important is not sufficient

Design and Analysis of Algorithms

Performance Analysis

- Uses a high-level description of the algorithm instead of an implementation
- Characterizes running time as a function of the input size, n.
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

THANK YOU

Vandana M L
Department of Computer Science & Engineering

vandanamd@pes.edu

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

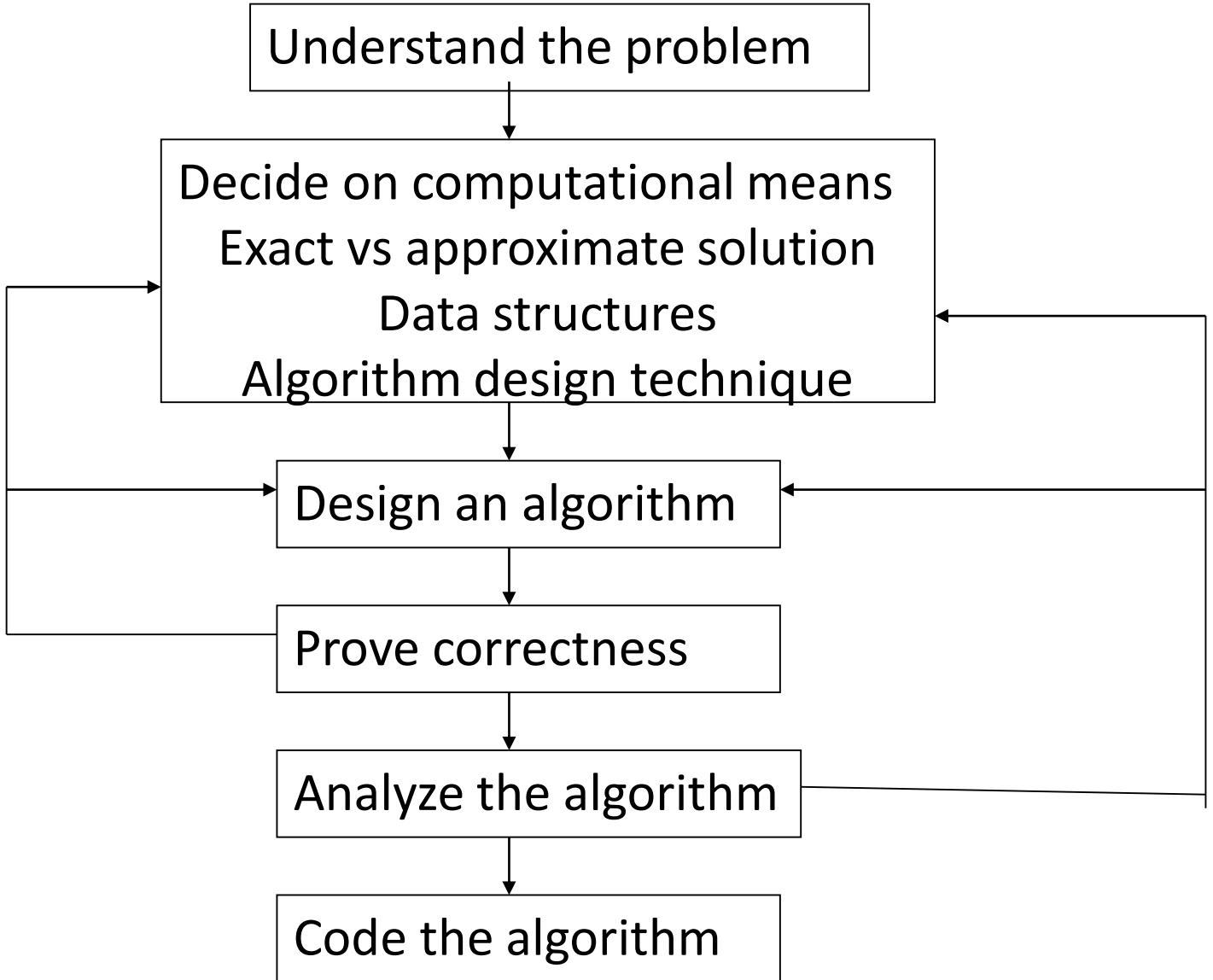
DESIGN AND ANALYSIS OF ALGORITHMS

Fundamentals of Algorithmic Problem Solving

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Algorithm Design and Analysis Process



Design and Analysis of Algorithms

Computational Means

Computational Device the algorithm is intended for

RAM Sequential Algorithms

PRAM Parallel Algorithms

Exact vs approximate solution

Travelling Salesman Problem

NP complete!!!

Approximate algorithm can be used to solve it

Design and Analysis of Algorithms

Deciding on Data Structure

- Linear

  Linear list, Stack, Queues
- Non Linear

  Trees, Graphs

Choice of Data structure for solving a problem using an algorithm may dramatically impact its time complexity

Dijkstra Algorithm

O(VlogV+E) with Fibonacci heap

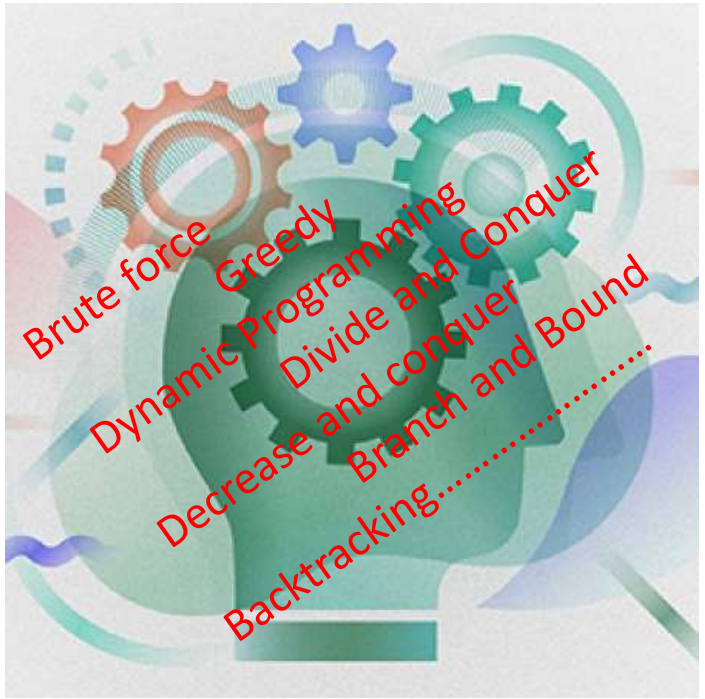
Design and Analysis of Algorithms

Algorithm Design Technique

General approach to solving problems algorithmically that is applicable to variety of problems from different areas of computing

ADT serves as heuristic for designing algorithms for new problems for which no satisfactory algorithm exists!!!

Algorithm Designer’s Toolkit



Abstract Data Types (ADTs) serve as a foundation for designing algorithms, especially for new problems without satisfactory solutions. They define the behavior of data structures (like stacks, queues, graphs) independently of implementation, offering a structured way to approach problem-solving.

As part of an Algorithm Designer's Toolkit, ADTs act as heuristics, guiding the selection of data structures and operations. For instance:

Graphs model network problems.
Priority Queues help in pathfinding algorithms like Dijkstra’s.
Hash Tables enable fast lookups.
By leveraging ADTs, designers can break down problems into manageable components, enabling efficient exploration and development of solutions.

Design and Analysis of Algorithms

Specifying an algorithm

- Natural Language
- Pseudo Code
- Flowchart

Design and Analysis of Algorithms

Specifying an algorithm

- Natural Language
- Pseudo Code
- Flowchart

Design and Analysis of Algorithms

Proving Correctness

Exact algorithms

Proving that algorithm yields a correct result for legitimate input in finite amount of time

Approximation algorithms

Error produced by algorithm does not exceed a predefined limit

Analyzing an algorithm

- Efficiency
  - Time efficiency
  - Space efficiency
- Simplicity
- Generality
  - Design an algorithm for the problem posed in more general terms
  - Design an algorithm that can handle a range of inputs that is natural for the problem at hand

Coding an algorithm

- Efficient implementation
- Correctness of program
  - Mathematical Approach: Formal verification for small programs
  - Practical Methods: Testing and Debugging
- Code optimization

THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Important Problem Types

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Design and Analysis of Algorithms

Important Problem Types

- sorting
- searching
- string processing
- graph problems
- combinatorial problems
- geometric problems
- numerical problems

Design and Analysis of Algorithms

Important Problem Types: Sorting

- Rearrange the items of a given list in ascending order.
  - Input: A sequence of n numbers <a1, a2, …, an>
  - Output: A reordering <a'_1, a'_2, …, a'_n> of the input sequence such that $a'_1 \leq a'_2 \leq \ldots \leq a'_n$.
- Why sorting?
  - Help searching
  - Algorithms often use sorting as a key subroutine.
- Sorting key

  A specially chosen piece of information used to guide sorting.

  Example: sort student records by SRN.

Important Problem Types: Sorting

- Rearrange the items of a given list in ascending order.
- Examples of sorting algorithms
  - Selection sort
  - Bubble sort
  - Insertion sort
  - Merge sort
  - Heap sort …
- Evaluate sorting algorithm complexity: the number of key comparisons.
- Two properties
  - Stability: A sorting algorithm is called stable if it preserves the relative order of any two equal elements in its input.
  - In place : A sorting algorithm is in place if it does not require extra memory, except, possibly for a few memory units.

Design and Analysis of Algorithms

Important Problem Types: Searching

Find a given value, called a search key, in a given set.

Examples of searching algorithms

- Sequential searching
- Binary searching…

Important Problem Types: String Processing

A string is a sequence of characters from an alphabet.

Text strings: letters, numbers, and special characters.

String matching: searching for a given word/pattern in a text.

Text: I am a computer science graduate

Pattern: computer

Important Problem Types: Graph Problems

Definition

Graph G is represented as a pair G= (V, E),
where V is a finite set of vertices and E is a finite set of edges

Modeling real-life problems

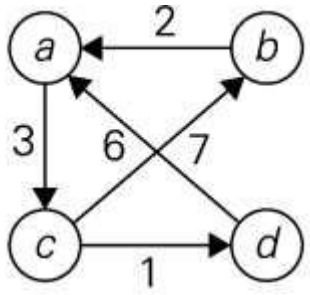
- Modeling WWW
- communication networks
- Project scheduling …

Examples of graph algorithms

- Graph traversal algorithms
- Shortest-path algorithms
- Topological sorting

Shortest paths in a graph

To find the distances from each vertex to all other vertices.



(a)

$$W = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{array}$$

(b)

$$D = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 7 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix} \end{array}$$

(c)

FIGURE 8.5 (a) Digraph. (b) Its weight matrix. (c) Its distance matrix.

Minimum cost spanning tree

- A spanning tree of a connected graph is its connected acyclic sub graph (i.e. a tree).

A spanning tree is defined as a tree-like subgraph of a connected, undirected graph that includes all the vertices of the graph. Or, to say in Layman's words, it is a subset of the edges of the graph that forms a tree (acyclic) where every node of the graph is a part of the tree.

E = V-1

The minimum spanning tree has all the properties of a spanning tree with an added constraint of having the minimum possible weights among all possible spanning trees.

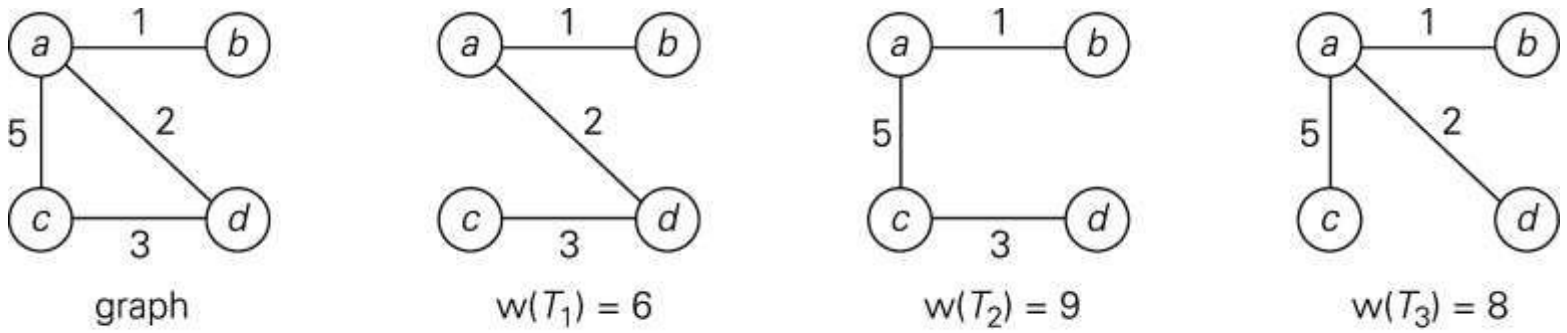
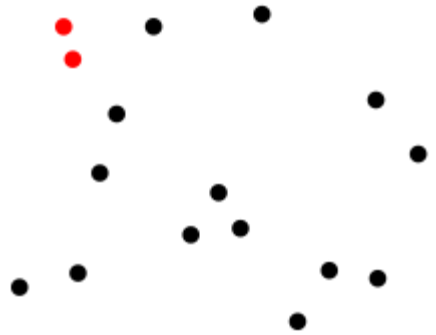


FIGURE 9.1 Graph and its spanning trees; T_1 is the minimum spanning tree

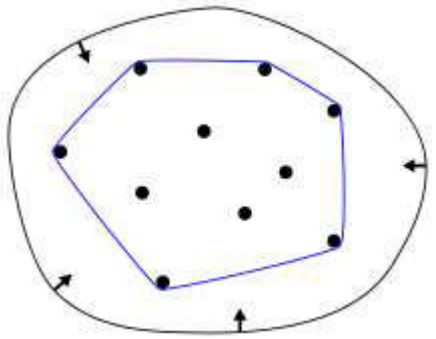
Important Problem Types: Geometric Problems

PES UNIVERSITY ONLINE

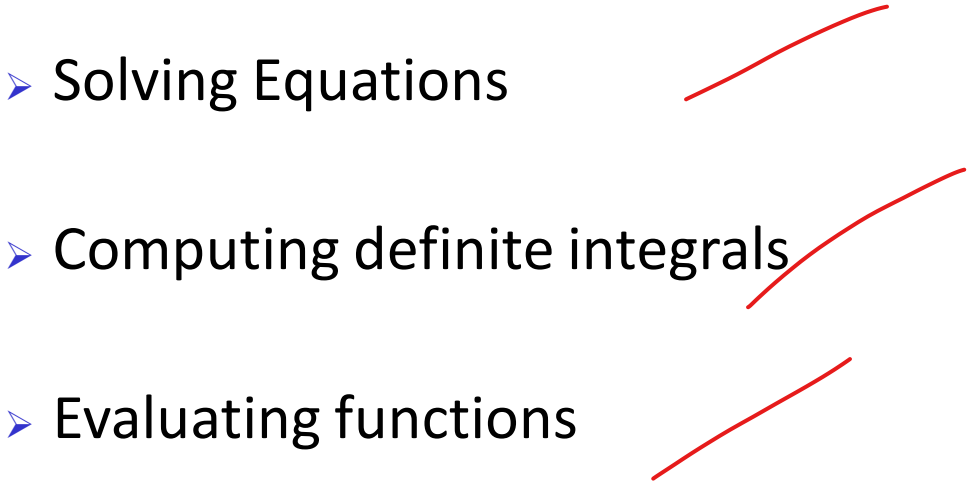
Closest Pair problem



Convex Hull Problem



Important Problem Types: Numerical Problems



- Solving Equations
- Computing definite integrals
- Evaluating functions

THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Analysis Framework

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Analysis Framework

What do you mean by analysing an algorithm?

Investigation of Algorithm's efficiency with respect to two resources

- Time
- Space

What is the need for Analysing an algorithm?

- To determine resource consumption
  - CPU time
  - Memory space
- Compare different methods for solving the same problem before actually implementing them and running the programs.
- To find an efficient algorithm

Design and Analysis of Algorithms

Complexity of an Algorithm

- A measure of the performance of an algorithm
- An algorithm's performance depends on
  - *internal* factors
    - Time required to run
    - Space (memory storage)required to run
  - *external* factors
    - Speed of the computer on which it is run
    - Quality of the compiler
    - Size of the input to the algorithm

Performance of Algorithm

important Criteria for performance:

- Space efficiency - the memory required, also called, space complexity
- Time efficiency - the time required, also called time complexity

Space Complexity

inpependet of input characterstics

S(P)=C+SP(I)

- Fixed Space Requirements (C)
  Independent of the characteristics of the inputs and outputs
  - instruction space
  - space for simple variables, fixed-size structured variable, constants
- Variable Space Requirements (SP(I))
  dependent on the instance characteristic I
  - number, size, values of inputs and outputs associated with I
  - recursive stack space, formal parameters, local variables, return address

dependent on input characterstics

Design and Analysis of Algorithms

Space Complexity

$S(P)=C+S_P(I)$

```
float rsum(float list[ ], int n)
{
  if (n)
    return rsum(list, n-1) + list[n-1]
  return 0
 }
```

$S_{sum}(I)=S_{sum}(n)=6n$

total space per recursive call

Space Complexity (Ssum(I)):
Let's denote n as the input size (number of recursive calls).
At any point in time, the maximum depth of recursion will be n (since the function calls itself n times before returning).
Thus, the total space used by the recursion stack is:
Ssum(I)=6 x n bytes

Type	Name	Number of bytes
parameter: float	list [ ]	2
parameter: integer	n	2
return address:(used internally)		2
TOTAL per recursive call		6

S(P)=C+6n,

C: Constant space for the code, which isn't affected by recursion.
SP(I): Space required for the recursion stack.

Design and Analysis of Algorithms

Time Complexity

$T(P)=C+T_P(I)$

- Compile time (C)
  independent of instance characteristics

- run (execution) time TP

Compile Time (C):

This is the constant time taken during the compilation phase of the program. It includes tasks like:
-Syntax checking
-Code optimization
-Generating machine code
Compile time is independent of the input size or instance characteristics. It remains constant and does not affect runtime.

Execution Time (TP):
This is the run-time complexity or the time taken to execute the program for a specific input. It depends on the size and characteristics of the input
Tp(I) is the execution time for a specific input instance I.

Why is this division important?
-Compile time is irrelevant once the program is built; it happens only once.
-Run time (TP) is what determines the efficiency of the program for different inputs. Hence, the focus in algorithm analysis is on TP(I)

PES UNIVERSITY ONLINE

How to measure time complexity?

- Theoretical Analysis
- Experimental study

Experimental study

- Write a program implementing the algorithm
- Run the program with inputs of varying size and composition
- Get an accurate measure of the actual running time
- Use a method like System.currentTimeMillis()
- Plot the results

Limitations of Experimental study

- It is necessary to implement the algorithm, which may be difficult
- Results may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same hardware and software environments must be used
- Experimental data though important is not sufficient

Theoretical Analysis

- Uses a high-level description of the algorithm instead of an implementation
- Characterizes running time as a function of the input size, n.
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

Design and Analysis of Algorithms

Theoretical Analysis

When to Use Which?
Use Order of Magnitude:
- When you need a broad overview of how the algorithm scales with large input sizes.
- To categorize and compare algorithms quickly (e.g., O(n) vs. O(n^2)).
- When input size is large and small differences in execution time don't matter.

Use Estimation of Running Time:
- When input size is small and the real-world performance difference is critical.
- When algorithms fall into the same asymptotic category (e.g., two O(n) algorithms) and you need to compare their actual performance.
- For optimizing specific implementations.

Two approaches:

1.Order of magnitude/asymptotic categorization –

This uses coarse categories and gives a general idea of performance.

If algorithms fall into the same category, if data size is small,
or if performance is critical, use method 2

data size is large and you need a general idea of performance

2.Estimation of running time -

1. *operation counts* - select operation(s) that are executed most frequently and determine how many times each is executed.
2. *step counts* - determine the total number of steps, possibly lines of code, executed by the program.

Analysis Framework

- Measuring an input's size
- Measuring running time
- Orders of growth (of the algorithm's efficiency function)
- Worst-base, best-case and average efficiency

Measuring an input's size

Efficiency is defined as a function of input size.

Input size depends on the problem.

Example 1, what is the input size of the problem of sorting n numbers?

Example 2, what is the input size of adding two n by n matrices?

Design and Analysis of Algorithms

Units for Measuring Running Time

- Measure the running time using standard unit of time measurements, such as seconds, minutes?

  Depends on the speed of the computer.

  so not a good idea as it will not be generalized for all machines

- count the number of times each of an algorithm's operations is executed.

  (step count method)

  Difficult and unnecessary

- count the number of times an algorithm's basic operation is executed.

  Basic operation: the most important operation of the algorithm, the operation contributing the most to the total running time.

  For example, the basic operation is usually the most time-consuming operation in the algorithm's innermost loop.

IMP

Analysis in the RAM Model

SmartFibonacci(n)		*cost*	*times* ($n > 1$)
1	**if** $n = 0$	c_1	1
2	**then return** 0	c_2	0
3	**elseif** $n = 1$	c_3	1
4	**then return** 1	c_4	0
5	**else** $pprev \leftarrow 0$	c_5	1
6	$prev \leftarrow 1$	c_6	1
7	**for** $i \leftarrow 2$ **to** n	c_7	n
8	**do** $f \leftarrow prev + pprev$	c_8	$n-1$
9	$pprev \leftarrow prev$	c_9	$n-1$
10	$prev \leftarrow f$	c_{10}	$n-1$
11	**return** f	c_{11}	1

$$T(n) = c_1 + c_3 + c_5 + c_6 + c_{11} + nc_7 + (n-1)(c_8 + c_9 + c_{10})$$

$$T(n) = nC_1 + C_2 \quad \Rightarrow T(n) \text{ is a } \textit{linear function} \text{ of } n$$

Measuring Running Time: Basic operation count

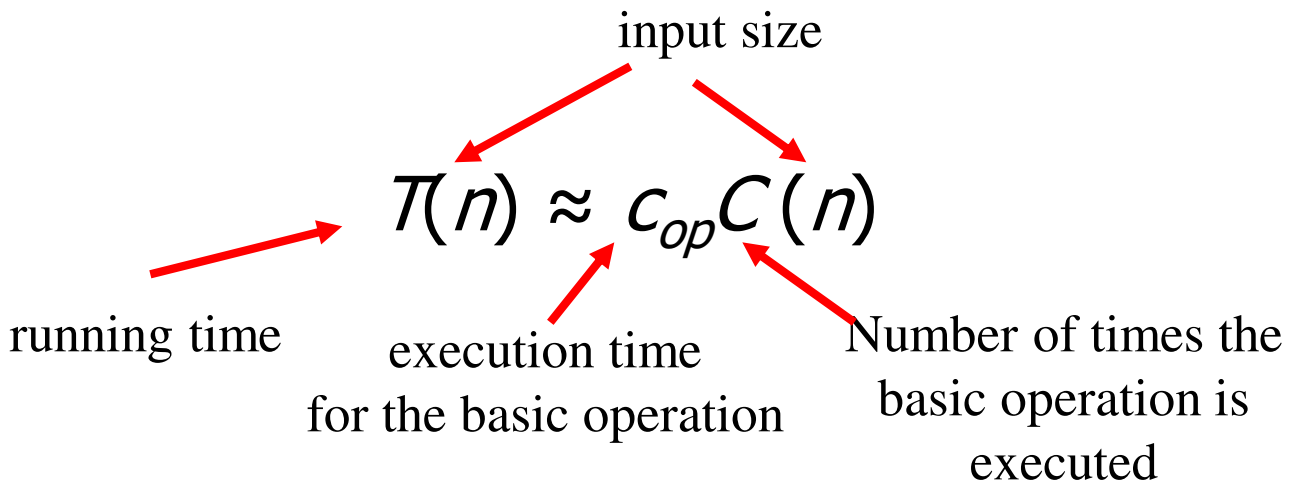
PES UNIVERSITY ONLINE

Input Size and Basic Operation Examples

Problem	***Input size measure***	***Basic operation***
Search for a key in a list of *n* items	Number of items in list, *n*	Key comparison
Add two *n* by *n* matrices	Dimensions of matrices, *n*	addition
multiply two matrices	Dimensions of matrices, *n*	multiplication

Theoretical Analysis of Time Efficiency: Basic operation count

Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size.



Order of Growth

C(n) Basic Operation Count

- The efficiency analysis framework ignores the multiplicative constants of C(n) and focuses on the orders of growth of the C(n).
- Simple characterization of the algorithm's efficiency by identifying relatively significant term in the C(n).

Order of Growth

Why do we care about the order of growth of an algorithm's efficiency function, i.e., the total number of basic operations?

- Because, for smaller inputs, it is difficult to distinguish inefficient algorithms vs. efficient ones.
- For example, if the number of basic operations of two algorithms to solve a particular problem are n and n^2 respectively, then
  - if $n = 2$, Basic operation will be executed 2 and 4 times respectively for algorithm1 and 2.

    Not much difference!!!
  - On the other hand, if $n = 10000$, then it does makes a difference whether the number of times the basic operation is executed is n or n^2.

Order of Growth

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms

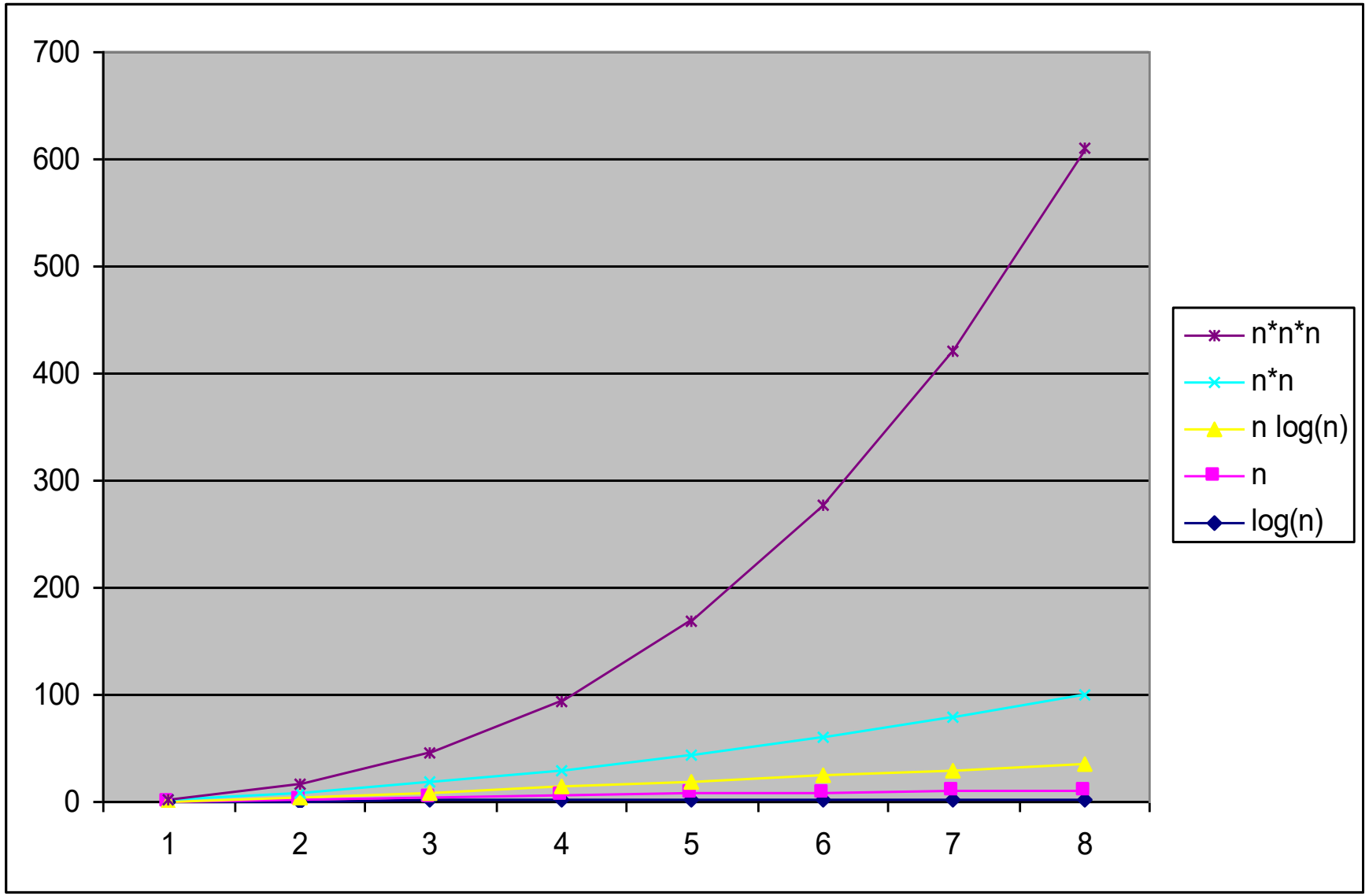
Exponential-growth functions

Orders of growth:

- consider only the leading term of a formula
- ignore the constant coefficient.

Design and Analysis of Algorithms

Order of Growth



Basic Efficiency Classes

1	constant
$\log n$	logarithmic
n	linear
$n \log n$	n-log-n
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial

Design and Analysis of Algorithms

Best, Worst and Average case Analysis

- Algorithm efficiency depends on the input size n
- For some algorithms efficiency depends on type of input.

  Example: Sequential Search

  Problem: Given a list of n elements and a search key K, find an element equal to K, if any.

  Algorithm: Scan the list and compare its successive elements with K until either a matching element is found (successful search) or the list is exhausted (unsuccessful search)

Given a sequential search problem of an input size of n,
what kind of input would make the running time the longest?
How many key comparisons?

Design and Analysis of Algorithms

Best, Worst and Average case Analysis

- Worst case Efficiency
  - Efficiency (# of times the basic operation will be executed) for the worst case input of size n.
  - The algorithm runs the longest among all possible inputs of size n.
- Best case
  - Efficiency (# of times the basic operation will be executed) for the best case input of size n.
  - The algorithm runs the fastest among all possible inputs of size n.
- Average case:
  - Efficiency (#of times the basic operation will be executed) for a typical/random input of size n.
  - NOT the average of worst and best case
- How to find the average case efficiency?

IMP

Best, Worst and Average case Analysis

```
ALGORITHM SequentialSearch(A[0..n-1], K)
  //Searches for a given value in a given array by sequential
  search
  //Input: An array A[0..n-1] and a search key K
  //Output: Returns the index of the first element of A that
  matches K or –1 if there are no matching elements
  i ←0
  while i < n and A[i] ǂ K do
        i ← i + 1
  if i < n          //A[I] = K
        return i
  else
        return -1
```

Best, Worst and Average case Analysis: Sequential Search

- Worst-Case: Cworst(n) = n
- Best-Case: Cbest(n) = 1
- Average-Case

from **(n+1)/2** to **(n+1)**

if the key is found then it will be found somewhere around middle in an average case scenario
so total searches n+ 1 extra search if i<n one will be also needed so total searches n+1 somewhere in middle =(n+1)/2

in case it's not found it would have made n searches and an additional extra search else return -1
so n+1

considering these both element will be fond in (n+1)/2 to n+1 searches

Average case Analysis: Sequential Search

Let **'p'** be the probability that key is found in the list

Assumption: All positions are equally probable

Case1: key is found in the list

$C_{avg,case1}(n) = p*(1 + 2 + \ldots + n) / n = p*(n + 1) / 2$

Case2: key is not found in the list

$C_{avg,\ case2}(n) = (1-p)*(n)$

$\mathbf{C_{avg}(n) = p}(n + 1) / 2 \mathbf{+ (1 - p)}(n)$

key is found ,
might be found in 1 comparison 2 comparioson ..... n comparison who knows that's why add all 1 to n

you checked full array but it was not found so you did total n comparison

THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Asymptotic Notations

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Design and Analysis of Algorithms

Asymptotic Notations

Order of growth of an algorithm's basic operation count is important How do we compare order of growth??

Using Asymptotic Notations

A way of comparing functions that ignores constant factors and small input sizes

- O($g(n)$): class of functions $f(n)$ that grow *no faster* than $g(n)$ <= lesser than equal to (tigthly upper bound)
- Ω($g(n)$): class of functions $f(n)$ that grow *at least as fast* as $g(n)$ >= greater than or equal to(tightly upper bound)
- Θ ($g(n)$): class of functions $f(n)$ that grow *at same rate* as $g(n)$ = equal to(same rate growth)
- o($g(n)$): class of functions $f(n)$ that grow *at slower rate* than $g(n)$ < lesser than (losely upper bound)
- ω($g(n)$): class of functions $f(n)$ that grow *at faster rate* than $g(n)$ > greater than (losely upper bound)

same rate but is bounded with some positive multiplese of g(n) for all large n

O-notation

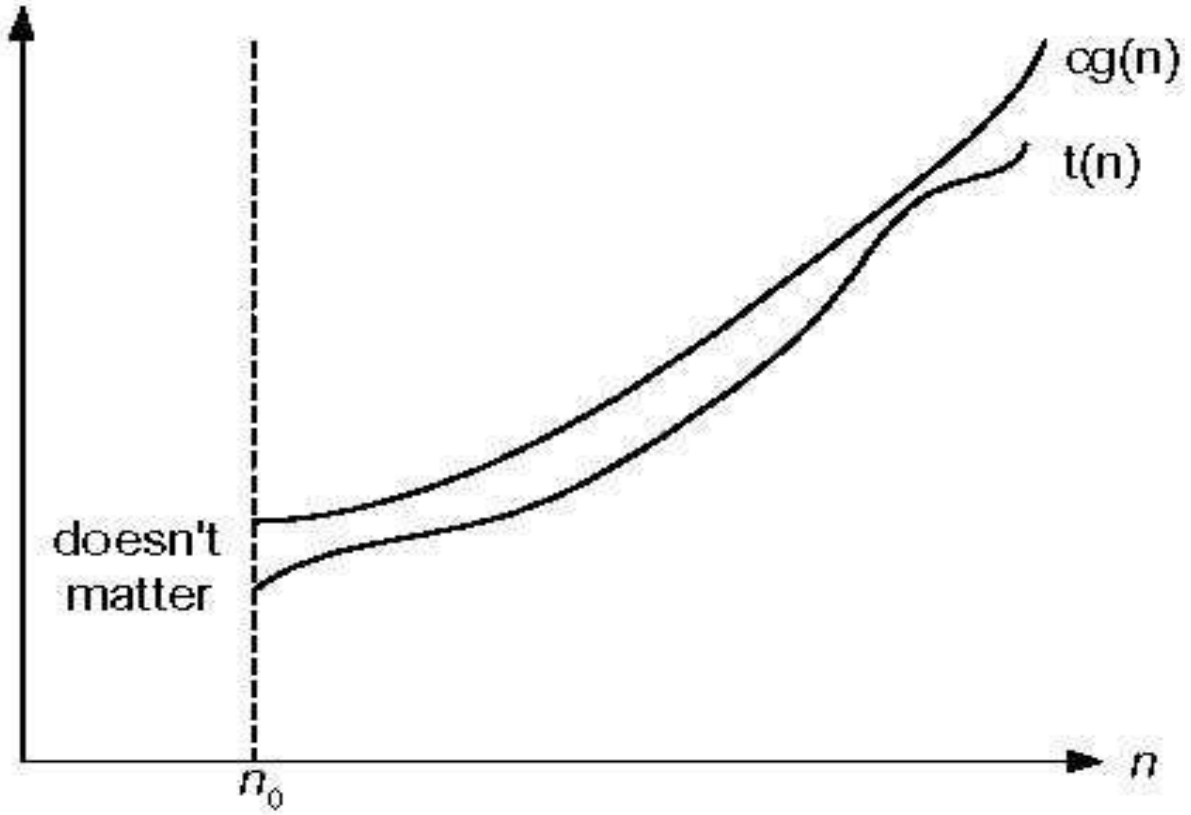


Figure 2.1 Big-oh notation: $t(n) \in O(g(n))$

O-notation

Formal definition

A function *t(n)* is said to be in *O(g(n)),* denoted *t(n)* $\in O(g(n))$, if *t(n)* is bounded above by some constant multiple of *g(n)* for all large *n*,

i.e., if there exist some positive constant c and some nonnegative integer n_0 such that

$t(n) \leq cg(n)$ for all $n \geq n_0$

Exercises: prove the following using the above definition

- $10n^2 \in O(n^2)$
- $10n^2 + 2n \in O(n^2)$
- $100n + 5 \in O(n^2)$
- $5n+20 \in O(n)$

Design and Analysis of Algorithms

Ω-notation

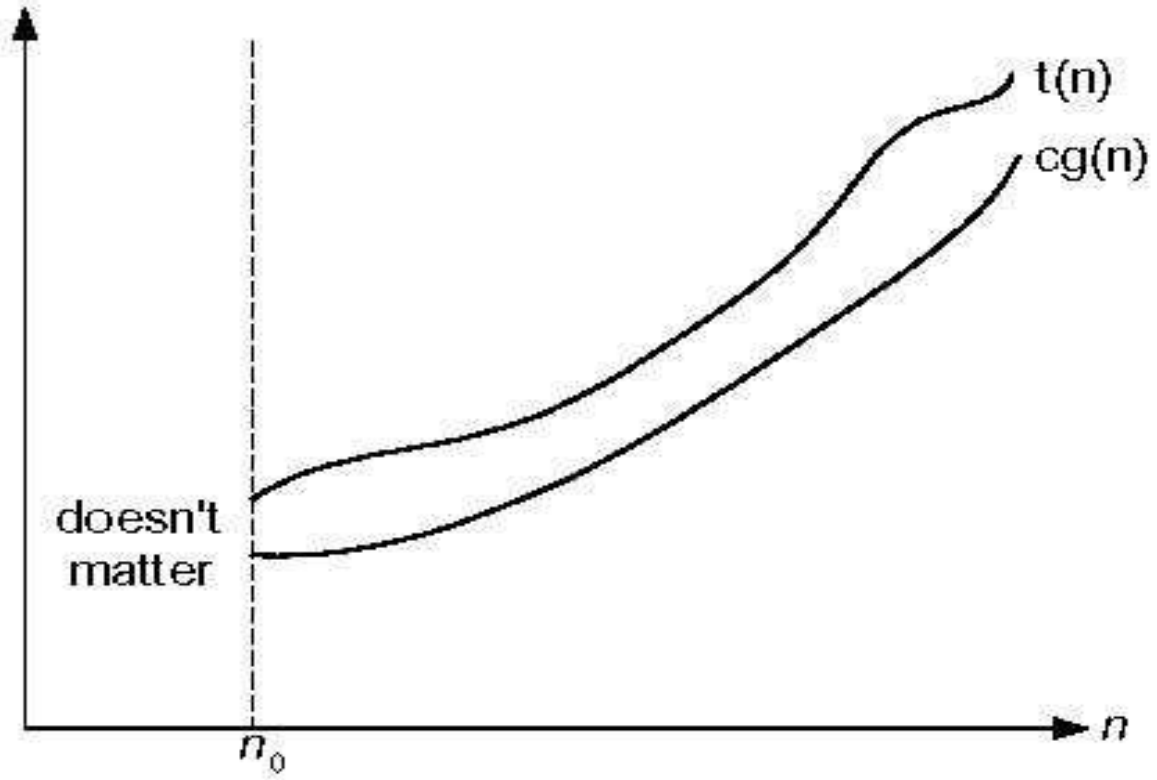


Fig. 2.2 Big-omega notation: $t(n) \in \Omega(g(n))$

Design and Analysis of Algorithms
Ω-notation

Formal definition

A function *t(n)* is said to be in Ω*(g(n)),* denoted *t(n)* ∈Ω*(g(n)),* if *t(n)* is bounded below by some constant multiple of *g(n)* for all large *n*,

i.e., if there exist some positive constant c and some nonnegative integer n_0 such that

$t(n) \geq cg(n)$ for all $n \geq n_0$

Exercises: prove the following using the above definition

$10n^2 \in \Omega(n^2)$

$10n^2 + 2n \in \Omega(n^2)$

$10n^3 \in \Omega(n^2)$

Design and Analysis of Algorithms

Θ-notation

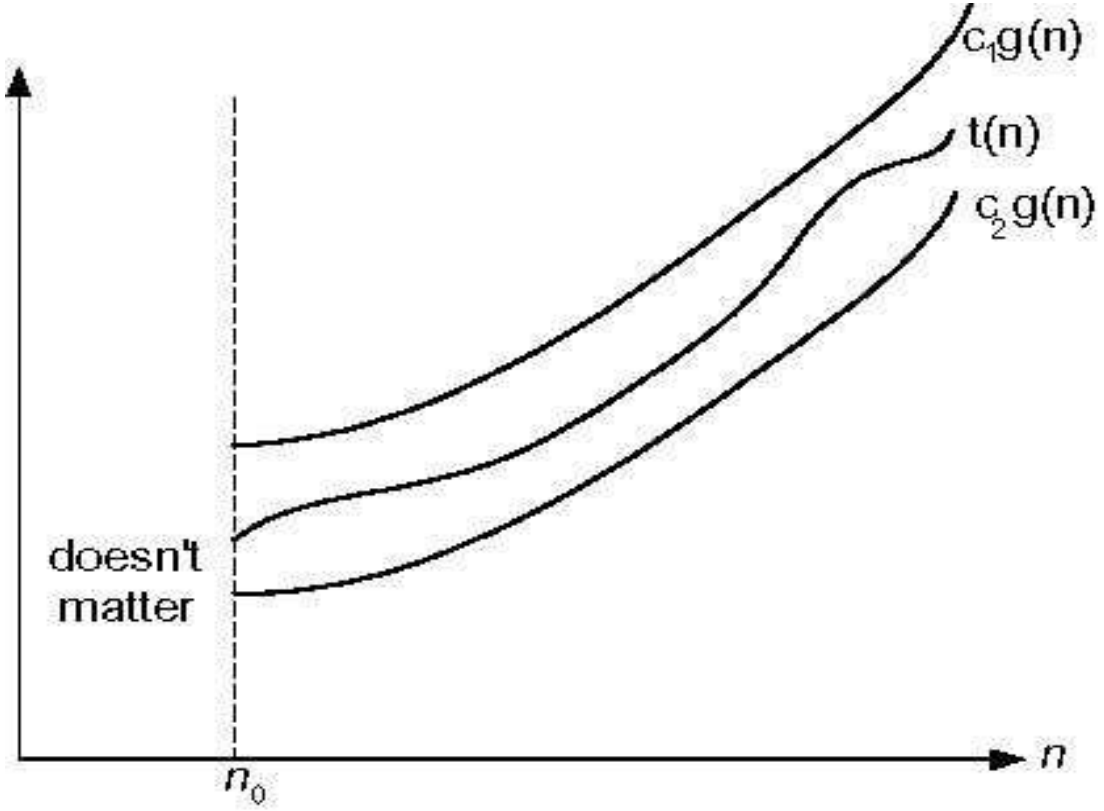


Figure 2.3 Big-theta notation: $t(n) \in \Theta(g(n))$

Θ-notation

Formal definition

A function *t(n)* is said to be in Θ*(g(n)),* denoted *t(n)* ∈Θ*(g(n)),* if *t(n)* is bounded both above and below by some positive constant multiples of *g(n)* for all large *n*,
i.e., if there exist some positive constants c_1 and c_2 and some nonnegative integer n_0 such that

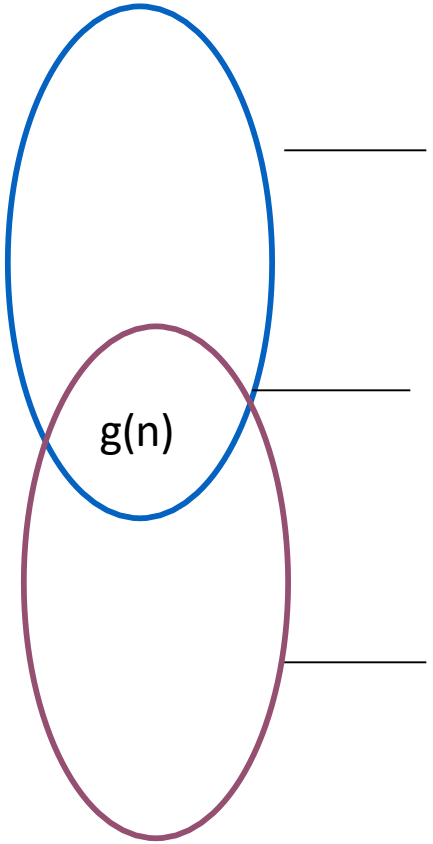
$$c_2\, g(n) \le t(n) \le c_1\, g(n) \text{ for all } n \ge n_0$$

Exercises: prove the following using the above definition

$10n^2 \in \Theta(n^2)$

$10n^2 + 2n \in \Theta(n^2)$

$(1/2)n(n-1) \in \Theta(n^2)$



>=

Ω(g(n)), functions that grow at least as fast as g(n)

=

Θ(g(n)), functions that grow at the same rate as g(n)

<=

O(g(n)), functions that grow no faster than g(n)

Little-o Notation

Formal Definition:

A function $t(n)$ is said to be in Little-o$(g(n))$, denoted $t(n) \in o(g(n))$, if for any positive constant c and some nonnegative integer n_0

$0 \leq t(n) < cg(n)$ for all $n \geq n_0$

Example: $n \in o(n^2)$

Little-o Notation

Example: $f(n) = 2n^2$ and $g(n) = n^2$ and $c = 2$

$f(n) = O(g(n))$ - Big-O

$f(n) \neq o(g(n))$ - little-o

If $f(n) = 2n$ & $g(n) = n^2$,

then for any value of $c>0$,

$f(n) < c(n^2)$

$f(n) \neq o(g(n))$

Note : For non-negative functions, f(n) and g(n),

f(n) is little o of g(n), if and only if,

$f(n) = O(g(n))$ and $f(n) \neq \theta(g(n))$

[strict upper bound, no lower bound]

Little Omega Notation

Formal Definition:

A function *t(n)* is said to be in Little- ω*(g(n)),* denoted *t(n)* ∈ω*(g(n)),*
if for any positive constant c and some nonnegative integer n_0

$t(n) > cg(n) \geq 0$ for all $n \geq n_0$

Example: $3\ n^2 + 2 \in \omega(n)$

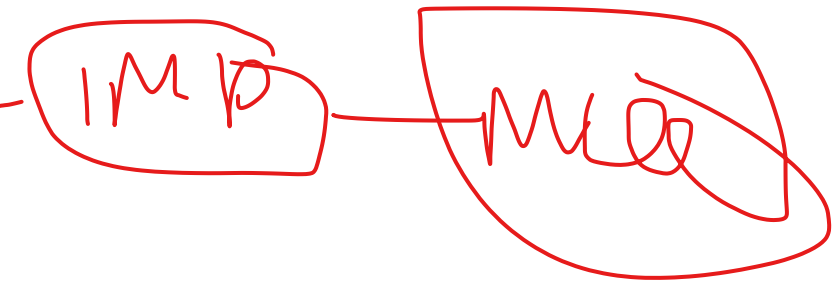
Little Omega Notation

Example : If $f(n) = 3n^2 + 2$, $g(n) = n$
then for any value of $c>0$
$f(n) > cg(n)$

$f(n) = \omega(n)$

Note : For non-negative functions, $f(n)$ *and* $g(n)$,
$f(n)$ is little ω of $g(n)$, if and only if

$f(n) = \Omega(g(n))$ and $f(n) \neq \theta(g(n))$
[strict lower bound, no upper bound]



Theorems

- If $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, then
  $t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$.
  For example,
  $5n^2 + 3n\log n \in O(n^2)$

- If $t1(n) \in \Theta(g1(n))$ and $t2(n) \in \Theta(g2(n))$, then
  $t1(n) + t2(n) \in \Theta(\max\{g1(n), g2(n)\})$

- $t1(n) \in \Omega(g1(n))$ and $t2(n) \in \Omega(g2(n))$, then
  $t1(n) + t2(n) \in \Omega(\max\{g1(n), g2(n)\})$

Implication: The algorithm's overall efficiency will be determined by the part with a larger
order of growth, i.e., its least efficient part

THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Basic Efficiency Classes
Problems based on Asymptotic notations

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

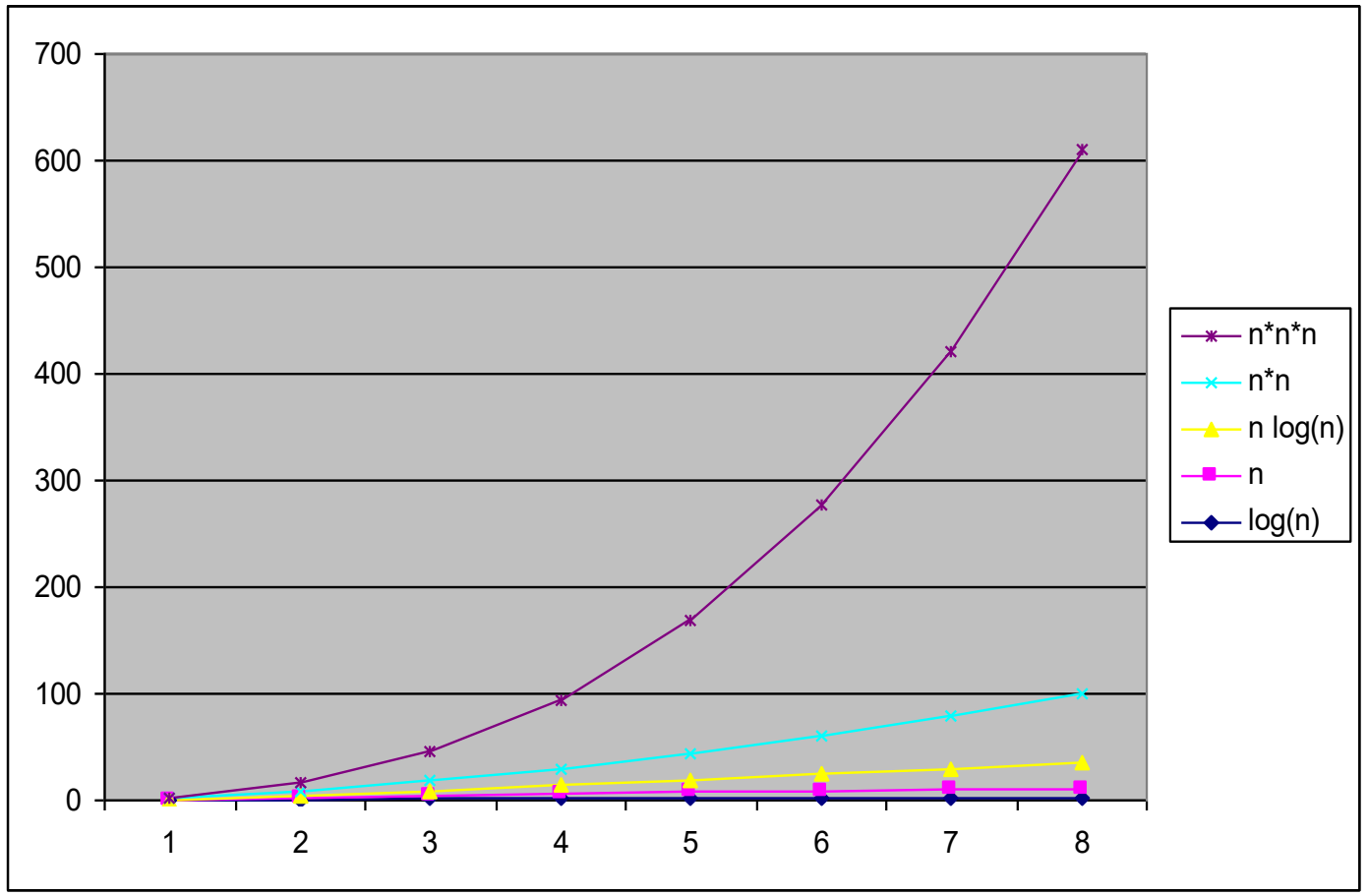
Design and Analysis of Algorithms

Basic Efficiency Classes

Class	Name	Example
1	constant	Best case for sequential search
log *n*	logarithmic	Binary Search
n	linear	Worst case for sequential search
n log *n*	*n*-log-*n*	Mergesort
n^2	quadratic	Bubble Sort
n^3	cubic	Matrix Multiplication
2^n	exponential	Subset generation
n!	factorial	TSP using exhaustive search

Design and Analysis of Algorithms

Basic Efficiency Classes



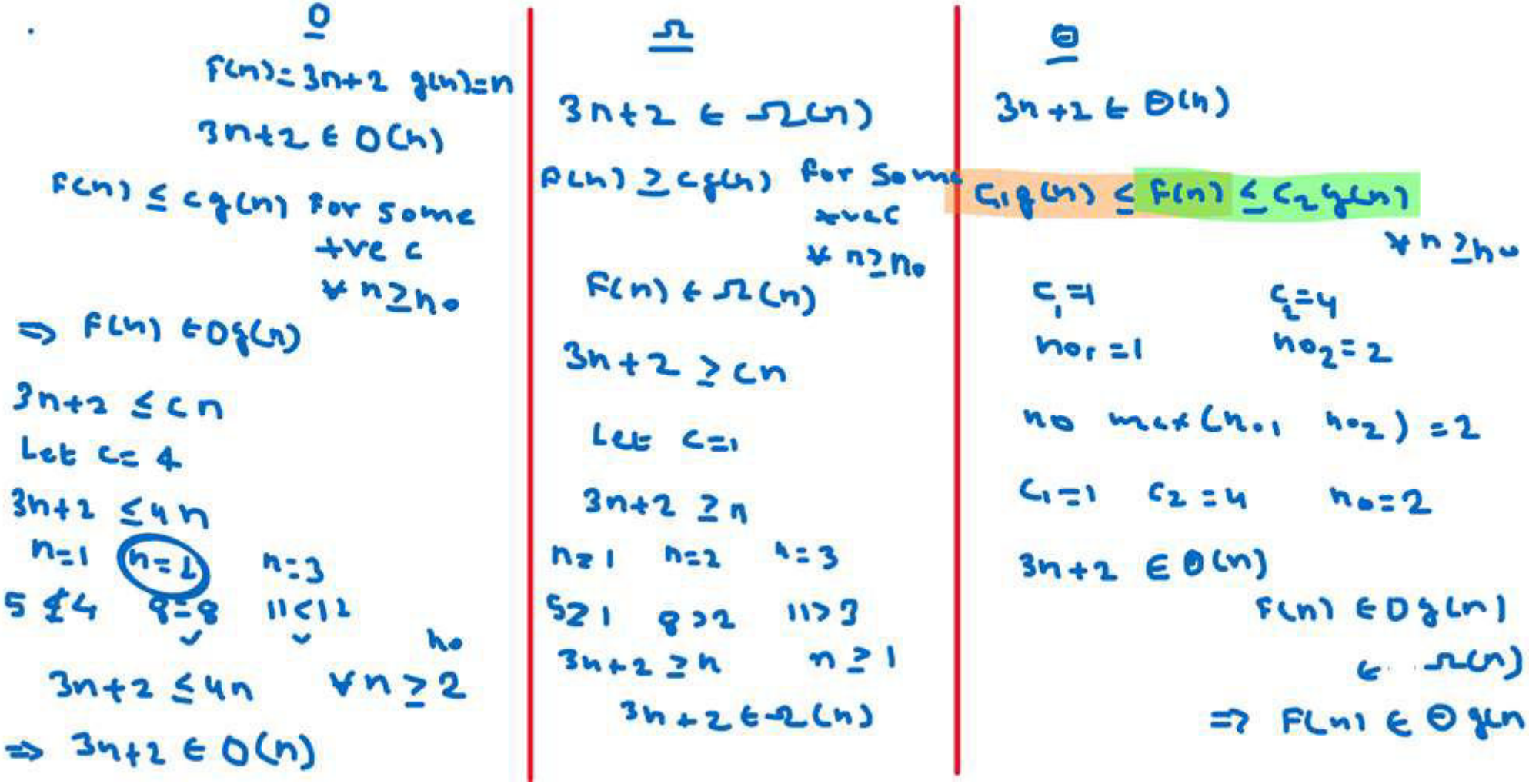
Basic Efficiency Classes

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms

Design and Analysis of Algorithms

Asymptotic notations



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Method of Limits for comparing order of Growth

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Design and Analysis of Algorithms

Using Limits to Compare Order of Growth

$$\lim_{n\to\infty}\frac{t(n)}{g(n)}=\begin{cases}0 & \text{implies that } t(n) \text{ has a smaller order of growth than } g(n),\\ c & \text{implies that } t(n) \text{ has the same order of growth as } g(n),\\ \infty & \text{implies that } t(n) \text{ has a larger order of growth than } g(n).\end{cases}$$

Case1: $t(n) \in O(g(n))$

Case2: $t(n) \in \Theta(g(n))$

Case3: $g(n) \in O(t(n))$

$t'(n)$ and $g'(n)$ are first-order derivatives of $t(n)$ and $g(n)$

L'Hopital's Rule $\lim_{n\to\infty}\frac{t(n)}{g(n)}=\lim_{n\to\infty}\frac{t'(n)}{g'(n)}$

Stirling's Formula $n!\approx\sqrt{2\pi n}\left(\frac{n}{e}\right)^n$ for large values of n

Using Limits to Compare Order of Growth: Example 1

Compare the order of growth of f(n) and g(n) using method of limits

$t(n) = 5n3 + 6n + 2\,, \qquad g(n) = n4$

$$\lim_{n\to\infty} \frac{t(n)}{g(n)} = \lim_{n\to\infty} \frac{5n^3 + 6n + 2}{n^4} = \lim_{n\to\infty} \left(\frac{5}{n} + \frac{6}{n^3} + \frac{2}{n^4} \right) = 0$$

As per case1

t(n) = O(g(n))

$5n^3$ + 6n + 2 = $O(n^4)$

Using Limits to Compare Order of Growth: Example 2

$$t(n) = \sqrt{5n^2 + 4n + 2}$$

using the Limits approach determine g(n) such that f(n) = Θ(g(n))

Leading term in square root n2

$$g(n) = \sqrt{n^2} = n$$

$$\lim_{n\to\infty} \frac{t(n)}{g(n)} = \lim_{n\to\infty} \frac{\sqrt{5n^2 + 4n + 2}}{\sqrt{n^2}}$$

$$= \lim_{n\to\infty} \sqrt{\frac{5n^2 + 4n + 2}{n^2}} = \lim_{n\to\infty} \sqrt{5 + \frac{4}{n} + \frac{2}{n^2}} = \sqrt{5}$$

non-zero constant

Hence, t(n) = Θ(g(n)) = Θ(n)

Design and Analysis of Algorithms

Using Limits to Compare Order of Growth

$$lim_{n \to \infty} t(n)/g(n) \neq 0, \infty \Rightarrow t(n) \in \Theta(g(n))$$

$$lim_{n \to \infty} t(n)/g(n) \neq \infty \Rightarrow t(n) \in O(g(n))$$

$$lim_{n \to \infty} t(n)/g(n) \neq 0 \Rightarrow t(n) \in \Omega(g(n))$$

$$lim_{n \to \infty} t(n)/g(n) = 0 \Rightarrow t(n) \in o(g(n))$$

$$lim_{n \to \infty} t(n)/g(n) = \infty \Rightarrow t(n) \in \omega(g(n))$$

byheart them and go
might come in mcq

Using Limits to Compare Order of Growth: Example 3

Compare the order of growth of t(n) and g(n) using method of limits

$t(n) = \log_2 n,\ g(n) = \sqrt{n}$

$$\lim_{n\to\infty} \frac{\log_2 n}{\sqrt{n}} = \lim_{n\to\infty} \frac{(\log_2 n)'}{(\sqrt{n})'} = \lim_{n\to\infty} \frac{(\log_2 e)\frac{1}{n}}{\frac{1}{2\sqrt{n}}} = 2\log_2 e \lim_{n\to\infty} \frac{1}{\sqrt{n}} = 0$$

$$\log_2 n \in o(\sqrt{n})$$

Design and Analysis of Algorithms
Orders of growth of some important functions

- All logarithmic functions $\log_a n$ belong to the same class

$\Theta(\log n)$ no matter what the logarithm's base $a > 1$ is

$\log_{10} n \in \Theta(\log_2 n)$

- All polynomials of the same degree k belong to the same class:

$a_k n^k + a_{k-1} n^{k-1} + \ldots + a_0 \in \Theta(n^k)$

- Exponential functions an have different orders of growth for different a's

$3^n \notin \Theta(2^n)$

- order $\log n$ < order n^α ($\alpha > 0$) < order a^n < order $n!$ < order n^n

How to Establish Orders of Growth of an Algorithm's Basic Operation Count

Summary

- Method 1: Using limits.
  - L' Hôpital's rule
- Method 2: Using the theorem.
- Method 3: Using the definitions of O-, Ω-, and Θ-notation.

THANK YOU

Vandana M L
Department of Computer Science & Engineering

vandanamd@pes.edu

Design and Analysis of Algorithms

Vandana M L

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Mathematical Analysis of Non-recursive Algorithms

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Time Efficiency of Non-recursive Algorithms

Steps in mathematical analysis of non-recursive algorithms:

- Decide on parameter n indicating input size
- Identify algorithm's basic operation
- Check whether the number of times the basic operation is executed depends only on the input size n. If it also depends on the type of input, investigate worst, average, and best case efficiency separately.
- Set up summation for C(n) reflecting the number of times the algorithm's basic operation is executed.
- Simplify summation using standard formulas

Design and Analysis of Algorithms

Useful Summation Formulas and Rules

$\Sigma_{l \le i \le u} 1 = 1+1+\ldots+1 = u - l + 1$

$\Sigma_{1 \le i \le n} i = 1+2+\ldots+n = n(n+1)/2$

$\Sigma_{1 \le i \le n} i^2 = 1^2+2^2+\ldots+n^2 = n(n+1)(2n+1)/6$

$\Sigma_{0 \le i \le n} a^i = 1 + a + \ldots + a^n = (a^{n+1} - 1)/(a - 1)$ for any $a \neq 1$

$\Sigma(a_i \pm b_i) = \Sigma a_i \pm \Sigma b_i \qquad \Sigma c a_i = c \Sigma a_i$

$\Sigma_{l \le i \le u} a_i = \Sigma_{l \le i \le m} a_i + \Sigma_{m+1 \le i \le u} a_i$

$$\sum_{i=l}^{u} 1 = (u - l + 1)$$

Example 1: Finding Max Element in a list

Algorithm *MaxElement (A[0..n-1])*
//Determines the value of the largest element
in a given array
//Input: An array A[0..n-1] of real numbers
//Output: The value of the largest element in A
maxval ← A[0]
for i ← 1 to n-1 do
    if A[i] > maxval
        maxval ← A[i]
return maxval

Let us denote C(n) the number of times this comparison is executed and try to find a formula expressing it as a function of size n. The algorithm makes one comparison on each execution of the loop, which is repeated for each value of the loop's variable i within the bounds 1 and n − 1, inclusive. Therefore, we get the following sum for C(n):

- The basic operation- comparison
- Number of comparisons is the same for all arrays of size n.
- Number of comparisons

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$$

abhi dekho hum pura summation kar rhe hai 1 to n-1 tak but inside loop one hi rhega na every iteration comparision happens once toh we will keep it 1 only

Example 2: Element Uniqueness Problem

Algorithm *UniqueElements (A[0..n-1])*
//Checks whether all the elements in a given array are distinct
//Input: An array A[0..n-1]
//Output: Returns true if all the elements in A are distinct and false otherwise

```
for i ← 0 to n - 2 do
        for j ← i + 1 to n – 1 do
                if A[i] = A[j] return false
return true
```

Best-case:

If the two first elements of the array are the same
No of comparisons in Best case = 1 comparison

Worst-case:

- Arrays with no equal elements
- Arrays in which only the last two elements are the pair of equal elements

Design and Analysis of Algorithms

Example 2: Element Uniqueness Problem

$$C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i)$$

$$= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2}$$

$$= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2} n^2$$

Best-case: 1 comparison

Worst-case: $n^2/2$ comparisons

$T(n)_{\text{worst case}} = O(n^2)$

Thenaturalmeasureoftheinput'ssizehereisagainn,thenumberofelements in the array. Since the innermost loop contains a single operation (the comparison of two elements), we should consider it as the algorithm's basic operation. Note, however, that the numberofelementcomparisonsdependsnotonlyonnbutalso on whether there are equal elements in the array and, if there are, which array positions they occupy. We will limit our investigation to the worst case only. Bydefinition,theworstcaseinputisanarrayforwhichthenumberofelement comparisons Cworst(n) is the largest among all arrays of size n. An inspection of the innermost loop reveals that there are two kinds of worst-case inputs—inputs for which the algorithm does not exit the loop prematurely: arrays with no equal elements and arrays in which the last two elements are the only pair of equal elements. For such inputs, one comparison is made for each repetition of the innermost loop, i.e., for each value of the loop variable j between its limits i + 1 and n−1; this is repeated for each value of the outer loop, i.e., for each value of the loop variable i between its limits 0 and n − 2. Accordingly, we get

Example 3:Matrix Multiplication

Algorithm *MatrixMultiplication(A[0..n-1, 0..n-1], B[0..n-1, 0..n-1])*

//Multiplies two square matrices of order n by the definition-based algorithm

//Input: two n-by-n matrices A and B

//Output: Matrix C = AB

```
for i ← 0 to n - 1 do
    for j ← 0 to n – 1 do
            C[i, j] ← 0.0
             for k ← 0 to n – 1 do
                        C[i, j] ← C[i, j] + A[i, k] * B[k, j]
return C
```

$M(n) \in \Theta(n^3)$

there is no best case worst case and all as the number of times basica operations addition and multiplication are executed depends only on matrix size that is the input so we don't take worst and best case into consdieration

hence average case is n^3

multiplication > addition

priority

THANK YOU

Vandana M L
Department of Computer Science & Engineering

vandanamd@pes.edu

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Mathematical Analysis of Recursive Algorithms

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Steps in Mathematical Analysis of Recursive Algorithms

- Decide on parameter n indicating input size
- Identify algorithm's basic operation
- If the number of times the basic operation is executed varies with different inputs of same sizes , investigate worst, average, and best case efficiency separately
- Set up a recurrence relation and initial condition(s) for C(n)-the number of times the basic operation will be executed for an input of size n
- Solve the recurrence or estimate the order of magnitude of the solution

Important Recurrence Types

- Decrease-by-one recurrences

A decrease-by-one algorithm solves a problem by exploiting a relationship between a given instance of size *n* and a smaller size *n – 1*.

Example: n!

The recurrence equation has the form

$T(n) = T(n-1) + f(n)$

- Decrease-by-a-constant-factor recurrences

A decrease-by-a-constant algorithm solves a problem by dividing its given instance of size *n* into several smaller instances of size *n/b*, solving each of them recursively, and then, if necessary, combining the solutions to the smaller instances into a solution to the given instance.

Example: binary search.

The recurrence has the form

$T(n) = aT(n/b) + f(n)$

Design and Analysis of Algorithms
Decrease-by-one Recurrences

- One (constant) operation reduces problem size by one.

  $T(n) = T(n-1) + c$ $\quad\quad$ $T(1) = d$

  Solution: $T(n) = (n-1)c + d$ $\quad\quad$ ***linear***

- A pass through input reduces problem size by one.

  $T(n) = T(n-1) + c\,n$ $\quad\quad$ $T(1) = d$

  Solution: $T(n) = [n(n+1)/2 - 1]\,c + d$ $\quad\quad$ ***quadratic***

Design and Analysis of Algorithms

Methods to solve recurrences

- Substitution Method
  - Mathematical Induction
  - Backward substitution
- Recursion Tree Method
- Master Method (Decrease by constant factor recurrences)

Design and Analysis of Algorithms

Recursive Evaluation of n!

$n! = 1 * 2 * \ldots * (n-1) * n$ for $n \geq 1$ and $0! = 1$

Recursive definition of $n!$:

$F(n) = F(n-1) * n$ for $n \geq 1$

$F(0) = 1$

input size?

basic operation?

Best/Worst/Average Case?

ALGORITHM $F(n)$

//Computes $n!$ recursively
//Input: A nonnegative integer n
//Output: The value of $n!$
if $n = 0$ **return** 1
else return $F(n-1) * n$

$$M(n) = M(n-1) + 1 \quad \text{for } n > 0,$$
$$M(0) = 0.$$

M(n-1) = M(n-2) + 1; **M(n-2) = M(n-3)+1**

M(n) = n

Overall time Complexity: Θ(n)

Counting number of binary digits in binary representation of a number

ALGORITHM *BinRec*(n)

```
//Input: A positive decimal integer n
//Output: The number of binary digits in n's binary representation
if n = 1 return 1
else return BinRec(⌊n/2⌋) + 1
```

input size?

basic operation?

Best/Worst/Average Case?

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{for } k > 0,$$
$$A(2^0) = 0.$$

$$\begin{aligned}
A(2^k) &= A(2^{k-1}) + 1 && \text{substitute } A(2^{k-1}) = A(2^{k-2}) + 1\\
&= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2 && \text{substitute } A(2^{k-2}) = A(2^{k-3}) + 1\\
&= [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3 && \dots\\
&\dots\\
&= A(2^{k-i}) + i\\
&\dots\\
&= A(2^{k-k}) + k.
\end{aligned}$$

$$A(n) = \log_2 n \in \Theta(\log n).$$

Tower of Hanoi

```
Algorithm TowerOfHanoi(n, Src, Aux, Dst)
    if (n = 0)
        return
    TowerOfHanoi(n-1, Src, Dst, Aux)
    Move disk n from Src to Dst
    TowerOfHanoi(n-1, Aux, Src, Dst)
```

Input Size: **n**

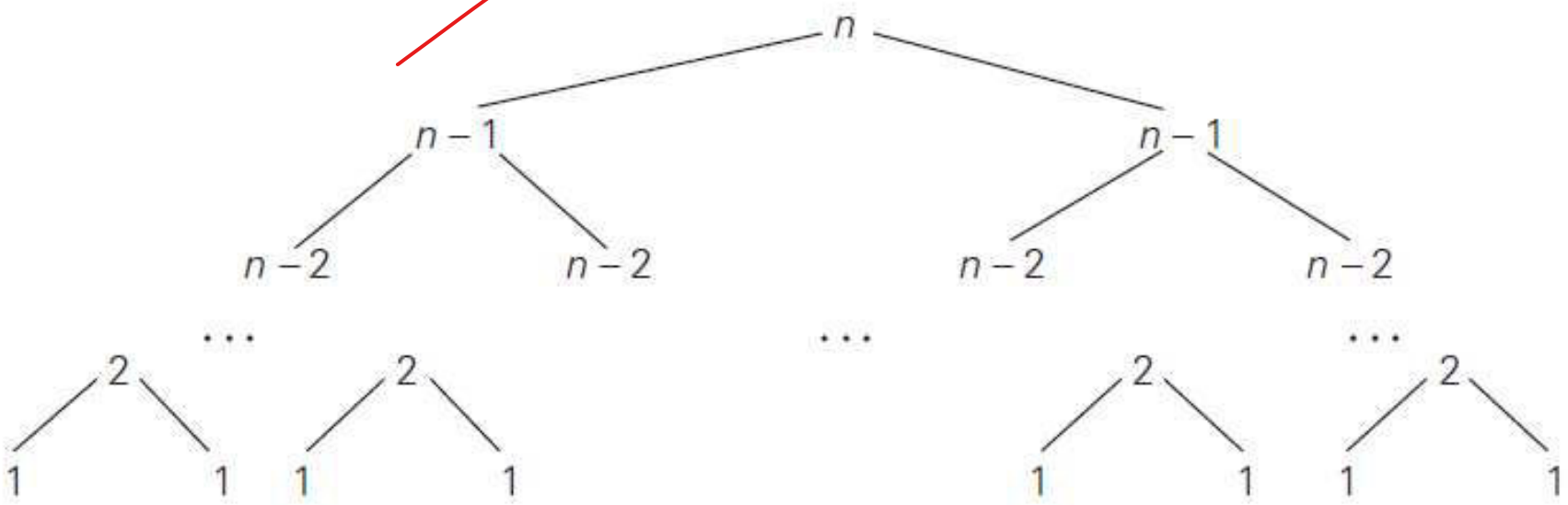
Basic Operation : **Move disk n from Src to Dst**

C(n) = **2C(n-1) + 1** for n > 0 and C(0)=0

= $2^n - 1 \in \Theta(2^n)$

Design and Analysis of Algorithms

Tower of Hanoi : Tree of Recursive calls



$$C(n) = \sum_{l=0}^{n-1} 2^l = 2^n - 1$$

THANK YOU

Vandana M L
Department of Computer Science & Engineering

vandanamd@pes.edu

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Solving Recurrences

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Solving Recurrences: Example1

$T(n) = T(n-1) + 1 \qquad n>0 \qquad T(0) = 1$

$T(n) = T(n-1) + 1$

$= T(n-2) + 1 + 1 = T(n-2) + 2$

$= T(n-3) + 1 + 2 = T(n-3) + 3$

$\ldots$

$= T(n-i) + i$

$\ldots$

$= T(n-n) + n = n = O(n)$

Solving Recurrences: Example2

$T(n) = T(n-1) + 2n - 1 \qquad T(0)=0$

$= [T(n-2) + 2(n-1) - 1] + 2n - 1$

$= T(n-2) + 2(n-1) + 2n - 2$

$= [T(n-3) + 2(n-2) - 1] + 2(n-1) + 2n - 2$

$= T(n-3) + 2(n-2) + 2(n-1) + 2n - 3$

...

$= T(n-i) + 2(n-i+1) + \ldots + 2n - i$

...

$= T(n-n) + 2(n-n+1) + \ldots + 2n - n$

$= 0 + 2 + 4 + \ldots + 2n - n$

$= 2 + 4 + \ldots + 2n - n$

$= 2*n*(n+1)/2 - n$

// arithmetic progression formula $1+\ldots+n = n(n+1)/2$ //

$= O(n^2)$

Design and Analysis of Algorithms

Solving Recurrences: Example3

$$T(n) = T(n/2) + 1 \quad n > 1$$
$$T(1) = 1$$

$$T(n) = T(n/2) + 1$$
$$= T(n/2^2) + 1 + 1$$
$$= T(n/2^3) + 1 + 1 + 1$$
.....
$$= T(n/2^i) + i$$
.....
$$= T(n/2^k) + k \quad (k = \log n)$$
$$= 1 + \log n$$
$$= O(\log n)$$

Design and Analysis of Algorithms

Solving Recurrences: Example4

$$T(n) = 2T(n/2) + cn \qquad n > 1 \qquad T(1) = c$$

$$T(n) = 2T(n/2) + cn$$

$$= 2(2T(n/2^2) + c(n/2)) + cn = 2^2 T(n/2^2) + cn + cn$$

$$= 2^2(2T(n/2^3) + c(n/2^2)) + cn + cn = 2^3 T(n/2^3) + 3cn$$

.....

$$= 2^i T(n/2^i) + icn$$

.....

$$= 2^k T(n/2^k) + kcn \quad (k = \log\ n)$$

$$= nT\ (1) + cn\log n = cn + cn\log n$$

$$= \mathrm{O}(n\log\ n)$$

THANK YOU

Vandana M L
Department of Computer Science & Engineering

vandanamd@pes.edu

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Performance Analysis Vs Performance Measurement

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Design and Analysis of Algorithms

Performance Evaluation of Algorithm

- Performance Analysis
  - Machine Independent
  - Prior Evaluation
- Performance Measurement
  - Machine Dependent
  - Posterior Evaluation

theoeretical analysis

experimental analysis

Design and Analysis of Algorithms

Performance Analysis of Sequential search :Worst Case

```
ALGORITHM SequentialSearch(A[0..n-1], K)
  //Searches for a given value in a given array by sequential search
  //Input: An array A[0..n-1] and a search key K
  //Output: Returns the index of the first element of A that matches K or –1 if there are no matching elements
  i ←0
  while i < n and A[i] ‡ K do
        i ← i + 1
  if i < n          //A[i] = K
        return i
  else
        return -1
```

Basic operation: A[i] ‡ K

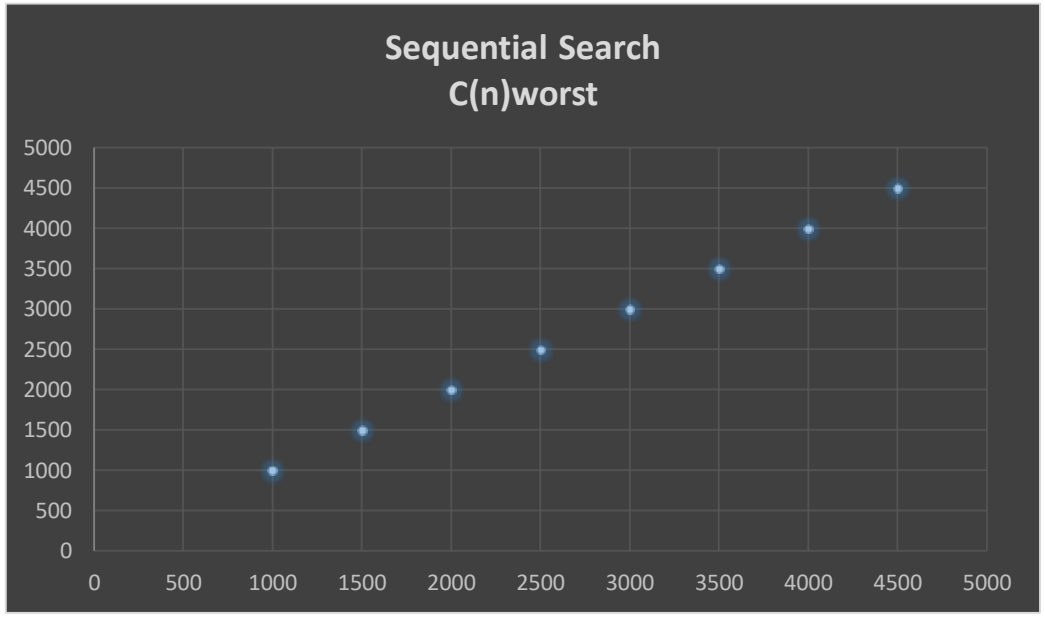
Basic operation count: n

Time Complexity: T(n) ∈O(n)

Design and Analysis of Algorithms

Performance Analysis of Sequential Search

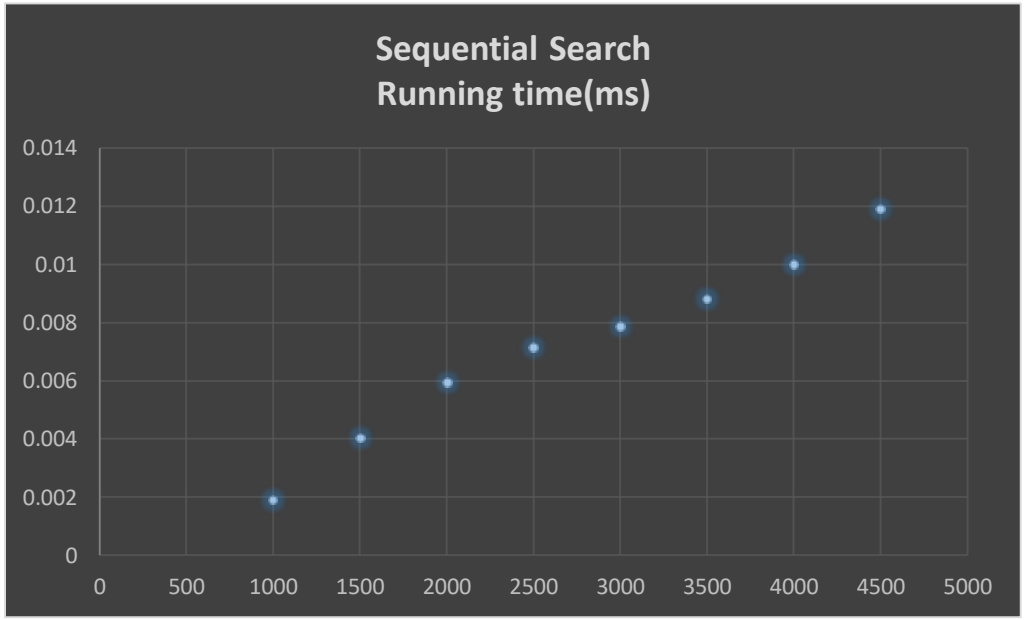
Input Size	Sequential Search C(n)worst
1000	1000
1500	1500
2000	2000
2500	2500
3000	3000
3500	3500
4000	4000
4500	4500



Design and Analysis of Algorithms

Performance Measurement of Sequential Search

Input Size	Sequential Search Actual Running Time(ms)
1000	0.001907
1500	0.004053
2000	0.00596
2500	0.007153
3000	0.007868
3500	0.008821
4000	0.010014
4500	0.011921



THANK YOU

Vandana M L
Department of Computer Science & Engineering

vandanamd@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS
UE19CS251

Shylaja S S
Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Brute Force: Selection Sort

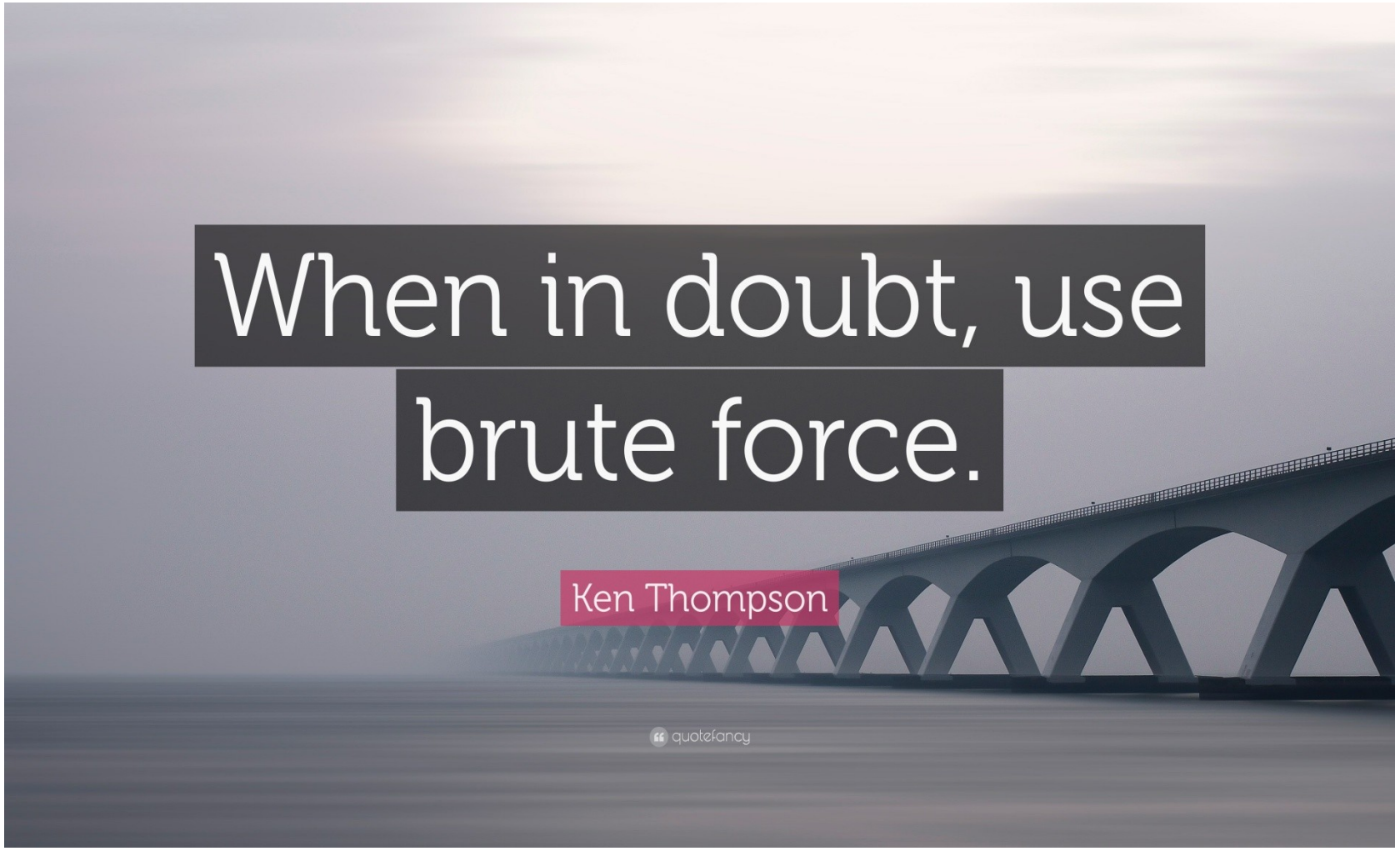
Major Slides Content: Anany Levitin

Shylaja S S

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Brute Force



Brute Force

- Brute Force is a straightforward approach to solving a problem, usually directly based on the problem statement and definitions of the concepts involved

Brute Force

- In computer science, **brute-force search** or **exhaustive search**, also known as **generate and test**, is a very general problem-solving technique and algorithmic paradigm that consists of:
  - systematically enumerating all possible candidates for the solution
  - checking whether each candidate satisfies the problem's statement

https://en.wikipedia.org/wiki/Brute-force_search

Brute Force

- A brute-force algorithm to find the divisors of a natural number n would
  - enumerate all integers from 1 to n
  - check whether each of them divides n without remainder
- A brute-force approach for the eight queens puzzle would
  - examine all possible arrangements of 8 pieces on the 64-square chessboard
  - check whether each (queen) piece can attack any other, for each arrangement
  - The brute-force method for finding an item in a table (linear search)
  - checks all entries of the table, sequentially, with the item

Brute Force

- A brute-force search is simple to implement, and will always find a solution if it exists
- But, its cost is proportional to the number of candidate solutions – which in many practical problems tends to grow very quickly as the size of the problem increases (Combinatorial explosion)
- Brute-force search is typically used
  - when the problem size is limited
  - when there are problem-specific heuristics that can be used to reduce the set of candidate solutions to a manageable size
  - when the simplicity of implementation is more important than speed

techniques or approaches

DESIGN AND ANALYSIS OF ALGORITHMS

Brute Force Sorting Algorithms

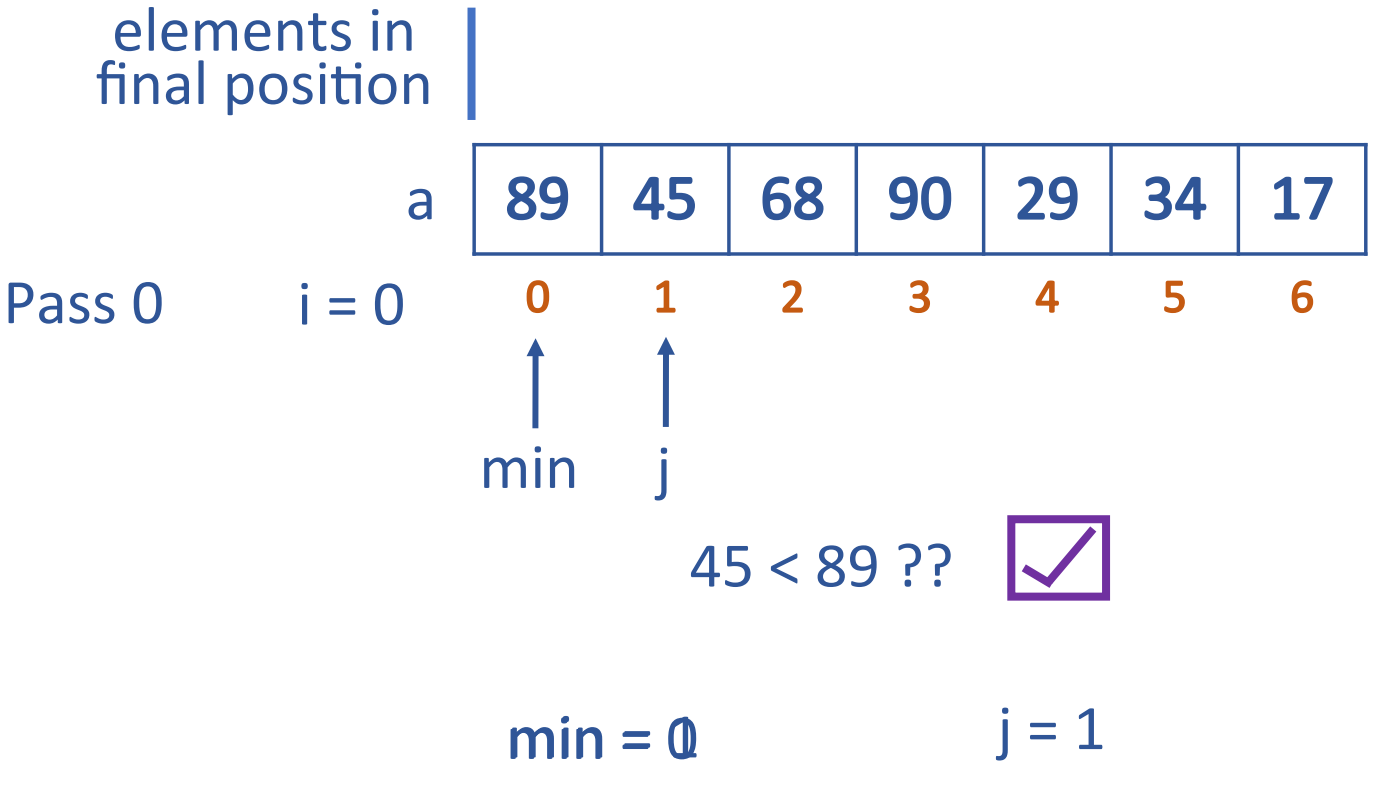
- Selection Sort
- Bubble Sort

Selection Sort

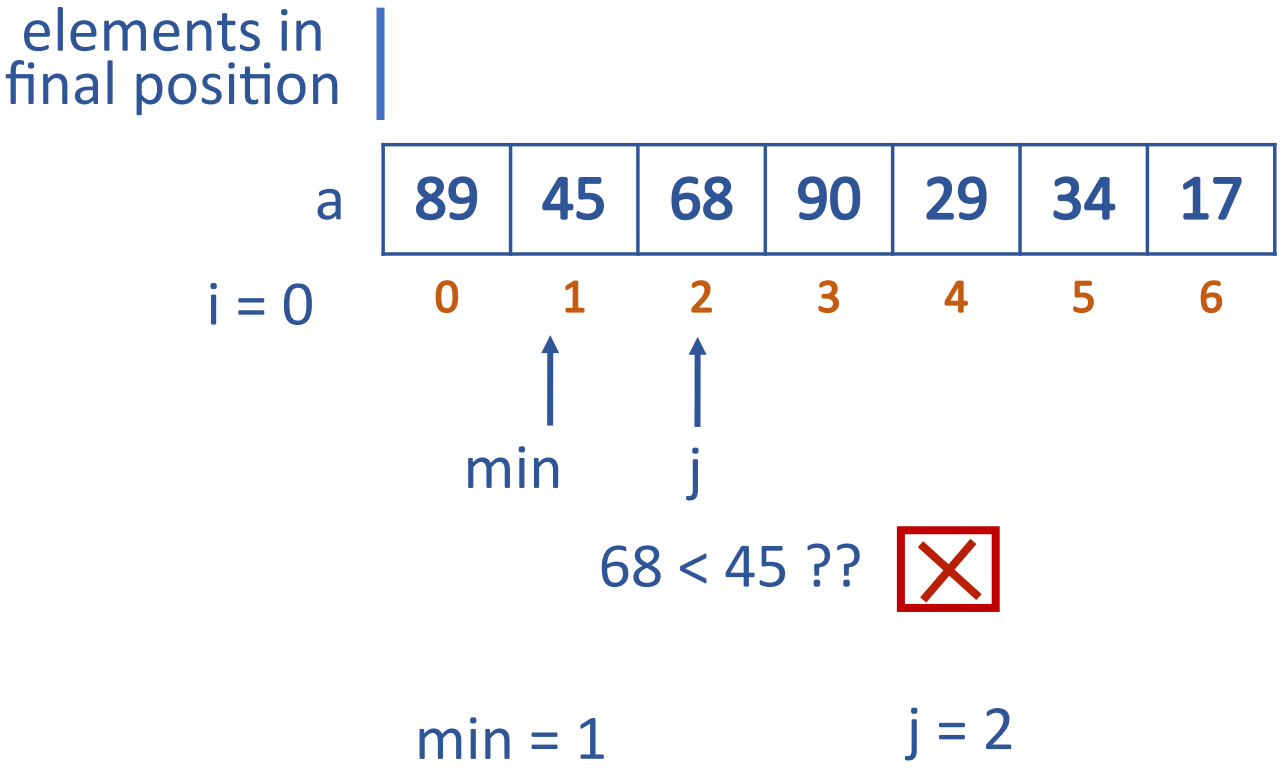
- Scan the array to find its smallest element and swap it with the first element, putting the smallest element in its final position in the sorted list
- Then, starting with the second element, scan the elements to the right of it to find the smallest among them and swap it with the second element, putting the second smallest element in its final position
- Generally, on pass i ($0 \leq i \leq n$-2), find the smallest element in $A[i..n\text{-}1]$ and swap it with $A[i]$

- $A[0] \leq A[1] \leq A[2] \ldots \leq A[i\text{-}1]$ | A[i], ….., A[min], …., A[n-1]
- in their final positions the last n – i elements

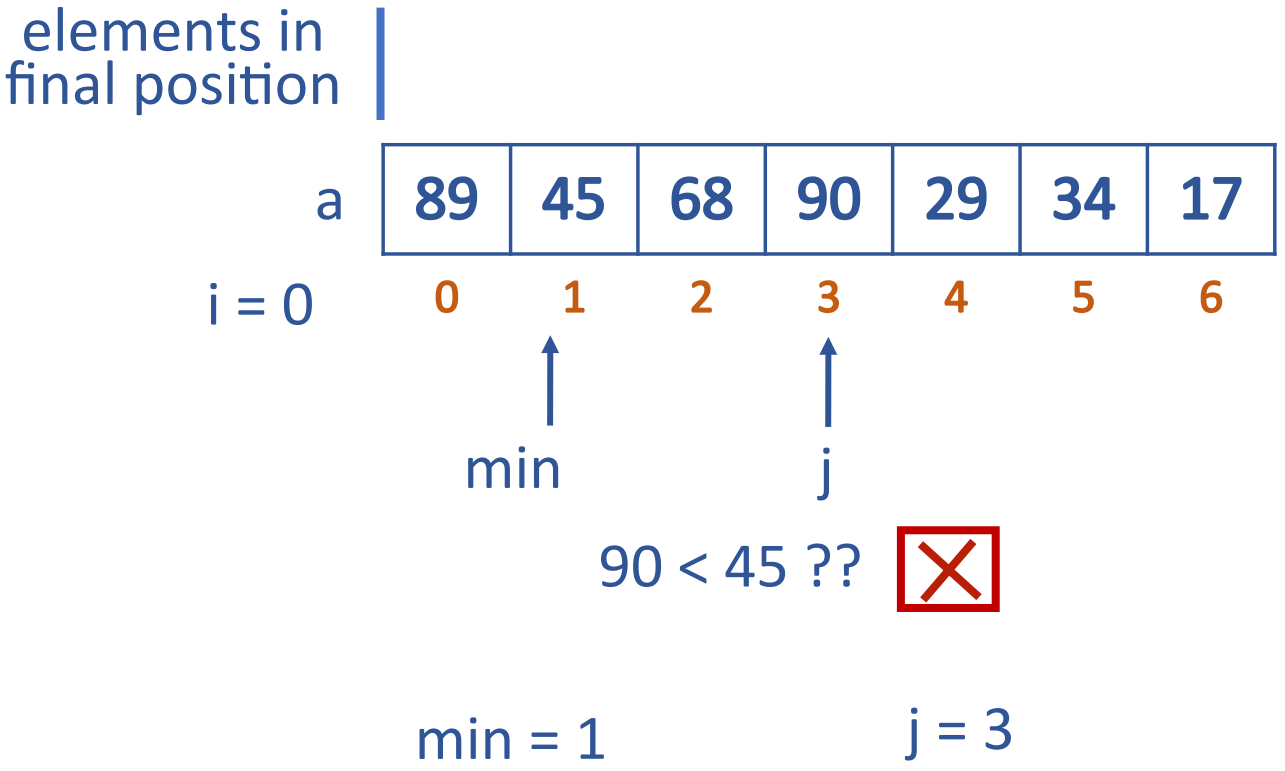
Selection Sort



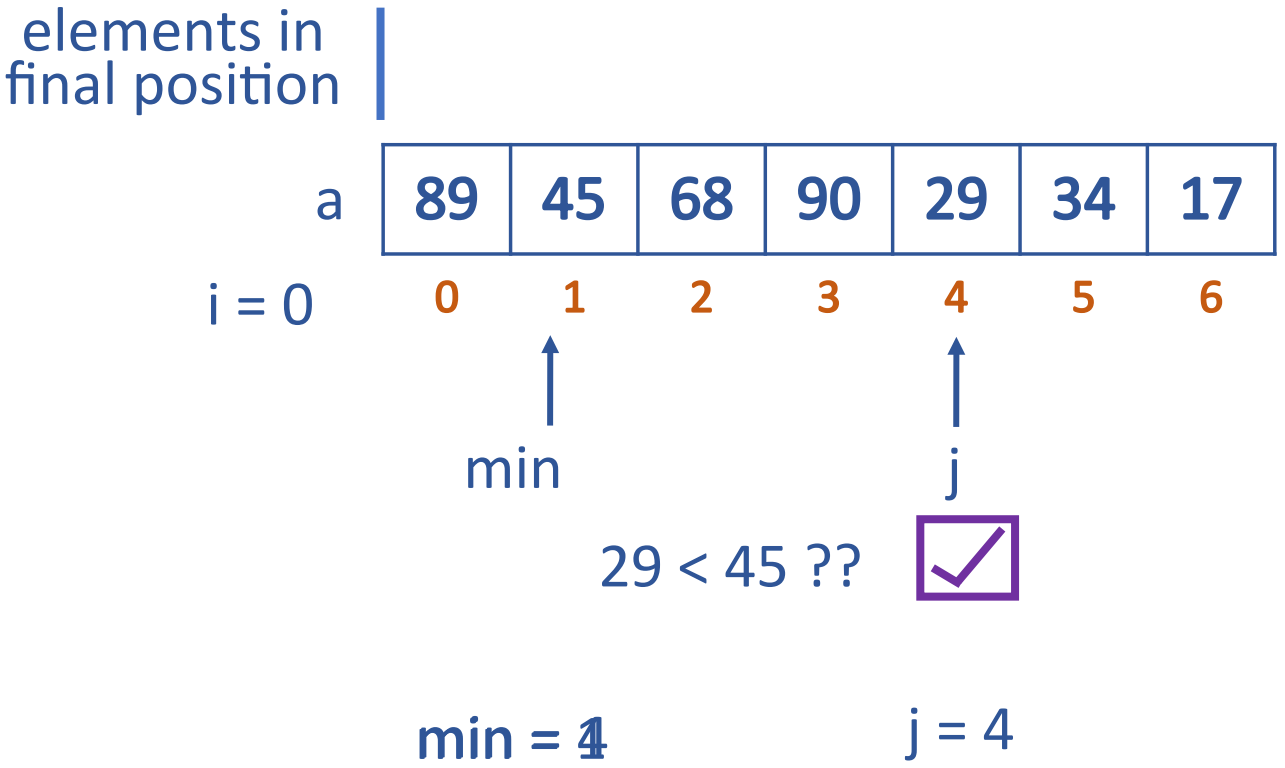
Selection Sort



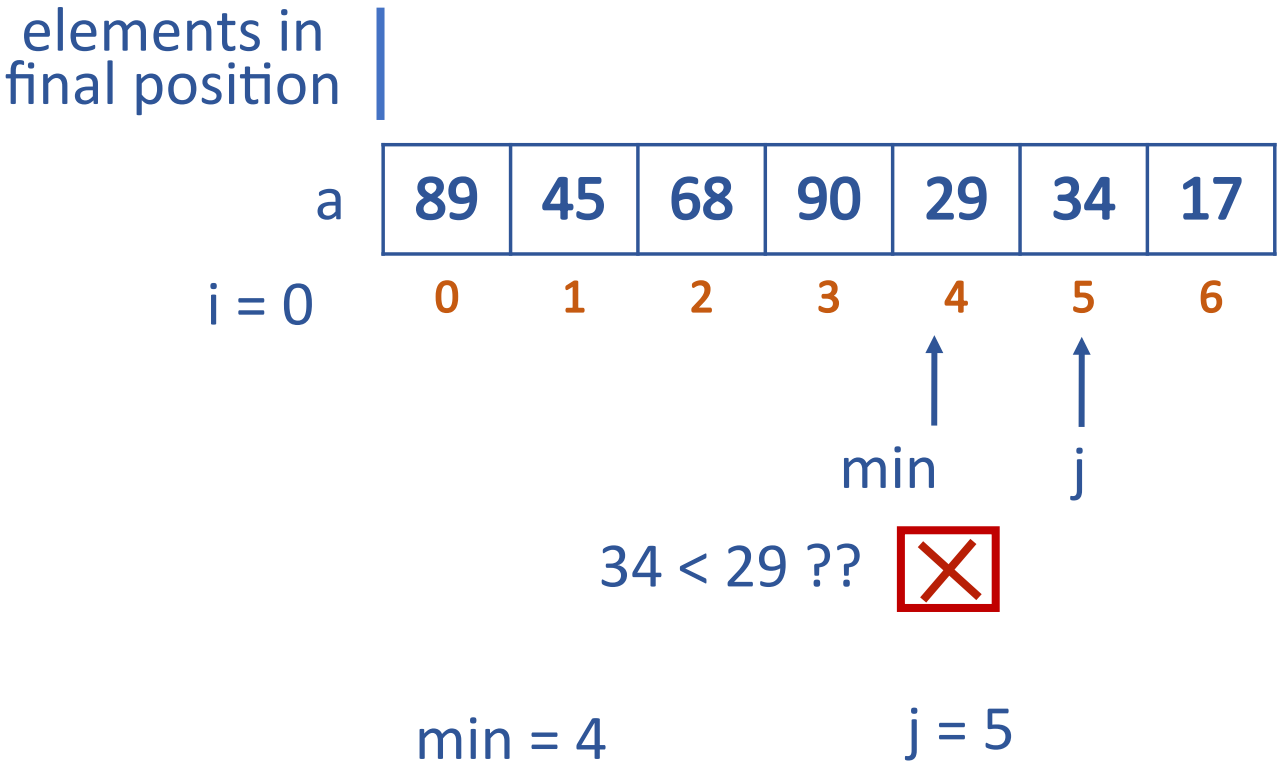
Selection Sort



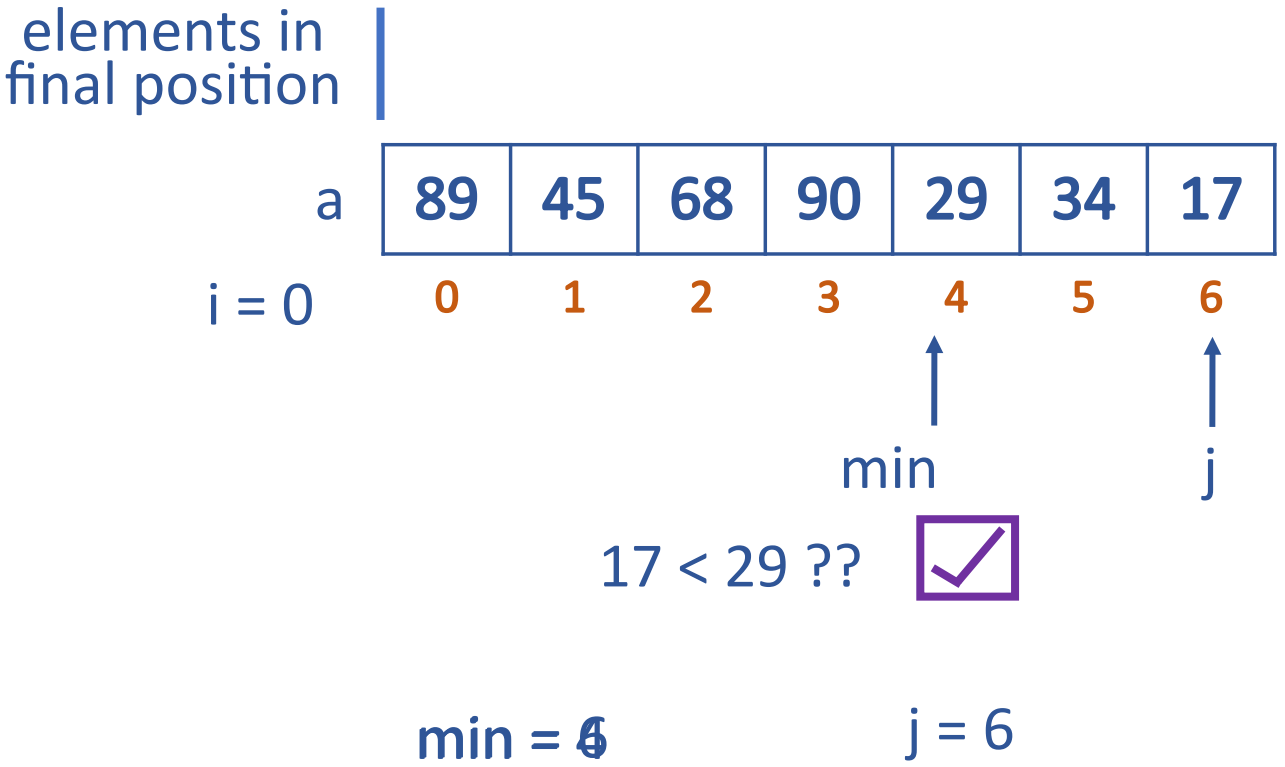
Selection Sort



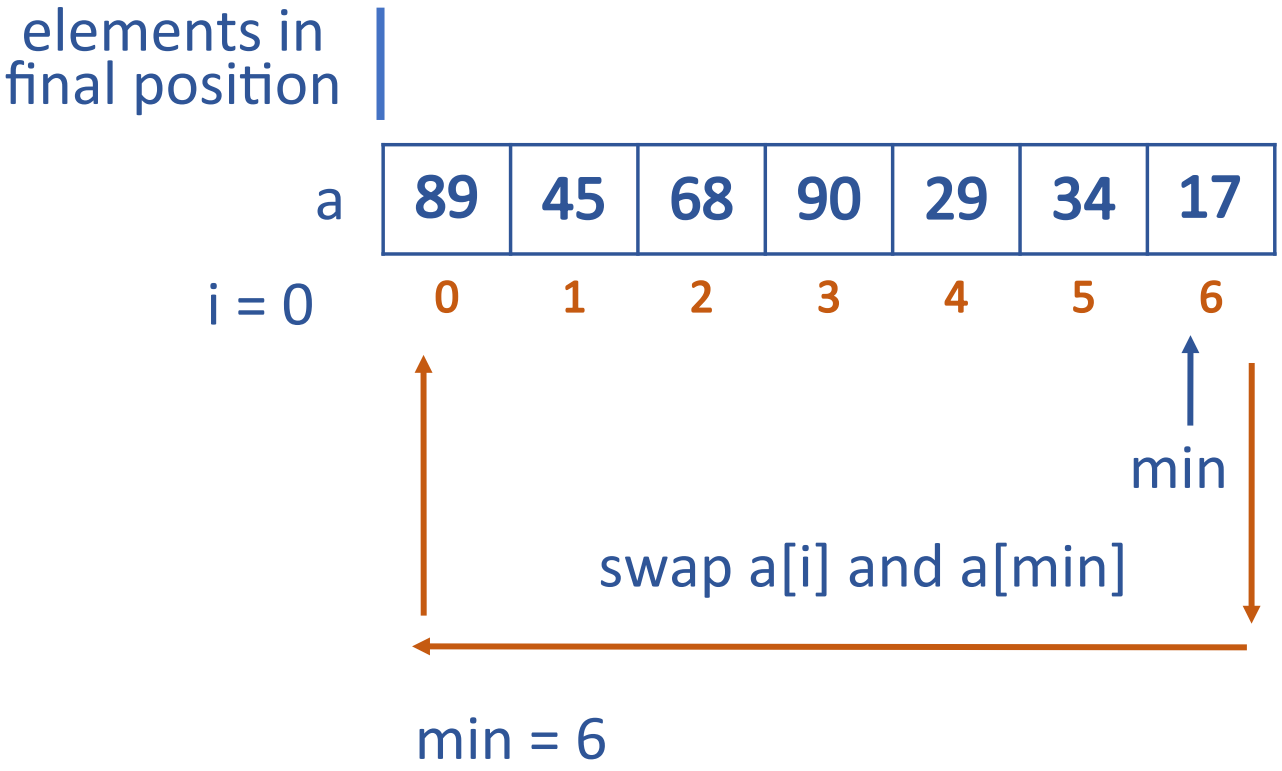
Selection Sort



Selection Sort



Selection Sort

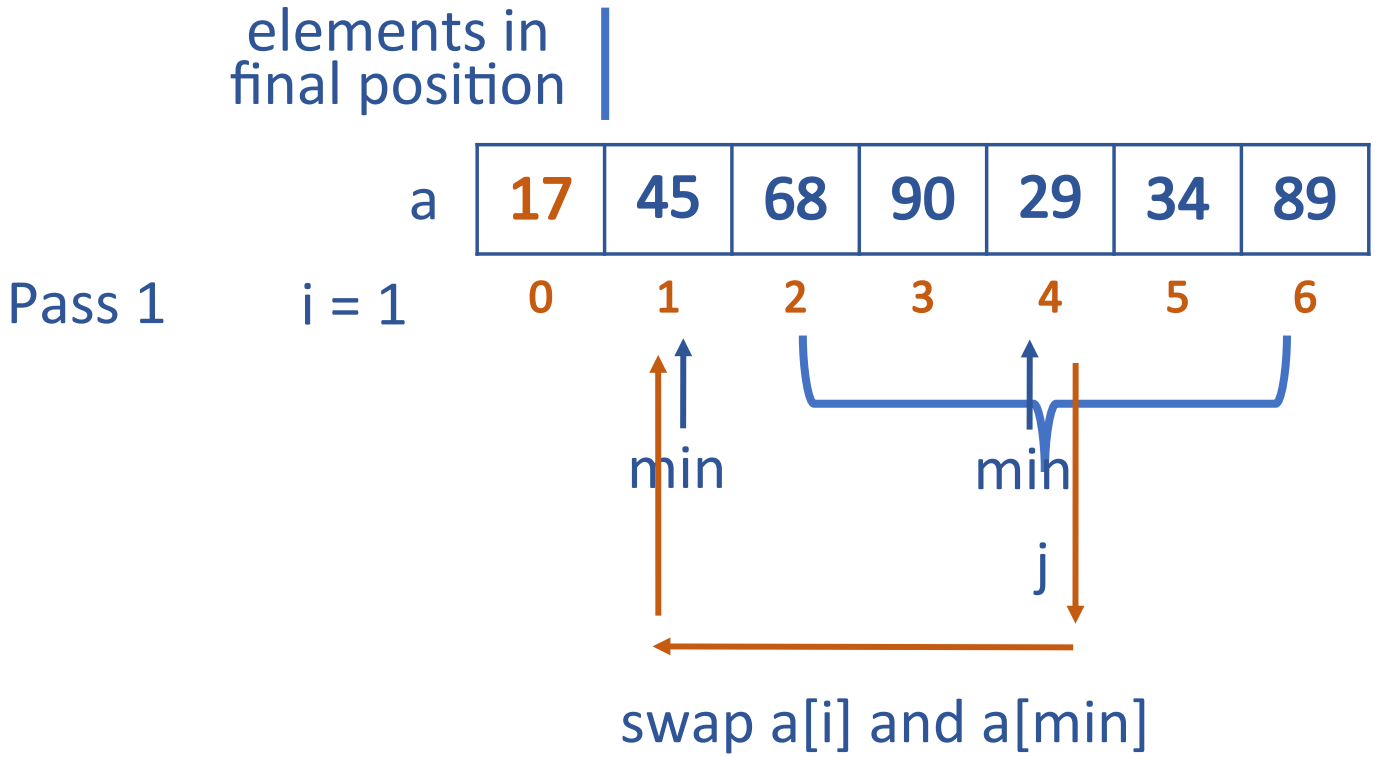


Selection Sort

elements in final position |

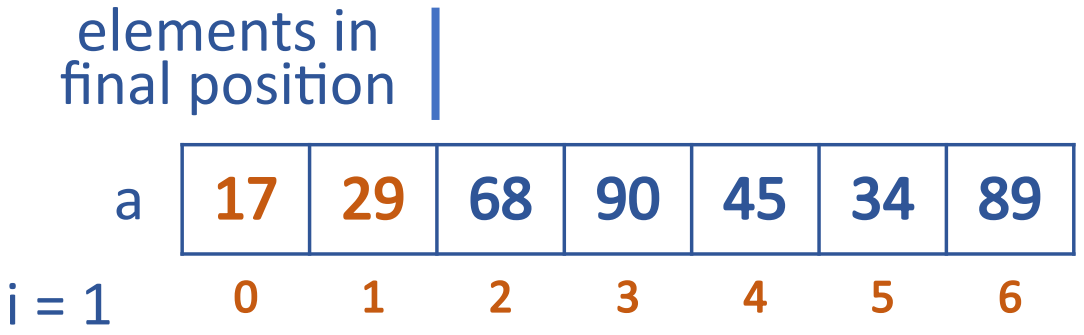
a	17	45	68	90	29	34	89
i = 0	0	1	2	3	4	5	6

Selection Sort

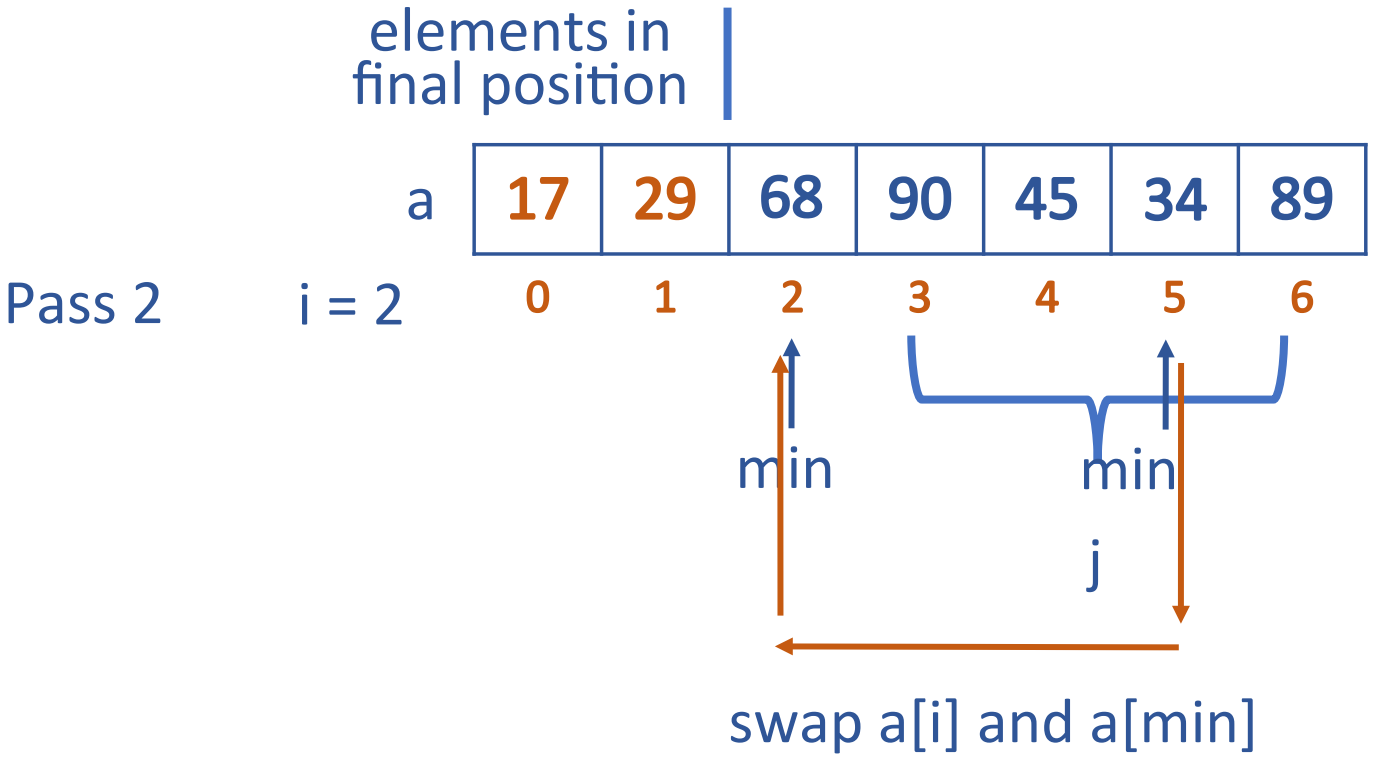


min = 4

Selection Sort

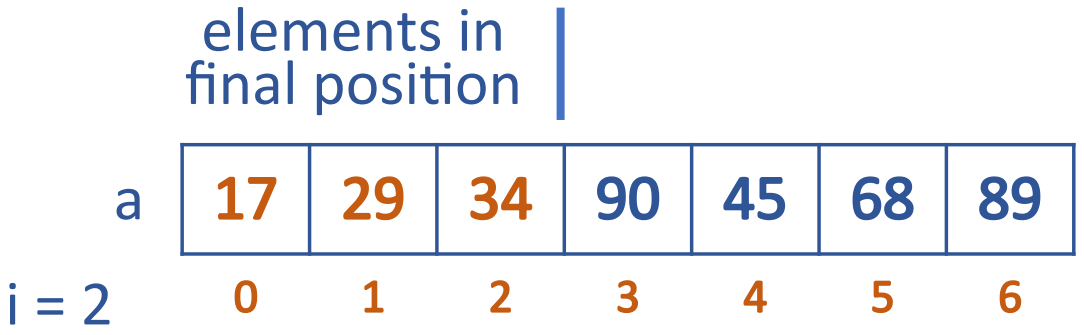


Selection Sort

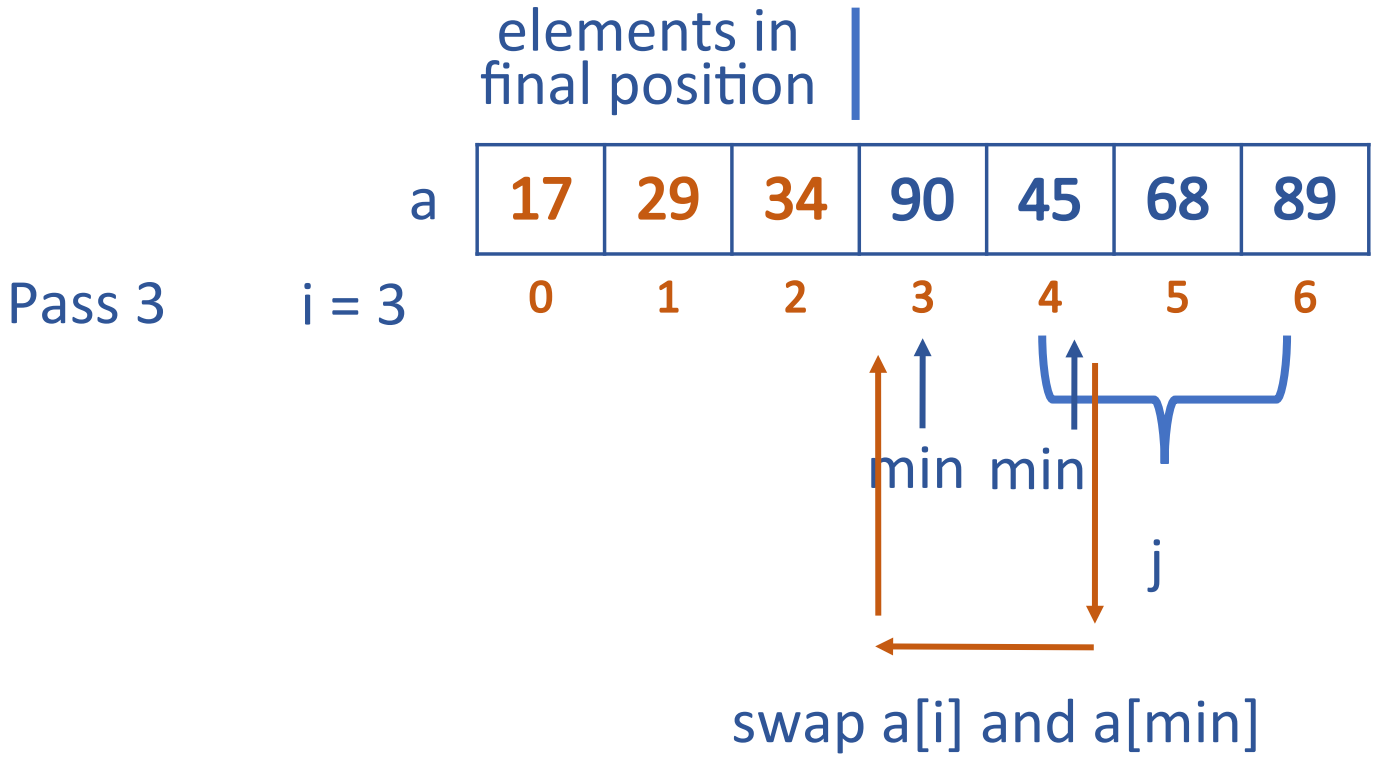


min = 5

Selection Sort

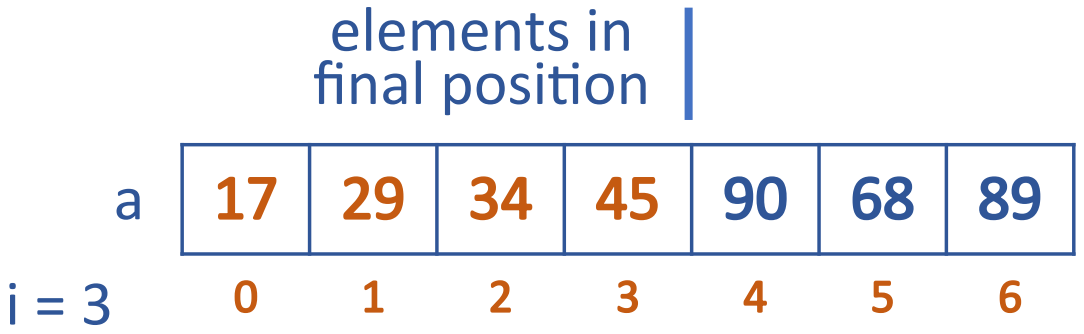


Selection Sort

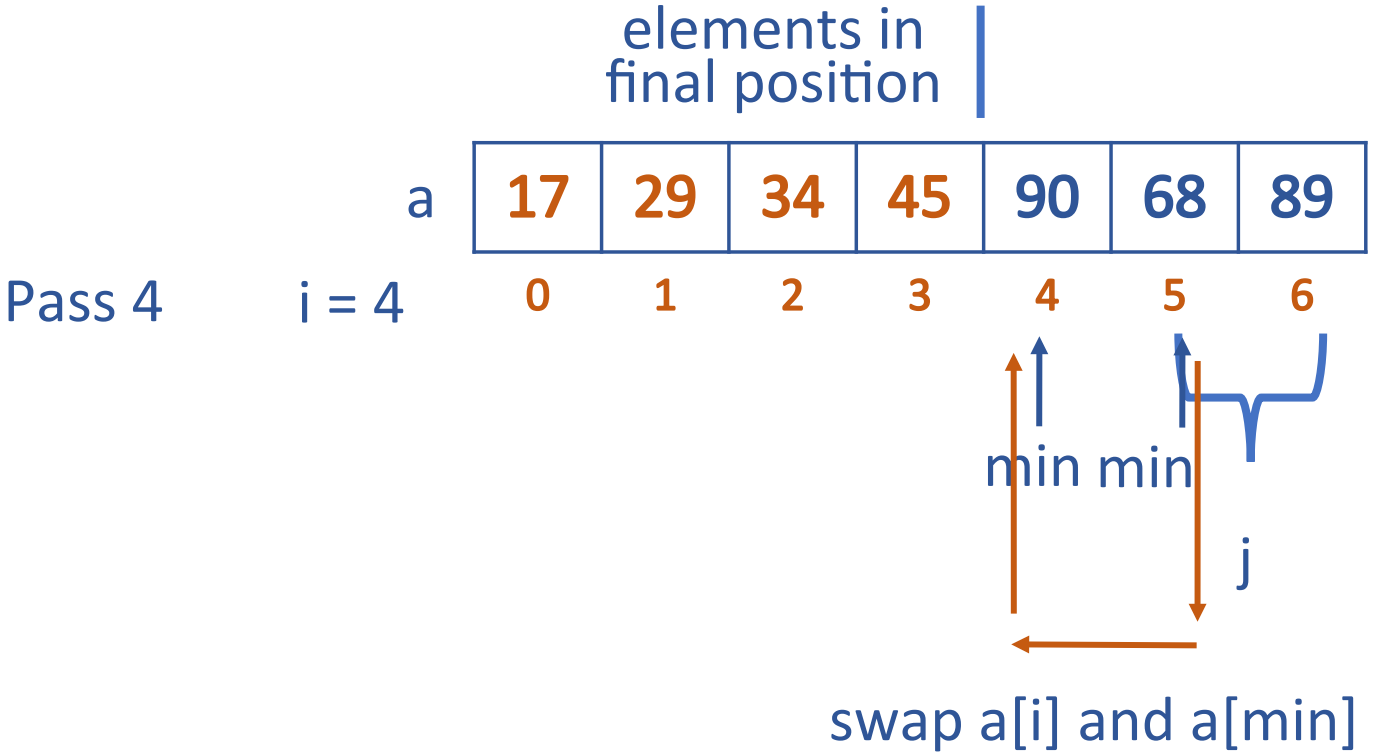


min = 4

Selection Sort

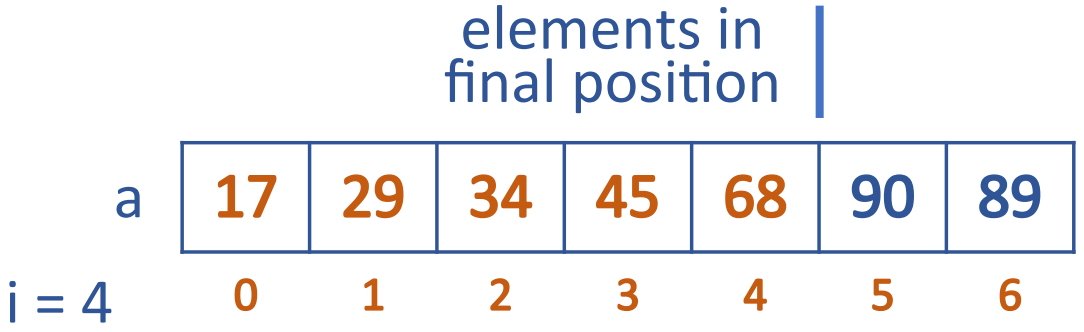


Selection Sort

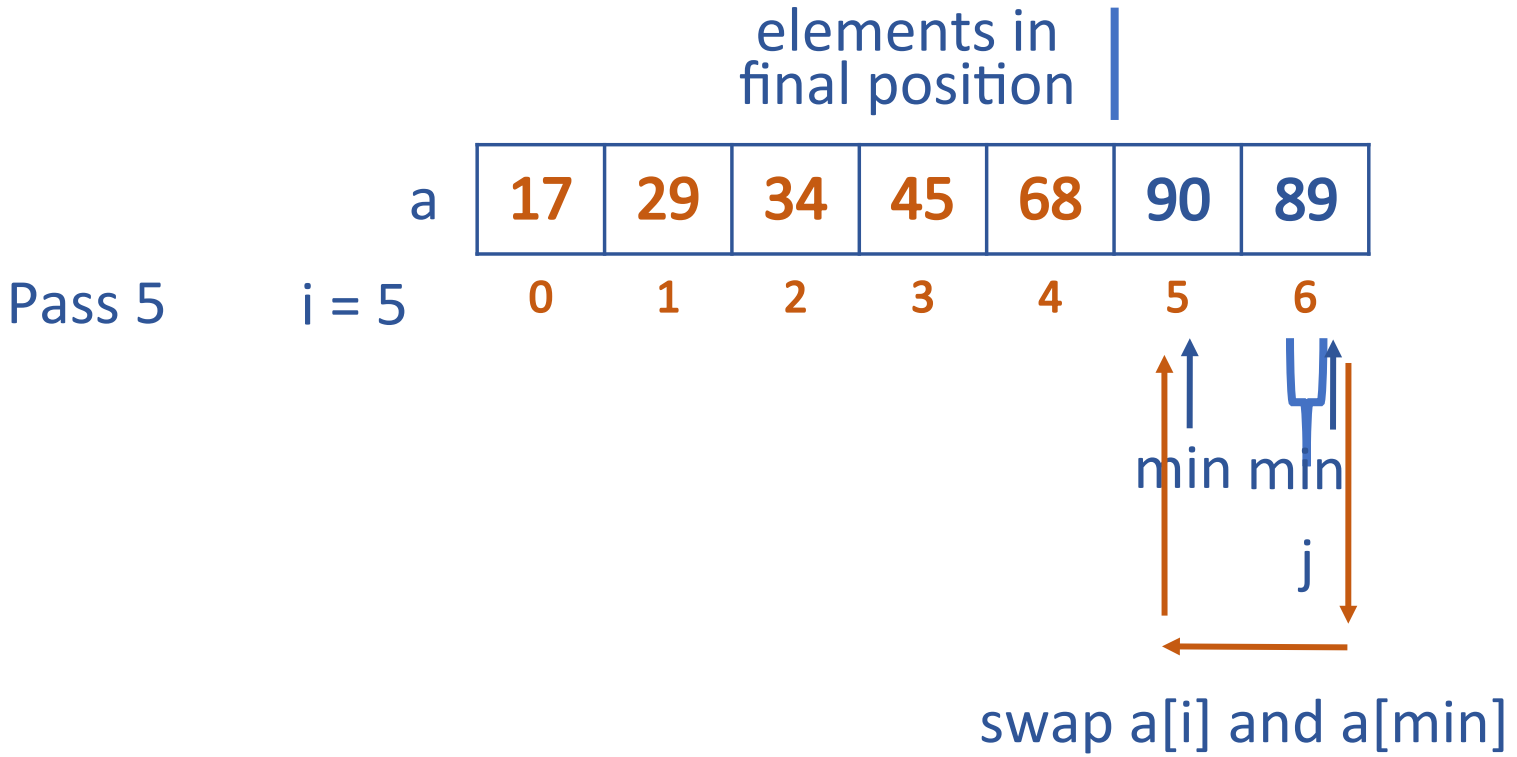


min = 5

Selection Sort

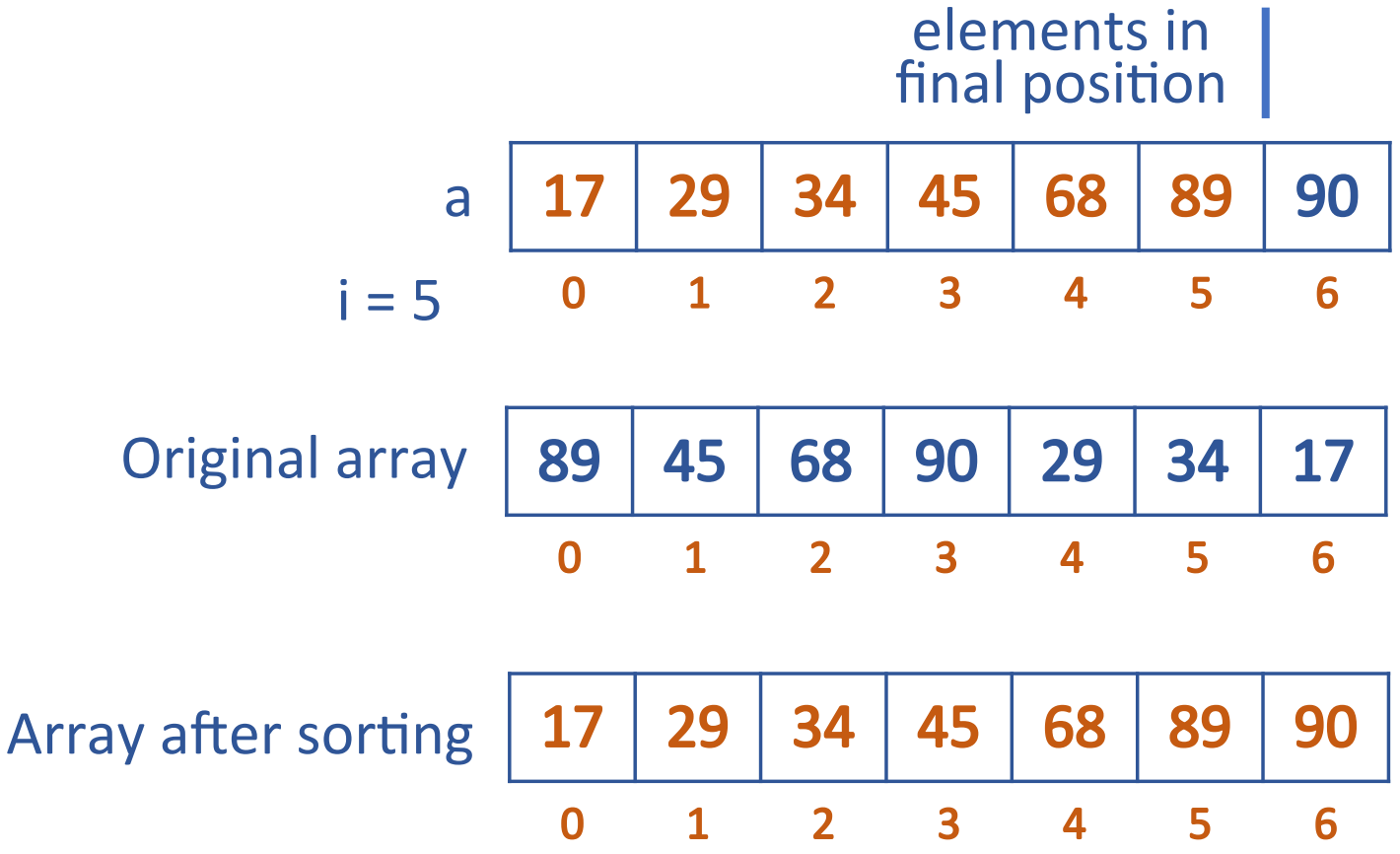


Selection Sort



min = 6

Selection Sort



Selection Sort

```
ALGORITHM  SelectionSort(A[0 .. n -1])
//Sorts a given array by selection sort
//Input: An array A[0 .. n - 1]  of orderable elements
//Output: Array A[0 .. n - 1]  sorted in ascending order
for i <- 0 to n - 2 do
        min <- i
        for j <- i+l to n-1 do
                if A[j] <  A[min]  min  <- j
        swap  A[i] and A[min]
```

Selection Sort

Selection Sort Analysis

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2}$$

Selection Sort is a $\Theta(n^2)$ algorithm

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS
UE19CS251

Shylaja S S
Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Bubble Sort

Major Slides Content: Anany Levitin

Shylaja S S

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Bubble Sort

- Compare adjacent elements of the list and exchange them if they are out of order
- By doing it repeatedly, we end up bubbling the largest element to the last position on the list
- The next pass bubbles up the second largest element and so on and after n – 1 passes, the list is sorted
- Pass i ($0 \leq i \leq n-2$) can be represented as follows:

$A[0], A[1], A[2], \ldots, A[j] \overset{?}{\longleftrightarrow} A[j+1], \ldots, A[n-i-1] \mid A[n-i] \leq \ldots \leq A[n-1]$

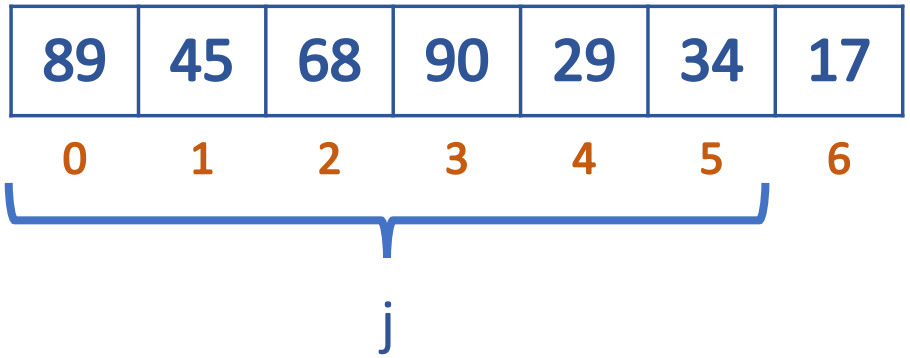
in their final positions

Another brute-force application to the sorting problem is to compare adjacent elements of the list and exchange them if they are out of order. By doing it repeatedly, we end up "bubbling up" the largest element to the last position on the list. The next pass bubbles up the second largest element, and so on, until after n − 1 passes the list is sorted. Pass i (0 <= i <= n − 2) of bubble sort can be represented by the following diagram

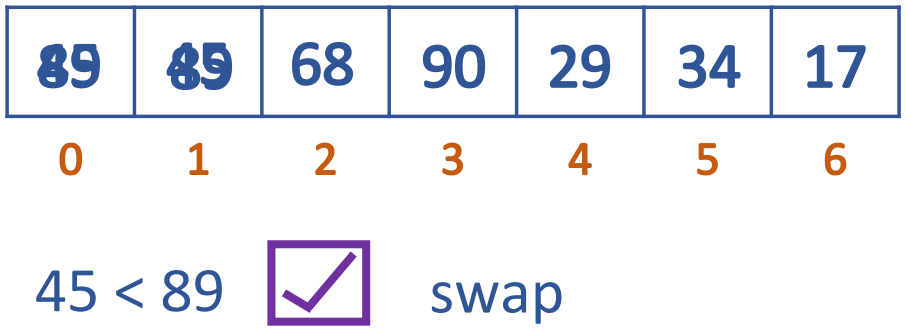
Bubble Sort

elements in final position

i = 0



j = 0



45 < 89 ☑ swap

Bubble Sort

elements in final position

i = 0

45	89	68	90	29	34	17
0	1	2	3	4	5	6

j

j = 1

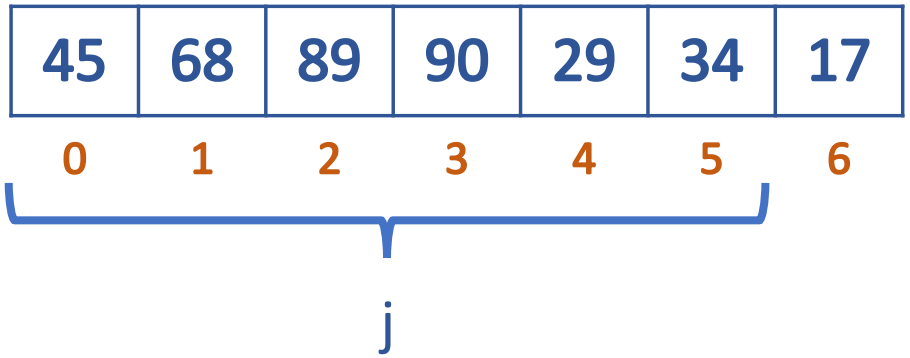
45	89	68	90	29	34	17
0	1	2	3	4	5	6

68 < 89 ☑ swap

Bubble Sort

elements in final position

i = 0



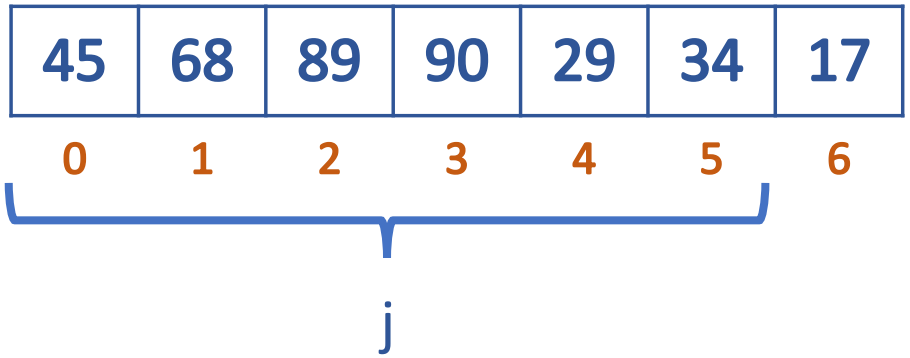
j = 2

90 < 89 ☒ no swap

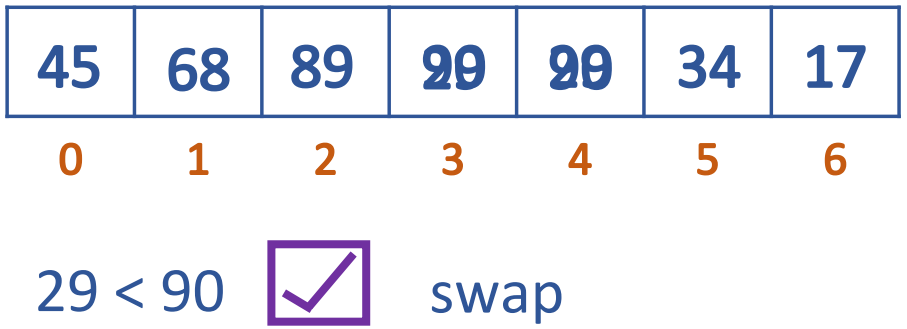
Bubble Sort

elements in final position

i = 0



j = 3

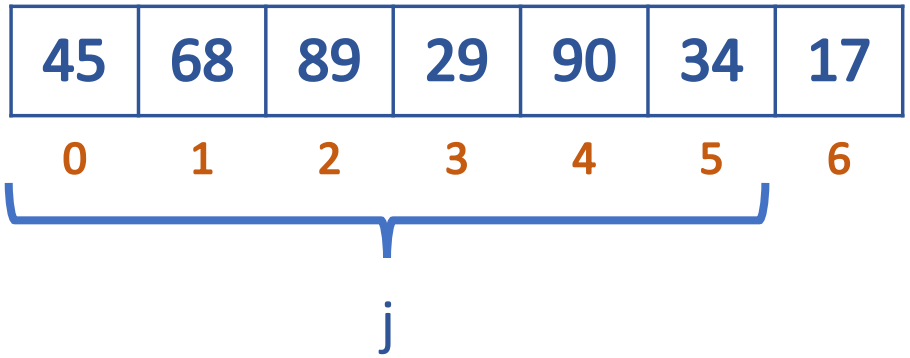


29 < 90 ☑ swap

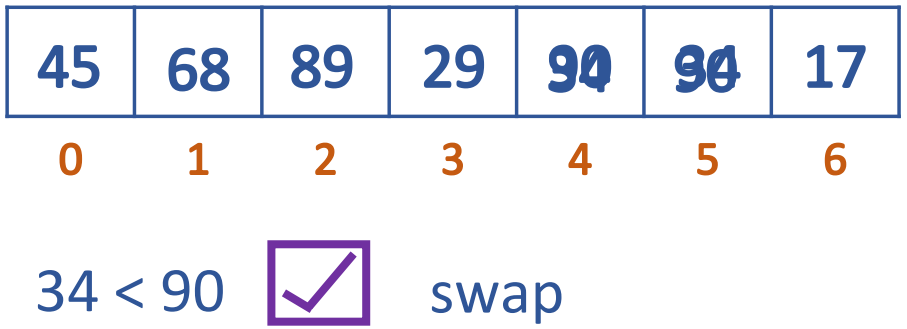
Bubble Sort

elements in final position

i = 0



j = 4

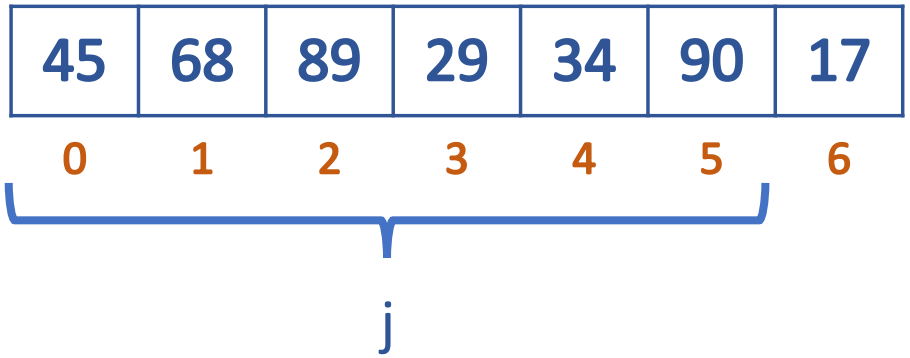


34 < 90 ☑ swap

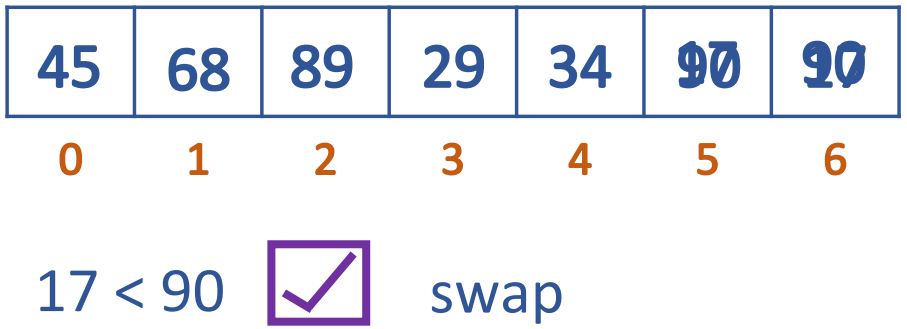
Bubble Sort

elements in final position

i = 0



j = 5

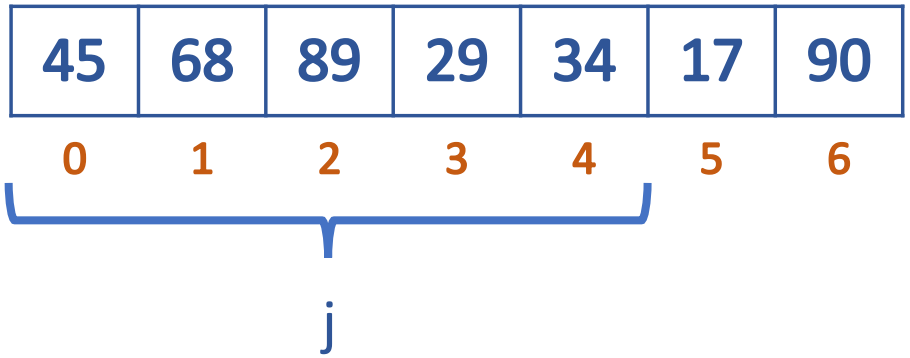


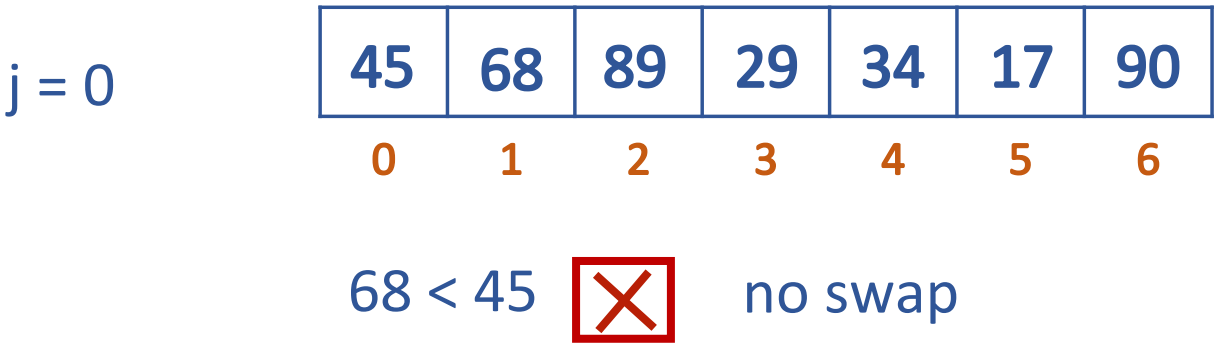
17 < 90 ☑ swap

Bubble Sort

After first iteration

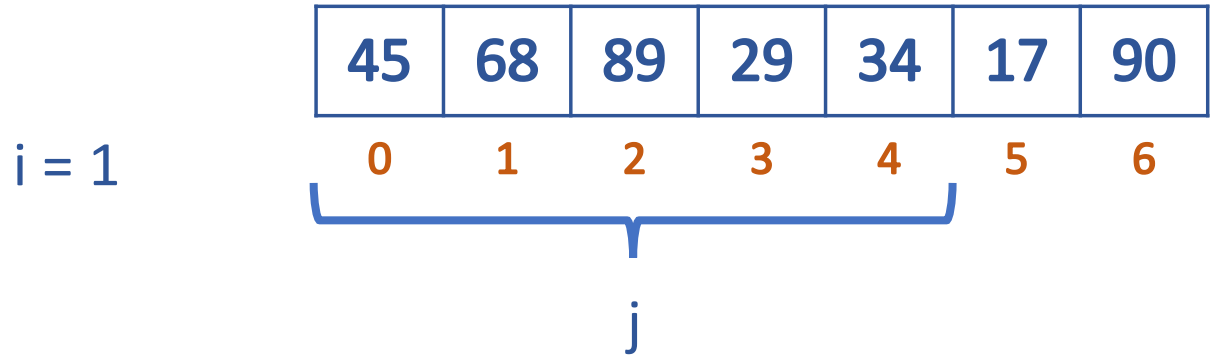
elements in final position

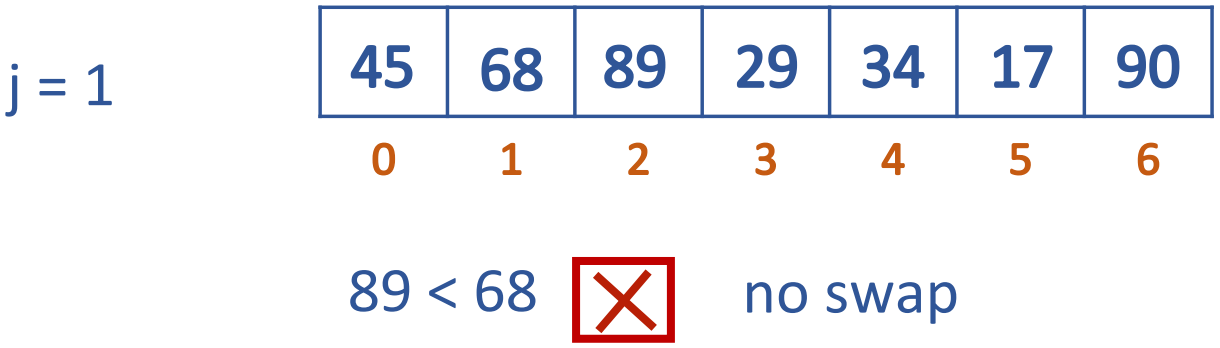




Bubble Sort

elements in final position





89 < 68 ☒ no swap

Bubble Sort

elements in final position

i = 1

45	68	89	29	34	17	90
0	1	2	3	4	5	6

j

j = 2

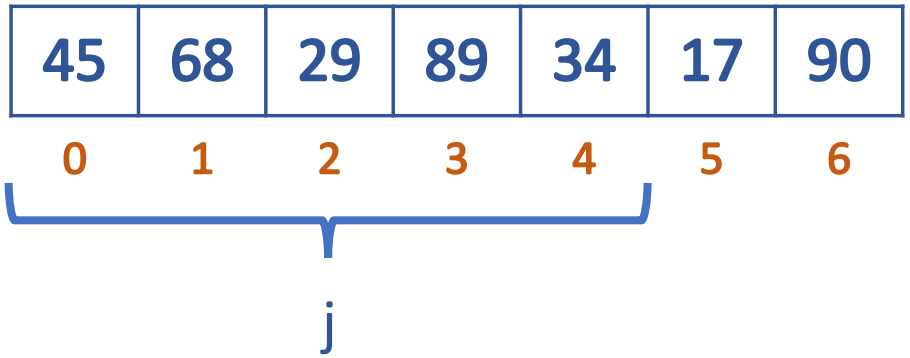
45	68	~~89~~	~~29~~	34	17	90
0	1	2	3	4	5	6

29 < 89 ☑ swap

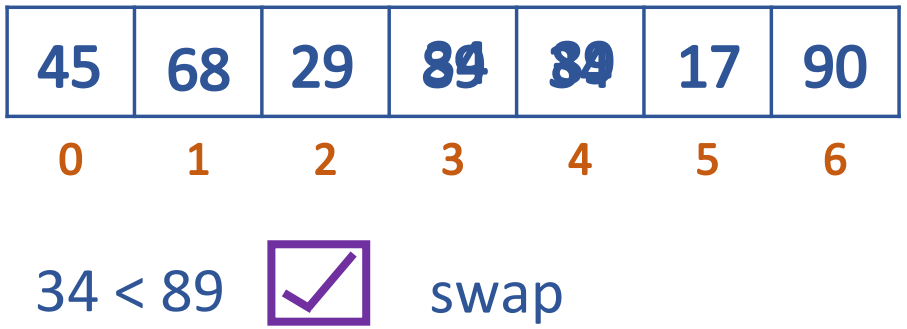
Bubble Sort

elements in final position

i = 1



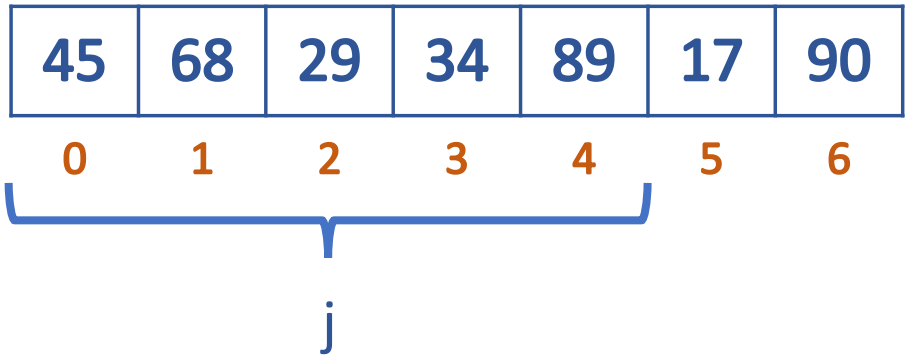
j = 3

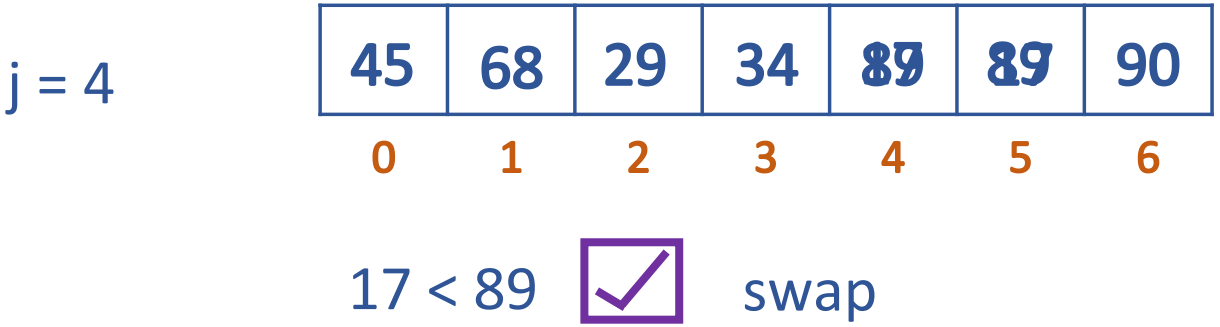


34 < 89 ☑ swap

Bubble Sort

elements in final position

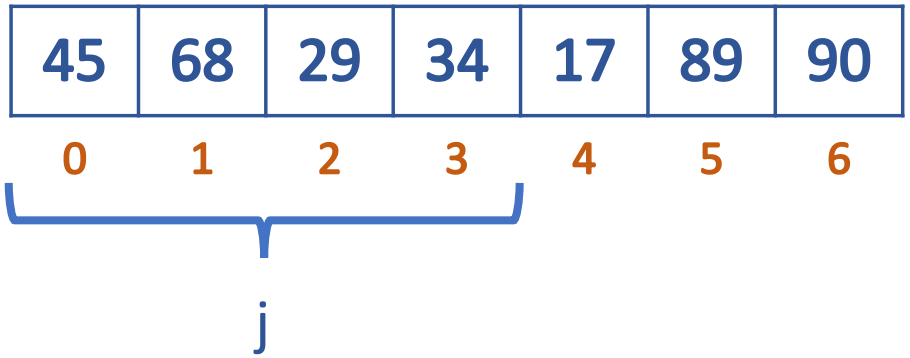


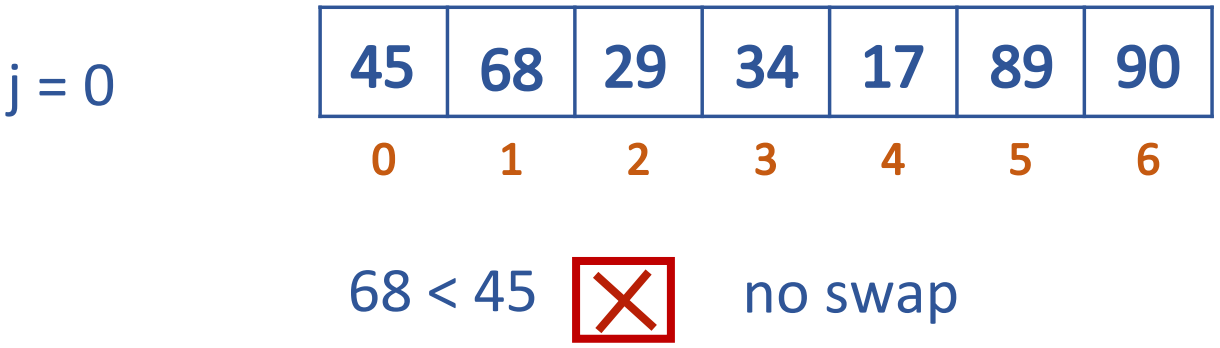


Bubble Sort

After second iteration

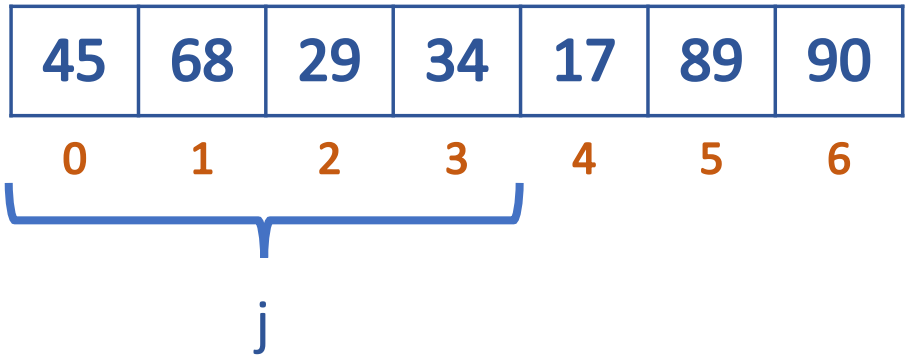
elements in final position

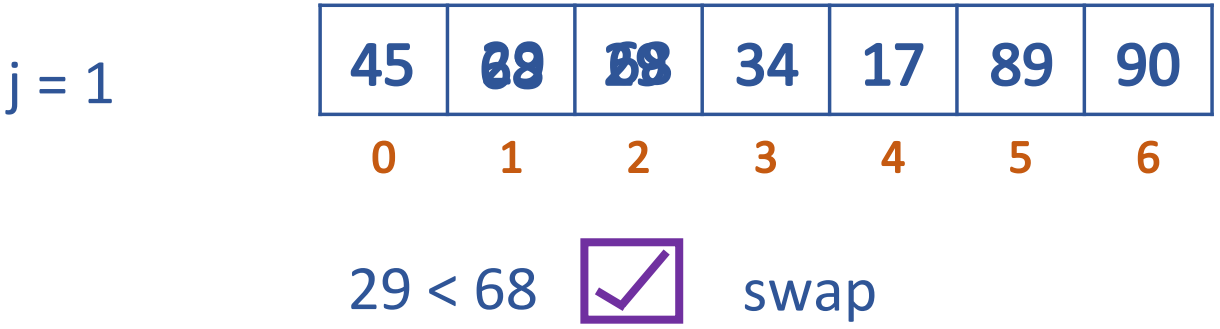




Bubble Sort

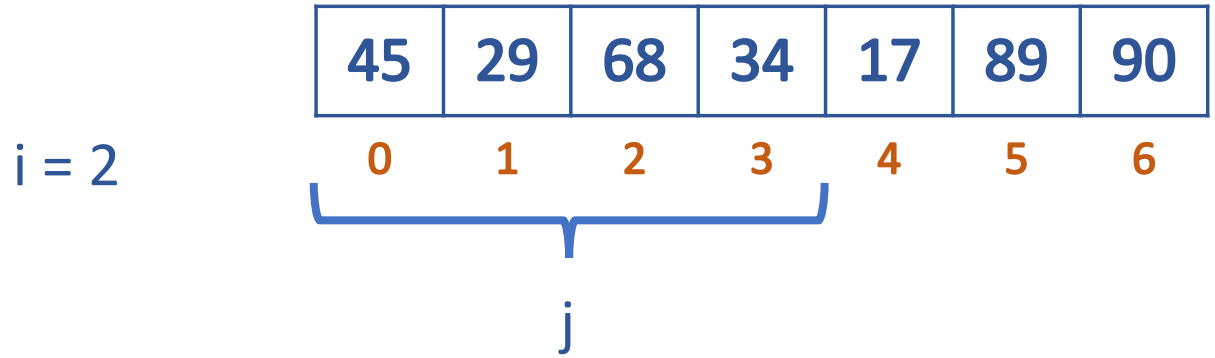
elements in final position

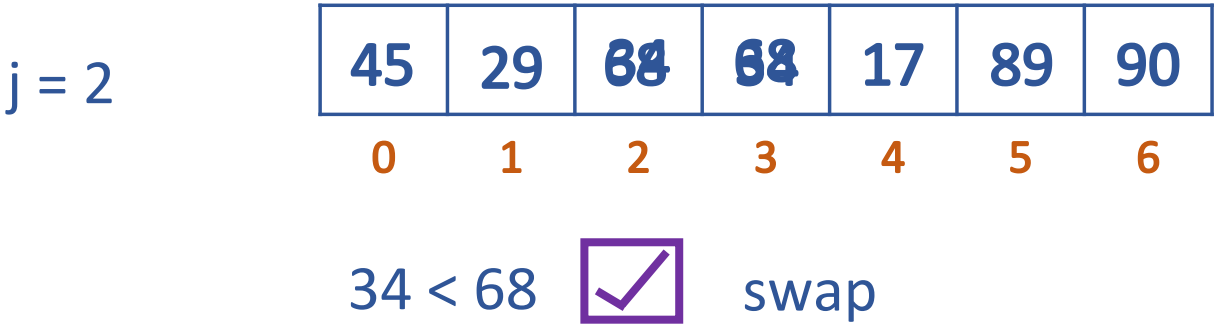




Bubble Sort

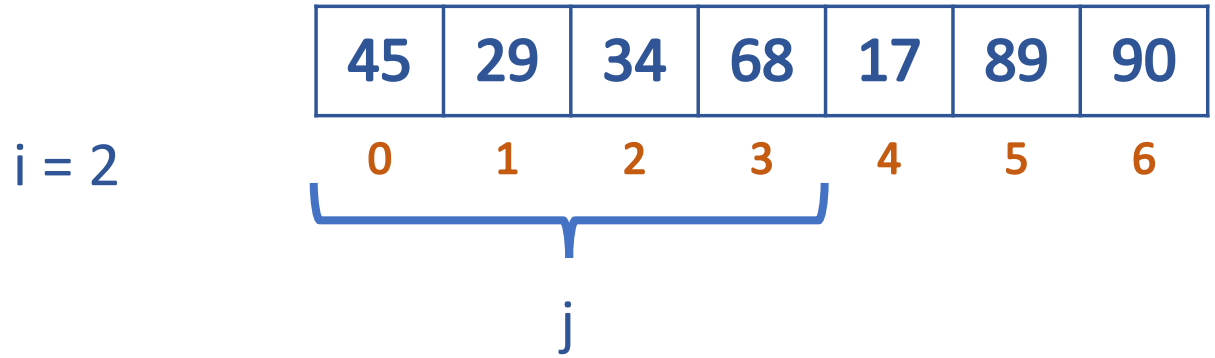
elements in final position

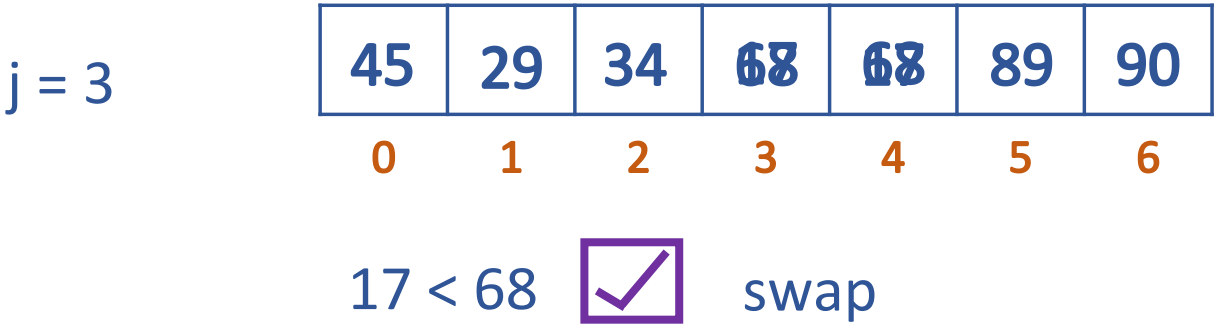




Bubble Sort

elements in final position

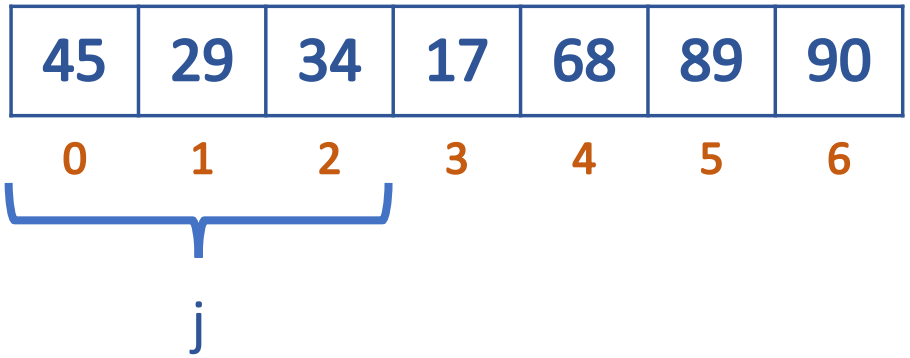


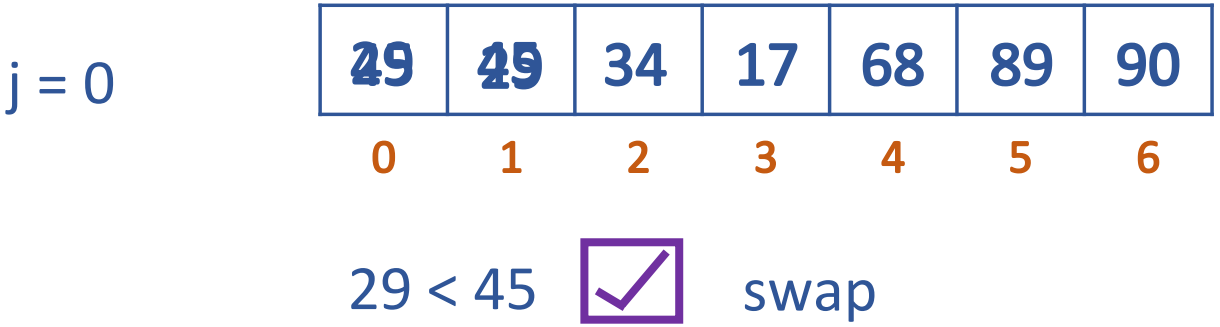


Bubble Sort

After third iteration

elements in final position

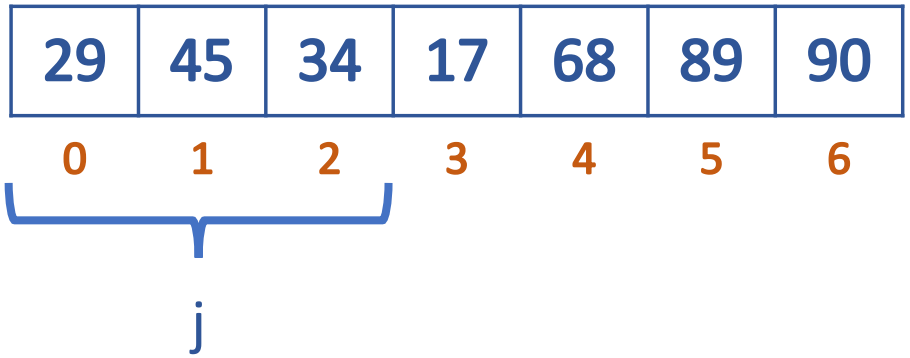




Bubble Sort

elements in final position

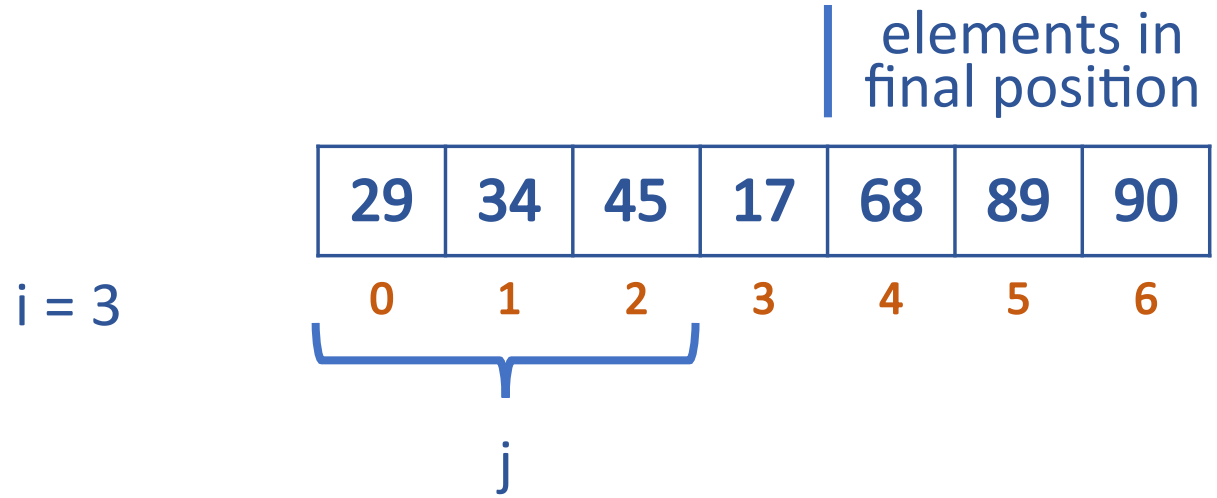
i = 3

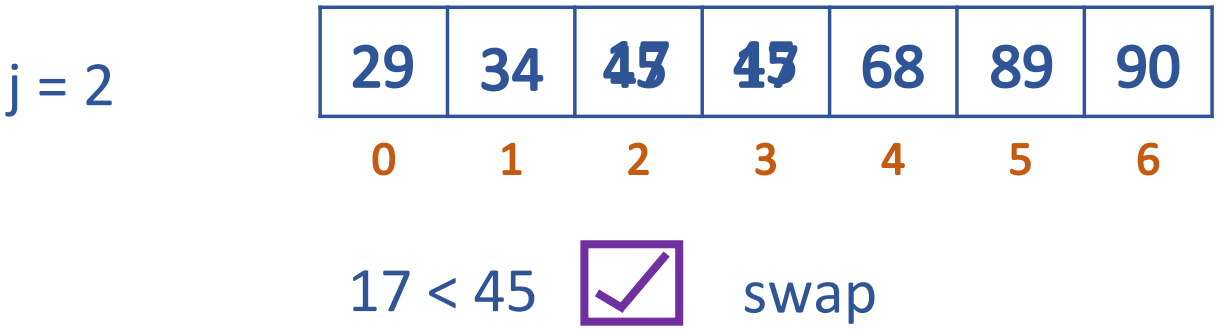


j = 1

34 < 45 ☑ swap

Bubble Sort

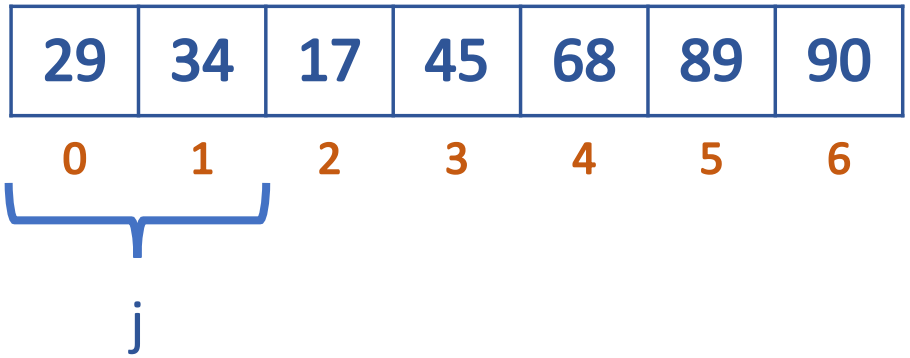

elements in final position
29 34 45 17 68 89 90
0 1 2 3 4 5 6
i = 3
j

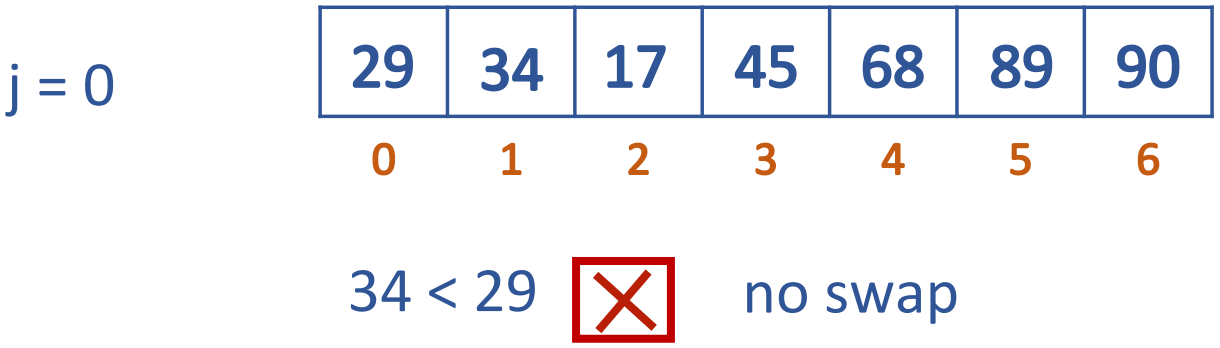

j = 2
29 34 45 17 68 89 90
0 1 2 3 4 5 6
17 < 45
swap

Bubble Sort

After fourth iteration

elements in final position

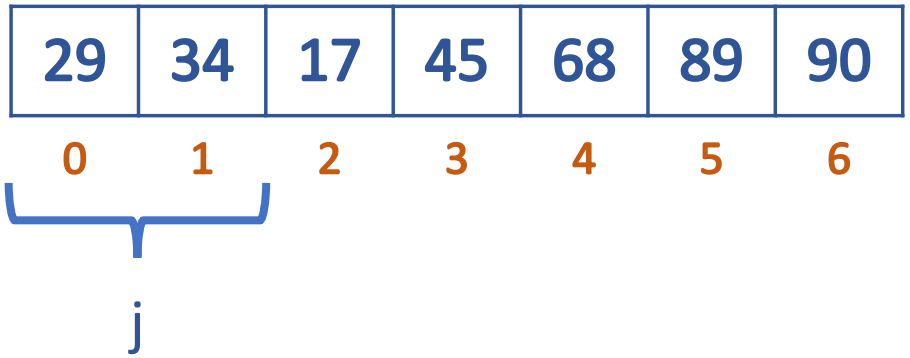


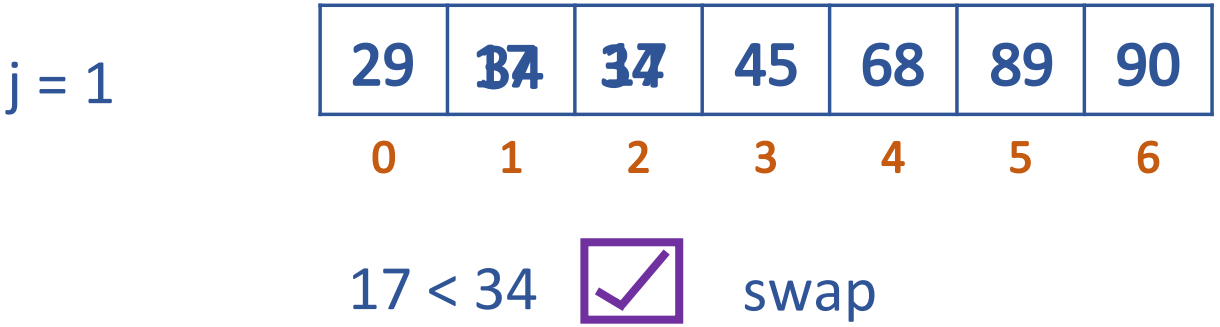


34 < 29 ☒ no swap

Bubble Sort

elements in final position

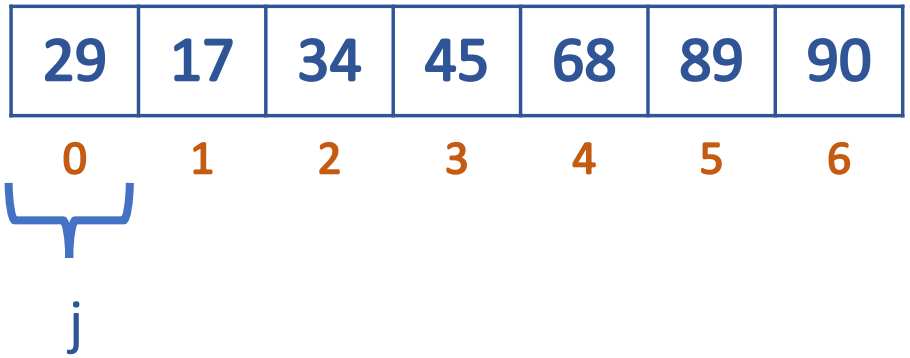


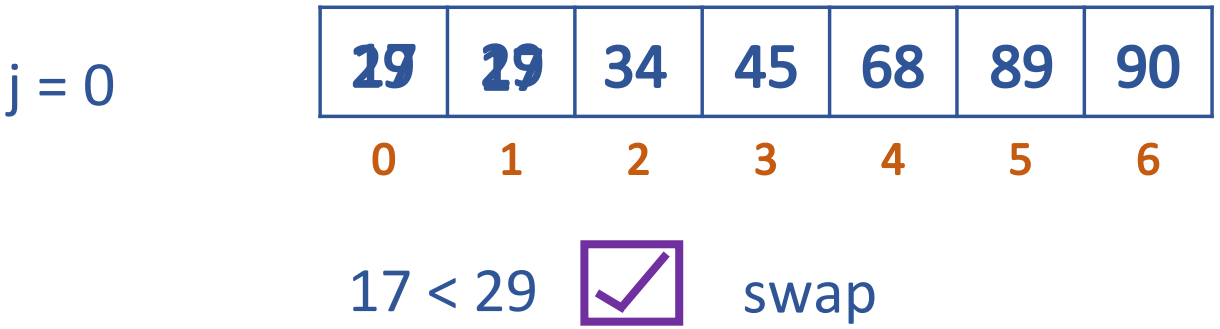


Bubble Sort

After fifth iteration

elements in final position





Bubble Sort

After sixth iteration | elements in final position

17	29	34	45	68	89	90
0	1	2	3	4	5	6

DESIGN AND ANALYSIS OF ALGORITHMS

Bubble Sort

ALGORITHM BubbleSort(A[0 .. n - 1])

//Sorts a given array by bubble sort in their final positions

//Input: An array A[0 .. n - 1] of orderable elements

//Output: Array A[0 .. n- 1] sorted in ascending order

for i <-- 0 to n - 2 do

  for j <-- 0 to n - 2 - i do

    if A[j + 1] < A[j] swap A[j] and A[j + 1]

how to optimize bubble sort done in notebook check there properly before exam

Bubble Sort

Bubble Sort Analysis

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1]$$

$$= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Bubble Sort is a $\Theta(n^2)$ algorithm

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS
UE19CS251

Shylaja S S
Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Sequential Search

Major Slides Content: Anany Levitin

Shylaja S S

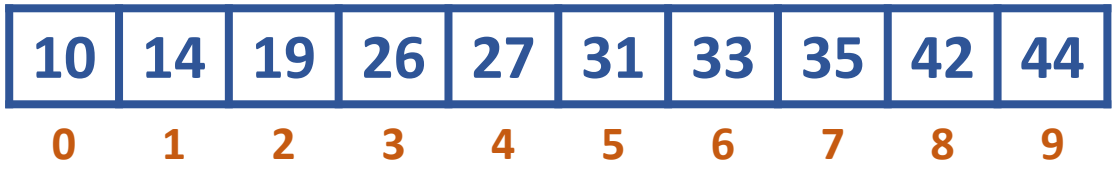
Department of Computer Science & Engineering

Sequential Search

- Compares successive elements of a given list with a given search key until:
  - A match is encountered (Successful Search)
  - List is exhausted without finding a match (Unsuccessful Search)
- An improvisation to the algorithm is to append the key to the end of the list
- This means the search has to be successful always and we can eliminate the end of list check

Sequential Search

- Sequential / Linear Search

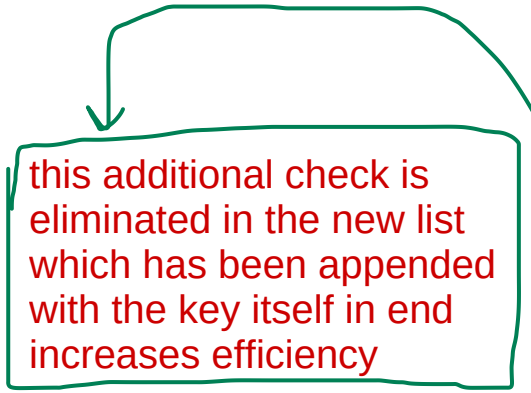


- For key = 33, 6 is returned
- For key = 50, -1 is returned

DESIGN AND ANALYSIS OF ALGORITHMS

Sequential Search

```
AlGORITHM  SequentialSearch2(A[0 .. n ],  K)
//Implements sequential search with a search key as  a sentinel
//Input: An array A of n elements and a search key  K
//Output: The index of the first element in A[0 .. n -1] whose value is
//  equal to K or -1 if no such element is found
A[n]<---K
i<---0
while A[i] ≠K  do
   i<--- i + 1
if i  <  n return i
else return -1
```




```
ALGORITHM SequentialSearch(A[0..n −1],K)
//Searches for a given value in a given array by sequential search
//Input: An array A[0..n − 1]and a search key K
//Output: The index of the first element in A that matches K
//
or −1if there are no matching elements
i <--0
while i<nand A[i]=K do
        i <--i+1
if i<n  return i
else return −1
```

Sequential Search

Sequential Search Analysis

- Sequential Search is a Θ(n) algorithm

Another straightforward improvement can be incorporated in sequential search if a given list is known to be sorted: searching in such a list can be stopped as soon as an element greater than or equal to the search key is encountered

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS
UE19CS251

Shylaja S S
Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

String Matching

Major Slides Content: Anany Levitin

Shylaja S S

Department of Computer Science & Engineering

Brute – Force String Matching

written in notebook properly

PES UNIVERSITY ONLINE

String Matching - Terms

- pattern:

  a string of m characters to search for

- text:

  a (longer) string of n characters to search in

- problem:

  find a substring in the text that matches the pattern

String Matching Idea

Step 1: Align pattern at beginning of text

Step 2: Moving from left to right, compare each character of pattern to the corresponding character in text until:

- all characters are found to match (successful search); or
- a mismatch is detected

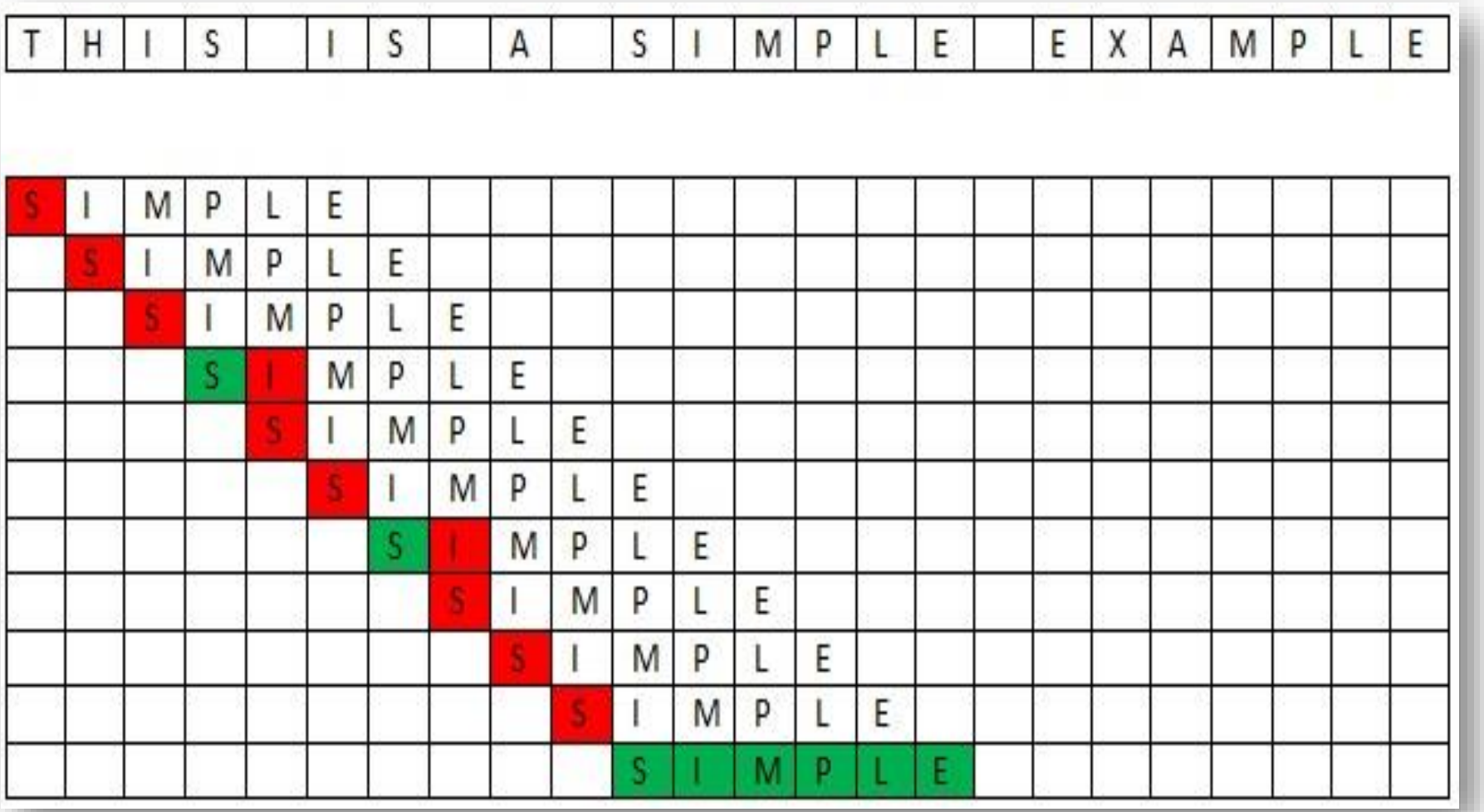
Step 3: While pattern is not found and the text is not yet exhausted, realign pattern one position to the right and repeat Step 2

String Matching

```
ALGORITHM  BruteForceStringMatch(T[0 .. n -1], P[0 .. m -1])
//Implements brute-force string matching
//Input: An array T[0 .. n  - 1]  of n  characters representing a text
// and an array  P[0 .. m - 1] of m characters representing a pattern
//Output: The index of the first character in the text that starts a
//matching substring or -1 if the search is unsuccessful
for i ← 0 to n-m do
   j ← 0
   while  j  <  m  and P[j] =  T[i + j] do
            j ←j+1
   if j  =  m  return i
return -1
```

DESIGN AND ANALYSIS OF ALGORITHMS

String Matching Example



String Matching Analysis

Worst Case:

- The algorithm might have to make all the ‘m’ comparisons for each of the (n-m+1) tries
- Therefore, the algorithm makes m(n-m+1) comparisons
- Brute Force String Matching is a O(nm) algorithm

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

UE19CS251

see textbook page number 141-142
before exam once

Shylaja S S
Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Travelling Salesman Problem

Major Slides Content: Anany Levitin

Shylaja S S

Department of Computer Science & Engineering

Exhaustive Search

- Exhaustive Search is a brute – force problem solving technique
- It suggests generating each and every element of the problem domain, selecting those of them that satisfy all the constraints and then finding a desired element
- The desired element might be one which minimizes or maximizes a certain characteristic
- Typically the problem domain involves combinatorial objects such as permutations, combinations and subsets of a given set

Exhaustive Search - Method

- Generate a list of all potential solutions to the problem in a systematic manner
- Evaluate potential solutions one by one, disqualifying infeasible ones and, for an optimization problem, keeping track of the best one found so far
- When search ends, announce the solution(s) found

DESIGN AND ANALYSIS OF ALGORITHMS

Travelling Salesman Problem

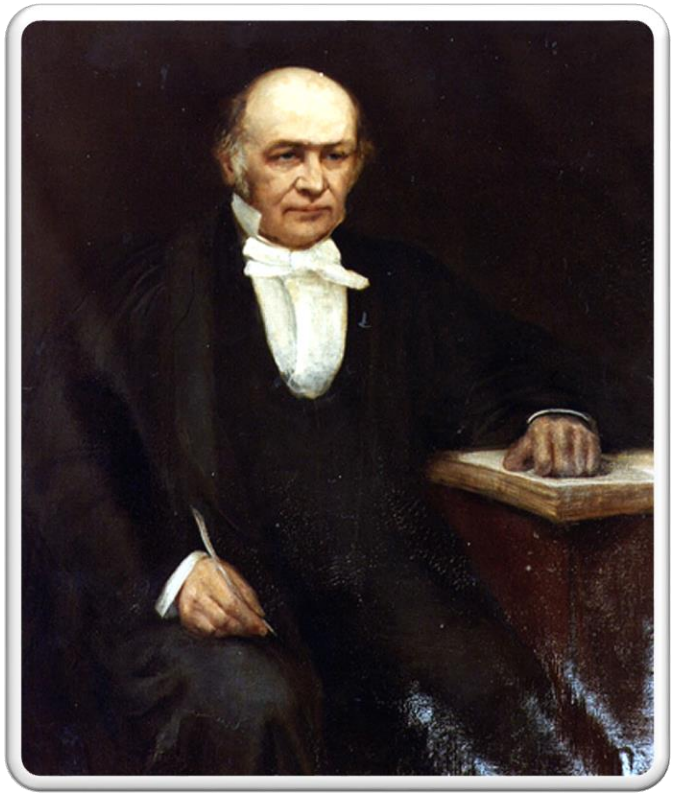
- Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city?

Alternative way to state the problem:

- Find the shortest Hamiltonian Circuit in a weighted connected graph

TSP: History and Relevance

- The Travelling Salesman Problem was mathematically formulated by Irish Mathematician Sir William Rowan Hamilton
- It is one of the most intensively studied problems in optimization
- It has applications in logistics and planning

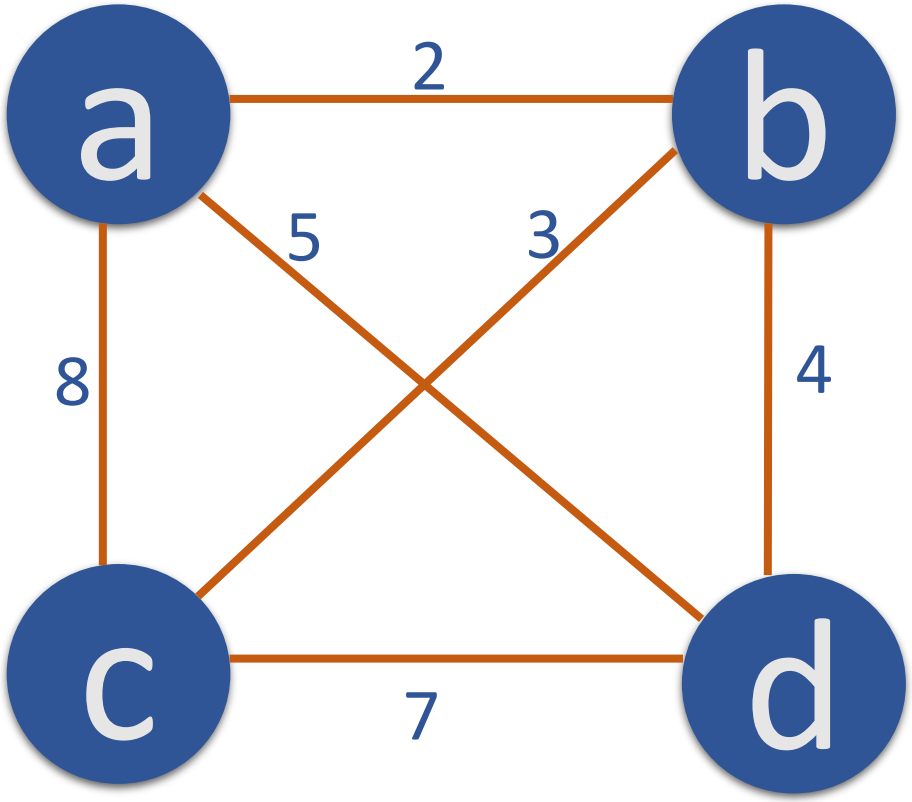


PES UNIVERSITY ONLINE

DESIGN AND ANALYSIS OF ALGORITHMS

Travelling Salesman Problem

Example

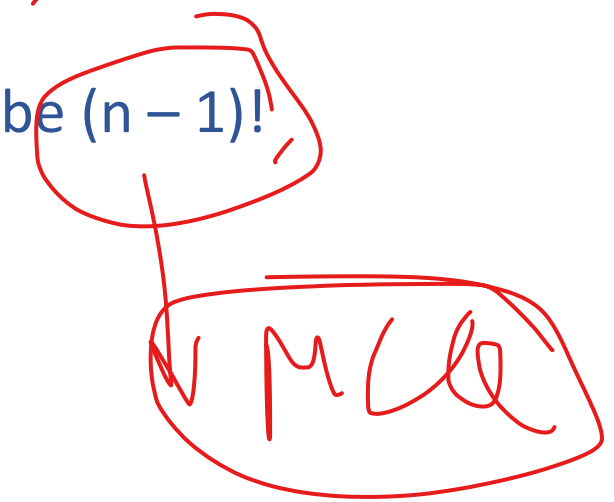


Tour	Length
a→b→c→d→a	**2+3+7+5 = 17**
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	**5+7+3+2 = 17**

Travelling Salesman Problem

Efficiency

- The Exhaustive Search solution to the Travelling Salesman problem can be obtained by keeping the origin city constant and generating permutations of all the other n – 1 cities
- Thus, the total number of permutations needed will be (n – 1)!



THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS
UE19CS251

Shylaja S S
Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Knapsack Problem

Major Slides Content: Anany Levitin

Shylaja S S

Department of Computer Science & Engineering

Knapsack Problem

Given n items:

- weights: w_1 w_2 … w_n
- values: v_1 v_2 … v_n
- a knapsack of capacity W

Find the most valuable subset of items that fit into the knapsack

Knapsack Problem

Example

Knapsack Capacity W = 16

Item	Weight	Value
1	2	20
2	5	30
3	10	50
4	5	10

Knapsack Problem

Subset	Total Weight	Total Value
{0}	0	0
{1}	2	20
{2}	5	30
{3}	10	50
{4}	5	10
{1, 2}	7	50
{1, 3}	12	70
{1, 4}	7	30
{2, 3}	15	80
{2, 4}	10	40
{3, 4}	15	60
{1, 2, 3}	17	Not Feasible
{1, 2, 4}	12	60
{1, 3, 4}	17	Not Feasible
{2, 3, 4}	20	Not Feasible
{1, 2, 3, 4}	22	Not Feasible

total subsets = 2^n
2^4=16 here
n=number of items or elements in the set

Knapsack Problem by Exhaustive Search

PES UNIVERSITY ONLINE

Knapsack Problem

- The Exhaustive Search solution to the Knapsack Problem is obtained by generating all subsets of the set of n items given and computing the total weight of each subset in order to identify the feasible subsets
- The number of subsets for a set of n elements is 2^n
- The Exhaustive Search solution to the Knapsack Problem belongs to $\Omega(2^n)$

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

see page number 145-146 before exam in textbook

DESIGN AND ANALYSIS OF ALGORITHMS
UE19CS251

Shylaja S S
Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Assignment Problem

Major Slides Content: Anany Levitin

Shylaja S S

Department of Computer Science & Engineering

The Assignment Problem

- There are n people who need to be assigned to n jobs, one person per job. The cost of assigning person i to job j is C[i, j]. Find an assignment that minimizes the total cost

The Assignment Problem

Example

	Job 1	Job 2	Job 3	Job 4
Person 1	9	2	7	8
Person 2	6	4	3	7
Person 3	5	8	1	8
Person 4	7	6	9	4

The Assignment Problem

Algorithmic Plan

1. Generate all legitimate assignments
2. Compute their costs
3. Select the cheapest one

The Assignment Problem

The Assignment Problem by Exhaustive Search

Assignment	Cost
1, 2, 3, 4	9 + 4 + 1 + 4 = 18
1, 3, 4, 2	9 + 8 + 9 + 7 = 33
1, 4, 3, 2	9 + 6 + 1 + 7 = 23
1, 4, 2, 3	9 + 6 + 3 + 8 = 26
1, 3, 2, 4	9 + 8 + 3 + 4 = 24
1, 2, 4, 3	9 + 4 + 9 + 8 = 30

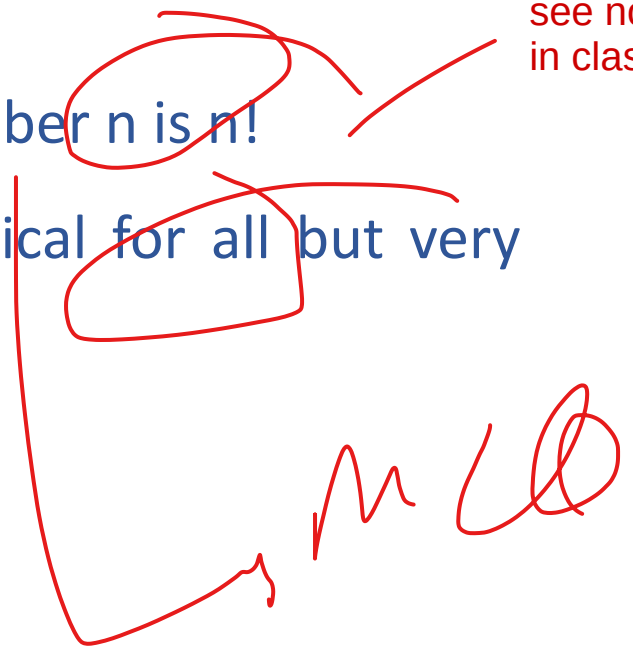
etc.,

The Assignment Problem

Efficiency

- The Assignment Problem is solved by generating all permutations of n
- The number of permutations for a given number n is n!
- Therefore, the exhaustive search is impractical for all but very small instances of the problem

see notebook also once the cw you wrote in class



THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

Design and Analysis of Algorithms

Vandana M L
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Important Problem Types

Slides courtesy of **Anany Levitin**

Vandana M L
Department of Computer Science & Engineering

Design and Analysis of Algorithms

Important Problem Types

- sorting
- searching
- string processing
- graph problems
- combinatorial problems
- geometric problems
- numerical problems

Design and Analysis of Algorithms

Important Problem Types: Sorting

- Rearrange the items of a given list in ascending order.
  - Input: A sequence of n numbers <a1, a2, …, an>
  - Output: A reordering <a'_1, a'_2, …, a'_n> of the input sequence such that $a'_1 \leq a'_2 \leq \ldots \leq a'_n$.
- Why sorting?
  - Help searching
  - Algorithms often use sorting as a key subroutine.
- Sorting key

  A specially chosen piece of information used to guide sorting.

  Example: sort student records by SRN.

Important Problem Types: Sorting

- Rearrange the items of a given list in ascending order.
- Examples of sorting algorithms
  - Selection sort
  - Bubble sort
  - Insertion sort
  - Merge sort
  - Heap sort …
- Evaluate sorting algorithm complexity: the number of key comparisons.
- Two properties
  - Stability: A sorting algorithm is called stable if it preserves the relative order of any two equal elements in its input.
  - In place : A sorting algorithm is in place if it does not require extra memory, except, possibly for a few memory units.

Design and Analysis of Algorithms

Important Problem Types: Searching

Find a given value, called a search key, in a given set.

Examples of searching algorithms

- Sequential searching
- Binary searching…

Important Problem Types: String Processing

A string is a sequence of characters from an alphabet.

Text strings: letters, numbers, and special characters.

String matching: searching for a given word/pattern in a text.

Text: I am a computer science graduate

Pattern: computer

Important Problem Types: Graph Problems

Definition

Graph G is represented as a pair G= (V, E),
where V is a finite set of vertices and E is a finite set of edges

Modeling real-life problems

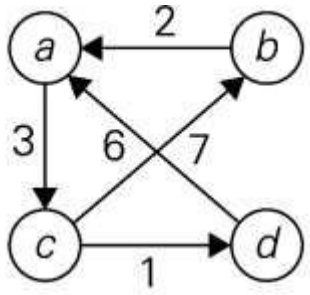
- Modeling WWW
- communication networks
- Project scheduling …

Examples of graph algorithms

- Graph traversal algorithms
- Shortest-path algorithms
- Topological sorting

Shortest paths in a graph

To find the distances from each vertex to all other vertices.



(a)

$$W = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{array}$$

(b)

$$D = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 7 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix} \end{array}$$

(c)

FIGURE 8.5 (a) Digraph. (b) Its weight matrix. (c) Its distance matrix.

Minimum cost spanning tree

- A spanning tree of a connected graph is its connected acyclic sub graph (i.e. a tree).

A spanning tree is defined as a tree-like subgraph of a connected, undirected graph that includes all the vertices of the graph. Or, to say in Layman's words, it is a subset of the edges of the graph that forms a tree (acyclic) where every node of the graph is a part of the tree.

E = V-1

The minimum spanning tree has all the properties of a spanning tree with an added constraint of having the minimum possible weights among all possible spanning trees.

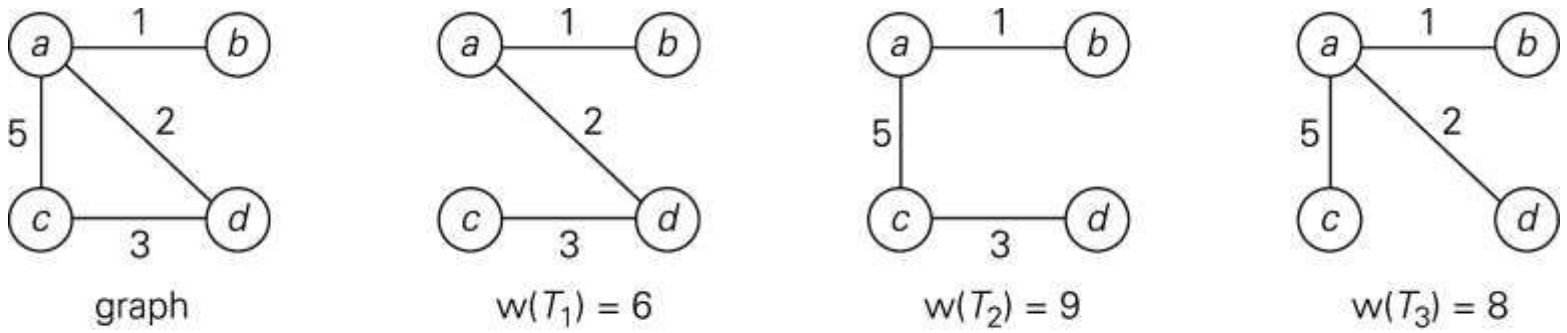
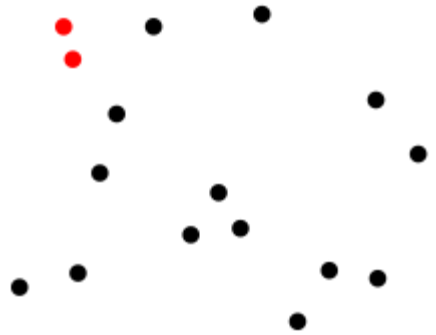


FIGURE 9.1 Graph and its spanning trees; T_1 is the minimum spanning tree

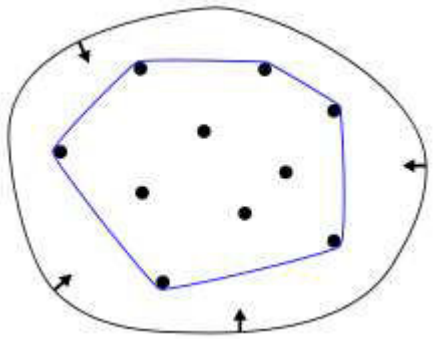
Important Problem Types: Geometric Problems

PES UNIVERSITY ONLINE

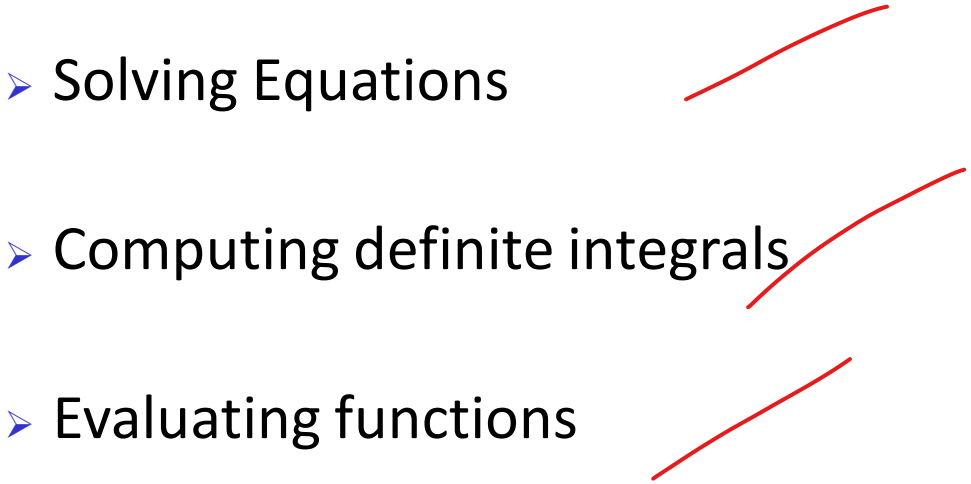
Closest Pair problem



Convex Hull Problem



Important Problem Types: Numerical Problems



- Solving Equations
- Computing definite integrals
- Evaluating functions

THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

Surabhi Narayan

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

DECREASE AND CONQUER

Surabhi Narayan
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Decrease and Conquer

- **Decrease** or reduce problem instance to smaller instance of the same problem and extend solution.
- **Conquer** the problem by solving smaller instance of the problem.
- **Extend** solution of smaller instance to obtain solution to original problem .
- **Exploit** the relationship between a solution to a given instance of a problem and a solution to its smaller instance.

- Can be implemented either top-down or bottom-up
- Also referred to as *inductive* or *incremental* approach

usually recursive implementaion

usually iterative solution

also called incremental approach

Decrease and Conquer

3 Types of Decrease and Conquer

Decrease by a constant (usually by 1):
- insertion sort
- graph traversal algorithms (DFS and BFS)
- topological sorting
- algorithms for generating permutations, subsets

Decrease by a constant factor (usually by half)
- binary search and bisection method
- exponentiation by squaring
- multiplication à la russe

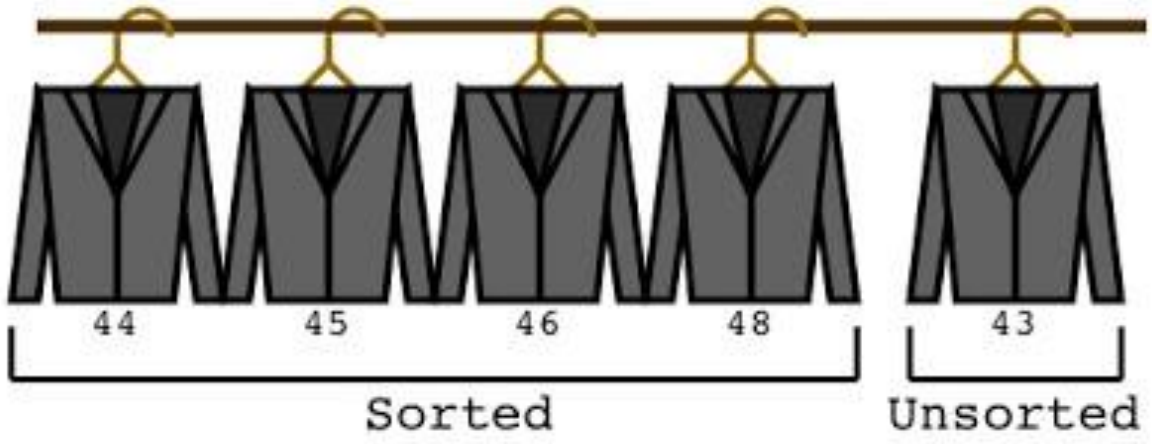
Variable-size decrease
- Euclid's algorithm
- selection by partition
- Nim-like games

This usually results in a recursive algorithm.

Insertion Sort

Imagine a card game

Cards in your hand are sorted.
The dealer hands you exactly one new card.
How would you rearrange your cards



Insertion Sort

- Insertion sort is based on the idea that one element from the input elements is consumed in each iteration to find its correct position i.e, the position to which it belongs in a sorted array.
- grows the sorted array at each iteration
- compares the current element with the largest value in the sorted array.
- If the current element is greater, then it leaves the element in its place and moves on to the next element else it finds its correct position in the sorted array and moves it to that position.
- This is done by shifting all the elements, which are larger than the current element, in the sorted array to one position ahead

Insertion Sort

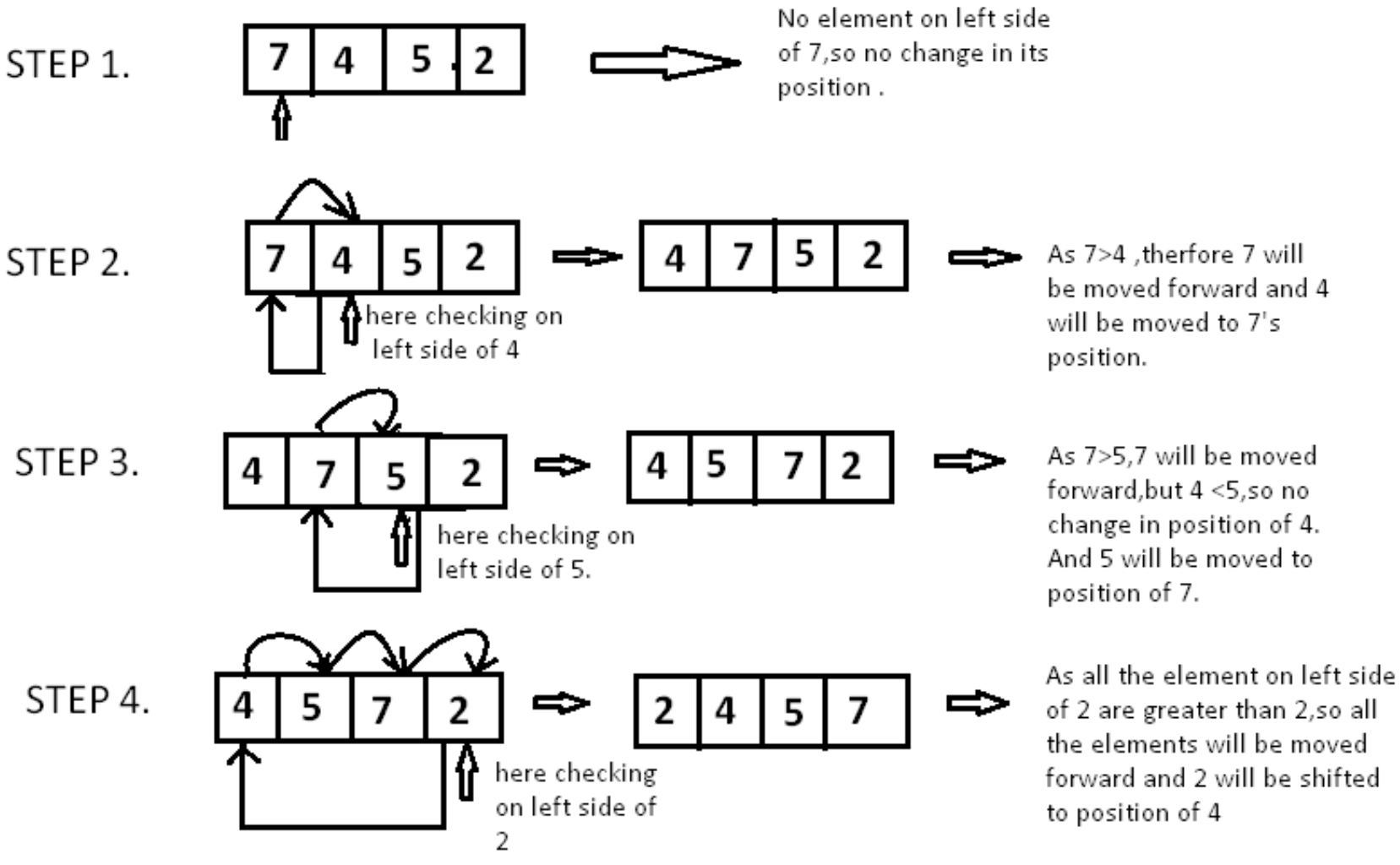
To sort array A[0..*n*-1], sort A[0..*n*-2] recursively and then insert A[*n*-1] in its proper place among the sorted A[0..*n*-2]

Usually implemented bottom up (nonrecursively)

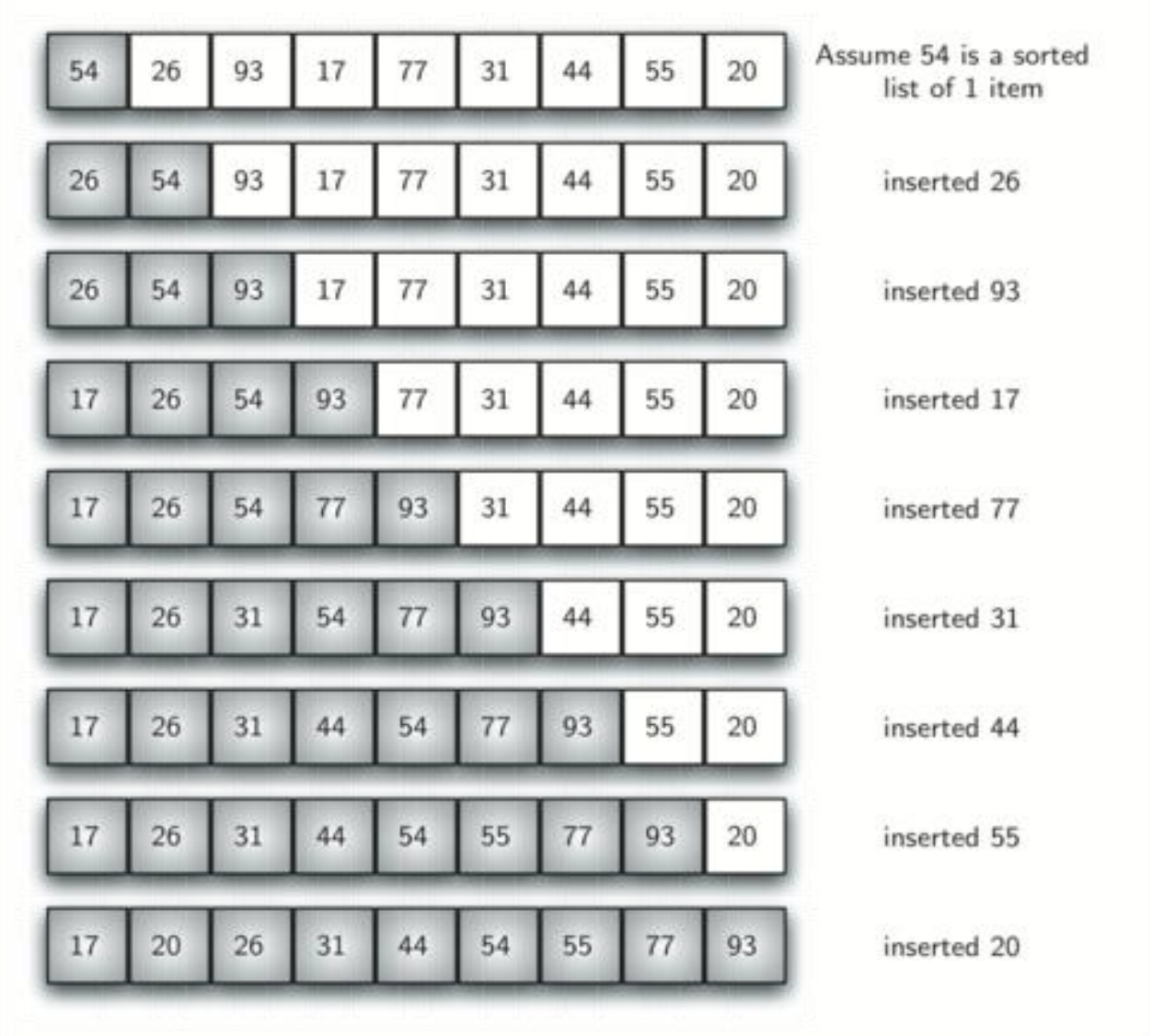
Example: Sort 6, 4, 1, 8, 5

6 | 4 1 8 5
4 6 | 1 8 5
1 4 6 | 8 5
1 4 6 8 | 5
1 4 5 6 8

Insertion Sort



Insertion Sort



ALGORITHM *InsertionSort*$(A[0..n-1])$

```
//Sorts a given array by insertion sort
//Input: An array A[0..n − 1] of n orderable elements
//Output: Array A[0..n − 1] sorted in nondecreasing order
for i ← 1 to n − 1 do
    v ← A[i]
    j ← i − 1
    while j ≥ 0 and A[j] > v do
        A[j + 1] ← A[j]
        j ← j − 1
    A[j + 1] ← v
```

Time efficiency

$C_{worst}(n) = n(n\text{-}1)/2 \in \Theta(n^2)$

$C_{avg}(n) \approx n^2/4 \in \Theta(n^2)$

$C_{best}(n) = n - 1 \in \Theta(n)$ (also fast on almost sorted arrays)

Space efficiency: in-place

Stability: yes

Best elementary sorting algorithm overall

Binary insertion sort

THANK YOU

Surabhi Narayan
Department of Computer Science &Engineering

surabhinarayan@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

Surabhi Narayan

Department of Computer Science & Engineering

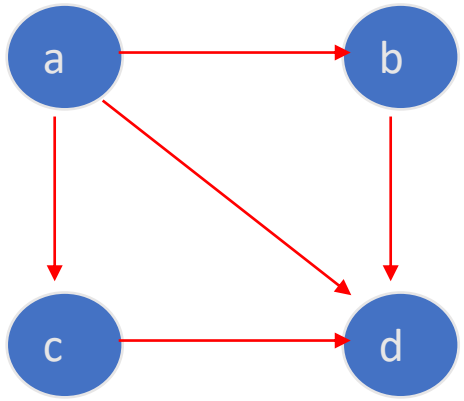
DESIGN AND ANALYSIS OF ALGORITHMS

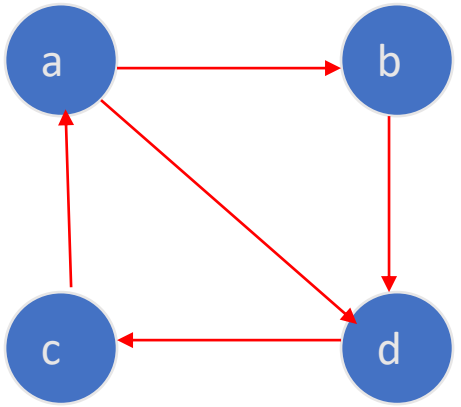
DECREASE AND CONQUER

Surabhi Narayan
Department of Computer Science & Engineering

DAGs and Topological Sorting

DAG: a directed acyclic graph, i.e. a directed graph with no (directed) cycles





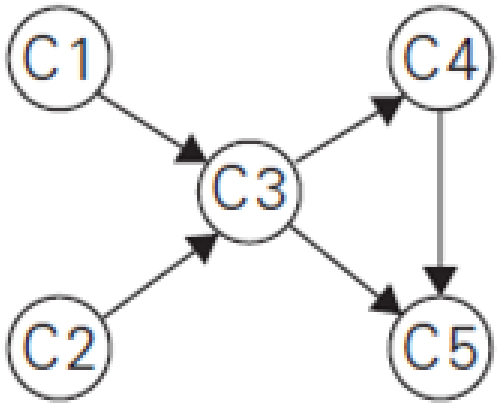
Arise in modeling many problems that involve prerequisite constraints (construction projects, document version control)

Vertices of a dag can be linearly ordered so that for every edge its starting vertex is listed before its ending vertex (*topological sorting*). Being a dag is also a necessary condition for topological sorting to be possible.

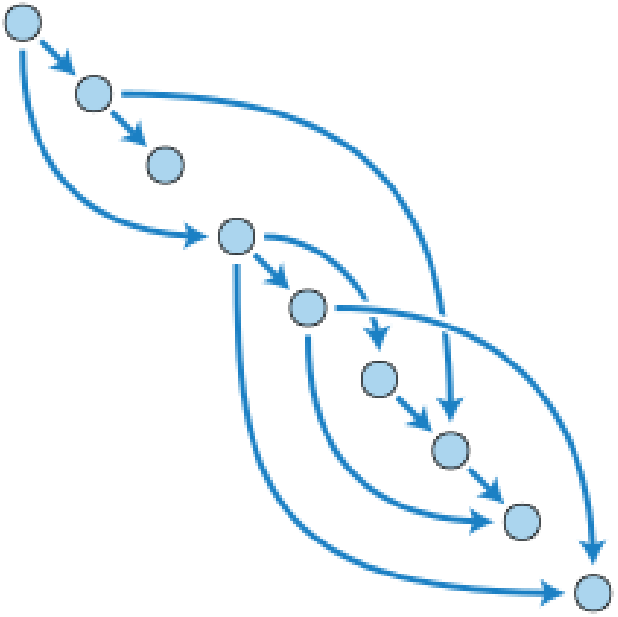
Topological Sorting

Topological Sorting: is listing vertices of a directed graph in such an order that for every edge in the graph, the vertex where the edge starts is listed before the vertex where the edge ends.

A digraph has a topological sorting iff it is a **dag**.



condition for topological sorting is
the graph has to be directed acyclic meaning
shouldn't have cycles



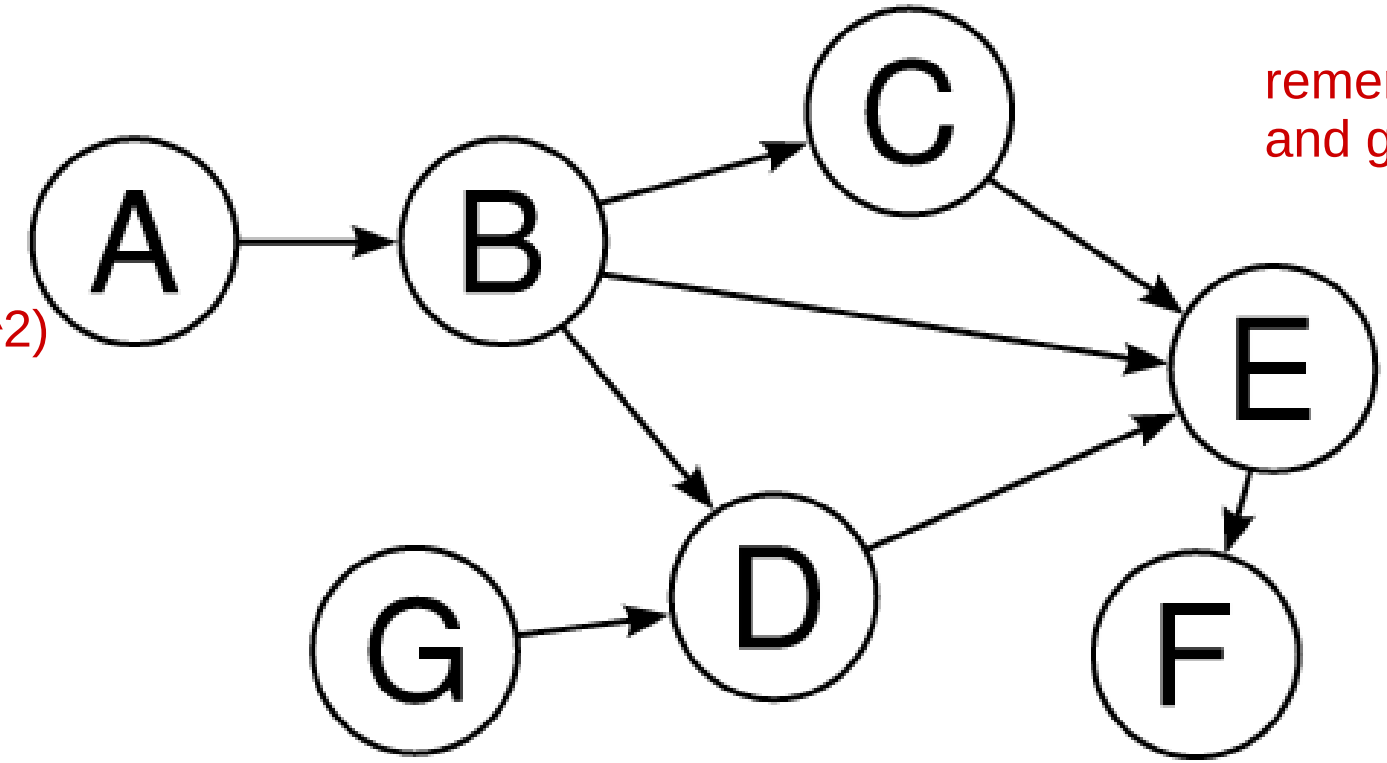
Finding a **Topological Sorting** of the vertices of a dag:

- **DFS-based** algorithm
- **Source-removal** algorithm

Read DFS first from page 149 - 151
read textbook marked mcqs and all there
please see them and go

remember the algorithm
and go

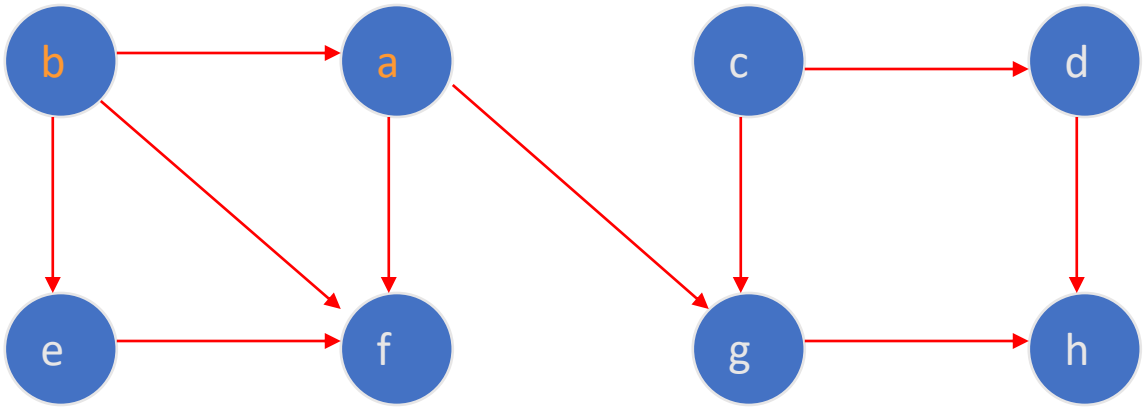
traversal time for
DFS algo
Adjacency matrix= Θ(V^2)
Adjacency list= Θ(V+E)



DFS-based algorithm for topological sorting

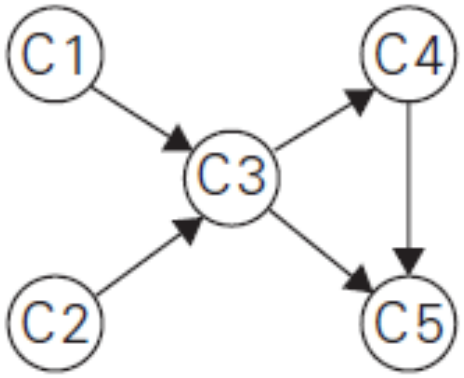
- Perform DFS traversal, noting the order vertices are popped off the traversal stack
- Reverse order solves topological sorting problem
- Back edges encountered?→ NOT a dag!

Example:



Efficiency: The same as that of DFS.

DFS-based algorithm for finding **Topological Sorting**



(a)

$C5_1$
$C4_2$
$C3_3$
$C1_4$ $C2_5$

(b)

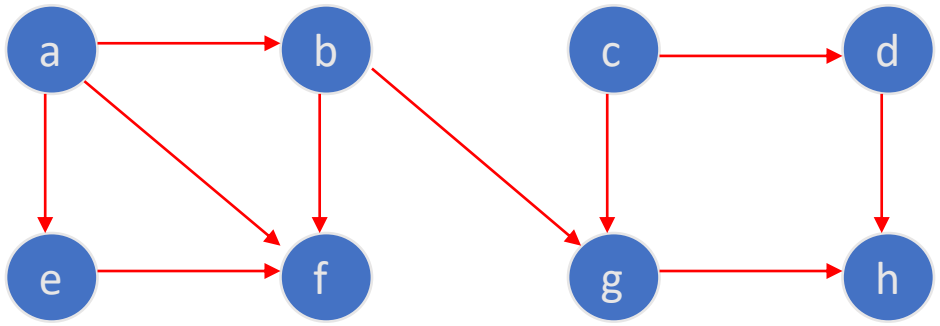
The popping-off order:
C5, C4, C3, C1, C2
The topologically sorted list:

(c)

Source removal algorithm

Repeatedly identify and remove a *source* (a vertex with no incoming edges) and all the edges incident to it until either no vertex is left or there is no source among the remaining vertices (not a dag)

Example:



"Invert" the adjacency lists for each vertex to count the number of incoming edges by going thru each adjacency list and counting the number of times that each vertex appears in these lists. To remove a source, decrement the count of each of its neighbors by one.

Decrease and Conquer

Algorithm SourceRemoval_Toposort(V, E)

L ← Empty list that will contain the sorted vertices

S ← Set of all vertices with no incoming edges

while S is non-empty **do**

    remove a vertex v from S

    add v to *tail* of L

    for each vertex m with an edge *e* from v to m **do**

        remove edge e from the graph

        if m has no other incoming edges **then**

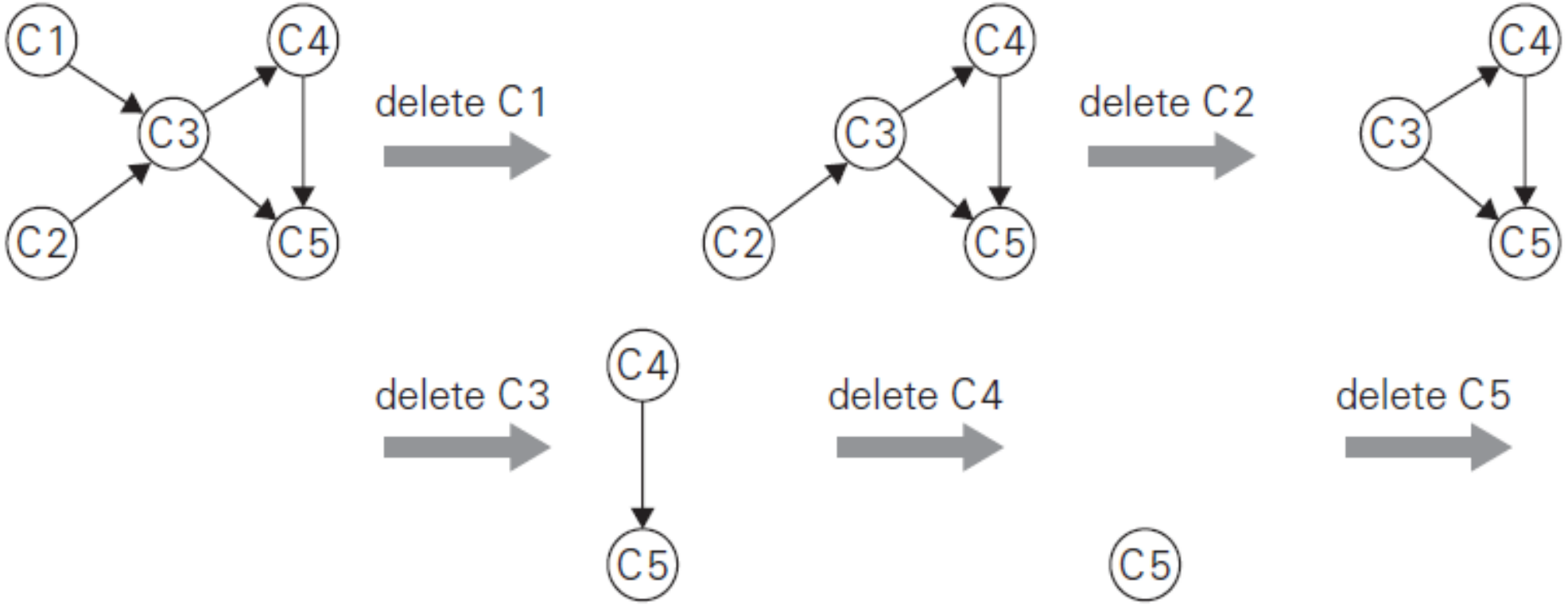
            insert m into S

if graph has edges **then**

    return error (not a DAG)

else return L (a topologically sorted order)

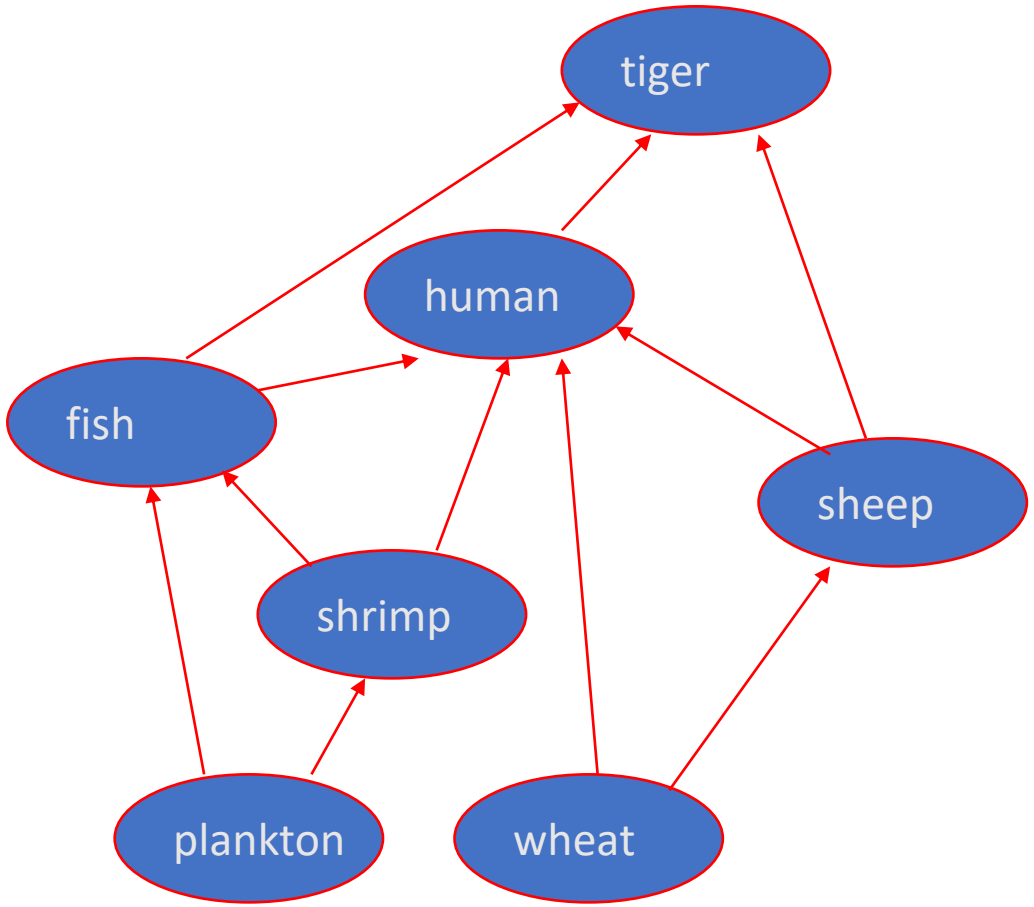
Decrease and Conquer



The solution obtained is C1, C2, C3, C4, C5

PES UNIVERSITY ONLINE

Order the following items in a food chain



THANK YOU

Surabhi Narayan
Department of Computer Science &Engineering

surabhinarayan@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

Surabhi Narayan

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

DECREASE AND CONQUER

Surabhi Narayan
Department of Computer Science & Engineering

Combinatorial Objects

- Permutations
- Combinations
- Subsets of a given set

Generating Permutations

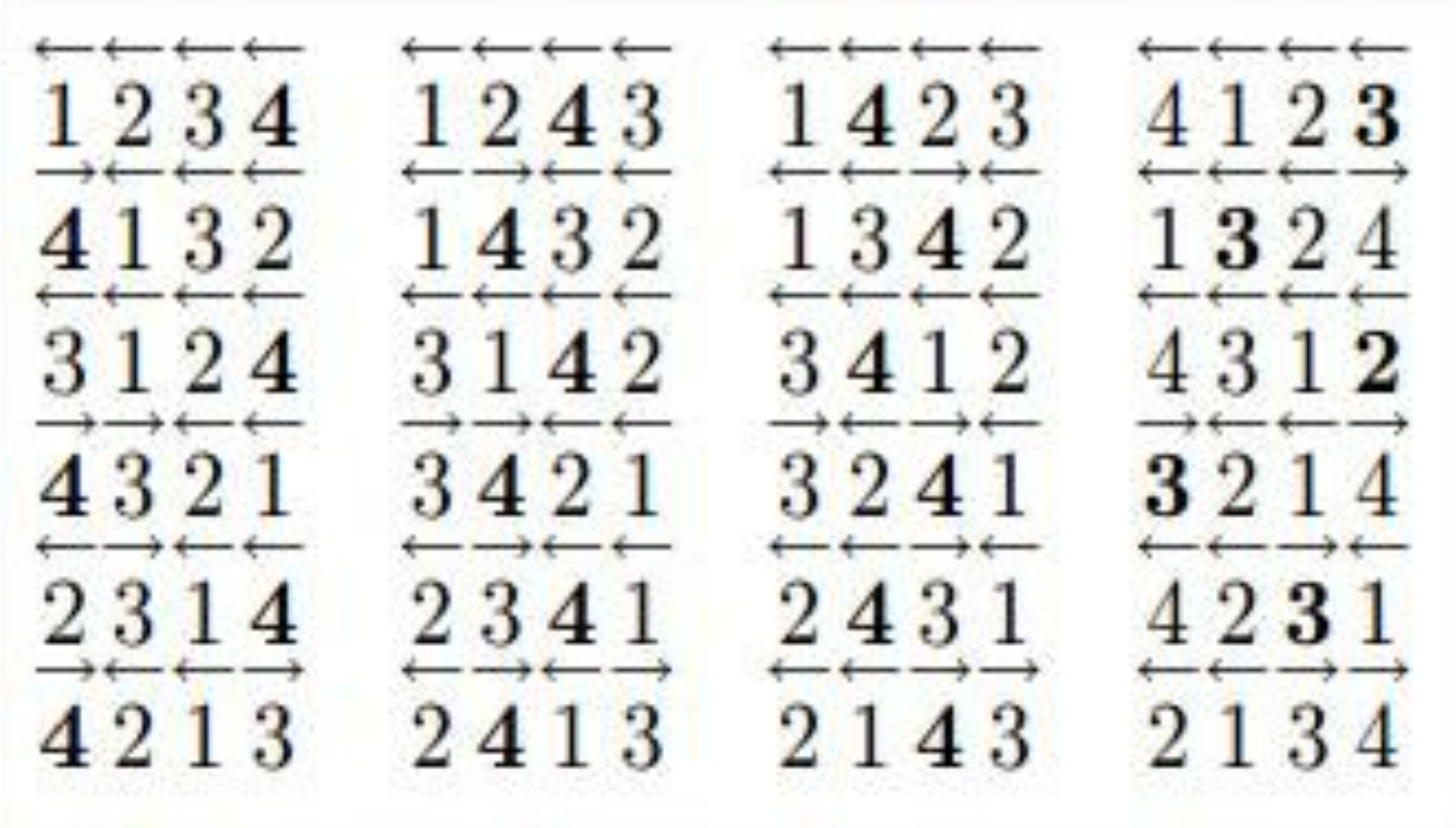
- Underlying set elements are to be permuted
- Decrease and conquer approach
- Satisfies the minimal change requirement
- Example: Johnson- Trotter algorithm

Generating Permutations

see notebook

ALGORITHM *JohnsonTrotter*(n)

//Implements Johnson-Trotter algorithm for generating permutations
//Input: A positive integer n
//Output: A list of all permutations of $\{1, \ldots, n\}$
initialize the first permutation with $\overleftarrow{1}\ \overleftarrow{2} \ldots \overleftarrow{n}$
while the last permutation has a mobile element **do**
    find its largest mobile element k
    swap k with the adjacent element k's arrow points to
    reverse the direction of all the elements that are larger than k
    add the new permutation to the list


1 2 3 4
4 1 3 2
3 1 2 4
4 3 2 1
2 3 1 4
4 2 1 3
1 2 4 3
1 4 3 2
3 1 4 2
3 4 2 1
2 3 4 1
2 4 1 3
1 4 2 3
1 3 4 2
3 4 1 2
3 2 4 1
2 4 3 1
2 1 4 3
4 1 2 3
1 3 2 4
4 3 1 2
3 2 1 4
4 2 3 1
2 1 3 4

Permutations in Lexicographic order

ALGORITHM *LexicographicPermute*(n)

//Generates permutations in lexicographic order
//Input: A positive integer n
//Output: A list of all permutations of $\{1, \ldots, n\}$ in lexicographic order
initialize the first permutation with $12 \ldots n$
while last permutation has two consecutive elements in increasing order **do**
    let i be its largest index such that $a_i < a_{i+1}$ $\quad //a_{i+1} > a_{i+2} > \cdots > a_n$
    find the largest index j such that $a_i < a_j$ $\quad //j \geq i+1$ since $a_i < a_{i+1}$
    swap a_i with a_j $\quad //a_{i+1}a_{i+2} \ldots a_n$ will remain in decreasing order
    reverse the order of the elements from a_{i+1} to a_n inclusive
    add the new permutation to the list

see notebook

Generating Subsets:

Knapsack problem needed to find the most valuable subset of items that fits a knapsack of a given capacity.

Powerset: set of all subsets of a set. Set A={1, 2, ..., n} has $\mathbf{2^n}$ subsets.

Generate all subsets of the set A={1, 2, ..., n}.

Any **decrease-by-one** idea?
# of subsets of { } = 2^0 = 1, which is **{ }** itself
Suppose, we know how to generate all subsets of {1,2,...,n-1}
Now, how can we generate all subsets of {1,2,...,n} ?

Generating Subsets:

All subsets of {1,2,...,n-1}: 2^{n-1} such subsets

All subsets of {1,2,...,n}:
2^{n-1} subsets of {1,2,...,n-1} and
another 2^{n-1} subsets of {1,2,...,n-1} having **'n'** with them.

That adds up to all 2^n subsets of {1,2,...,n}

0	$\varnothing$							
1	$\varnothing$	$\{a_1\}$						
2	$\varnothing$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$				
3	$\varnothing$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$	$\{a_3\}$	$\{a_1, a_3\}$	$\{a_2, a_3\}$	$\{a_1, a_2, a_3\}$

Alternate way of Generating Subsets:

Knowing the binary nature of either having **n**th element or not, any idea involving binary numbers itself?

One-to-one correspondence between all 2^n bit strings $b_1b_2...b_n$ and 2^n subsets of $\{a_1, a_2, \ldots, a_n\}$.

Each bit string $b_1b_2...b_n$ could correspond to a subset.
In a bit string $b_1b_2...b_n$, depending on whether b_i is 1 or 0, a_i is in the subset or not in the subset.

000	001	010	011	100	101	110	111
$\varnothing$	$\{a_3\}$	$\{a_2\}$	$\{a_2, a_3\}$	$\{a_1\}$	$\{a_1, a_3\}$	$\{a_1, a_2\}$	$\{a_1, a_2, a_3\}$

Generating Subsets in Squashed order:

Squashed order: any subset involving a_j can be listed only after all the subsets involving $a_1, a_2, \ldots, a_{j-1}$

Both of the previous methods does generate subsets in squashed order.

000	001	010	011	100	101	110	111
$\varnothing$	$\{a_3\}$	$\{a_2\}$	$\{a_2, a_3\}$	$\{a_1\}$	$\{a_1, a_3\}$	$\{a_1, a_2\}$	$\{a_1, a_2, a_3\}$

see from textbook + your notebok+ teja's notebook before exam

whole generating subsets wala part properly

Generating Subsets in Squashed order:

Squashed order: any subset involving a_j can be listed only after all the subsets involving $a_1, a_2, \ldots, a_{j-1}$

Can we do it with minimal change in bit-string (actually, just one-bit change to get the next bit string)? This would mean, to get a new subset, just change one item (remove one item or add one item).

Binary reflected gray code:

000 001 011 010 110 111 101 100

THANK YOU

Surabhi Narayan
Department of Computer Science &Engineering

surabhinarayan@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

Surabhi Narayan

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

DECREASE AND CONQUER

Surabhi Narayan
Department of Computer Science & Engineering

PES UNIVERSITY

Decrease-by-a-Constant-Factor Algorithms:

```
Algorithm BinarySearchRec(A[0..n-1], K)
   if(n <= 0)
      return -1
   m = ⌊n/2⌋
   if(k = A[m])
      return m
   if(k < A[m])
      return BinarySearchRec(A[0..m-1], K)
   else
      return BinarySearchRec(A[m+1..n-1], K)
```

Worst Case Time complexity: $O(Log_2 n)$

understand the problem properly by reading here and see the textbook page number 178-179also see the solution of problem 10 which deals with divide by 3 algortihm

Fake-Coin Problem

Among n identical-looking coins, one is fake.With a balance scale, we can compare any two sets of coins.

The problem is to design an efficient algorithm for detecting the fake coin.

The most natural idea for solving this problem is to divide n coins into two piles of $n/2$ coins each, leaving one extra coin aside if n is odd, and put the two piles on the scale.

If the piles weigh the same, the coin put aside must be fake; otherwise, we can proceed in the same manner with the lighter pile, which must be the one with the fake coin.

$W(n) = W(n/2) + 1$ for $n > 1$, $W(1) = 0$.

$W(n) = \log_2 n$.

It would be more efficient to divide the coins not into two but into *three* piles of about $n/3$ coins each.

$\log_3 n$

Russian Peasant Multiplication

Let n and m be positive integers whose product we want to compute, and let us measure the instance size by the value of n.
if n is even, an instance of half the size has to deal with $n/2$, and we have an obvious formula relating the solution to the problem's larger instance to the solution to the smaller one:
$n \cdot m = (n/2) * 2m$

If n is odd, we need only a slight adjustment of this formula:
$n \cdot m = ((n-1)/2) \cdot 2m + m$.
Using these formulas and the trivial case of $1 \cdot m = m$ to stop, we can compute product $n \cdot m$ either recursively or iteratively.

n	m	
50	65	
25	130	
12	260	(+130)
6	520	
3	1040	
1	2080	(+1040)
	2080	+(130 + 1040) = 3250

(a)

n	m	
50	65	
25	130	130
12	260	
6	520	
3	1040	1040
1	2080	2080
		3250

(b)

FIGURE 4.11 Computing $50 \cdot 65$ by the Russian peasant method.

Josephus problem

Named for Flavius Josephus, a famous Jewish historian who participated in and chronicled the Jewish revolt of 66–70 c.e. against the Romans.
Josephus, as a general, managed to hold the fortress of Jotapata for 47 days, but after the fall of the city he took refuge with 40 diehards in a nearby cave. There, the rebels voted to perish rather than surrender. Josephus proposed that each man in turn should dispatch his neighbor, the order to be determined by casting lots. Josephus contrived to draw the last lot, and, as one of the two surviving men in the cave, he prevailed upon his intended victim to surrender to the Romans.

Josephus Problem

for n = even

J(2k)=2J(k)-1

for n = odd

J(2k+1)=2J(k)+1

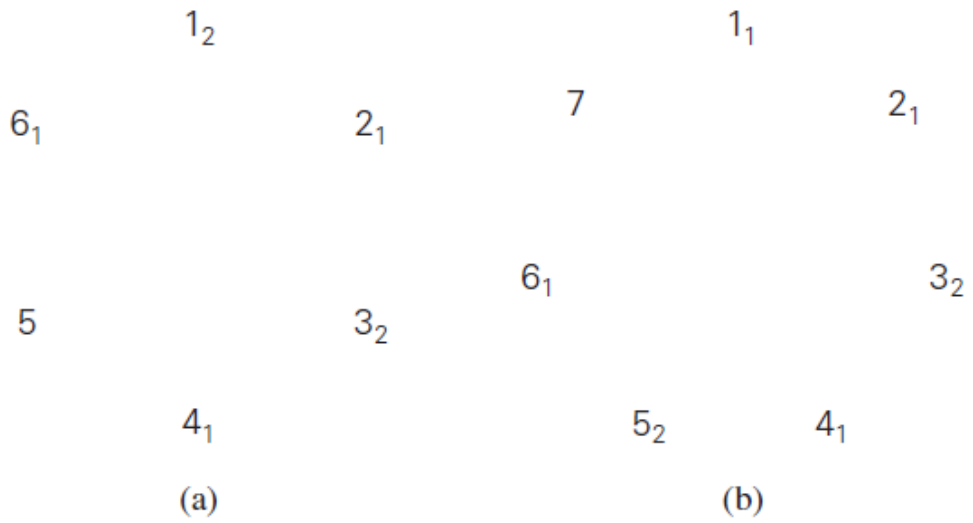


FIGURE 4.12 Instances of the Josephus problem for (a) $n = 6$ and (b) $n = 7$. Subscript numbers indicate the pass on which the person in that position is eliminated. The solutions are $J(6) = 5$ and $J(7) = 7$, respectively.

THANK YOU

Surabhi Narayan
Department of Computer Science &Engineering

surabhinarayan@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

Shylaja S S
Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Binary Search

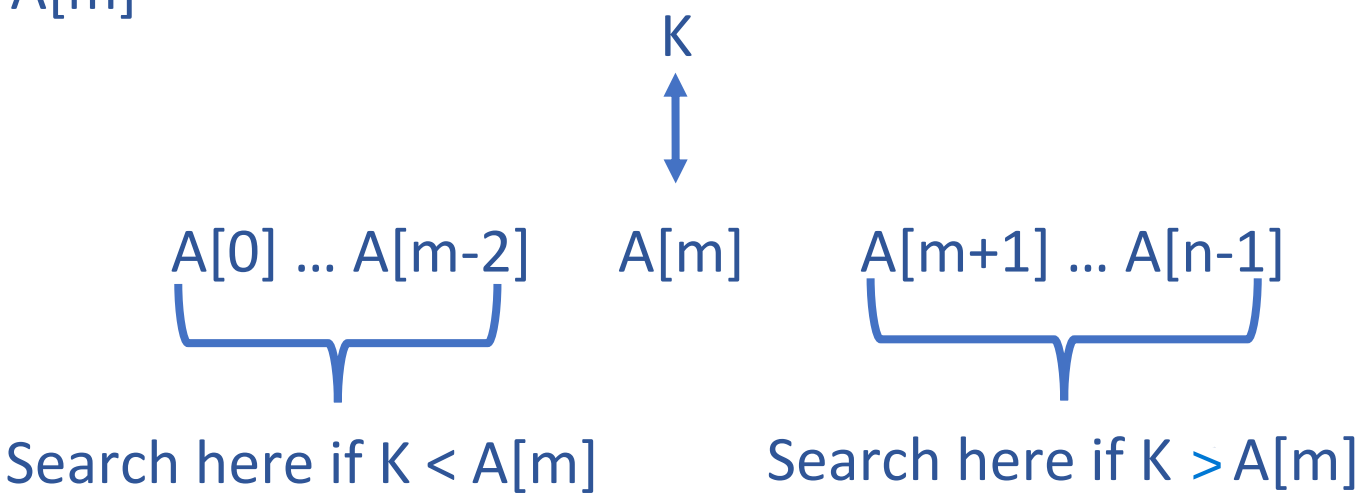
Major Slides Content: Anany Levitin

Shylaja S S

Department of Computer Science & Engineering

Binary Search - Idea

- Binary Search is a remarkably efficient algorithm for searching in a sorted array
- It works by comparing the search key K with the array's middle element A[m]
- If they match, the algorithm stops
- Otherwise, the same operation is repeated recursively for the first half of the array if K < A[m] and for the second half if K > A[m]



Binary Search - Algorithm

```
ALGORITHM  BinarySearch(A[0 .. n  -1], K)
// Implements non recursive binary search
// Input: An array A[0 .. n - 1] sorted in  ascending order  and a
   // search key K
// Output: An index of the array's element that is  equal to K or
   // -1 if there is no such element
l □ 0; r □ n-1
while l ≤ r do
   m ← ⌊(l + r)/2⌋
   if K = A[m] return m
   else if K < A[m]  r □ m-1
   else l□m+1
return -1
```

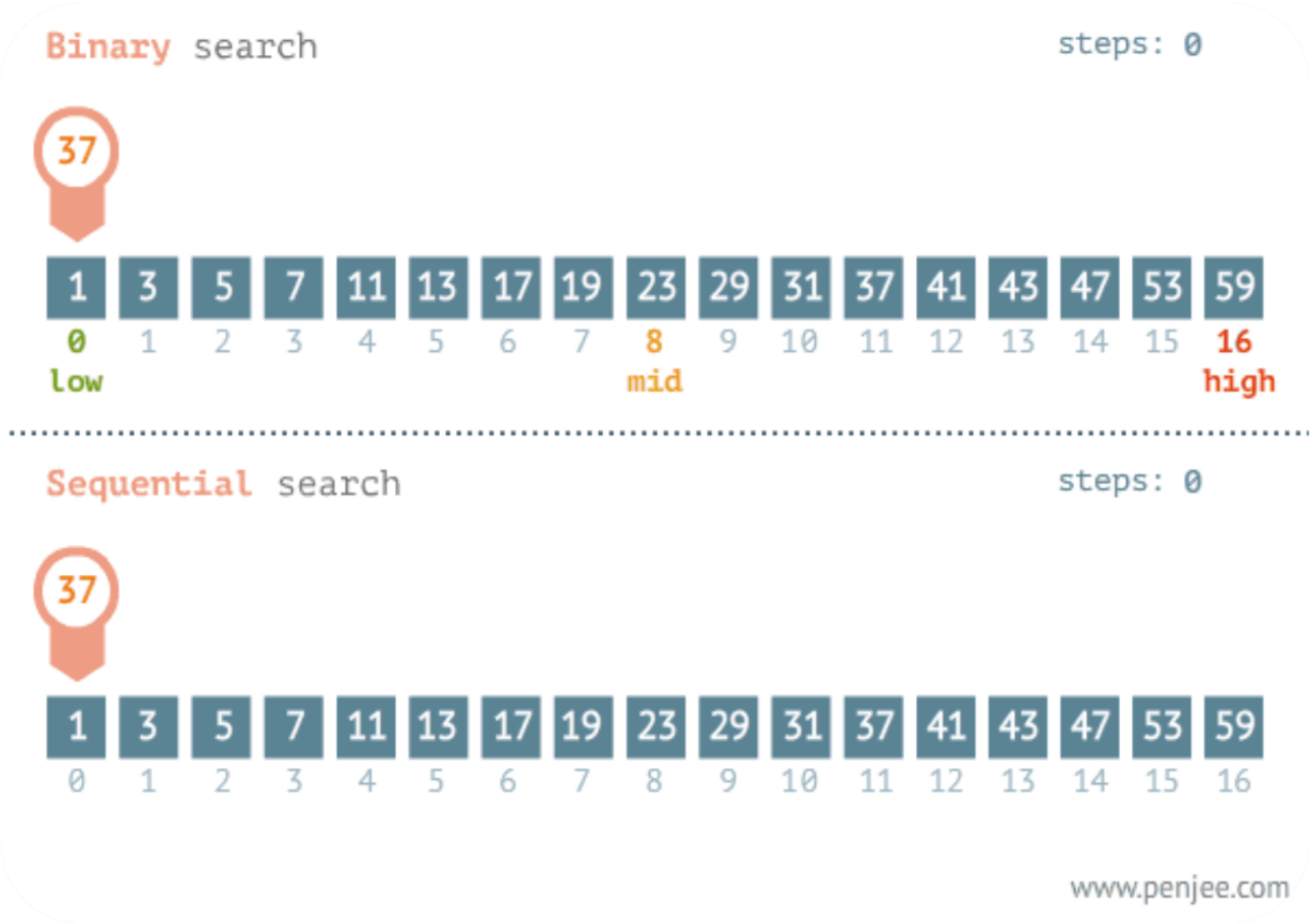
Binary Search - Example

Search Key K = 70

index	0	1	2	3	4	5	6	7	8	9	10	11	12
value	3	14	27	31	39	42	55	70	74	81	85	93	98
iteration 1	*l*						*m*						*r*
iteration 2								*l*		*m*			*r*
iteration 3								*l,m*	*r*				

DESIGN AND ANALYSIS OF ALGORITHMS

Binary Search Vs Linear Search



Binary Search - Analysis: Worst Case

The basic operation is the comparison of the search key with an element of the array

The number of comparisons made are given by the following recurrence:

$$C_{worst}(n) = C_{worst}\left(\left\lfloor n/2 \right\rfloor\right) + 1 \; for \; n > 1, C_{worst}(1) = 1$$

For the initial condition $C_{worst}(1) = 1$, we obtain:

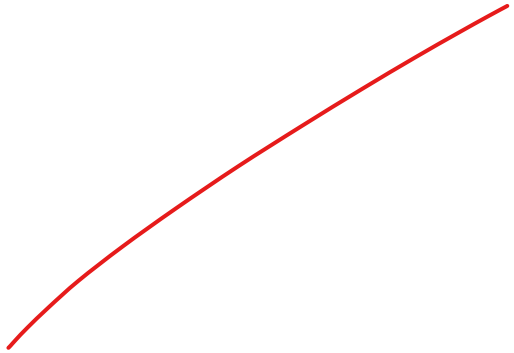
$$C_{worst}(2^k) = k + 1 = \log_2 n + 1$$

For any arbitrary positive integer, n:

$$C_{worst}(n) = \lfloor \log_2 n \rfloor + 1$$

Binary Search - Analysis: Average Case

$$C_{avg} \approx \log_2 n$$



THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

DESIGN AND ANALYSIS OF ALGORITHMS

Decrease by Variable size Algorithm

The third principal variety of decrease-and-conquer, the **size reduction pattern** varies from one iteration of the algorithm to another.

This algorithm helps optimize:

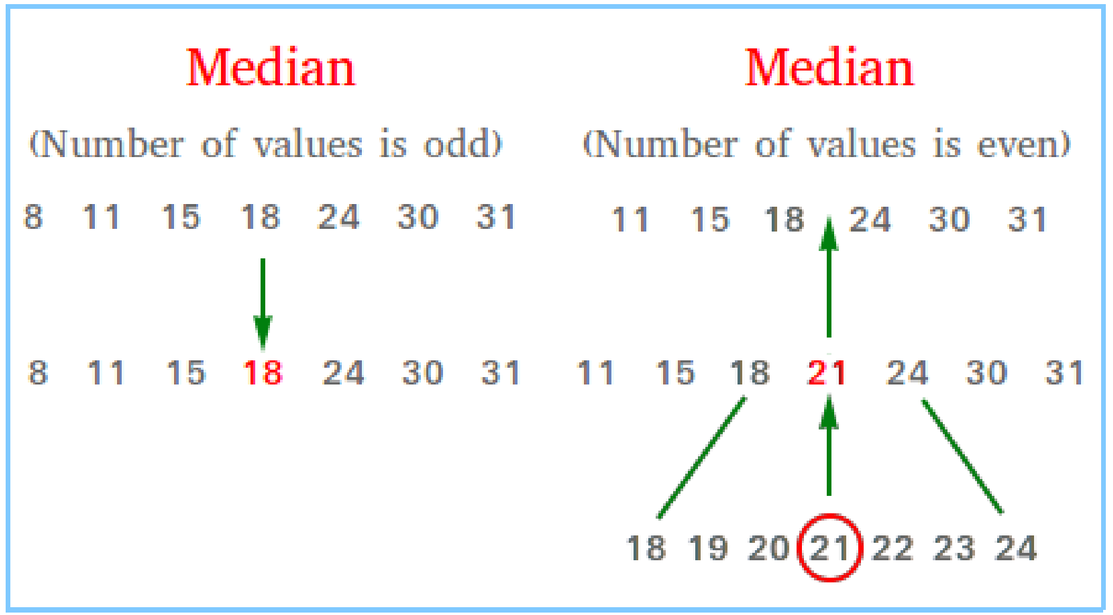
1. Computing a Median and the Selection Problem
2. Interpolation Search
3. Searching and Insertion in a Binary Search Tree
4. The Game of Nim

We will look at **Computing a Median and the Selection Problem** in detail.

Computing a Median and the Selection Problem

Objective: Find the median of an odd array in the best time.

What is the Median?
The median is the middle value of a sorted array.



Naïve Approach:

1. Sort the array.
2. Find the element in the middle position.

Time complexity:

- Sorting: O(n log n)
- Selecting the middle element: O(1)
- Total: O(n log n)

PES UNIVERSITY

Computing a Median and the Selection Problem

Issues with the Naïve Approach:

- Sorting places all elements in their correct positions.
- We only need the middle element, so sorting the entire array is unnecessary.

What can be optimized:

- Instead of sorting all elements, directly try to find the element at the n/2 position.
- This problem of finding the k-th smallest or largest element is called the **Selection Problem.**

Proposed Solution:

- QuickSelect to solve the Selection Problem

Computing a Median and the Selection Problem

Quick Select:

- Similar to Quick Sort, we pick a **pivot** element and **partition** the array:
  - Elements **smaller** than the pivot go to the **left**.
  - Elements **larger** than the pivot go to the **right**.
  - The **pivot** is placed in its **correct** position.
  - This partition technique is called **Lomuto Partition**.

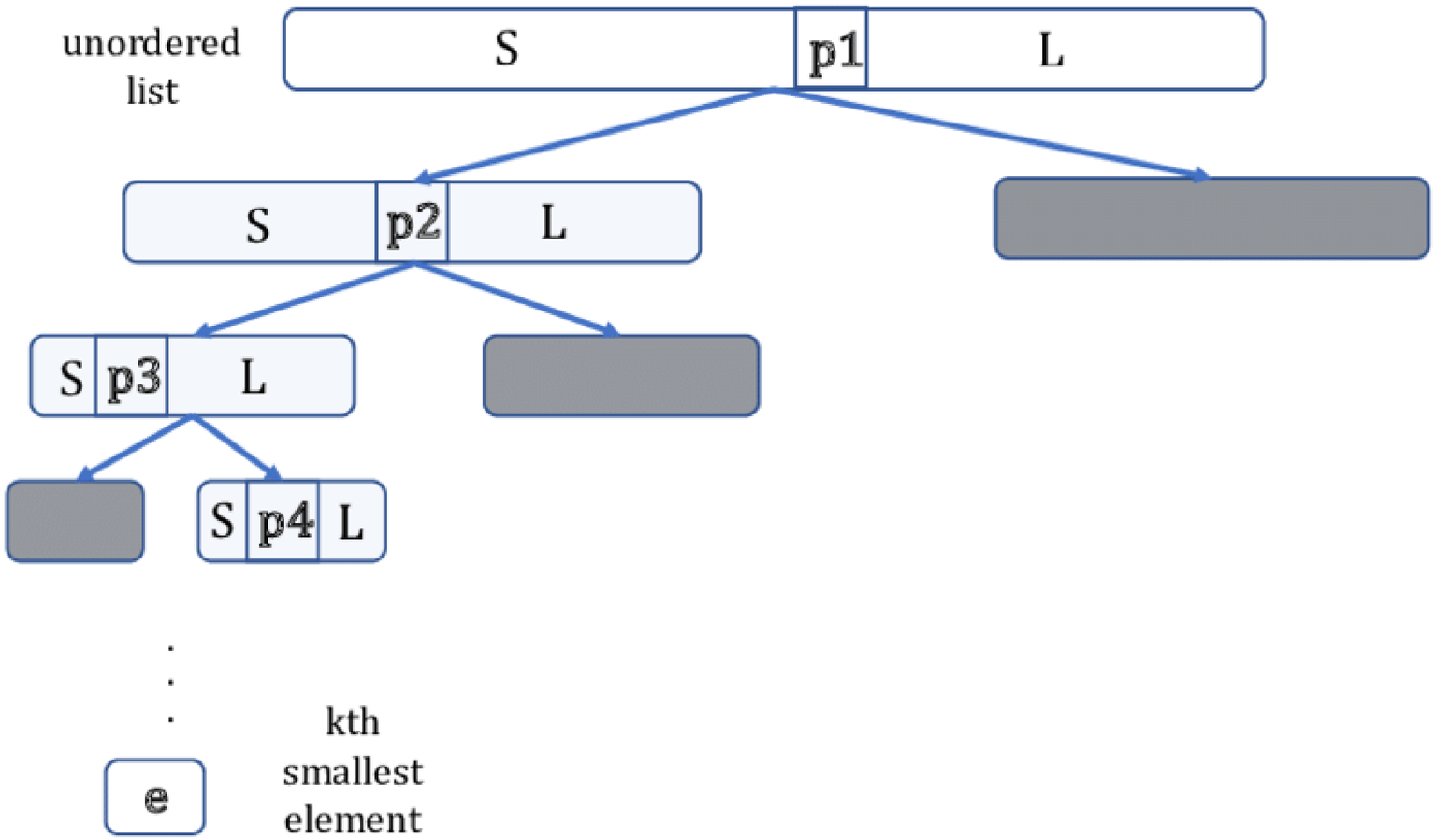
- Difference from Quick Sort:
  - Quick Sort recursively sorts the entire array.
  - Quick Select **stops** once the pivot is at the k-th position (n/2 for median).

Algorithm for QuickSelect

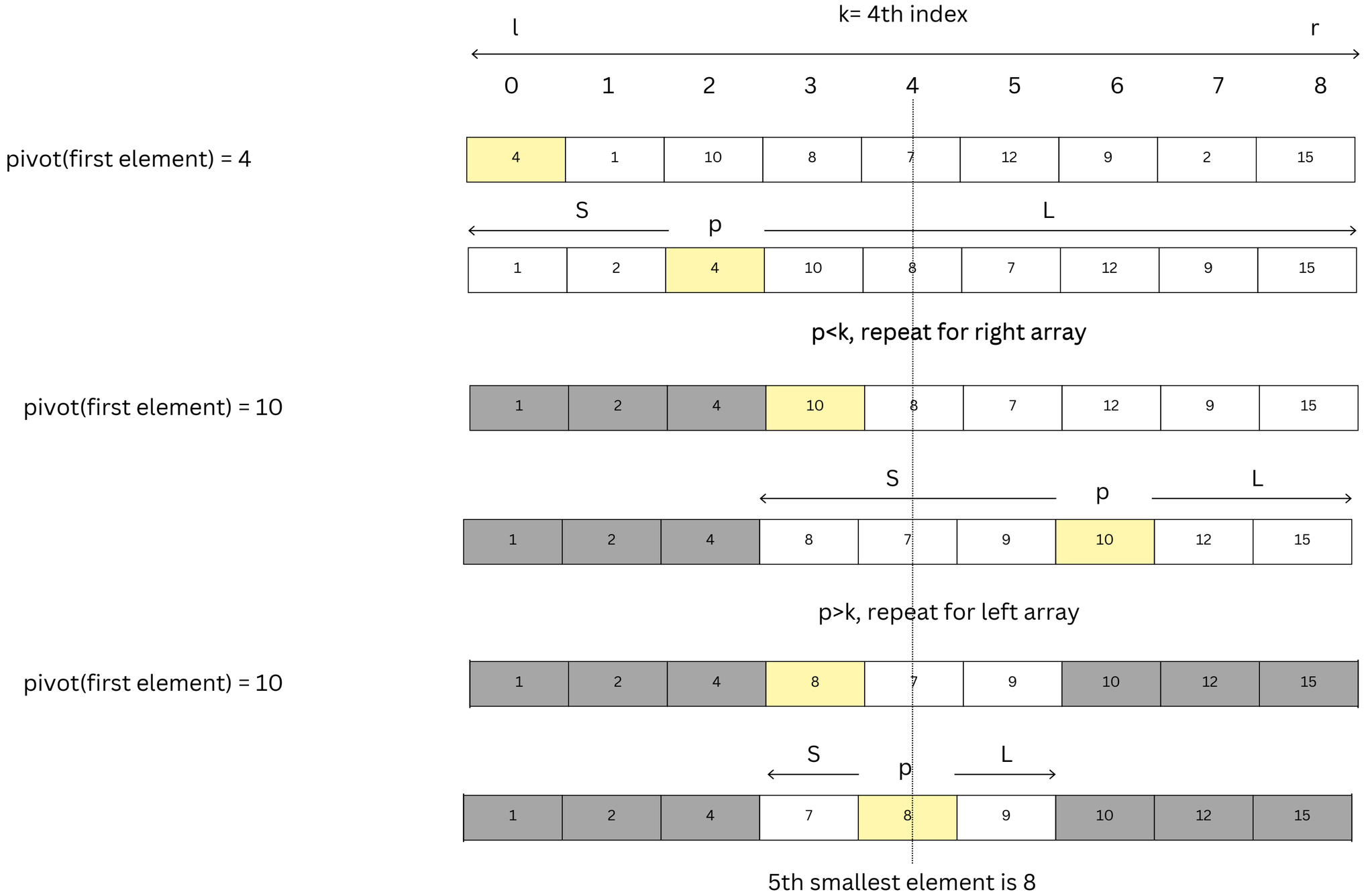
Input : List, left is first position of list, right is last position of list and k is k-th smallest element.
Output : A new list is partitioned.

```
quickSelect(list, left, right, k)
   if left = right
      return list[left]

   // Select a pivotIndex between left and right
   pivotIndex <= partition(list, left, right, pivotIndex)
   if k = pivotIndex
      return list[k]
   else if k < pivotIndex
      right <= pivotIndex - 1
   else
      left <= pivotIndex + 1
```

QuickSelect



Find the 5th smallest element in the given array



Time Complexities for QuickSelect

Best Case:

- The best case occurs when the pivot chosen is always the median of the array. This perfectly divides the array in half at each step.
- The recurrence relation: $T(n)=T(n/2)+O(n)$
- Solving this gives $O(n)$ time complexity, and the problem size is reduced exponentially.

Worst Case:

- The worst case occurs when the pivot selection is poor, e.g., always choosing the smallest or largest element. This results in highly unbalanced partitions, where one side has n - 1 elements and the other has 0.
- The recurrence becomes: $T(n)=T(n-1)+O(n)$
- Solving this gives $O(n^2)$ time complexity.

Improve the Worst Case Complexity

There are two key ways to achieve better worst-case complexity:

1. Choosing a Better Pivot Selection Strategy such as Median of Medians Algorithm.
2. Using an Improved Partitioning Algorithm such as using Hoare's Partition Scheme instead of Lomuto's Partition Scheme can improve efficiency by reducing swaps.

Median of Medians Algorithm (Guaranteed O(n) Worst-Case):

We need to **choose a pivot close to the median** consistently. This ensures better partitioning and prevents highly unbalanced cases.

- Divide the array into groups of 5 elements (or another small fixed size).
- Find the median of each group.
- Recursively find the median of these medians to use as the pivot.
- Ensures a pivot that is reasonably close to the true median, avoiding worst-case $O(n^2)$.
- Guarantees O(n) worst-case time complexity.

Variable Decrease and Conquer

By utilizing **variable-size Decrease and Conquer**, we optimized the Selection Algorithm, making median computation more efficient.

Unlike traditional divide-and-conquer, where the problem size is reduced by a fixed factor (e.g., halving in binary search), **variable-size** reduction allows for **adaptive problem** shrinking at each step. In Quick Select, this means:

- Instead of processing the entire array, we eliminate a portion based on the pivot's position.
- The size reduction varies **dynamically**, depending on where the pivot is placed.
- This flexibility enables an **O(n)** average-time complexity, significantly improving over naive sorting methods.

By using variable Decrease and Conquer, we were able to directly target the element we needed, making the solution far more efficient.

THANK YOU

DESIGN AND ANALYSIS OF ALGORITHMS

Shylaja S S
Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Divide and Conquer

Major Slides Content: Anany Levitin

Shylaja S S

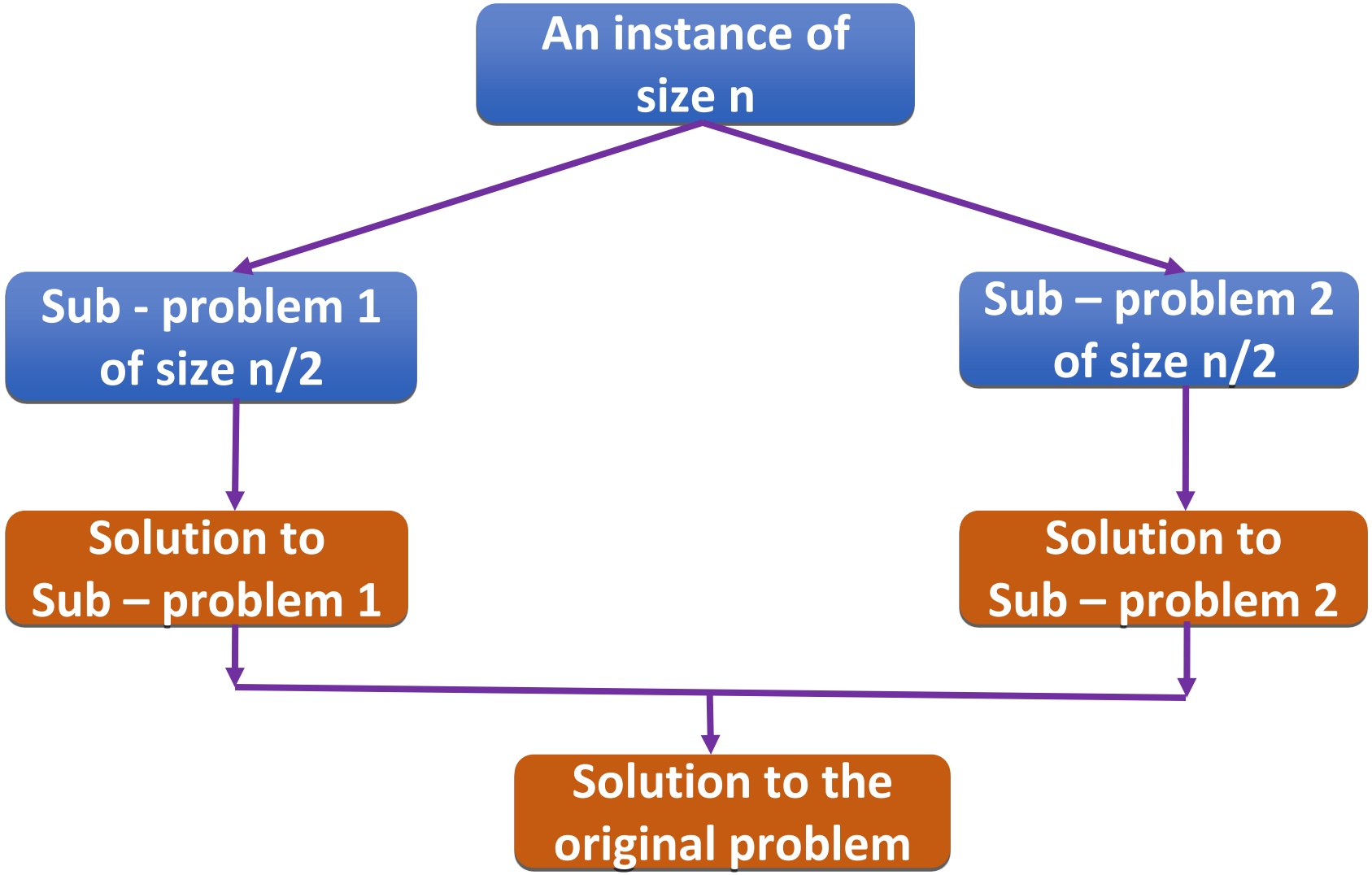
Department of Computer Science & Engineering

Divide and Conquer – Idea

- Divide and Conquer is one of the most well – known algorithm design strategies
- The principle underlying Divide and Conquer strategy can be stated as follows:
  - Divide the given instance of the problem into two or more smaller instances
  - Solve the smaller instances recursively
  - Combine the solutions of the smaller instances and obtain the solution for the original instance

Divide and Conquer – Idea

- Divide and Conquer



General Divide and Conquer

Recurrence

- In the most typical cases of Divide and Conquer, a problem's instance of size n can be divided into b instances of size n/b, with a of them needing to be solved
- Here a and b are constants; a >= 1 and b >= 1
- Assuming that size n is a power of b, we get the following recurrence for the running time:

$$T(n) = a * T(n/b) + f(n)$$

- f(n) is a function that accounts for the time spent on dividing the problem and combining the solutions

Master Theorem

Recurrence

- For the recurrence:

$$\mathbf{T(n) = a * T(n/b) + f(n)}$$

- If $f(n) \in \Theta(n^d)$, where $d >= 0$ in the recurrence relation, then:
  - If $a < b^d$, $T(n) \in \Theta(n^d)$
  - If $a = b^d$, $T(n) \in \Theta(n^d \log n)$
  - If $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$

- Analogous results hold for O and Ω as well!

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

1.3 Master theorem

The master theorem is a formula for solving recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a \geq 1$ and $b > 1$ and $f(n)$ is asymptotically positive. (Asymptotically positive means that the function is positive for all sufficiently large n.)

This recurrence describes an algorithm that divides a problem of size n into a subproblems, each of size n/b, and solves them recursively. (Note that n/b might not be an integer, but in section 4.6 of the book, they prove that replacing $T(n/b)$ with $T(\lfloor n/b \rfloor)$ or $T(\lceil n/b \rceil)$ does not affect the asymptotic behavior of the recurrence. So we will just ignore floors and ceilings here.)

The theorem is as follows:

> ***Theorem 4.1 (Master theorem)***
> Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence
>
> $$T(n) = aT(n/b) + f(n)\,,$$
>
> where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ has the following asymptotic bounds:
>
> 1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
> 2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
> 3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$. ∎

The master theorem compares the function $n^{\log_b a}$ to the function $f(n)$. Intuitively, if $n^{\log_b a}$ is larger (by a polynomial factor), then the solution is $T(n) = \Theta(n^{\log_b a})$. If $f(n)$ is larger (by a polynomial factor), then the solution is $T(n) = \Theta(f(n))$. If they are the same size, then we multiply by a logarithmic factor.

Be warned that these cases are not exhaustive – for example, it is possible for $f(n)$ to be asymptotically larger than $n^{\log_b a}$, but not larger by a polynomial factor (no matter how small the exponent in the polynomial is). For example, this is true when $f(n) = n^{\log_b a} \log n$. In this situation, the master theorem would not apply, and you would have to use another method to solve the recurrence.

1.3.1 Examples:

To use the master theorem, we simply plug the numbers into the formula.

Example 1: $T(n) = 9T(n/3)+n$. Here $a = 9$, $b = 3$, $f(n) = n$, and $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$. Since $f(n) = O(n^{\log_3 9-\epsilon})$ for $\epsilon = 1$, case 1 of the master theorem applies, and the solution is $T(n) = \Theta(n^2)$.

Example 2: $T(n) = T(2n/3)+1$. Here $a = 1$, $b = 3/2$, $f(n) = 1$, and $n^{\log_b a} = n^0 = 1$. Since $f(n) = \Theta(n^{\log_b a})$, case 2 of the master theorem applies, so the solution is $T(n) = \Theta(\log n)$.

Example 3: $T(n) = 3T(n/4) + n\log n$. Here $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$. For $\epsilon = 0.2$, we have $f(n) = \Omega(n^{\log_4 3+\epsilon})$. So case 3 applies if we can show that $af(n/b) \leq cf(n)$ for some $c < 1$ and all sufficiently large n. This would mean $3\frac{n}{4}\log\frac{n}{4} \leq cn\log n$. Setting $c = 3/4$ would cause this condition to be satisfied.

Example 4: $T(n) = 2T(n/2)+n\log n$. Here the master method does not apply. $n^{\log_b a} = n$, and $f(n) = n\log n$. Case 3 does not apply because even though $n\log n$ is asymptotically larger than n, it is not polynomially larger. That is, the ratio $f(n)/n^{\log_b a} = \log n$ is asymptotically less than n^ϵ for all positive constants ϵ.

read merge sort from textbook pg 198-200 and your notes which you wrote before exam

DESIGN AND ANALYSIS OF ALGORITHMS

Shylaja S S
Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Merge Sort

Major Slides Content: Anany Levitin

A noteworthy advantage of Merge Sort over Quick Sort and Heap Sort—two important advanced sorting algorithms to be discussed later—is its stability.

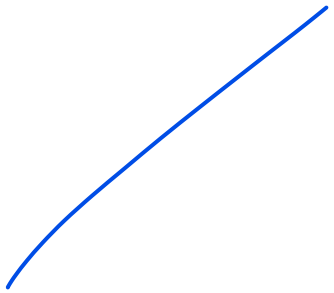
The principal shortcoming of Merge Sort is the linear amount of extra storage the algorithm requires. Though merging can be done in-place, the resulting algorithm is quite complicated and is of theoretical interest only.

Shylaja S S

Department of Computer Science & Engineering

Merge Sort - Idea

- Split array A[0..n-1] into about equal halves and make copies of each half in arrays B and C
- Sort arrays B and C recursively
- Merge sorted arrays B and C into array A as follows:
  - Repeat the following until no elements remain in one of the arrays:
    - compare the first elements in the remaining unprocessed portions of the arrays
    - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
  - Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A



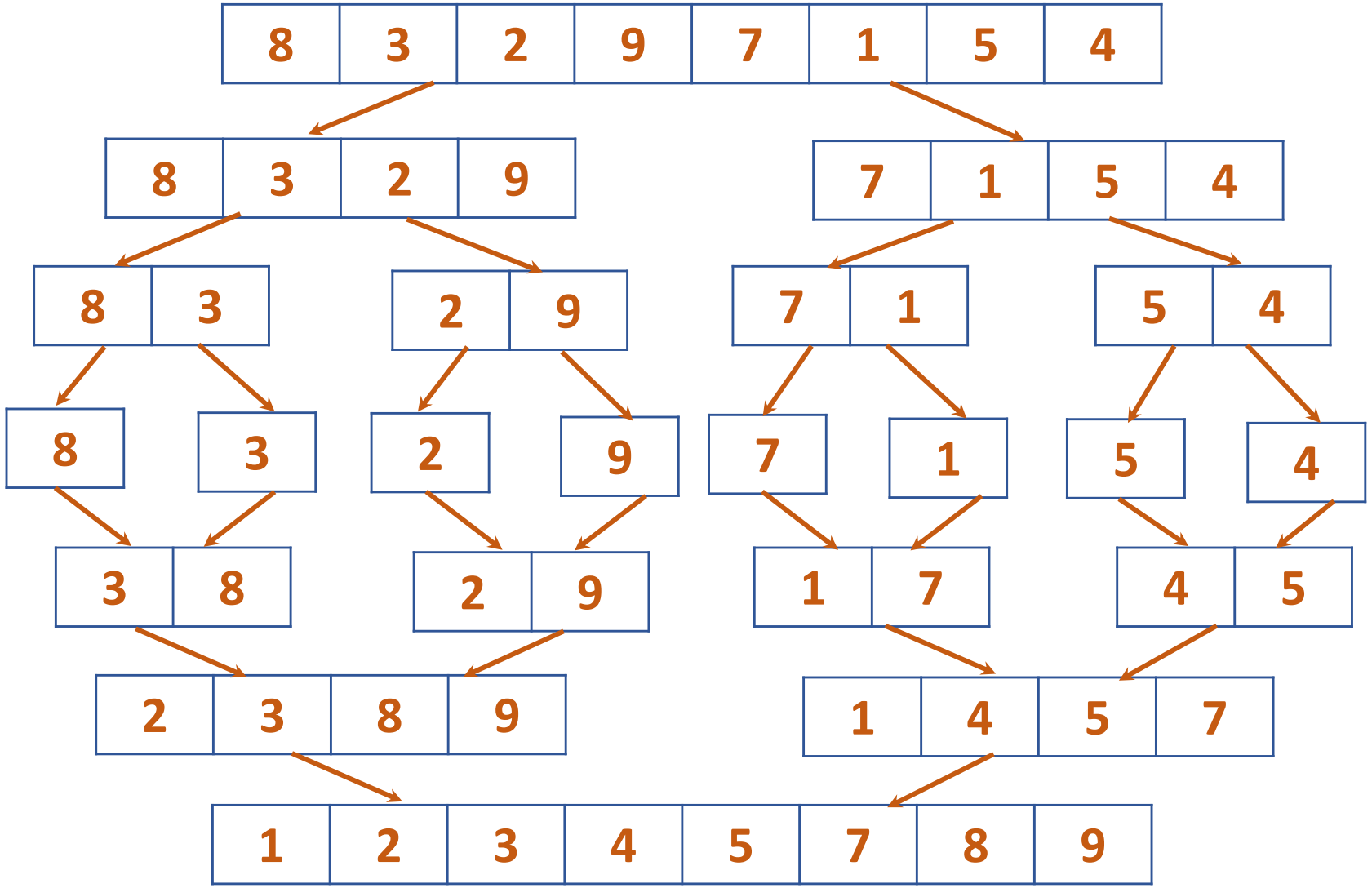
Merge Sort - Algorithm

ALGORITHM Mergesort(A[0 .. n-1])
//Sorts array A[0 .. n-1] by recursive mergesort
//Input: An array A[0 .. n-1] of orderable elements
//Output: Array A[0 .. n-1] sorted in non decreasing order
if n > 1
  copy A[0 .. $\lfloor n/2 \rfloor$ -1] to B[0 .. $\lfloor n/2 \rfloor$ -1]
  copy A[$\lfloor n/2 \rfloor$ .. n -1] to C[0..$\lceil n/2 \rceil$ -1]
  Mergesort(B[0 .. $\lfloor n/2 \rfloor$ - 1])
  Mergesort(C[0 ..$\lceil n/2 \rceil$ -1])
  Merge(B, C, A)

Merge Sort - Algorithm

```
ALGORITHM  Merge(B[0 .. p- 1],  C[0 .. q  -1], A[0 .. p  + q  -1])
//Merges two sorted arrays into one sorted array
//Input: Arrays B[0 .. p -1] and C[0 .. q -1] both sorted
//Output: Sorted array A[0 .. p  + q  -1] of the elements of B and
   C
i<-0;  j<-0;  k<-0
while i < p and j < q do
   if B[i] ≤ C[j]
            A[k] <-B[i];  i <-i  + 1
   else   A[k]<-C[j];   j<-j+1
   k<-K+1
if i =  p
   copy C[j .. q-1] to A[k .. p + q - 1]
else
   copy B[i .. p -1] to A[k .. p + q -1]
```

Merge Sort - Example



Merge Sort - Algorithm

- Assuming for simplicity that n is a power of 2, the recurrence relation for the number of key comparisons C(n) is:

$$C(n) = 2C(n/2) + C_{merge}(n) \text{ [for } n > 1], C(1) = 0$$

- The number of key comparisons performed during the merging stage in the worst case is:

$$C_{merge}(n) = n - 1$$

- Using the above equation:

$$C_{worst}(n) = 2C_{worst}(n/2) + n - 1 \text{ [for } n > 1], C_{worst}(1) = 0$$

- Applying Master Theorem to the above equation:

$$C_{worst}(n) \in \Theta(n \log n)$$

THANK YOU

Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

Shylaja S S

Department of Computer Science
& Engineering

see your notebook mostly
textbook if you feel like with time left
see this slide but just once from upar upar se1

DESIGN AND ANALYSIS OF ALGORITHMS

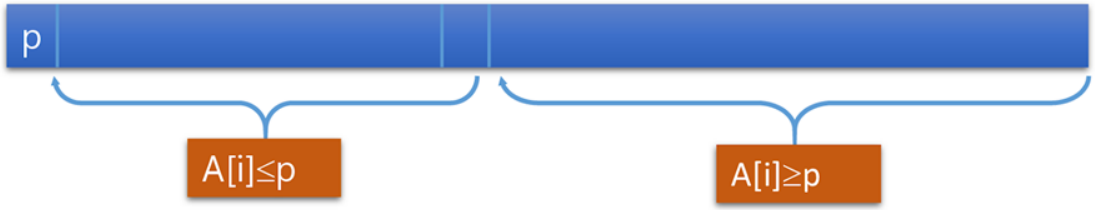
Quick Sort

Major Slides Content: Anany Levitin

Shylaja S S
Department of Computer Science & Engineering

Quick Sort

- Select a pivot (partitioning element) – here, the first element
- Rearrange the list so that all the elements in the first s positions are smaller than or equal to the pivot and all the elements in the remaining n-s positions are larger than or equal to the pivot



- Exchange the pivot with the last element in the first (i.e., ≤) subarray — the pivot is now in its final position
- Sort the two subarrays recursively

Quick Sort - Algorithm

```
ALGORITHM  Quicksort(A[l .. r])
// Sorts a subarray by quicksort
// Input: A subarray A[l … r] of A[0 .. n  -1], defined by its left and
// right indices l  and r
// Output: Subarray A[l .. r]  sorted in non decreasing order
if l  <  r
   s ←Partition(A[l .. r])          //s is a split position
   Quicksort(A[l  .. s - 1])
   Quicksort(A[s + 1 .. r])
```

Quick Sort - Algorithm

```
ALGORITHM  Partition(A[l .. r])
// Partitions a subarray by using its first  element as  a pivot
// Input: A  subarray A[l..r] of A[0 .. n  - 1],  defined by its left and right  indices l and r  (l  <  r)
// Output: A partition of A[l .. r ], with the split position returned as this function's value
p ← A[l]
i  ← l;  j ← r  + 1
repeat
   repeat i ← i + 1 until A[i] ≥ p
   repeat j ← j  - 1 until A[j] ≤ p
   swap(A[i],  A[j])
until i ≥ j
swap(A[i],  A[j])          //undo last swap when i ≥ j
swap(A[i],  A[j])
return  j
```

Quick Sort - Example

5 3 1 9 8 2 4 7

2 3 1 4 5 8 9 7

1 2 3 4 5 7 8 9

1 2 3 4 5 7 8 9

1 2 3 4 5 7 8 9

1 2 3 4 5 7 8 9

Quick Sort - Analysis: Best Case

- The number of comparisons in the best case satisfies the recurrence:
- $C_{best}(n) = 2C_{best}(n/2) + n$ for $n > 1$, $C_{best}(1) = 0$

- According to Master Theorem

$$C_{best}(n) \in \Theta(n \log_2 n)$$

Quick Sort - Analysis: Worst Case

- The number of comparisons in the worst case satisfies the recurrence

$$C_{worst}(n) = (n+1) + n + \cdots + 3 = \frac{(n+1)(n+2)}{2} - 3 \in \theta(n^2)$$

Quick Sort - Analysis: Average Case

Let $C_{avg}(n)$ be the number of key comparisons made by Quick Sort on a randomly ordered array of size n

$$C_{avg}(n) = \frac{1}{n}\sum_{s=0}^{n-1}[(n+1) + C_{avg}(s) + C_{avg}(n-1-s)] \quad \text{for n>1}$$

The solution for the above recurrence is:

$$C_{avg}(n) \approx 2n \ln n \approx 1.38n \log_2 n$$

see average case and go from here just in case

THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

read textbook and go for this topic mandatory page no 208-211

.

if you have time have a glance over tree codes of unit3 once they might ask to write algorithms for those. so do it :)

DESIGN AND ANALYSIS OF ALGORITHMS

Shylaja S S

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Binary Tree

Major Slides Content: Anany Levitin

Shylaja S S
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Binary Tree

- A *binary tree* **T** is defined as a finite set of nodes that is either empty or consists of a root and two disjoint binary trees $\mathbf{T_L}$ and $\mathbf{T_R}$ called as the left and right subtree of the root

- The definition itself divides the Binary Tree into two smaller structures and hence many problems concerning the binary trees can be solved using the Divide – And – Conquer technique

- The binary tree is a Divide – And – Conquer ready structure ☺

PES UNIVERSITY

Height of a Binary Tree

- Height of a Binary Tree: Length of the longest path from root to leaf

ALGORITHM Height(T)

//Computes recursively the height of a binary tree

//Input: A binary tree T

//Output: The height of T

if T = Ø return -1

else return max(Height(T_L), Height(T_R))+ 1

PES UNIVERSITY

DESIGN AND ANALYSIS OF ALGORITHMS

Height of a Binary Tree - Analysis

- The measure of input's size is the number of nodes in the given binary tree. Let us represent this number as n(T)
- Basic Operation: Addition
- The recurrence relation is setup as follows:

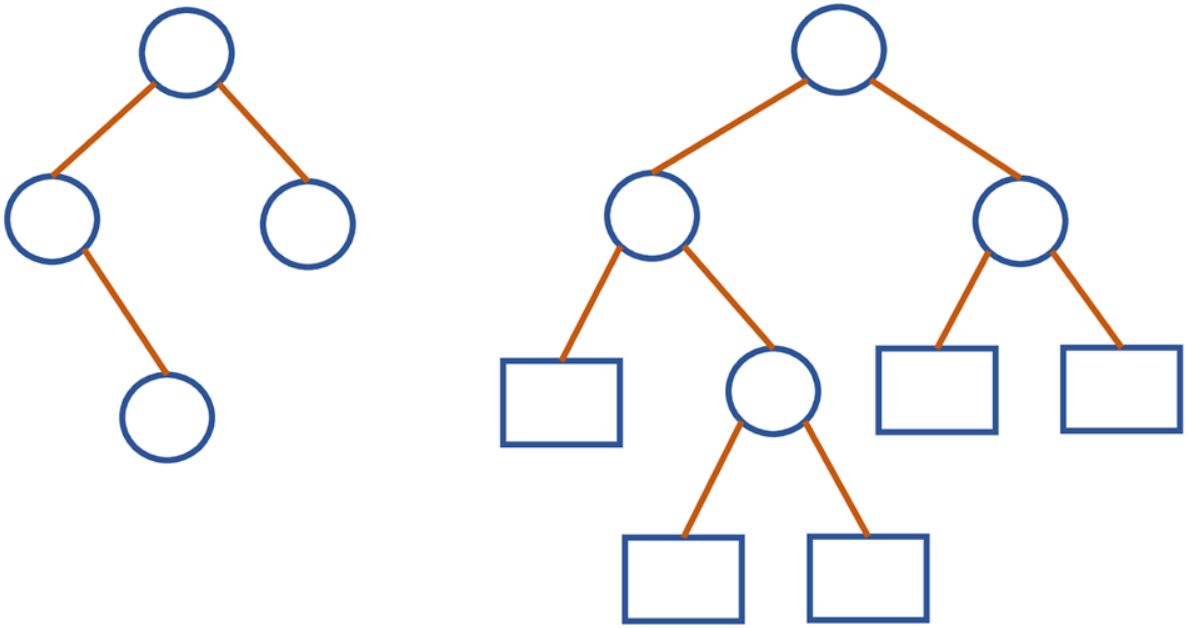
$$A(n(T)) = A(n(T_L)) + A(n(T_R)) + 1, \quad \text{for } n(T) > 0$$

$$A(0) = 0$$

DESIGN AND ANALYSIS OF ALGORITHMS

Height of a Binary Tree - Analysis

- In the analysis of tree algorithms, the tree is extended by replacing empty subtrees by special nodes called external nodes



DESIGN AND ANALYSIS OF ALGORITHMS

Height of a Binary Tree - Analysis

- x – Number of external nodes
- n – Number of internal nodes

$$x = n + 1$$

- The number of comparisons to check whether a tree is empty or not:

$$C(n) = n + x = 2n + 1$$

- The number of additions is:

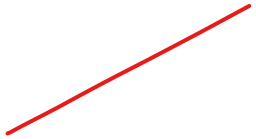
$$A(n) = n$$

DESIGN AND ANALYSIS OF ALGORITHMS

Binary Tree Traversals

- The three classic traversals for a binary tree are inorder, preorder and postorder traversals

- In the preorder traversal, the root is visited before the left and right subtrees are visited (in that order)
- In the inorder traversal, the root is visited after visiting its left subtree but before visiting the right subtree
- In the postorder traversal, the root is visited after visiting the left and right subtrees (in that order)



DESIGN AND ANALYSIS OF ALGORITHMS

Binary Tree Traversals

```
Algorithm Inorder(T)
if T ≠ ∅
    Inorder(Tleft)
    print(root of T)
    Inorder(Tright)
```

```
Algorithm Preorder(T)
if T ≠ ∅
    print(root of T)
    Preorder(T_left)
    Preorder(T_right)
```

```
Algorithm Postorder(T)
if T ≠ ∅
    Postorder(T_left)
    Postorder(T_right)
    print(root of T)
```

PES UNIVERSITY

not there in textbook
see from here and go

THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

read from your notebook
and for matrix multipliation refer
textboo(pg no 212-217)also along
with your notes

DESIGN AND ANALYSIS OF ALGORITHMS

Shylaja S S

Department of Computer Science
& Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Multiplication of Large Integers and Strassen's Matrix Multiplication

Major Slides Content: Anany Levitin

Shylaja S S
Department of Computer Science & Engineering

Multiplication of large integers - Idea

- Let the two numbers being multiplied be a and b
- a and b are n–digit integers, where n is a positive even number
- Let the first half of a's digits be a_1 and second half be a_0
- Similarly, let the first half of b's digits be b_1 and second half be b_0
- In these notations, $a = a_1a_0$ implies $a = a_1*10^{n/2} + a_0$ and $b = b_1b_0$ implies $b = b_1*10^{n/2} + b^0$

Multiplication of large integers - Idea

$c = a*b = (a_1 10^{n/2} + a_0) * (b_1 10^{n/2} + b_0)$

$= (a_1 * b_1)10^n + (a_1 * b_0 + a_0 * b_1)10^{n/2} + (a_0 * b_0)$

$= c_2 10^n + c_1 10^{n/2} + c_0$

where

$c_2 = a_1 * b_1$ is the product of their first halves

$c_0 = a_0 * b_0$ is the product of their second halves

$c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$ is the product of the sum of the a's halves and the sum of the b's halves minus the sum of c_2 and c_0

Multiplication of large integers - Analysis

- $M(n) = 3M(n/2)$ for $n > 1$, $M(1) = 1$
- Solving it by backward substitutions for $n = 2^k$ yields:

$M(2^k) = 3M(2^{k-1}) = 3[3M(2^{k-2})] = 3^2M(2^{k-2})$

$= \ldots = 3^i M(2^{k-i}) = \ldots = 3^kM(2^{k-k}) = 3^k$

- Since $k = \log_2 n$: $M(n) = 3^{\log_2 n} = n^{\log_2 3} = n^{1.585}$
- The number of additions is given by:

$A(n) = 3A(n/2) + cn$ for $n > 1$, $A(1) = 1$

$A(n)$ belongs to $\Theta(n^{\log_2 3})$

Multiplication of large integers - Example

2135 * 4014

$= (21*10^2 + 35) * (40*10^2 + 14)$

$= (21*40)*10^4 + c1*10^2 + 35*14$

where c1 = (21+35)*(40+14) - 21*40 - 35*14, and

21*40 = (2*10 + 1) * (4*10 + 0)

$= (2*4)*10^2 + c2*10 + 1*0$

where c2 = (2+1)*(4+0) - 2*4 - 1*0, etc.,

Strassen's Matrix Multiplication

- This algorithm was published by V Strassen in 1969
- The principal insight of the algorithm lies in the discovery that we can find product of two 2 – by – 2 matrices A and B with seven multiplications as opposed to the eight required by the Brute – Force algorithm
- This is accomplished by the following formulae:

$$\begin{bmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} * \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix}$$

$$= \begin{bmatrix} m_1 + m_4 - m_5 + m_7 & m_3 + m_5 \\ m_2 + m_4 & m_1 + m_3 - m_2 + m_6 \end{bmatrix}$$

Strassen's Matrix Multiplication

$m_1 = (a_{00} + a_{11}) * (b_{00} + b_{11})$

$m_2 = (a_{10} + a_{11}) * b_{00}$

$m_3 = a_{00} * (b_{01} - b_{11})$

$m_4 = a_{11} * (b_{10} - b_{00})$

$m_5 = (a_{00} + a_{01}) * b_{11}$

$m_6 = (a_{10} - a_{00}) * (b_{00} + b_{01})$

$m_7 = (a_{01} - a_{11}) * (b_{10} + b_{11})$

Strassen's Matrix Multiplication – General Formula

For any two matrices A and B of size n – by – n, we can divide A, B and the product C as follows:

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} * \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix}$$

The sub – matrices can be treated as numbers to get the correct product

Strassen's Matrix Multiplication – Analysis

- If M(n) is the number of multiplications made by Strassen's algorithm in multiplying two matrices n – by – n, we get the following recurrence relation for it:

$M(n) = 7M(n/2)$ for $n>1$, $M(1) = 1$

Since $n = 2^k$,

$$M(2^k) = 7M(2^{k-1}) = 7[7M(2^{k-2})] = 7^2M(2^{k-2}) = \ldots$$
$$= 7^i\, M(2^{k-i}) \ldots = 7^k\, M\,(2^{k-k}) = 7^k$$

Since $k = \log_2 n$,

$$\mathbf{M(n) = 7^{\log_2 n} = n^{\log_2 7} \approx n^{2.807}}$$

Strassen's Matrix Multiplication – Analysis

- The number of additions are given by the following recurrence:

  $A(n) = 7\,A(n/2) + 18(n/2)^2$ for $n>1$, $A(1) = 0$

- According to Master's Theorem, A(n) belongs to $\Theta(n^{\log_2 7})$

THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

Surabhi Narayan

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Transform and Conquer

Surabhi Narayan
Department of Computer Science & Engineering

Transform and Conquer

Transform and Conquer:

- In Transformation stage, the problem's instance is modified to be, for one reason or another, more amenable to solution.
- Then, in the second or conquering stage, it is solved.

There are three major variations that differ by what we transform a given instance to

- Transformation to a simpler or more convenient instance of the same problem—we call it ***instance simplification***.
- Transformation to a different representation of the same instance—we call it ***representation change***.
- Transformation to an instance of a different problem for which an algorithm is already available—we call it ***problem reduction***.

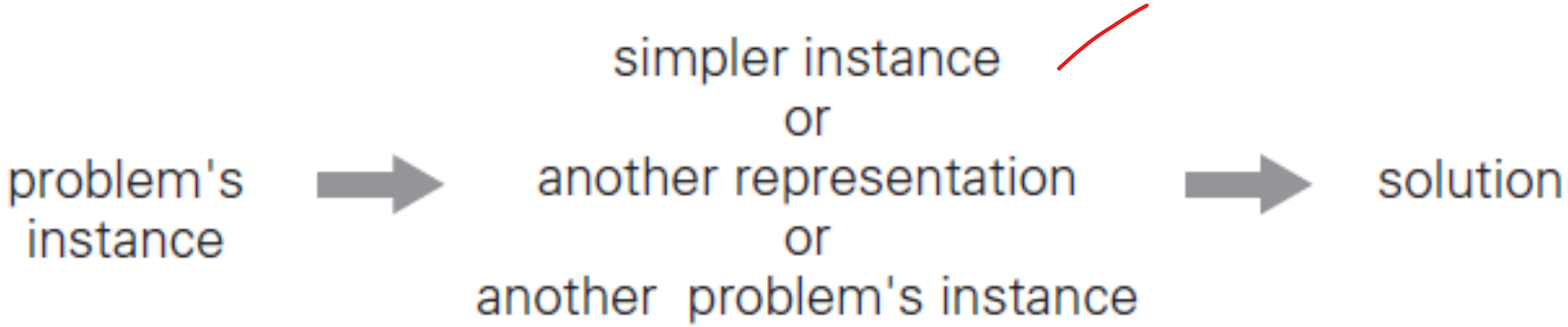


FIGURE 6.1 Transform-and-conquer strategy.

Checking element uniqueness in an array

The brute-force algorithm compared pairs of the array's elements until either two equal elements were found or no more pairs were left. Its worst-case efficiency was in (n^2).

Alternatively, we can sort the array first and then check only its consecutive elements: if the array has equal elements, a pair of them must be next to each other, and vice versa.

Presorting means sorting data before applying an algorithm that either:
*Requires sorted input to work correctly (like binary search), or
*Works faster when the input is sorted (like in merge-based algorithms or greedy strategies).

algorithms like quicksort and mergesort

ALGORITHM *PresortElementUniqueness*$(A[0..n-1])$

//Solves the element uniqueness problem by sorting the array first
//Input: An array $A[0..n-1]$ of orderable elements
//Output: Returns "true" if A has no equal elements, "false" otherwise

sort the array A

for $i \leftarrow 0$ **to** $n-2$ **do**
if $A[i] = A[i+1]$ **return false**
return true

The running time of this algorithm is the sum of the time spent on sorting and the time spent on checking consecutive elements. Since the former requires at least n log n comparisons and the latter needs no more than n −1comparisons, it is the sorting part that will determine the overall efficiency of the algorithm. So, if we use aquadratic sorting algorithm here, the entire algorithm will not be more efficient than the brute-force one. But if we use a good sorting algorithm, such as mergesort, with worst-case efficiency in (nlog n), the worst-case efficiency of the entire presorting-based algorithm will be also in (nlog n):

$$T(n) = T_{sort}(n) + T_{scan}(n) \in \Theta(n \log n) + \Theta(n) = \Theta(n \log n).$$

Computing mode with presorting

ALGORITHM *PresortMode*$(A[0..n-1])$

```
//Computes the mode of an array by sorting it first
//Input: An array A[0..n − 1] of orderable elements
//Output: The array's mode
sort the array A
i ← 0                          //current run begins at position i
modefrequency ← 0              //highest frequency seen so far
while i ≤ n − 1 do
    runlength ← 1;  runvalue ← A[i]
    while i + runlength ≤ n − 1 and A[i + runlength] = runvalue
        runlength ← runlength + 1
    if runlength > modefrequency
        modefrequency ← runlength;  modevalue ← runvalue
    i ← i + runlength
return modevalue
```

$$T(n) = T_{sort}(n) + T_{scan}(n) \in \Theta(n \log n) + \Theta(n)$$

$$= \Theta(n \log n)$$

the efficiency remains same as the above i.e. nlogn

before going to exam read the part in tb for example 1 and example 2 pg no:228-231

THANK YOU

Surabhi Narayan
Department of Computer Science &Engineering

surabhinarayan@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

Surabhi Narayan

Department of Computer Science & Engineering

Now, where do heap and heapsort fall?
They fall under:
(ii) Representation Change
Why?
In heapsort, we transform the array into a heap (specifically, a binary heap – a special kind of binary tree stored as an array).
This transformation gives us an efficient way to repeatedly extract the largest or smallest element.
So we're not simplifying the problem or changing it into another kind of problem — we're just changing the data representation to something more powerful for our task.

DESIGN AND ANALYSIS OF ALGORITHMS

Heap and Heap Sort

Surabhi Narayan
Department of Computer Science & Engineering

Heap and Heap Sort

Heap is partially ordered data structure that is especially suitable for implementing priority queues.

A ***priority queue*** is a multiset of items with an orderable characteristic called an item's ***priority***, with the following operations:

- finding an item with the highest (i.e., largest) priority
- deleting an item with the highest priority
- adding a new item to the multiset

Heap and Heap Sort

DEFINITION : A ***heap*** can be defined as a binary tree with keys assigned to its nodes, one key per node, provided the following two conditions are met:

1. The ***shape property***—the binary tree is ***essentially complete*** (or simply ***complete***), i.e., all its levels are full except possibly the last level, where only some rightmost leaves may be missing.

2. The ***parental dominance*** or ***heap property***—the key in each node is greater than or equal to the keys in its children. (This condition is considered automatically satisfied for all leaves.)

Heap and Heap Sort

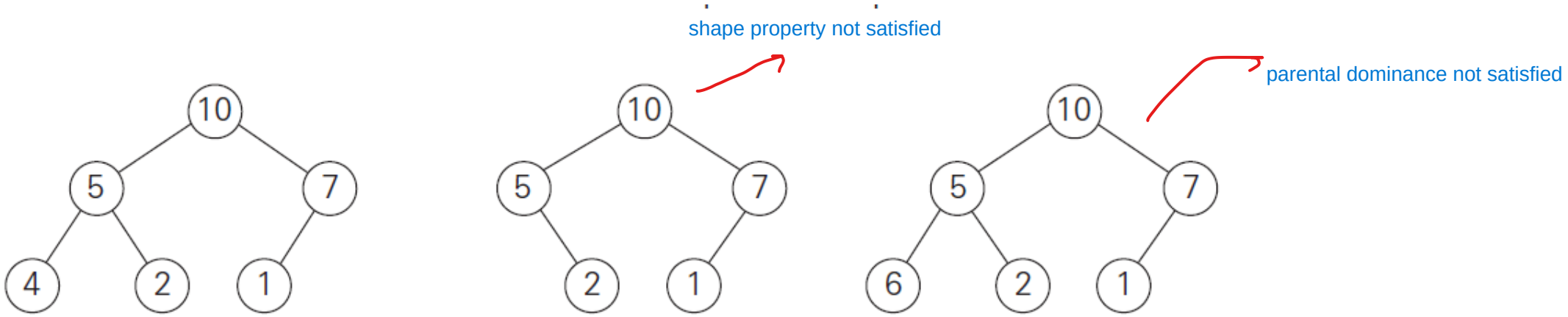


FIGURE 6.9 Illustration of the definition of heap: only the leftmost tree is a heap.

Heap and Heap Sort

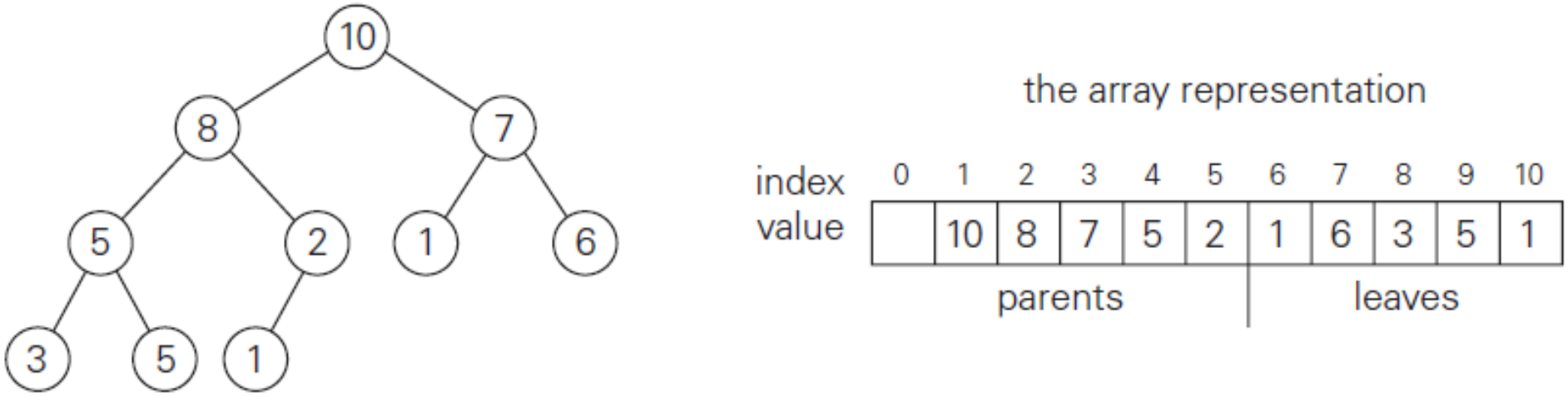


FIGURE 6.10 Heap and its array representation.

Properties of Heap

There exists exactly one essentially complete binary tree with n nodes. Its height is equal to $\log_2 n$.

2. The root of a heap always contains its largest element.

3. A node of a heap considered with all its descendants is also a heap.

4. A heap can be implemented as an array by recording its elements in the topdown, left-to-right fashion. It is convenient to store the heap's elements in positions 1 through n of such an array, leaving $H[0]$ either unused or putting there a sentinel whose value is greater than every element in the heap. In such a representation,

a. the parental node keys will be in the first $n/2$ positions of the array, while the leaf keys will occupy the last $n/2$ positions;

b. the children of a key in the array's parental position i $(1 \le i \le n/2)$ will be in positions $2i$ and $2i + 1$, and, correspondingly, the parent of a key in position i $(2 \le i \le n)$ will be in position $i/2$.

Heap and Heap Sort

The first is the bottom-up heap construction algorithm illustrated in Figure 6.11. It initializes the essentially complete binary tree with n nodes by placing keys in the order given and then “heapifies” the tree as follows. Starting with the last parental node, the algorithm checks whether the parental dominance holds for the key in this node. If it does not, the algorithm exchanges the node’s key K with the larger key of its children and checks whether the parental dominance holds for K in its new position. This process continues until the parental dominance for K is satisfied (Eventually, it has to because it holds automatically for any key in a leaf.) After completing the “heapification” of the subtreerootedatthecurrentparentalnode,thealgorithmproceedstodothesame for the node’s immediate predecessor. The algorithm stops after this is done for the root of the tree.

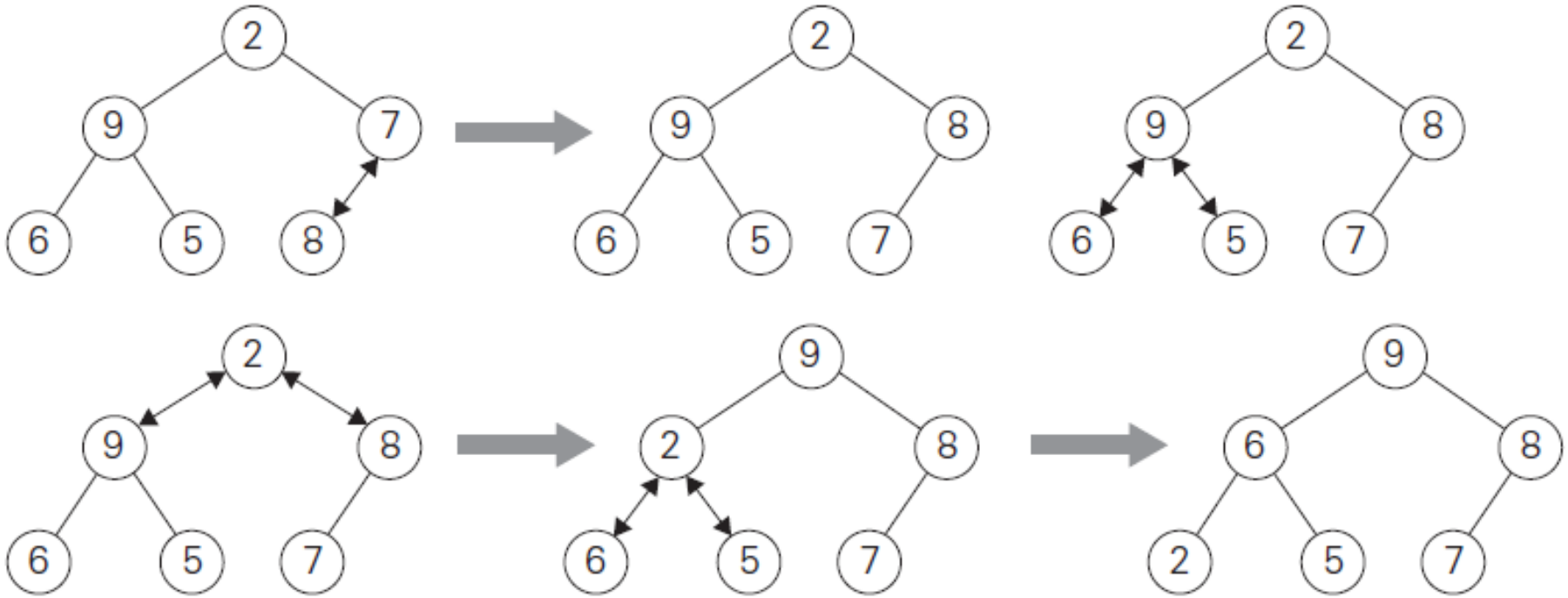


FIGURE 6.11 Bottom-up construction of a heap for the list 2, 9, 7, 6, 5, 8. The double-headed arrows show key comparisons verifying the parental dominance.

Heap and Heap Sort

ALGORITHM *HeapBottomUp*$(H[1..n])$

```
//Constructs a heap from elements of a given array
// by the bottom-up algorithm
//Input: An array H[1..n] of orderable items
//Output: A heap H[1..n]
for i ← ⌊n/2⌋ downto 1 do
    k ← i;   v ← H[k]
    heap ← false
    while not heap and 2 * k ≤ n do
        j ← 2 * k
        if j < n    //there are two children
            if H[j] < H[j + 1]  j ← j + 1
        if v ≥ H[j]
            heap ← true
        else H[k] ← H[j];    k ← j
    H[k] ← v
```

Worst Case,Best Case ,Average case all are O(n) for Bottom Up Heap construction

just see the basic count part of it from textbook and go before exam

Top Down Approach

- Insert a new key K into a heap by attaching a new node with key K in it after the last leaf of the existing heap.
- Sift K up to its appropriate place in the new heap as follows.
  - Compare K with its parent's key: if the latter is greater than or equal to K, stop (the structure is a heap);
  - otherwise, swap these two keys and compare K with its new parent.
- This swapping continues until K is not greater than its last parent or it reaches the root
- Insertion operation cannot require more key comparisons than the heap's height.
- The time efficiency of insertion is in $O(\log n)$.

Heap and Heap Sort

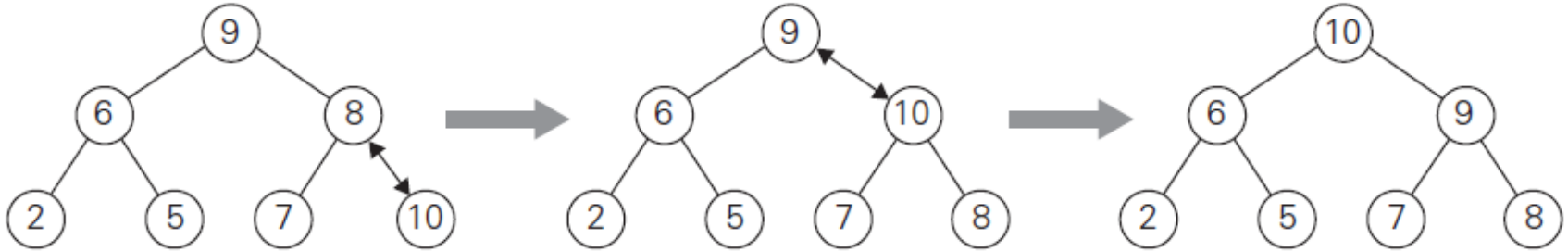


FIGURE 6.12 Inserting a key (10) into the heap constructed in Figure 6.11. The new key is sifted up via a swap with its parent until it is not larger than its parent (or is in the root).

Heap and Heap Sort

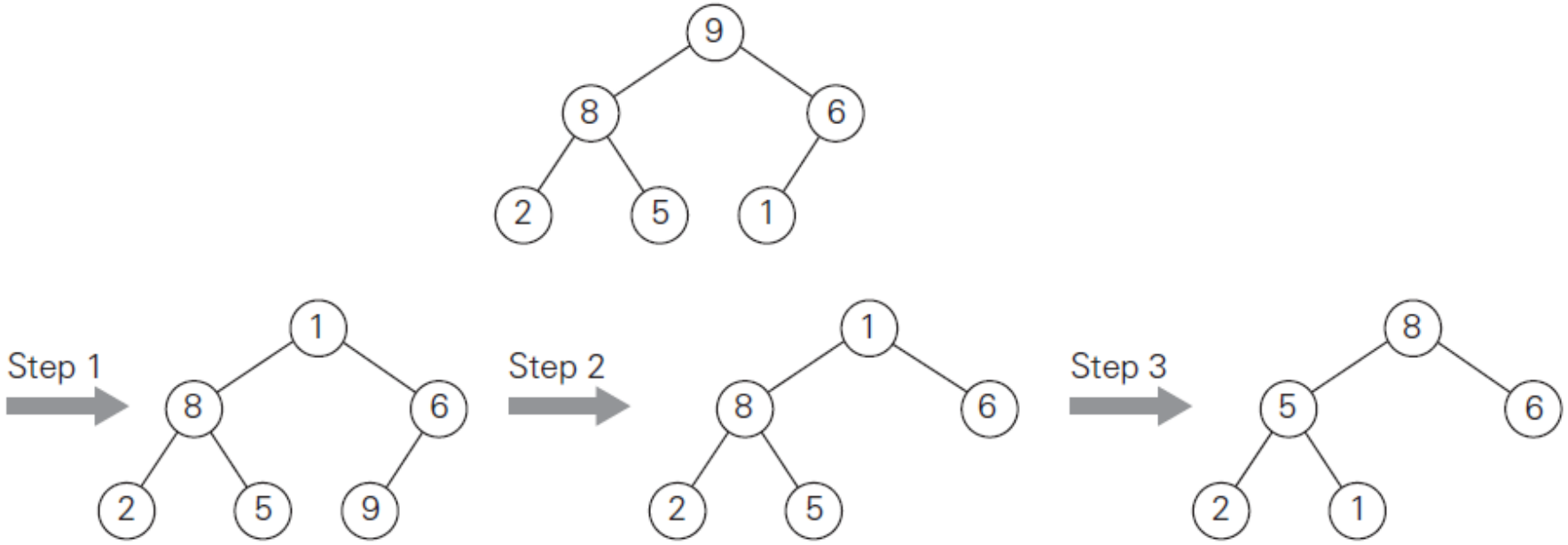


FIGURE 6.13 Deleting the root's key from a heap. The key to be deleted is swapped with the last key after which the smaller tree is "heapified" by exchanging the new key in its root with the larger key in its children until the parental dominance requirement is satisfied.

Maximum Key Deletion from a heap

Step 1 Exchange the root's key with the last key K of the heap.

Step 2 Decrease the heap's size by 1.

Step 3 "Heapify" the smaller tree by sifting K down the tree exactly in the same way we did it in the bottom-up heap construction algorithm. That is, verify the parental dominance for K: if it holds, we are done; if not, swap K with the larger of its children and repeat this operation until the parental dominance condition holds for K in its new position.

The efficiency of deletion is determined by the number of key comparisons needed to "heapify" the tree after the swap has been made and the size of the tree is decreased by 1. Since this cannot require more key comparisons than twice the heap's height, the time efficiency of deletion is in O(log n) as well.

Heap Sort

Two-stage algorithm that works as follows:

Stage 1 (heap construction): Construct a heap for a given array.
Stage 2 (maximum deletions): Apply the root-deletion operation $n - 1$ times to the remaining heap.

As a result, the array elements are eliminated in decreasing order. But since under the array implementation of heaps an element being deleted is placed last, the resulting array will be exactly the original array sorted in increasing order.

Heap and Heap Sort

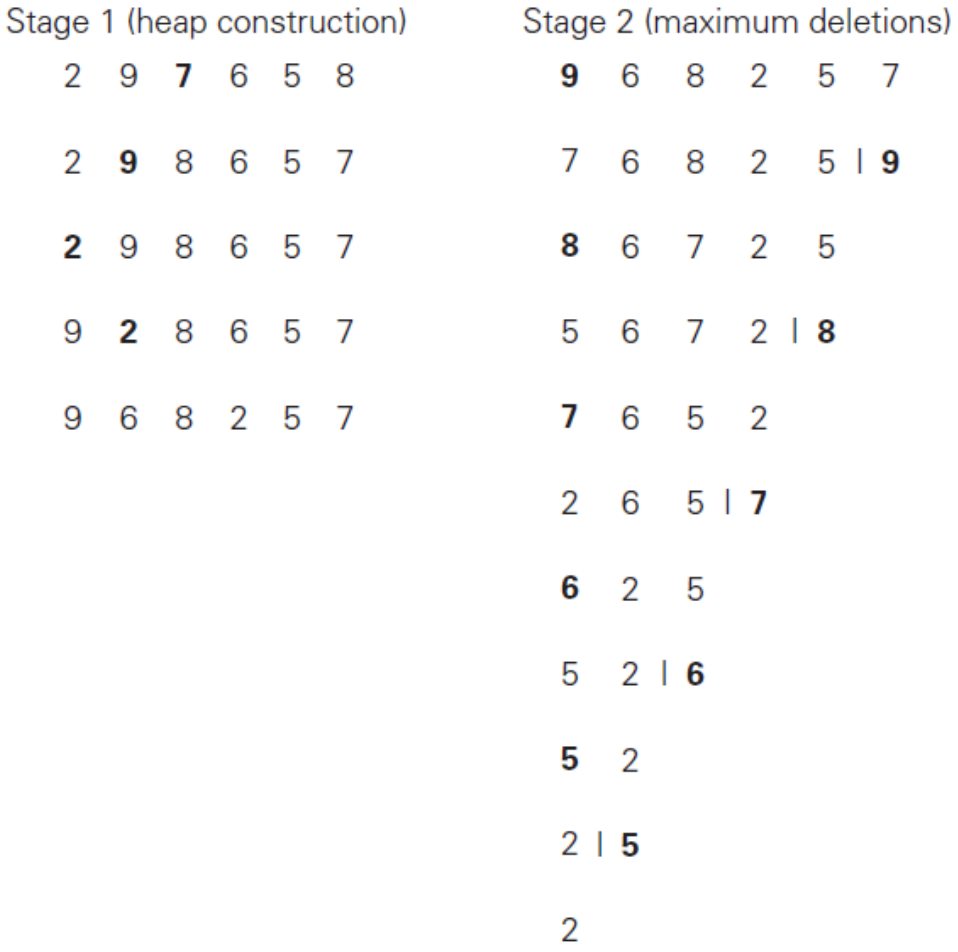


FIGURE 6.14 Sorting the array 2, 9, 7, 6, 5, 8 by heapsort.

Complexity Analysis of Heap Sort

$$C(n) \le 2\lfloor \log_2(n-1) \rfloor + 2\lfloor \log_2(n-2) \rfloor + \cdots + 2\lfloor \log_2 1 \rfloor \le 2 \sum_{i=1}^{n-1} \log_2 i$$

$$\le 2 \sum_{i=1}^{n-1} \log_2(n-1) = 2(n-1)\log_2(n-1) \le 2n \log_2 n.$$

This means that C(n) = O(n log n) for the second stage of heapsort.
For both stages, we get:
O(n) + O(n log n) = O(n log n)
A more detailed analysis shows that the time efficiency of heapsort is, in fact, in (n log n) in both the worst and average cases.
Thus, heapsort's time efficiency falls in the same class as that of mergesort.
Unlike the latter, heapsort is in-place, i.e., it does not require any extra storage.
Timing experiments on random files show that heapsort runs more slowly than quicksort but can be competitive with mergesort.

THANK YOU

Surabhi Narayan
Department of Computer Science &Engineering

surabhinarayan@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

Surabhi Narayan

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

RED BLACK TREE

Let’s reinterpret what they mean by instance simplification here — it's not the typical "reduce input" like in sorting.
Instead, they’re saying:
"We take an unbalanced BST (which is inefficient) and transform it into a balanced one (which is efficient) — through AVL or Red-Black strategies."
So, the input instance (the BST you’re working with) is being simplified structurally to make operations efficient.

Thus, in this sense:
AVL Tree / Red-Black Tree = instance-simplification of an unbalanced BST.

It’s a slightly broader interpretation of instance simplification — not simplifying raw data, but simplifying the structure of a data structure.

Surabhi Narayan

Department of Computer Science & Engineering

these come into instance simplification part of transform and conquer

Red Black Tree

Balanced Binary Search Tree

Revisit AVL Tree

What is a Self-Balancing BST?
A normal Binary Search Tree (BST) can degrade to a linked list (worst case) if the elements are inserted in sorted order. Self-balancing BSTs avoid that by keeping the height of the tree small, typically O(log n).

AVL Tree - "Strictly Balanced"
AVL (Adelson-Velsky and Landis) Tree ensures that:
- # For every node, the balance factor (difference in height between left and right subtrees) is in {-1, 0, 1}.
- # If not, it performs rotations to rebalance.

Advantages:
- # More rigidly balanced --> faster lookups (O(log n) and usually with smaller constants).
- # Great for read-heavy applications.

searching

Downsides:
- # More rotations on insert/delete --> overhead for write-heavy operations.

Red Black Tree

Balanced Binary Search Tree

Revisit AVL Tree

Red-Black Tree — "Loosely Balanced"
Red-Black Tree maintains balance using color rules:
- # Every node is red or black.
- # No two red nodes in a row.
- # Every path from root to null has the same number of black nodes (black-height).

This allows a bit more imbalance than AVL but still guarantees O(log n) operations.

Advantages:
- # Fewer rotations than AVL in insert/delete --> better for write-heavy applications (like in real-time systems).
- # Used in many library implementations (e.g., C++ STL map, Java TreeMap).

Downsides:
- # Lookup may be slightly slower than AVL due to looser balancing.

Red Black Tree

Balanced Binary Search Tree

Revisit AVL Tree

When to Use Which?

AVL Tree: When you need fast searching, like in database indexing or in-memory search systems.

Red-Black Tree: When you want faster insertion/deletion, like in OS schedulers, or standard libraries where balanced behavior is more critical than speed extremes

Red Black Tree

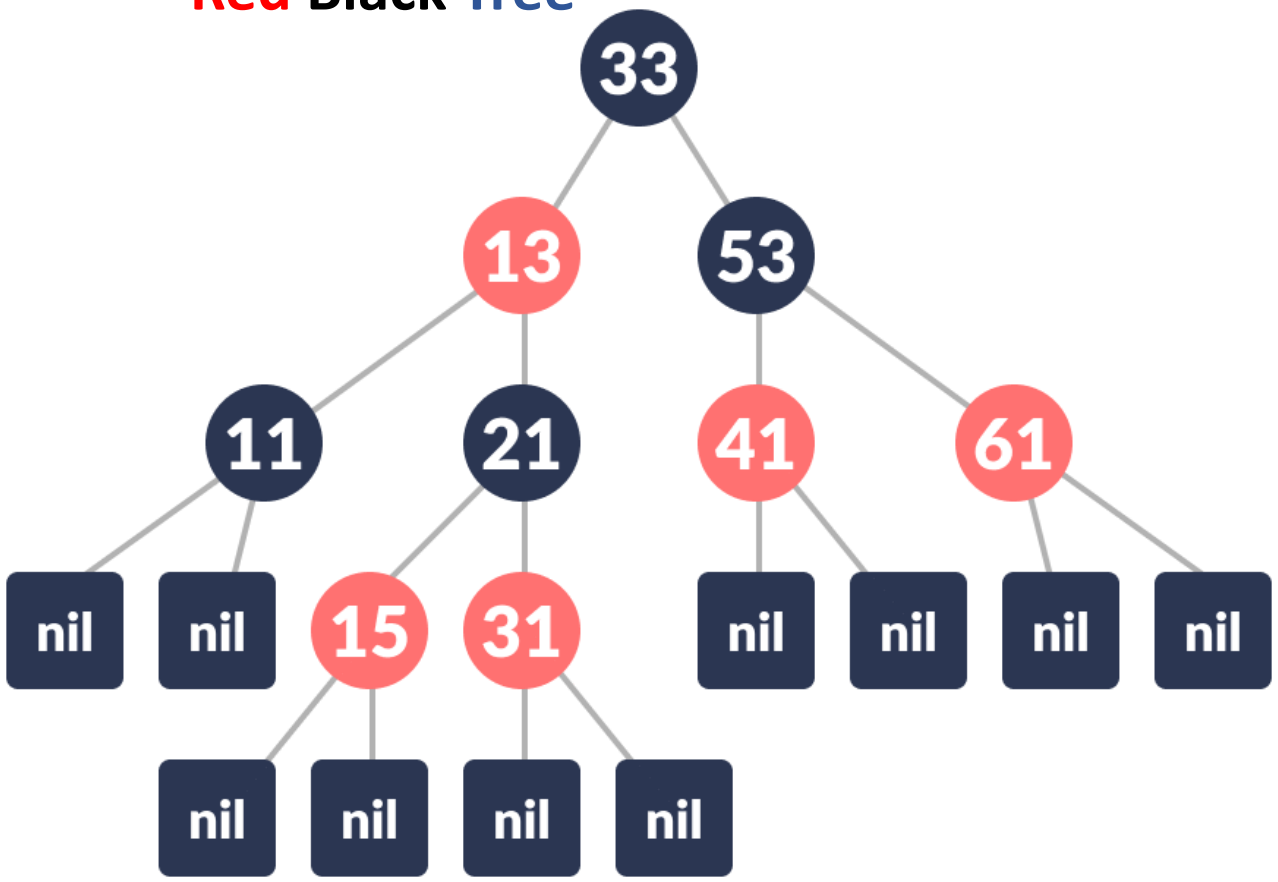
- A Red Black Tree is a category of the self-balancing binary search tree.
- Created in 1972 by Rudolf Bayer who termed them **"symmetric binary B-trees**."
- A red-black tree is a Binary tree where a particular node has color as an extra attribute, either red or black.
- These colours are used to ensure that the tree remains balanced during insertions and deletions.
- Although the balance of the tree is not perfect, it is good enough to reduce the searching time and maintain it around O(log n) time, where n is the total number of elements in the tree.

Red Black Tree

A red-black tree satisfies the following properties:

- **Red/Black Property:** Every node is colored, either red or black.
- **Root Property:** The root is black.
- **Leaf Property:** Every leaf (NIL) is black.
- **Red Property:** If a red node has children then, the children are always black.
- **Depth Property:** For each node, any simple path from this node to any of its descendant leaf has the same black-depth (the number of black nodes).

Red Black Tree



Each node has the following attributes:

- Color
- key
- leftChild
- rightChild
- parent (except root node)

Interesting points about Red-Black Tree:

- Black height of the red-black tree is the number of black nodes on a path from the root node to a leaf node. Leaf nodes are also counted as black nodes. So, a red-black tree of height h has black height >= h/2. *Number of nodes from a node to its farthest descendant leaf is no more than twice as the number of nodes to the nearest descendant leaf.*

- Height of a red-black tree with n nodes is $h <= 2 \log_2(n + 1)$.
- All leaves (NIL) are black.
- The black depth of a node is defined as the number of black nodes from the root to that node i.e the number of black ancestors.
- Every red-black tree is a special case of a binary tree.

Red Black Tree

Applications:

- Most of the self-balancing BST library functions like map and set in C++ (OR TreeSet and TreeMap in Java) use Red-Black Tree.
- It is used to implement CPU Scheduling Linux. Completely Fair Scheduler uses it.
- Besides they are used in the K-mean clustering algorithm for reducing time complexity.
- Moreover, MySQL also uses the Red-Black tree for indexes on tables.

Operations on a Red-Black Tree

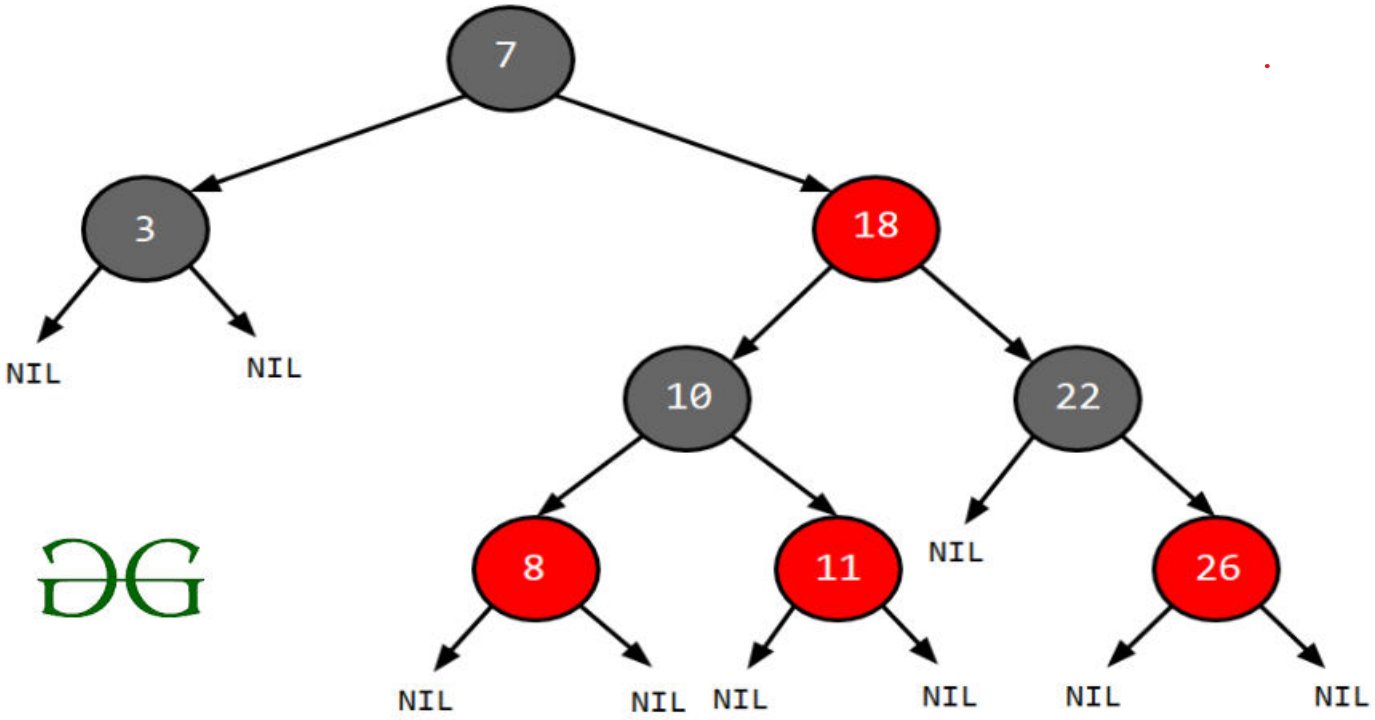
- insert a key value (insert)
- determine whether a key value is in the tree (lookup/search)
- remove key value from the tree (delete)
- print all of the key values in sorted order (print)

Search in Red-black Tree:

As every red-black tree is a special case of a binary tree so the searching algorithm of a red-black tree is similar to that of a binary tree.

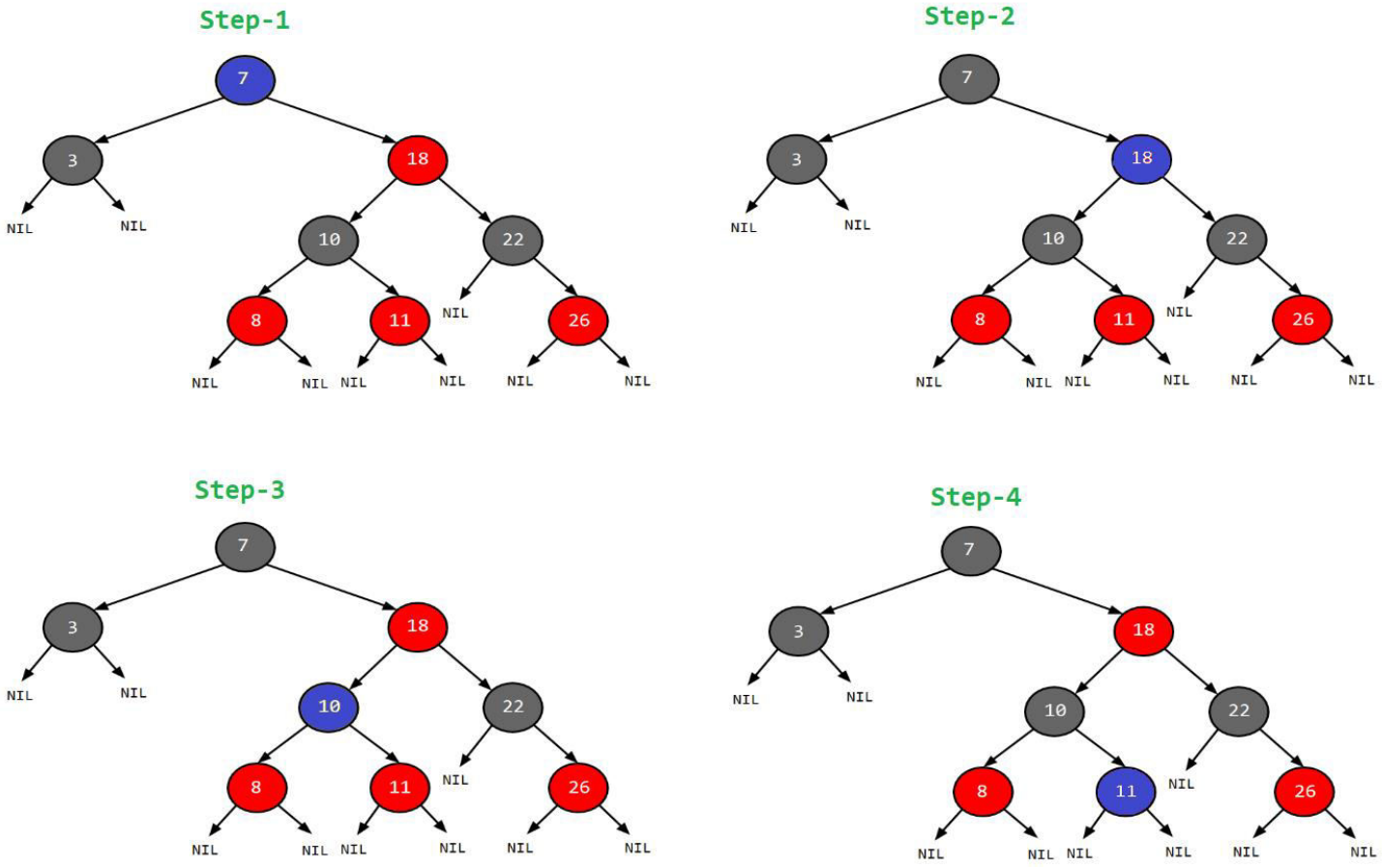
```
searchElement (tree, val)
Step 1: If tree -> data = val OR tree = NULL
Return tree
    Else If val data
            Return searchElement (tree -> left, val)
        Else Return searchElement (tree -> right, val)
        [ End of if ]
[ End of if ]

Step 2: END
```

Search in Red-black Tree: search for the key 11



1.Start from the root.
2.Compare the inserting element with root, if less than root, then recurse for left, else recurse for right.
3.If the element to search is found anywhere, return true, else return false.

Search in Red-black Tree: search for the key 11



DESIGN AND ANALYSIS OF ALGORITHMS

Red Black Tree - Insertion

see the rules from your notebook

Let x be the newly inserted node.

1.Perform standard BST insertion and make the colour of newly inserted nodes as RED.

2.If x is the root, change the colour of x as BLACK (Black height of complete tree increases by 1).

3.Do the following if the color of x's parent is not BLACK **and** x is not the root.

a) If x's uncle is RED (Grandparent must have been black from property 4)

(i) Change the colour of parent and uncle as BLACK.

(ii) Colour of a grandparent as RED.

(iii) Change x = x's grandparent, repeat steps 2 and 3 for new x.

b) If x's uncle is BLACK, then there can be four configurations for x, x's parent (**p**) and x's grandparent (**g**) (This is similar to AVL Tree)

(i) Left Left Case (p is left child of g and x is left child of p)

(ii) Left Right Case (p is left child of g and x is the right child of p)

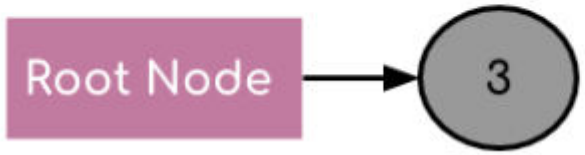
(iii) Right Right Case (Mirror of case i)

(iv) Right Left Case (Mirror of case ii)

Red Black Tree - Insertion

Create a red-black tree with elements 3, 21, 32 and 15.

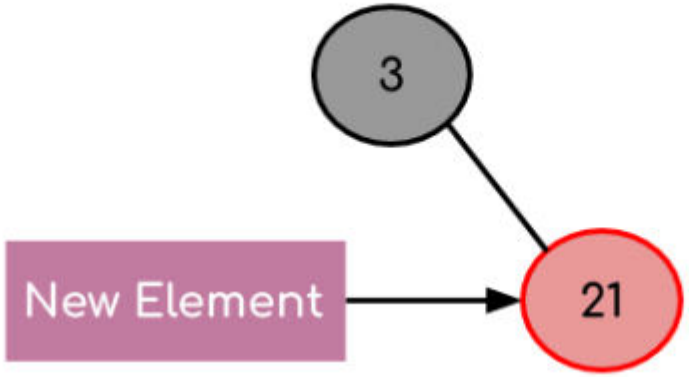
Step 1: Inserting element 3 inside the tree.



Red Black Tree - Insertion

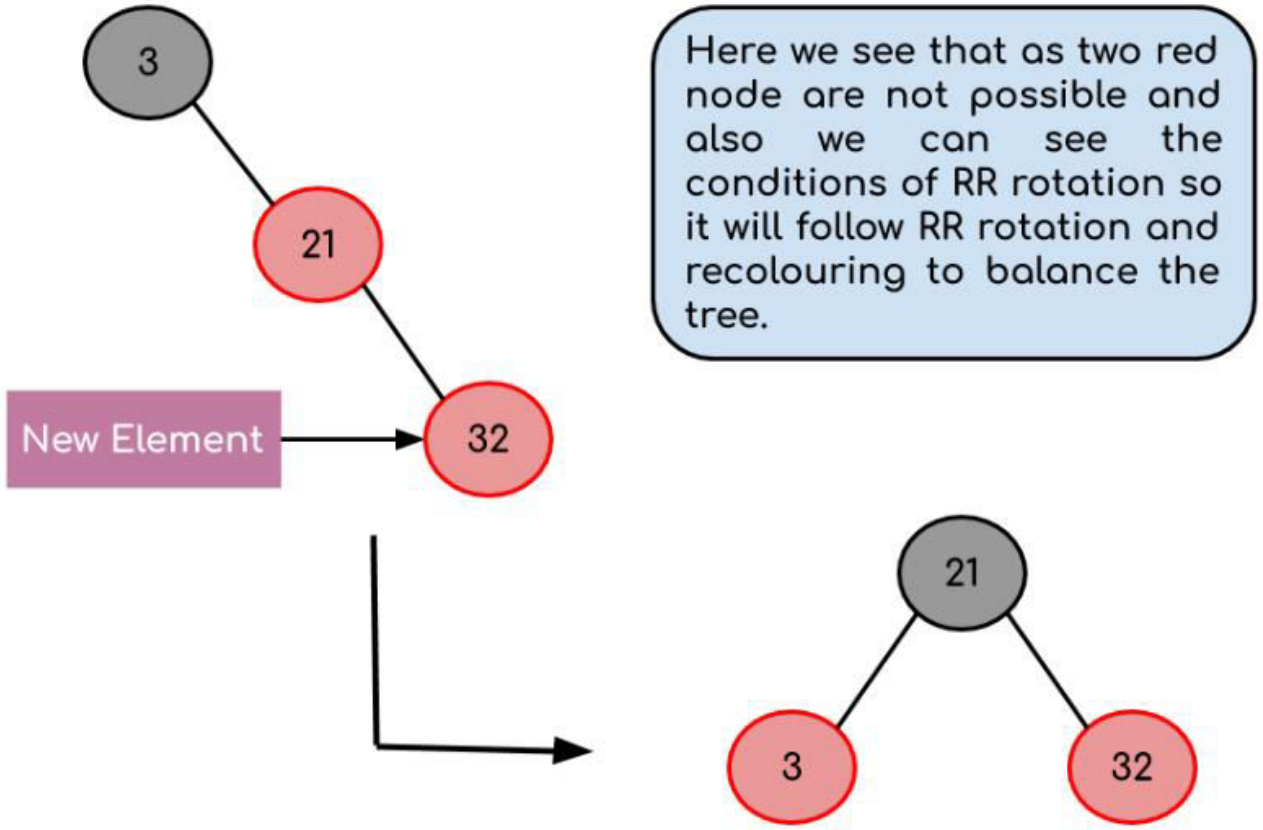
Create a red-black tree with elements 3, 21, 32 and 15.

Step 2: Inserting element 21 inside the tree.



Create a red-black tree with elements 3, 21, 32 and 15.

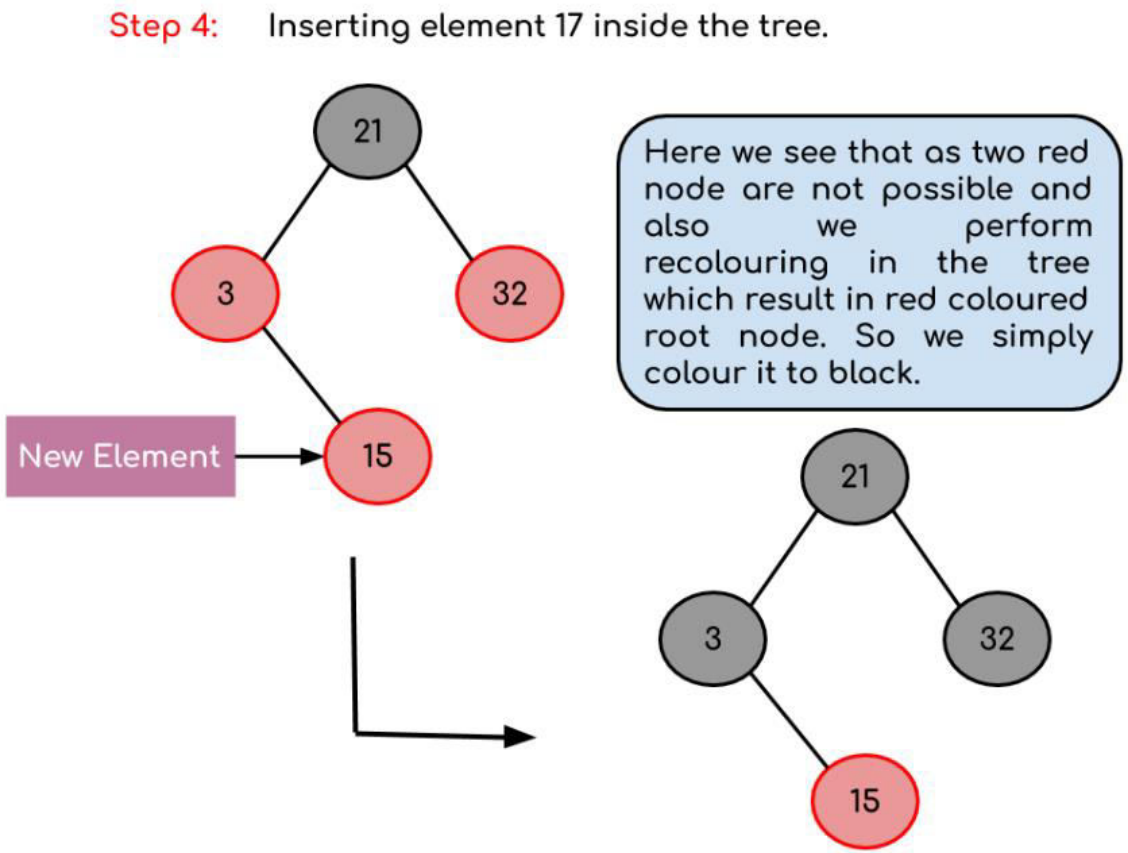
Step 3: Inserting element 32 inside the tree.



Red Black Tree - Insertion

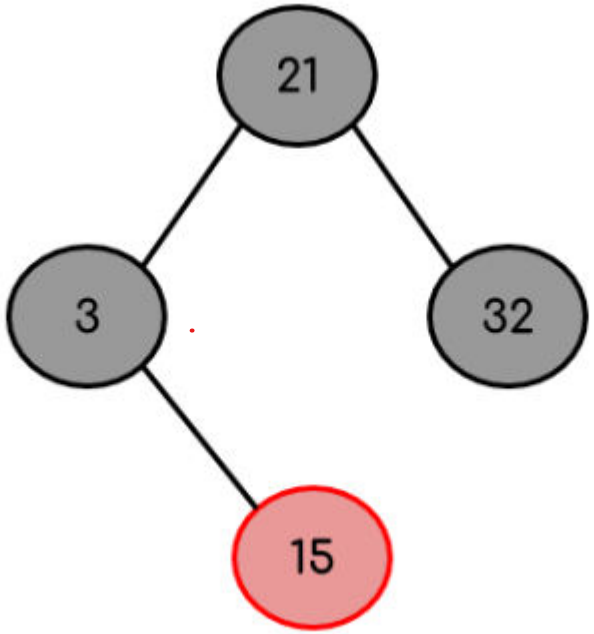
PES UNIVERSITY ONLINE

Create a red-black tree with elements 3, 21, 32 and 15.



Red Black Tree - Insertion

Create a red-black tree with elements 3, 21, 32 and 15.



DESIGN AND ANALYSIS OF ALGORITHMS

Red Black Tree - Insertion

Create a red-black tree with elements 1,2,3,4,5,6,7,8,9,10,11,12,13.

THANK YOU

Surabhi Narayan
Department of Computer Science &Engineering

surabhinarayan@pes.edu

Design And Analysis of Algorithms

Unit 3: 2-3 Trees

Dr. Divyashree N
Department of Computer Science and Engineering

Design And Analysis of Algorithms

2-3 Trees

Topics Covered

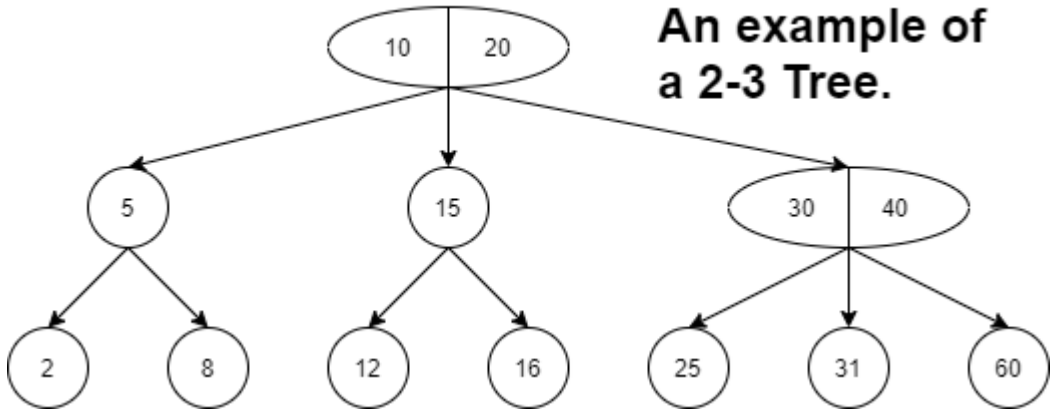
- Introduction to Tree Data Structures
- Definition of 2-3 Trees
- Properties of 2-3 Trees
- Structure of 2-3 Nodes
- Searching in 2-3 Trees
- Insertion in 2-3 Trees
- Deletion in 2-3 Trees
- Balancing Mechanism
- Comparison with Other Trees
- Advantages of 2-3 Trees
- Disadvantages of 2-3 Trees

Introduction to 2-3 Trees

A 2-3 Tree is a tree data structure where every node with children has either two children and one data element or three children and two data elements.

Ensures balance by maintaining all leaves at the same depth.

A special case of B-trees with order 3.



An example of a 2-3 Tree.

Properties of 2-3 Trees

A 2-3 tree follows the below mentioned properties.

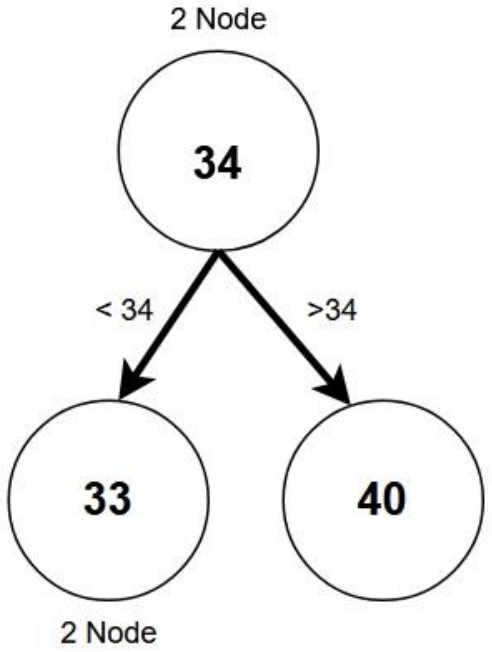
1. Every internal node in the tree is a **2-node** or a **3-node** i.e it has either one value or two values.
2. A node with one value is either a leaf node or has exactly two children. **Values in left sub tree < value in node < values in right sub tree**.
3. A node with two values is either a leaf node or has exactly 3 children. It cannot have 2 children. **Values in left sub tree < first value in node < values in middle sub tree < second value in node < value in right sub tree**.
4. All leaf nodes are at the same level.

Note: A 4-NODE, with three data elements, may be temporarily created during manipulation of the tree but is never persistently stored in the tree.

Structure of 2-3 Nodes

2-node:

- Exactly two children.
- Left child with smaller weight.
- Right child with greater weight.
- Can be a leaf node with no child.



3-node

Exactly three children.

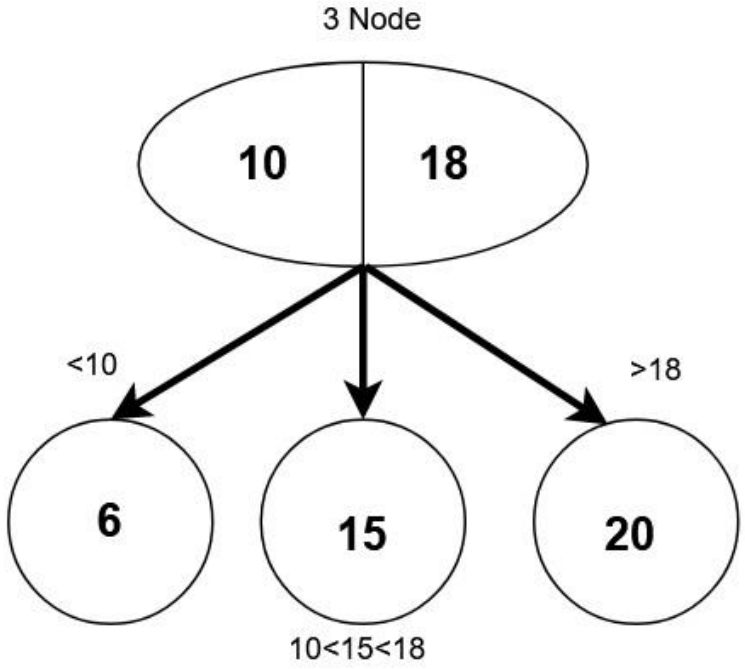
2 data values.

Left child with weight smaller than left data value.

Middle child with weight in between both data values.

Right child with weight greater than right data value.

Can never be a leaf node.



Searching in 2-3 Trees

1. Similar to the search operation in a binary search tree as the data is sorted.
2. To search X in a 2-3 tree.
3. If tree is empty → return False
4. If we reach root node then return False (not found)
5. If X is less than the left data part, then search the left subtree
6. If X is greater than left data and more than right data, search the middle subtree.
7. If X is greater than the right data part, then search right subtree.

- Efficiency: Time complexity is O(logN)

Unit 3: 2-3 Trees

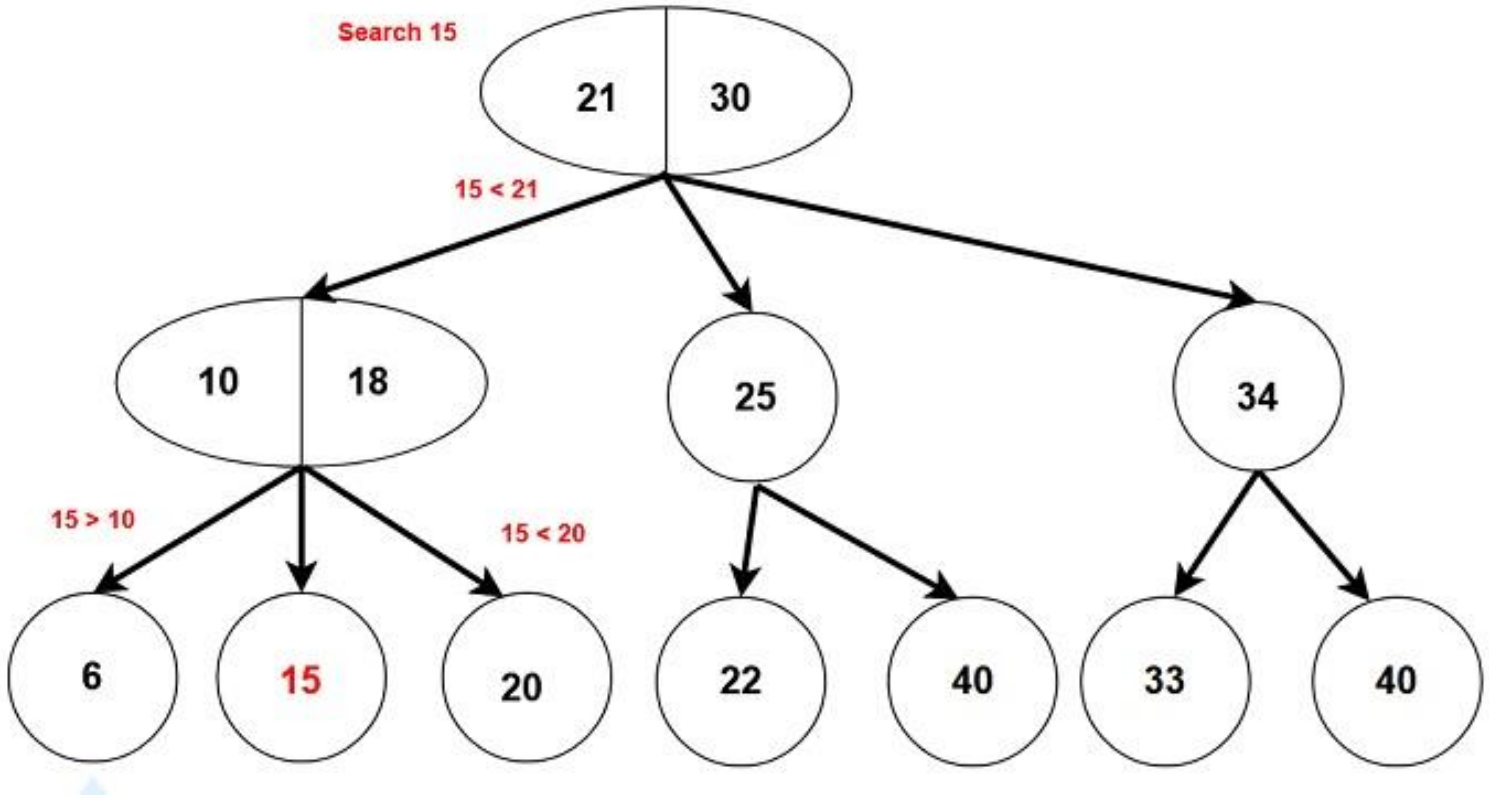
Searching in 2-3 Trees

Algorithm for search:

```
Function Search(node, key):
    If node is NULL:
        Return "Key not found"

    If key == node.key1 OR (node has two keys AND key == node.key2):
        Return "Key found"

    If key < node.key1:
        Return Search(node.left, key)  // Go to left child

    Else if node has two keys AND key > node.key2:
        Return Search(node.right, key) // Go to right child

    Else:
        Return Search(node.middle, key) // Go to middle child
```

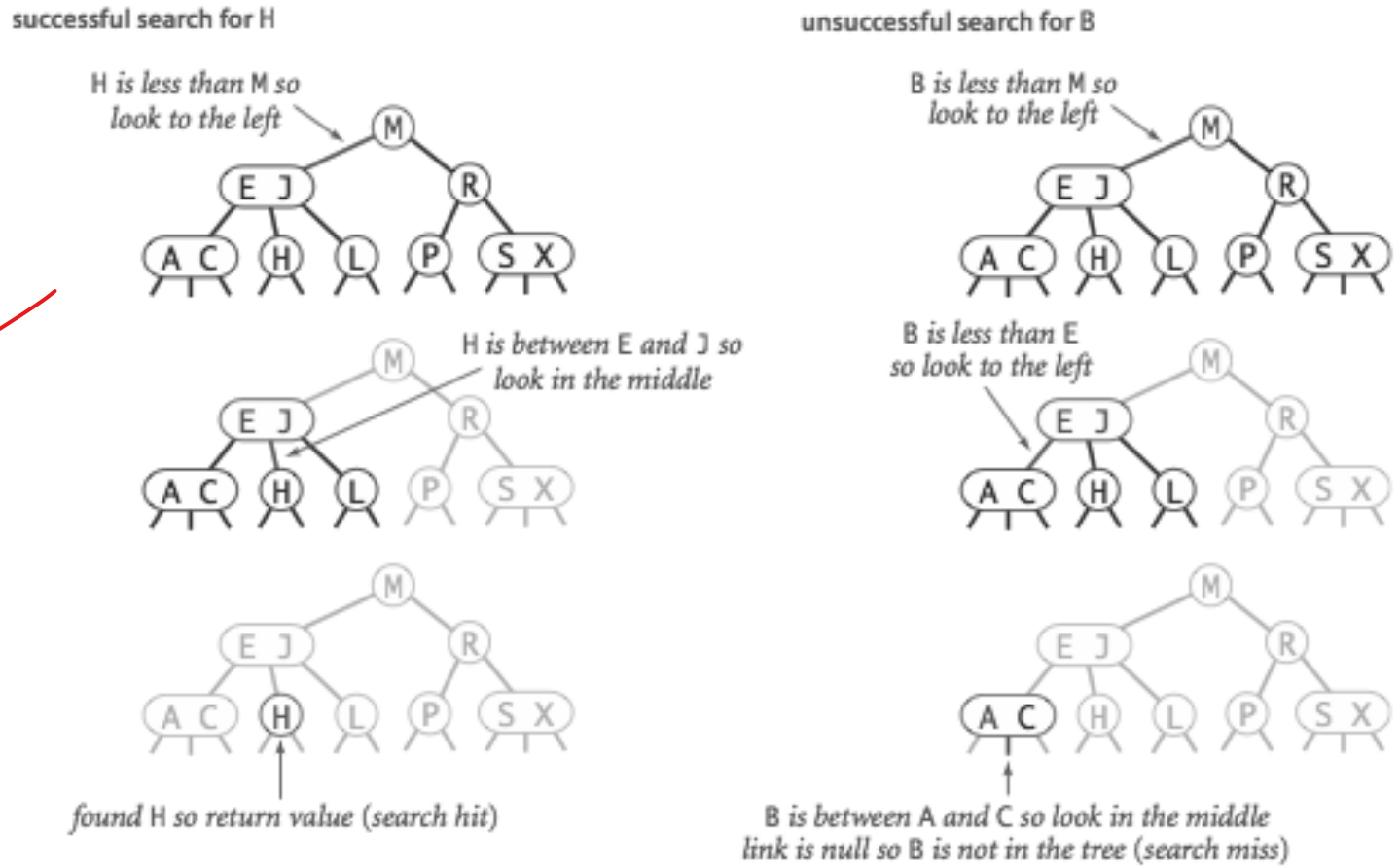
Searching in 2-3 Trees

Let’s Try it out with an example:



Searching in 2-3 Trees

Let's try this one here:

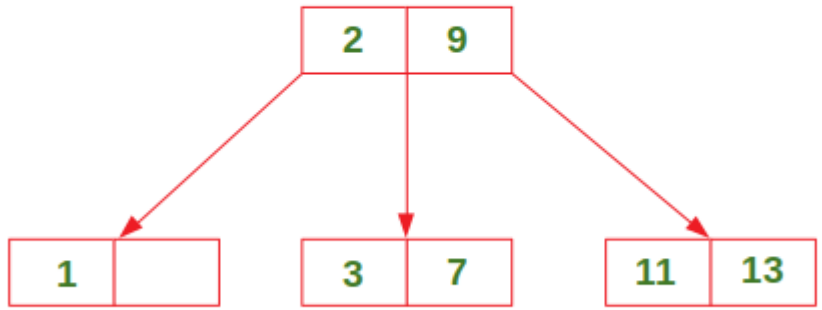


Search hit (left) and search miss (right) in a 2-3 tree

Searching in 2-3 Trees

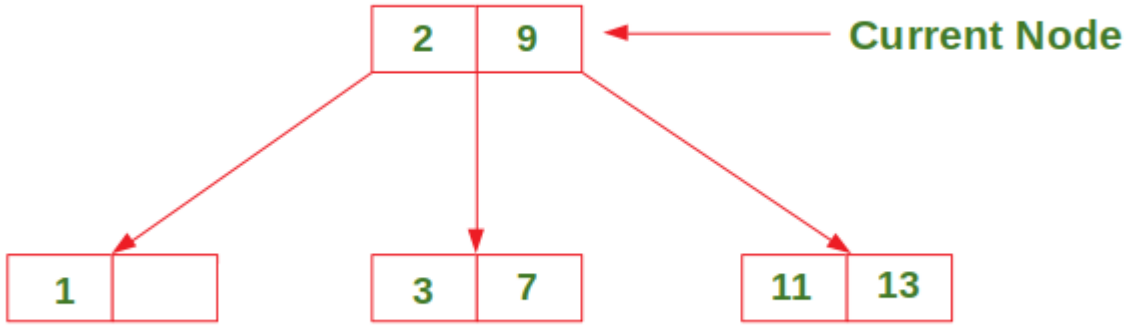
An example of an unsuccessful search:

Search 5 in the following 2-3 Tree:



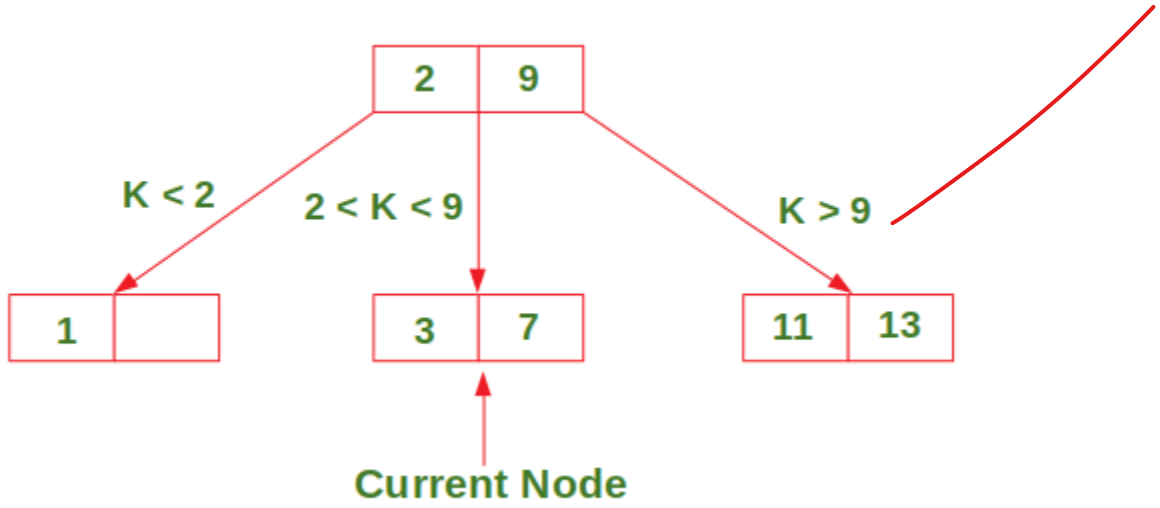
Searching in 2-3 Trees

An example of an unsuccessful search:



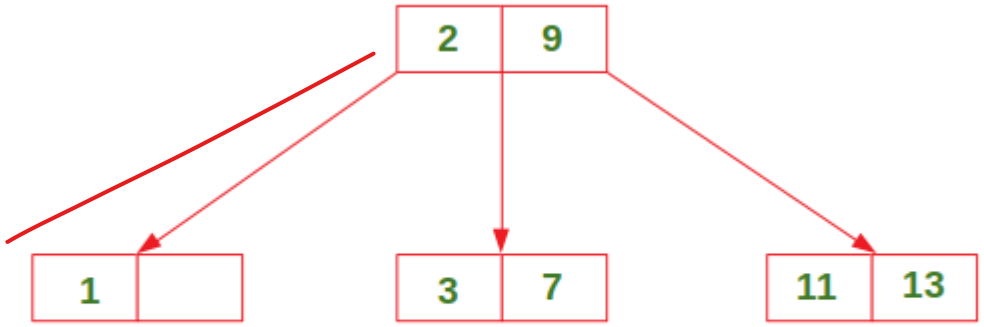
Searching in 2-3 Trees

An example of an unsuccessful search:



Searching in 2-3 Trees

An example of an unsuccessful search:



5 Not Found. Return False

Insertion in 2-3 Trees

- To insert X in a 2-3 tree.
- If the tree is empty → Add X as root.
- Else: Search for the right position of X and add it as a leaf node.
- If there is a leaf node with one data part, add X with and leaf node becomes 2 - Node.
- If the leaf node has two data parts, add X. Split temporary 3-nodes and move data to parent nodes according to the sorting order.

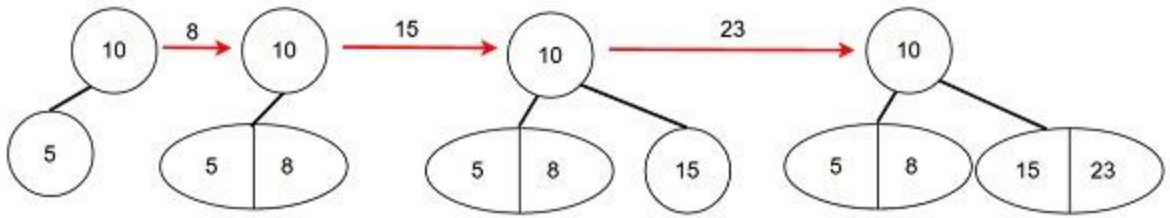
Insertion in 2-3 Trees

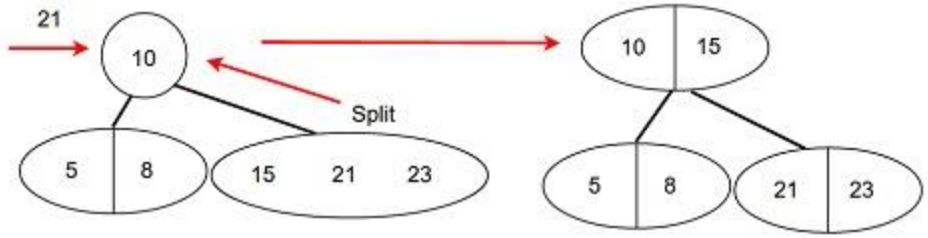
Algorithm for insertion:

```
Function Insert(node, key):
    If node is a leaf:
        Insert key into node in sorted order

        If node now has three keys:
            Return Split(node)
        Else:
            Return node

    If key < node.key1:
        newChild, newKey = Insert(node.left, key)

    Else if node has two keys AND key > node.key2:
        newChild, newKey = Insert(node.right, key)

    Else:
        newChild, newKey = Insert(node.middle, key)

    If newChild is not NULL:
        Insert newKey into node in sorted order
        Attach newChild to the correct position

        If node now has three keys:
            Return Split(node)

    Return node, NULL

Function Split(node):
    Create newNode
    newNode.key1 = node.key2

    If node has children:
        newNode.left = node.middle
        newNode.right = node.right

    Return newNode, node.key1  // Promote middle key
```

Insertion in 2-3 Trees

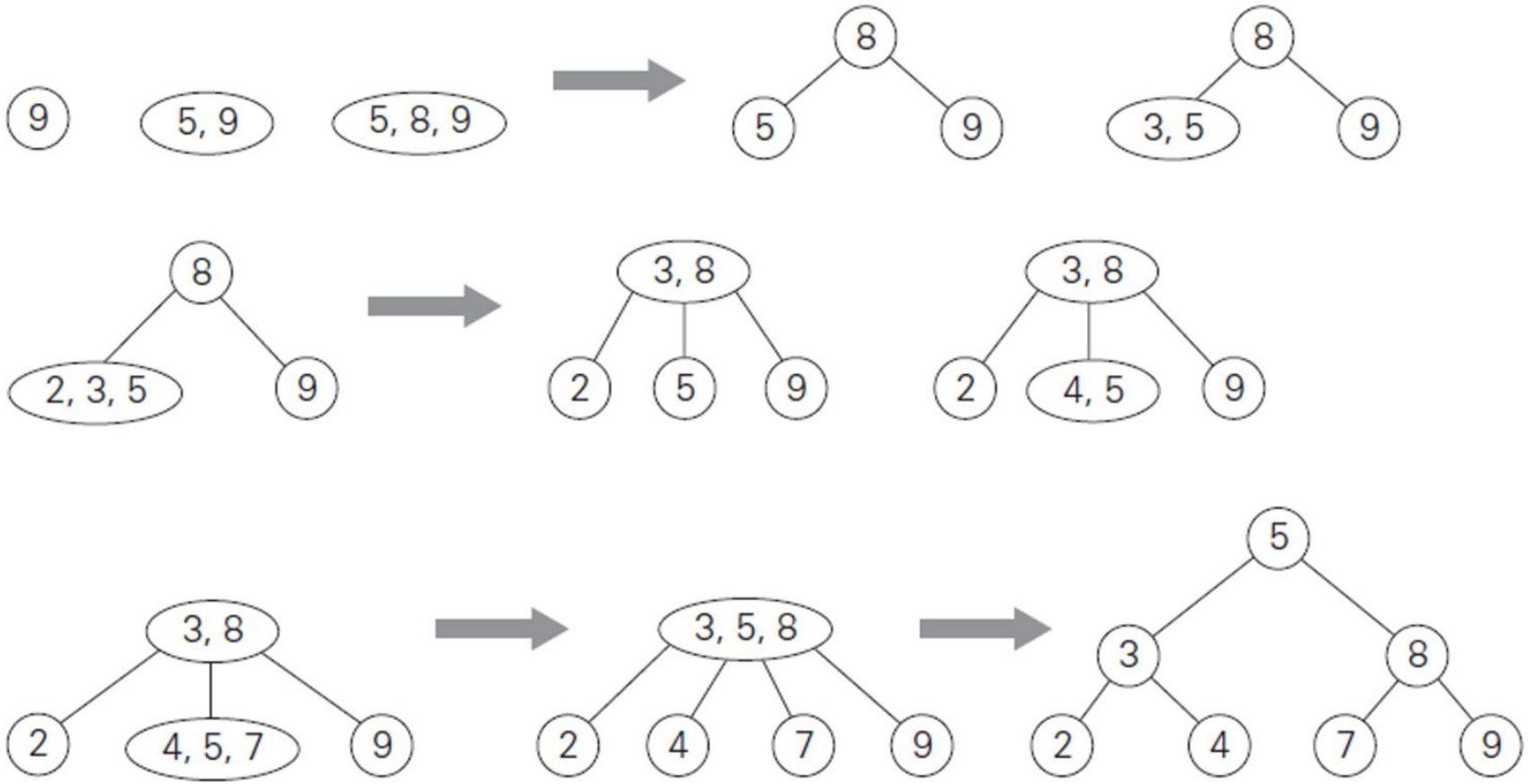
Make 2-3 tree and add nodes in order → 10, 5, 8, 15, 23, 21





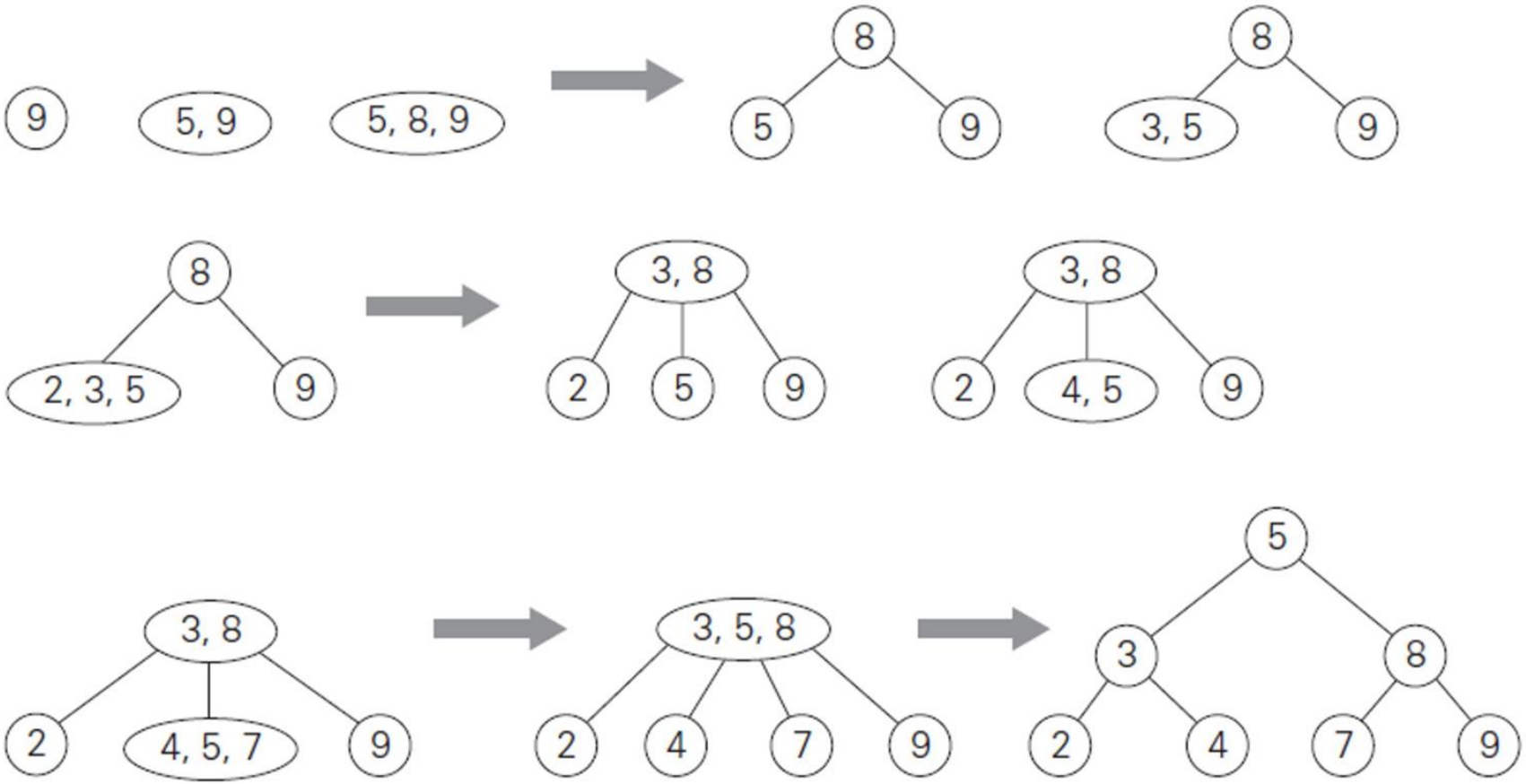
Insertion in 2-3 Trees

Let us insert **9, 5, 8, 3, 2, 4, 7** starting from an empty tree.



Insertion in 2-3 Trees

Let us insert **9, 5, 8, 3, 2, 4, 7** starting from an empty tree.



Deletion in 2-3 Trees

To delete X in a 2-3 tree.

If the tree is empty return false.

Else: Search for the position of X and delete it, then adjust the tree.

If X is part of 3 node delete X and adjust left value and middle value. Also adjust node's ancestor's left and middle value if necessary.

If X is part of 2 node then adjust and split the tree in a recursive manner arranging nodes in sorted order.

Deletion in 2-3 Trees

Algorithm for Delete:

```
Function Delete(node, key):
//Case 1:
    If node is NULL:
        Return NULL
    If key is in a leaf node:
        Remove key from node
        If node has no keys left:
            HandleUnderflow(node)
        Return node
```

Deletion in 2-3 Trees

Algorithm for Delete:

```
Function Delete(node, key):
// Case 2:
    If key is in an internal node:
        inorderSuccessor = FindMin(node.right)
        node.key1 = inorderSuccessor
        Delete(node.right, inorderSuccessor)
    Else if key < node.key1:
        Delete(node.left, key)
    Else if node has two keys AND key > node.key2:
        Delete(node.right, key)
    Else:
        Delete(node.middle, key)
    If node has underflow:
        HandleUnderflow(node)
    Return node
```

Deletion in 2-3 Trees

Algorithm for Delete:

Function Delete(node, key):

//Case 3:

Function HandleUnderflow(node):

    If sibling has extra key:

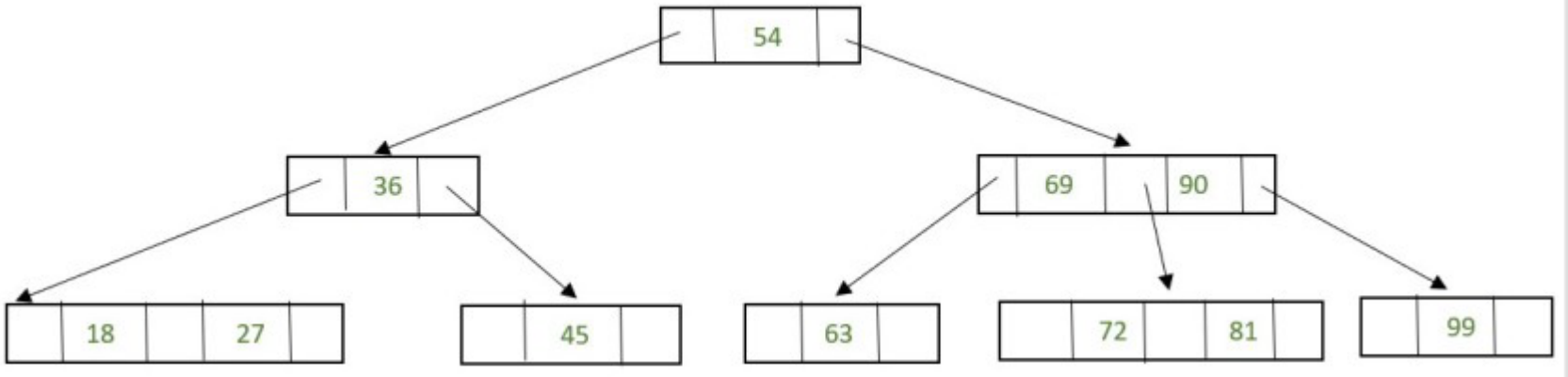
        Borrow key from sibling (Redistribution)

    Else:

        Merge node with sibling

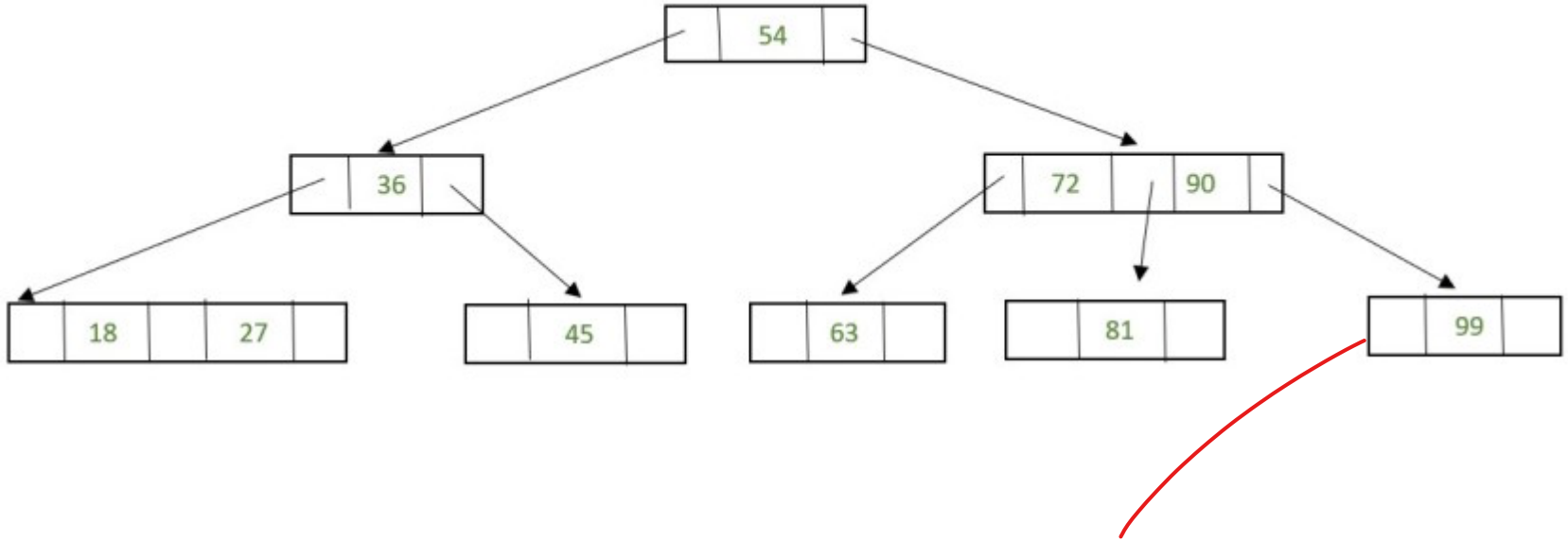
Deletion in 2-3 Trees

Consider the 2-3 tree given below : delete the following values from it: 69,72, 99, 81.



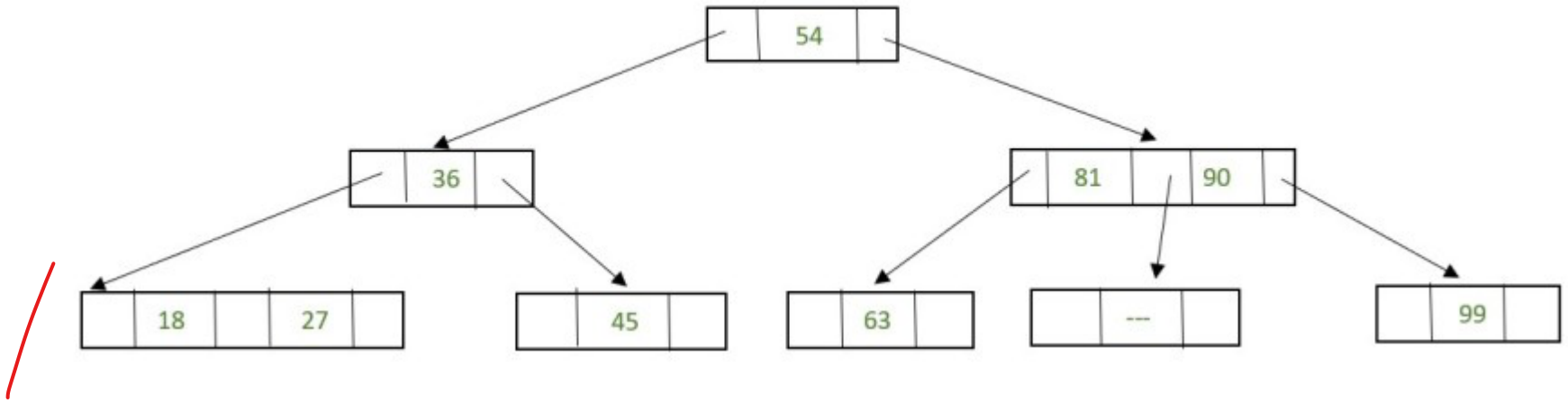
Deletion in 2-3 Trees

To delete 69, swap it with its in-order successor, that is, 72. 69 now comes in the leaf node. Remove the value 69 from the leaf node.



Deletion in 2-3 Trees

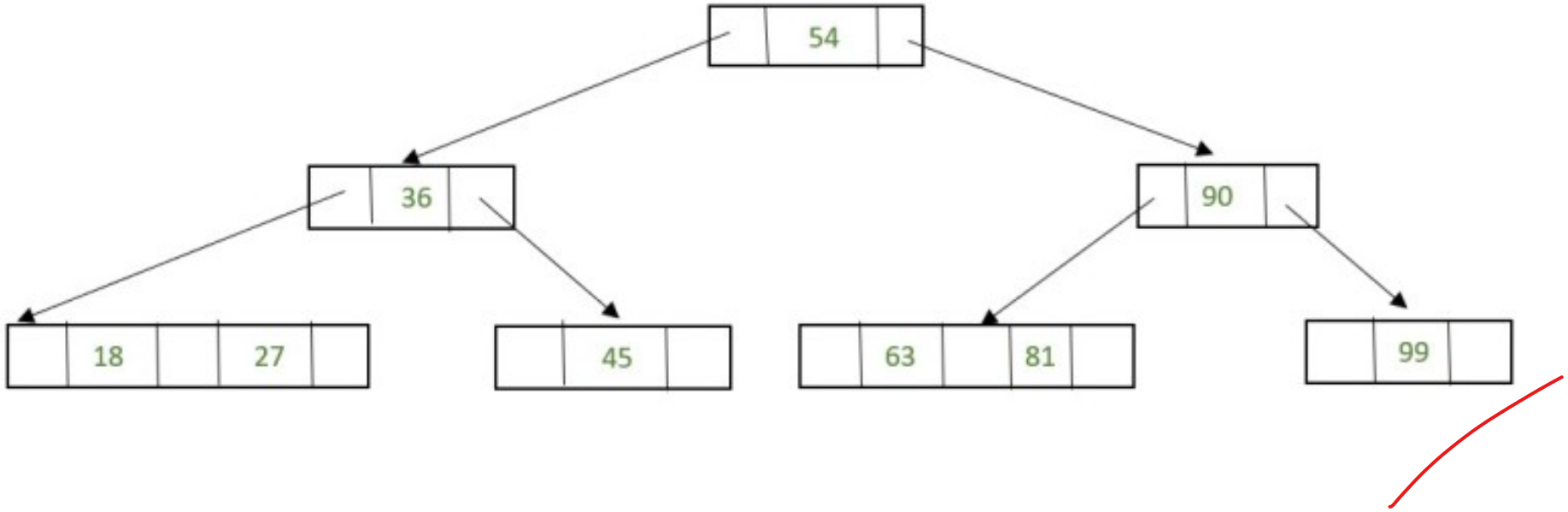
To delete 72, 72 is an internal node. To delete this value swap 72 with its in-order successor 81 so that 72 now becomes a leaf node. Remove the value 72 from the leaf node.



Deletion in 2-3 Trees

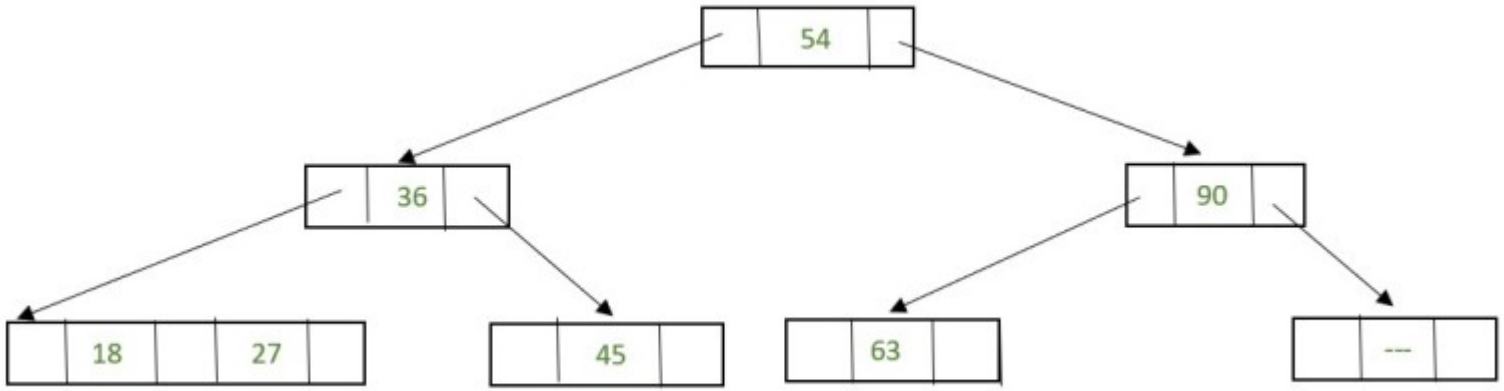
Now there is a leaf node that has less than 1 data value thereby violating the property of a 2-3 tree. So the node must be merged.

To merge the node, pull down the lowest data value in the parent's node and merge it with its left sibling.



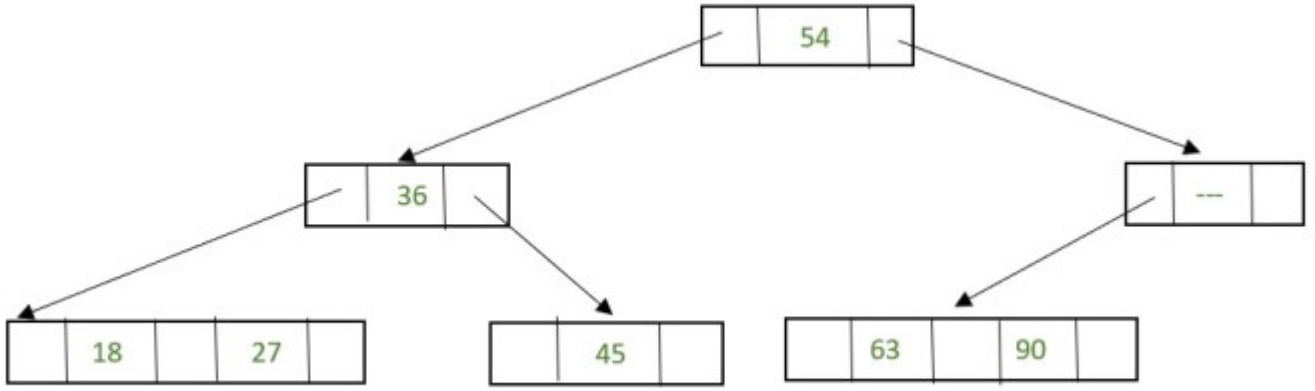
Deletion in 2-3 Trees

To delete 81, 81 is an internal node. To delete this value swap 81 with its in-order successor 90 so that 81 now becomes a leaf node. Remove the value 81 from the leaf node.



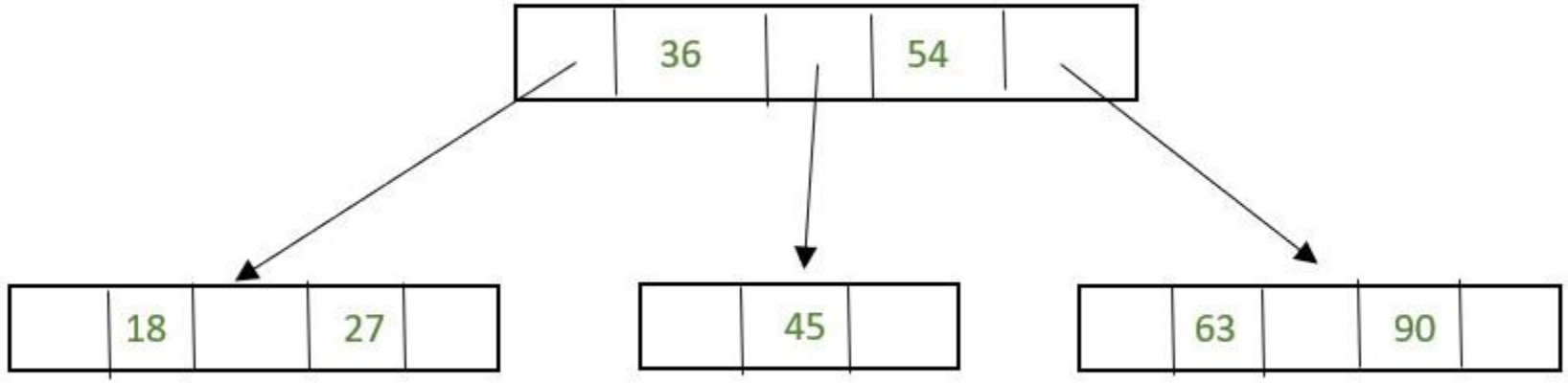
Deletion in 2-3 Trees

Now there is a leaf node that has less than 1 data value, thereby violating the property of a 2-3 tree. So the node must be merged. To merge the node, pull down the lowest data value in the parent's node and merge it with its left sibling.



Deletion in 2-3 Trees

As internal node cannot be empty. So now pull down the lowest data value from the parent's node and merge the empty node with its left sibling.



The Balancing Mechanism

The Key to a 2-3 Trees' efficiency is its ability to a have a search time complexity of O(log(n)).

This efficiency is maintained, as the tree is re-balanced for each insertion or deletion.

Here's how Balance is Maintained:

- Node splitting during insertion.
- Key redistribution or merging during deletion.

Comparison with Other Trees

Feature	Binary Search Tree	AVL Tree	Red-Black Tree	2-3 Tree
Balancing Mechanism	None	Rotations	Rotations	Splitting/Merging
Time Complexity	O(N)	$O(\log N)$	$O(\log N)$	$O(\log N)$
Use Case	Simple applications	In-memory sorting	Complex dynamic sets	Database indexing

Merits and Demerits

Advantages of 2-3 Trees

1. Strictly balanced structure ensures predictable performance.
2. Guaranteed $O(\log N)$ time complexity for search, insert, and delete operations.

Disadvantages of 2-3 Trees

1. More complex to implement compared to simpler structures like BSTs or AVL trees.
2. Overhead in maintaining balance during frequent insertions or deletions.

References

1. https://www.geeksforgeeks.org/2-3-trees-search-and-insert/ (step by step guide for search, insertion and deletion)
2. https://iq.opengenus.org/2-3-trees/ (understanding tree properties)
3. https://www.tutorialspoint.com/2-3-trees-data-structures-and-algorithms-in-cplusplus (illustration)

THANK YOU

Teaching Associate

Varun Bharadwaj

CMSC 420: Lecture 6
2-3 Trees

2-3 Trees: In the previous lecture, we presented one way to establish balance in a binary search tree, namely through the AVL tree's height-balance condition. Today, we will explore an alternative approach which is achieved by allowing nodes to variable "widths." In particular, we will allow internal nodes to have either two or three children (see Fig. 1(a) and (b)). When a node has three children, it stores two keys. Given the two key values b and d, the three subtrees A, C, and E must satisfy the *generalized inorder property* that

$$A < b < C < d < E,$$

These are called *2-nodes* and *3-nodes*, respectively.

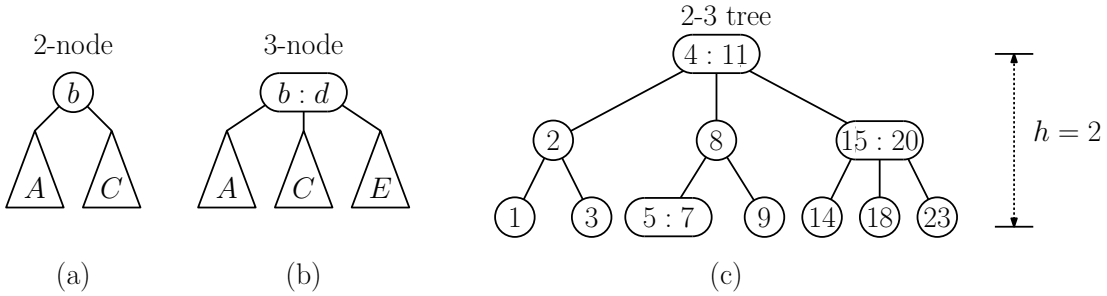


Fig. 1: (a) 2-node, (b) 3-node, and (c) a possible 2-3 tree of height 2.

A *2-3 tree* of height $h \geq -1$ is defined recursively as follows (see Fig. 1(c)). It is either:

- empty (i.e., `null`) when $h = -1$, or
- its root is a 2-node, and its two subtrees are each 2-3 trees of heights $h - 1$, or
- its root is a 3-node, and its three subtrees are each 2-3 trees of heights $h - 1$.

Since all the leaves are at the same level, and the sparsest possible 2-3 tree is a complete binary tree, which implies the following:

Theorem: A 2-3 tree with n nodes has height $O(\log n)$.

It is easy to see how to perform the operation `find(x)` in a 2-3 tree. We apply the usual recursive descent as for a standard binary tree, but whenever we come to a 3-node, we need to check the relationship between x and the two key values in this node in order to decide which of the three subtrees to visit. The important issues are insertion and deletion, which we discuss next.

For conceptual purposes, it will be convenient to temporarily allow for the existence of "invalid" *1-nodes* and *4-nodes*. As soon as one of these exceptional nodes comes into existence, we will take action to replace them with proper 2-nodes and 3-nodes.

Insertion into a 2-3 tree: The insertion procedure follows the general structure that we have established with prior binary search trees. We first search for the insertion key, and make a note of the last node we visited just before falling out of the tree. Because all leaf nodes are at the same level, we always fall out at the lowest level of the tree. We insert the new key

into this leaf node. If the node was a 2-node, it now becomes a 3-node, and we are fine. If it was a 3-node, it now becomes a 4-node, and we need to fix it.

While the initial insertion takes place at a leaf node, we will see that the restructuring process can propagate to internal nodes. So, let us consider how to remedy the problem of a 4-node in a general setting. A 4-node has three keys, say b, d, and f and four children, say A, C, E, and G (see Fig. 2(a)).

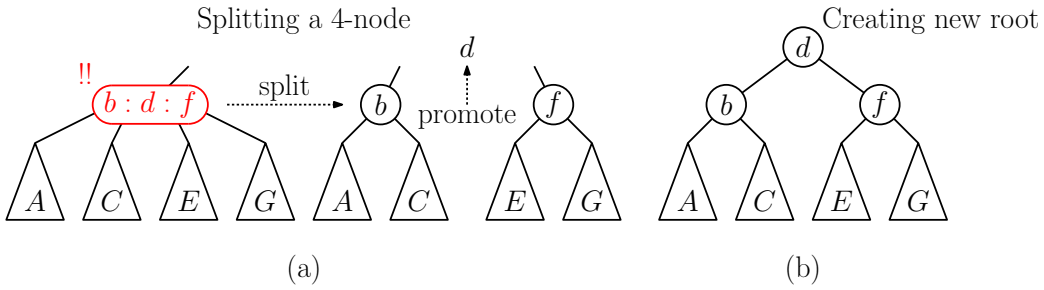


Fig. 2: 2-3 tree insertion: (a) splitting a 4-node into two 2-nodes and (b) creating a new root.

To resolve the problem we *split* this node into two 2-nodes: one for b with A and C as subtrees, and the other for f with E and G as subtrees. We then *promote* the middle key d by inserting it (recursively) into the parent node. What if there is no parent, because the current node is the root? We create a new root node storing d whose two children are b and f (see Fig. 2(b)). It is easy to check that this process preserves the 2-3 tree structural properties, and so we won't bother giving a proof.

In the figure below, we present an example of the result of inserting key 6 into a 2-3 tree. It requires two splits to restore the tree's structure.

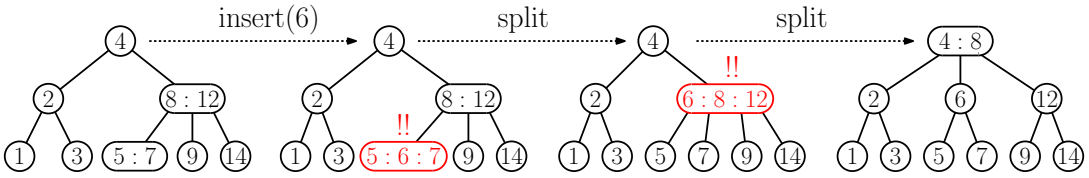


Fig. 3: 2-3 tree insertion involving two splits.

Deletion from a 2-3 tree: Consistent with our experience with binary search trees, deletion is more complicated than insertion. The general process follows the usual pattern. First, we find the key to be deleted. If it does not appear in a leaf node, then we identify a replacement key as the inorder successor. (The inorder predecessor would work equally well.) We copy the replacement key-value pair to replace the deleted key entry, and then we recursively delete the replacement key from its subtree. In this manner, we can always assume that we are deleting a key from a leaf node. So, let us focus on this.

As you might imagine, since insertion resulted in an overfull 4-node being split into two 2-nodes, the process of deletion will involve merging "underfull" nodes. This is indeed the case, but it will also be necessary to consider another restructuring primitive in which keys are taken or "adopted" from a sibling. This adoption process is also called *key rotation*, and I will use both terms interchangeably.

More formally, let us consider the deletion of an arbitrary key from an arbitrary node in the tree. If this is a 3-node, then the deletion results in a 2-node, which is fine. However, if this is a 2-node, the deletion results in an illegal 1-node, which has one subtree and zero keys. We remedy the situation in one of two ways.

Adoption (Key Rotation): Consider the left and right siblings of this node (if they exist). If either sibling is a 3-node, then it gives up an appropriate key (and subtree) to convert us back to 2-node. The tree is now properly structured.

Suppose, for the sake of illustration that the current node is an underfull 1-node, and it has a right sibling that is a 3-node (see Fig. 4(a)). Then, we adopt the leftmost key and subtree from this sibling, resulting in two 2-nodes. (A convenient mnemonic is the equation $1 + 3 = 2 + 2$.)

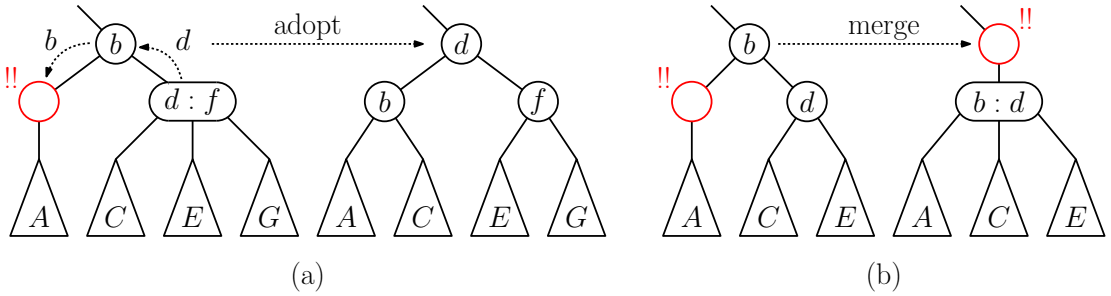


Fig. 4: 2-3 tree deletion: (a) adopting a key/subtree from a sibling and (b) merging two nodes.

Merging: On the other hand, if neither sibling can offer a key, then it follows that at least one sibling is a 2-node. Suppose for the sake of illustration that it is the right sibling (see Fig. 4(b)). In this case, we merge the 1-node with this 2-node to form a 3-node. (A convenient mnemonic is the equation $1 + 2 = 3$.)

But, we need a key to complete this 3-node. We take this key from our parent. If the parent was a 3-node, it now becomes a 2-node, and we are fine. If the parent was a 2-node, it has now become a 1-node (as in the figure), and the restructuring process continues on up the tree. Finally, if we ever arrive at a situation where the 1-node is the root of the tree, we remove this root and make its only child the new root.

An example of the deletion of a key is shown in Fig. 5. In this case, the initial deletion was from a 2-node, which left it as a 1-node. We merged it with its sibling to form a 3-node ($1 + 2 = 3$). This involved demoting the key 6 from the parent, which caused the parent to decrease from a 2-node to a 1-node. Since the parent has a 3-node sibling, we can adopt from it. (By the way, there is also a sibling which is a 2-node, containing the key 2. Could we have instead merged with this node? The answer is "yes", but it is not in our interest to do this. This is because merging results in more disruptions to ancestors of the tree, whereas a single adoption terminates the restructuring process.)

In Fig. 6 we illustrate two more examples of deletions from a 2-3 tree. In the upper example, we delete the key 3. Since its only sibling is a 2-node, we cannot perform an adoption, so we merge with the parent. (You might wonder, can't we adopt from our cousin, the $(6 : 7)$ node? The answer is "no." According to the 2-3 deletion algorithm, you can only adopt from your siblings. If we were to allow adoptions from arbitrarily distant cousins, the adoption process would take more than $O(1)$ time.) The parent node is now critical, and so we merge with its parent.

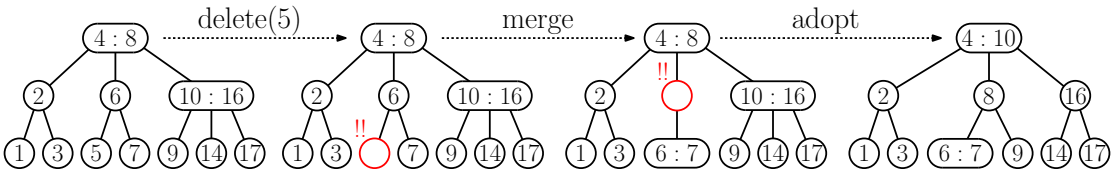


Fig. 5: 2-3 tree deletion involving a merge and an adoption.

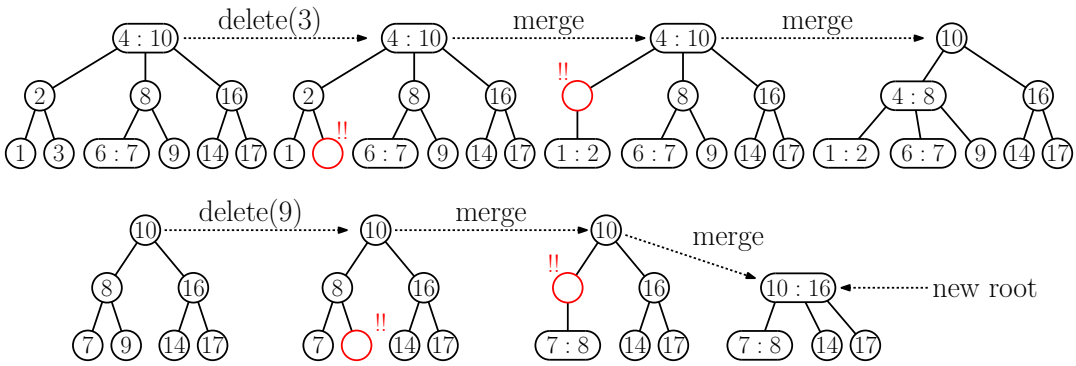


Fig. 6: Two more examples of 2-3 tree deletions.

In the second example in Fig. 6, we illustrate a case where the root node becomes critical. In this case, we remove the old root, and make its only child the new root.

Implementation? We will not discuss the 2-3 tree implementation in detail, since we will be presenting a number of related data structures, including red-black trees, AA trees, and B-trees in later lectures. The fact that there are two types of nodes might suggest using two different node classes (e.g., `TwoNode` and `ThreeNode`, which are both subclasses of a generic node class). However, because we often convert between one node type and the other (changing a 2-node into a 3-node and vice versa), it may be preferred to have just one node type that is capable of representing both, and adjusting the number of children/keys dynamically. An example is shown in the following code fragment.

2-3 Tree Node

```
class TwoThreeNode {
    int             nChildren;      // number of children (2 or 3)
    TwoThreeNode    child[3];       // children pointers
    Key             key[2];         // keys
    Value           value[2];       // values
}
```

In our examples, we temporarily created a 4-node, which we immediately split into two 2-nodes. This is really just a conceptual trick. In reality, a 4-node would never be generated. Instead, as soon as you see that you are about to create a 4-node, you would immediately invoke a split procedure to remedy the situation.

Another tricky implementation issue is that the current node will need to access its siblings on the left and right sides. To make this operation easier, it may be desirable to add a parent link to our node structure. Alternatively, the various recursive functions can be passed two node pointers as arguments, one to the current node and the other to its parent.

DESIGN AND ANALYSIS OF ALGORITHMS

Surabhi Narayan

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

2-3 Trees

Surabhi Narayan
Department of Computer Science & Engineering

2-3 Tree

2-3 tree is a tree that can have nodes of two kinds: 2-nodes and 3-nodes.

- A ***2-node*** contains a single key*K* and has two children:
  the left child serves as the root of a subtree whose keys are less than *K*, and the right child serves as the root of a subtree whose keys are greater than *K*.

- A ***3-node*** contains two ordered keys *K*1 and *K*2 (*K*1<*K*2) and has three children.

- The leftmost child serves as the root of a subtree with keys less than *K*1, the middle child serves as the root of a subtree with keys between *K*1 and *K*2, and the rightmost child serves as the root of a subtree with keys greater than *K*2
- The last requirement of the 2-3 tree is that all its leaves must be on the same level. In other words, a 2-3 tree is always perfectly height-balanced: the length of a path from the root to a leaf is the same for every leaf.

2-3 Tree

Here are the properties of a 2-3 tree:

- each node has either one value or two value
- a node with one value is either a leaf node or has exactly two children (non-null). Values in left subtree < value in node < values in right subtree
- a node with two values is either a leaf node or has exactly three children (non-null). Values in left subtree < first value in node < values in middle subtree < second value in node < value in right subtree.
- all leaf nodes are at the same level of the tree

2-3 Tree

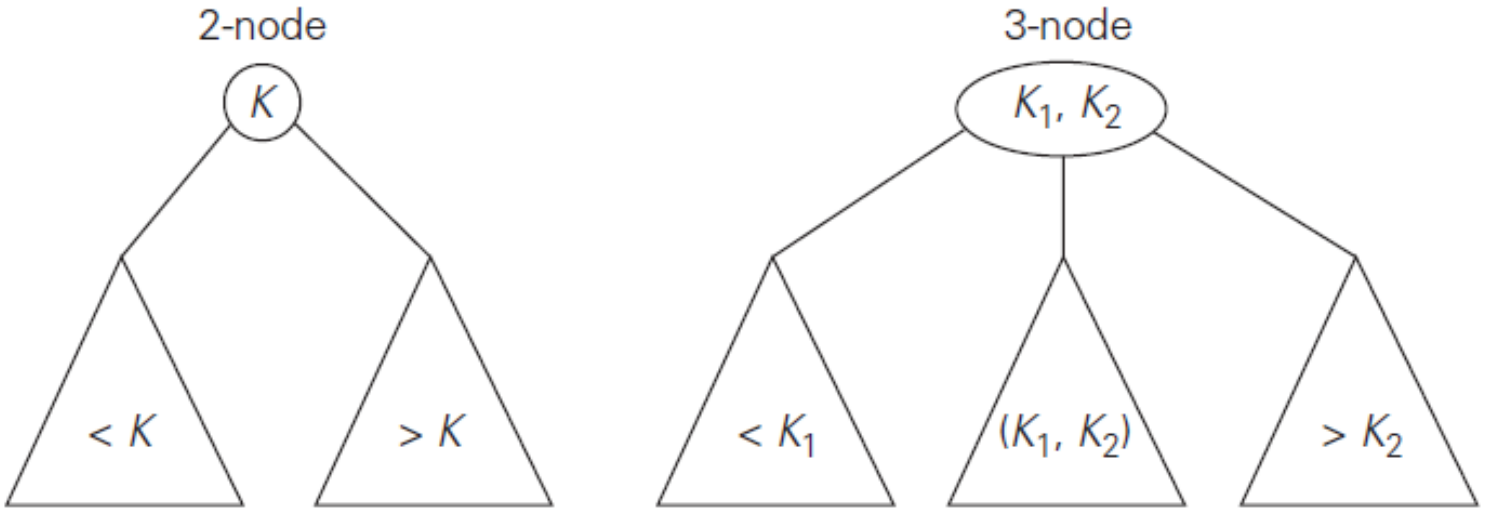


FIGURE 6.7 Two kinds of nodes of a 2-3 tree.

2-3 Tree

Search: To search a key **K** in given 2-3 tree **T**, we follow the following procedure:

Base cases:

1. If **T** is empty, return False (key cannot be found in the tree).
2. If current node contains data value which is equal to **K**, return True.
3. If we reach the leaf-node and it doesn't contain the required key value **K**, return False.

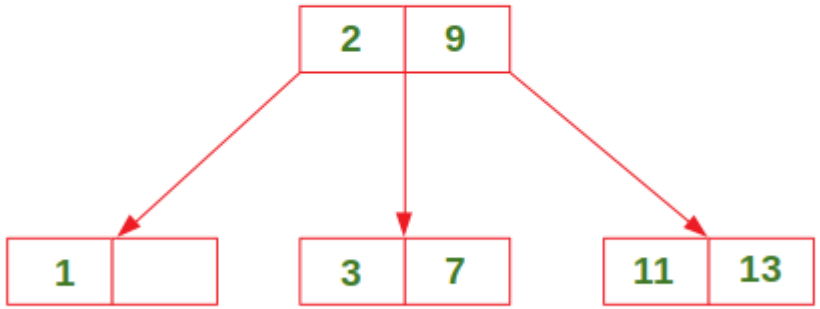
Recursive Calls:

1. If **K** < currentNode.leftVal, we explore the left subtree of the current node.
2. Else if currentNode.leftVal < **K** < currentNode.rightVal, we explore the middle subtree of the current node.
3. Else if **K** > currentNode.rightVal, we explore the right subtree of the current node.

2-3 Tree

PES UNIVERSITY ONLINE

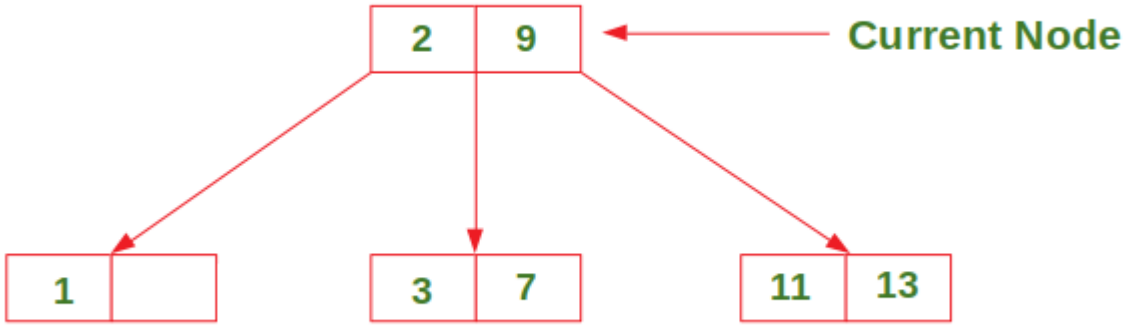
Search 5 in the following 2-3 Tree:



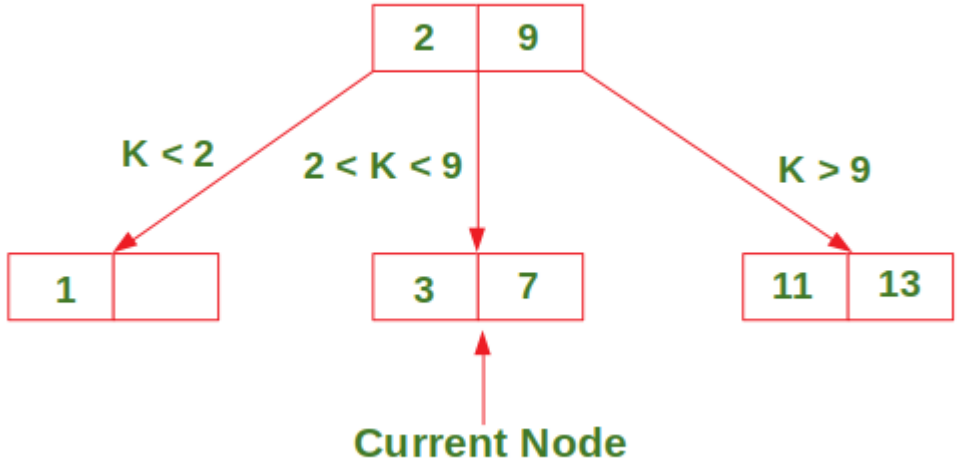
2-3 Tree

PES UNIVERSITY ONLINE

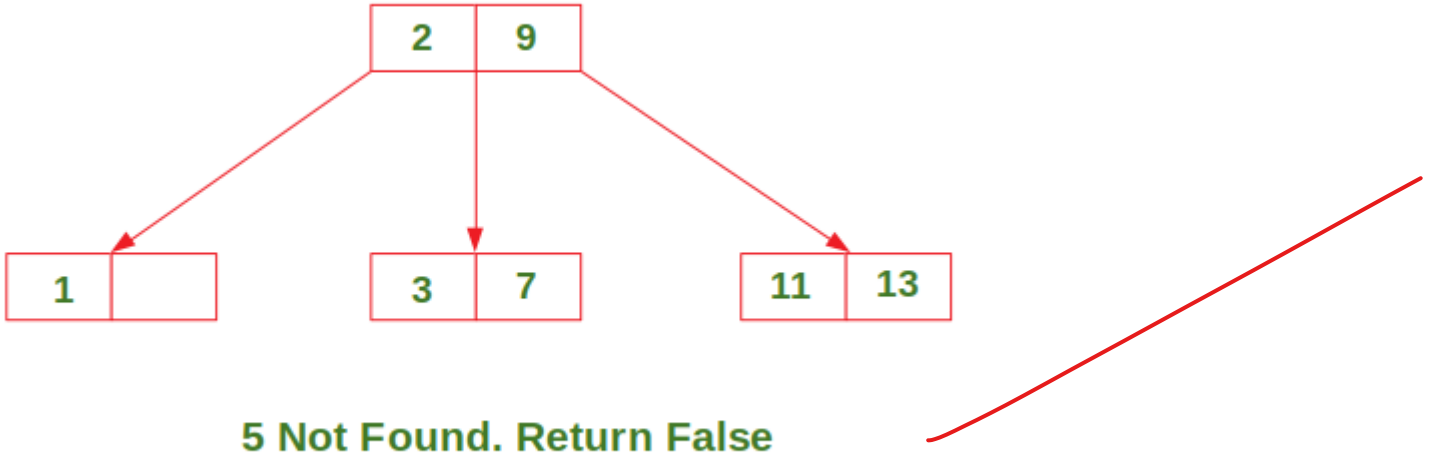
Search 5



Search 5



Search 5



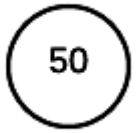
Insertion

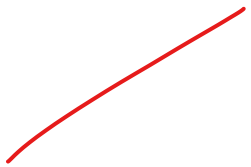
The insertion algorithm into a two-three tree is quite different from the insertion algorithm into a binary search tree. In a two-three tree, the algorithm will be as follows:

- If the tree is empty, create a node and put value into the node
- Otherwise find the leaf node where the value belongs.
- If the leaf node has only one value, put the new value into the node
- If the leaf node has more than two values, split the node and promote the median of the three values to parent.
- If the parent then has three values, continue to split and promote, forming a new root node if necessary

2-3 Tree

Insert 50



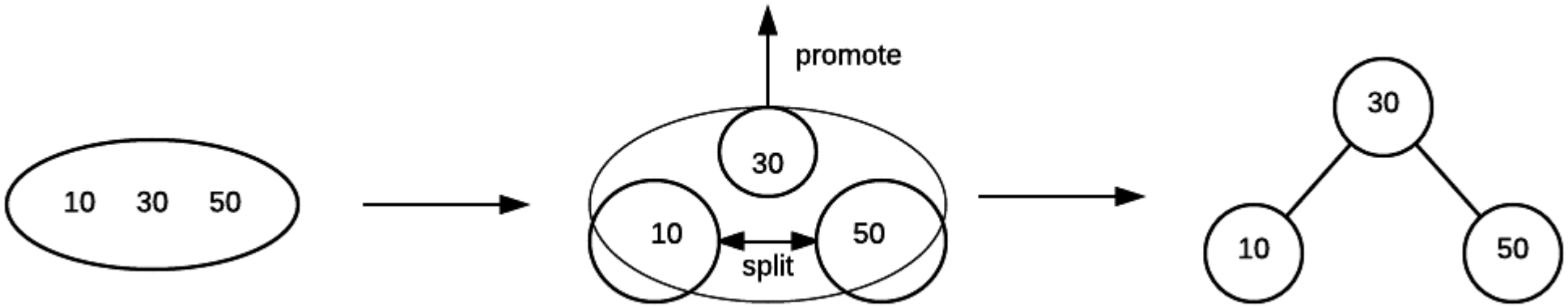


2-3 Tree

Insert 30

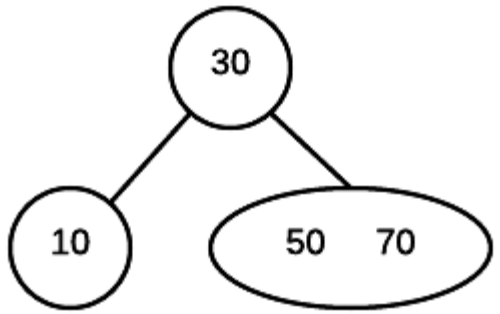
2-3 Tree

Insert 10



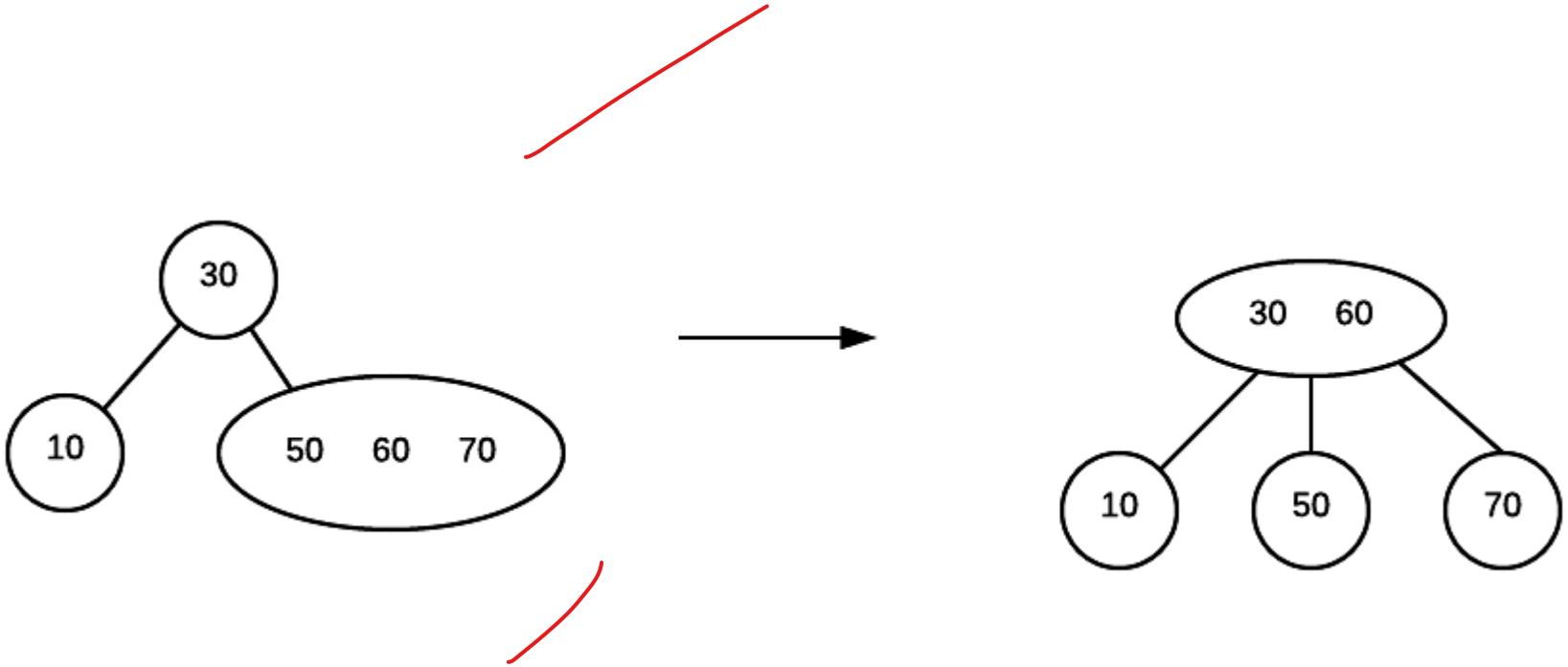
2-3 Tree

Insert 70



2-3 Tree

Insert 60



2-3 Tree

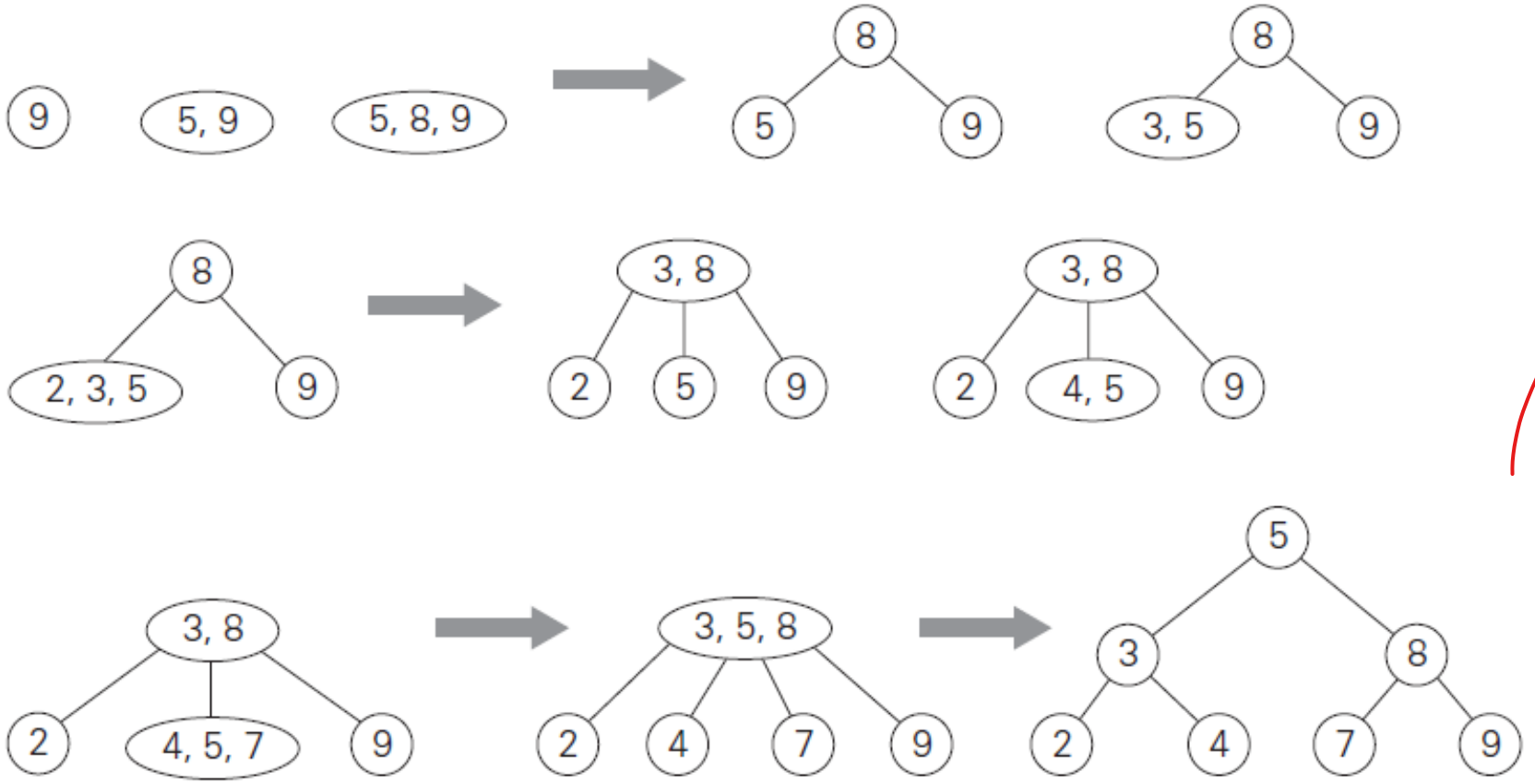


FIGURE 6.8 Construction of a 2-3 tree for the list 9, 5, 8, 3, 2, 4, 7.

Time Complexity

The property of being perfectly balanced, enables the **2-3 Tree** operations of insert, delete and search to have a time complexity of O(log (n)).

THANK YOU

Surabhi Narayan
Department of Computer Science &Engineering

surabhinarayan@pes.edu

DESIGN AND ANALYSIS OF ALGORITHMS

Unit 3: B Trees

Dr. Divyashree N
Department of Computer Science & Engineering

PES UNIVERSITY
CELEBRATING 50 YEARS

DESIGN AND ANALYSIS OF ALGORITHMS

B Trees

DESIGN AND ANALYSIS OF ALGORITHMS

B Tree

- Self-balancing search tree
- Each node can contain more than one key
- Can have more than two children.
- All data records (or record keys) are stored at the leaves, in increasing order of the keys each parental node contains $m - 1$ ordered keys.

It is also known as a height-balanced m-way tree.

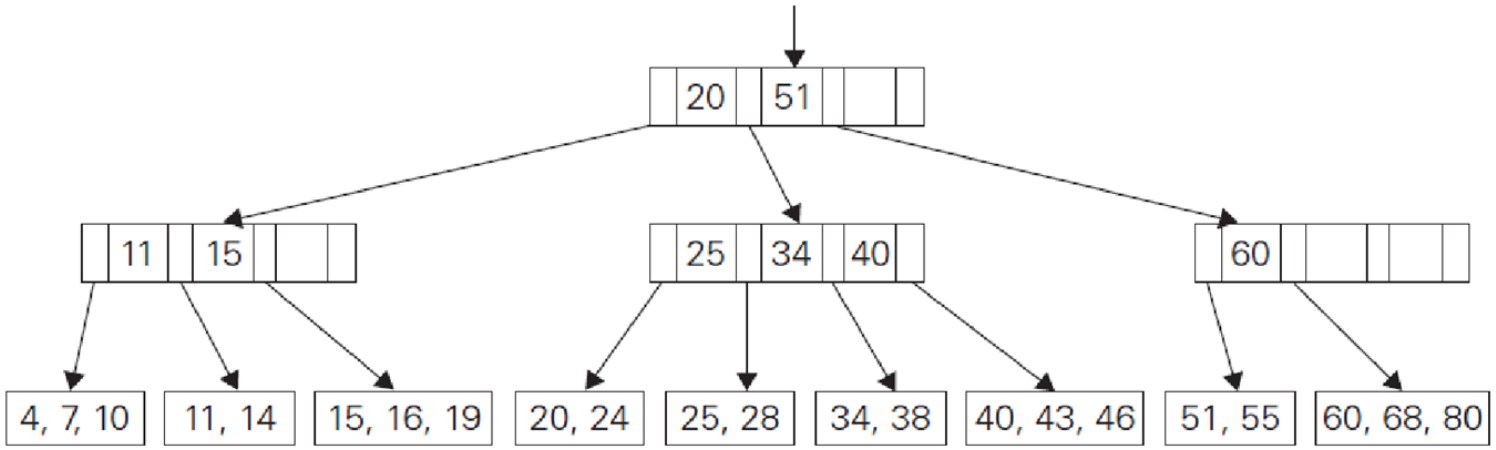


FIGURE 7.8 Example of a B-tree of order 4.

B Tree

Structural properties:

- All leaves are at the same level.
- For each node x, the keys are stored in increasing order.
- If n is the order of the tree, each internal node can contain at most m - 1 keys along with a pointer to each child.
- Each node except root can have at most n children and at least m/2 children.
- All leaves have the same depth (i.e. height-h of the tree).
- The root has at least 2 children and contains a minimum of 1 key.
- If $m \geq 1$, then for any n-key B-tree of height h and minimum degree $t \geq 2$, $h \geq \log_t (m+1)/2$
- B-Tree grows and shrinks from the root which is unlike Binary Search Tree. Binary Search Trees grow downward and also shrink from downward.
- Like other balanced Binary Search Trees, time complexity to search, insert and delete is O(log n).

PES UNIVERSITY
CELEBRATING 50 YEARS

B Tree

Operations on a B Tree:

- Searching
- Insertion
- Deletion

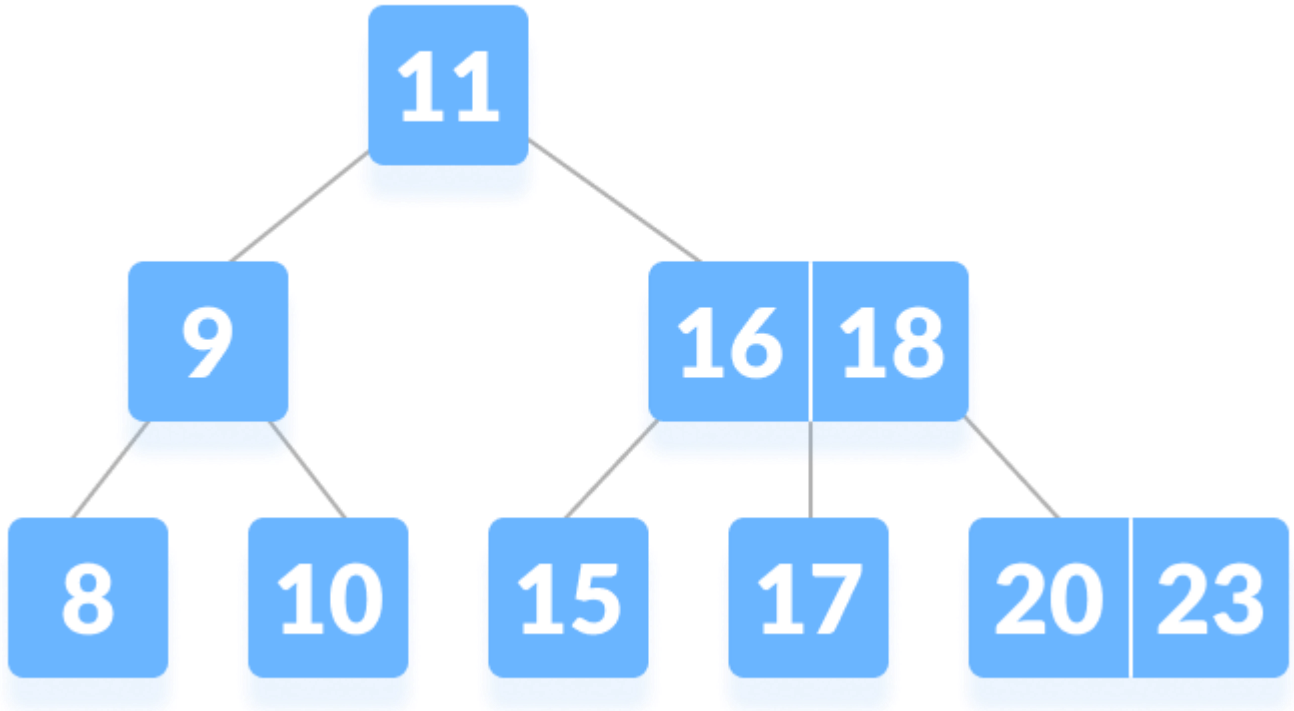
PES UNIVERSITY
CELEBRATING 50 YEARS

Search

1. Starting from the root node, compare k with the first key of the node.
2. If k = the first key of the node, return the node and the index
3. If k.leaf = true, return NULL, i.e. not found
4. If k < the first key of the root node, search the left child of this key recursively.
5. If there is more than one key in the current node and k > the first key compare k with the next key in the node.
6. If k < next key search the left child of this key (ie. k lies in between the first and the second keys). Else, search the right child of the key
7. Repeat steps 1 to 4 until the leaf is reached.

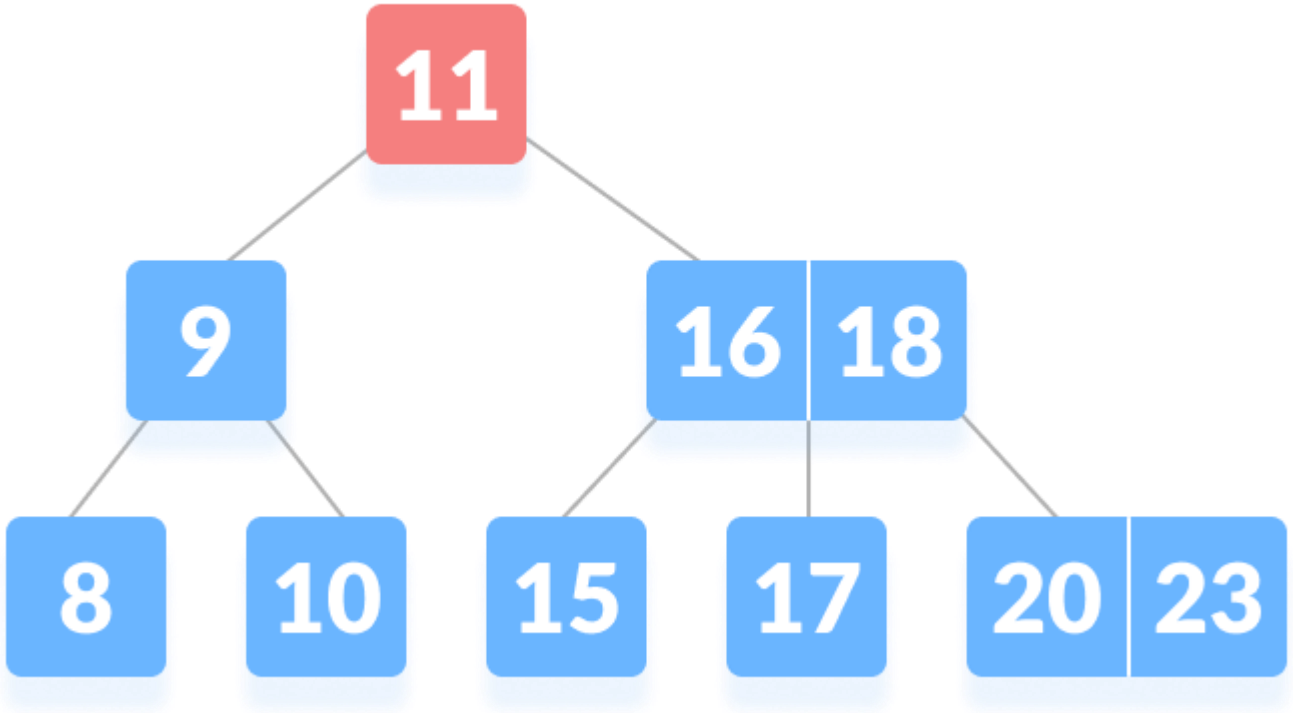
B Tree

Let us search key , k = 17



B Tree

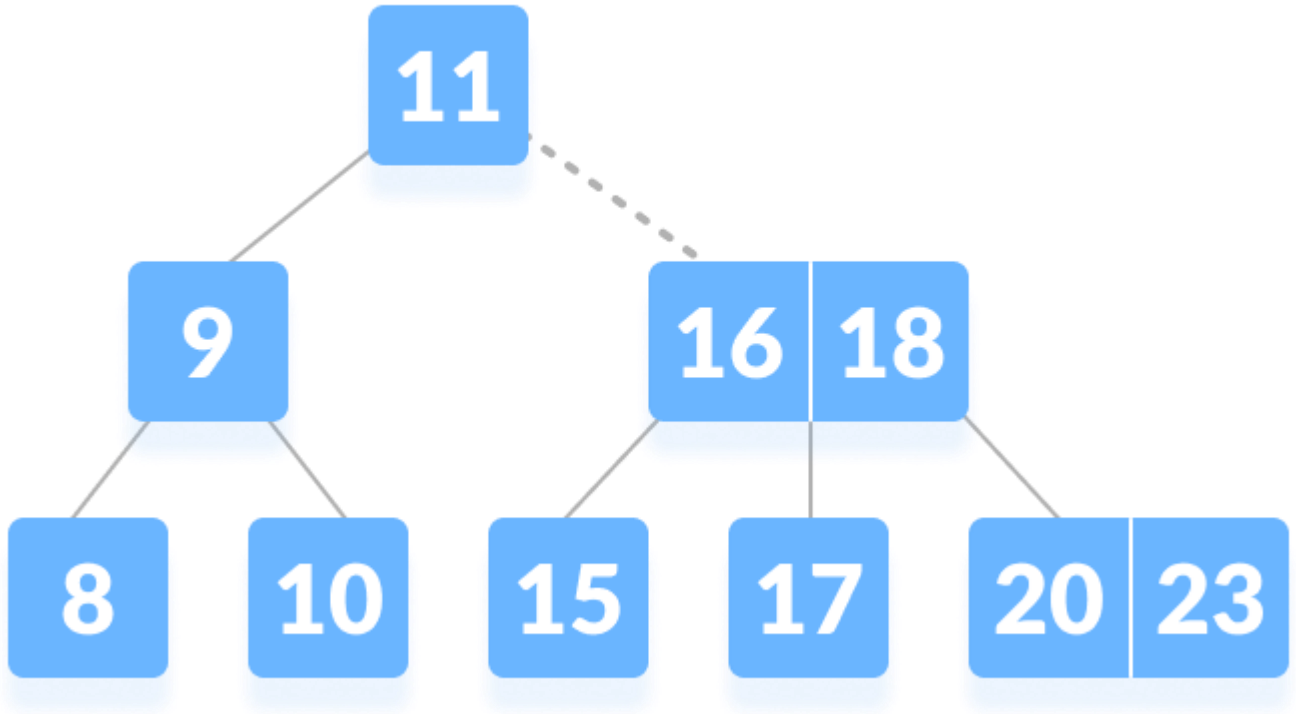
17 is not found in the root so, compare it with the root key



PES UNIVERSITY
CELEBRATING 50 YEARS

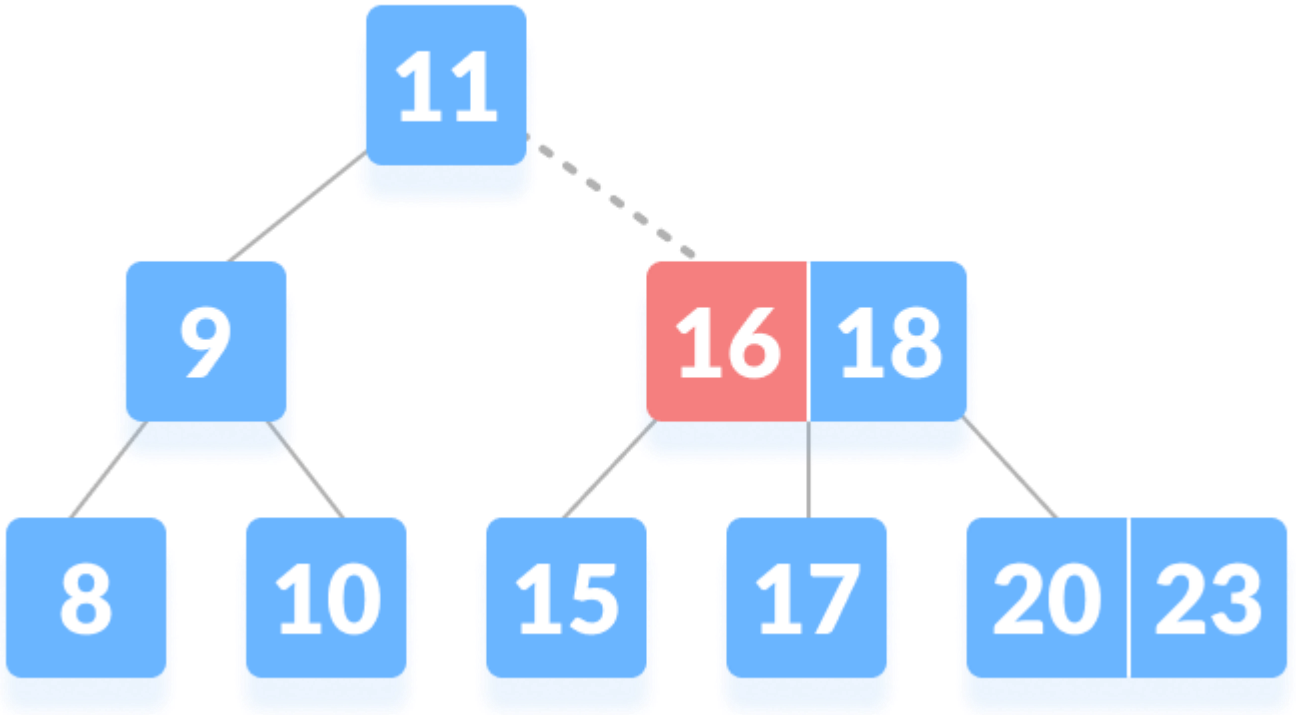
B Tree

Since k > 11 go to the right child of the root node.



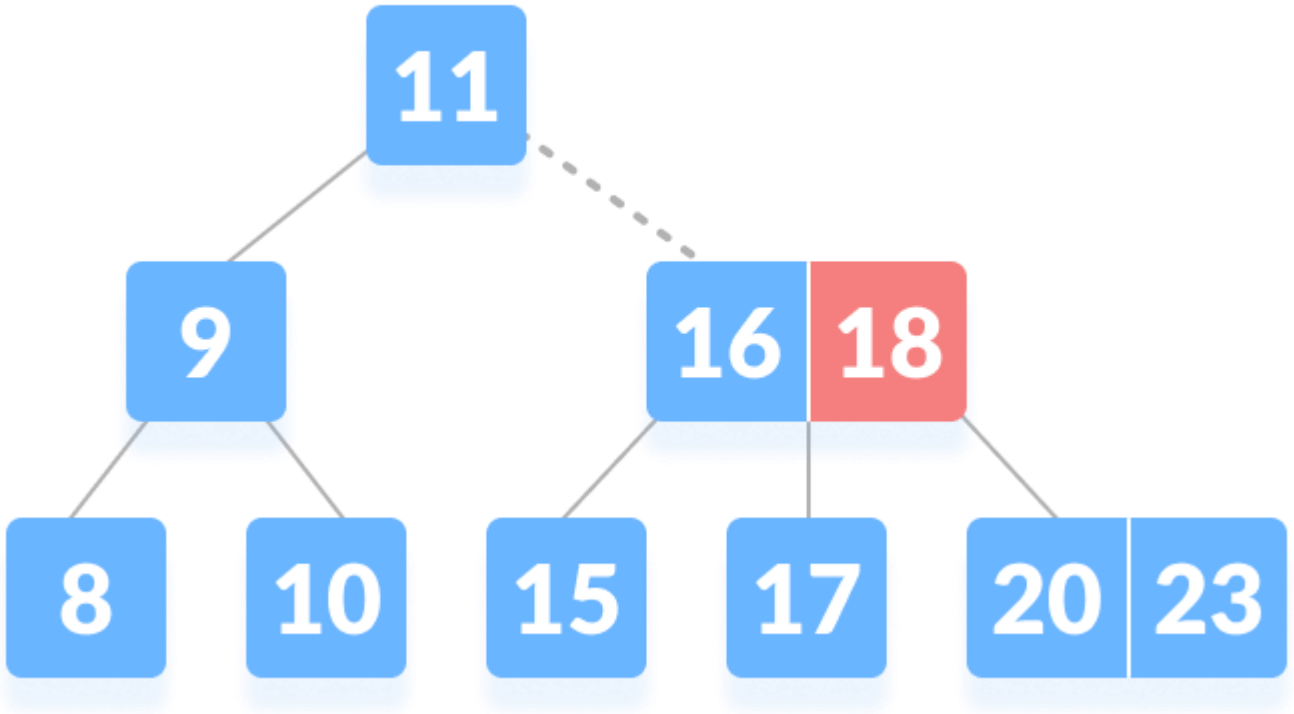
B Tree

Compare k with 16. Since k > 16 compare k with the next key 18



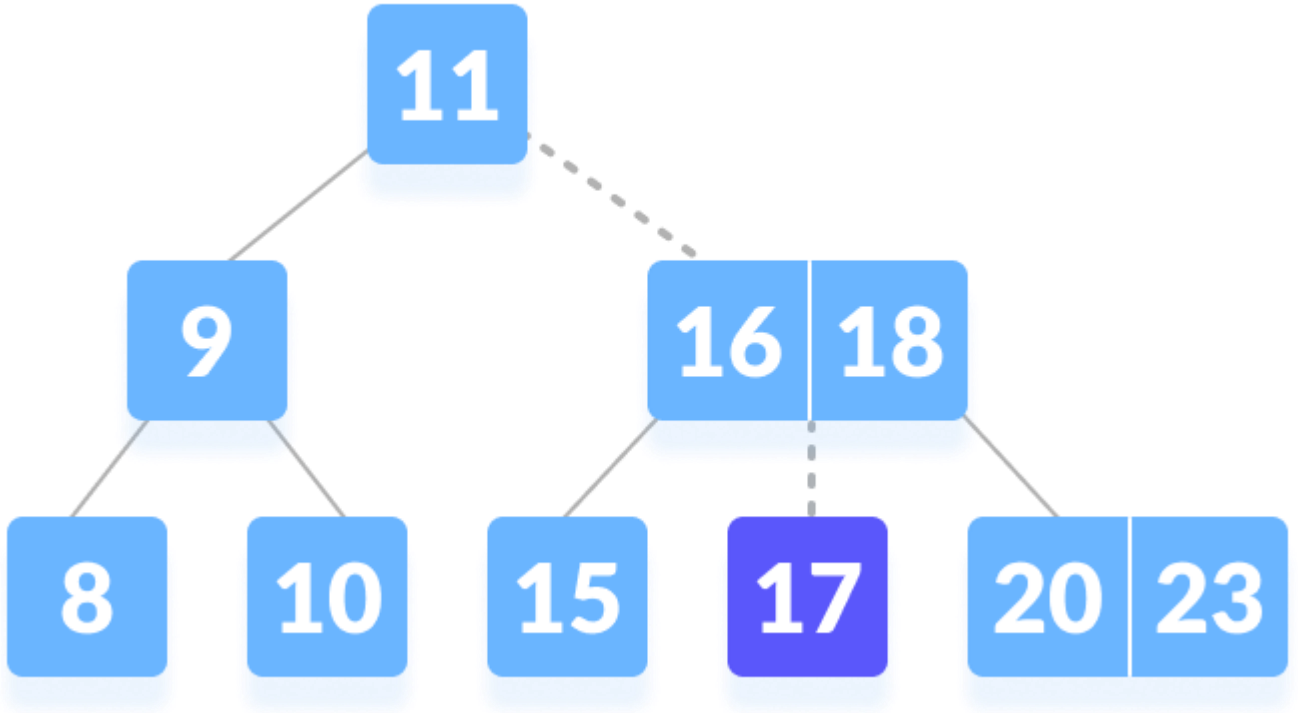
B Tree

Since k < 18, k lies between 16 and 18. Search in the right child of 16 or the left child of 18.



B Tree

k is found



B Tree - Insertion

Steps for Insertion -

- Locate the Leaf Node: Find the appropriate leaf node where the new key should be inserted.
- Insert in Sorted Order: Place the key in the correct position within the node.
- Check for Overflow: If the node exceeds the maximum allowed keys, it needs to be split.

Handling Overflow (Node Splitting)-

- Middle Key Promotion: If a node has more than 2t - 1 keys (where t is the minimum degree), split it into two.
- Split Process:
  - The median key moves up to the parent.
  - The left and right halves form two separate nodes.
- Recursion to Root: If the parent overflows, repeat the split upwards. If the root splits, the tree height increases.

B Tree - Insertion

- Edge Cases
  - Root Node Split: The only case where the height of the B-Tree increases.
  - Insertion in a Full Leaf Node: Requires splitting before inserting the key.
  - Multiple Splits: Can propagate up the tree if several nodes overflow.
- Complexity Analysis
  - Search for insertion: O(logn)
  - Insertion (with possible splits): O(logn)
  - Space Complexity: Efficient due to high branching factor.

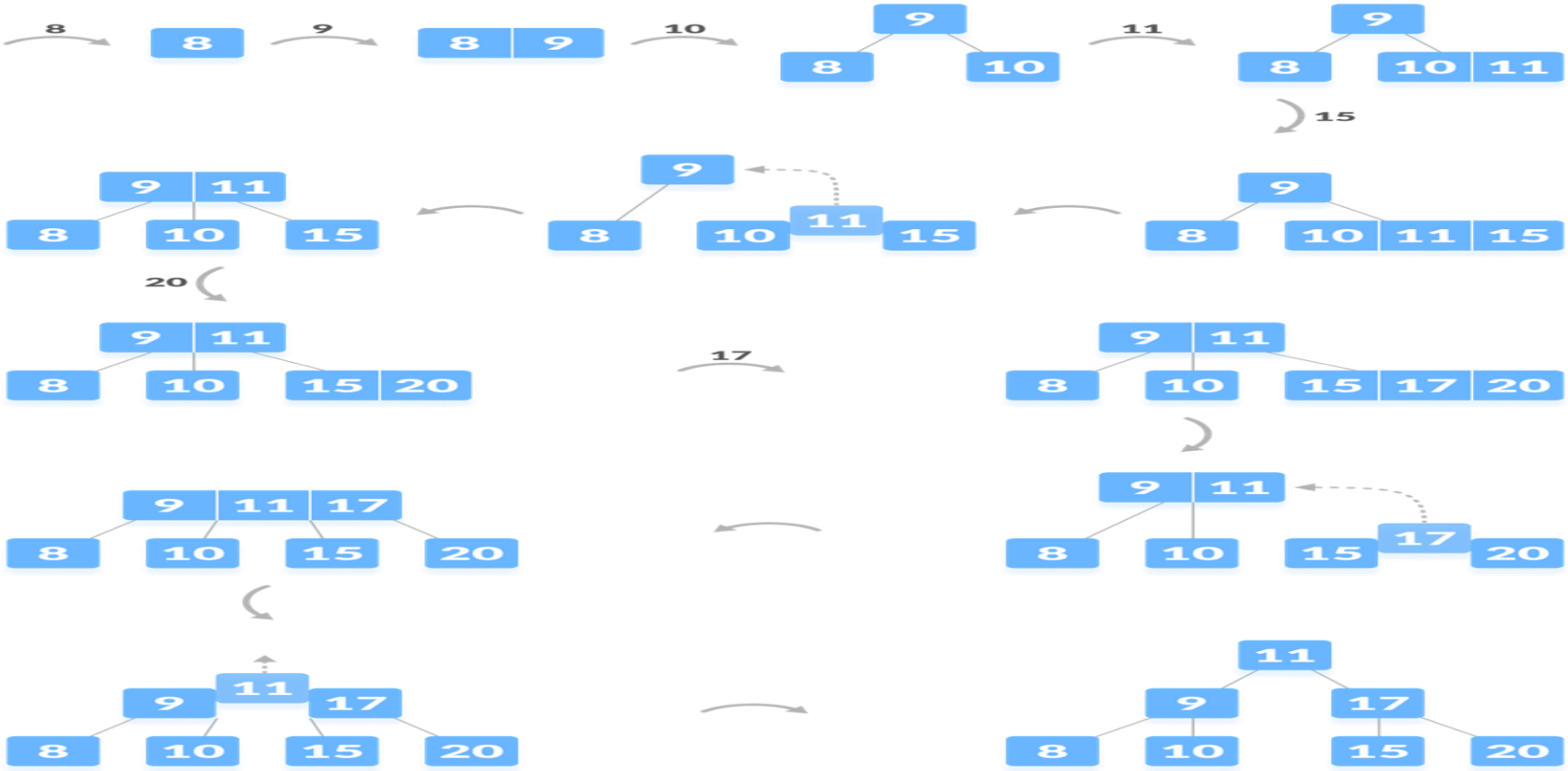
Minimum number of keys -> t-1, (m/2) - 1
Maximum number of keys -> 2t-1, m -1
Maximum number of children -> 2t, m
Where t is the minimum degree and m is the order of the tree

B Tree - Insertion Algorithm

```
1: procedure B-Tree-Insert (Node x, Key k)
2:    find i such that x:keys[i] > k or i >=numkeys(x)
3:    if x is a leaf then
4:       Insert k into x.keys at i
5:    else
6:      if x:child[i] is full then
7:         Split x:child[i]
8:         if k > x:key[i] then
9:            i = i + 1
10:        end if
11:      end if
12:     B-Tree-Insert(x:child[i]; k)
13:  end if
14: end procedure
```

DESIGN AND ANALYSIS OF ALGORITHMS

B Tree Insertion Example



Insertion Example

Let us understand the insertion operation with the illustrations below.
The elements to be inserted are 8, 9, 10, 11, 15, 16, 17, 18, 20, 23.

B Tree - Deletion

PES UNIVERSITY
CELEBRATING 50 YEARS

Deletion Operation on a B-Tree:

Deleting an element on a B-tree consists of three main events: **searching the node where the key to be deleted exists**, deleting the key and balancing the tree if required.

While deleting a node in a tree, a condition called **underflow** may occur. Underflow occurs when a node contains less than the minimum number of keys it should hold.

The terms to be understood before studying deletion operation are:

1. **Inorder Predecessor**

The largest key on the left child of a node is called its inorder predecessor.

2. **Inorder Successor**

The smallest key on the right child of a node is called its inorder successor.

DESIGN AND ANALYSIS OF ALGORITHMS

B Tree - Deletion

Before going through the steps below, one must know these facts about a B tree of degree **m**.

1. A node can have a maximum of m children. (i.e. 3)
2. A node can contain a maximum of m - 1 keys. (i.e. 2)
3. A node should have a minimum of [m/2] children. (i.e. 2)
4. A node (except root node) should contain a minimum of [m/2] - 1 keys. (i.e. 1)

There are three main cases for deletion operation in a B tree.

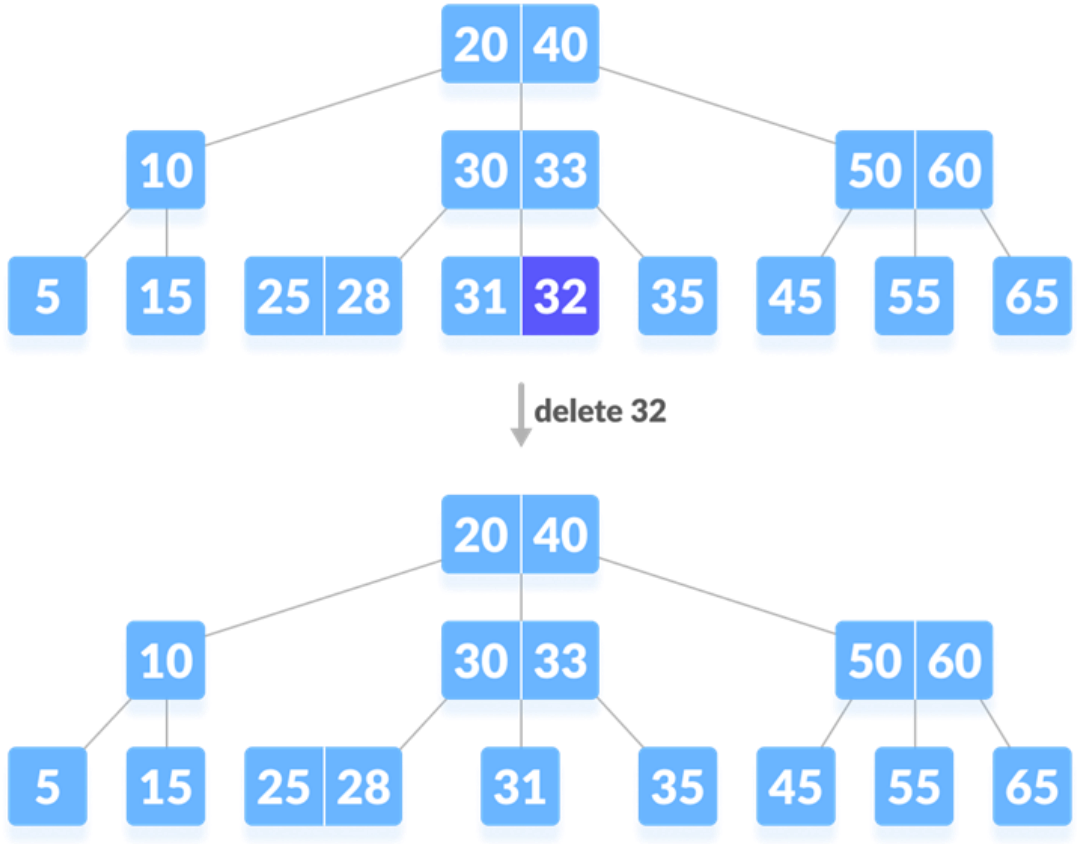
B Tree - Deletion

Case I

The key to be deleted lies in the leaf. There are two cases for it.

1.The deletion of the key does not violate the property of the minimum number of keys a node should hold.

In the tree below, deleting 32 does not violate the above properties.



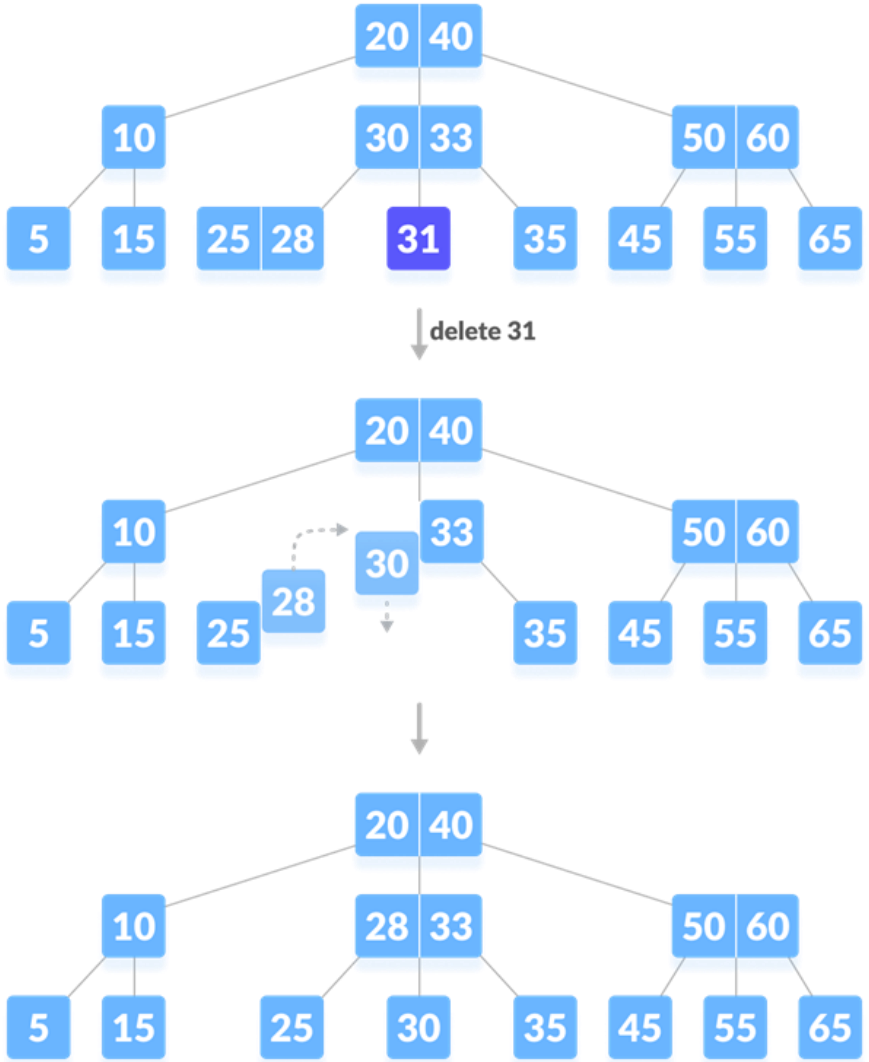
B Tree - Deletion

2. The deletion of the key violates the property of the minimum number of keys a node should hold. In this case, we borrow a key from its immediate neighboring sibling node in the order of left to right.

First, visit the immediate left sibling. If the left sibling node has more than a minimum number of keys, then borrow a key from this node.

Else, check to borrow from the immediate right sibling node.

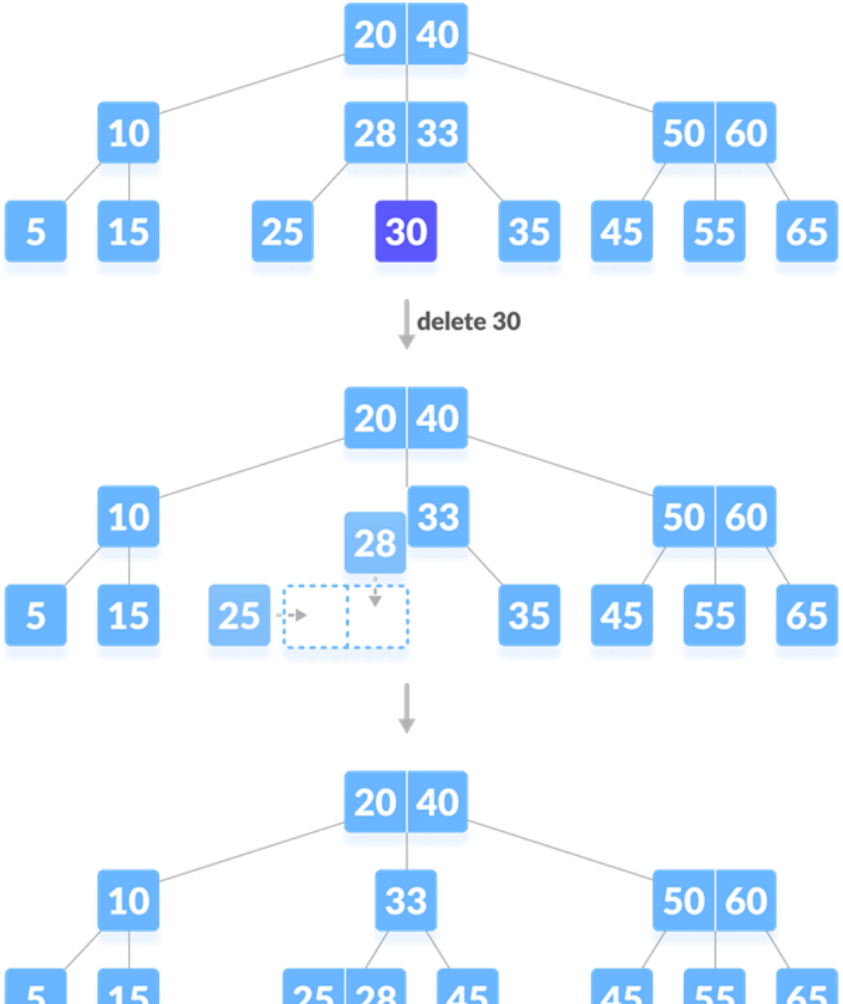
In the tree below, deleting 31 results in the above condition. Let us borrow a key from the left sibling node.



B Tree - Deletion

If both the immediate sibling nodes already have a minimum number of keys, then merge the node with either the left sibling node or the right sibling node. **This merging is done through the parent node.**

Deleting 30 results in the above case.

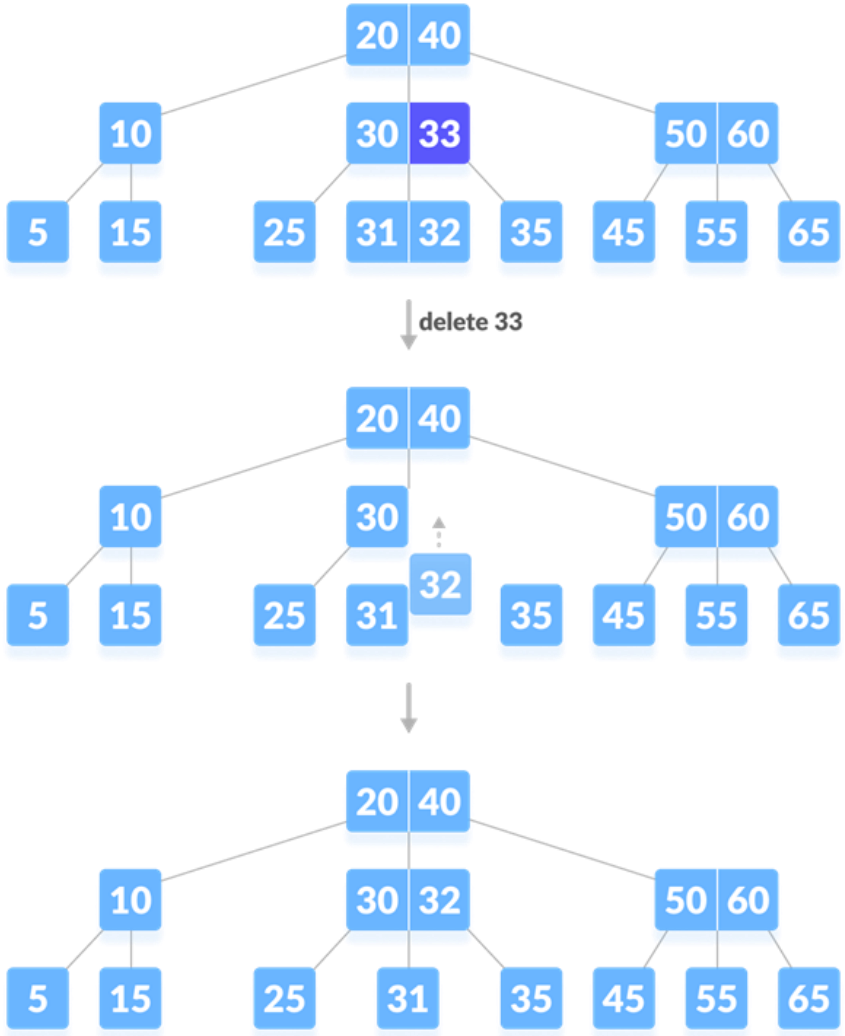


B Tree - Deletion

Case II

If the key to be deleted lies in the internal node, the following cases occur.

1.The internal node, which is deleted, is replaced by an inorder predecessor if the left child has more than the minimum number of keys.

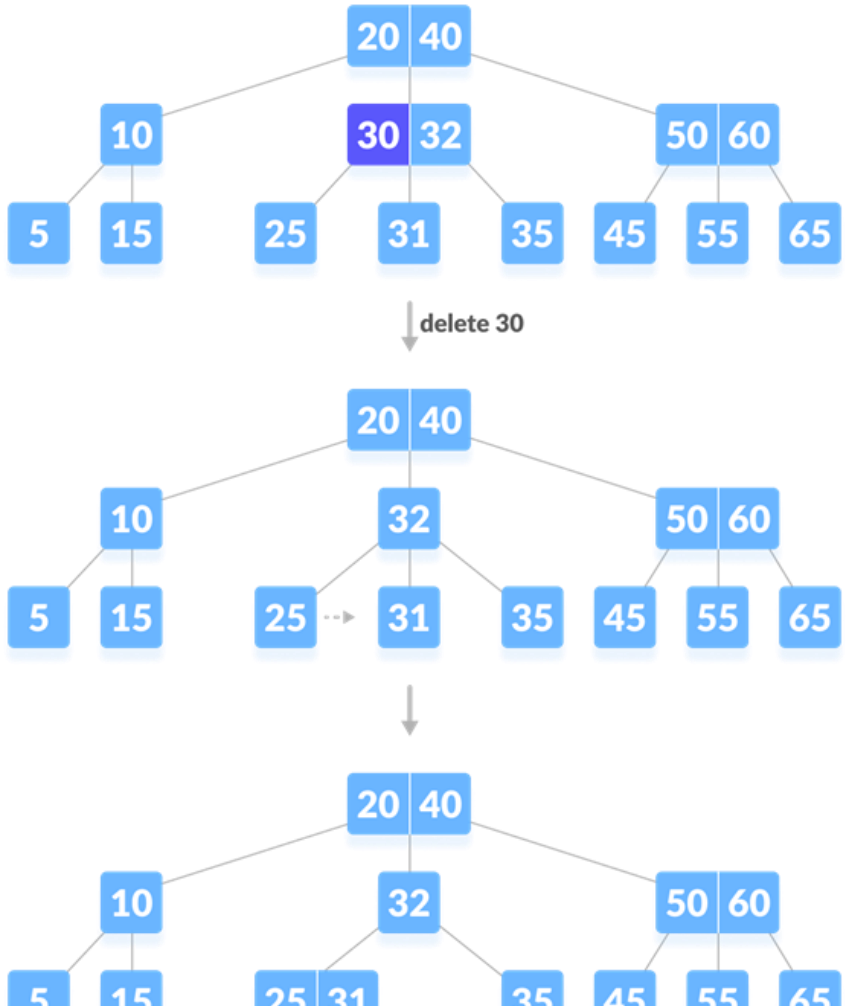


B Tree - Deletion

2. The internal node, which is deleted, is replaced by an inorder successor if the right child has more than the minimum number of keys.

3. If either child has exactly a minimum number of keys then, merge the left and the right children.

After merging if the parent node has less than the minimum number of keys then, look for the siblings as in Case I.

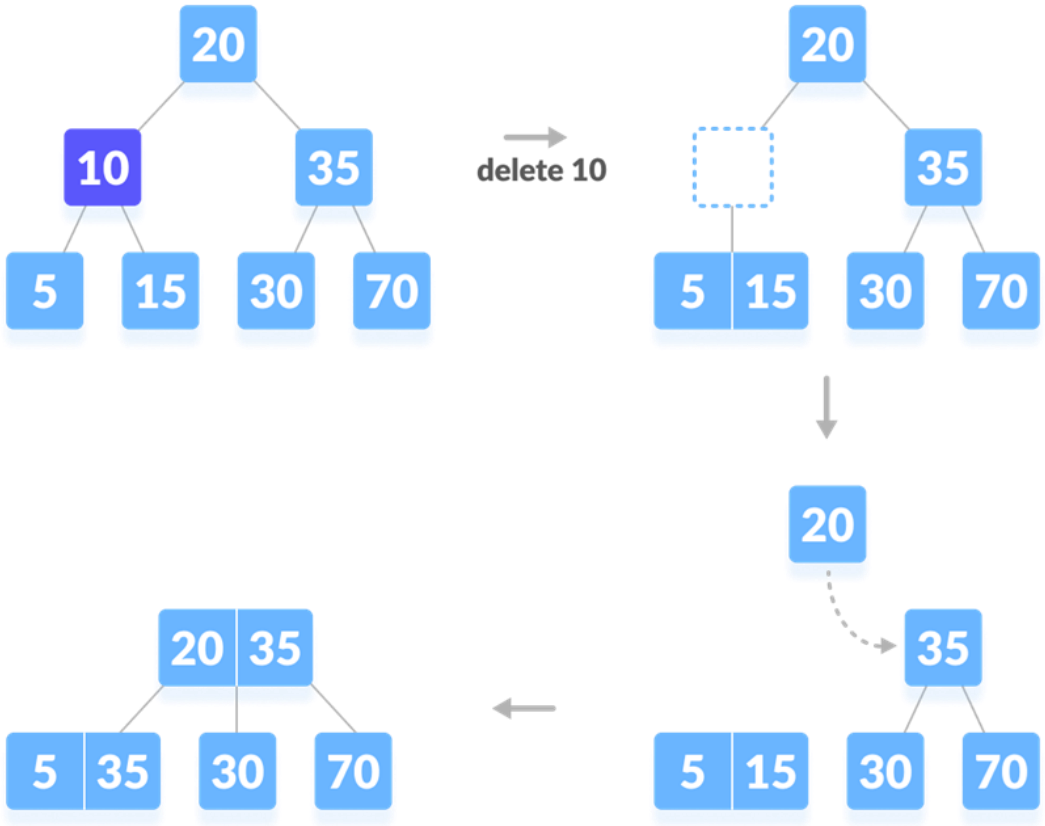


B Tree - Deletion

Case III

In this case, the height of the tree shrinks. If the target key lies in an internal node, and the deletion of the key leads to a fewer number of keys in the node (i.e. less than the minimum required), then look for the inorder predecessor and the inorder successor. If both the children contain a minimum number of keys then, borrowing cannot take place. This leads to Case II(3) i.e. merging the children.

Again, look for the sibling to borrow a key. But, if the sibling also has only a minimum number of keys then, merge the node with the sibling along with the parent. Arrange the children accordingly (increasing order).



B Tree - Deletion

Deletion Complexity

Best case Time complexity: Θ(log n)

Average case Space complexity: Θ(n)

Worst case Space complexity: Θ(n)

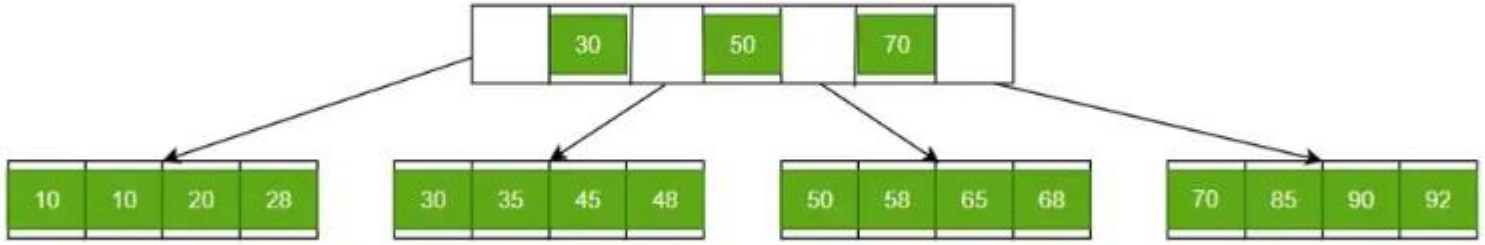
PES UNIVERSITY
CELEBRATING 50 YEARS

DESIGN AND ANALYSIS OF ALGORITHMS

B+ Trees
(additional Information- not in syllabus)

B+ Tree

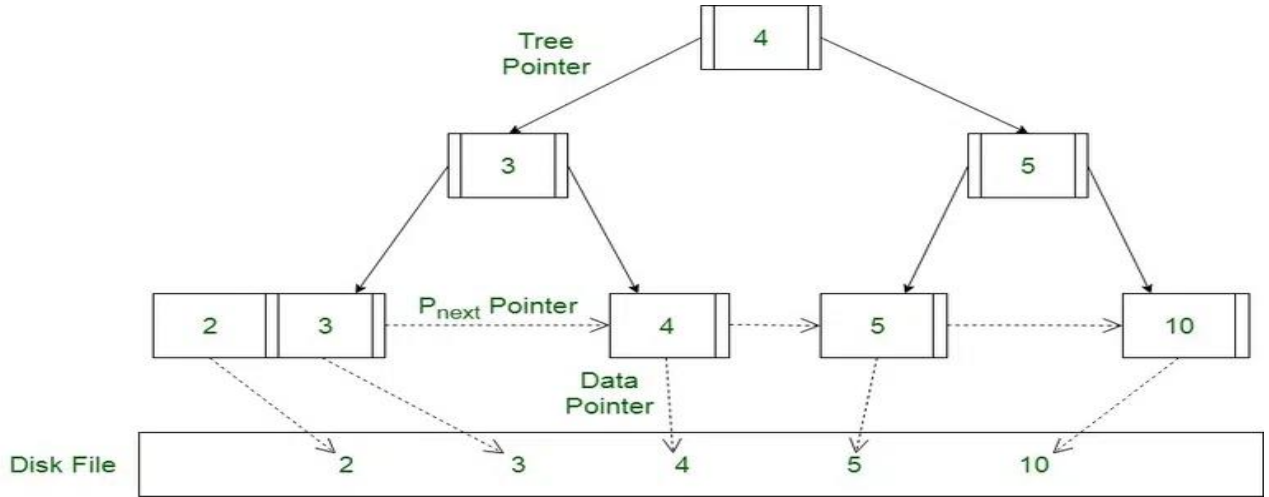
- B + Tree is a variation of the B-tree data structure. In a B + tree, data pointers are stored only at the leaf nodes of the tree.
- In a B+ tree structure of a leaf node differs from the structure of internal nodes.
  - The leaf nodes have an entry for every value of the search field, along with a data pointer to the record (or to the block that contains this record). The leaf nodes of the B+ tree are linked together to provide ordered access to the search field to the records.
  - Internal nodes of a B+ tree are used to guide the search. Some search field values from the leaf nodes are repeated in the internal nodes of the B+ tree.



B+ Tree

- Properties of B+ Trees
  - Each node except root can have a maximum of M children and at least ceil(M/2) children.
  - Each node can contain a maximum of M – 1 keys and a minimum of ceil(M/2) – 1 keys.
  - The root has at least two children and atleast one search key.
  - While insertion overflow of the node occurs when it contains more than M – 1 search key values

Here M is the order of the B Tree



B+ Tree

PES UNIVERSITY
CELEBRATING 50 YEARS

Operations on a B+ Tree:

- Insertion
- Deletion

B+ Tree - Insertion

Properties for insertion B+ Tree

- Case 1: Overflow in leaf node
  - Split the leaf node into two nodes.
  - First node contains ceil((m-1)/2) values.
  - Second node contains the remaining values.
  - Copy the smallest search key value from second node to the parent node.(Right biased)

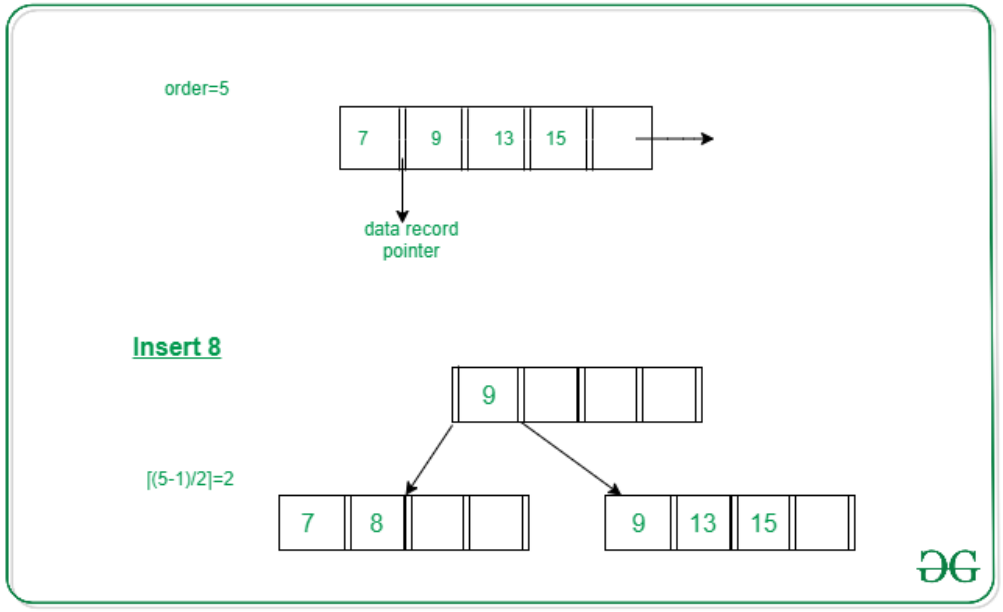


Illustration of inserting 8 into B+ Tree of order of 5

B+ Tree - Insertion

Properties for insertion B+ Tree

- Case 2: Overflow in non-leaf node
  - Split the non leaf node into two nodes.
  - First node contains ceil(m/2)-1 values.
  - Move the smallest among remaining to the parent.
  - Second node contains the remaining keys.

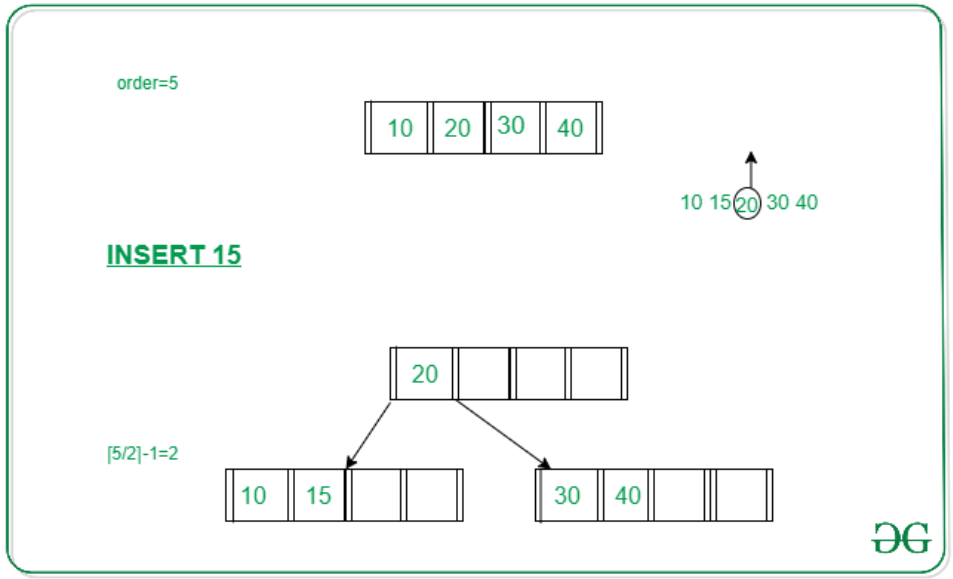
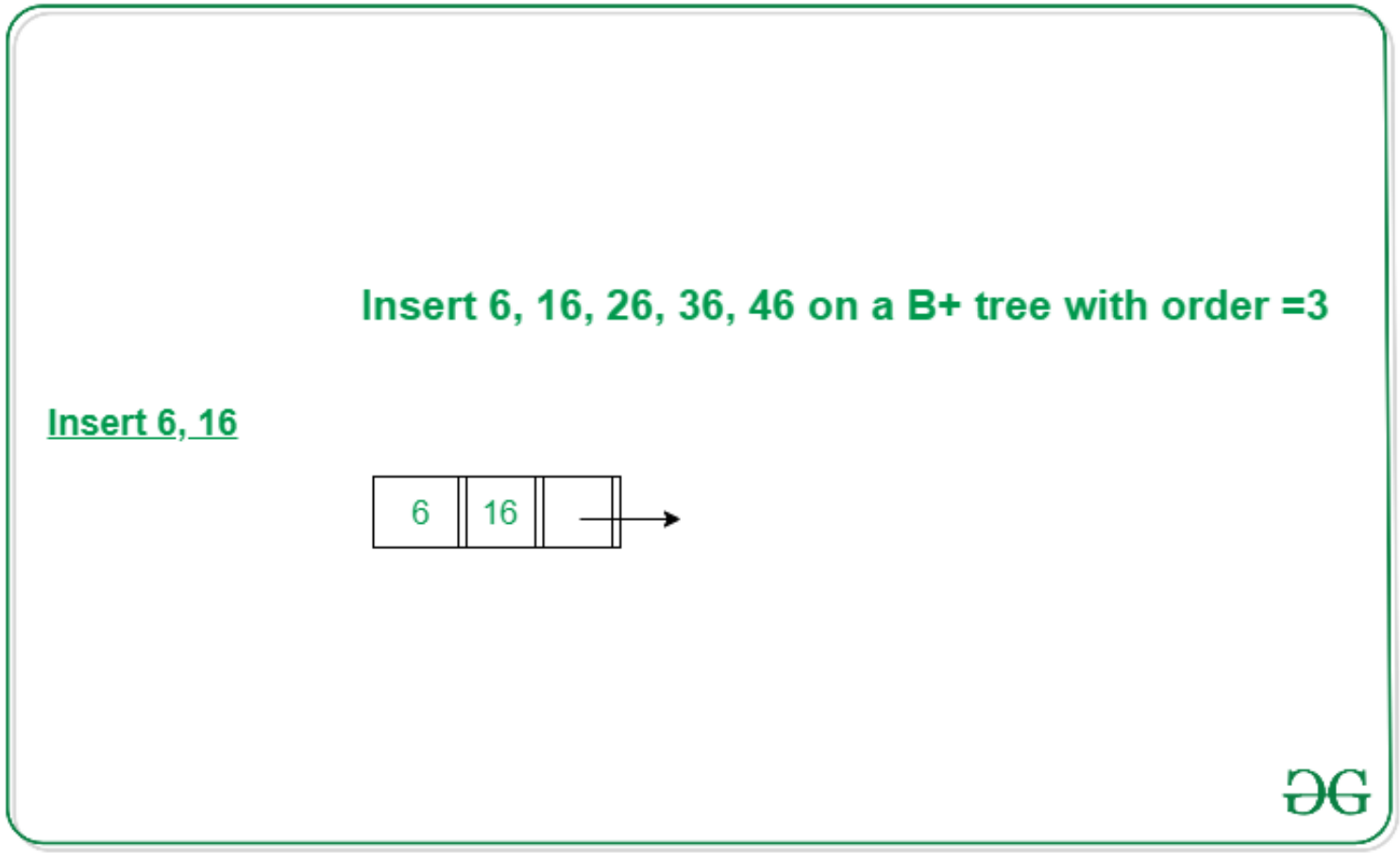


Illustration of inserting 15 into B+ Tree of order of 5

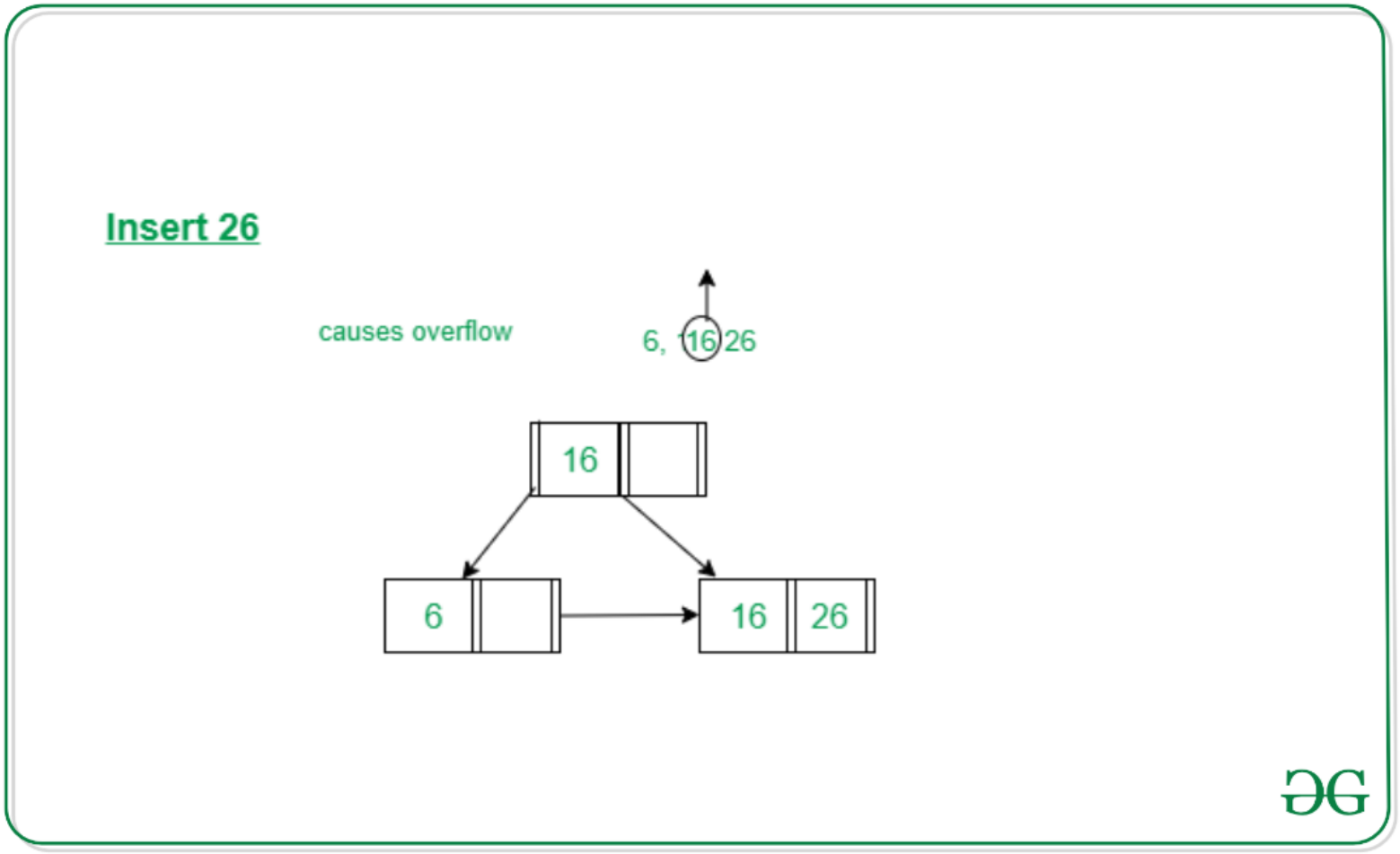
B+ Tree - Insertion Example

Problem: Insert the following key values 6, 16, 26, 36, 46 on a B+ tree with order = 3.



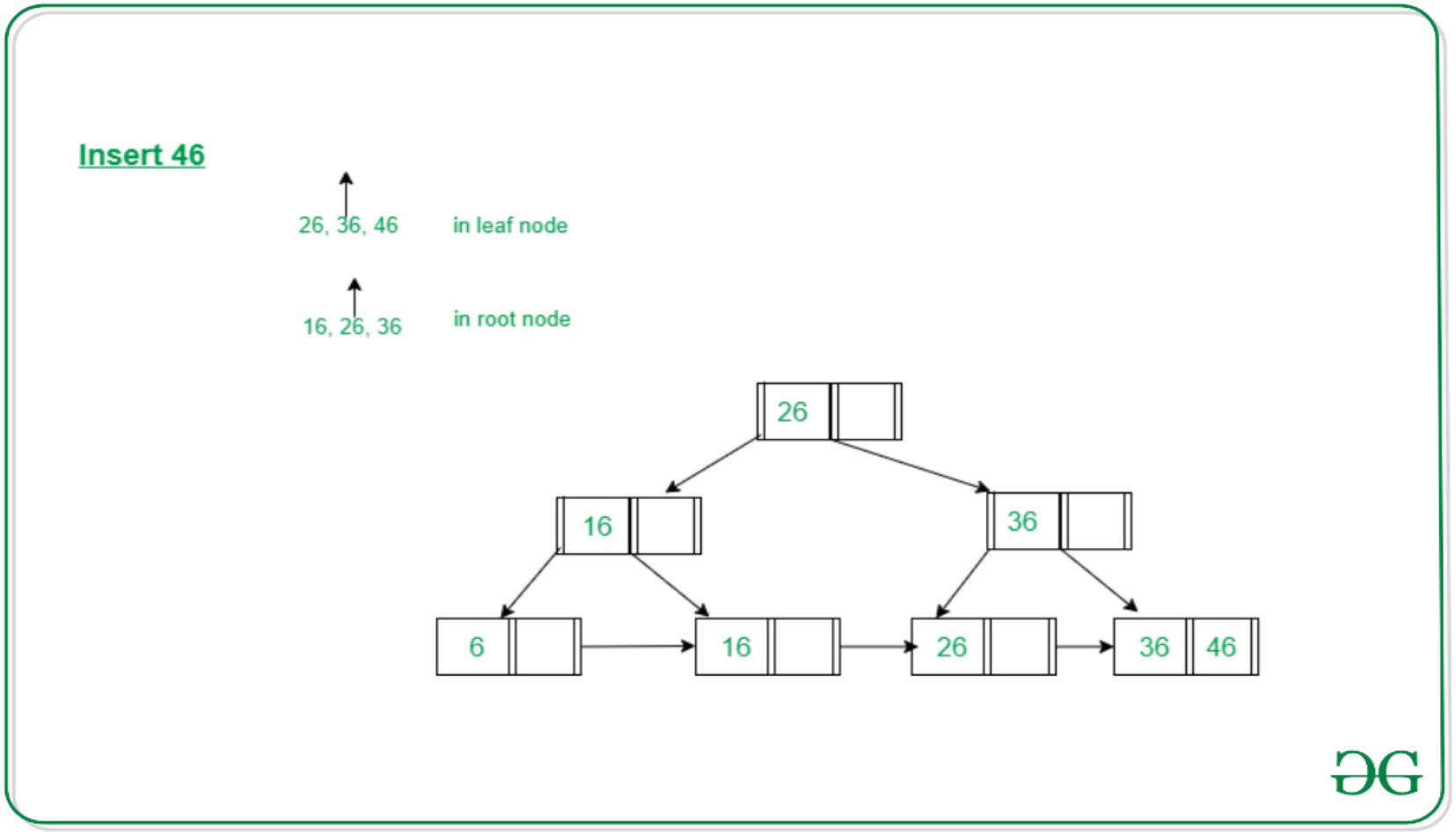
B+ Tree - Insertion Example

Problem: Insert the following key values 6, 16, 26, 36, 46 on a B+ tree with order = 3.



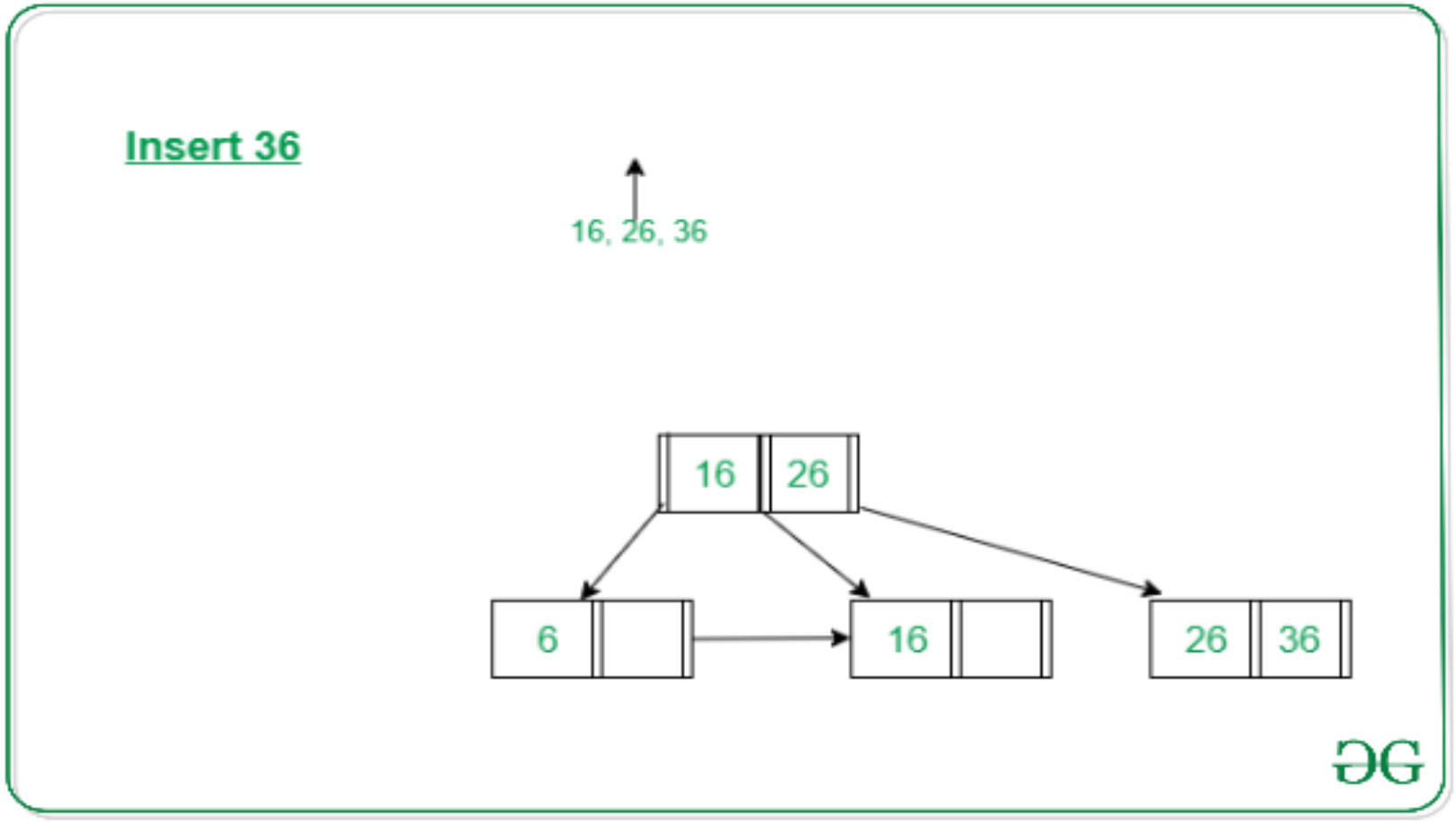
B+ Tree - Insertion Example

Problem: Insert the following key values 6, 16, 26, 36, 46 on a B+ tree with order = 3.



B+ Tree - Insertion Example

Problem: Insert the following key values 6, 16, 26, 36, 46 on a B+ tree with order = 3.



B+ Tree - Insertion Example

Problem: Insert the following key values 5,15, 25, 35, 45 on a B+ tree with order = 3.

1) Insert 5

B+ Tree - Insertion Example

Problem: Insert the following key values 5,15, 25, 35, 45 on a B+ tree with order = 3.

2) Insert 15

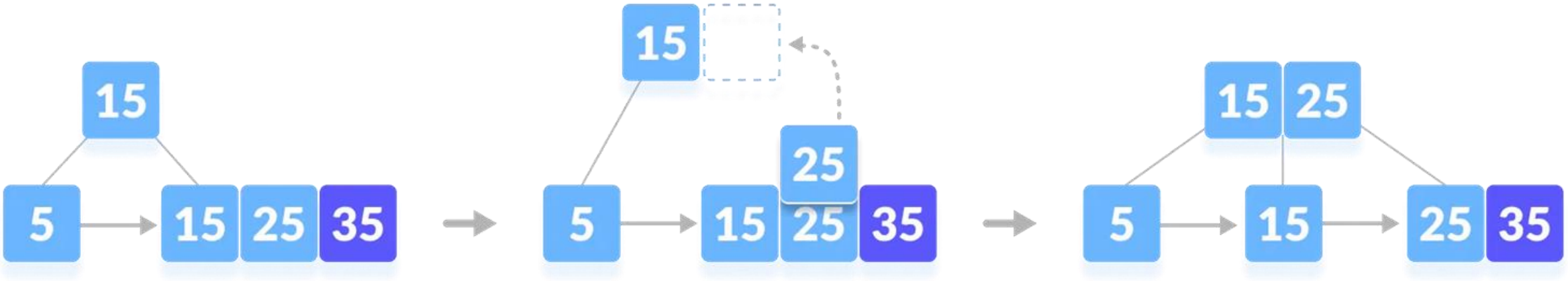
B+ Tree - Insertion Example

Problem: Insert the following key values 5,15, 25, 35, 45 on a B+ tree with order = 3.

3) Insert 25

B+ Tree - Insertion Example

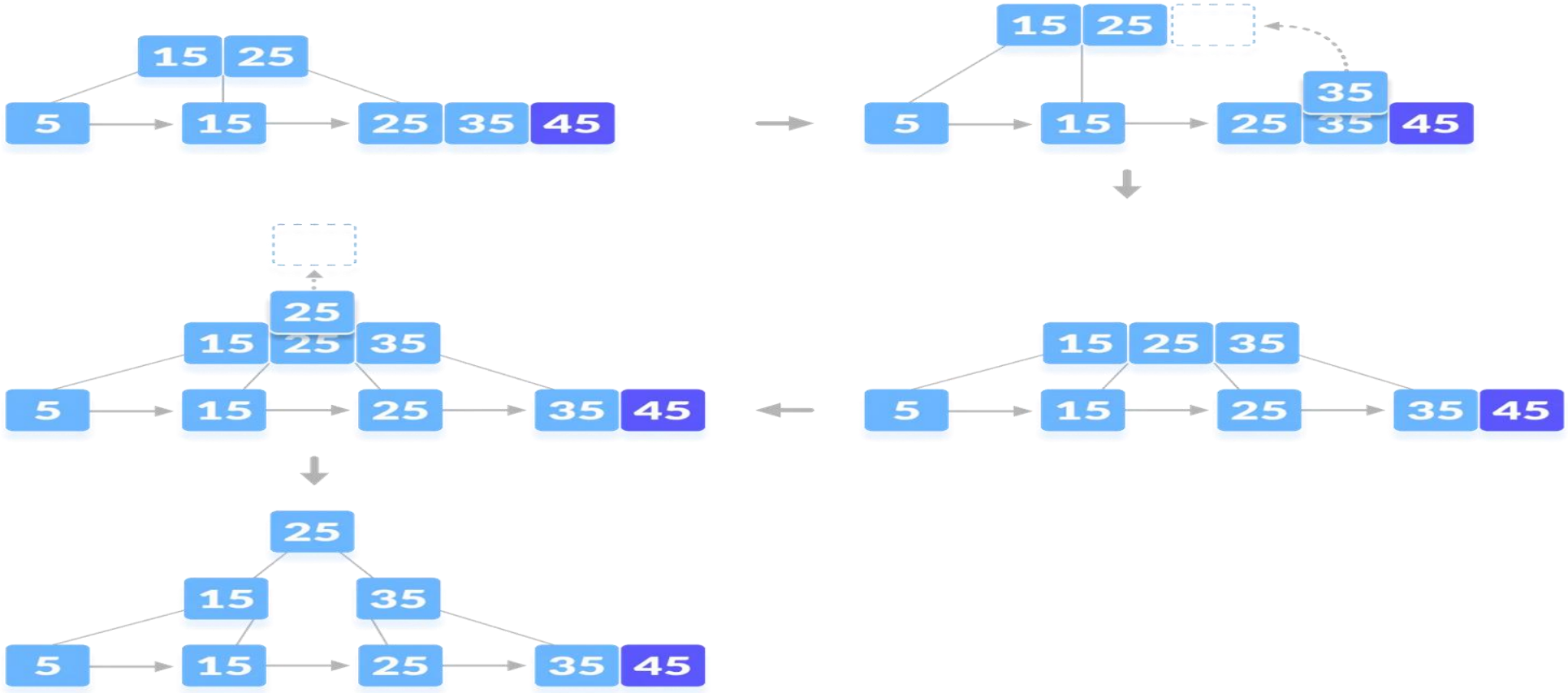
Problem: Insert the following key values 5,15, 25, 35, 45 on a B+ tree with order = 3.



4) Insert 35

B+ Tree - Insertion Example

Problem: Insert the following key values 5,15, 25, 35, 45 on a B+ tree with order = 3.



4) Insert 45

B+ Tree - Deletion

- Deleting an element on a B+ tree consists of three main events: searching the node where the key to be deleted exists, deleting the key and balancing the tree if required.
- Underflow is a situation when there is less number of keys in a node than the minimum number of keys it should hold.
- While deleting a key, we have to take care of the keys present in the internal nodes (i.e. indexes) as well because the values are redundant in a B+ tree.
  - Search the key to be deleted then follow the following steps.

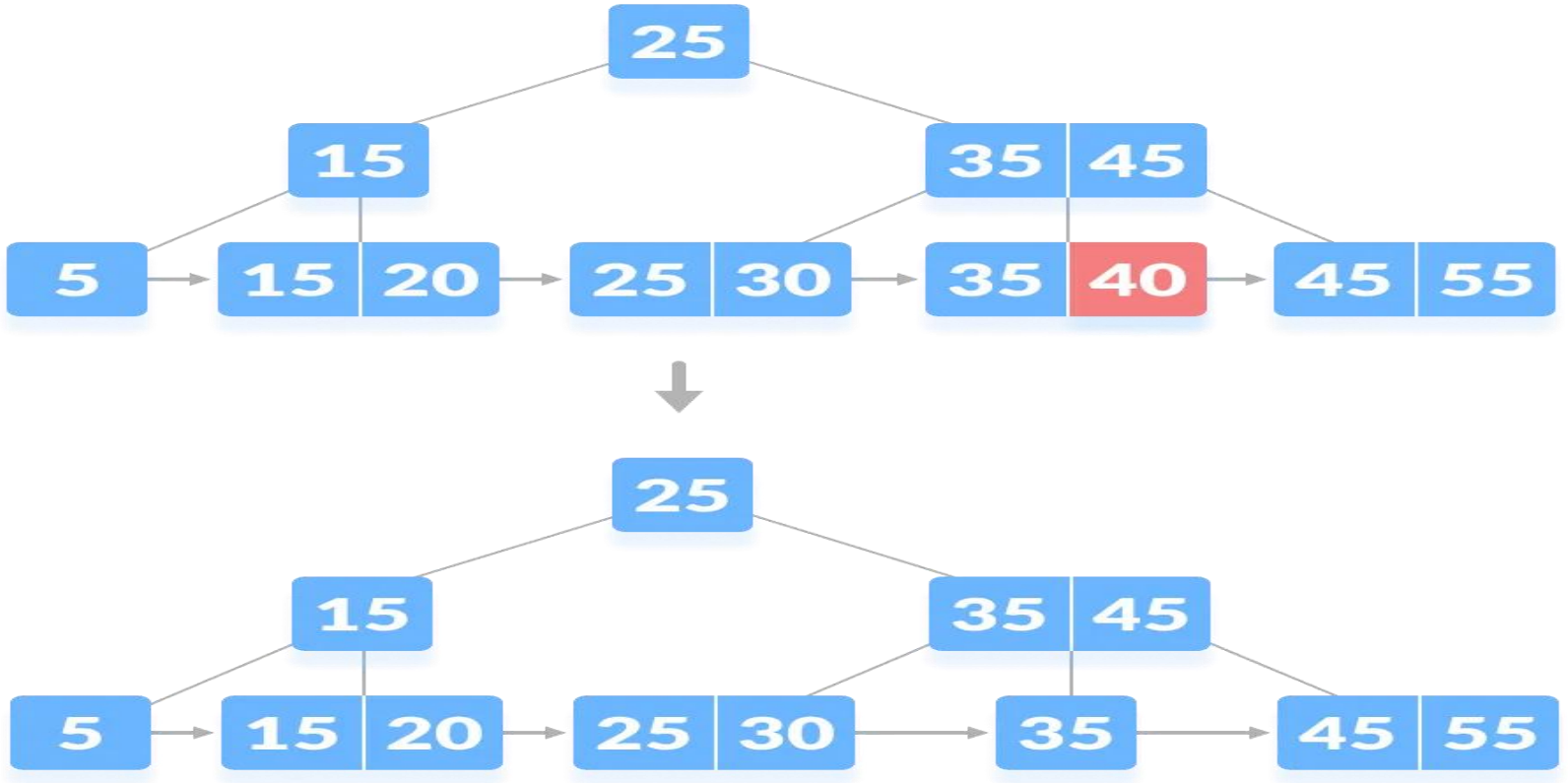
B+ Tree - Deletion Strategy

- Look to locate the deleted key in the leaf nodes.
- Delete the key and its associated value if the key is discovered in a leaf node.
- One of the following steps should be taken if the node underflows (number of keys is less than half the maximum allowed):
  - Get a key by borrowing it from a sibling node if it contains more keys than the required minimum.
  - If the minimal number of keys is met by all of the sibling nodes, merge the underflow node with one of its siblings and modify the parent node as necessary.
- Remove all references to the deleted leaf node from the internal nodes of the tree.
- Remove the old root node and update the new one if the root node is empty.

B+ Tree - Deletion Case 1

- Case 1 - The key to be deleted is present only at the leaf node not in the indexes (or internal nodes). There are two cases -
  - Case 1.1 - There is more than the minimum number of keys in the node. Simply delete the key

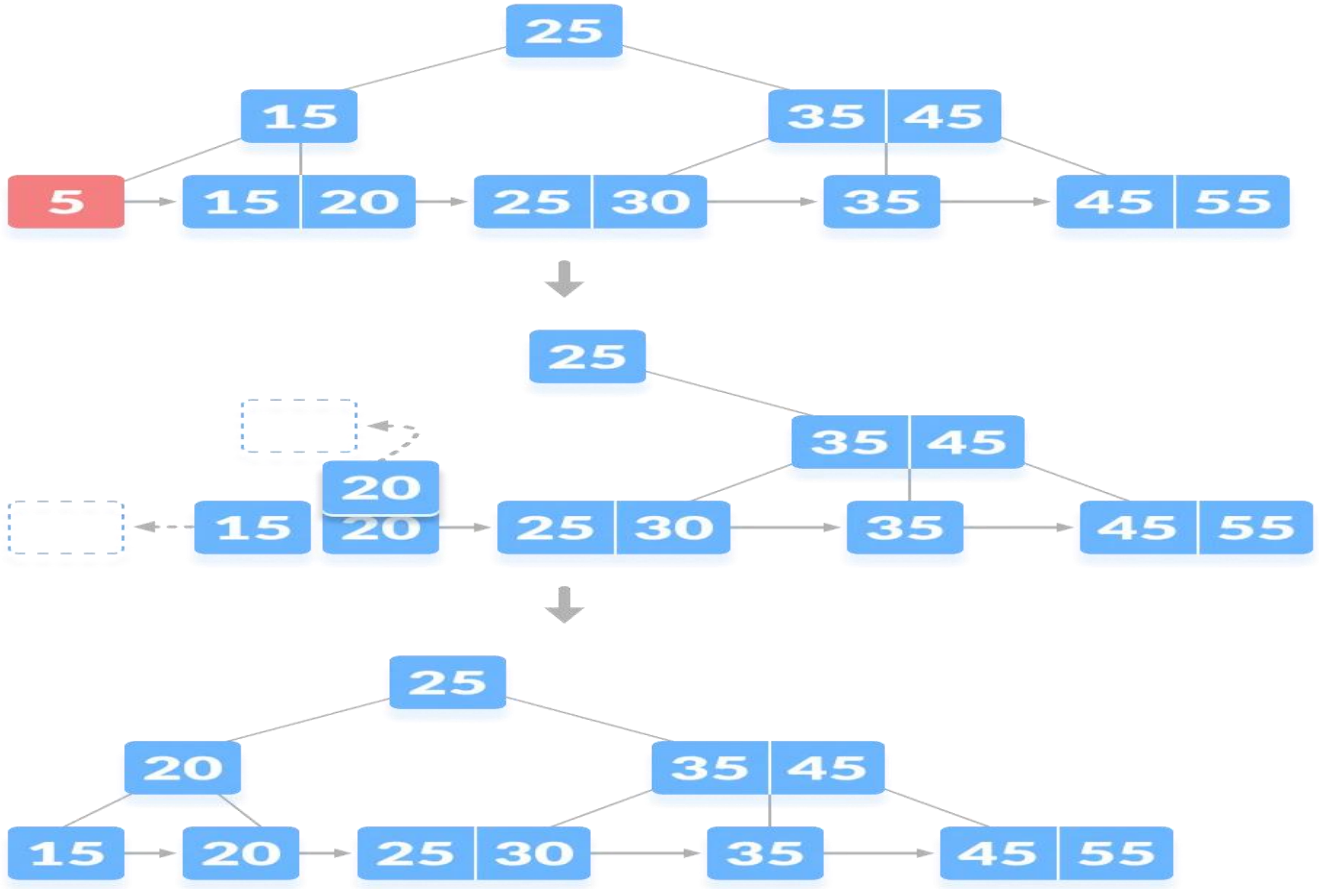
Deleting 40



B+ Tree - Deletion Case 1

- Case 1.2 - There is an exact minimum number of keys in the node. Delete the key and borrow a key from the immediate sibling. Add the median key of the sibling node to the parent.

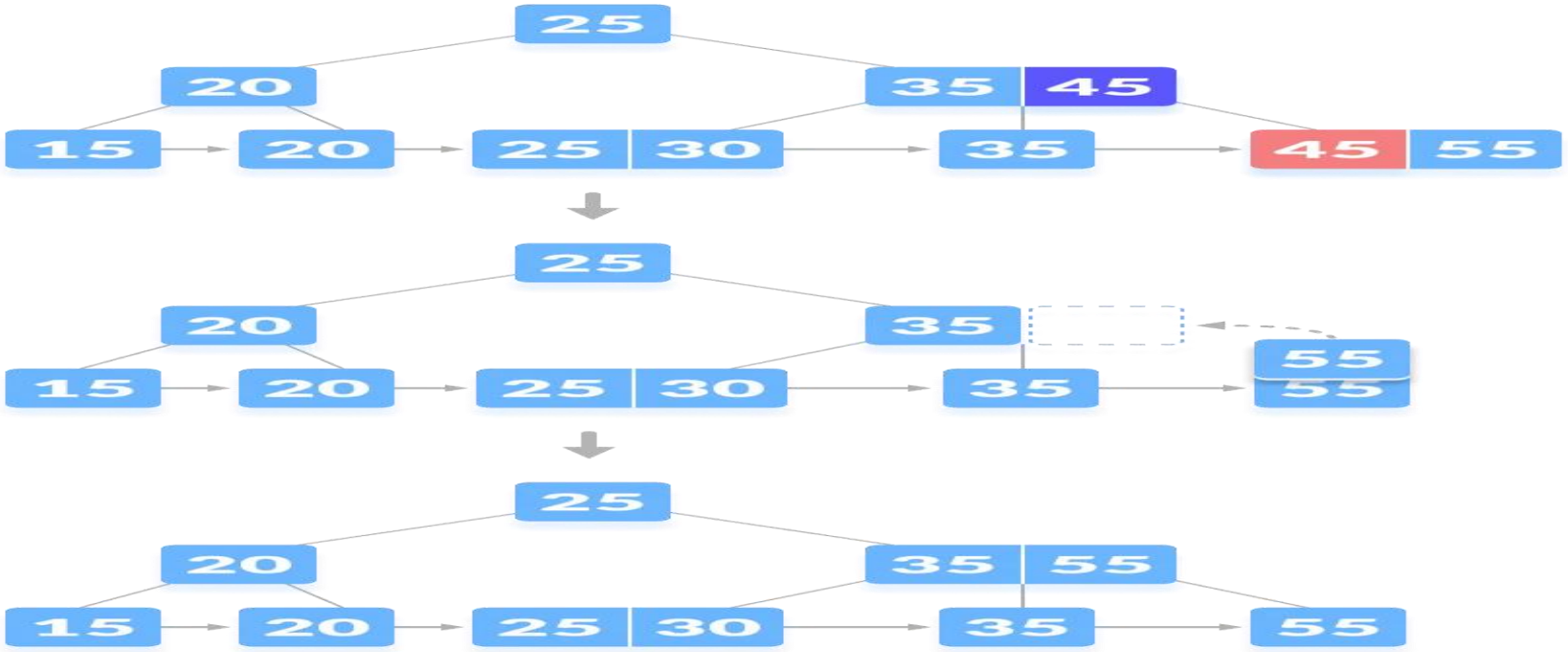
Deleting 5



B+ Tree - Deletion Case 2

- Case 2 - The key to be deleted is present in the internal nodes as well. Then we have to remove them from the internal nodes as well. There are three cases -
  - Case 2.1 - If there is more than the minimum number of keys in the node, simply delete the key from the leaf node and delete the key from the internal node as well. Fill the empty space in the internal node with the inorder successor.

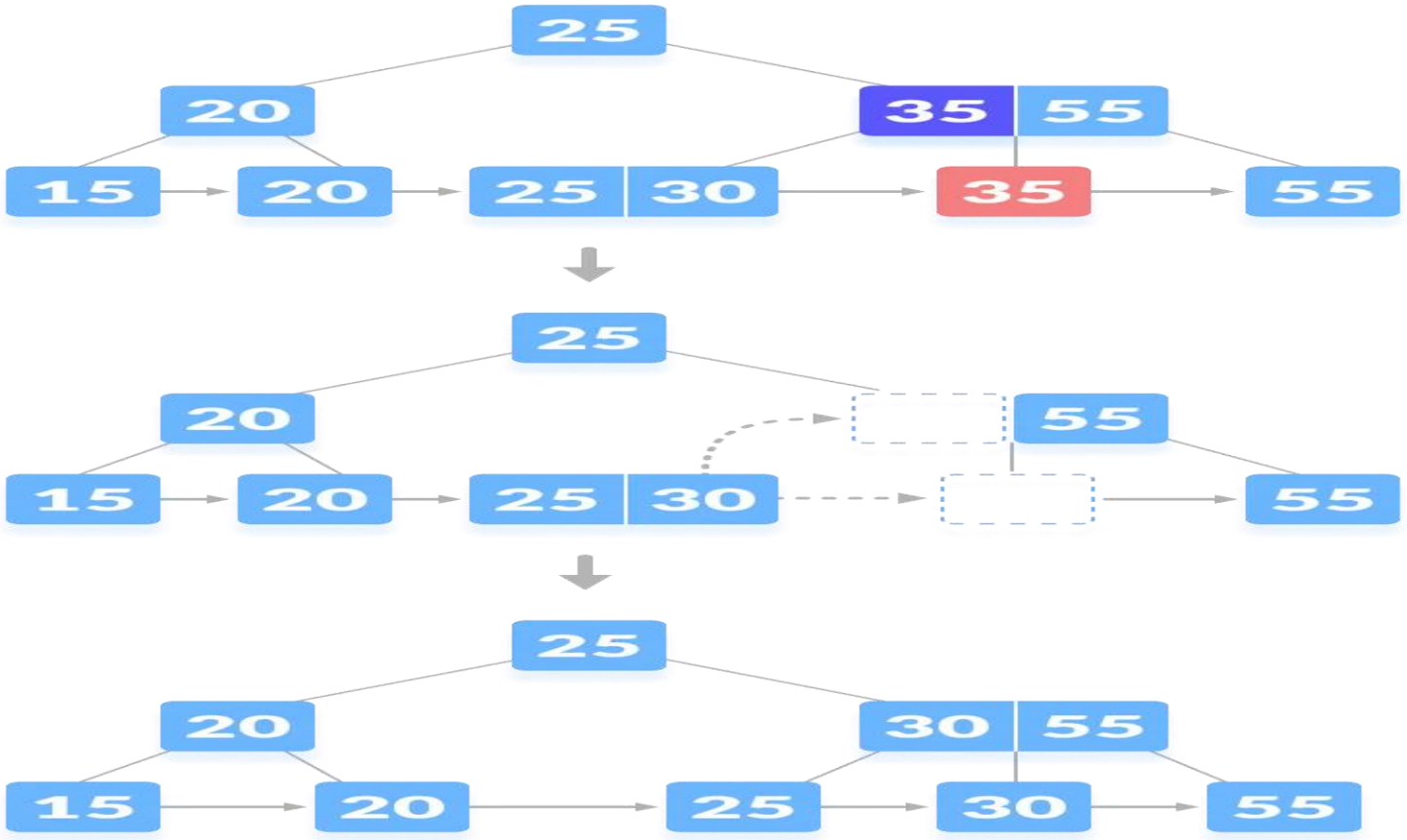
Deleting 45



B+ Tree - Deletion Case 2

- Case 2.2 - If there is an exact minimum number of keys in the node, then delete the key and borrow a key from its immediate sibling (through the parent). Fill the empty space created in the index (internal node) with the borrowed key.

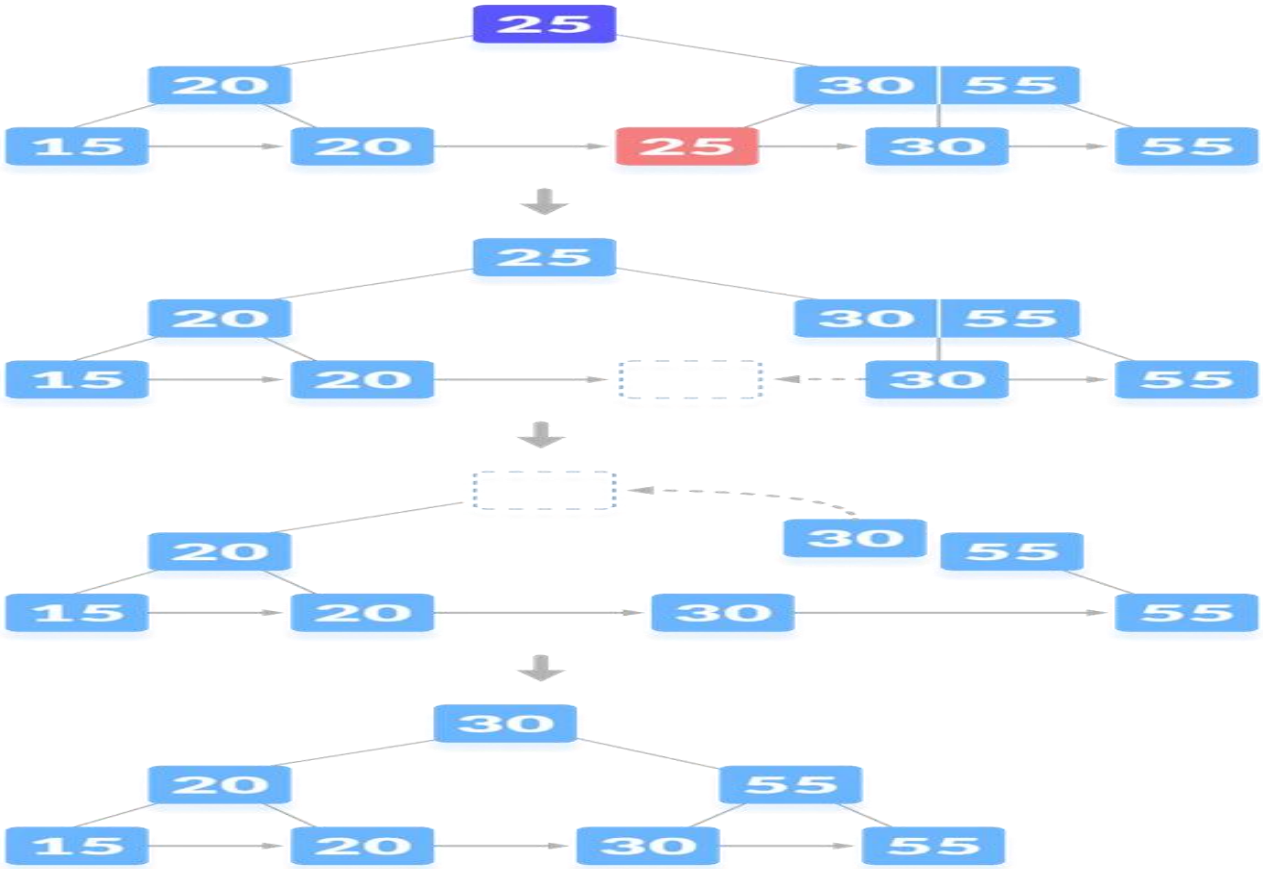
Deleting 35



B+ Tree - Deletion Case 2

- Case 2.3 - This case is similar to Case II(1) but here, empty space is generated above the immediate parent node. After deleting the key, merge the empty space with its sibling. Fill the empty space in the grandparent node with the inorder successor.

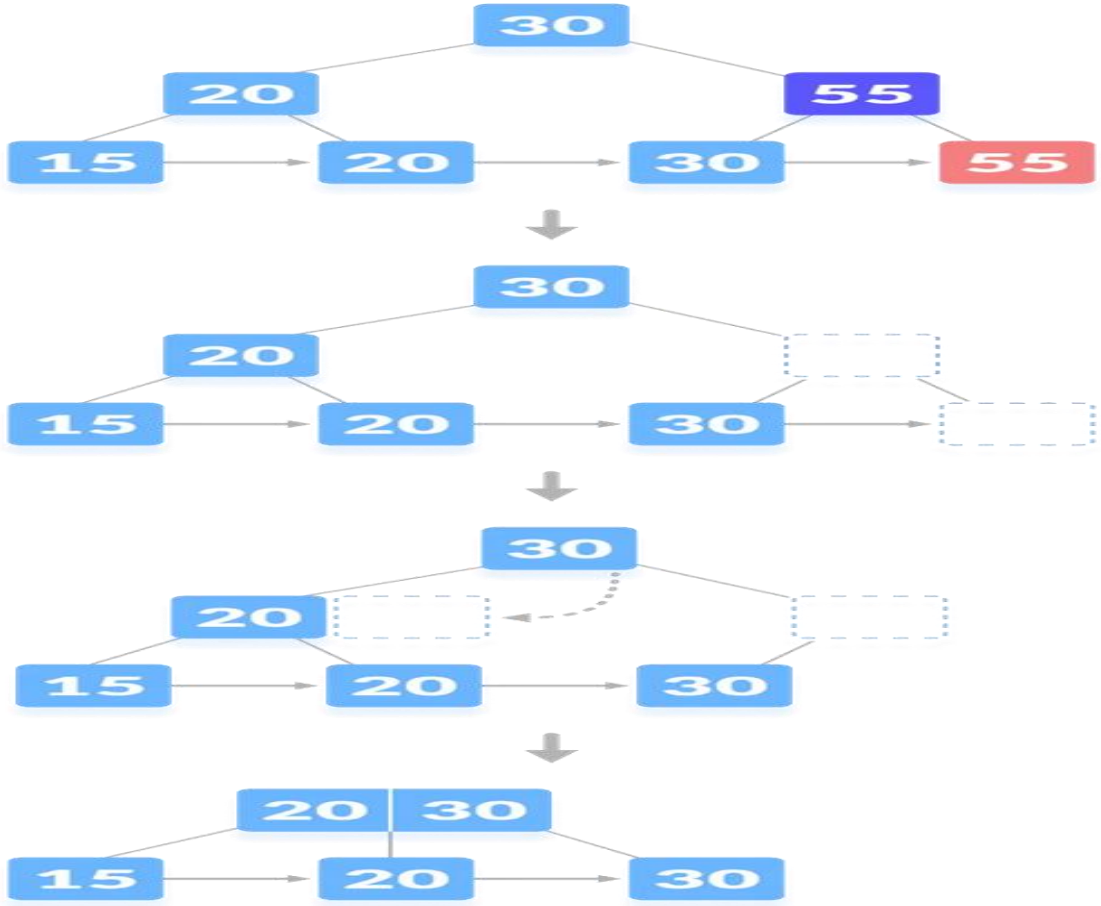
Deleting 25



B+ Tree - Deletion Case 3

- Case 3 - In this case, the height of the tree gets shrinked. It is a little complicated.Deleting 55 from the tree below leads to this condition.

Deleting 55



B+ Tree - Insertion and Deletion Complexity

PES UNIVERSITY
CELEBRATING 50 YEARS

- Insertion
  - Time complexity: O(log n)
  - Auxiliary Space: O(log n)

- Deletion
  - Time Complexity: O(log N)
  - Auxiliary Space: O(N)

DESIGN AND ANALYSIS OF ALGORITHMS

Difference between B Tree and B+ Tree

B Tree	B+ Tree
Nodes store both keys and data values	Separate leaf nodes for data storage and internal nodes for indexing
Leaf nodes do not form a linked list	Leaf nodes form a linked list for efficient range-based queries
Lower order (fewer keys)	Higher order (more keys)
Usually does not allow key duplication	Typically allows key duplication in leaf nodes
Deletion of the internal node is very complex and the tree has to undergo a lot of transformations.	Deletion of any node is easy because all node are found at leaf.
Insertion takes more time and it is not predictable sometimes.	Insertion is easier and the results are always the same.
Requires less memory as keys and values are stored in the same node	Requires more memory for internal nodes
Use Cases - In-memory data structures, databases, general-purpose use	Use cases - Database systems, file systems, where range queries are common

PES UNIVERSITY
CELEBRATING 50 YEARS

References

https://www.programiz.com/dsa/deletion-from-a-b-tree
https://www.programiz.com/dsa/insertion-into-a-b-tree
https://www.geeksforgeeks.org/insert-operation-in-b-tree/
https://www.geeksforgeeks.org/introduction-of-b-tree/
https://www.geeksforgeeks.org/difference-between-b-tree-and-b-tree/?ref=lbp
https://www.geeksforgeeks.org/insertion-in-a-b-tree/
https://www.programiz.com/dsa/insertion-on-a-b-plus-tree
https://www.programiz.com/dsa/deletion-from-a-b-plus-tree
https://www.geeksforgeeks.org/deletion-in-b-tree/

THANK YOU

Teaching Assistants

Suma Grandhi

Vaishnavi M

B Tree

Slide Courtesy: Cormen

Introduction to B-Tree

- B-trees are balanced search tree.
- More than two children are possible.

- B-Tree, stores all information in the leaves and stores only keys and Child pointer.

- If an internal B-tree node x contains n[x] keys then x has n[x]+1 children.

Example of B-Tree

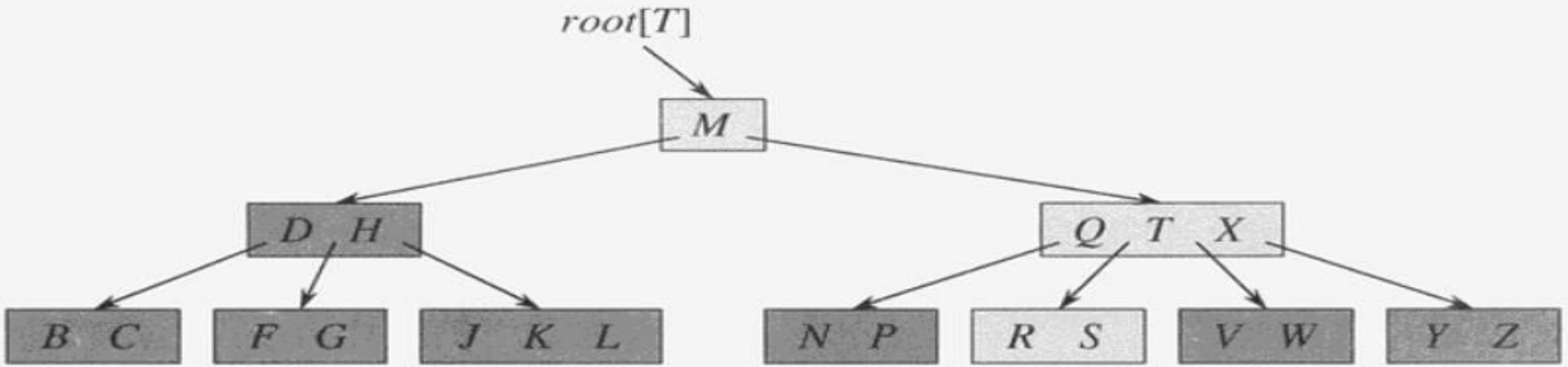


Figure 18.1 A B-tree whose keys are the consonants of English. An internal node x containing $n[x]$ keys has $n[x] + 1$ children. All leaves are at the same depth in the tree. The lightly shaded nodes are examined in a search for the letter R.

Properties of B-Tree

- A B-Tree T is a rooted tree having the following properties:

1. Every node x has the following fields:
    1. n[x] the no. of keys currently stored in node x
    2. The n[x] keys themselves, stored in non-decreasing order, so that $key_1[x] \leq key_2[x] \ldots\ldots \leq key_{n-1}[x] \leq key_n[x]$.
    3. Leaf[x], a Boolean value that is TRUE if x is a leaf and FALSE if x is an internal node.
2. Each internal node x also contains n[x]+1 pointers (Childs) $c_1[x]$, $c_2[x]$,---------$c_{n[x]+1}[x]$.
3. All leaves have the same depth, which is the tree's height h.

Properties of B-Tree (cont.)

4. There are lower and upper bounds on the no. of keys a node can contains: these bounds can be expressed in terms of a fixed integer $t \geq 2$ called the minimum degree of B-Tree.
   - Every node other than the root must have at least t-1 keys, then root has at least t children if the tree is non empty the root must have at least one key.
   - Every node can contain at most 2t-1 keys. Therefore, an internal node can have at most 2t children we say that a node is full if it contains exactly 2t-1 keys.

Height of B-tree

If $n \geq 1$, then for any n-key B-tree T of height h and minimum degree $t \geq 2$,

$$h \leq \log_t (n+1)/2$$

Basic operation on B-tree

- B-TREE-SEARCH :-Searching in B Tree
- B-TREE-INSERT :-Inserting key in B Tree
- B-TREE-CREATE :-Creating a B Tree
- B-TREE-DELETE :- Deleting a key from B Tree

Insert

- Cannot just create a new leaf node and insert it
  - resulting tree is not B-tree
- Insert new key into an existing leaf node
- If leaf node is full
  - Split full node y (with 2t-1) keys around its median $key_t[y]$ into two nodes each having t-1 keys
  - Move the median key into y's parent.
  - If parent is full, recursively split, all the way to the root node if necessary.
  - If root is full, split root - height of tree increase by one.

INSERT

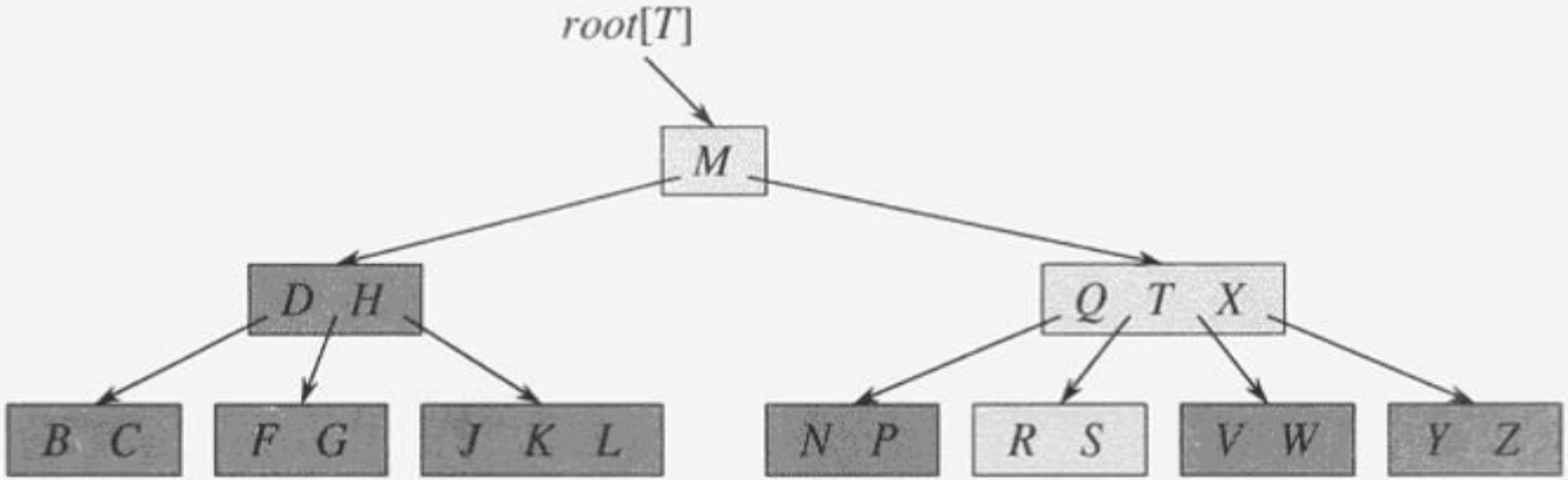


Figure 18.1 A B-tree whose keys are the consonants of English. An internal node x containing $n[x]$ keys has $n[x] + 1$ children. All leaves are at the same depth in the tree. The lightly shaded nodes are examined in a search for the letter R.

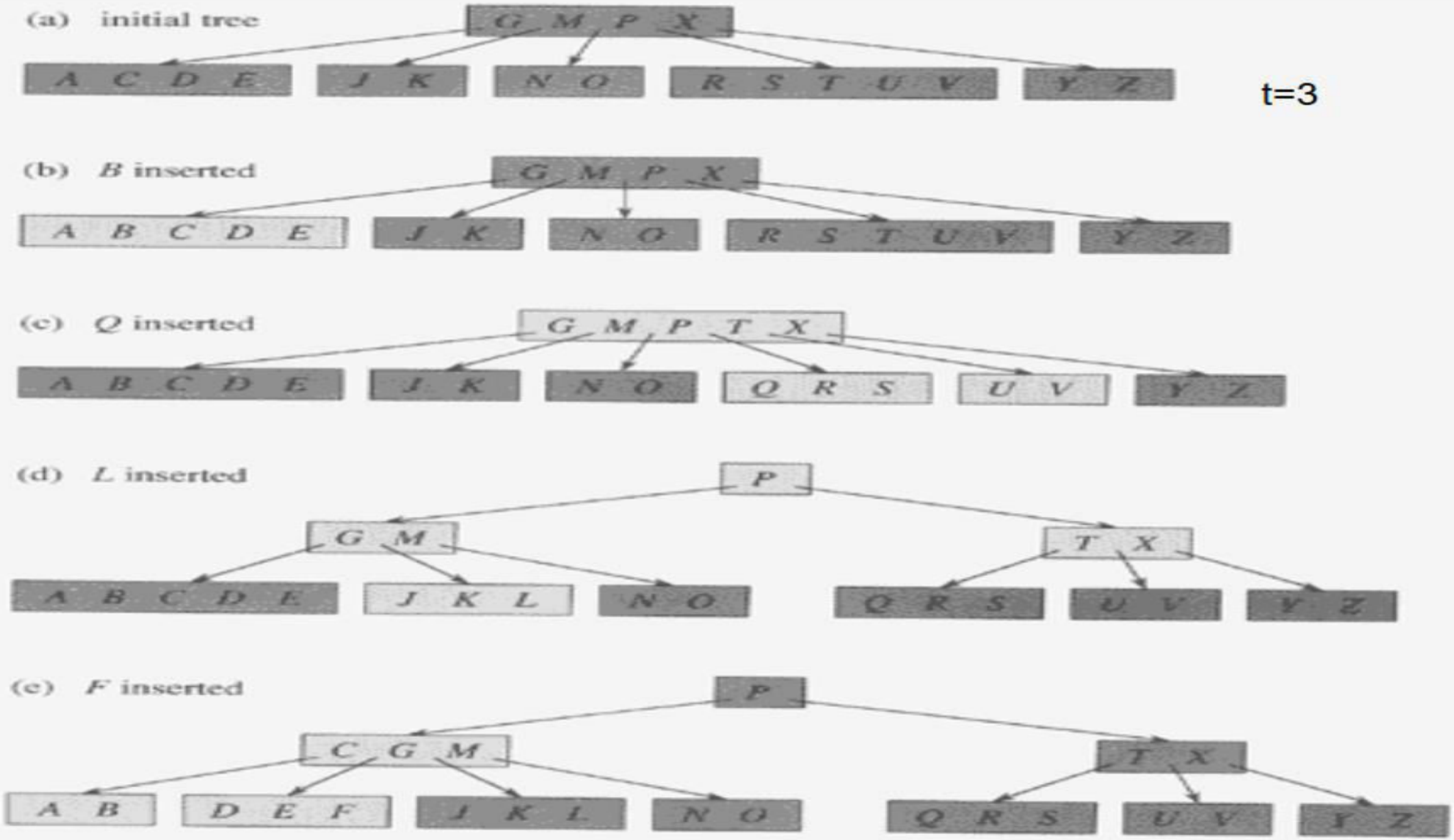


Figure 18.7 Inserting keys into a B-tree. The minimum degree t for this B-tree is 3

Deleting from B-Trees

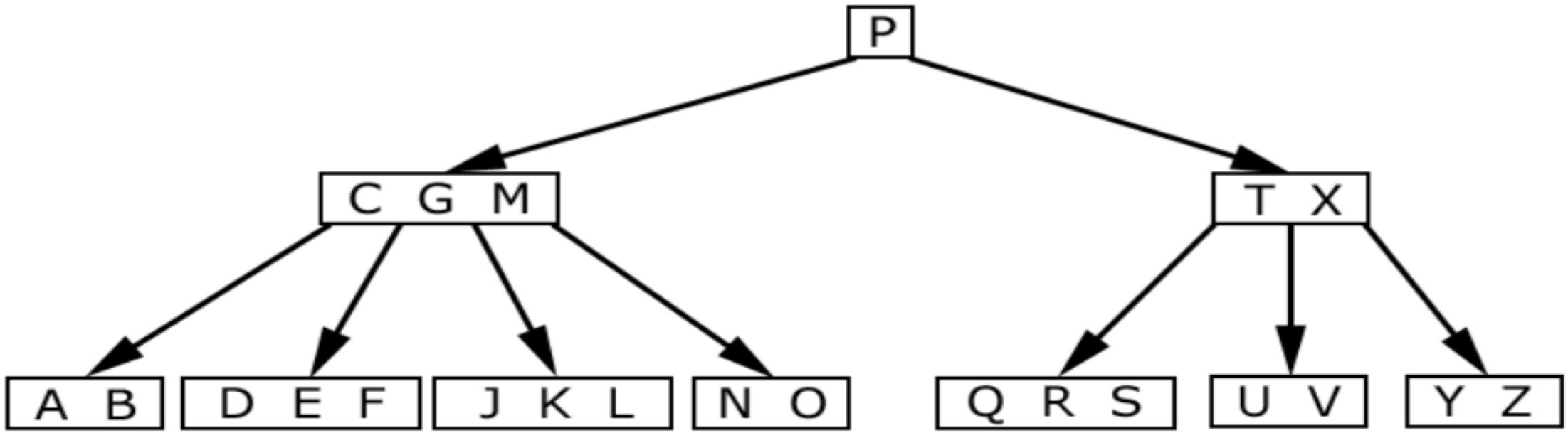
The Concept

- You can delete a key entry from any node.
- ->Therefore, you must ensure that before/after deletion, the B-Tree maintains its properties.
- When deleting, you have to ensure that a node doesn't get *too small* (minimum node size is T – 1). We prevent this by *combining* nodes together.

Lets look at an example:

We're given this valid B-Tree

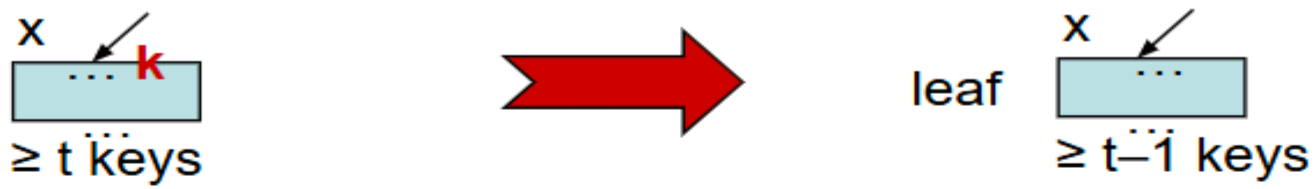
Note: T = 3



Source: Introduction to Algorithms, Thomas H. Cormen

Deletion Cases

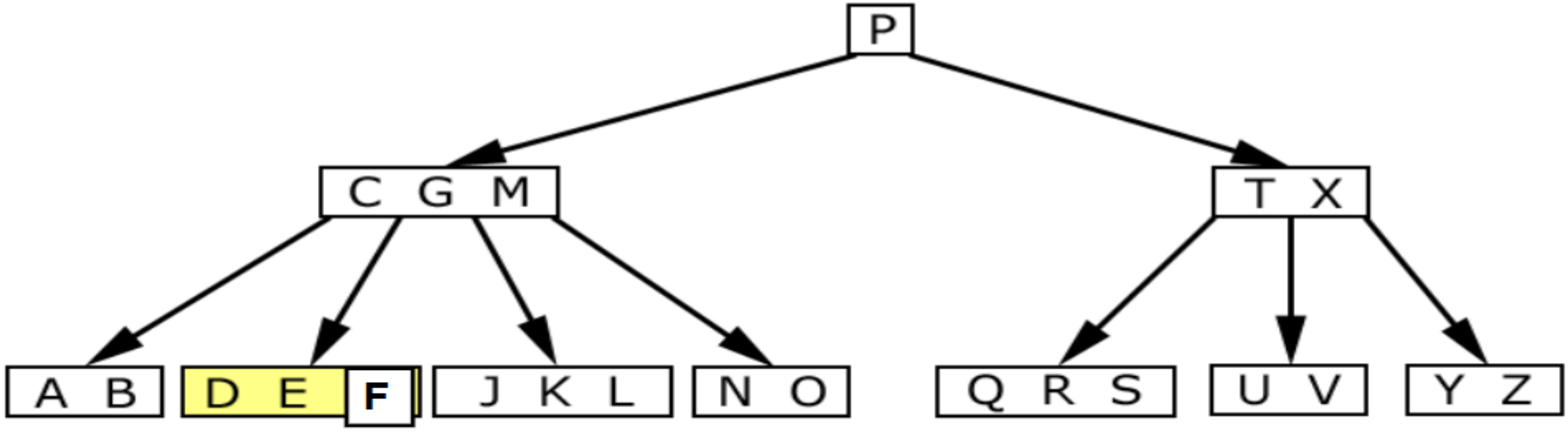
- **Case 1:** If the key k is in node x and x is a leaf node having atleast t keys - then delete k from x.



Simple Deletion

Case 1: We delete “F”

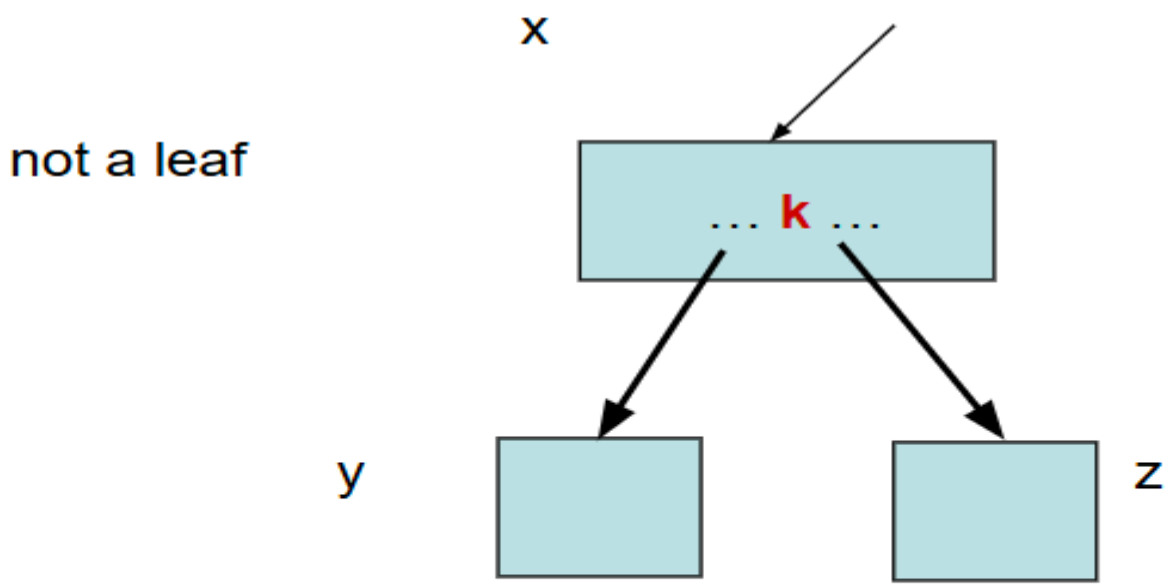
Result: We remove “F” from the leaf node. No further action needed.



Source: Introduction to Algorithms, Thomas H. Cormen

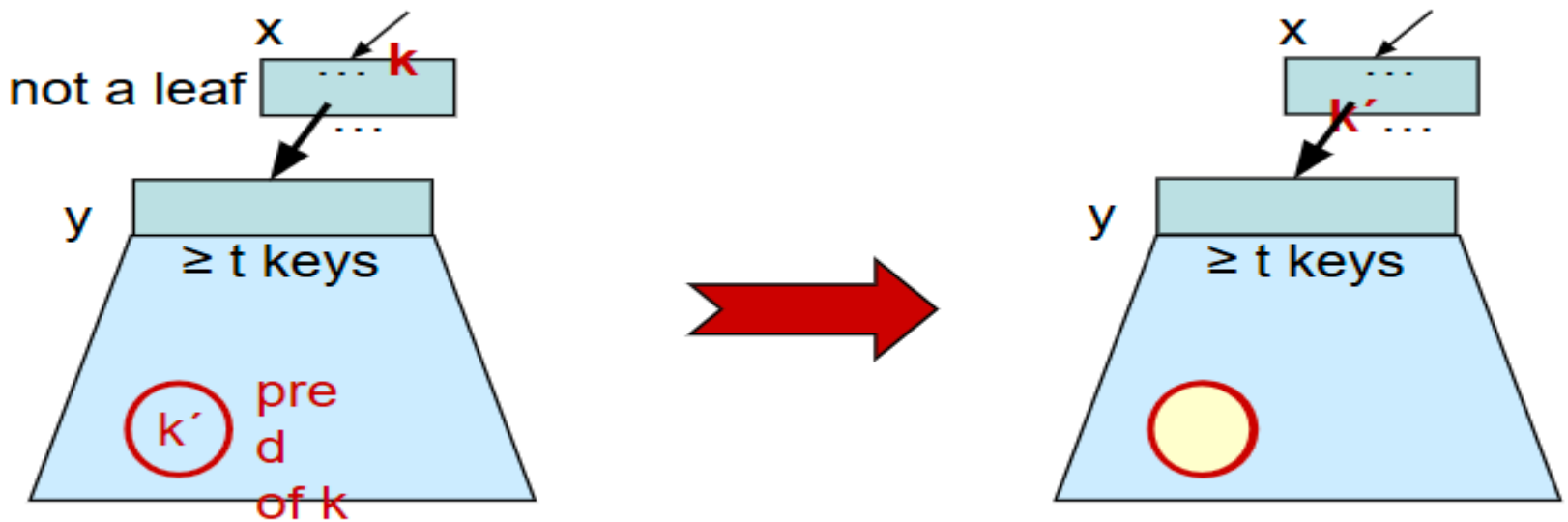
Deletion Cases (Continued)

- **Case 2:** If the child key k is in node x and x is an internal node, do the following:



Deletion Cases (Continued)

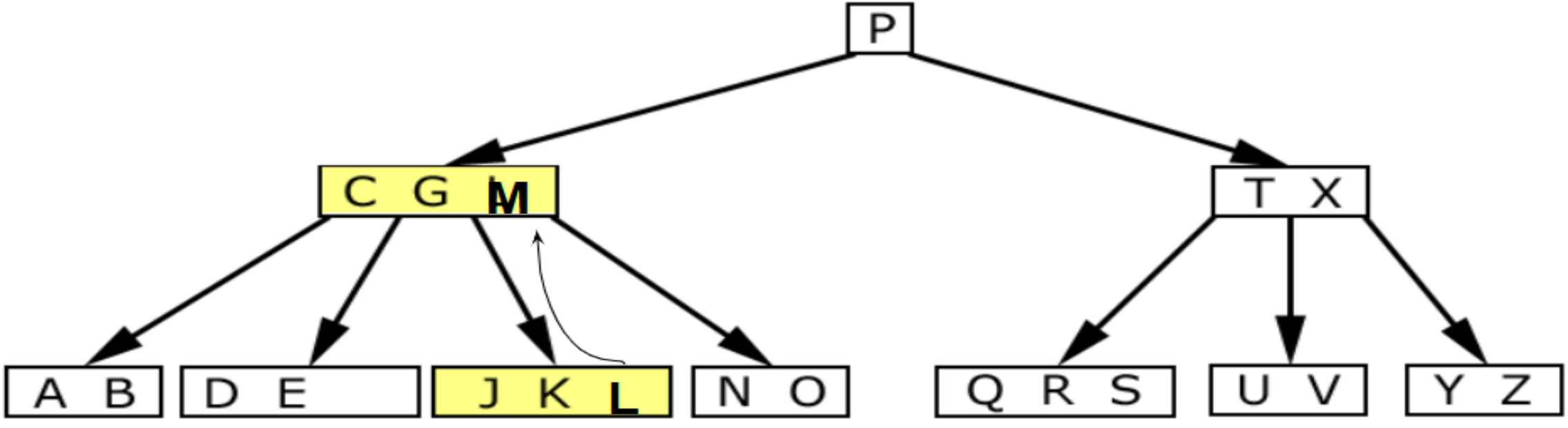
- **Subcase a:** If the child y that precedes k has at least t keys then find predecessor k´ of k in subtree rooted at y, recursively delete k´ and replace k by k´ in x.



Deleting and shifting

Case 2a: We deleted “M”

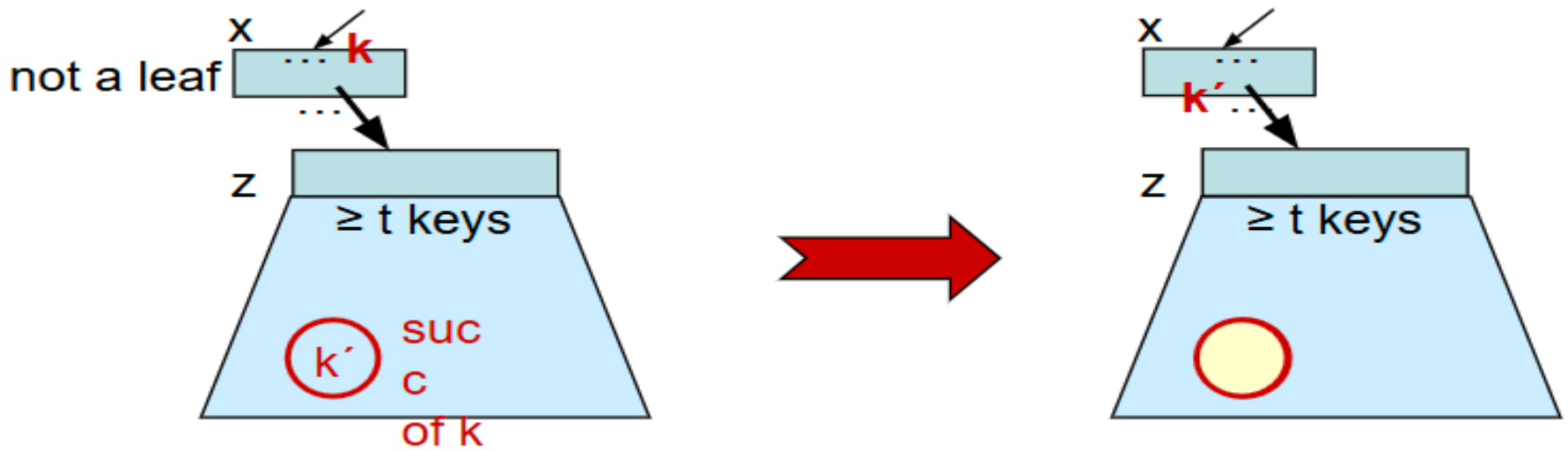
Result: We remove “M” from the parent node. Since there are four nodes and two letters, we move “L” to replace “M”. Now, the “N O” node has a parent again.



Source: Introduction to Algorithms, Thomas H. Cormen

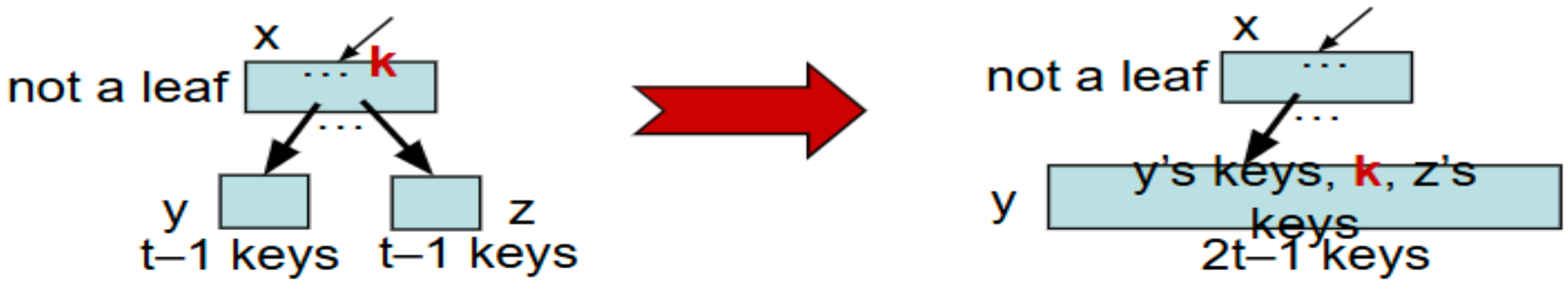
Deletion Cases (Continued)

Subcase B: Symmetrically, if the child z that follows k in node x has at least t keys then find successor k´ of k in subtree rooted at z, recursively delete k´and replace k by k´ in x.



Deletion Cases (Continued)

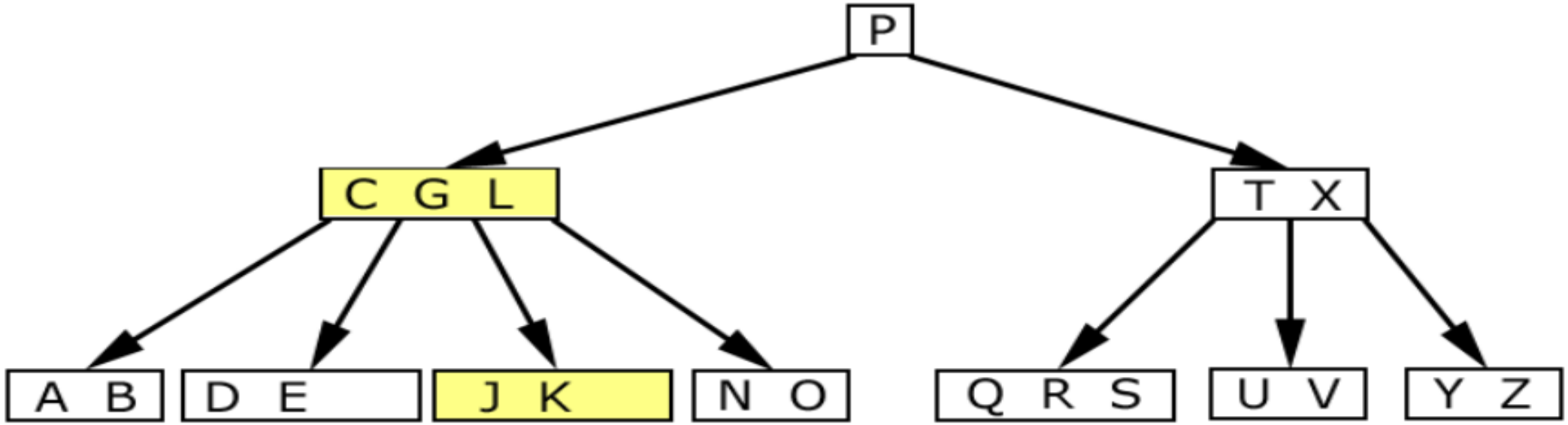
Subcase C: y and z both have t–1 keys -- merge k and z into y, free z, recursively delete k from y.



Combining and Deleting

Case 2c: Now, we delete “G”

Result: First, we combine nodes “DE” and “JK”. Then, we push down “G” into the “DEJK” node and delete it as a leaf.

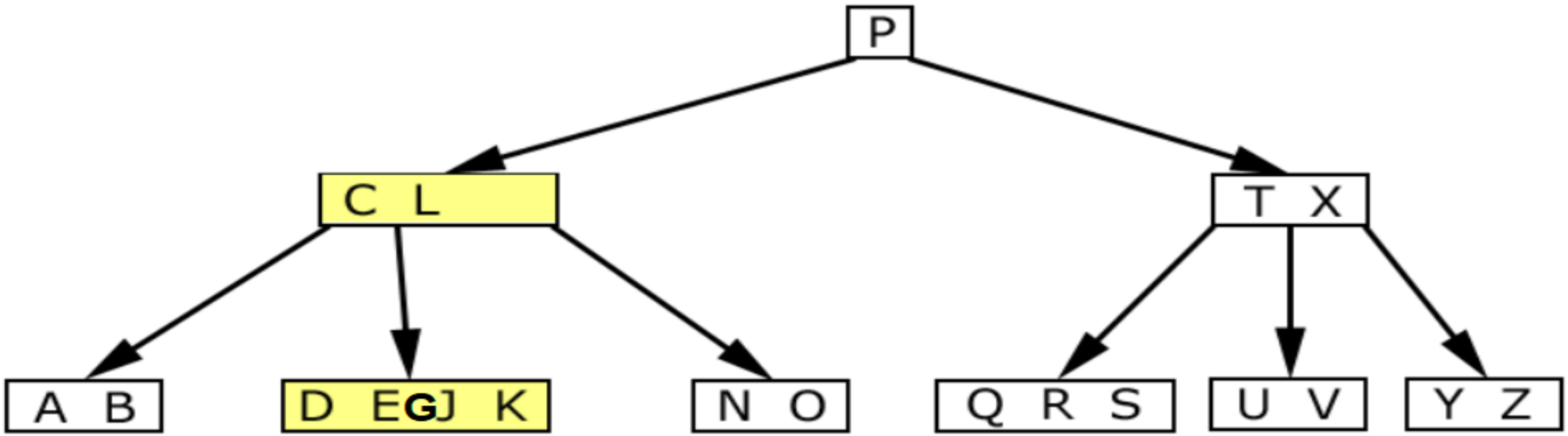


Source: Introduction to Algorithms, Thomas H. Cormen

Combining and Deleting

Case 2c: Now, we delete “G”

Result: First, we combine nodes “DE” and “JK”. Then, we push down “G” into the “DEJK” node and delete it as a leaf.



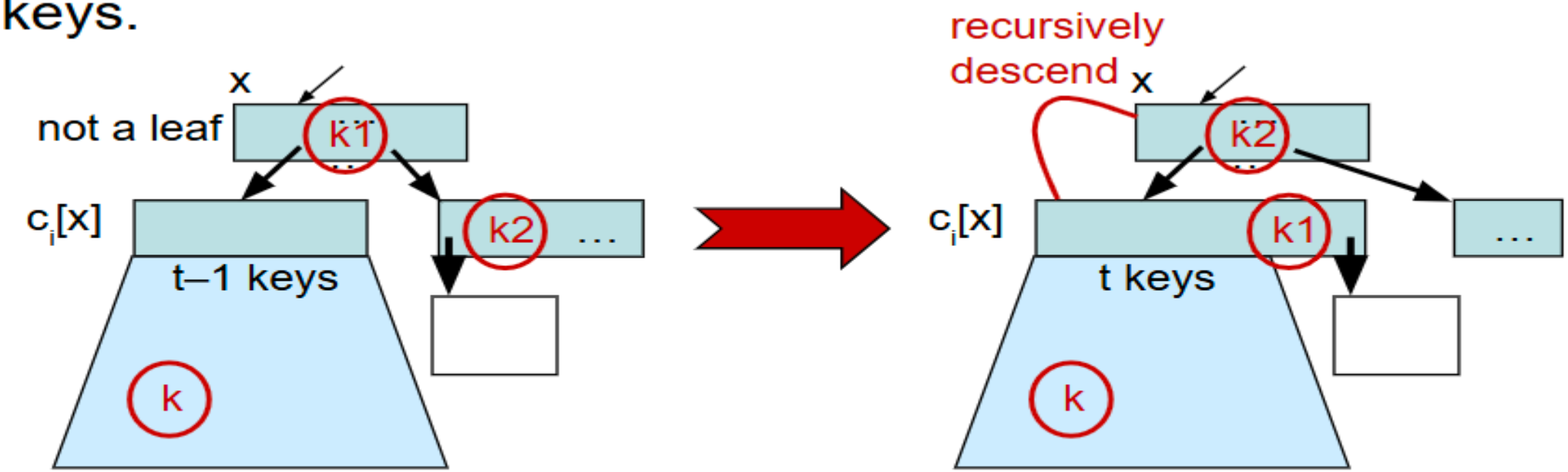
Source: Introduction to Algorithms, Thomas H. Cormen

Deletion Cases (Continued)

Case 3: k not in internal node. Let $c_i[x]$ be the root of the subtree that must contain k, if k is in the tree.
If $c_i[x]$ has at least t keys, then recursively descend; otherwise, execute 3.A and 3.B as necessary.

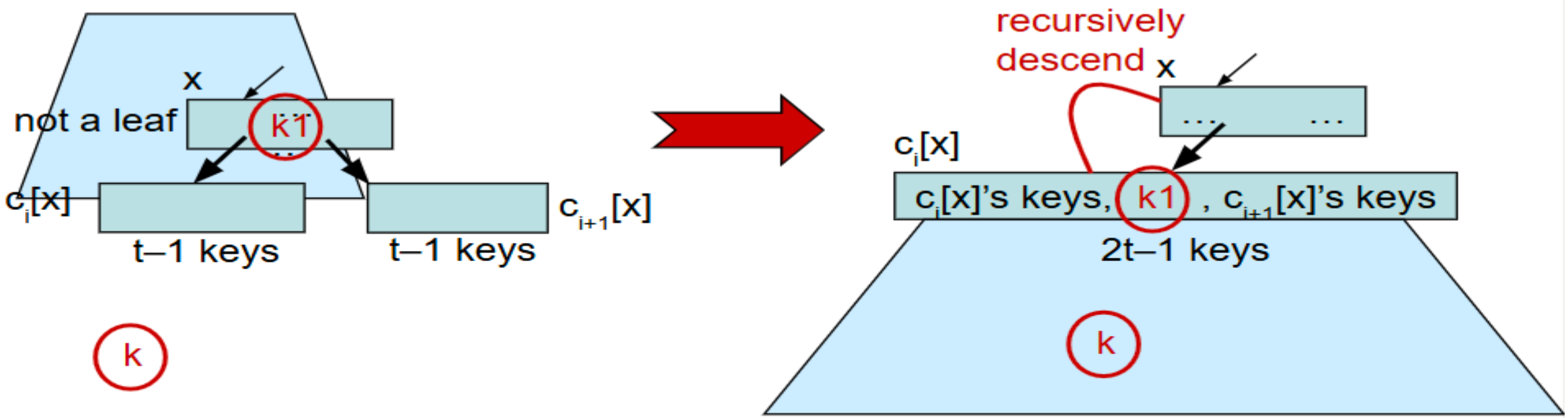
Deletion Cases (Continued)

Subcase A: $c_i[x]$ has t–1 keys, some sibling has at least t keys.



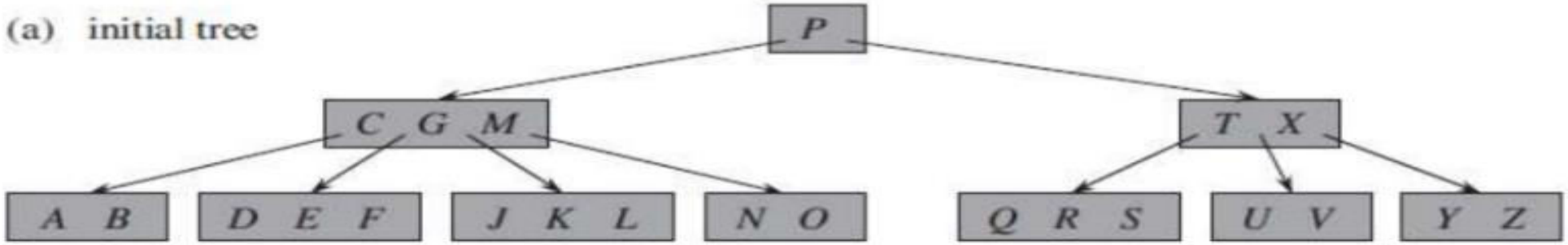
Deletion Cases (Continued)

Subcase B: $c_i[x]$ and sibling both have t–1 keys.

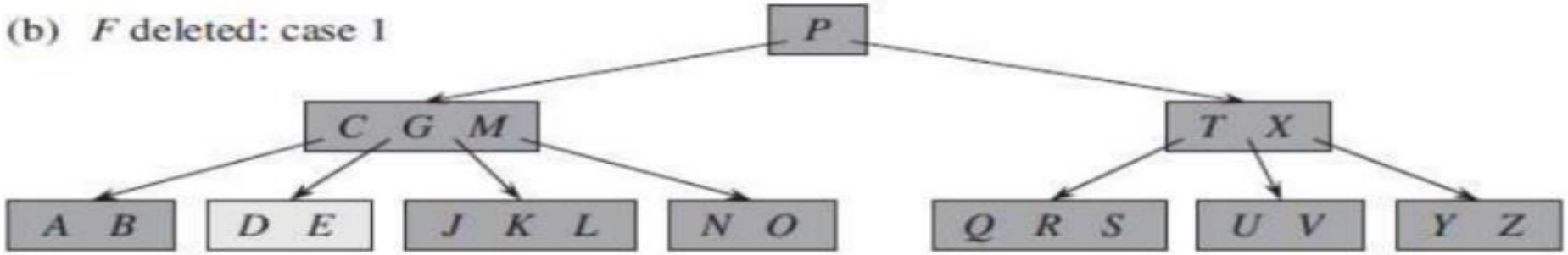


Deleting a key from a B-tree

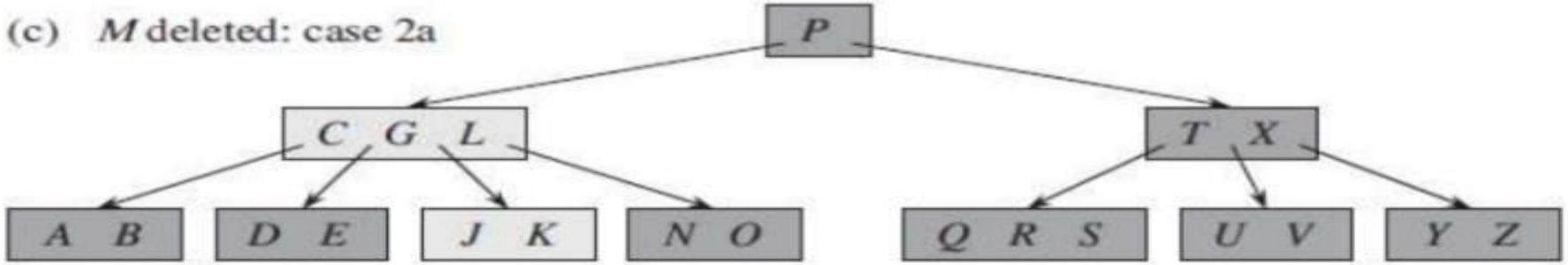
(a) initial tree



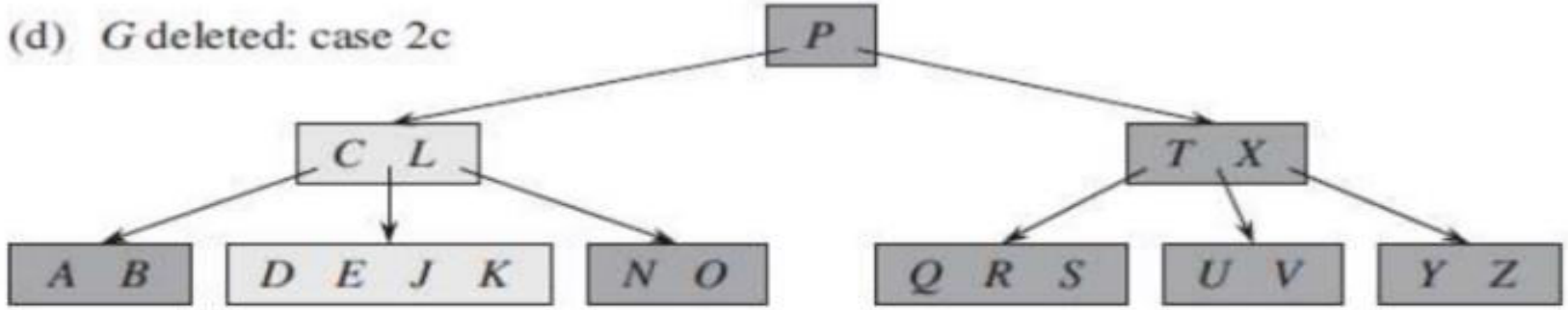
(b) *F* deleted: case 1



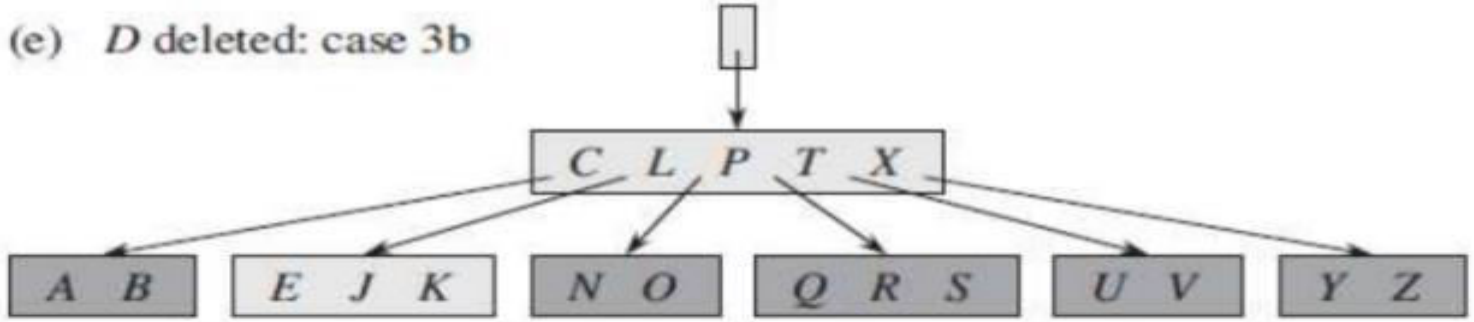
(c) *M* deleted: case 2a



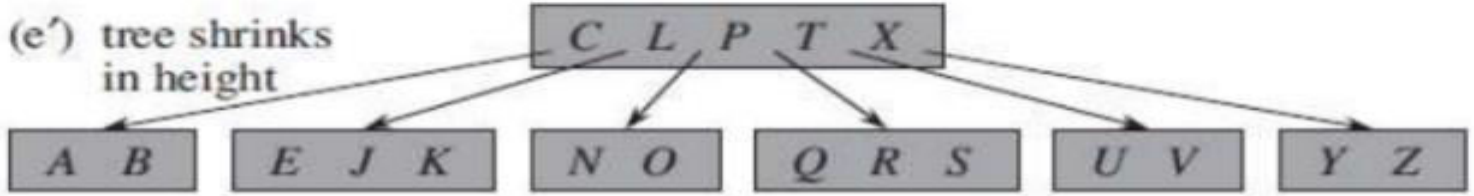
(d) *G* deleted: case 2c



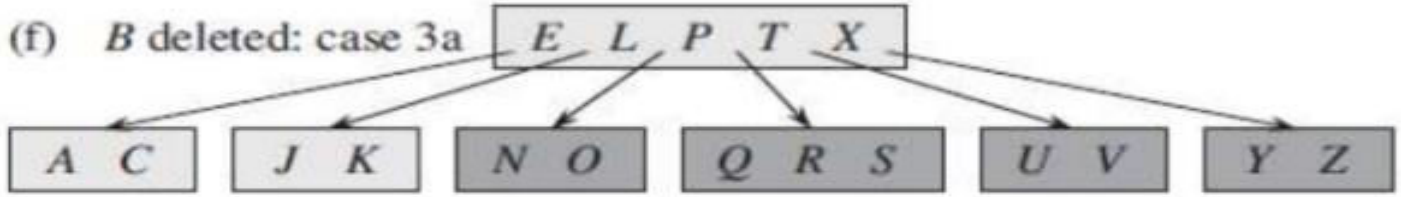
(e) *D* deleted: case 3b



(e′) tree shrinks in height



(f) *B* deleted: case 3a



DESIGN AND ANALYSIS OF ALGORITHMS

Surabhi Narayan

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

B Trees

Surabhi Narayan
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

B Tree

- Self-balancing search tree
- Each node can contain more than one key
- Can have more than two children.
- All data records (or record keys) are stored at the leaves, in increasing order of the keys each parental node contains $n - 1$ ordered keys.

It is also known as a height-balanced m-way tree.

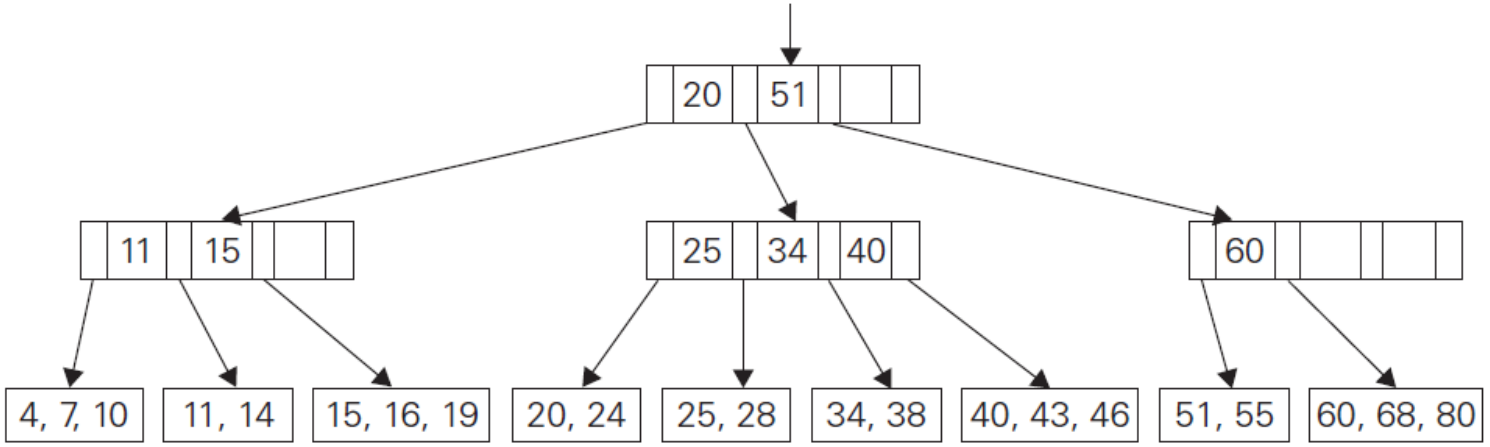


FIGURE 7.8 Example of a B-tree of order 4.

B Tree

Structural properties:

- All leaves are at the same level.
- For each node x, the keys are stored in increasing order.
- If n is the order of the tree, each internal node can contain at most n - 1 keys along with a pointer to each child.
- Each node except root can have at most n children and at least n/2 children.
- All leaves have the same depth (i.e. height-h of the tree).
- The root has at least 2 children and contains a minimum of 1 key.
- If $n \geq 1$, then for any n-key B-tree of height h and minimum degree $t \geq 2$, $h \geq \log_t (n+1)/2$
- B-Tree grows and shrinks from the root which is unlike Binary Search Tree. Binary Search Trees grow downward and also shrink from downward.
- Like other balanced Binary Search Trees, time complexity to search, insert and delete is O(log n).

B Tree

Operations on a B Tree:

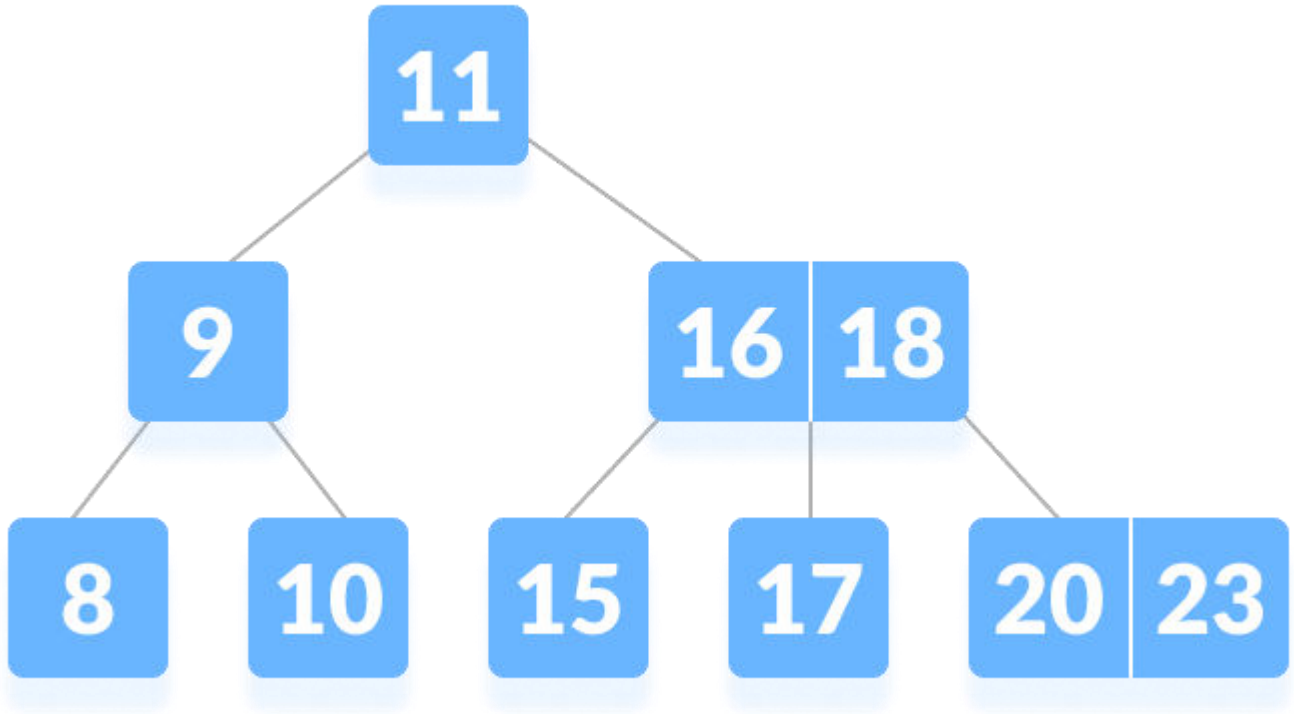
- Searching
- Insertion
- Deletion

Search

1. Starting from the root node, compare k with the first key of the node.
2. If k = the first key of the node, return the node and the index
3. If k.leaf = true, return NULL, i.e. not found
4. If k < the first key of the root node, search the left child of this key recursively.
5. If there is more than one key in the current node and k > the first key compare k with the next key in the node.
6. If k < next key search the left child of this key (ie. k lies in between the first and the second keys). Else, search the right child of the key
7. Repeat steps 1 to 4 until the leaf is reached.

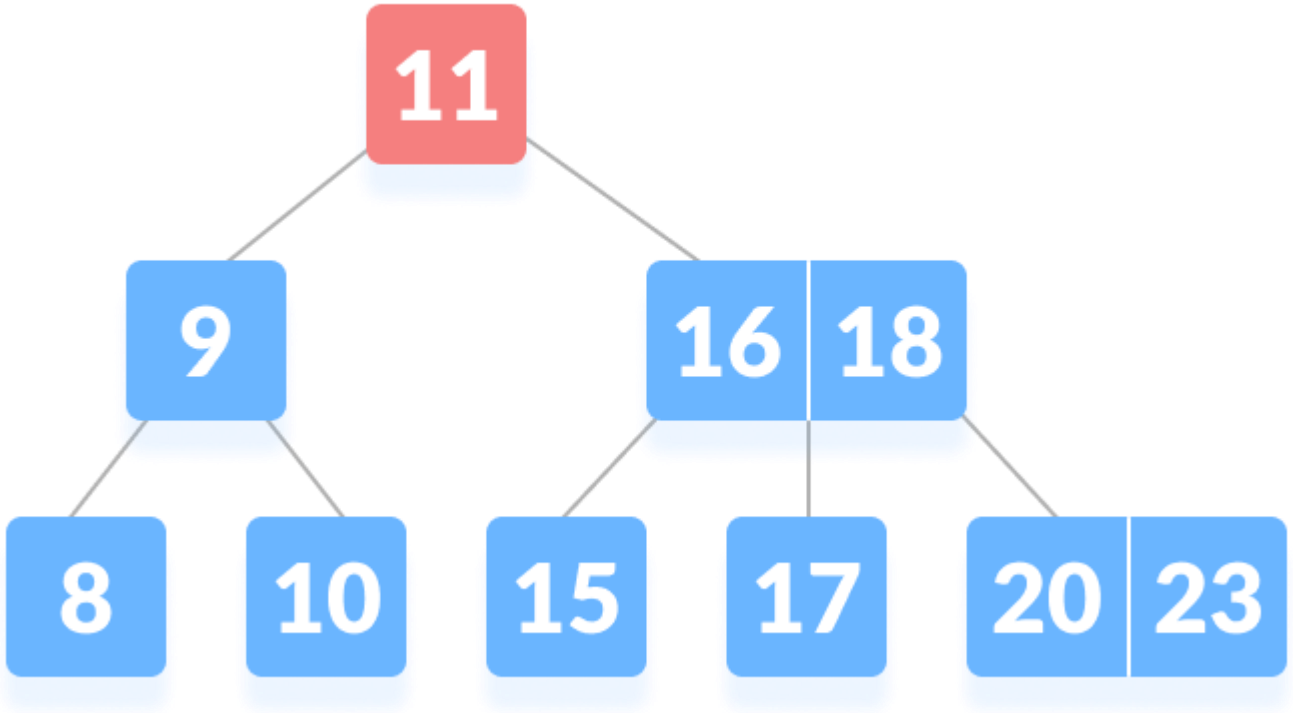
B Tree

Let us search key , k = 17



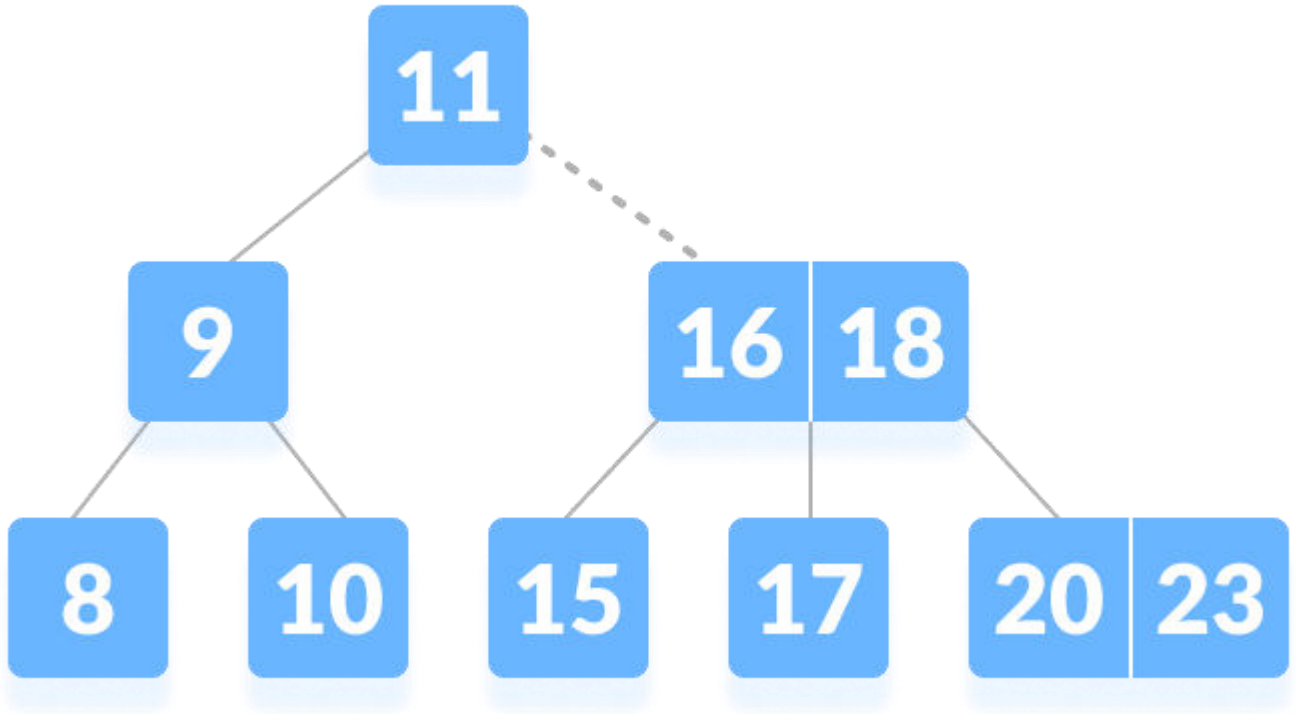
B Tree

K is not found in the root so, compare it with the root key



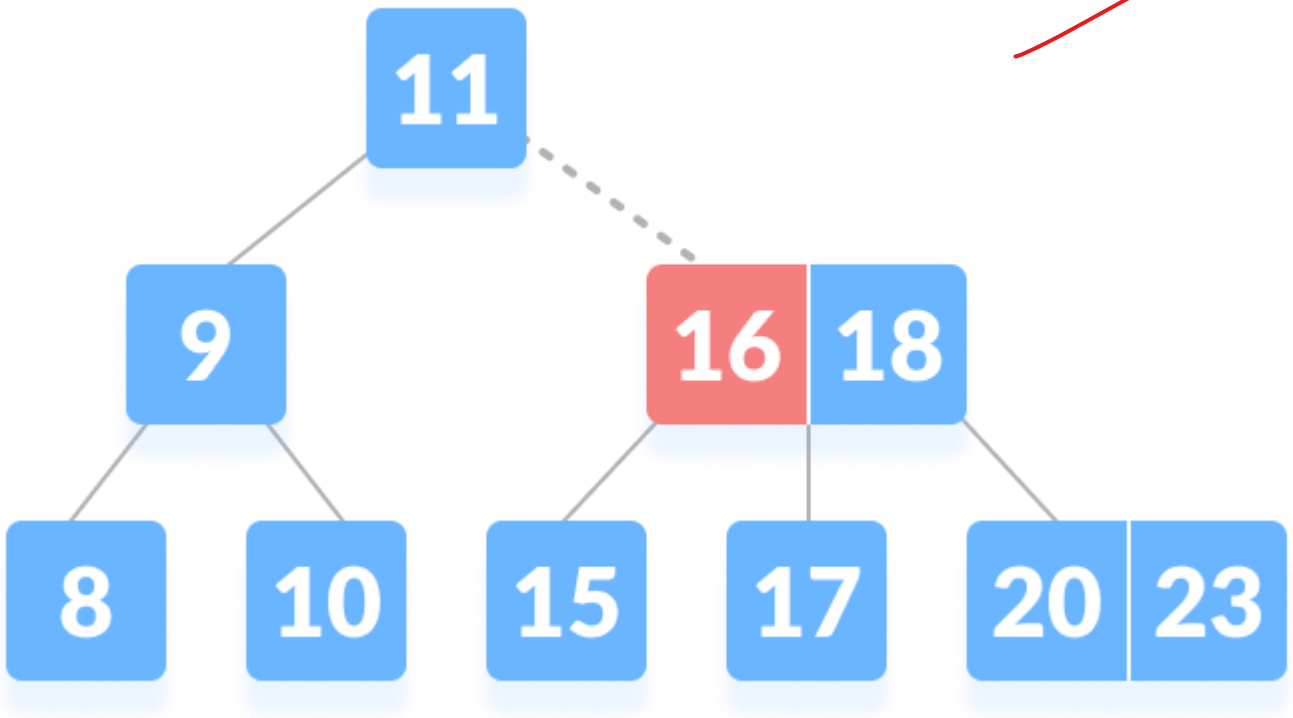
B Tree

Since k > 11 go to the right child of the root node.



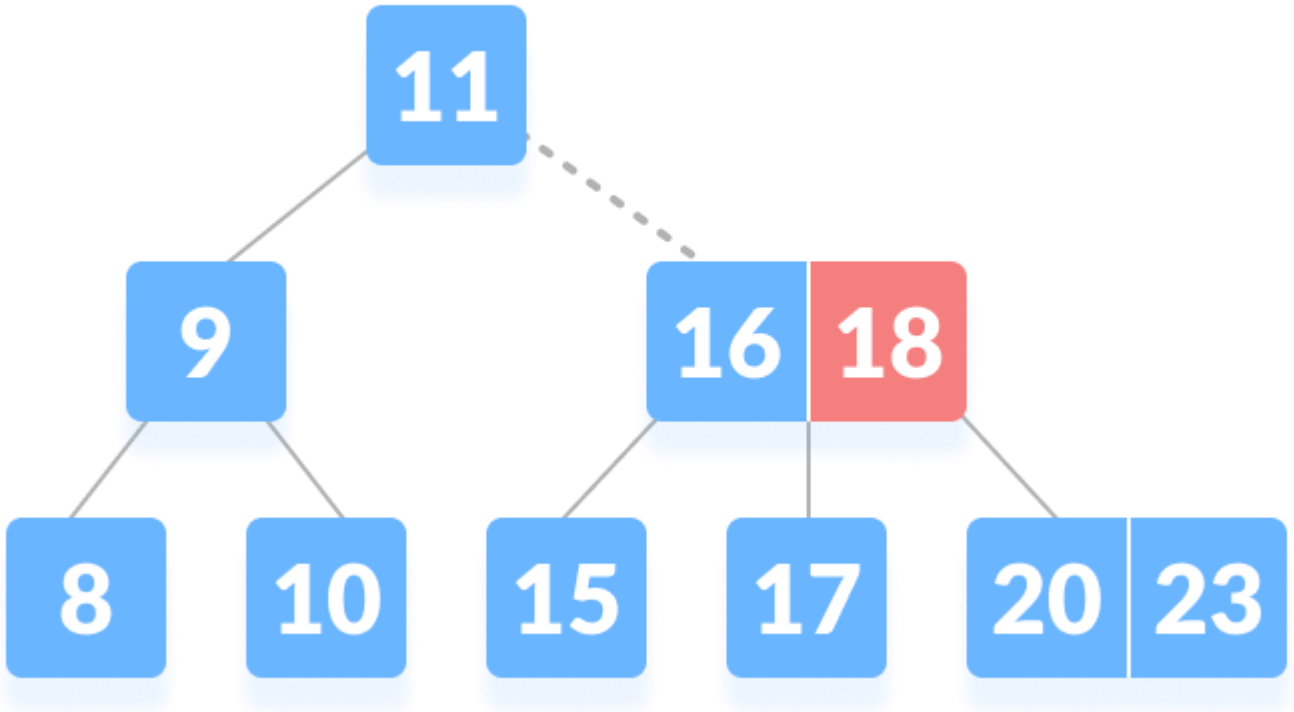
B Tree

Compare k with 16. Since k > 16, compare k with the next key 18



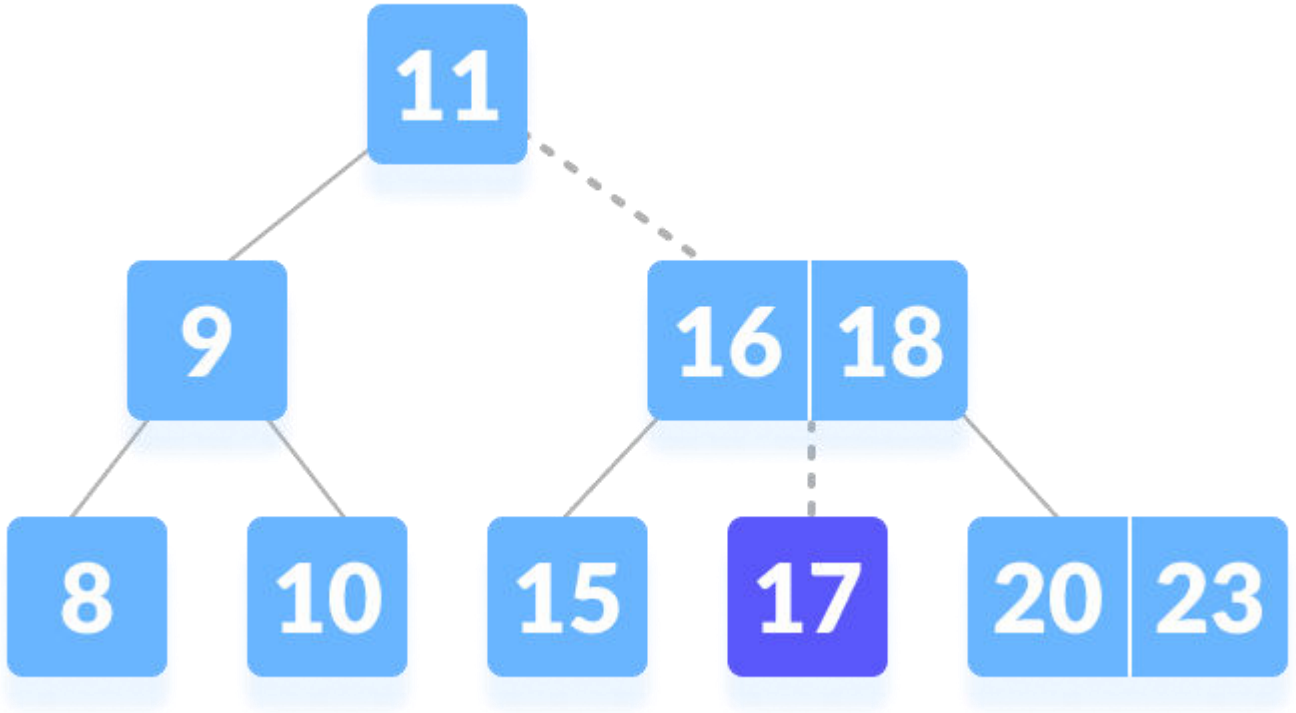
B Tree

Since k < 18, k lies between 16 and 18. Search in the right child of 16 or the left child of 18.



B Tree

k is found



Insertion into a B-tree

Inserting an element on a B-tree consists of two events: **searching the appropriate node** to insert the element and **splitting the node** if required.Insertion operation always takes place in the bottom-up approach.

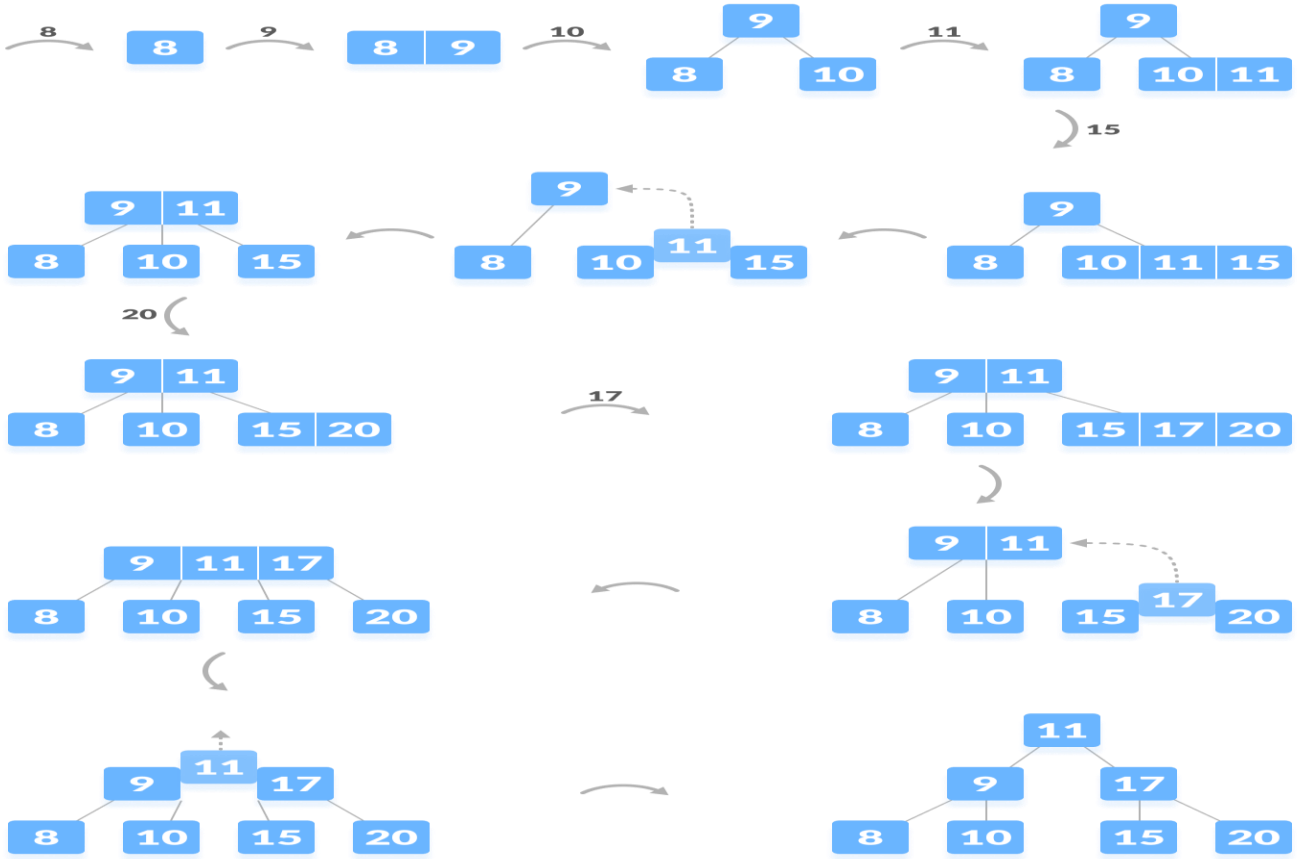
Insertion Operation

1.Traverse the B Tree in order to find the appropriate leaf node at which the node can be inserted.
2.If the leaf node contain less than m-1 keys then insert the element in the increasing order.

3. Else, if the leaf node contains m-1 keys, then follow the following steps.
   - Insert the new element in the increasing order of elements.
   - Split the node into the two nodes at the median.
   - Push the median element upto its parent node.
   - If the parent node also contain m-1 number of keys, then split it too by following the same steps.

DESIGN AND ANALYSIS OF ALGORITHMS

B Tree



Insertion Example

Let us understand the insertion operation with the illustrations below.
The elements to be inserted are 8, 9, 10, 11, 15, 16, 17, 18, 20, 23.

B Tree

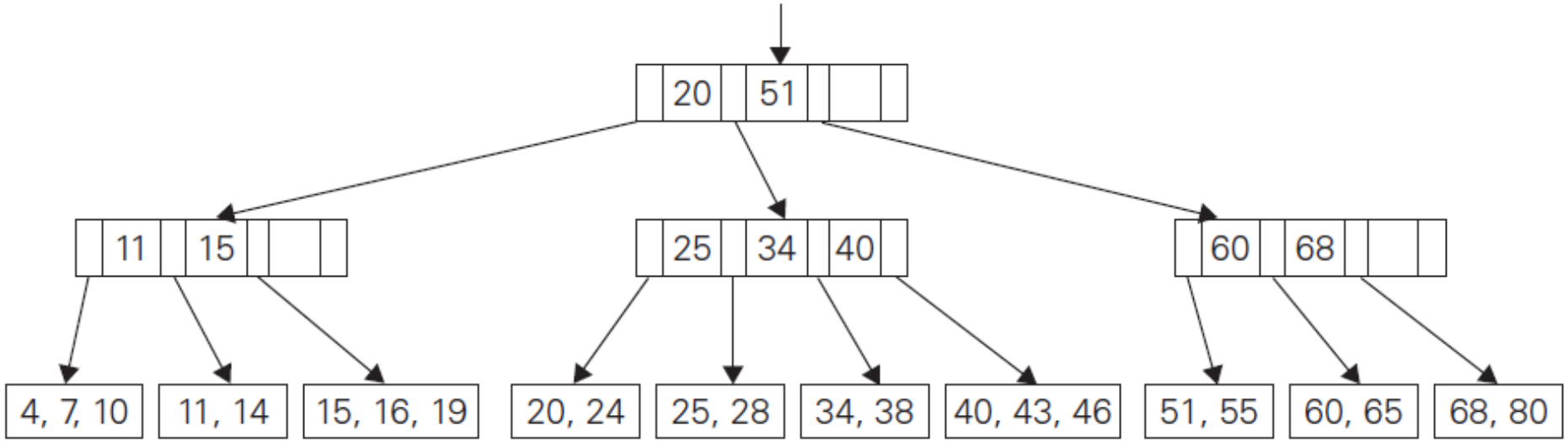


FIGURE 7.9 B-tree obtained after inserting 65 into the B-tree in Figure 7.8.

THANK YOU

Surabhi Narayan
Department of Computer Science &Engineering

surabhinarayan@pes.edu

Design and Analysis of Algorithms
Unit -4

Bharathi R

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Unit 4: Space and Time Tradeoffs

Space and Time Tradeoffs - Sorting by Counting

Bharathi R

Department of Computer Science & Engineering

Design and Analysis of Algorithms

Space and Time Tradeoffs- Introduction

- Space and time trade-offs in algorithm design are a well-known issue for both theoreticians and practitioners of computing.

- As an algorithm design technique, trading space for time is much more prevalent than trading time for space.

In somewhat more general terms, the idea is to preprocess the problem's input, in whole or in part, and store the additional information obtained to accelerate solving the problem afterward. We call this approach input enhancement and discuss the following algorithms based on it: # counting methods for sorting (Section 7.1)

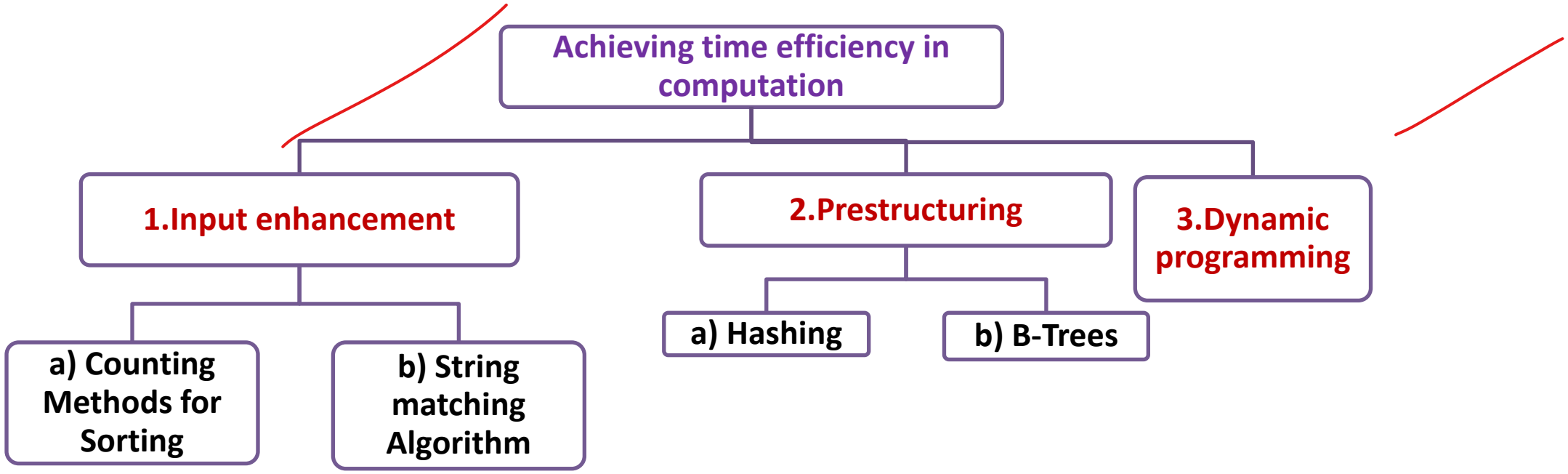
Boyer-Moore algorithm for string matching and its simplified version suggested by Horspool (Section 7.2)

The other type of technique that exploits space-for-time trade-offs simply uses extra space to facilitate faster and/or more flexible access to the data. We call this approach prestructuring. This name highlights two facets of this variation of the space-for-time trade-off: some processing is done before a problem in question is actually solved but, unlike the input-enhancement variety, it deals with access structuring. We illustrate this approach by: #hashing(Section 7.3)

indexingwithB-trees (Section 7.4)

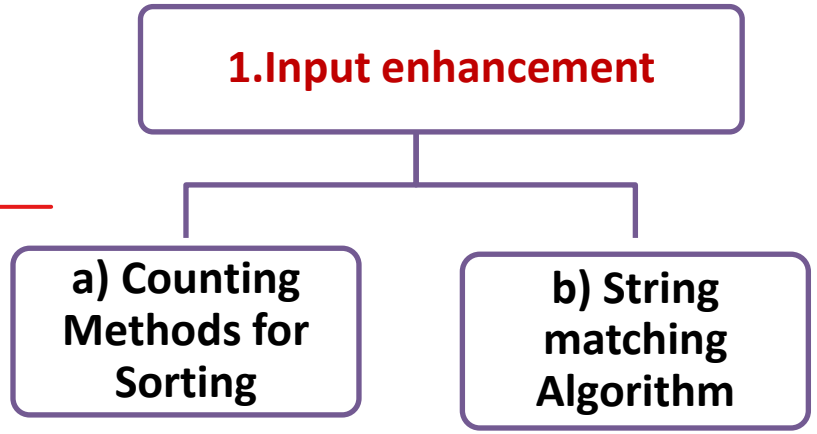
There is one more algorithm design technique related to the space-for-time trade-off idea: dynamic programming. This strategy is based on recording solutions to overlapping subproblems of a given problem in a table from which a solution to the problem in question is then obtained. We discuss this well-developed technique separately, in the next chapter of the book.

Principal Varieties for trading space for time in Algorithm design



1.Input enhancement

- **Input Enhancement**
  - Preprocess the problem's input, in whole or in part, and store the additional information obtained to accelerate solving the problem afterward.
  - Eg:
    1. Comparison counting sort
    2. Distribution Counting,
    3. Horspool's algorithm,
    4. Boyer-Moore's algorithm



1.Input enhancement

a)Sorting by Counting

1. **Comparison Counting Sorting**
   I. For each element of the list, count the total number of elements smaller than this element.
   II. These numbers will indicate the positions of the elements in the sorted list.

2. **Distribution Counting Sorting**
   I. Suppose the elements of the list to be sorted belong to a finite set (aka domain).
   II. Count the frequency of each element of the set in the list to be sorted.
   III. Scan the set in order of sorting and print each element of the set according to its frequency, which will be the required sorted list.

Design and Analysis of Algorithms

1.Input Enhancement

→ Sorting by Counting

→→ Comparison Counting Sorting

1. Find the numbers that are less than a[0] i.e, 62, by scanning the array from the index 1 to 5

	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
Array *a*	62	31	84	96	19	47

2. Maintain another array called ***Count*** the elements that are **lesser than 62**

	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
Array *a*	62	31	84	96	19	47
Array *Count*	**3**					

Step 1: Compare each pair and count
We go through all pairs (i, j) where i < j:
Compare A[0]=62 with others:
62 < 31 --> No --> Count[0]++
62 < 84 --> Yes --> Count[2]++
62 < 96 --> Yes --> Count[3]++
62 < 19 --> No --> Count[0]++
62 < 47 --> No --> Count[0]++
Now:
Count = [3, 0, 1, 1, 0, 0]
Continue the rest:
i = 1 (31):
31 < 84 --> Yes --> Count[2]++
31 < 96 --> Yes --> Count[3]++
31 < 19 --> No --> Count[1]++
31 < 47 --> Yes --> Count[5]++
Now:
Count = [3,1,2,2,0,1]
i = 2 (84):
84 < 96 --> Yes --> Count[3]++
84 < 19 --> No --> Count[2]++
84 < 47 --> No --> Count[2]++
Now:
Count = [3,1,4,3,0,1]
i = 3 (96):
96 < 19 --> No --> Count[3]++
96 < 47 --> No --> Count[3]++
Count = [3,1,4,5,0,1]
i = 4 (19):
19 < 47 --> Yes --> Count[5]++
Count = [3,1,4,5,0,2-->

Final Count[]:
Count = [3, 1, 4, 5, 0, 2]
This means:
A[0] = 62 --> 3rd position
A[1] = 31 --> 1st position
A[2] = 84 --> 4th position
A[3] = 96 --> 5th position
A[4] = 19 --> 0th position
A[5] = 47 --> 2nd position

Step 2: Fill Sorted Array S
S[3] = 62
S[1] = 31
S[4] = 84
S[5] = 96
S[0] = 19
S[2] = 47

Final S[]:
S=[19,31,47,62,84,96]

Example of sorting by comparison counting

Array $A[0..5]$		62	31	84	96	19	47
Initially	*Count* []	0	0	0	0	0	0
After pass $i = 0$	*Count* []	3	0	1	1	0	0
After pass $i = 1$	*Count* []		1	2	2	0	1
After pass $i = 2$	*Count* []			4	3	0	1
After pass $i = 3$	*Count* []				5	0	1
After pass $i = 4$	*Count* []					0	2
Final state	*Count* []	3	1	4	5	0	2

Array $S[0..5]$	19	31	47	62	84	96

Algorithm for Sorting by Counting

ALGORITHM *ComparisonCountingSort*$(A[0..n-1])$

```
//Sorts an array by comparison counting
//Input: An array A[0..n − 1] of orderable elements
//Output: Array S[0..n − 1] of A's elements sorted
for i ← 0 to n − 1 do Count[i] ← 0
for i ← 0 to n − 2 do
    for j ← i + 1 to n − 1 do
        if A[i] < A[j]
            Count[j] ← Count[j] + 1
        else Count[i] ← Count[i] + 1
for i ← 0 to n − 1 do S[Count[i]] ← A[i]
return S
```

Time Efficiency

It should be quadratic because the algorithm considers all the different pairs of an *n*-element array More formally,the number of times its basic operation, the comparison A[i]<A[j], is executed is equal to the sum we have encountered several times already

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

$$= \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}$$

Thus, the algorithm makes the same number of key comparisons as selection sort and in addition uses a linear amount of extra space. On the positive side, the algorithm makes the minimum number of key moves possible, placing each of them directly in their final position in a sorted array.

Drawbacks

1. Thus, the algorithm makes the same number of key comparisons as selection sort,
2. In addition it uses a linear amount of extra space.

References

"Introduction to the Design and Analysis of Algorithms", Anany Levitin, Pearson Education, Delhi (Indian Version), 3rd edition, 2012.
Chapter- 7

THANK YOU

Bharathi R

Department of Computer Science & Engineering

rbharathi@pes.edu

Before reading this slide have a look at the slide:DAA_UE23CS241B_U3_S50_Sort_by_Counting
https://drive.google.com/file/d/1UW2rmPHdtvIFOiiF_RpvqEHpgFM1nBpl/view?usp=drive_link

Design and Analysis of Algorithms
Unit -4

Bharathi R

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Unit 4: Space and Time Tradeoffs

Distribution Counting Sort

Slides from **Anany Levitin**

Bharathi R
Department of Computer Science & Engineering

Design and Analysis of Algorithms

1.Input enhancement

a)Sorting by Counting

1. **Comparison Counting Sorting**
   I. For each element of the list, count the total number of elements smaller than this element.
   II. These numbers will indicate the positions of the elements in the sorted list.

2. **Distribution Counting Sorting**
   I. Suppose the elements of the list to be sorted belong to a finite set (aka domain).
   II. Count the frequency of each element of the set in the list to be sorted.
   III. Scan the set in order of sorting and print each element of the set according to its frequency, which will be the required sorted list.

1. Input Enhancement

→ Sorting by Counting →→ ii)Distribution Counting Sorting

- A sorting method in which, with the help of some **associated information** of the elements , the elements can be placed in an array at their relative positions.

- The required information which is used to place the elements at proper positions is **accumulated sum of frequencies** which is also called as distribution in statistics.

- Hence this method is called as **Distribution counting method** for sorting.

1. Input Enhancement

→ Sorting by Counting →→ ii)Distribution Counting Sorting

Consider sorting the array whose values are known to come from the set {11, 12, 13} and should not be overwritten in the process of sorting. The frequency and distribution arrays are as follows:

13	11	12	13	12	12

Array values	11	12	13
Frequencies	1	3	2
Distribution values	1	4	6

11	12	12	12	13	13

Design and Analysis of Algorithms

Example of sorting by distribution counting.

Let us consider a more realistic situation of sorting a list of items with some other information associated with their keys so that we cannot overwrite the list's elements.Then we can copy elements into a new array S[0..n−1] to hold the sorted list as follows. The elements of A whose values are equal to the lowest possible value l are copied into the first F[0] elements of S,i.e.,positions 0 through F[0]− 1; the elements of value l +1 are copied to positions from F[0]to (F[0]+ F[1]) −1; and so on. Since such accumulated sums of frequencies are called a distribution in statistics, the method itself is known as distribution counting

13	11	12	13	12	12

Array values	11	12	13
Frequencies	1	3	2
Distribution values	1	4	6

11	12	12	12	13	13

The distribution values being decremented are shown in bold.

	$D[0..2]$		
$A[5] = 12$	1	**4**	6
$A[4] = 12$	1	**3**	6
$A[3] = 13$	1	2	**6**
$A[2] = 12$	1	**2**	5
$A[1] = 11$	**1**	1	5
$A[0] = 13$	0	1	**5**

$S[0..5]$

			12		
		12			
					13
	12				
11					
				13	

Design and Analysis of Algorithms

Algorithm for Distribution Counting

ALGORITHM *DistributionCountingSort*$(A[0..n-1], l, u)$

//Sorts an array of integers from a limited range by distribution counting
//Input: An array $A[0..n-1]$ of integers between l and u $(l \le u)$
//Output: Array $S[0..n-1]$ of A's elements sorted in nondecreasing order

```
for j ← 0 to u − l do D[j] ← 0                    //initialize frequencies
for i ← 0 to n − 1 do D[A[i] − l] ← D[A[i] − l] + 1 //compute frequencies
for j ← 1 to u − l do D[j] ← D[j − 1] + D[j]      //reuse for distribution
for i ← n − 1 downto 0 do
    j ← A[i] − l
    S[D[j] − 1] ← A[i]
    D[j] ← D[j] − 1
return S
```

here k= u - l+1

efficiency of this loop --> O(k)

efficiency of this loop --> O(n)

efficiency of this loop --> O(k)

efficiency of this loop --> O(n)

cumulative values

we are reading backwards to maintain stability.

overall time efficiency : O(k) + O(n) + O(k) + O(n)= O(n)

Distribution Counting Sorting

1. This is a better time-efficiency class than that of the most efficient sorting algorithms—mergesort, quicksort, and heapsort—we have encountered.

2. It is important to remember, however, that this efficiency is obtained by exploiting the specific nature of inputs for which sorting by distribution counting works, in addition to trading space for time.

see your notebook before exam (CW)

References

"Introduction to the Design and Analysis of Algorithms", Anany Levitin, Pearson Education, Delhi (Indian Version), 3rd edition, 2012. Chapter- 7

THANK YOU

Bharathi R

Department of Computer Science & Engineering

rbharathi@pes.edu

Design and Analysis of Algorithms
Unit -4

Bharathi R

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Unit 4: Space and Time Tradeoffs

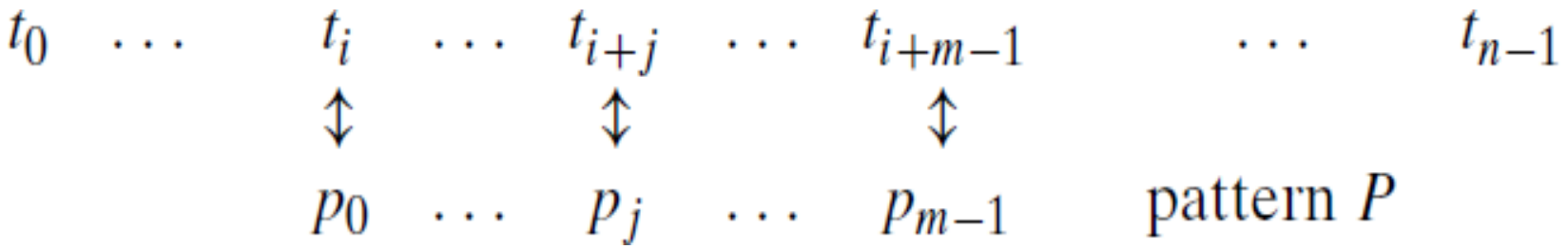
Input Enhancement in String Matching

Bharathi R

Department of Computer Science & Engineering

String Matching

String matching requires finding an occurrence of a given string of *m* characters called the ***pattern*** in a longer string of *n* characters called the ***text***.

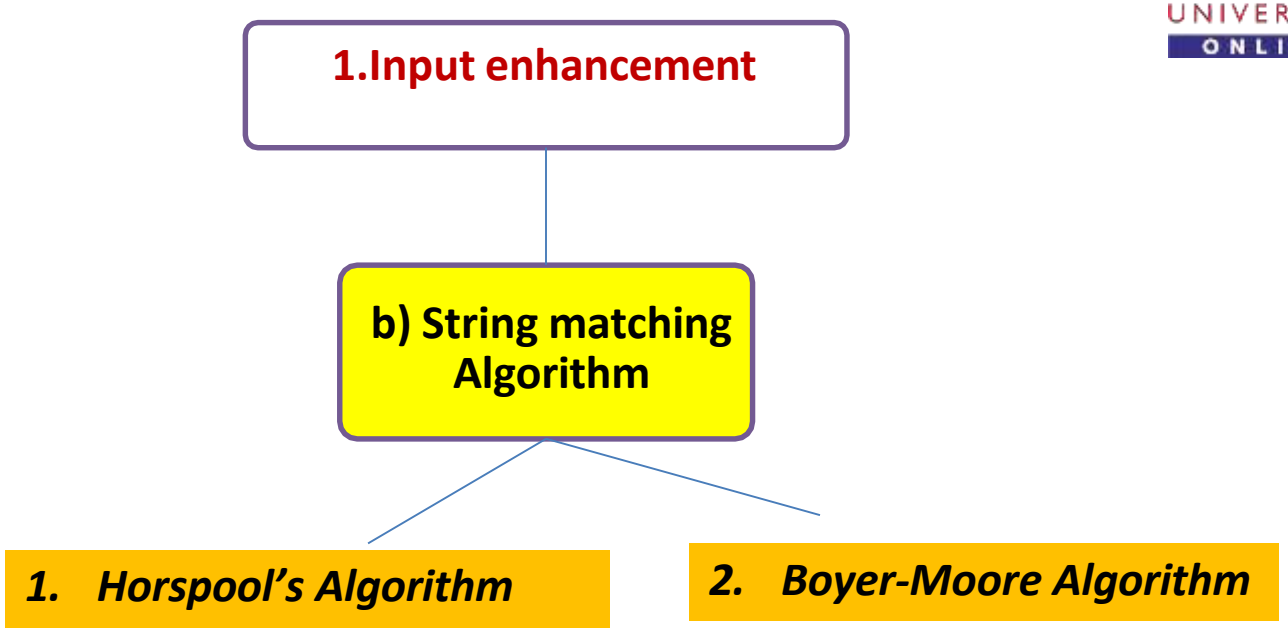


Design and Analysis of Algorithms

Input Enhancement in String Matching

Input-enhancement idea:

Preprocess the pattern to get some information about it, store this information in a table, and then use this information during an actual search for the pattern in a given text.



String matching by Brute force

pattern: a string of *m* characters to search for
text**:** a (long) string of *n* characters to search in

Brute force algorithm

Step 1 Align pattern at beginning of text

Step 2 Moving from left to right, compare each character of pattern to the corresponding character in text until either all characters are found to match (successful search) or a mismatch is detected

Step 3 While a mismatch is detected and the text is not yet exhausted, realign pattern one position to the right and repeat Step 2

Time complexity (worst-case): O(mn)

Since the maximum number of such trials is n − m+1and, in the worst case, m comparisons need to be made on each of them, the worst-case efficiency of the brute-force algorithm is in the O(nm) class. On average, however ,we should expect just a few comparisons before a pattern's shift, and for random natural-language texts, the average-case efficiency indeed turns out to be in O(n+ m).

Design and Analysis of Algorithms

String matching by preprocessing

Several string searching algorithms are based on the input enhancement idea of preprocessing the **pattern**

1. **Knuth-Morris-Pratt (KMP)** algorithm preprocesses pattern left to right to get useful information for later searching

2. **Boyer -Moore algorithm** preprocesses pattern right to left and store information into two tables

3. **Horspool's algorithm** simplifies the Boyer-Moore algorithm by using just one table

Horspool's Algorithm

A simplified version of Boyer-Moore algorithm:

Preprocesses pattern to generate a "shift table" that determines how much to shift the pattern when a mismatch occurs

Always makes a shift based on the text's character **c** aligned with the last compared (mismatched) character in the pattern according to the shift table's entry for ***c***

How far to shift?

Look at first (rightmost) character in text that was compared:

1. The character is not in the pattern

```
    ........c..... (c not in pattern)
            ≠
    BAOBAB
```

2. The character is in the pattern (but not the rightmost)

```
    .....O..... (O occurs once in pattern)
    BAOBAB

    .....A..... (A occurs twice in pattern)
    BAOBAB
```

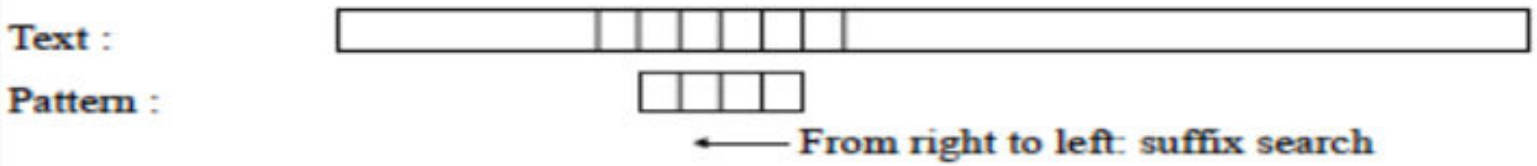
3. The rightmost characters do match

```
    .....B.....
    BAOBAB
```

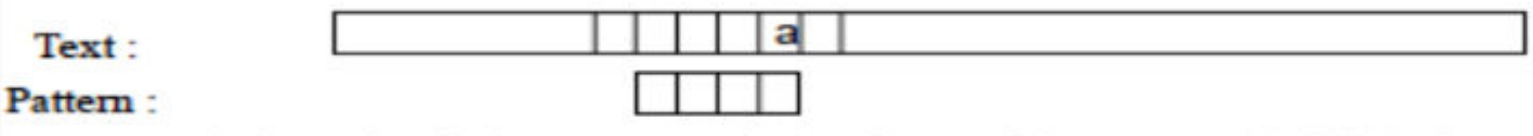
Horspool's Algorithm

String Matching: Horspool algorithm

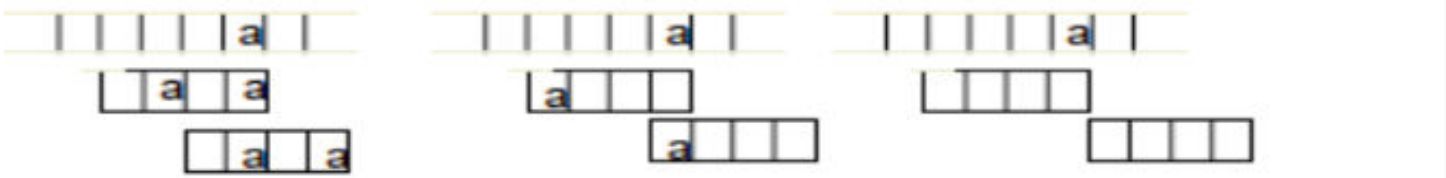
- How the comparison is made?



- Which is the next position of the window?



It depends of where appears the last letter of the text, say it 'a', in the pattern:



Then it is necessary a preprocess that determines the length of the shift.

Design and Analysis of Algorithms

Shift Table

Shift sizes can be precomputed by the formula

$$t(c) = \begin{cases} \text{the pattern's length } m, \\ \text{if } c \text{ is not among the first } m-1 \text{ characters of the pattern;} \\ \\ \text{the distance from the rightmost } c \text{ among the first } m-1 \text{ characters} \\ \text{of the pattern to its last character, otherwise.} \end{cases}$$

By scanning pattern before search begins and stored in a table called ***shift table***. After the shift, the right end of pattern is *t(c)* positions to the right of the last compared character in text.

Shift table is indexed by text and pattern alphabet Eg: for **BAOBAB**:

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	2	6	6	6	6	6	6	6	6	6	6	6	6	3	6	6	6	6	6	6	6	6	6	6	6

Design and Analysis of Algorithms

Shift Table

$$t(c) = \begin{cases} \text{the pattern's length } m, \\ \text{if } c \text{ is not among the first } m-1 \text{ characters of the pattern;} \\ \\ \text{the distance from the rightmost } c \text{ among the first } m-1 \text{ characters} \\ \text{of the pattern to its last character, otherwise.} \end{cases}$$

ALGORITHM *ShiftTable*$(P[0..m-1])$

//Fills the shift table used by Horspool's and Boyer-Moore algorithms
//Input: Pattern $P[0..m-1]$ and an alphabet of possible characters
//Output: $Table[0..size-1]$ indexed by the alphabet's characters and
// filled with shift sizes computed by formula (7.1)
for $i \leftarrow 0$ **to** $size-1$ **do** $Table[i] \leftarrow m$
for $j \leftarrow 0$ **to** $m-2$ **do** $Table[P[j]] \leftarrow m-1-j$
return *Table*

size here is 26 charcters+1 blank character

Horspool Matching

ALGORITHM *HorspoolMatching*$(P[0..m-1], T[0..n-1])$

```
//Implements Horspool's algorithm for string matching
//Input: Pattern P[0..m − 1] and text T[0..n − 1]
//Output: The index of the left end of the first matching substring
//        or −1 if there are no matches
ShiftTable(P[0..m − 1])          //generate Table of shifts
i ← m − 1                        //position of the pattern's right end
while i ≤ n − 1 do
    k ← 0                        //number of matched characters
    while k ≤ m − 1 and P[m − 1 − k] = T[i − k] do
        k ← k + 1
    if k = m
        return i − m + 1
    else i ← i + Table[T[i]]
return −1
```

returns the index where the occurrence of word was found

Horspool Matching -Example

$s_0 \quad \ldots \quad c \quad \ldots \quad s_{n-1}$

B A R B E R

character c	A	B	C	D	E	F	. . .	R	. . .	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6

J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
B A R B E R
B A R B E R
B A R B E R
B A R B E R

Design and Analysis of Algorithms

Example of Horspool's algorithm

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	-
1	2	6	6	6	6	6	6	6	6	6	6	6	6	3	6	6	6	6	6	6	6	6	6	6	6	6

```
BARD LOVED BANANAS
BAOBAB
      BAOBAB
           BAOBAB
                    BAOBAB (unsuccessful search)
```

Horspool Matching

The worst-case efficiency of Horspool's algorithm is in O(nm) But for random texts, it is in $\Theta(n)$ and, although in the same efficiency class, Horspool's algorithm is obviously faster on average than the brute-force algorithm.

References

"Introduction to the Design and Analysis of Algorithms", Anany Levitin, Pearson Education, Delhi (Indian Version), 3rd edition, 2012. Chapter- 7

THANK YOU

Bharathi R

Department of Computer Science & Engineering

rbharathi@pes.edu

Design and Analysis of Algorithms
Unit -4

Bharathi R
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Unit 4: Space and Time Tradeoffs

Input Enhancement in String Matching- The Boyer-Moore Algorithm

Bharathi R

Department of Computer Science & Engineering

String Matching

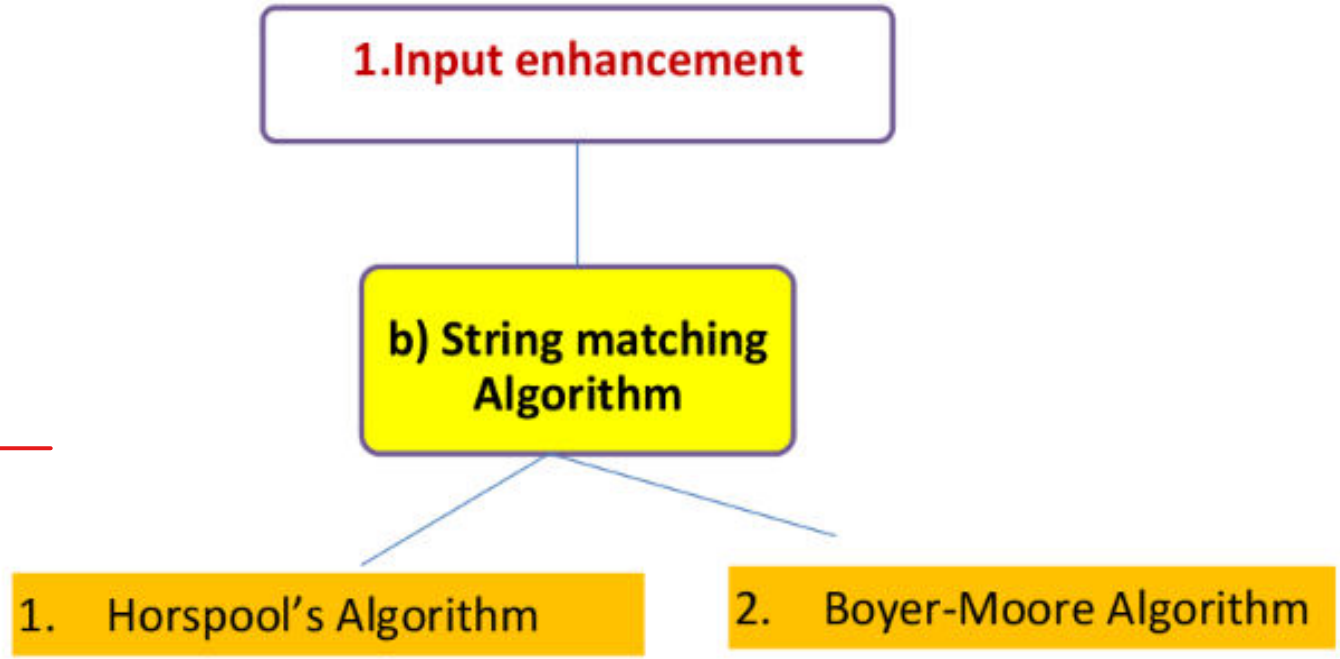
String matching requires finding an occurrence of a given string of *m* characters called the ***pattern*** in a longer string of *n* characters called the ***text***.

$$
\begin{array}{ccccccccccc}
t_0 & \ldots & t_i & \ldots & t_{i+j} & \ldots & t_{i+m-1} & \ldots & t_{n-1} \\
 & & \updownarrow & & \updownarrow & & \updownarrow & & \\
 & & p_0 & \ldots & p_j & \ldots & p_{m-1} & \text{pattern } P &
\end{array}
$$

Input Enhancement in String Matching

Input-enhancement idea:

Preprocess the pattern to get some information about it, store this information in a table, and then use this information during an actual search for the pattern in a given text.



The Boyer-Moore algorithm

Based on the same two ideas:

comparing pattern characters to text from right to left

precomputing shift sizes in **two** tables

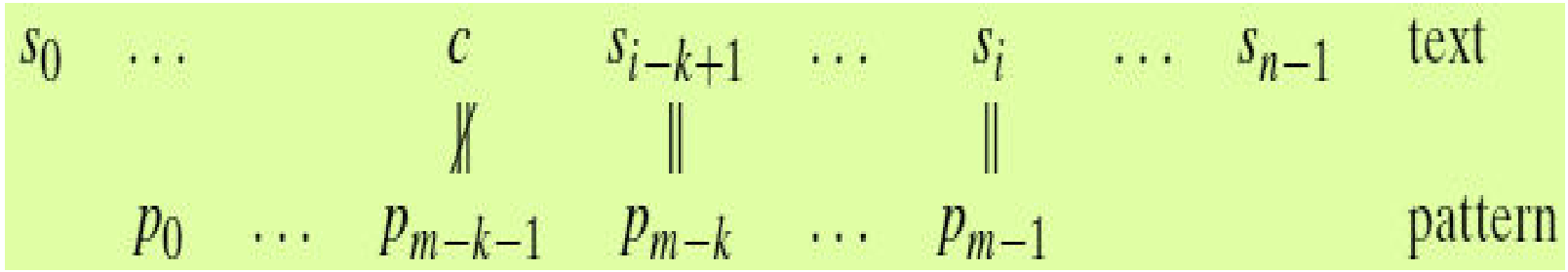
1. ***bad-symbol table*** indicates how much to shift based on text's character causing a mismatch

2. ***good-suffix table*** indicates how much to shift based on matched part (suffix) of the pattern (taking advantage of the periodic structure of the pattern)

Design and Analysis of Algorithms

Bad-symbol shift in Boyer-Moore algorithm

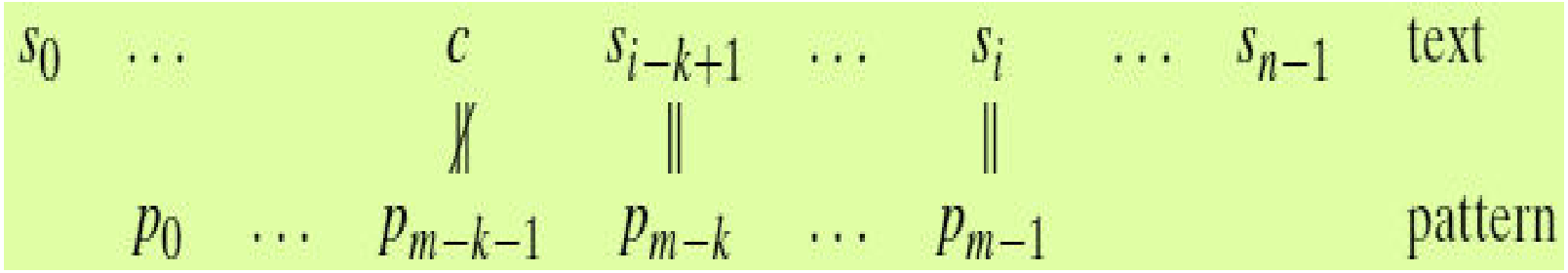
1. If the rightmost character of the pattern doesn't match, BM algorithm acts as Horspool's
2. If the rightmost character of the pattern does match, BM compares preceding characters right to left until either all pattern's characters match or a mismatch on text's character c is encountered after $k > 0$ matches



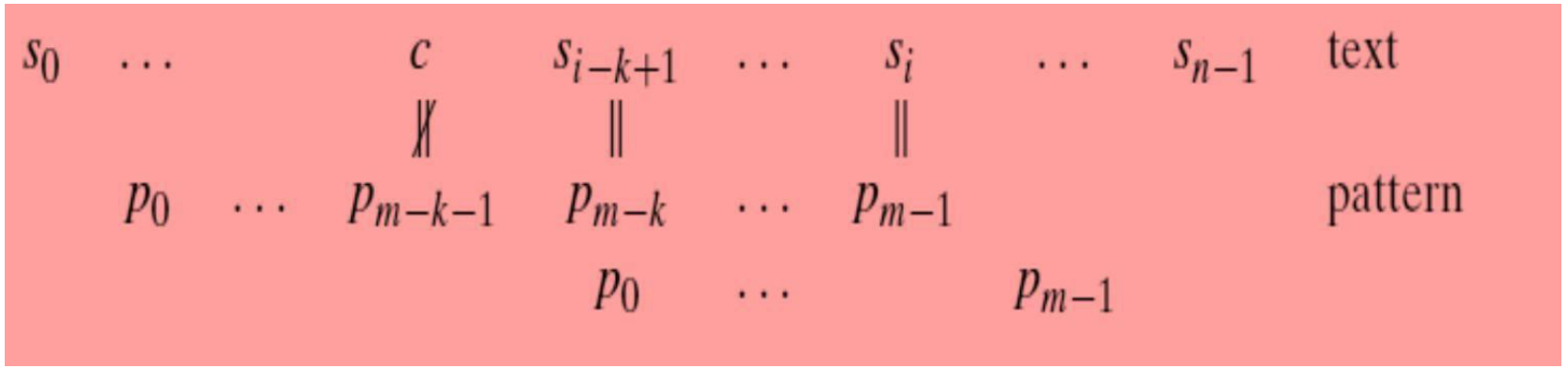
In this situation, the Boyer-Moore algorithm determines the shift size by considering two quantities. The first one is guided by the text's character c that caused a mismatch with its counterpart in the pattern. Accordingly, it is called the ***bad symbol shift***.

Design and Analysis of Algorithms

Bad-symbol shift in Boyer-Moore algorithm



If *c* is not in the pattern, we shift the pattern to just pass this *c* in the text. Conveniently, the size of this shift can be computed by the formula *t1(c)* − *k* where *t1(c)* is the entry in the precomputed table used by Horspool's algorithm and *k* is the number of matched characters:



Bad-symbol shift in Boyer-Moore algorithm- Example

character c	A	B	C	D	E	F	. . .	R	. . .	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6

For example, if we search for the pattern BARBER in some text and match the last two characters before failing on letter S in the text, we can shift the pattern by $t_1(\text{S}) - 2 = 6 - 2 = 4$ positions:

```
s0  . . .        S E R              . . .  sn−1
                 ✗ ‖ ‖
          B A R B E R
                   B A R B E R
```

Bad-symbol shift in Boyer-Moore algorithm- Example

character c	A	B	C	D	E	F	. . .	R	. . .	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6

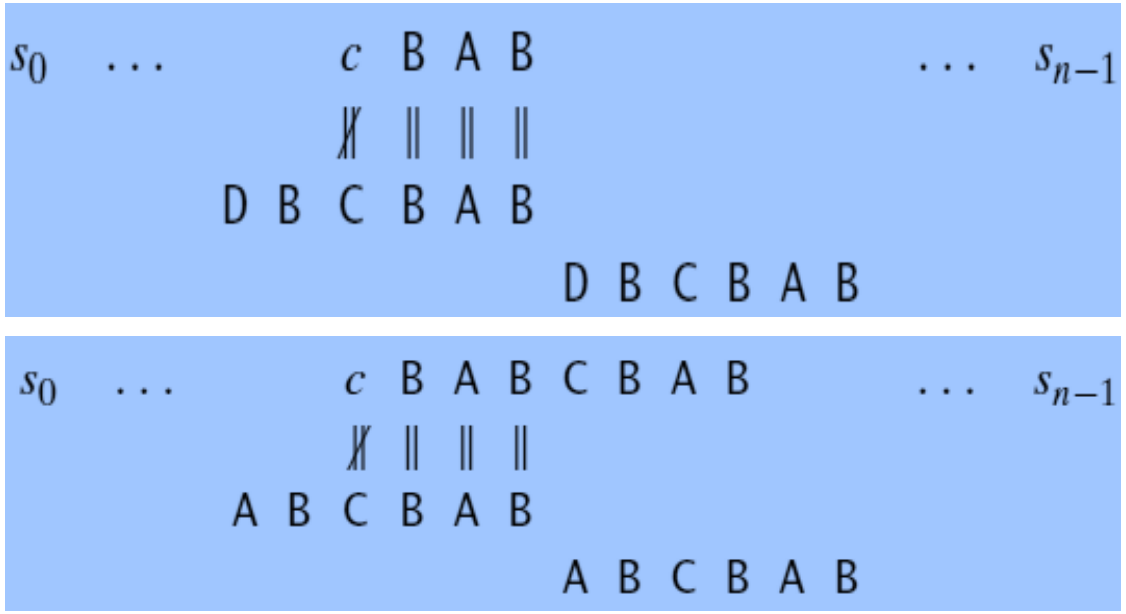
The same formula can also be used when the mismatching character c of the text occurs in the pattern, provided $t_1(c) - k > 0$. For example, if we search for the pattern BARBER in some text and match the last two characters before failing on letter A, we can shift the pattern by $t_1(\text{A}) - 2 = 4 - 2 = 2$ positions:

```
s0  . . .        A  E  R          . . .  sn−1
                 ✗  ‖  ‖
         B  A  R  B  E  R
               B  A  R  B  E  R
```

$$d_1 = \max\{t_1(c) - k, 1\}$$

Design and Analysis of Algorithms

Good-suffix shift in Boyer-Moore algorithm and when to use?



Erroneous shift will happen.
To avoid that a shift based on matched suffix of the pattern.
Called "Good Suffix shift". It is denoted by d_2.

Good-suffix shift in Boyer-Moore algorithm

$d_2(k)$ = Distance between matched suffix of size k and its rightmost occurrence in the pattern that is not preceded by the same character as the suffix

k	pattern	d_2
1	ABC$\overline{B}$AB	2
2	$\overline{AB}$CBAB	4
3	$\overline{A}$BCBAB	4
4	$\overline{AB}$CBAB	4
5	$\overline{A}$BCBAB	4

$$d = \begin{cases} d_1 & \text{if } k = 0, \\ \max\{d_1, d_2\} & \text{if } k > 0, \end{cases}$$

where $d_1 = \max\{t_1(c) - k, 1\}$

Good-suffix shift in Boyer-Moore algorithm

- Good-suffix shift d_2 is applied after $0 < k < m$ last characters were matched

- $d_2(k)$ = the distance between (the last letter of) the matched suffix of size k and (the last letter of) its rightmost occurrence in the pattern that is not preceded by the same character preceding the suffix

  Example: `CABABA` $d_2(1) = 4$

- If there is no such occurrence, match the longest part (tail) of the k-character suffix with corresponding prefix;
  if there are no such suffix-prefix matches, $d_2(k) = m$

  Example: `WOWWOW` $d_2(2) = 5$, $d_2(3) = 3$, $d_2(4) = 3$, $d_2(5) = 3$

Boyer-Moore algorithm

Step 1 Fill in the bad-symbol shift table
Step 2 Fill in the good-suffix shift table
Step 3 Align the pattern against the beginning of the text
Step 4 Repeat until a matching substring is found or text ends:
Compare the corresponding characters right to left.
If no characters match, retrieve entry $t_1(c)$ from the bad-symbol table for the text's character c causing the mismatch and shift the pattern to the right by $t_1(c)$.
If $0 < k < m$ characters are matched, retrieve entry $t_1(c)$ from the bad-symbol table for the text's character c causing the mismatch and entry $d_2(k)$ from the good-suffix table and shift the pattern to the right by

$$d = \max \{d_1, d_2\}$$

where $d_1 = \max\{t_1(c) - k, 1\}$.

Design and Analysis of Algorithms

Example of Boyer-Moore alg. application

BAOBAB

c	A	B	C	D	. . .	O	. . .	Z	_
$t_1(c)$	1	2	6	6	6	3	6	6	6

k	pattern	d_2
1	BAO$\overline{\text{B}}$A$\underline{\text{B}}$	2
2	$\overline{\text{B}}$AOB$\underline{\text{AB}}$	5
3	$\overline{\text{B}}$AO$\underline{\text{BAB}}$	5
4	$\overline{\text{B}}$A$\underline{\text{OBAB}}$	5
5	$\overline{\text{B}}$$\underline{\text{AOBAB}}$	5

B E S S _ K N E W _ A B O U T _ B A O B A B S

B A O B A B

$d_1 = t_1(K) - 0 = 6$

B A O B A B

$d_1 = t_1(_) - 2 = 4$

$d_2 = 5$

$d = \max\{4, 5\} = 5$

B A O B A B

$d_1 = t_1(_) - 1 = 5$

$d_2 = 2$

$d = \max\{5, 2\} = 5$

B A O B A B

Analysis

- The worst-case efficiency of the Boyer-Moore algorithm is known to be linear.
- Though this algorithm runs very fast, especially on large alphabets (relative to the length of the pattern),many people prefer its simplified versions, such as Horspool's algorithm, when dealing with natural-language–like strings.

References

Chapter 7 ,Introduction to The Design and Analysis of Algorithms by Anany Levitin

THANK YOU

Bharathi R

Department of Computer Science & Engineering

rbharathi@pes.edu

Design and Analysis of Algorithms
Unit -4

Bharathi R
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Unit 4: Greedy Technique

What is the Greedy Approach?
The Greedy Approach is a method used to solve optimization problems (i.e., problems where we want the best result—like least time, minimum cost, or maximum profit).

How does it work?
The idea is simple: You build the solution step-by-step, and at each step, you pick the best possible option available right now (this is called locally optimal). Once you pick something, you don't go back and change it.

Each choice must follow 3 rules:

Feasible – It must satisfy the rules or constraints of the problem.
Locally Optimal – It must be the best choice among all options available at that step.
Irrevocable – Once you make a choice, it stays. You don't undo or revise it later.

The Greedy Approach

The greedy approach suggests constructing a solution through a sequence of steps, each expanding a partially constructed solution obtained so far, until a complete solution to the problem is reached. On each step, the choice made must be:

-*feasible*: it has to satisfy the problem's constraints

-*locally optimal*: it has to be the best local choice among all feasible choices available on that step

- *irrevocable*: once decision was made, it cannot be changed on subsequent steps of the algorithm

These requirements explain the technique's name: on each step, it suggests a "greedy" grab of the best alternative available in the hope that a sequence of locally optimal choices will yield a (globally) optimal solution to the entire problem.

Examples of Greedy Algorithms:

- Coin-change problem
- Minimum Spanning Tree (MST)
  - Prim's Algorithm
  - Kruskal's Algorithm
- Single-source shortest paths
  - Dijkstra's Algorithm
- Huffman codes

Coin Change Problem

A greedy algorithm to find the minimum number of coins for making the change of a given amount of money. Usually, this problem is referred to as the change-making problem.

Design and Analysis of Algorithms
Change Making Problem

- In the change-making problem, we're provided with an array, D = { d1, d2, d3,.....dm} of m distinct coin denominations.
- Now we need to find an array(subset) s having minimum number of coins that add up to a given amount of money **n**, provided that there exists a viable solution.
- Let's consider a real-life example for a better understanding of the change-making problem.
- Let's assume that we're working at a cash counter and have an infinite supply of **D ={1,2,5,10}** valued coins.

see the algorithm for this one and go given as first question in tb excersise 9.1

- A person buys things worth **Rs. 72** and gives a **Rs. 100** bill. How does the cashier give change for **Rs. 28**?

Change Making Problem

Change-making problem:

How can a given amount of money be made with the least number of coins of given denominations?

Example:

Change for Rs. **28**

Option	Choosen Coins
28-10 = 18	10
18-10 =8	10 10
8-5 =3	10 10 5
3-2 = 1	10 10 5 2
1-1 =0	10 10 5 2 1

Design and Analysis of Algorithms

Change Making Problem

Iteration 1
D ={1,2,5,10}
n = 28 and i=4

Select D[3] =10 as 28>= 10
Decrease n by 10 (n= 18)
Add 10 to set S = {10}

Iteration 2
D ={1,2,5,10}
n = 28 and i=3

Select D[3] =10 as 18>= 10
Decrease n by 10 (n= 8)
Add 10 to set S = {10, 10}

Iteration 3
D ={1,2,5,10}
n = 8 and i=2

Select D[2] =5 as 8>= 5
Decrease n by 5 (n= 3)
Add 5 to set S = {10, 10,5}

Iteration 4
D ={1,2,5,10}
n = 3 and i=1

Select D[1] =2 as 3>= 2
Decrease n by 2 (n= 1)
Add 2 to set S = {10, 10, 5,2}

Iteration 5
D ={1,2,5,10}
n = 1 and i=0

Select D[0] =1 as 1>= 1
Decrease n by 1 (n= 0)
Add 1 to set S = {10, 10, 5,2,1}

Summary

The ***greedy technique*** suggests constructing a solution to an optimization problem through a sequence of steps, each expanding a partially constructed solution obtained so far, until a complete solution to the problem is reached.

On each step, the choice made must be ***feasible***, ***locally optimal***, and ***irrevocable***.

Design and Analysis of Algorithms

Text Books

Chapter 9 ,Introduction to The Design and Analysis of Algorithms by Anany Levitin

THANK YOU

Bharathi R

Department of Computer Science & Engineering

rbharathi@pes.edu

Design and Analysis of Algorithms
Unit -4

Bharathi R
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Unit 4: Greedy Technique

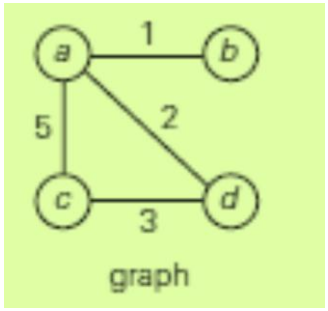
Prim's algorithm

Bharathi R

Department of Computer Science & Engineering

Minimum Spanning Tree : DEFINITIONs

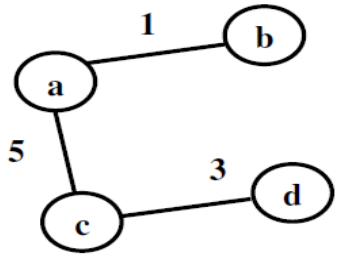
A *spanning tree* of an undirected connected graph is its connected acyclic subgraph (i.e., a tree) that contains all the vertices of the graph.



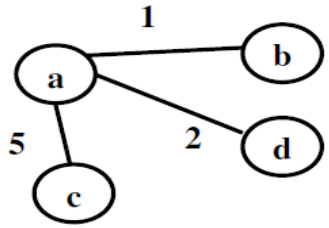
graph

Example

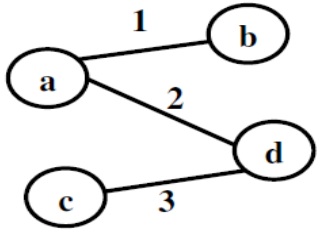
The spanning trees for the above graph are as follows:



Weight (T_1) = 9



Weight (T_2) = 8

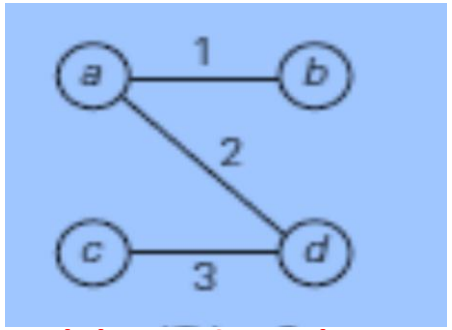


Weight (T_3) = 6

Spanning tree of a graph consists of all vertices in the graph and has min set of edges which connects all the vertices to form a tree(a graph with no cycles)

told by ma'am

Minimum Spanning Tree (MST) of a weighted, connected graph G is a spanning tree of G with minimum total weight.



Minimum Spanning Tree

Design and Analysis of Algorithms

Prim's Algorithm : Greedy Approach

mst only has v-1 edges
you wrote in your notes
from abdul bari video

- Start with a tree, T_1, consisting of one vertex (V).
- Adjacent vertices of the vertex in T_1 are "fringe" vertices of T_1.
- For i = 1 to $|V|-1$ do
  - Construct T_i from T_{i-1} by adding the fringe vertex with the minimum weight edge from the set. The vertex is removed from the set of fringe vertices.
  - Add the adjacent vertices of the vertex to the set of fringe vertices which are not in T_i.
  - Remove vertices from the set of fringe vertices where the new vertex is one of the terminal vertex of the edge.
- Return T_n which is a minimum spanning tree.

Step-by-Step Explanation of Your Pseudocode:

Start with one vertex
Let's call this initial tree T1, and it starts with one vertex (say, A).
Identify fringe vertices
The fringe vertices are those not in the MST yet, but are directly connected to a vertex in the MST.
Imagine T1 = {A}, and A is connected to B with weight 2, and C with weight 3. Then B and C are fringe vertices.

For i = 1 to V minus 1
Loop until we include all the vertices (that is, form an MST):

a. Add the fringe vertex with the smallest edge weight to the tree.
This is the greedy choice -- pick the best local option.
Example: Between B with weight 2 and C with weight 3, we choose B.

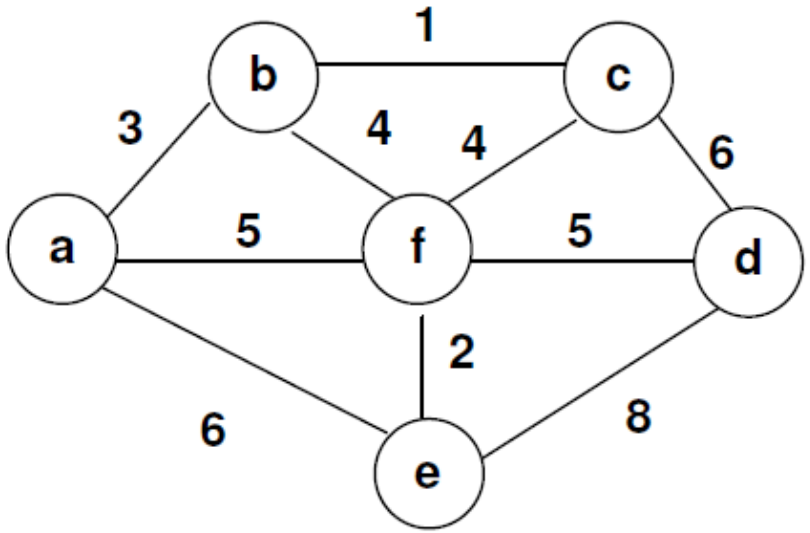
b. Add its adjacent vertices (that are not yet in the MST) to the fringe list.
If B is now added, and it connects to D with weight 4, then D becomes a new fringe vertex.

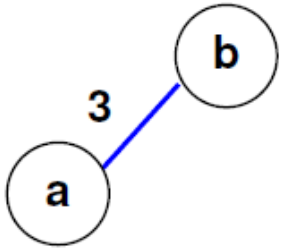
c. Remove any fringe vertex if it is now part of the MST.
Once a vertex is added to the tree, it is no longer a fringe.

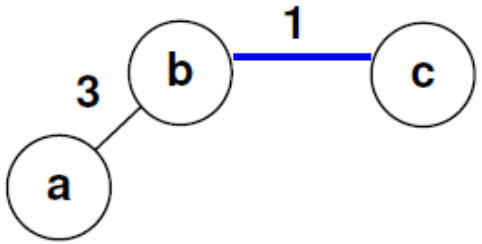
Repeat this until you have added all V minus 1 edges.
This forms the Minimum Spanning Tree.

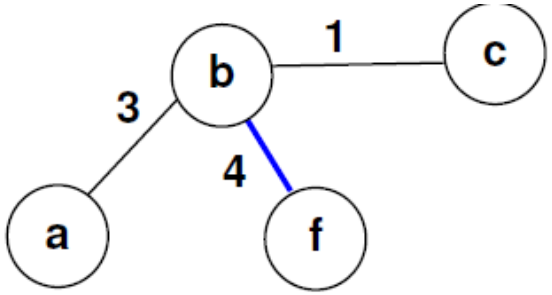
Design and Analysis of Algorithms

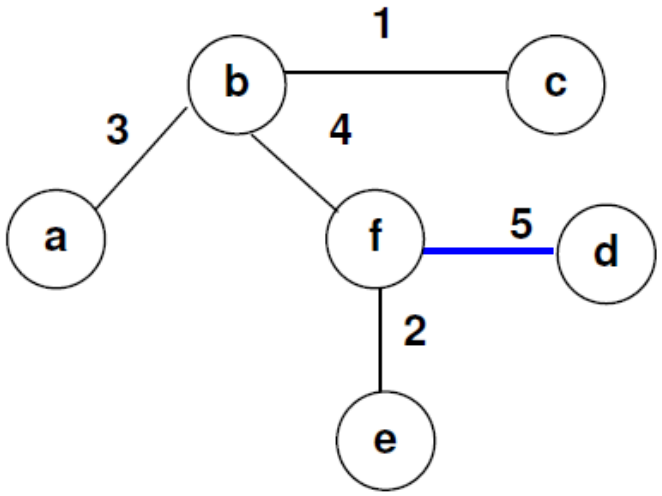
Prim's Algorithm

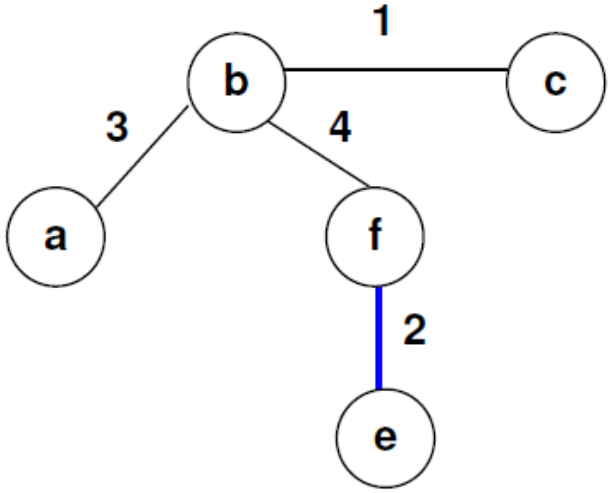




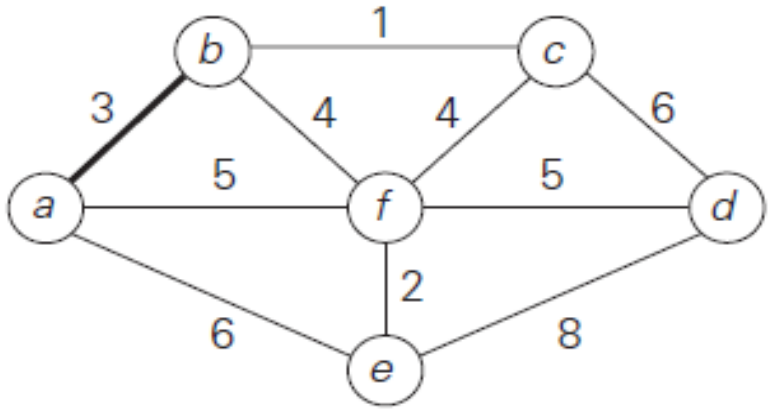
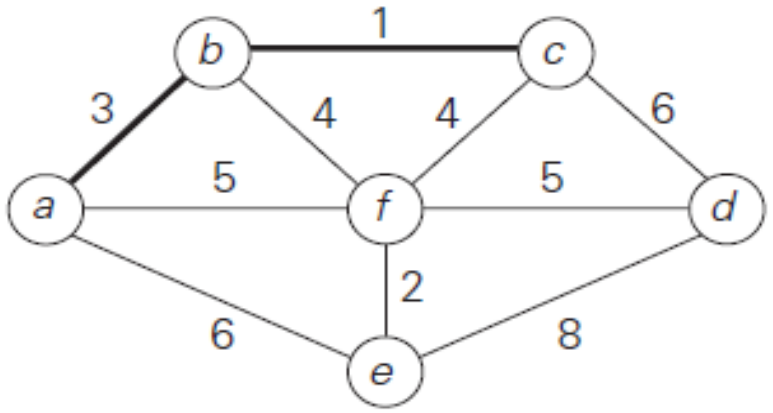






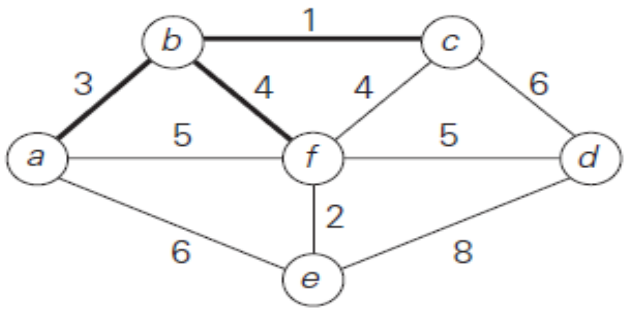
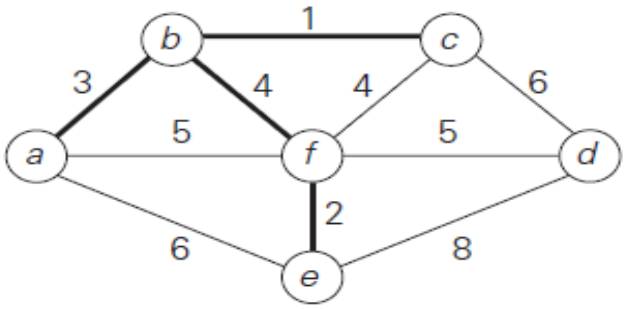
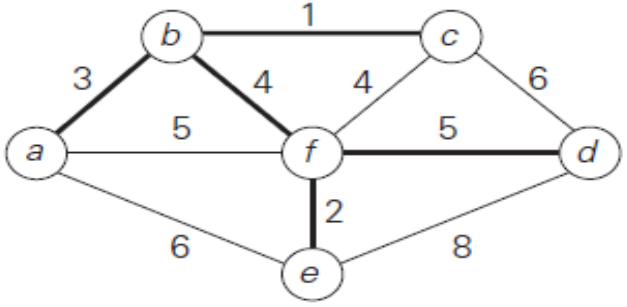


Prim's Algorithm

Tree vertices	Remaining vertices	Illustration
a(–, –)	**b(a, 3)** c(–, ∞) d(–, ∞) e(a, 6) f(a, 5)	
b(a, 3)	**c(b, 1)** d(–, ∞) e(a, 6) f(b, 4)	

Design and Analysis of Algorithms

Prim's Algorithm

Tree vertices	Remaining vertices	Illustration
c(b, 1)	d(c, 6) e(a, 6) **f(b, 4)**	
f(b, 4)	d(f, 5) **e(f, 2)**	
e(f, 2)	**d(f, 5)**	
d(f, 5)		

Design and Analysis of Algorithms

Prim's Algorithm

ALGORITHM *Prim*(G)

//Prim's algorithm for constructing a minimum spanning tree
//Input: A weighted connected graph $G = \langle V, E \rangle$
//Output: E_T, the set of edges composing a minimum spanning tree of G
$V_T \leftarrow \{v_0\}$ //the set of tree vertices can be initialized with any vertex
$E_T \leftarrow \varnothing$
for $i \leftarrow 1$ **to** $|V| - 1$ **do**
    find a minimum-weight edge $e^* = (v^*, u^*)$ among all the edges (v, u)
    such that v is in V_T and u is in $V - V_T$
    $V_T \leftarrow V_T \cup \{u^*\}$
    $E_T \leftarrow E_T \cup \{e^*\}$
return E_T

Step-by-Step Explanation of the Algorithm:
1.Start with a random vertex from the graph. Let's call it v0. Add it to VT.
This means our tree currently has only one vertex and no edges.

2.Initialize the MST edge set ET as empty.
We haven't picked any edges yet.

3.Loop |V| - 1 times
Since a tree with |V| vertices has exactly |V| - 1 edges, we repeat the next step this many times.

4.In each iteration, do the following:
a. Find the edge with the smallest weight that connects:
   # One vertex (v) already in the tree (VT)
   # One vertex (u) not yet in the tree (V - VT)
This is the greedy step — always pick the lightest edge that expands the tree.
b. Add the new vertex u* to the tree vertex set VT.
c. Add the chosen edge e* to the MST edge set ET.
5.Repeat until all vertices are included.
6.When the loop is done, VT contains all vertices, and ET contains all the edges of the MST.
Return the set of MST edges ET.

Understanding the Variables:
G = (V, E): Your graph G has a set of vertices V and edges E.
VT: Set of vertices that are already included in the tree (initially starts with just one vertex).
ET: Set of edges in the MST (starts empty).
e* = (v*, u*): The edge with the smallest weight between a vertex v* in the tree and a vertex u* not yet in the tree.

find a minimum-weight edge $e^* = (v^*, u^*)$ among all the edges (v, u) such that v is in V_T and u is in $V - V_T$

After we have identified a vertex u^* to be added to the tree, we need to perform two operations:

Move u^* from the set $V - V_T$ to the set of tree vertices V_T.

For each remaining vertex u in $V - V_T$ that is connected to u^* by a shorter edge than the u's current distance label, update its labels by u^* and the weight of the edge between u^* and u, respectively.

How efficient is Prim's Algorithm

- The answer depends on the data structures chosen for the graph itself and for the priority queue of the set $V - V_T$ whose vertex priorities are the distances to the nearest tree vertices.

- If a graph is represented by its weight matrix and the priority queue is implemented as an unordered array, the algorithm's running time will be in $\Theta(|V|^2)$

- If a graph is represented by its adjacency lists and the priority queue is implemented as a min-heap, the running time of the algorithm is in $O(|E| \log |V|)$.

Summary

Prim's algorithm is very similar to Kruskal's: whereas Kruskal's "grows" a forest of trees, Prim's algorithm grows a single tree until it becomes the minimum spanning tree.

Both algorithms use the greedy approach - they add the cheapest edge that will not cause a cycle. But rather than choosing the cheapest edge that will connect *any* pair of trees together, Prim's algorithm only adds edges that join nodes to the existing tree.

(In this respect, Prim's algorithm is very similar to Dijkstra's algorithm for finding shortest paths.)

Design and Analysis of Algorithms

Text Books

Chapter 9 ,Introduction to The Design and Analysis of Algorithms by Anany Levitin

THANK YOU

Bharathi R

Department of Computer Science & Engineering

rbharathi@pes.edu

Cycle Detection in undirected graph using Union-Find Algorithm

By

P.Kavitha

Dept. of CSE

PES University (EC Campus), Bangalore

Disjoint Sets

- Two sets are called **disjoint sets** if they don't have any element in common. The disjoint set data structure is used to store such sets. It supports following operations:
- Merging two disjoint sets to a single set using **Union** operation.
- Finding representative of a disjoint set using **Find** operation.
- Check if two elements belong to same set or not. Find the representative of both and check if same.

Disjoint Sets

- It is a data structure that keeps track of a set of elements partitioned into a number of disjoint(non overlapping) subsets

$\Rightarrow S1 \cap S2 = \Phi \quad , S2 \cap S3 = \Phi \quad , S1 \cap S3 = \Phi$

Disjoint Set

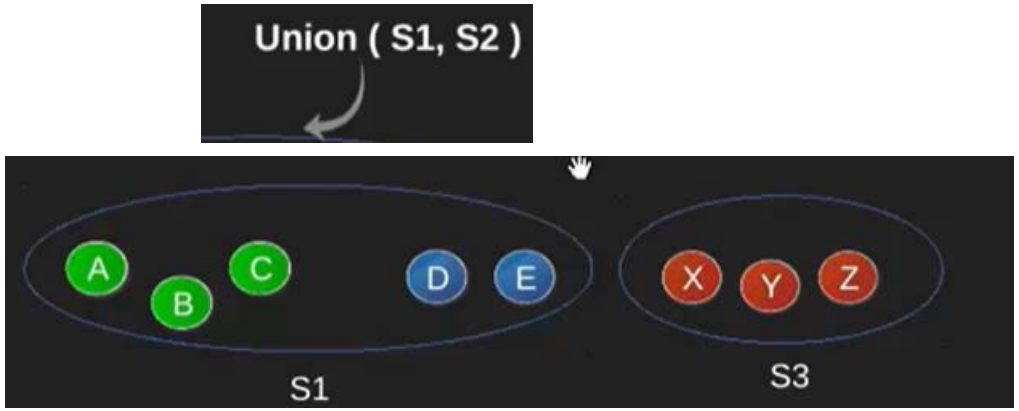
- It is a data structure that keeps track of a set of elements partitioned into a number of disjoint(non overlapping) subsets

No element is common in the sets

$\Rightarrow S1 \cap S2 = \Phi \quad , \; S2 \cap S3 = \Phi \quad , \; S1 \cap S3 = \Phi$

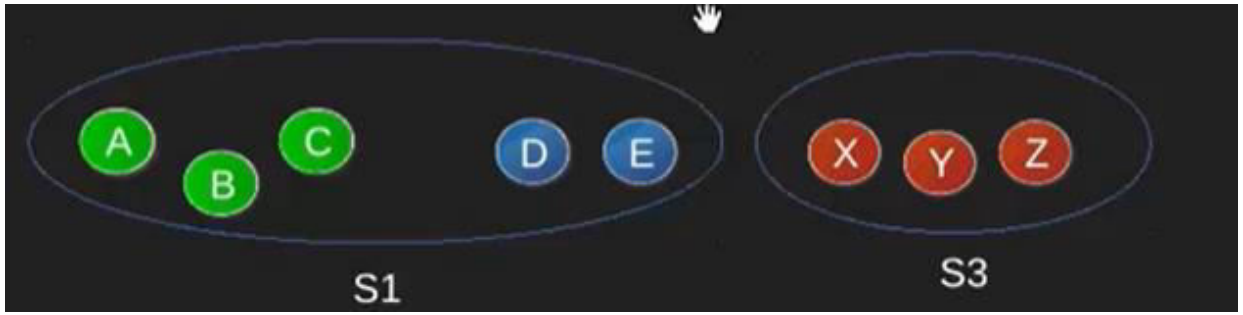
Disjoint Set

- Operations
- Union()
- Find()



Disjoint Set

- Operations
- Union()
- Find()



Find (B) - S1
Find (D) - S1
Find (Z) - S3

Cycle Detection

- Problem:

For each edge, make subsets using both the vertices of the edge.

If both the vertices are in the same subset, a cycle is found.

Cycle Detection

- Problem:

For each edge, make subsets using both the vertices of the edge.

If both the vertices are in the same subset, a cycle is found.

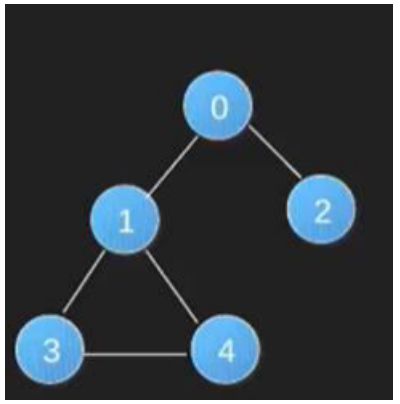
```
for each unvisited edge (u, v) in edge set E
{
    If( Find(u) == Find(v) )
    {
        // cycle detected
    }
    else
        Union(x, y);
}
```

Cycle Detection

Nodes	0	1	2	3	4
Parent	-1	-1	-1	-1	-1

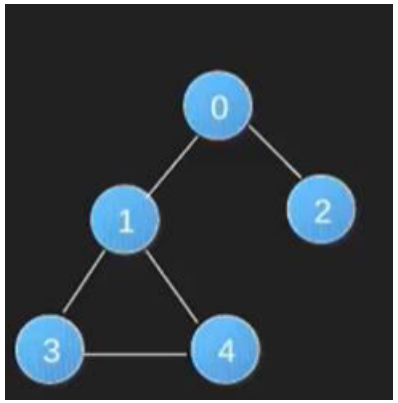
Initially all the nodes are parent of itself

Undirected graph



Cycle Detection

Undirected graph



Nodes	0	1	2	3	4
Parent	-1	-1	-1	-1	-1

Initially all the nodes are parent of itself

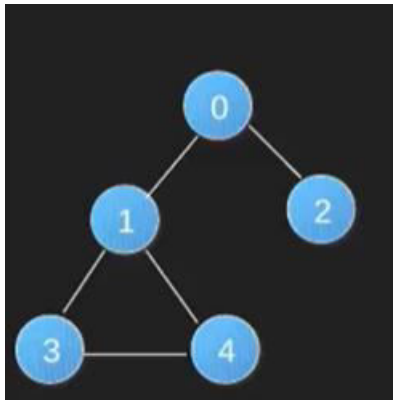
Select one edge at a time :

Edge(0,1)

Find(0)=0 , Find(1)=1 , both are parent nodes or representing itself

Cycle Detection

Undirected graph



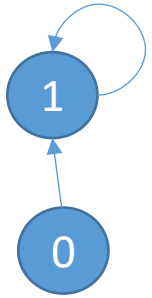
Nodes	0	1	2	3	4
Parent	-1	-1	-1	-1	-1

Initially all the nodes are parent of itself

Select one edge at a time :

Edge(0,1)

Find(0)=0 , Find(1)=1 , both are parent nodes or representing itself

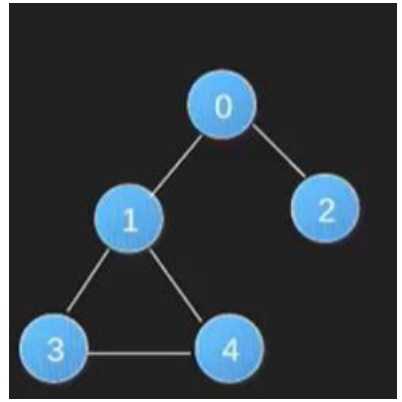


Nodes 0,1 belongs to a different set

=>Apply Union(0,1)

Nodes	0	1	2	3	4
Parent	1	-1	-1	-1	-1

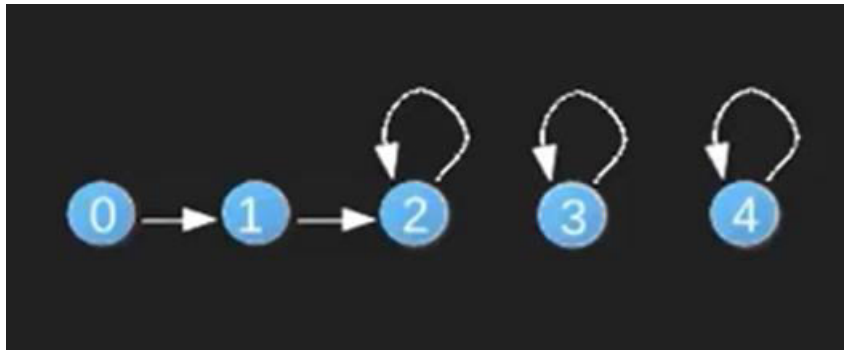
Cycle Detection

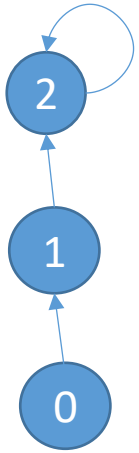


Nodes	0	1	2	3	4
Parent	1	-1	-1	-1	-1

- Edge(0,2)
- Find(0)=1 , Find(2)=2
- Parent of 0 is 1 and 2 is a parent of itself.

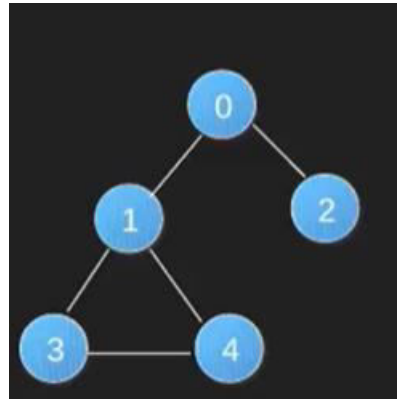
  Nodes 0,2 belongs to a different subsets =>Apply Union(1,2)





Nodes	0	1	2	3	4
Parent	1	2	-1	-1	-1

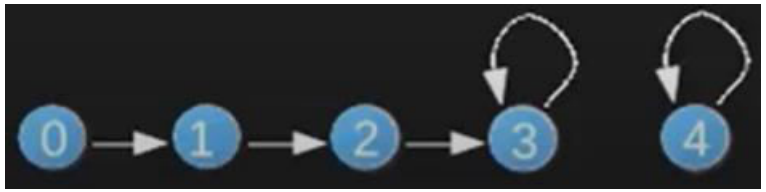
Cycle Detection

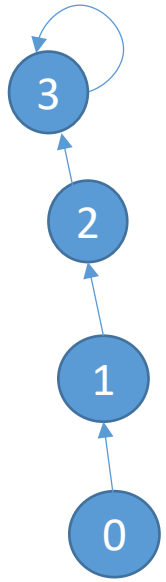


Nodes	0	1	2	3	4
Parent	1	2	-1	-1	-1

- Edge(1,3)
- Find(1)=2 , Find(3)=3
- Parent of 1 is 2 & 3 is parent of itself.

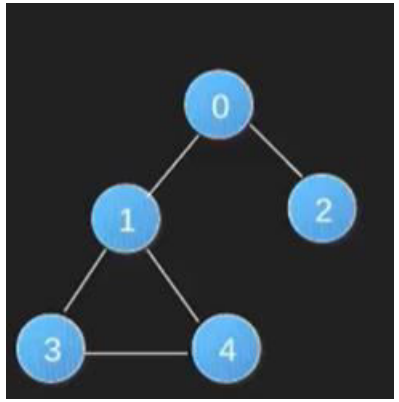
  Nodes 1,3 belongs to a different subsets =>Apply Union(2,3)





Nodes	0	1	2	3	4
Parent	1	2	3	-1	-1

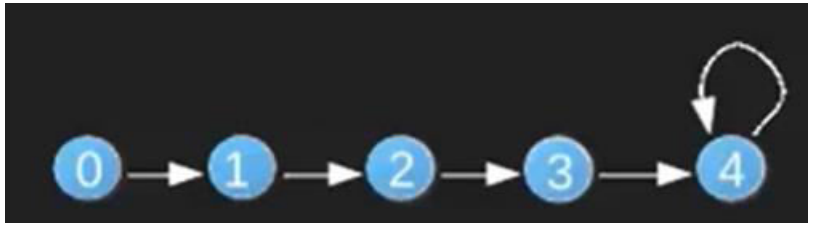
Cycle Detection



Nodes	0	1	2	3	4
Parent	1	2	3	-1	-1

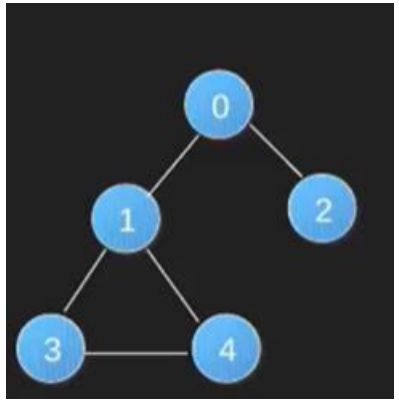
- Edge(1,4)
- Find(1)=3 , Find(4)=4
- Parent of 1 is 3 & 4 is parent of itself.

  Nodes 3,4 belongs to a different subsets =>Apply Union(3,4)



Nodes	0	1	2	3	4
Parent	1	2	3	4	-1

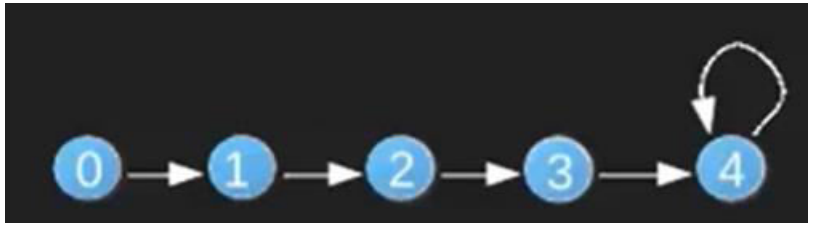
Cycle Detection



Nodes	0	1	2	3	4
Parent	1	2	3	4	-1

- Edge(3,4)
- Find(3)=4 , Find(4)=4
- Parent of 3 is 4 & 4 is parent of itself.

  Nodes belongs to the same set=> Cycle Detected!



Implementation

```
// The main function to check whether a given graph contains
// cycle or not
int isCycle( struct Graph* graph )
{
    // Allocate memory for creating V subsets
    int *parent = (int*) malloc( graph->V * sizeof(int) );

    // Initialize all subsets as single element sets
    memset(parent, -1, sizeof(int) * graph->V);

    // Iterate through all edges of graph, find subset of both
    // vertices of every edge, if both subsets are same, then
    // there is cycle in graph.
    for(int i = 0; i < graph->E; ++i)
    {
        int x = find(parent, graph->edge[i].src);
        int y = find(parent, graph->edge[i].dest);

        if (x == y)
            return 1;

        Union(parent, x, y);
    }
    return 0;
}
```

Source: geeks for geeks

Prims Vs. Kruskals Algorithm

- Prim's and Kruskal's algorithms are both used to find the **Minimum Spanning Tree (MST)** of a graph, but they work in different ways.

Feature	Prim's Algorithm	Kruskal's Algorithm
Approach	Greedy approach that grows a single tree by adding the smallest edge	Greedy approach that sorts edges and adds them while avoiding cycles
Starting Point	Starts from an arbitrary node	Starts with the smallest edge
Graph Representation	Works well with adjacency matrix and adjacency list	Works well with edge list
Edge Selection	Picks the smallest edge that connects a node in the MST to a node outside	Picks the smallest edge in the entire graph (sorted order)
Cycle Formation	Ensures connectivity by always expanding from the MST set	Uses the Disjoint Set Union (DSU) to avoid cycles
Best for Dense/Sparse Graphs	Works better for dense graphs (because adjacency matrix performs well)	Works better for sparse graphs (as sorting edges is efficient)
Time Complexity	$O(V^2)$ (with adjacency matrix) or O(E log V) (with priority queue)	O(E log E) (due to sorting)

Key Differences:

- **Prim's** works by **growing** a single MST tree, while **Kruskal's** works by **merging** components.
- **Prim's** selects edges based on connected nodes, while **Kruskal's** sorts edges and picks the smallest.
- **Kruskal's** uses **Disjoint Set (Union-Find)** to check cycles, while **Prim's** uses a **priority queue** or **min heap**.

Reference

- https://www.geeksforgeeks.org/introduction-to-disjoint-set-data-structure-or-union-find-algorithm/?ref=ml_lbp

Design and Analysis of Algorithms
Unit -4

Bharathi R
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Unit 4: Greedy Technique

Bharathi R

Department of Computer Science & Engineering

Minimum Spanning Tree : Kruskal's Algorithm

This algorithm was described by **Joseph Kruskal** in 1956.

Prim's algorithm and Kruskal's algorithm solve the same problem (**Minimum Spanning Tree**) by applying the **greedy approach** in two different ways, and both of them always yield an optimal solution.

Design and Analysis of Algorithms

Minimum Spanning Tree : Kruskal's Algorithm

Kruskal's algorithm initially places all the nodes of the original graph isolated from each other, to form a forest of single node trees, and then gradually merges these trees, combining at each iteration any two of all the trees with some edge of the original graph.

Before the execution of the algorithm, all edges are sorted by weight (in non-decreasing order).

Then begins the process of unification: pick all edges from the first to the last (in sorted order), and if the ends of the currently picked edge belong to different subtrees, these subtrees are combined, and the edge is added to the answer.

After iterating through all the edges, all the vertices will belong to the same sub-tree, and we will get the answer.

Design and Analysis of Algorithms

Kruskal's Algorithm

Let G = {V,E} be weighted connected graph

The vertices are numbered 0 through $n - 1$.

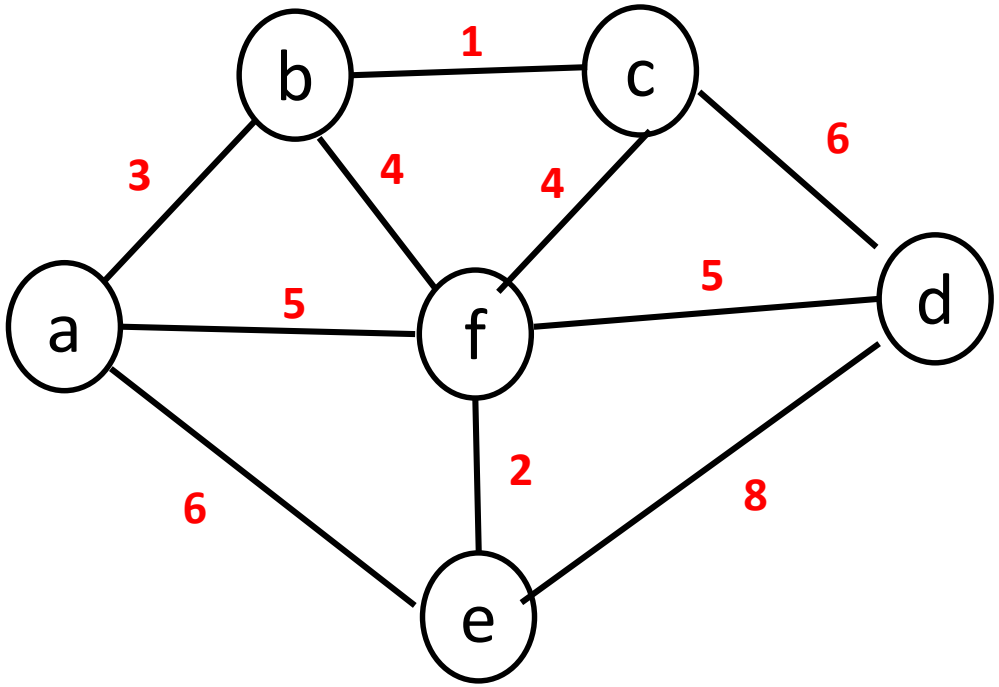
V = {0,1,….n-1} a nd E =[e_1,e_2,….e_n}

Sort the edges in non-decreasing order of their weights.

- Initially, trees of the forest are the vertices (no edges).
- Start with an empty minimum spanning tree and mark each edge as unvisited,
- While there are edges not yet visited or while the minimum spanning tree does not have $n - 1$ edges:
  - Find the edge with minimum weight that has not yet been visited.
  - Mark that edge as visited.
  - If adding that edge does not create a cycle (that is, the two vertices are not already connected), add that edge to the minimum spanning tree.

Kruskal's Algorithm

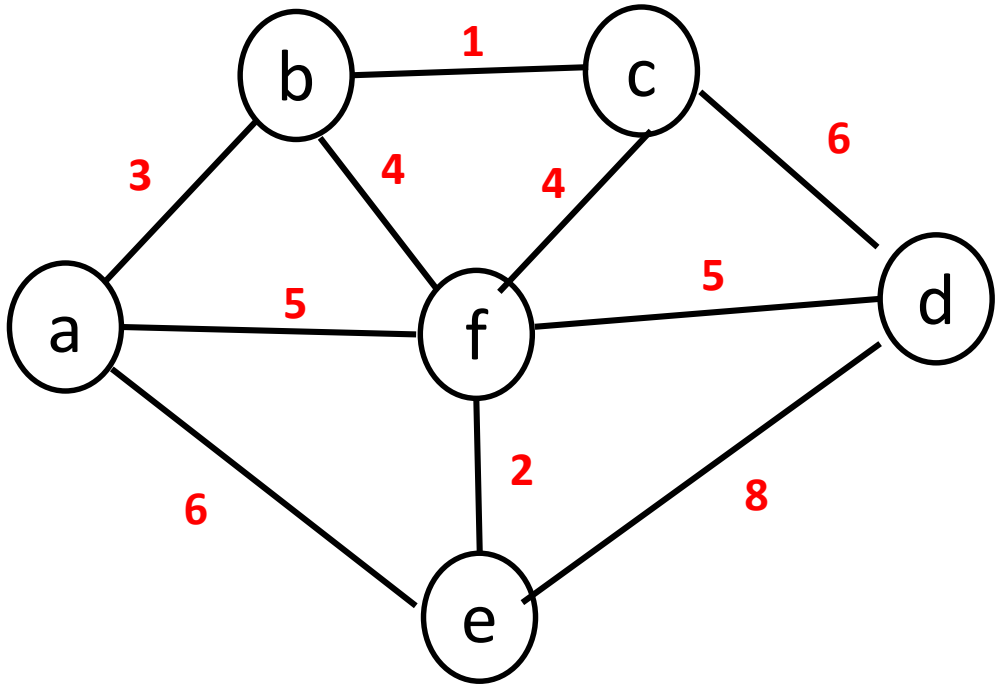
Given : Graph <V,E>



Design and Analysis of Algorithms

Kruskal's Algorithm

Given : Graph<V,E>



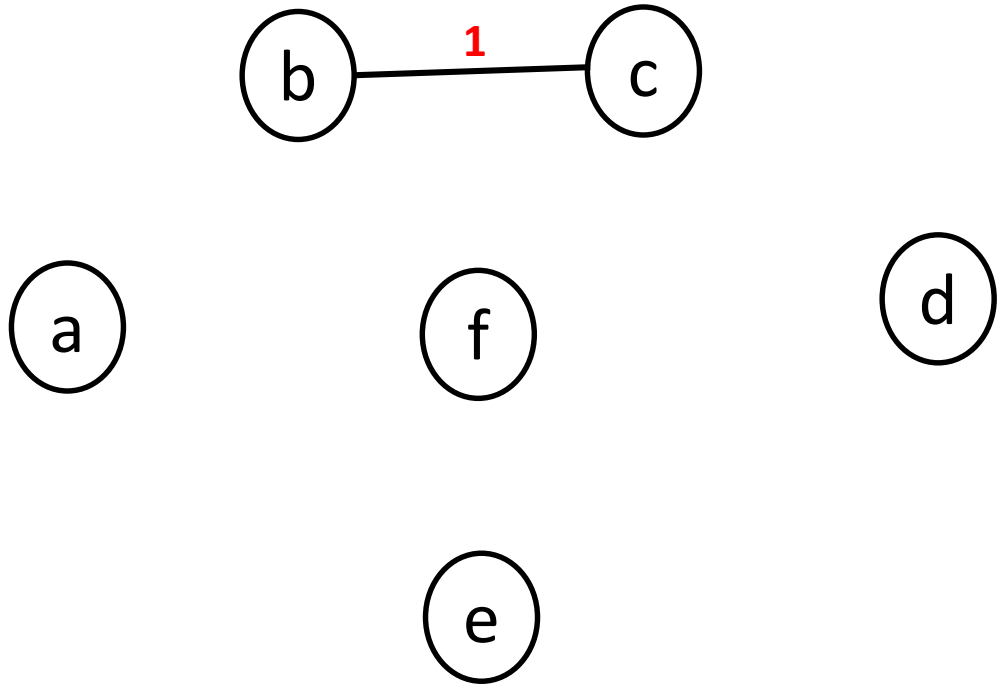
Sorted list of edges in non decreasing order

Edge(u,v)	Weight
bc	1
ef	2
ab	3
bf	4
cf	4
af	5
df	5
ae	6
cd	6
de	8

Design and Analysis of Algorithms

Kruskal's Algorithm

Given : Graph<V,E>



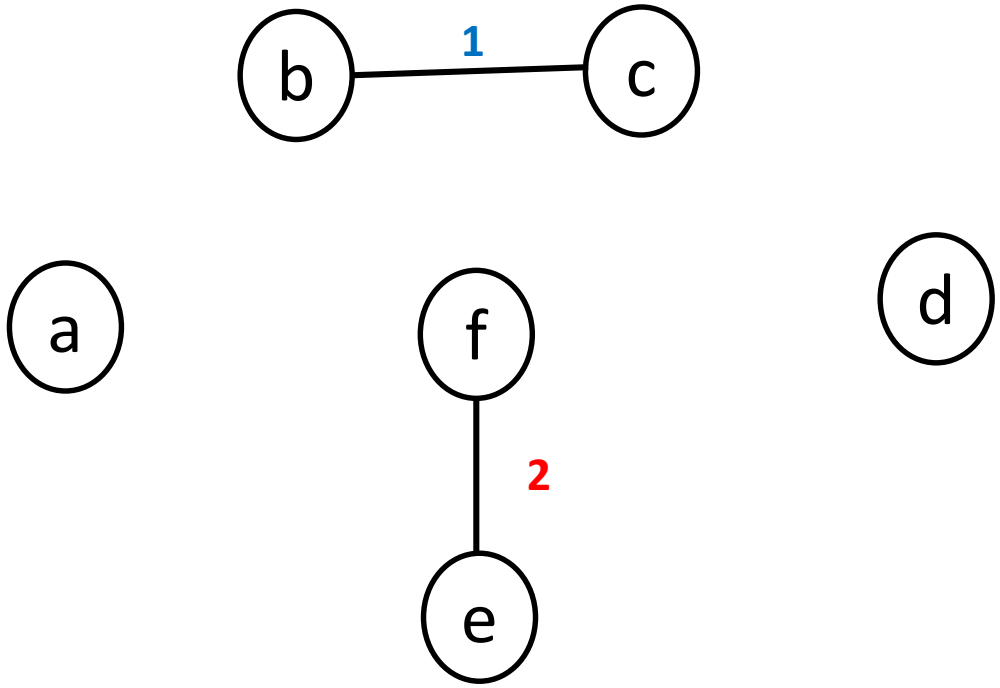
Sorted list of edges in non decreasing order

Edge(u,v)	Weight
bc	**1**
ef	2
ab	3
bf	4
cf	4
af	5
df	5
ae	6
cd	6
de	8

Design and Analysis of Algorithms

Kruskal's Algorithm

Given : Graph<V,E>



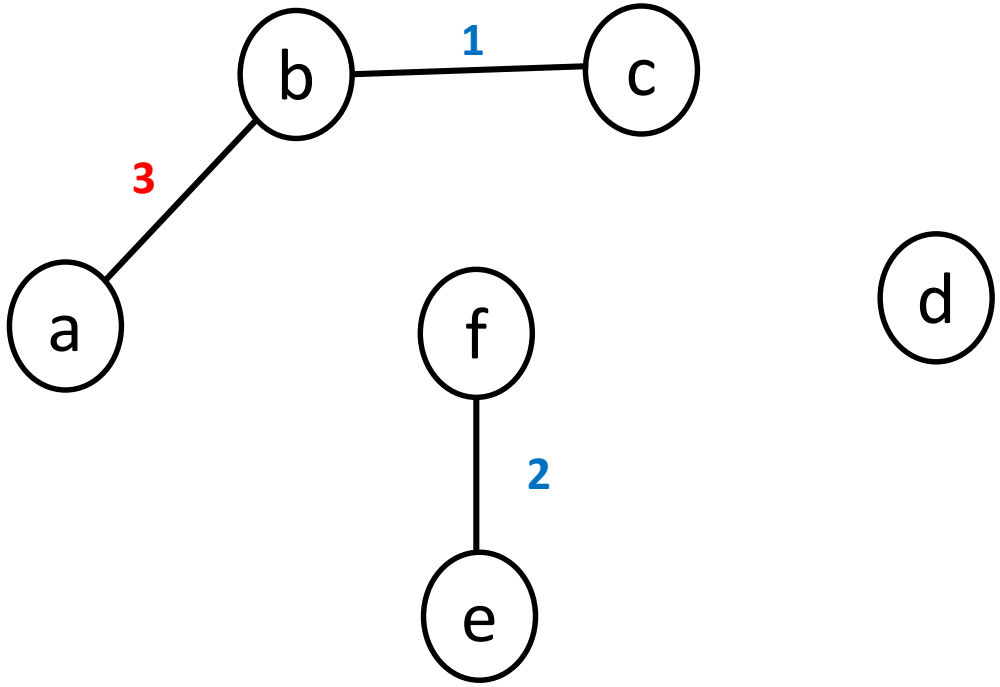
Sorted list of edges in non decreasing order

Edge(u,v)	Weight
bc	1
ef	2
ab	3
bf	4
cf	4
af	5
df	5
ae	6
cd	6
de	8

Design and Analysis of Algorithms

Kruskal's Algorithm

Given : Graph<V,E>



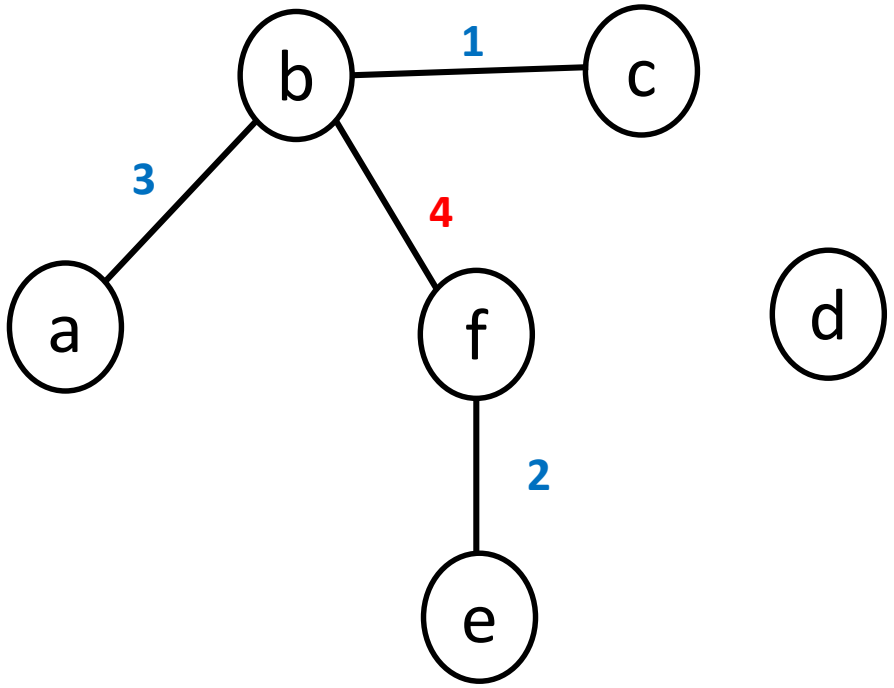
Sorted list of edges in non decreasing order

Edge(u,v)	Weight
bc	1
ef	2
ab	3
bf	4
cf	4
af	5
df	5
ae	6
cd	6
de	8

Design and Analysis of Algorithms

Kruskal's Algorithm

Given : Graph<V,E>



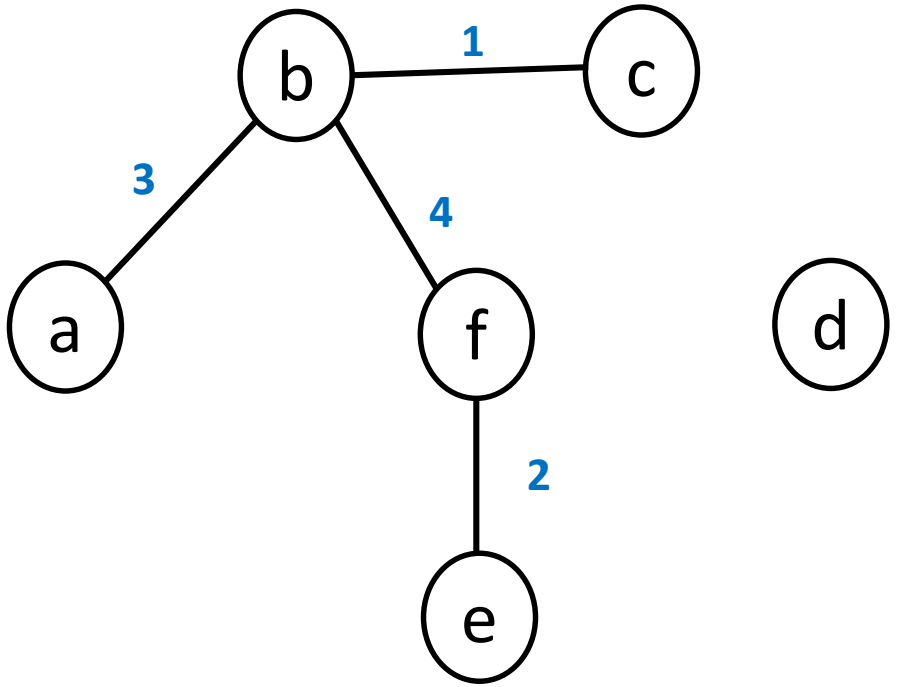
Sorted list of edges in non decreasing order

Edge(u,v)	Weight
bc	1
ef	2
ab	3
bf	4
cf	4
af	5
df	5
ae	6
cd	6
de	8

Design and Analysis of Algorithms

Kruskal's Algorithm

Given : Graph<V,E>



Sorted list of Edges in non decreasing Order

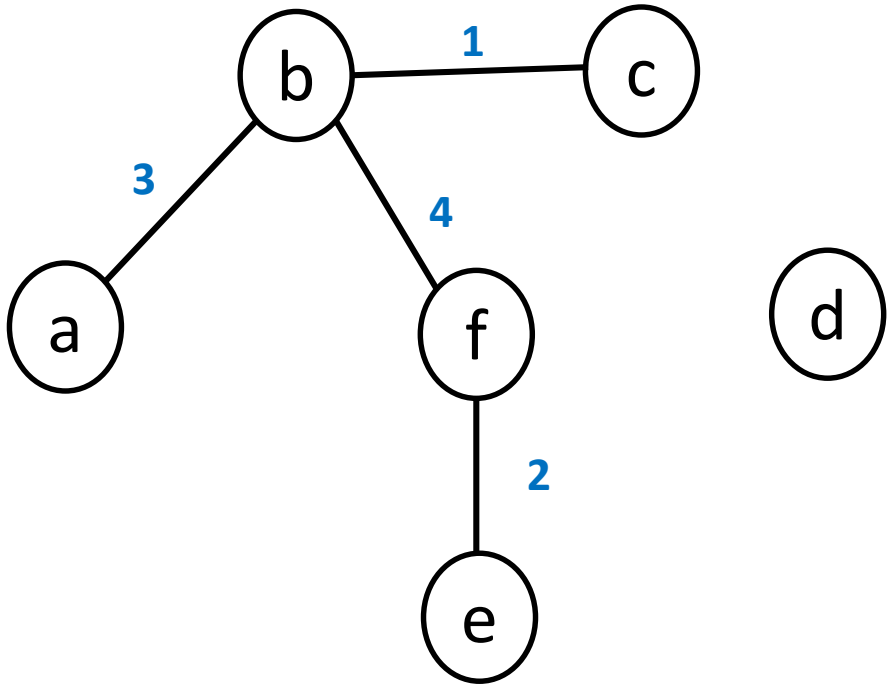
Edge(u,v)	Weight
bc	1
ef	2
ab	3
bf	4
cf	4 x
af	5
df	5
ae	6
cd	6
de	8

Cyclic

Design and Analysis of Algorithms

Kruskal's Algorithm

Given : Graph<V,E>



Sorted list of edges in non decreasing order

Edge(u,v)	Weight
bc	1
ef	2
ab	3
bf	4
cf	4
af	5
df	5
ae	6
cd	6
de	8

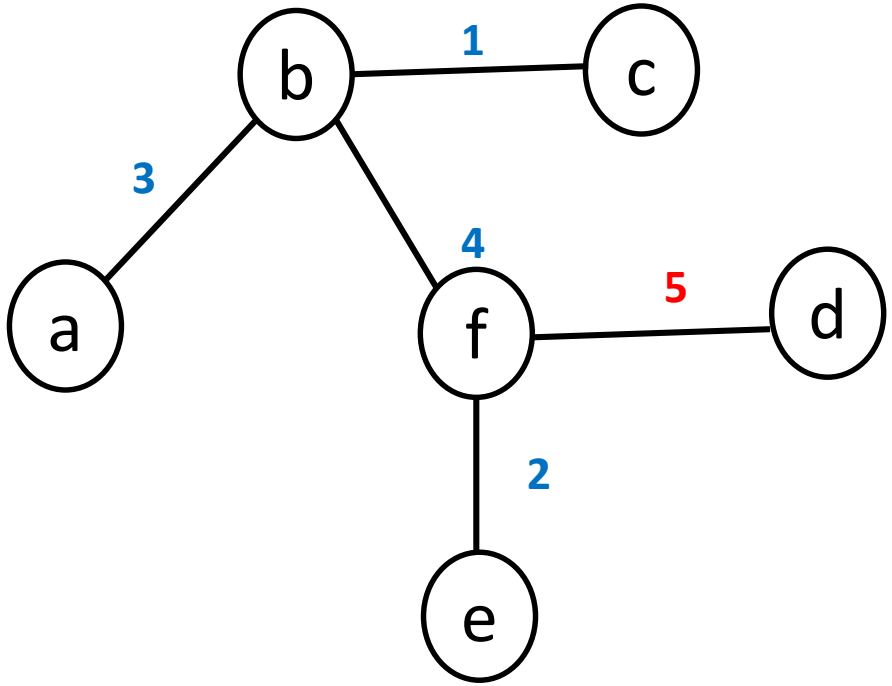
Cyclic

X

Design and Analysis of Algorithms

Kruskal's Algorithm

Given : Graph<V,E>



Sorted list of edges in non decreasing order

Edge(u,v)	Weight
bc	1
ef	2
ab	3
bf	4
cf	4
af	5
df	5
ae	6
cd	6
de	8

Acyclic

Before formalizing the above idea, lets quickly review the disjoint-set data structure.

1. *makeset*(***v***): Create a new set whose only member is pointed to by **v**. Note that for this operation v must already be in a set.
2. *find*(***v***): Returns a pointer to the set containing *v*.
3. *union*(**u,v**): Unites the dynamic sets that contain *u* and *v* into a new set that is union of these two sets.

For example, let ***S* = {1, 2, 3, 4, 5, 6}.**
Then ***makeset(i)*** creates the set ***{i}*** and applying this operation six times initializes the structure to the collection of six singleton sets:

{1}, {2}, {3}, {4}, {5}, {6}.

Performing ***union(1, 4)*** and ***union(5, 2)*** yields

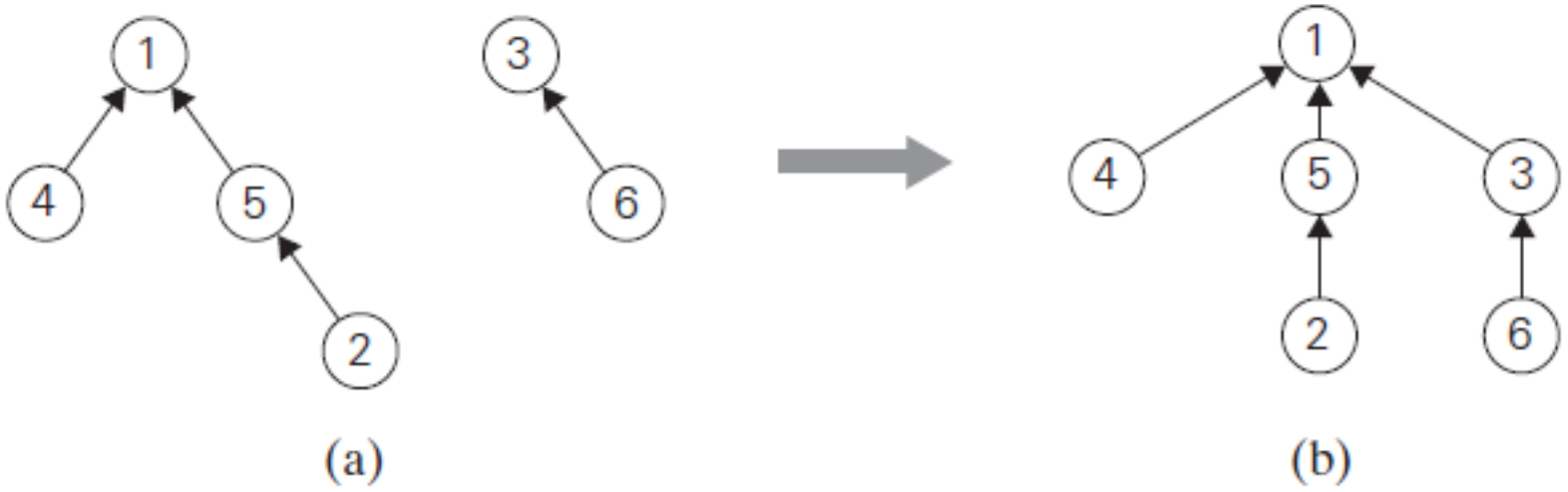
{1, 4}, {5, 2}, {3}, {6},

and, if followed by ***union(4, 5)*** and then by ***union(3, 6)***, we end up with the disjoint subsets

{1, 4, 5, 2}, {3, 6}.

(a) Forest representation of subsets {1, 4, 5, 2} and {3, 6}

(b) Result of *union*(5, 6).



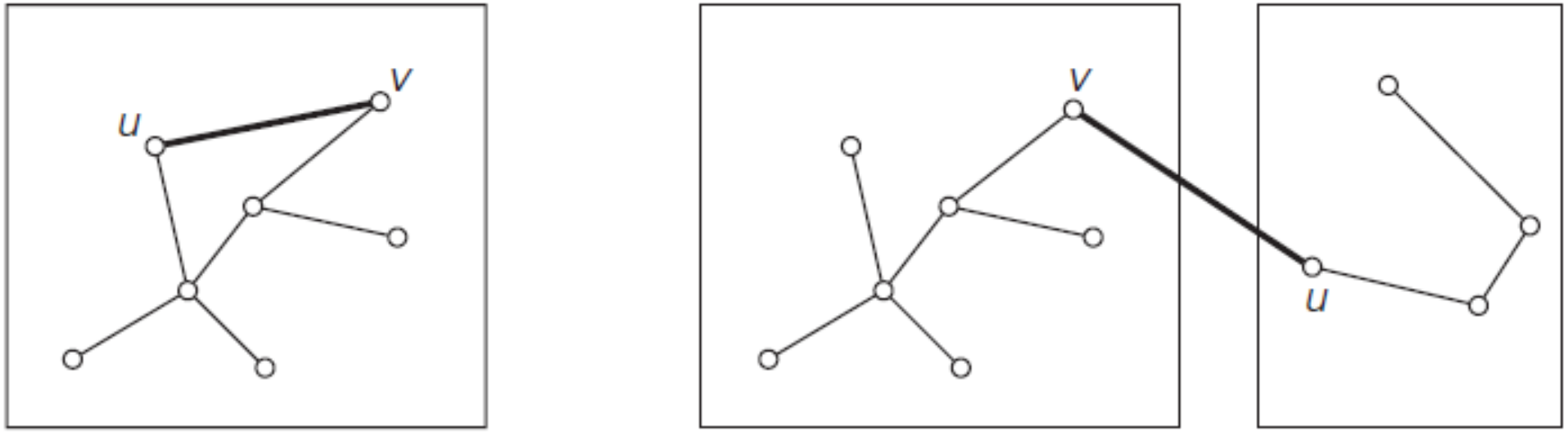
Kruskal's Algorithm

ALGORITHM *Kruskal(G)*

//Kruskal's algorithm for constructing a minimum spanning tree
//Input: A weighted connected graph $G = \langle V, E \rangle$
//Output: E_T, the set of edges composing a minimum spanning tree of G
sort E in nondecreasing order of the edge weights $w(e_{i_1}) \leq \cdots \leq w(e_{i_{|E|}})$
$E_T \leftarrow \varnothing$; $ecounter \leftarrow 0$ //initialize the set of tree edges and its size
$k \leftarrow 0$ //initialize the number of processed edges
while $ecounter < |V| - 1$ **do**
    $k \leftarrow k + 1$
    if $E_T \cup \{e_{i_k}\}$ is acyclic
        $E_T \leftarrow E_T \cup \{e_{i_k}\}$; $ecounter \leftarrow ecounter + 1$
return E_T

Design and Analysis of Algorithms

Kruskal's Algorithm



With an efficient Sorting algorithm and an Union-Find algorithm.

Efficiency: $\Theta(|E| \log |E|)$

Fortunately, there are efficient algorithms for doing so, including the crucial check for whether two vertices belong to the same tree. They are called union find algorithms. We discuss them in the following subsection. With an efficient union-find algorithm, the running time of Kruskal's algorithm will be dominated by the time needed for sorting the edge weights of a given graph. Hence, with an efficient sorting algorithm, the time efficiency of Kruskal's algorithm will be in $O(|E| \log |E|)$

6. **Algorithm** *Kruskal*(G)

```
//Kruskal's algorithm with explicit disjoint-subsets operations
//Input: A weighted connected graph G = ⟨V, E⟩
//Output: E_T, the set of edges composing a minimum spanning tree of G
sort E in nondecreasing order of the edge weights w(e_{i_1}) ≤ ... ≤ w(e_{i_|E|})
for each vertex v ∈ V make(v)
E_T ← ∅; ecounter ← 0        //initialize the set of tree edges and its size
k ← 0                        //the number of processed edges
while ecounter < |V| − 1
    k ← k + 1
    if find(u) ≠ find(v)     //u, v are the endpoints of edge e_{i_k}
        E_T ← E_T ∪ {e_{i_k}}; ecounter ← ecounter + 1
        union(u, v)
return E_T
```

if it's in the same group then we dont' create the edge between vertices-- done using find()
if it's in differnt groups then we merge them-done using- union()

Watch this video to understand the concept:
https://www.youtube.com/watch?v=JZBQLXgSGfs

Design and Analysis of Algorithms

Text Book

Chapter 9 ,Introduction to The Design and Analysis of Algorithms by Anany Levitin

THANK YOU

Bharathi R

Department of Computer Science & Engineering

rbharathi@pes.edu

Design and Analysis of Algorithms
Unit -4

Bharathi R

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Unit 4: Greedy Technique

Dijkstra's Algorithm

Bharathi R

Department of Computer Science & Engineering

Dijkstra's Algorithm

Single Source Shortest Paths Problem: Given a weighted connected (directed) graph G, find shortest paths from source vertex ***s*** to each of the other vertices

Dijkstra's algorithm: Similar to Prim's MST algorithm, with a different way of computing numerical labels: Among vertices not already in the tree, it finds vertex ***u*** with the smallest sum

$$d_v + w(v,u)$$

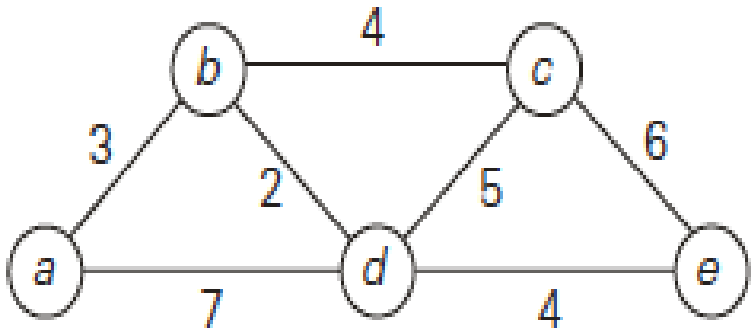
where

v is a vertex for which shortest path has been already found on preceding iterations (such vertices form a tree rooted at ***s***)

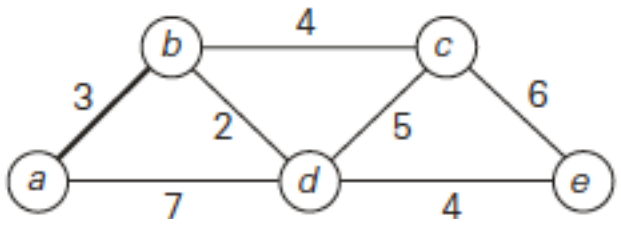
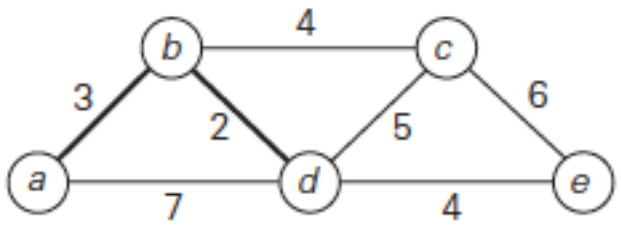
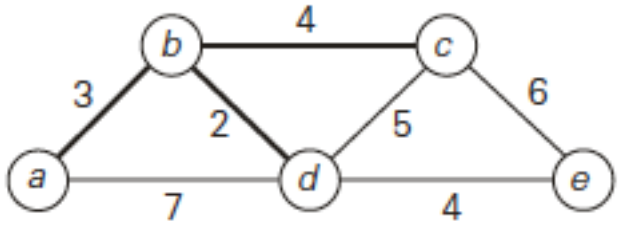
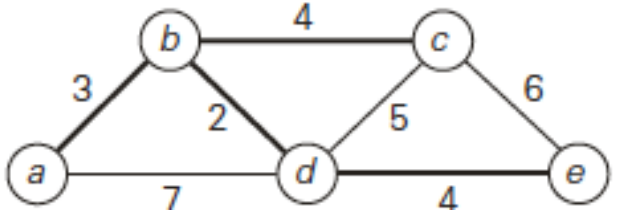
d_v is the length of the shortest path from source ***s*** to ***v***

$w(v,u)$ is the length (weight) of edge from ***v*** to ***u***

Example



Dijkstra's Algorithm-Example

Tree vertices	Remaining vertices	Illustration
a(−, 0)	**b(a, 3)** c(−, ∞) d(a, 7) e(−, ∞)	
b(a, 3)	c(b, 3 + 4) **d(b, 3 + 2)** e(−, ∞)	
d(b, 5)	**c(b, 7)** e(d, 5 + 4)	
c(b, 7)	**e(d, 9)**	
e(d, 9)		

The next closest vertex is shown in bold.

Design and Analysis of Algorithms

Dijkstra's Algorithm

ALGORITHM *Dijkstra*(G, s)

```
//Dijkstra's algorithm for single-source shortest paths
//Input: A weighted connected graph G = ⟨V, E⟩ with nonnegative weights
//        and its vertex s
//Output: The length d_v of a shortest path from s to v
//         and its penultimate vertex p_v for every vertex v in V
Initialize(Q)  //initialize priority queue to empty
for every vertex v in V
    d_v ← ∞;  p_v ← null
    Insert(Q, v, d_v)  //initialize vertex priority in the priority queue
d_s ← 0;  Decrease(Q, s, d_s)  //update priority of s with d_s
V_T ← ∅
for i ← 0 to |V| − 1 do
    u* ← DeleteMin(Q)  //delete the minimum priority element
    V_T ← V_T ∪ {u*}
    for every vertex u in V − V_T that is adjacent to u* do
        if d_u* + w(u*, u) < d_u
            d_u ← d_u* + w(u*, u);  p_u ← u*
            Decrease(Q, u, d_u)
```

Dijkstra's Algorithm- Time Efficiency

The time efficiency of Dijkstra's algorithm depends on the data structures used for implementing the priority queue and for representing an input graph itself.

- **$O(|V|^2)$** for graphs represented by weight matrix and array implementation of priority queue
- **$O(|E|\log|V|)$** for graphs represented by adj. lists and min-heap implementation of priority queue

Notes on Dijkstra's Algorithm

- Correctness can be proven by induction on the number of vertices.

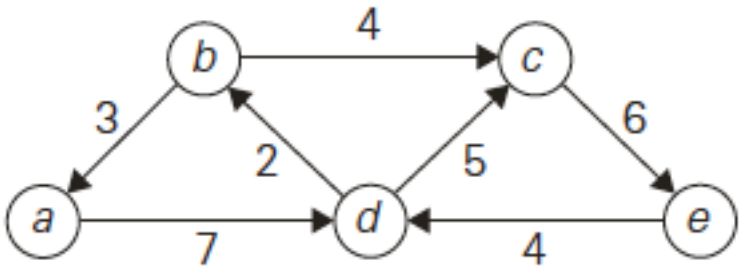
- Doesn't work for graphs with negative weights (whereas Floyd's algorithm does, as long as there is no negative cycle).

- Applicable to both undirected and directed graphs

- Don't mix up Dijkstra's algorithm with Prim's algorithm!

Exercises

Solve the following instances of the single-source shortest-paths problem with vertex a as the source:



References

Chapter-9 Greedy Technique

Introduction to the Design & Analysis of Algorithms- Anany Levitin

THANK YOU

Bharathi R

Department of Computer Science & Engineering

rbharathi@pes.edu

Design and Analysis of Algorithms
Unit -4

Bharathi R

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Unit 4: Greedy Technique- Huffman Trees

Bharathi R

Department of Computer Science & Engineering

Huffman Trees

The Huffman trees are constructed for encoding a given text of n characters.

While encoding a given text, each character is associated with some of bits called the ***codeword***.

- **Fixed length encoding**: Assigns to each character a bit string of the same length.
- **Variable length encoding**: Assigns codewords of different lengths to different characters.
- **Prefix free code**: In Prefix free code, no codeword is a prefix of a codeword of another character.

Binary prefix code :

- The characters are associated with the leaves of a binary tree.
- All left edges are labeled as 0, and right edges as 1.
- Codeword of a character is obtained by recording the labels on the simple path from the root to the character's leaf.
- Since, there is no simple path to a leaf that continues to another leaf, no codeword can be a prefix of another codeword.

About Huffman Algorithm:

- Invented by David A Huffman in 1951.
- Constructs binary prefix code tree.
- Huffman's algorithm achieves data compression by finding the best variable length binary encoding scheme for the symbols that occur in the file to be compressed. Huffman coding uses frequencies of the symbols in the string to build a variable rate prefix code
  - Each symbol is mapped to a binary string
  - More frequent symbols have shorter codes
  - No code is a prefix of another code (prefix free code)
- Huffman Codes for data compression achieves 20-90% Compression.

Huffman Algorithm:

Input: Alphabet and frequency of each symbol in the text.

Step 1: Initialize n one-node trees (forest) and label them with the symbols of the alphabet given. Record the frequency of each symbol in its tree's root to indicate the tree's weight. (Generally, the weight of a tree will be equal to the sum of the frequencies in the tree's leaves.)

Step 2: Repeat the following operation until a single tree is obtained. Find two trees with the smallest weight. Make them the left and right subtree of a new tree and record the sum of their weights in the root of the new tree as its weight.

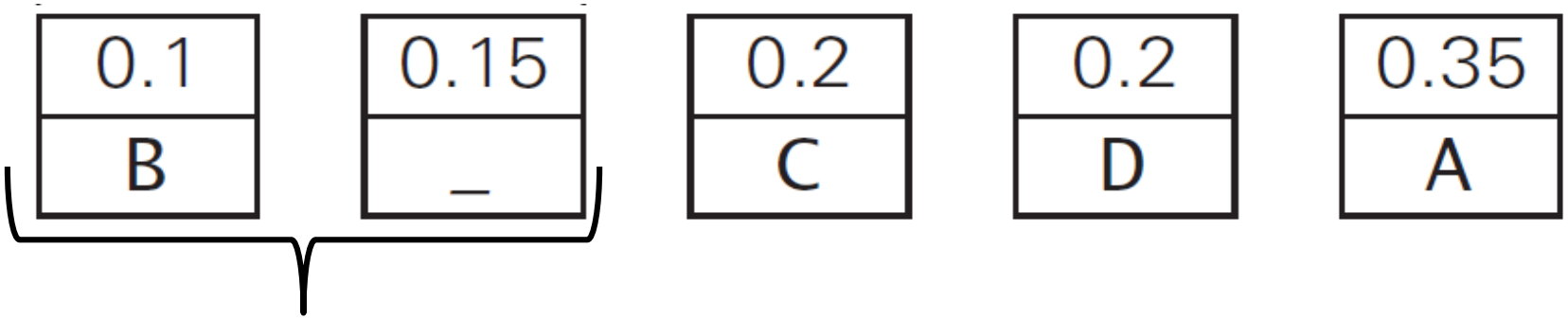
Design and Analysis of Algorithms

Huffman Algorithm:

EXAMPLE

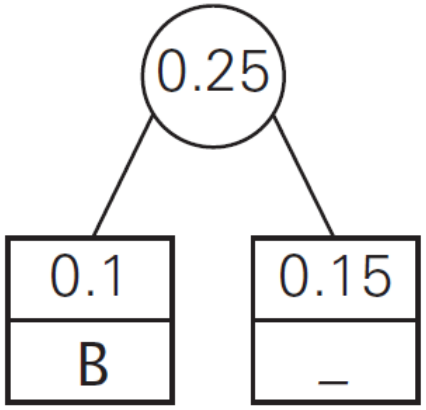
symbol	A	B	C	D	_
frequency	0.35	0.1	0.2	0.2	0.15

1.First arrange the characters in ascending order of their probabilities

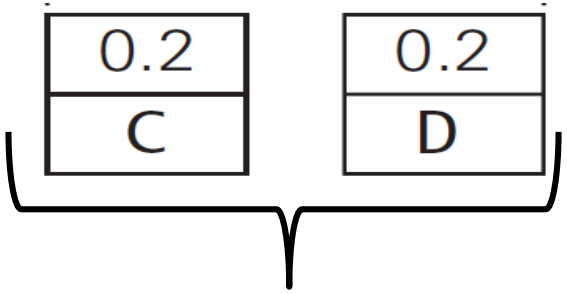


Combine these two nodes and form a single node

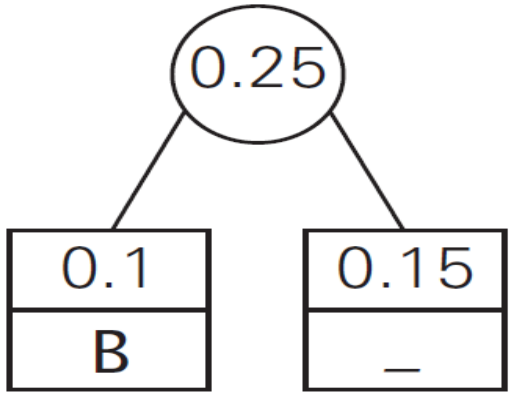
Huffman Algorithm:

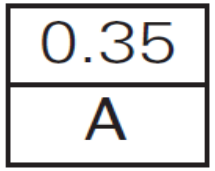


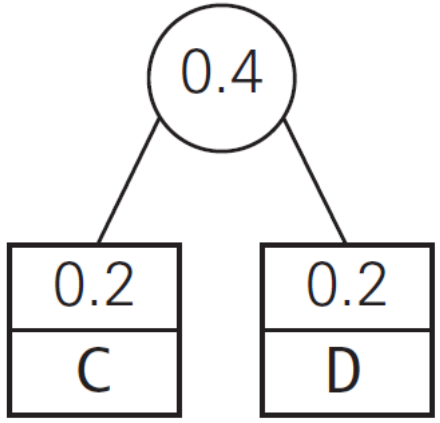
2.Again arrange probabilities in ascending order



Combine these nodes



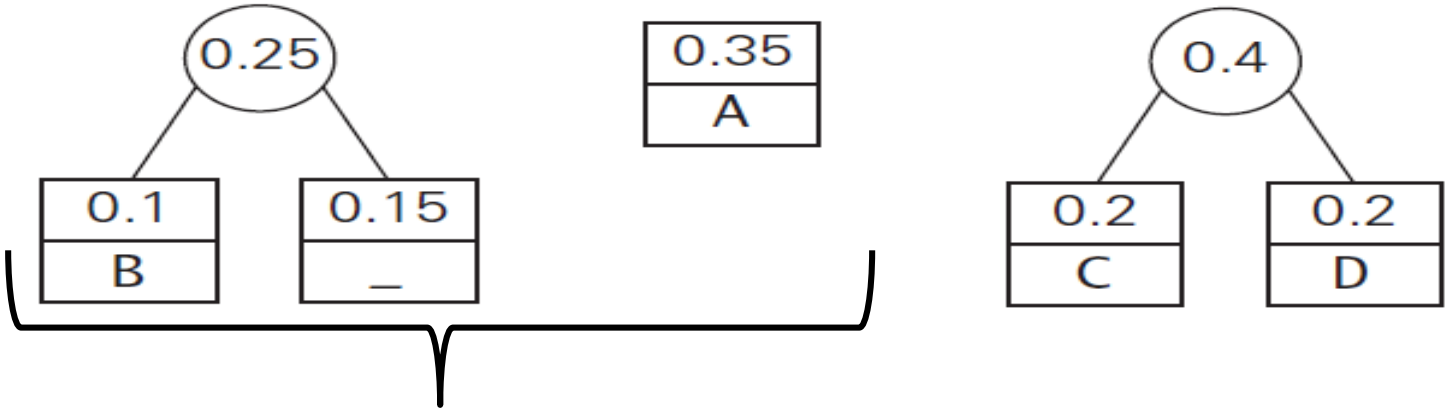




Design and Analysis of Algorithms

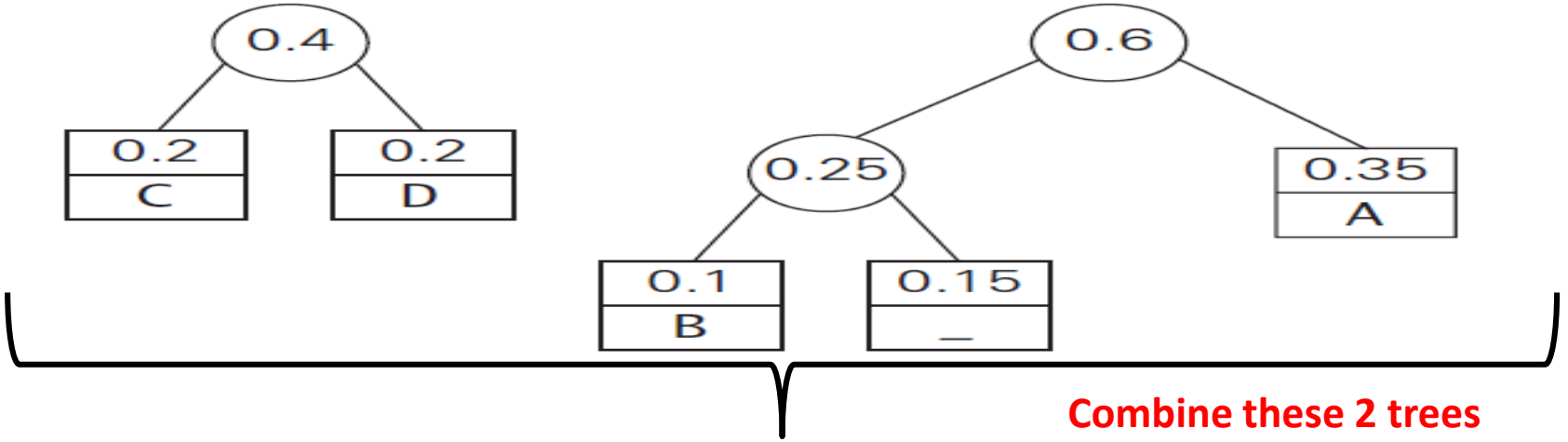
Huffman Algorithm:

3.Again arrange probabilities in ascending order



Combine these 2 trees

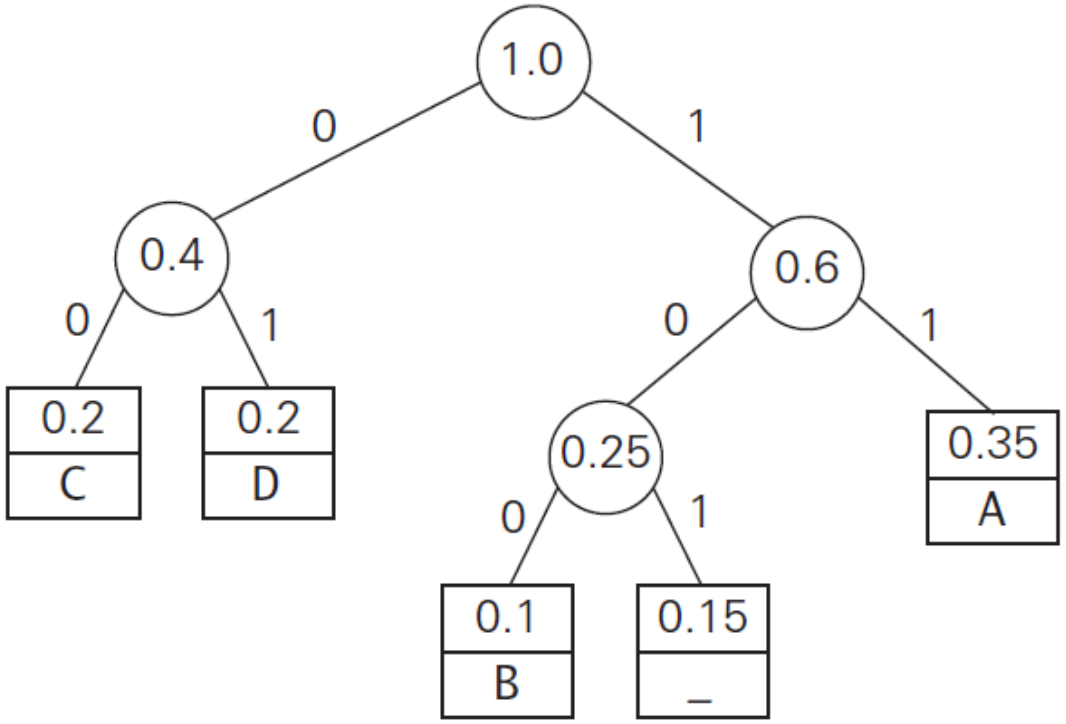
4. Arrange probabilities in ascending order



Combine these 2 trees

Design and Analysis of Algorithms

Huffman Tree



DAD is encoded as 011101, and
10011011011101 is decoded as BAD_AD

The resulting codewords are as follows

symbol	A	B	C	D	_
frequency	0.35	0.1	0.2	0.2	0.15
codeword	11	100	00	01	101

Compression Ratio

Compression Ratio : A standard measure of a compression algorithm's efficiency.

Character	A	B	C	D	-
probability	0.4	0.1	0.2	0.15	0.15

Character	A	B	C	D	-
probability	0.4	0.1	0.2	0.15	0.15
Code word	0	100	111	101	110

Compute compression ratio:

Bits per character = Codeword length * Frequency

= (1 * 0.4) + (3 * 0.1) + (3 * 0.2) + (3 * 0.15) + (3 * 0.15)

= 2.20

Compression ratio is = (3 – 2.20)/ 3 . 100% = 26.6%

Features of Huffman's Encoding

- Huffman's encoding is one of the most important file-compression methods.
- In addition to its simplicity and versatility, it yields an optimal, i.e., minimal-length, encoding (provided the frequencies of symbol occurrences are independent and known in advance).
- The simplest version of Huffman compression calls, in fact, for a preliminary scanning of a given text to count the frequencies of symbol occurrences in it. Then these frequencies are used to construct a Huffman coding tree and encode the text.

Design and Analysis of Algorithms

Text Books

Chapter-9 Greedy Technique
Introduction to the Design & Analysis of Algorithms- Anany Levitin

THANK YOU

Bharathi R

Department of Computer Science & Engineering

rbharathi@pes.edu

Design and Analysis of Algorithms

Dynamic Programming

Reetinder Sidhu

Department of Computer Science and Engineering

Design and Analysis of Algorithms

Dynamic Programming

Reetinder Sidhu

Department of Computer Science and Engineering

Dynamic Programming

Unit 5: Limitations of Algorithmic Power and Coping with the Limitation

- Dynamic Programming
  - **Computing a Binomial Coefficient**
  - The Knapsack Problem
  - Memory Functions
  - Warshall's and Floyd's Algorithms
- Limitations of Algorithmic Power
  - Lower-Bound Arguments
  - Decision Trees
  - P, NP, and NP-Complete, NP-Hard Problems
- Coping with the Limitations
  - Backtracking
  - Branch-and-Bound

Concepts covered

- Dynamic Programming
  - Introduction
  - Fibonacci numbers
  - Binomial Coefficients

Dynamic Programming

Introduction

Dynamic Programming is a general algorithm design technique for solving problems defined by recurrences with overlapping subproblems

- Invented by American mathematician Richard Bellman in the 1950s to solve optimization problems and later assimilated by CS
- "Programming" here means "planning"
- Main idea:
  - set up a recurrence relating a solution to a larger instance to solutions of some smaller instances
  - solve smaller instances once
  - record solutions in a table
  - extract solution to the initial instance from that table

Dynamic Programming
Example: Fibonacci Numbers

- Recall definition of Fibonacci numbers:

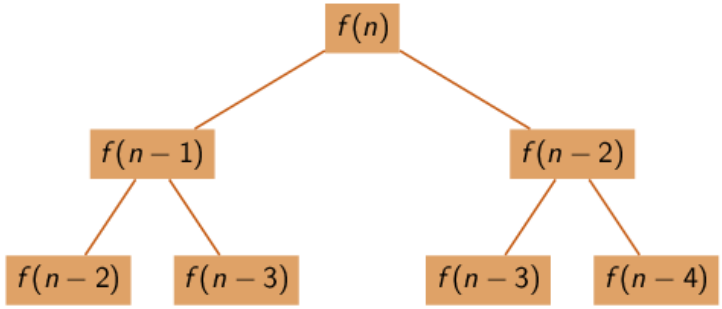
$$f(n) = f(n-1) + f(n-2)$$
$$f(0) = 0$$
$$f(1) = 1$$

Dynamic Programming
Example: Fibonacci Numbers

- Recall definition of Fibonacci numbers:

$$f(n) = f(n-1) + f(n-2)$$
$$f(0) = 0$$
$$f(1) = 1$$

- Computing the n^{th} Fibonacci number recursively (top-down):



Dynamic Programming
Example: Fibonacci Numbers

Computing the nth Fibonacci number using bottom-up iteration and recording results:

$$
\begin{aligned}
f(0) &= 0 \\
f(1) &= 1 \\
f(2) &= 0 + 1 = 1 \\
f(3) &= 1 + 1 = 2 \\
f(4) &= 1 + 2 = 3 \\
&\vdots
\end{aligned}
$$

Dynamic Programming
Example: Fibonacci Numbers

Computing the nth Fibonacci number using bottom-up iteration and recording results:

$$f(0) = 0$$
$$f(1) = 1$$
$$f(2) = 0 + 1 = 1$$
$$f(3) = 1 + 1 = 2$$
$$f(4) = 1 + 2 = 3$$
$$\vdots$$

Efficiency:

- time: $\Theta(n)$
- space: $\Theta(n)$ or $\Theta(1)$

Dynamic Programming
Algorithm Examples

- Computing a binomial coefficient
- Warshall's algorithm for transitive closure
- Floyd's algorithm for all-pairs shortest paths
- Constructing an optimal binary search tree
- Some instances of difficult discrete optimization problems:
  - traveling salesman
  - knapsack

Dynamic Programming

Binomial Coefficient

- Binomial coefficients are coefficients of the binomial formula:

$$(a+b)^n = C(n,0)a^n b^0 + \ldots + C(n,k)a^{n-k}b^k + \ldots + C(n,n)a^0 b^n$$

Dynamic Programming

Binomial Coefficient

- Binomial coefficients are coefficients of the binomial formula:

$$(a+b)^n = C(n,0)a^n b^0 + \ldots + C(n,k)a^{n-k}b^k + \ldots + C(n,n)a^0 b^n$$

- Recurrence:

$C(n,k) = C(n-1,k) + C(n-1,k-1) \quad \text{for } n > k > 0$

$C(n,0) = 1, C(n,n) = 1 \quad \text{for } n \geq 0$

Dynamic Programming

Binomial Coefficient

- Binomial coefficients are coefficients of the binomial formula:

$$(a+b)^n = C(n,0)a^n b^0 + \ldots + C(n,k)a^{n-k}b^k + \ldots + C(n,n)a^0 b^n$$

- Recurrence:

$$C(n,k) = C(n-1,k) + C(n-1,k-1) \quad \text{for } n > k > 0$$

$$C(n,0) = 1, C(n,n) = 1 \quad \text{for } n \geq 0$$

- Value of C(n,k) can be computed by filling a table:

	0	1	2	.	.	.	k-1	k
0	1							
1	1	1						
.								
.								
.								
n-1							C(n-1,k-1)	C(n-1,k)
n								C(n,k)

Dynamic Programming
Binomial Coefficient Algorithm

Dynamic Programming Binomial Coefficient Algorithm

```
1: procedure BINOMIAL(n, k)
2:     ▷ Input: Integers n ≥ 0, k ≥ 0
3:     ▷ Output: C(n, k)
4:     for i ← 0 to n do
5:         for j ← 0 to min(i, k) do
6:             if j=0 or j=i then
7:                 C(i, j) ← 1
8:             else C[i, j] = C[i − 1, j] + C[i − 1, j − 1]
9:     return C[n, k]
```

Dynamic Programming
Binomial Coefficient Algorithm

Dynamic Programming Binomial Coefficient Algorithm

```
1: procedure BINOMIAL(n, k)
2:     ▷ Input: Integers n ≥ 0, k ≥ 0
3:     ▷ Output: C(n, k)
4:     for i ← 0 to n do
5:         for j ← 0 to min(i, k) do
6:             if j=0 or j=i then
7:                 C(i, j) ← 1
8:             else C[i, j] = C[i − 1, j] + C[i − 1, j − 1]
9:     return C[n, k]
```

- Time: $\Theta(nk)$
- Space: $\Theta(nk)$

Dynamic Programming

Think About It

- What does dynamic programming have in common with divide-and-conquer? What is a principal difference between them?

Dynamic Programming

Think About It

- What does dynamic programming have in common with divide-and-conquer? What is a principal difference between them?
- The coin change problem does not have an optimal greedy solution in all cases (*ex:* coins 1,20,25 and amount 40). Is there a dynamic programming based algorithm that can solve all cases of the coin change problem?

Design and Analysis of Algorithms

The Knapsack Problem

Reetinder Sidhu

Department of Computer Science and Engineering

Design and Analysis of Algorithms

The Knapsack Problem

Reetinder Sidhu

Department of Computer Science and Engineering

The Knapsack Problem

Unit 5: Limitations of Algorithmic Power and Coping with the Limitations

- Dynamic Programming
  - Computing a Binomial Coefficient
  - **The Knapsack Problem**
  - Memory Functions
  - Warshall's and Floyd's Algorithms
- Limitations of Algorithmic Power
  - Lower-Bound Arguments
  - Decision Trees
  - P, NP, and NP-Complete, NP-Hard Problems
- Coping with the Limitations
  - Backtracking
  - Branch-and-Bound. Architecture (microprocessor instruction set)

Concepts covered

- The Knapsack Problem
  - Introduction
  - Recurrence
  - Example

The Knapsack Problem

Problem Definition

- Given
  - n items of integer weights : w_1 w_2 ... w_n
    values : v_1 v_2 ... v_n
  - knapsack of capacity W (integer $W > 0$)
- Find the most valuable subset of items such that sum of their weights does not exceed W

The Knapsack Problem
Kanpsack Recurrence

- To design a dynamic programming algorithm, we need to derive a recurrence relation that expresses a solution to an instance of the knapsack problem in terms of solutions to its smaller subinstances
- Consider the smaller knapsack problem where number of items is i ($i \leq n$) and the knapsack capacity id j ($j \leq W$)
- Then

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ \ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12

	capacity j				
i	1	2	3	4	5
1					
2					
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12

	capacity j				
i	1	2	3	4	5
1	0				
2					
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12

	capacity j				
i	1	2	3	4	5
1	0	12			
2					
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12

	capacity j				
i	1	2	3	4	5
1	0	12	12		
2					
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	
2					
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2					
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2					
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10				
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12			
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22		
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3					
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12			
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22		
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	32
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	32
4					

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	32
4	10				

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	32
4	10	15			

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	32
4	10	15	25		

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	32
4	10	15	25	30	

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	32
4	10	15	25	30	37

What is the maximum value that can be stored in a knapsack of capacity 6?

The Knapsack Problem
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

What is the maximum value that can be stored in a knapsack of capacity 6?

	capacity j				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	32
4	10	15	25	30	37

Given above 6 items, maximum value that can be stored in a knapsack of capacity 5 is **37**

The Knapsack Problem
Complexity

- Space complexity: $\Theta(nW)$
- Time complexity: $\Theta(nW)$
- Time to compose optimal solution: $O(n)$

Design and Analysis of Algorithms

Memory Function Knapsack

Reetinder Sidhu

Department of Computer Science and Engineering

Design and Analysis of Algorithms

Memory Function Knapsack

Reetinder Sidhu

Department of Computer Science and Engineering

Memory Function Knapsack

Unit 5: Limitations of Algorithmic Power and Coping with the Limitations

- Dynamic Programming
  - Computing a Binomial Coefficient
  - **The Knapsack Problem**
  - Memory Functions
  - Warshall's and Floyd's Algorithms
- Limitations of Algorithmic Power
  - Lower-Bound Arguments
  - Decision Trees
  - P, NP, and NP-Complete, NP-Hard Problems
- Coping with the Limitations
  - Backtracking
  - Branch-and-Bound. Architecture (microprocessor instruction set)

Concepts covered

- Memory Function Knapsack
  - Motivation
  - Algorithm
  - Example

Memory Function Knapsack
Bottom Up Approach

Memory Function Knapsack
Bottom Up Approach

- Advantage of bottom up approach: each value computed only once

Memory Function Knapsack
Bottom Up Approach

- Advantage of bottom up approach: each value computed only once
- Example computed bottom up:

	capacity *j*				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	32
4	10	15	25	30	37

Memory Function Knapsack
Bottom Up Approach

- Advantage of bottom up approach: each value computed only once
- Example computed bottom up:

	capacity *j*				
i	1	2	3	4	5
1	0	12	12	12	12
2	10	12	22	22	22
3	10	12	22	30	32
4	10	15	25	30	37

- Disadvantage of bottom up approach: values not required also computed

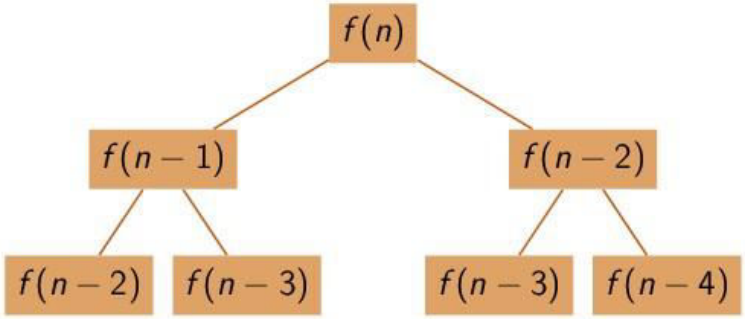
Memory Function Knapsack
Top Down Approach

Memory Function Knapsack
Top Down Approach

- Disadvantage of top down approach: same problem solved multiple times

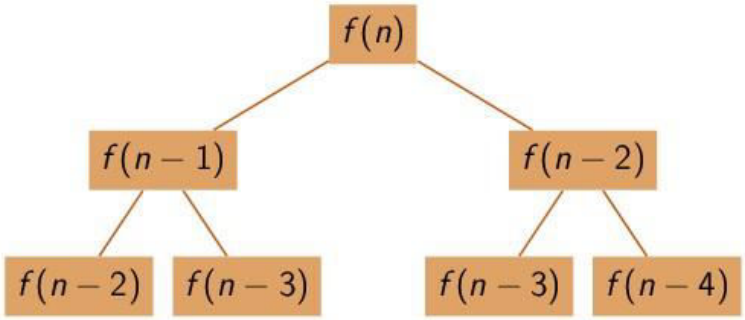
Memory Function Knapsack
Top Down Approach

- Disadvantage of top down approach: same problem solved multiple times
- Example computed top down:



Memory Function Knapsack
Top Down Approach

- Disadvantage of top down approach: same problem solved multiple times
- Example computed top down:



- Advantage of top down approach: only the required subproblems solved

Memory Function Knapsack
Memory Function Dynamic Programming

- Combine the advantages of bottom up and top down approaches:
  - compute each subproblem only once
  - compute only the required subproblems

Memory Function Knapsack
MF-DP Algorithm

Algorithm for Memory Function Dynamic Programming

```
1:  procedure MFKnapsack(i, j)
2:      ▷ Inputs: i indicating the number items, and
3:      ▷ j, indicating the knapsack capacity
4:      ▷ Output: The value of an optimal feasible subset of the first i items
5:      ▷ Note: Uses global variables input arrays Weights[1 . . . n], Values[1 . . . n],
6:      ▷ and table F[0 . . . n, 0 . . . W] whose entries are initialized with −1's except
7:      ▷ row 0 and column 0 is initialized with 0
8:      if F[i, j] < 0 then
9:          if j < Weights[i] then
10:             value ← MFKnapsack(i − 1, j)
11:         else value ← max(MFKnapsack(i − 1, j), Values[i] + MFKnapsack(i − 1, j − Weights[i]))
12:             F[i, j] ← value
13:     return F[i, j]
```

Memory Function Knapsack
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

What is the maximum value that can be stored in a knapsack of capacity 5?

Memory Function Knapsack
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

What is the maximum value that can be stored in a knapsack of capacity 5?

	capacity j					
i	0	1	2	3	4	5
0	0					
1						
2						
3						
4						□

Memory Function Knapsack
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

What is the maximum value that can be stored in a knapsack of capacity 5?

	capacity j					
i	0	1	2	3	4	5
0	0					
1						
2						
3						□
4						□

Memory Function Knapsack
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

What is the maximum value that can be stored in a knapsack of capacity 5?

	capacity j					
i	0	1	2	3	4	5
0	0					
1						
2						
3				□		□
4						□

Memory Function Knapsack
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

What is the maximum value that can be stored in a knapsack of capacity 5?

	capacity j					
i	0	1	2	3	4	5
0	0					
1						
2						
3						
4						

Memory Function Knapsack
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

What is the maximum value that can be stored in a knapsack of capacity 5?

	capacity j					
i	0	1	2	3	4	5
0	0	□	□	□	□	□
1	□	□	□	□	□	□
2	□	–	□	□	–	□
3	□	–	–	□	–	□
4	□	–	–	–	–	□

Memory Function Knapsack
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

What is the maximum value that can be stored in a knapsack of capacity 5?

	capacity j					
i	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	–	12	22	–	22
3	0	–	–	22	–	32
4	0	–	–	–	–	37

Memory Function Knapsack
Example

$$F(i,j) = \begin{cases} \max(F(i-1,j),\ v_i + F(i-1,j-w_i)) & \text{if } j - w_i \geq 0 \\ F(i-1,j) & \text{if } j - w_i < 0 \end{cases}$$

Dynamic Programming Example

item i	weight w_i	value v_i
1	2	12
2	1	10
3	3	20
4	2	15

What is the maximum value that can be stored in a knapsack of capacity 5?

	capacity j					
i	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	–	12	22	–	22
3	0	–	–	22	–	32
4	0	–	–	–	–	37

Knapsack problem solved by

- computing 21 out 30 possible subproblems
- reusing subproblem entry $(1,2)$

Memory Function Knapsack
Complexity

- Constant factor imrpovement in efficiency
  - Space complexity: $\Theta(nW)$
  - Time complexity: $\Theta(nW)$
  - Time to compose optimal solution: $O(n)$
- Bigger gains possible where computation of a subproblem takes more than constant time

Memory Function Knapsack

Think About It

- Consider the use of the MF technique to compute binomial coefficient using the recurrence

$$C(n,k) = C(n-1,k-1) + C(n-1,k)$$

  - How many table entires are filled?
  - How many are reused?

Design and Analysis of Algorithms

Transitive Closure (Warshall's Algorith

Reetinder Sidhu

Department of Computer Science and Engineering

Design and Analysis of Algorithms

Transitive Closure (Warshall's Algorithm)

Reetinder Sidhu

Department of Computer Science and Engineering

Transitive Closure (Warshall's Algorithm)

Unit 5: Limitations of Algorithmic Power and Coping with the Limitations

- Dynamic Programming
  - Computing a Binomial Coefficient
  - The Knapsack Problem
  - Memory Functions
  - **Warshall's and Floyd's Algorithms**
- Limitations of Algorithmic Power
  - Lower-Bound Arguments
  - Decision Trees
  - P, NP, and NP-Complete, NP-Hard Problems
- Coping with the Limitations
  - Backtracking
  - Branch-and-Bound. Architecture (microprocessor instruction set)

Concepts covered

- Transitive Closure (Warshall's Algorithm)
  - Motivation
  - Algorithm
  - Example

Transitive Closure (Warshall's Algorithm)

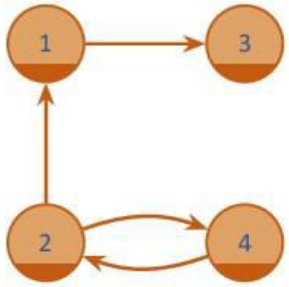
Transitive Closure

- Computes the transitive closure of a relation
- Alternatively: existence of all nontrivial paths in a digraph (directed graph)
- Example of transitive closure:

TRANSITIVE CLOSURE (WARSHALL'S ALGORITHM)

Transitive Closure

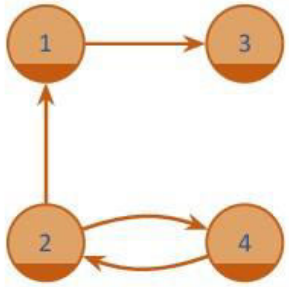
- Computes the transitive closure of a relation
- Alternatively: existence of all nontrivial paths in a digraph (directed graph)
- Example of transitive closure:



Transitive Closure (Warshall's Algorithm)

Transitive Closure

- Computes the transitive closure of a relation
- Alternatively: existence of all nontrivial paths in a digraph (directed graph)
- Example of transitive closure:

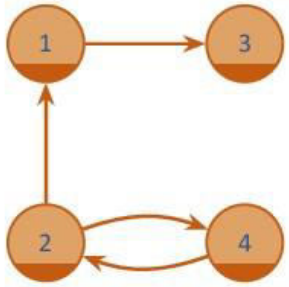


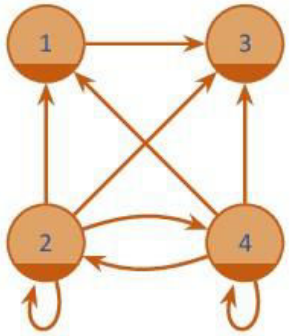
	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0

Transitive Closure (Warshall's Algorithm)

Transitive Closure

- Computes the transitive closure of a relation
- Alternatively: existence of all nontrivial paths in a digraph (directed graph)
- Example of transitive closure:



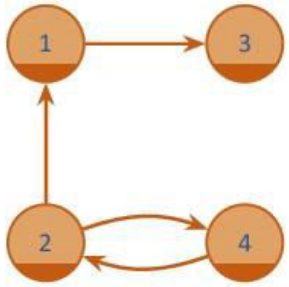


	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0

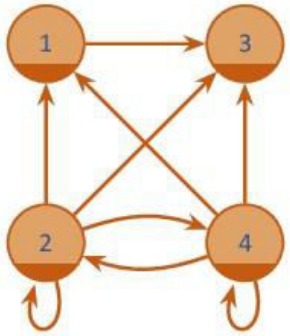
Transitive Closure (Warshall's Algorithm)

Transitive Closure

- Computes the transitive closure of a relation
- Alternatively: existence of all nontrivial paths in a digraph (directed graph)
- Example of transitive closure:



	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0



	1	2	3	4
1	0	0	1	0
2	1	**1**	**1**	1
3	0	0	0	0
4	**1**	1	**1**	**1**

Transitive Closure (Warshall's Algorithm)

Warshall's Algorithm

- Constructs transitive closure T as the last matrix in the sequence of $n \times n$ matrices $R^{(0)}, \ldots, R^{(k)}, \ldots, R^{(n)}$ where $R^{(k)}[i,j] = 1$ iff there is nontrivial path from i to j with only first k vertices allowed as intermediate vertices
  - $R^{(0)} = A$ (adjacency matrix), $R^{(n)} = T$ (transitive closure)
- On the k^{th} iteration, the algorithm computes $R^{(k)}$

$$R^{(k)}[i,j] = \begin{cases} 1 & \text{if path from } i \text{ to } k \text{ and } k \text{ to } j, \text{ i.e., } R^{(k-1)}[i,k] = R^{(k-1)}[k,j] = 1 \\ R^{(k-1)}[i,j] & \text{otherwise} \end{cases}$$

Transitive Closure (Warshall's Algorithm)

Warshall's Algorithm

- Constructs transitive closure T as the last matrix in the sequence of $n \times n$ matrices $R^{(0)}, \ldots, R^{(k)}, \ldots, R^{(n)}$ where $R^{(k)}[i,j] = 1$ iff there is nontrivial path from i to j with only first k vertices allowed as intermediate vertices
  - $R^{(0)} = A$ (adjacency matrix), $R^{(n)} = T$ (transitive closure)
- On the k^{th} iteration, the algorithm computes $R^{(k)}$

$$R^{(k)}[i,j] = \begin{cases} 1 & \text{if path from } i \text{ to } k \text{ and } k \text{ to } j, \text{ i.e., } R^{(k-1)}[i,k] = R^{(k-1)}[k,j] = 1 \\ R^{(k-1)}[i,j] & \text{otherwise} \end{cases}$$

$$R^{(k)}[i,j] = R^{(k-1)}[i,j] \textbf{ or } (R^{(k-1)}[i,k] \textbf{ and } R^{(k-1)}[k,j])$$

Transitive Closure (Warshall's Algorithm)

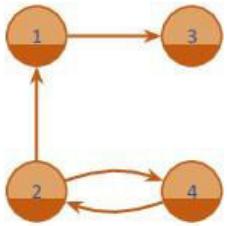
Algorithm

Transitive Closure (Warshall's Algorithm)

1: **procedure** WARSHALL(()$A[1 \ldots n, 1 \ldots n]$)
2: ▷ **Input:** The adjacency matrix A of a digraph with n vertices
3: ▷ **Output:** The transitive closure of the digraph
4: $R^{(0)} \leftarrow A$
5: **for** $k \leftarrow 1$ **to** n **do**
6: **for** $i \leftarrow 1$ **to** n **do**
7: **for** $j \leftarrow 1$ **to** n **do**
8: $R^{(k)}[i,j] \leftarrow R^{(k-1)}[i,j]$ **or** $(R^{(k-1)}[i,k]$ **and** $R^{(k-1)}[k,j])$;
9: **return** R

Transitive Closure (Warshall's Algorithm)

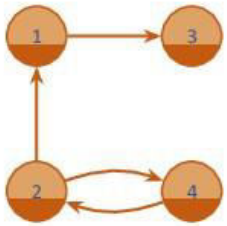
Warshall's Algorithm



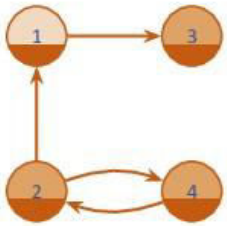
	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0

Transitive Closure (Warshall's Algorithm)

Warshall's Algorithm



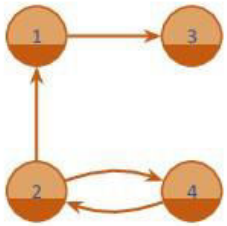
	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0



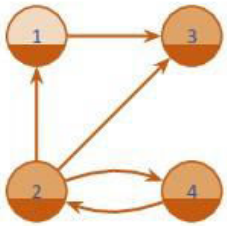
	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0

Transitive Closure (Warshall's Algorithm)

Warshall's Algorithm



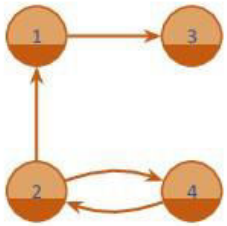
	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0



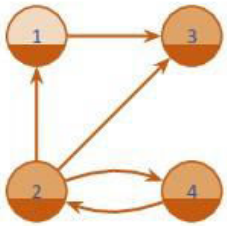
	1	2	3	4
1	0	0	1	0
2	1	0	**1**	1
3	0	0	0	0
4	0	1	0	0

TRANSITIVE CLOSURE (WARSHALL'S ALGORITHM)

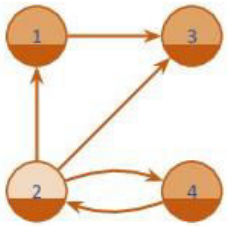
Warshall's Algorithm



	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0



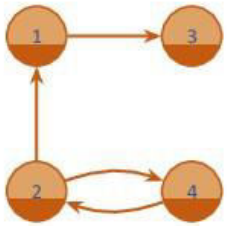
	1	2	3	4
1	0	0	1	0
2	1	0	**1**	1
3	0	0	0	0
4	0	1	0	0



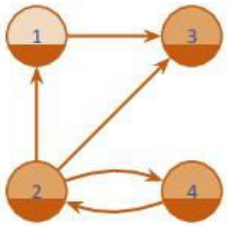
	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	0	1	0	0

TRANSITIVE CLOSURE (WARSHALL'S ALGORITHM)

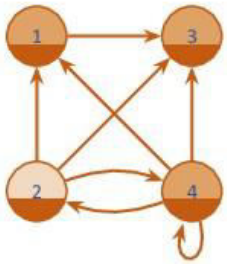
Warshall's Algorithm



	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0



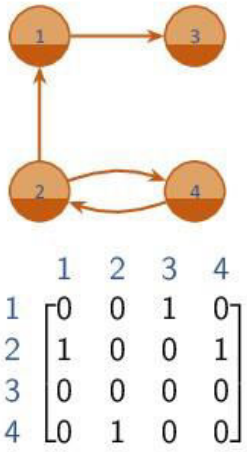
	1	2	3	4
1	0	0	1	0
2	1	0	**1**	1
3	0	0	0	0
4	0	1	0	0



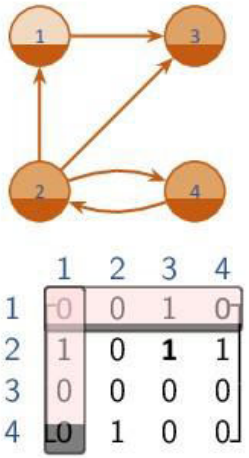
	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	**1**	**1**	**1**	**1**

TRANSITIVE CLOSURE (WARSHALL'S ALGORITHM)

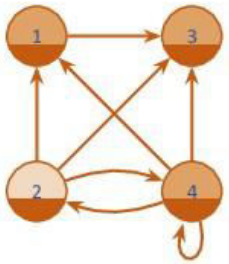
Warshall's Algorithm



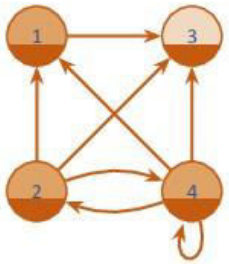
	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0



	1	2	3	4
1	0	0	1	0
2	1	0	**1**	1
3	0	0	0	0
4	0	1	0	0



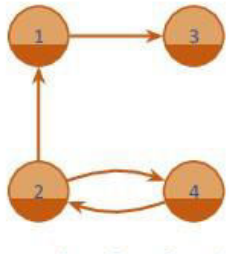
	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	**1**	**1**	**1**	**1**



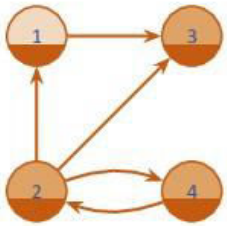
	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	1	1	1	1

TRANSITIVE CLOSURE (WARSHALL'S ALGORITHM)

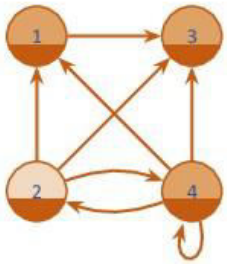
Warshall's Algorithm



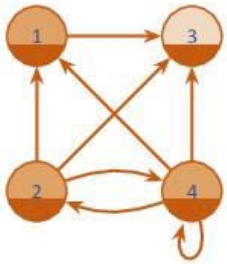
	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0



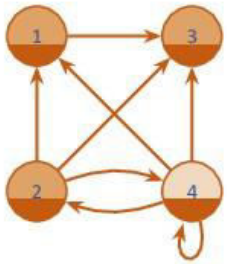
	1	2	3	4
1	0	0	1	0
2	1	0	**1**	1
3	0	0	0	0
4	0	1	0	0



	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	**1**	**1**	**1**	**1**



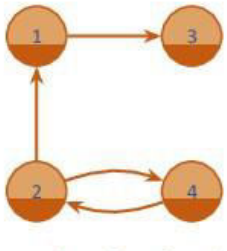
	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	1	1	1	1



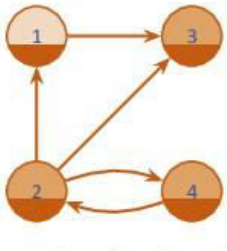
	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	1	1	1	1

TRANSITIVE CLOSURE (WARSHALL'S ALGORITHM)

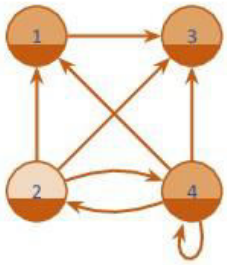
Warshall's Algorithm



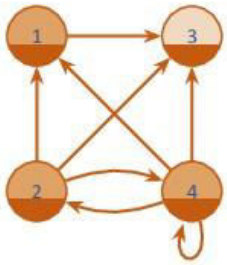
	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0



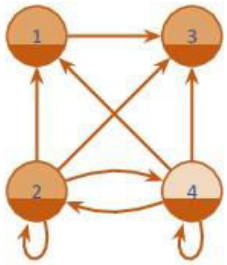
	1	2	3	4
1	0	0	1	0
2	1	0	**1**	1
3	0	0	0	0
4	0	1	0	0



	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	**1**	**1**	**1**	**1**



	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	1	1	1	1



	1	2	3	4
1	0	0	1	0
2	1	**1**	1	1
3	0	0	0	0
4	1	1	1	1

Transitive Closure (Warshall's Algorithm)

Think About It

- Is Warshall's algorithm efficient for sparse graphs? Why / why not?
- Can Warshall's algorithm be used to determine if a graph is a DAG (Directed Acyclic Graph)? If so, how?

1. Is Warshall's algorithm efficient for sparse graphs? Why / why not?

Warshall's algorithm (or Floyd-Warshall) has a time complexity of **O(V^3)**, where V is the number of vertices in the graph.

- **Efficiency Concern for Sparse Graphs:**
  - A **sparse graph** has significantly fewer edges compared to the maximum possible edges (V^2).
  - Warshall's algorithm does not take advantage of sparsity and still performs **O(V^3)** operations, even if the graph has very few edges.
  - More efficient algorithms for sparse graphs include **Depth-First Search (DFS) or Breadth-First Search (BFS)** for reachability computation, which run in **O(V + E)**.
- **Conclusion:**
  Warshall's algorithm is **not efficient** for sparse graphs due to its cubic complexity, making it slower compared to specialized algorithms that can exploit sparsity.

2. Can Warshall's algorithm be used to determine if a graph is a DAG (Directed Acyclic Graph)? If so, how?

Yes, Warshall's algorithm can be used to check if a graph is a **DAG**.

- **How?**
  - Compute the **transitive closure** using Warshall's algorithm.
  - Check the **diagonal elements** of the resultant reachability matrix.
  - If any **diagonal element (i, i) = 1**, it means there is a cycle (i.e., a path from a vertex to itself).
  - If **all diagonal elements are 0**, the graph is acyclic (DAG).
- **Why does this work?**
  - A **DAG** has **no cycles**, so no node should be reachable from itself through an indirect path.
- **Alternative Efficient Method:**
  - A **topological sorting** approach (using DFS or Kahn's algorithm) is a **better and more efficient** way to check if a graph is a DAG in **O(V + E)** time.

Design and Analysis of Algorithms

All Pairs Shortest Path (Floyd's Algorithm

Reetinder Sidhu

Department of Computer Science and Engineering

Design and Analysis of Algorithms

All Pairs Shortest Path (Floyd's Algorithm)

Reetinder Sidhu

Department of Computer Science and Engineering

All Pairs Shortest Path (Floyd's Algorithm)

Unit 5: Limitations of Algorithmic Power and Coping with the Limitations

- Dynamic Programming
  - Computing a Binomial Coefficient
  - The Knapsack Problem
  - Memory Functions
  - **Warshall's and Floyd's Algorithms**
- Limitations of Algorithmic Power
  - Lower-Bound Arguments
  - Decision Trees
  - P, NP, and NP-Complete, NP-Hard Problems
- Coping with the Limitations
  - Backtracking
  - Branch-and-Bound. Architecture (microprocessor instruction set)

Concepts covered

- All Pairs Shortes Path (Floyd's Algorithm)
  - Definition
  - Algorithm
  - Example

All Pairs Shortest Path (Floyd's Algorithm)

Problem Definition

- Given a undirected or directed graph, with weighted edges, find the shortest path between every pair of vertices
  - Dijkstra's algorithm found shortest paths from given vertex to remaining $n-1$ vertices ($\Theta(n)$ paths)
  - Current problem is to find the shortest path between every pair of vertices ($\Theta(n^2)$ paths)
- Solution approach is similar to the transitive closure approach: Compute transitive closure via sequence of $n \times n$ matrices $R^{(0)}, \ldots, R^{(k)}, \ldots, R^{(n)}$ where $R^{(k)}[i,j] = 1$ iff there is nontrivial path from i to j with only first k vertices allowed as intermediate vertices
- Compute all pairs shortest paths via sequence of $n \times n$ matrices $D^{(0)}, \ldots, D^{(k)}, \ldots, D^{(n)}$ where $D^{(k)}[i,j]$ is the shortest path from i to j with only first k vertices allowed as intermediate vertices

All Pairs Shortest Path (Floyd's Algorithm)

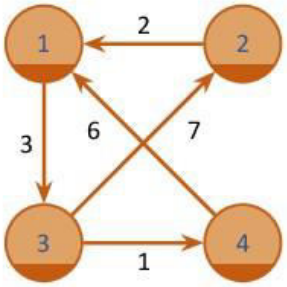
Example

- Example of all pairs shortest paths:

All Pairs Shortest Path (Floyd's Algorithm)

Example

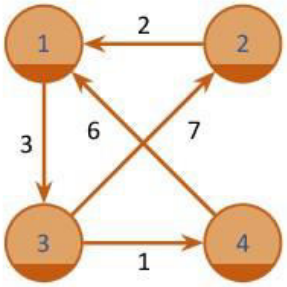
- Example of all pairs shortest paths:



All Pairs Shortest Path (Floyd's Algorithm)

Example

- Example of all pairs shortest paths:

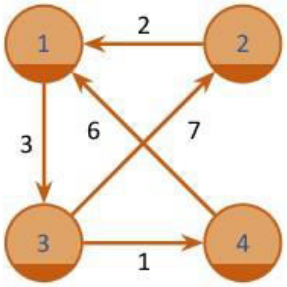


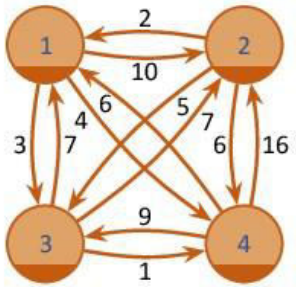
$$
\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array}
\begin{array}{c}
\begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\
\begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix}
\end{array}
$$

All Pairs Shortest Path (Floyd's Algorithm)

Example

- Example of all pairs shortest paths:



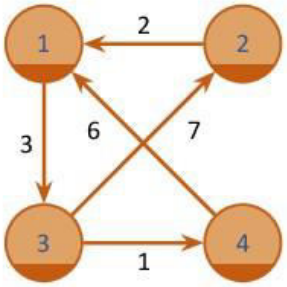


$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{array}$$

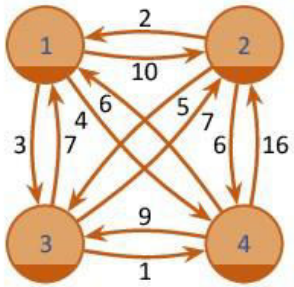
All Pairs Shortest Path (Floyd's Algorithm)

Example

- Example of all pairs shortest paths:



	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0



	1	2	3	4
1	0	10	3	4
2	2	0	5	6
3	7	7	0	1
4	6	16	9	0

All Pairs Shortest Path (Floyd's Algorithm)

Algorithm

Transitive Closure (Floyd's Algorithm)

```
1: procedure FLOYD(()A[1 . . . n, 1 . . . n])
2:     ▷ Input: Weight matrix A of a graph with no negative length cycles
3:     ▷ Output: Distance matrix of shortest paths
4:     D ← W
5:     for k ← 1 to n do
6:         for i ← 1 to n do
7:             for j ← 1 to n do
8:                 D[i, j] ← min(D[i, j], D[i, k] + D[k, j])
9:     return D
```

All Pairs Shortest Path (Floyd's Algorithm)

Algorithm

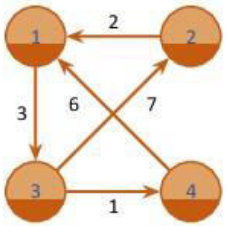
Transitive Closure (Floyd's Algorithm)

```
1: procedure FLOYD(()A[1 . . . n, 1 . . . n])
2:     ▷ Input: Weight matrix A of a graph with no negative length cycles
3:     ▷ Output: Distance matrix of shortest paths
4:     D ← W
5:     for k ← 1 to n do
6:         for i ← 1 to n do
7:             for j ← 1 to n do
8:                 D[i, j] ← min(D[i, j], D[i, k] + D[k, j])
9:     return D
```

- Complexity: $\Theta(n^3)$

All Pairs Shortest Path (Floyd's Algorithm)

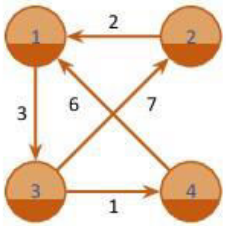
Example

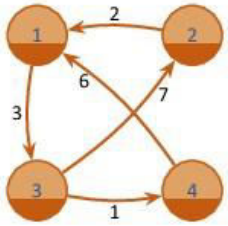


	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0

All Pairs Shortest Path (Floyd's Algorithm)

Example



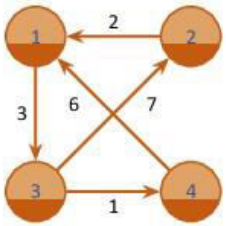


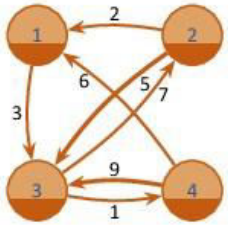
	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0

	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0

All Pairs Shortest Path (Floyd's Algorithm)

Example



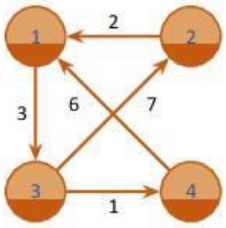


	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0

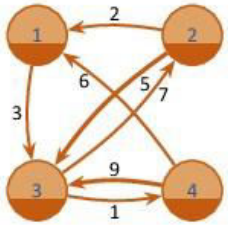
	1	2	3	4
1	0	∞	3	∞
2	2	0	**5**	∞
3	∞	7	0	1
4	6	∞	**9**	0

All Pairs Shortest Path (Floyd's Algorithm)

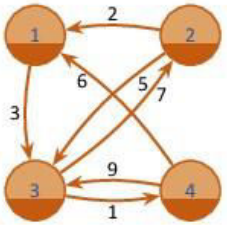
Example



	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0



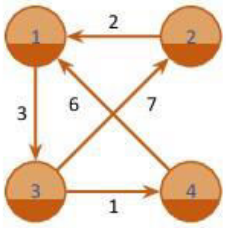
	1	2	3	4
1	0	∞	3	∞
2	2	0	**5**	∞
3	∞	7	0	1
4	6	∞	**9**	0



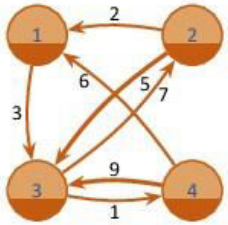
	1	2	3	4
1	0	∞	3	∞
2	2	0	5	∞
3	∞	7	0	1
4	6	∞	9	0

All Pairs Shortest Path (Floyd's Algorithm)

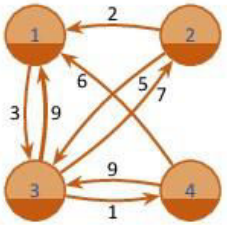
Example



	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0



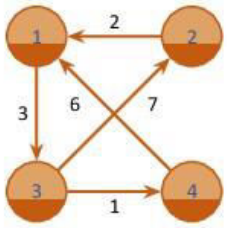
	1	2	3	4
1	0	∞	3	∞
2	2	0	**5**	∞
3	∞	7	0	1
4	6	∞	**9**	0



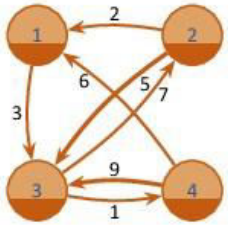
	1	2	3	4
1	0	∞	3	∞
2	2	0	5	∞
3	**9**	7	0	1
4	6	∞	9	0

All Pairs Shortest Path (Floyd's Algorithm)

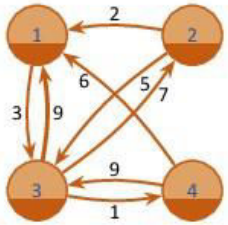
Example



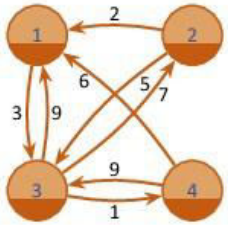
	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0



	1	2	3	4
1	0	∞	3	∞
2	2	0	**5**	∞
3	∞	7	0	1
4	6	∞	**9**	0



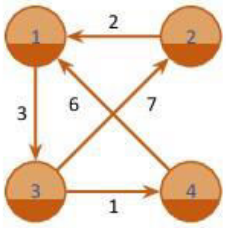
	1	2	3	4
1	0	∞	3	∞
2	2	0	5	∞
3	**9**	7	0	1
4	6	∞	9	0



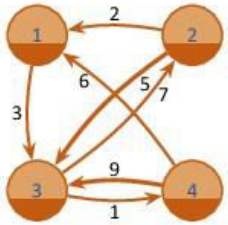
	1	2	3	4
1	0	∞	3	∞
2	2	0	5	∞
3	9	7	0	1
4	6	∞	9	0

All Pairs Shortest Path (Floyd's Algorithm)

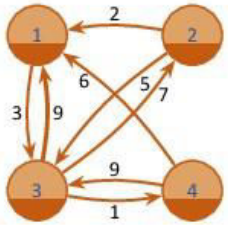
Example



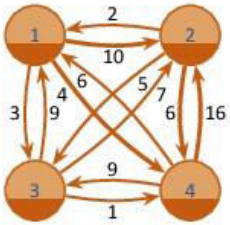
	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0



	1	2	3	4
1	0	∞	3	∞
2	2	0	**5**	∞
3	∞	7	0	1
4	6	∞	**9**	0



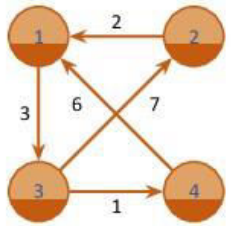
	1	2	3	4
1	0	∞	3	∞
2	2	0	5	∞
3	**9**	7	0	1
4	6	∞	9	0



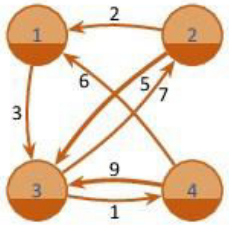
	1	2	3	4
1	0	**10**	3	**4**
2	2	0	5	**6**
3	9	7	0	1
4	6	**16**	9	0

All Pairs Shortest Path (Floyd's Algorithm)

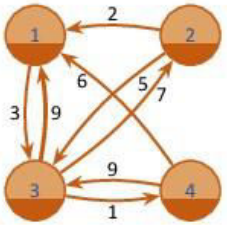
Example



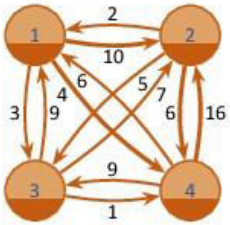
	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0



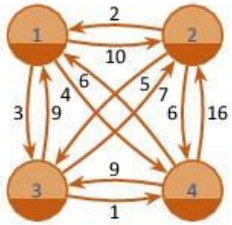
	1	2	3	4
1	0	∞	3	∞
2	2	0	**5**	∞
3	∞	7	0	1
4	6	∞	**9**	0



	1	2	3	4
1	0	∞	3	∞
2	2	0	5	∞
3	**9**	7	0	1
4	6	∞	9	0



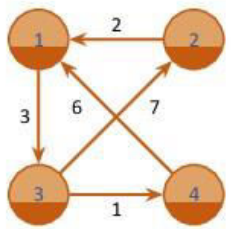
	1	2	3	4
1	0	**10**	3	**4**
2	2	0	5	**6**
3	9	7	0	1
4	6	**16**	9	0



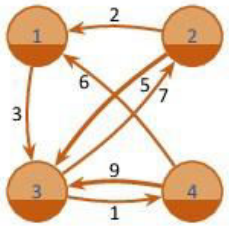
	1	2	3	4
1	0	10	3	4
2	2	0	5	6
3	9	7	0	1
4	6	16	9	0

All Pairs Shortest Path (Floyd's Algorithm)

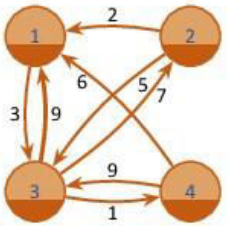
Example



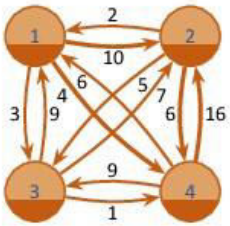
	1	2	3	4
1	0	∞	3	∞
2	2	0	∞	∞
3	∞	7	0	1
4	6	∞	∞	0



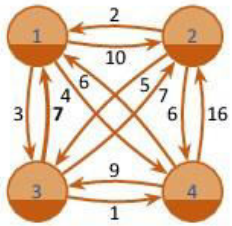
	1	2	3	4
1	0	∞	3	∞
2	2	0	**5**	∞
3	∞	7	0	1
4	6	∞	**9**	0



	1	2	3	4
1	0	∞	3	∞
2	2	0	5	∞
3	**9**	7	0	1
4	6	∞	9	0



	1	2	3	4
1	0	**10**	3	**4**
2	2	0	5	**6**
3	9	7	0	1
4	6	**16**	9	0



	1	2	3	4
1	0	10	3	4
2	2	0	5	6
3	**7**	7	0	1
4	6	16	9	0

All Pairs Shortest Path (Floyd's Algorithm)

Think About It

- Give an example of a graph with negative weights for which Floyd's algorithm does not yield the correct result
- Enhance Floyd's algorithm so that shortest paths themselves, not just their lengths, can be found

Design and Analysis of Algorithms

Lower-Bound Arguments

Reetinder Sidhu

Department of Computer Science and Engineering

Design and Analysis of Algorithms

Lower-Bound Arguments

Reetinder Sidhu

Department of Computer Science and Engineering

Lower-Bound Arguments

Unit 5: Limitations of Algorithmic Power and Coping with the Limitations

- Dynamic Programming
  - Computing a Binomial Coefficient
  - The Knapsack Problem
  - Memory Functions
  - Warshall's and Floyd's Algorithms
  - Optimal Binary Search Trees
- Limitations of Algorithmic Power
  - **Lower-Bound Arguments**
  - Decision Trees
  - P, NP, and NP-Complete, NP-Hard Problems
- Coping with the Limitations
  - Backtracking
  - Branch-and-Bound

Concepts covered

- Lower-Bound Arguments
  - Trivial lower bounds
  - Adversary arguments
  - Problem reduction

Lower-Bound Arguments
Limitations of Algorithmic Power

- There are no algorithms to solve some problems
  - Ex: halting problem
- Other problems can be solved algorithmically, but not in polynomial time
  - Ex: traveling salesman problem
- For polynomial time algorithms also, there are lower bounds

Lower-Bound Arguments

Definition

Lower Bound

An estimate on a minimum amount of work needed to solve a given problem (estimate can be less than the minimum amount of work but not greater)

- Examples:
  - number of comparisons needed to find the largest element in a set of n numbers
  - number of comparisons needed to sort an array of size n
  - number of comparisons necessary for searching in a sorted array
  - number of multiplications needed to multiply two $n \times n$ matrices

Lower-Bound Arguments
Bound Tightness

- A lower bound can be:
  - an exact count
  - an efficiency class (Ω)

Tight Lower Bound

There exists an algorithm with the same efficiency as the lower bound

Problem	Lower Bound	Tightness
Sorting	$\Omega(n \log n)$	yes
Searching a sorted array	$\Omega(\log n)$	yes
Element uniqueness	$\Omega(n \log n)$	yes
Integer multiplication ($n \times n$)	$\Omega(n)$	unknown
Matrix multiplication ($n \times n$)	$\Omega(n^2)$	unknown

Lower-Bound Arguments
Methods for Establishing Lower Bounds

- Trivial lower bounds
- Information-theoretic arguments (decision trees)
- Adversary arguments
- Problem reduction

Lower-Bound Arguments

Trivial Lower Bounds

Trivial Lower Bounds

Based on counting the number of items that must be processed in input and generated as output

- Examples
  - finding max element
  - polynomial evaluation
  - sorting
  - element uniqueness
  - Hamiltonian circuit existence
- Conclusions
  - may and may not be useful
  - be careful in deciding how many elements must be processed

Lower-Bound Arguments
Adversary Arguments

Adversary Argument

A method of proving a lower bound by playing role of adversary that makes algorithm work the hardest by adjusting input

- Example 1: "Guessing" a number between 1 and n with yes/no questions
  - Adversary: Puts the number in a larger of the two subsets generated by last question
- Example 2: Merging two sorted lists of size n $a_1 < a_2 < \ldots < a_n$ and $b_1 < b_2 < \ldots < b_n$
  - Adversary: $a_i < b_j$ iff $i < j$
    Output $b_1 < a_1 < b_2 < a_2 < \ldots < b_n < a_n$ requires $2n - 1$ comparisons of adjacent elements

Lower-Bound Arguments

Problem Reduction

- Basic idea: If problem P is at least as hard as problem Q, then a lower bound for Q is also a lower bound for P
- Hence, find problem Q with a known lower bound that can be reduced to problem P in question
- Example: P is finding MST for n points in Cartesian plane Q is element uniqueness problem (known to be in $\Omega(n \log n)$)

Lower-Bound Arguments

Think About It

- Prove that the classic recursive algorithm for the Tower of Hanoi puzzle makes the minimum number of disk moves
- Find a trivial lower-bound class and indicate if the bound is tight:
  - finding the largest element in an array
  - generating all the subsets of an n-element set
  - determining whether *n* given real numbers are all distinct

Design and Analysis of Algorithms

Decision Trees

Reetinder Sidhu

Department of Computer Science and Engineering

Design and Analysis of Algorithms

Decision Trees

Reetinder Sidhu

Department of Computer Science and Engineering

Decision Trees

Unit 5: Limitations of Algorithmic Power and Coping with the Limitations

- Dynamic Programming
  - Computing a Binomial Coefficient
  - The Knapsack Problem
  - Memory Functions
  - Warshall's and Floyd's Algorithms
  - Optimal Binary Search Trees
- Limitations of Algorithmic Power
  - Lower-Bound Arguments
  - **Decision Trees**
  - P, NP, and NP-Complete, NP-Hard Problems
- Coping with the Limitations
  - Backtracking
  - Branch-and-Bound

Concepts covered

- Decision Trees
  - Smallest of three numbers
  - Sorting
  - Searching

Decision Trees
Problem Types: Optimization and Decision

- Optimization problem: find a solution that maximizes or minimizes some objective function
- Decision problem: answer yes/no to a question
- Many problems have decision and optimization versions
  - Ex: traveling salesman problem
  - optimization: find Hamiltonian cycle of minimum length
  - decision: find Hamiltonian cycle of length m
- Decision problems are more convenient for formal investigation of their complexity

Decision Trees

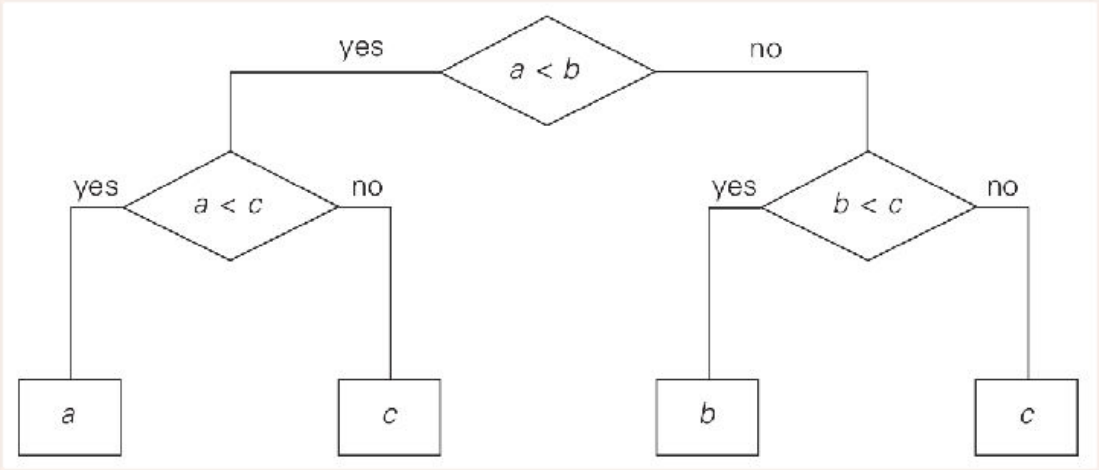
Introduction

- Many important algorithms, especially those for sorting and searching, work by comparing items of their inputs
- We can study the performance of such algorithms with a device called the decision tree

Decision Trees

Example: Decision tree for minimum of three

Decision tree for a determining the minimum of three numbers



Decision Trees

Central Idea

- The central idea behind this model lies in the observation that a tree with a given number of leaves, which is dictated by the number of possible outcomes, has to be tall enough to have that many leaves
- Specifically, it is not difficult to prove that for any binary tree with leaves and height h

$$h \geq \lceil \log_2 l \rceil$$

- A binary tree of height h with the largest number of leaves has all its leaves on the last level
- Hence, the largest number of leaves in such a tree is 2^h
- In other words, $2^h \geq l$ which implies $h \geq \lceil \log_2 l \rceil$

Decision Trees

Decision Trees for Sorting Algorithms

- Most sorting algorithms are comparison-based, i.e., they work by comparing elements in a list to be sorted
- By studying properties of binary decision trees, for comparison-based sorting algorithms, we can derive important lower bounds on time efficiencies of such algorithms
- We can interpret an outcome of a sorting algorithm as finding a permutation of the element indices of an input list that puts the list's elements in ascending order
- For example, for the outcome a < c < b obtained by sorting a list a, b, c
- The number of possible outcomes for sorting an arbitrary n-element list is equal to $n!$

Decision Trees for Sorting Algorithms

- The height of a binary decision tree for any comparison-based sorting algorithm and hence the worst -case number of comparisons made by such an algorithm cannot be less than

$$C_{worst}(n) \geq \lceil \log_2 n! \rceil$$

- Using Stirling's formula:

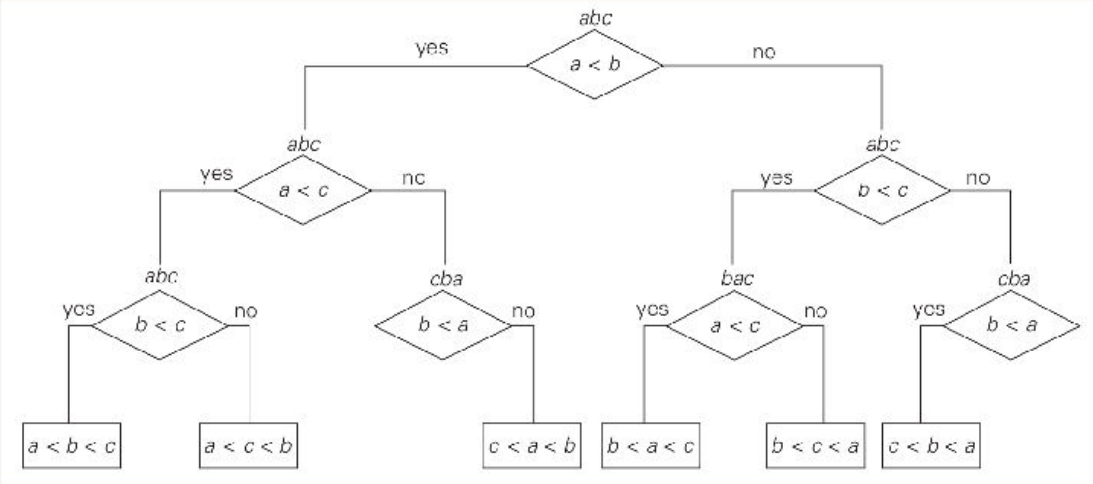
$$\lceil \log_2 n! \rceil \approx \log_2 \sqrt{2\pi n}\,(\frac{n}{e})^n = n\log_2 n - n\log_2 e + \frac{\log_2 n}{2} + \frac{\log_2 \pi}{2} \approx n\log_2 n$$

- About $n log_2 n$ comparisons are necessary to sort an arbitrary n-element list by any comparison-based sorting algorithm

Decision Trees

Decision Trees for Sorting Algorithms

Decision Tree for Three Element Selection Sort



Decision Trees for Sorting Algorithms

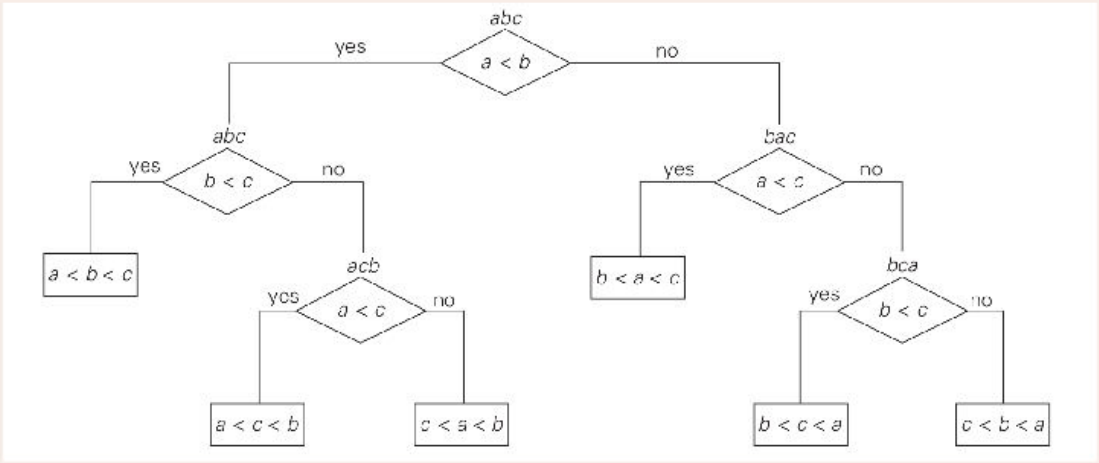
- We can also use decision trees for analyzing the average-case behavior of a comparison based sorting algorithm
- We can compute the average number of comparisons for a particular algorithm as the average depth of its decision tree's leaves, i.e., as the average path length from the root to the leaves
- For example, for the three-element insertion sort this number is:

$$\frac{2+3+3+2+3+3}{6} = 2\frac{2}{3}$$

Decision Trees

Decision Trees for Sorting Algorithms

Decision Tree for Three Element Insertion Sort



Decision Trees for Sorting Algorithms

- Under the standard assumption that all n! outcomes of sorting are equally likely, the following lower bound on the average number of comparisons Cavg made by any comparison-based algorithm in sorting an n-element list has been proved

$$C_{avg}(n) \geq \log_2 n!$$

Decision Trees for Searching Algorithms

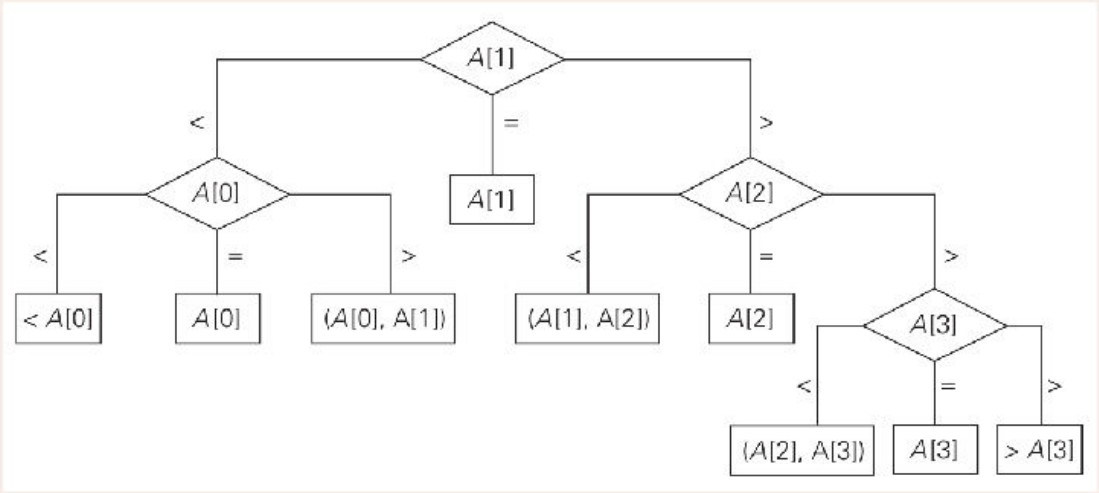
- Decision trees can be used for establishing lower bounds on the number of key comparisons in searching a sorted array of n keys: $A[O] < A[1] < \ldots < A[n-1]$
- The number of comparisons made by binary search in the worst case:

$$C^{bs}_{worst}(n) = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2 (n+1) \rceil$$

Decision Trees

Decision Trees for Searching Algorithms

Ternary Decision Tree for Four Element Array Binary Search



Decision Trees for Searching Algorithms

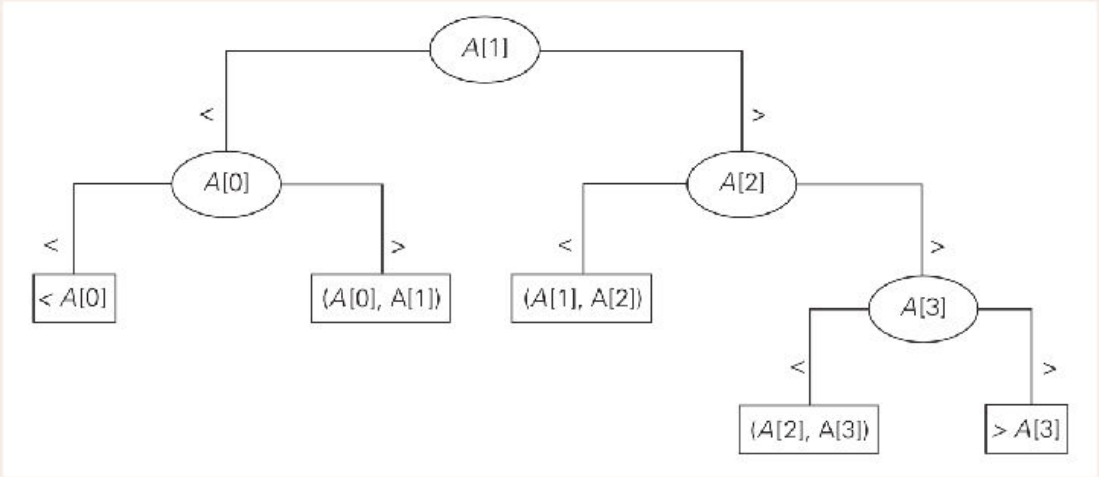
- For an array of n elements, all such decision trees will have 2n +1 leaves (n for successful searches and n + 1 for unsuccessful ones)
- Since the minimum height h of a ternary tree with l leaves is floor(log3l), we get the following lower bound on the number of worst-case comparisons:

$$C_{worst}(n) \geq \lceil \log_3(2n+1) \rceil$$

- This lower bound is smaller than $\lceil \log_2(n+1) \rceil$, the number of worst-case comparisons for binary search
- Can we prove a better lower bound, or is binary search far from being optimal?

DECISION TREES
Decision Trees for Searching Algorithms

Binary Decision Tree for Four Element Array Binary Search



- The binary decision tree is simply the ternary decision tree with all the middle subtrees eliminated

$$C_{worst}(n) \geq \lceil \log_2(n+1) \rceil$$

Design and Analysis of Algorithms

P, NP, and NP-Complete Problems

Reetinder Sidhu

Department of Computer Science and Engineering

Design and Analysis of Algorithms

P, NP, and NP-Complete Problems

Reetinder Sidhu

Department of Computer Science and Engineering

P, NP, and NP-Complete Problems

Unit 5: Limitations of Algorithmic Power and Coping with the Limitations

- Dynamic Programming
  - Computing a Binomial Coefficient
  - The Knapsack Problem
  - Memory Functions
  - Warshall's and Floyd's Algorithms
  - Optimal Binary Search Trees
- Limitations of Algorithmic Power
  - Lower-Bound Arguments
  - Decision Trees
  - **P, NP, and NP-Complete, NP-Hard Problems**
- Coping with the Limitations
  - Backtracking
  - Branch-and-Bound

Concepts covered

- Decision Trees
  - Smallest of three numbers
  - Sorting
  - Searching

P, NP, and NP-Complete Problems

Classifying Problem Complexity

- Is the problem tractable, i.e., is there a polynomial-time ($O(p(n))$ algorithm that solves it?
- Possible answers:
  - yes
  - no
    - because it's been proved that no algorithm exists at all (e.g., Turing's halting problem)
    - because it's been be proved that any algorithm takes exponential time
- unknown

P, NP, and NP-Complete Problems

Problem Types: Optimization and Decision

- Optimization problem: find a solution that maximizes or minimizes some objective function
- Decision problem: answer yes/no to a question

- Many problems have decision and optimization versions
- Example: traveling salesman problem
  - optimization: find Hamiltonian cycle of minimum length
  - decision: find Hamiltonian cycle of length $\leq m$
- Decision problems are more convenient for formal investigation of their complexity

P, NP, and NP-Complete Problems

Class P

Class P (Polynomial

The class of decision problems that are solvable in $O(p(n))$ time, where $p(n)$ is a polynomial of problem's input size n

- searching
- element uniqueness
- graph connectivity
- graph acyclicity
- primality testing

P, NP, and NP-Complete Problems

Class NP

Class NP (Nondeterministic Polynomial)

class of decision problems whose proposed solutions can be verified in polynomial time = solvable by a nondeterministic polynomial algorithm

- A nondeterministic polynomial algorithm is an abstract two-stage procedure that:
  - generates a random string purported to solve the problem checks
  - whether this solution is correct in polynomial time
- By definition, it solves the problem if it's capable of generating and verifying a solution on one of its tries
- Why this definition?
  - led to development of the rich theory called "computational complexity"

P, NP, and NP-Complete Problems

Example: CNF satisfiability

Boolean Satisfiability (CNF)

Is a Boolean expression in its conjunctive normal form (CNF) satisfiable, i.e., are there values of its variables that makes it true?

- This problem is in NP. Nondeterministic algorithm:
  - Guess truth assignment
  - Substitute the values into the CNF formula to see if it evaluates to true
- Example: Consider the Boolean expression in CNF form:

$$(a + \overline{b} + \overline{c})(\overline{a} + b)(\overline{a} + \overline{b} + \overline{c})$$

- Can values *false* and *true* (or 0 and 1) be assigned to a, b and c such that above expression evaluates to 1?
- $a = 1, b = 1, c = 0$
- Checking phase: $\Theta(n)$

P, NP, and NP-Complete Problems

What problems are in NP?

- Hamiltonian circuit existence
- Partition problem: Is it possible to partition a set of n integers into two disjoint subsets with the same sum?
- Decision versions of TSP, knapsack problem, graph coloring, and many other combinatorial optimization problems. (Few exceptions include: MST, shortest paths)
- All the problems in P can also be solved in this manner (but no guessing is necessary), so we have:

$$P \subseteq NP$$

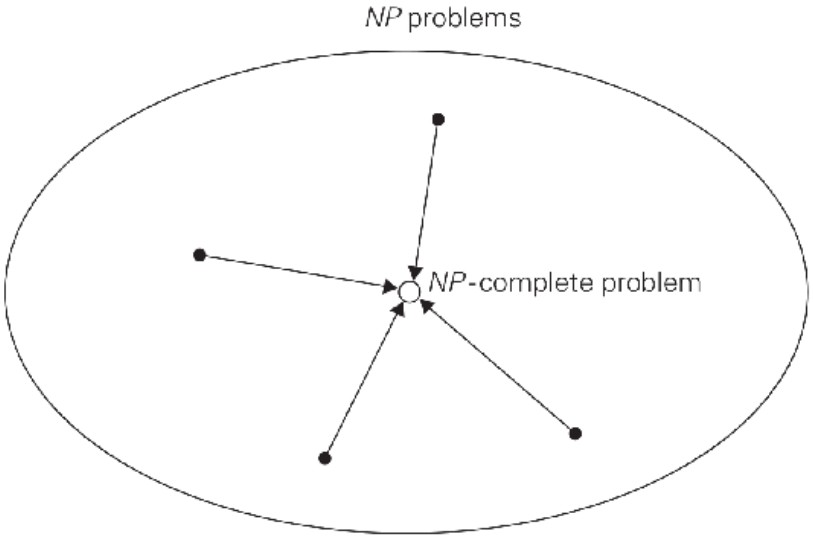
- Big question:

$$P = NP\ ?$$

P, NP, and NP-Complete Problems

NP-Complete Problems

- A decision problem *D* is *NP*-complete if it's as hard as any problem in *NP*, i.e.,
  - *D* is in *NP*
  - every problem in *NP* is polynomial-time reducible to *D*

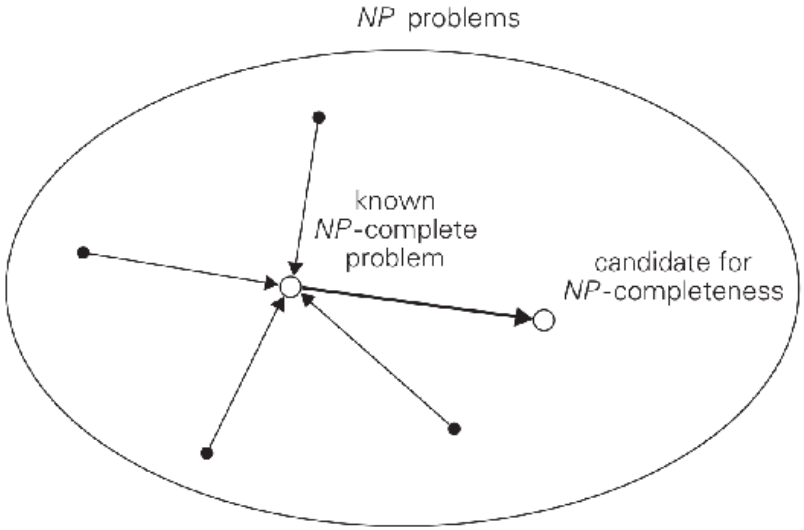


  - Cook's theorem (1971): CNF-sat is NP-complete

P, NP, and NP-Complete Problems

NP-Complete Problems

- Other NP-complete problems obtained through polynomial- time reductions from a known NP-complete problem

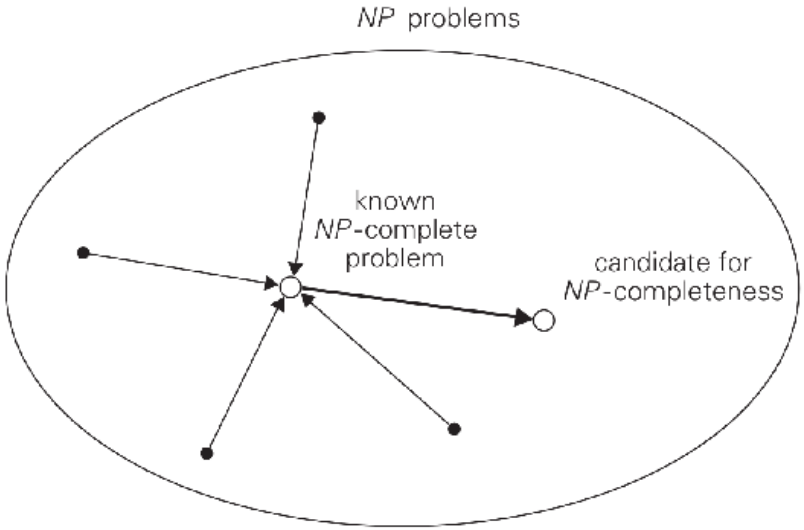


- Examples: TSP, knapsack, partition, graph-coloring and hundreds of other problems of combinatorial nature

P, NP, and NP-Complete Problems

P = NP ? Dilemma Revisited

- $P = NP$ would imply that every problem in NP, including all NP-complete problems, could be solved in polynomial time
- If a polynomial-time algorithm for just one NP-complete problem is discovered, then every problem in NP can be solved in polynomial time, i.e., $P = NP$



- Most but not all researchers believe that $P \neq NP$

Design and Analysis of Algorithms

Backtracking

Reetinder Sidhu

Department of Computer Science and Engineering

Design and Analysis of Algorithms

Backtracking

Reetinder Sidhu

Department of Computer Science and Engineering

BACKTRACKING

UNIT 5: Limitations of Algorithmic Power and Coping with the Limitations

- Dynamic Programming
  - Computing a Binomial Coefficient
  - The Knapsack Problem
  - Memory Functions
  - Warshall's and Floyd's Algorithms
  - Optimal Binary Search Trees
- Limitations of Algorithmic Power
  - Lower-Bound Arguments
  - Decision Trees
  - P, NP, and NP-Complete, NP-Hard Problems
- Coping with the Limitations
  - **Backtracking**
  - Branch-and-Bound

Concepts covered

- Backtracking
  - Introduction
  - *N* Queens
  - Hamiltonian Circuit
  - Subset Sum
  - Algorithm

BACKTRACKING
Tackling Difficult Combinatorial Problems

- There are two principal approaches to tackling difficult combinatorial problems (NP-hard problems):
  - Use a strategy that guarantees solving the problem exactly but doesn't guarantee to find a solution in polynomial time
  - Use an approximation algorithm that can find an approximate (sub-optimal) solution in polynomial time

BACKTRACKING
Exact Solution Strategies

- Exhaustive search (brute force)
  - useful only for small instances
- Dynamic programming
  - applicable to some problems (e.g., the knapsack problem)
- Backtracking
  - eliminates some unnecessary cases from consideration
  - yields solutions in reasonable time for many instances but worst case is still exponential
- Branch-and-bound
  - further refines the backtracking idea for optimization problems

BACKTRACKING

Introduction

- Construct the *state-space tree*
  - nodes: partial solutions
  - edges: choices in extending partial solutions
- Explore the state space tree using depth-first search
- "Prune" *nonpromising nodes*
  - DFS stops exploring subtrees rooted at nodes that cannot lead to a solution and backtracks to such a node's parent to continue the search

BACKTRACKING
Algorithm

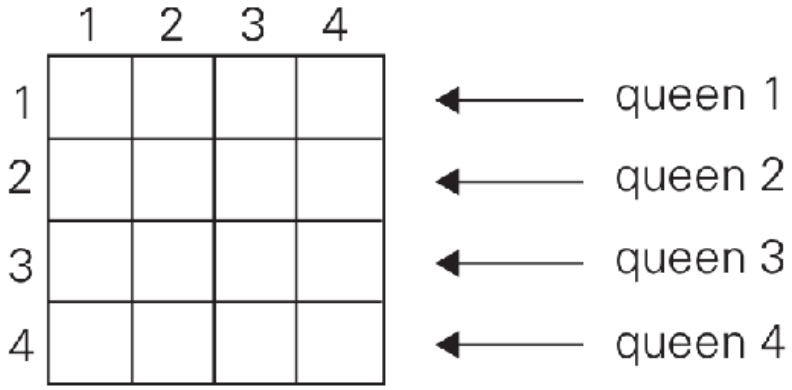
Backtrack Algorithm

```
1: procedure BACKTRACK(X[1 . . . i])
2:     ▷ Input: X[1 . . . i] specifies first i promising components of a solution
3:     ▷ Output: All the tuples representing the problem's solutions
4:     if X[1 . . . i] is a solution then
5:         write X[1 . . . i]
6:     else
7:         for each element x ∈ S_{i+1} consistent with X[1 . . . i] and the constraints do
8:             X[i + 1] ← x
9:             Backtrack (X[1 . . . i + 1])
```

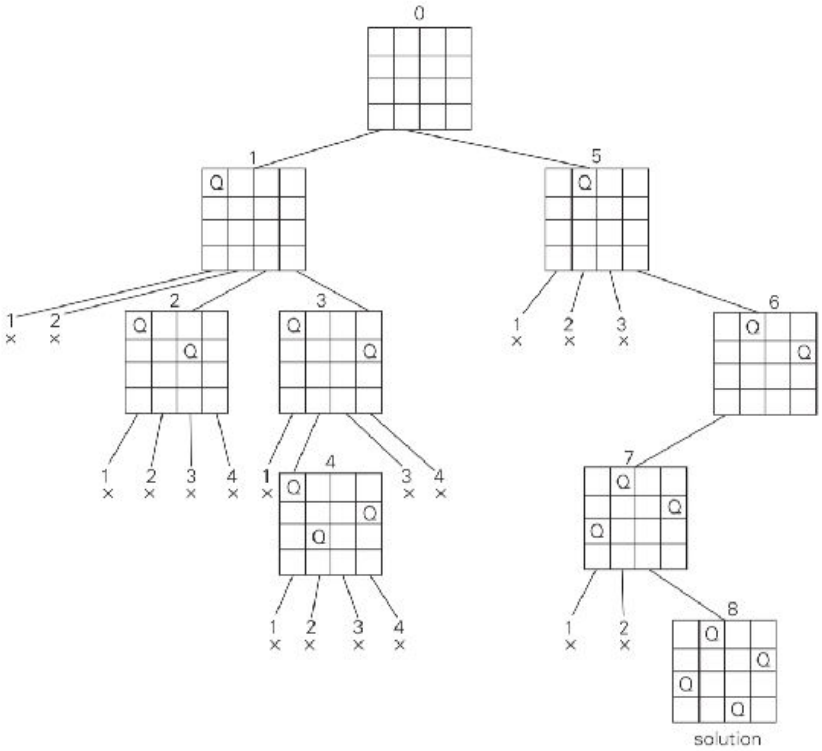
- Output: n-tuples $(x_1, x_2, \ldots, x_n)$
- Each $x_i \in S_i$, some finite linearly ordered set

BACKTRACKING
Example: *N*-Queens Problem

- Place N queens on an $N \times N$ chess board so that no two of them are in the same row, column, or diagonal



BACKTRACKING
State-Space Tree of the 4-Queens Problem



```
1   Algorithm Place(k, i)
2   // Returns true if a queen can be placed in kth row and
3   // ith column. Otherwise it returns false. x[ ] is a
4   // global array whose first (k − 1) values have been set.
5   // Abs(r) returns the absolute value of r.
6   {
7       for j := 1 to k − 1 do
8           if ((x[j] = i) // Two in the same column
9                 or (Abs(x[j] − i) = Abs(j − k)))
10                    // or in the same diagonal
11                then return false;
12      return true;
13  }
```

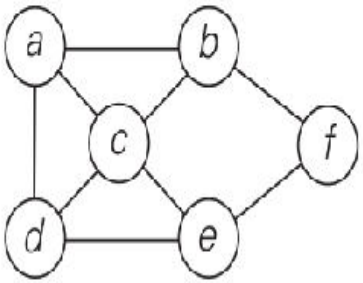
```
1   Algorithm NQueens(k, n)
2   // Using backtracking, this procedure prints all
3   // possible placements of n queens on an n × n
4   // chessboard so that they are nonattacking.
5   {
6       for i := 1 to n do
7       {
8           if Place(k, i) then
9           {
10              x[k] := i;
11              if (k = n) then write (x[1 : n]);
12              else NQueens(k + 1, n);
13          }
14      }
15  }
```

BACKTRACKING
Example: Hamiltonian Circuit Problem

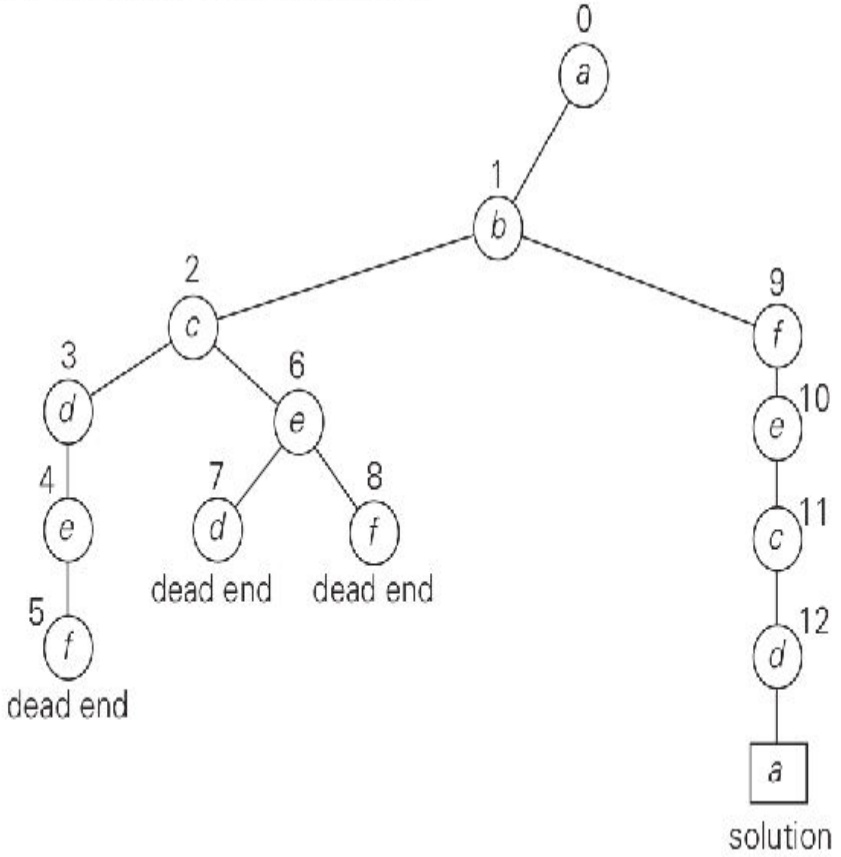
Hamiltonian Circuit

A Hamiltonian circuit is defined as a cycle that passes through all the vertices of the graph exactly once.

- Example graph:



- State-space tree for finding a Hamiltonian circuit (numbers above the nodes of indicate the order in which the nodes are generated):

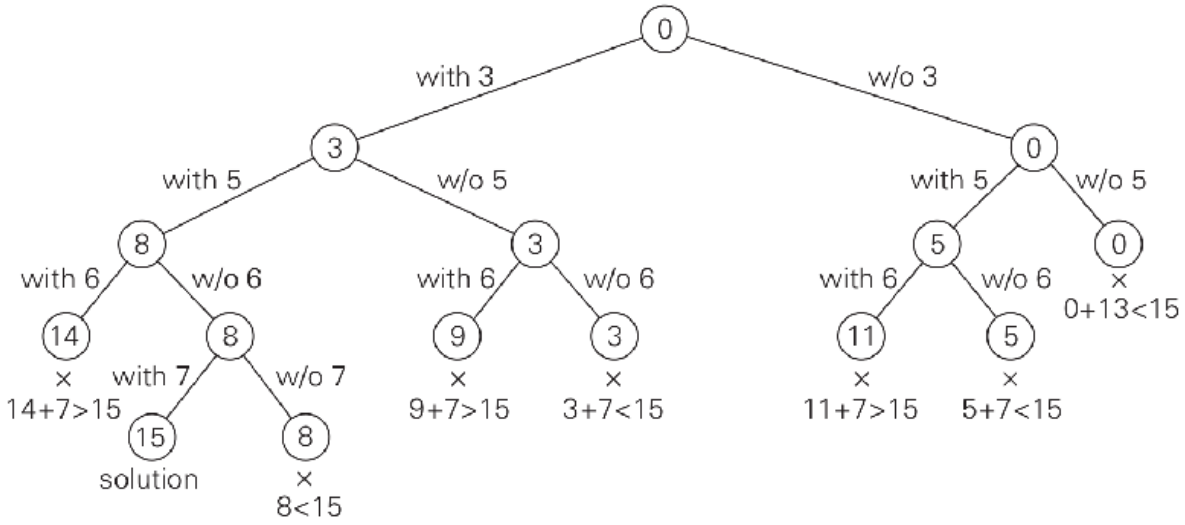


BACKTRACKING
Example: Subset Sum Problem

Subset Sum Problem

Given set $A = \{a_1, \ldots, a_n\}$ of n positive integers, find a subset whose sum is equal to a given positive integer d

- State space tree for $A = \{3, 5, 6, 7\}$ and $d = 15$ (number in each node is the sum so far):



BACKTRACKING

Think About It

- Continue the backtracking search for a solution to the four-queens problem, to find the second solution to the problem
- Explain how the board's symmetry can be used to find the second solution to the four-queens problem

Design and Analysis of Algorithms

Branch and Bound

Reetinder Sidhu

Department of Computer Science and Engineering

Design and Analysis of Algorithms

Branch and Bound

Reetinder Sidhu

Department of Computer Science and Engineering

Branch and Bound

Unit 5: Limitations of Algorithmic Power and Coping with the Limitations

- Dynamic Programming
  - Computing a Binomial Coefficient
  - The Knapsack Problem
  - Memory Functions
  - Warshall's and Floyd's Algorithms
  - Optimal Binary Search Trees
- Limitations of Algorithmic Power
  - Lower-Bound Arguments
  - Decision Trees
  - P, NP, and NP-Complete, NP-Hard Problems
- Coping with the Limitations
  - Backtracking
  - **Branch and Bound**

Concepts covered

- Backtracking
  - General Approach
  - Knapsack Problem
  - Assignment Problem
  - Travelling Salesman Problem

Branch and Bound

Introduction

- An enhancement of backtracking
- Applicable to optimization problems
- For each node (partial solution) of a state-space tree, computes a bound on the value of the objective function for all descendants of the node (extensions of the partial solution)
- Uses the bound for:
  - ruling out certain nodes as "nonpromising" to prune the tree (if a node's bound is not better than the best solution seen so far)
  - guiding the search through state-space

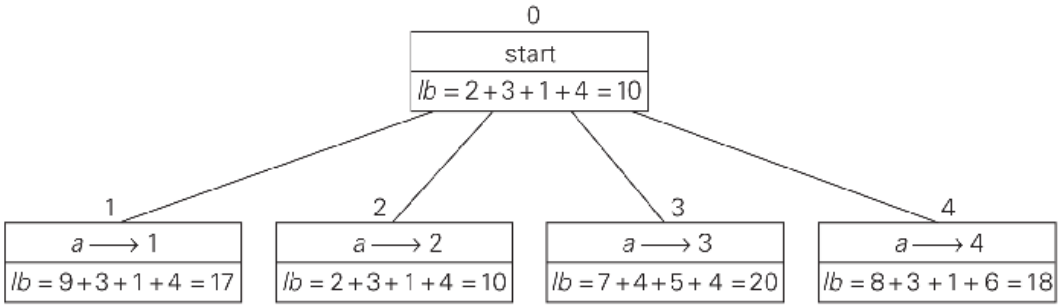
Branch and Bound
Example: Assignment Problem

- Select one element in each row of the cost matrix C so that:
  - no two selected elements are in the same column
  - the sum is minimized
- Example

	Job 1	Job 2	Job 3	Job 4
Person *a*	9	2	7	8
Person *b*	6	4	3	7
Person *c*	5	8	1	8
Person *d*	7	6	9	4

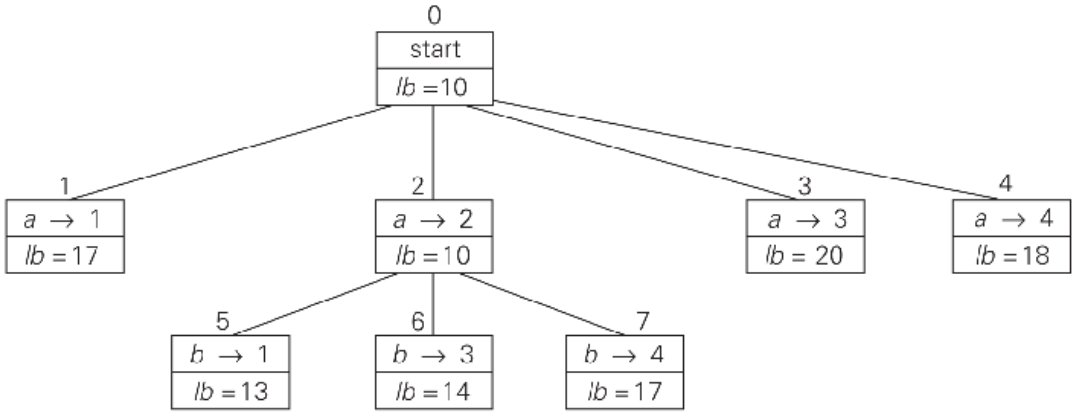
- Lower bound (sum of smallest elements in each row): $2 + 3 + 1 + 4 = 10$
- Best-first branch-and-bound variation: Generate all the children of the most promising node

Branch and Bound
Example: First two levels of the state-space

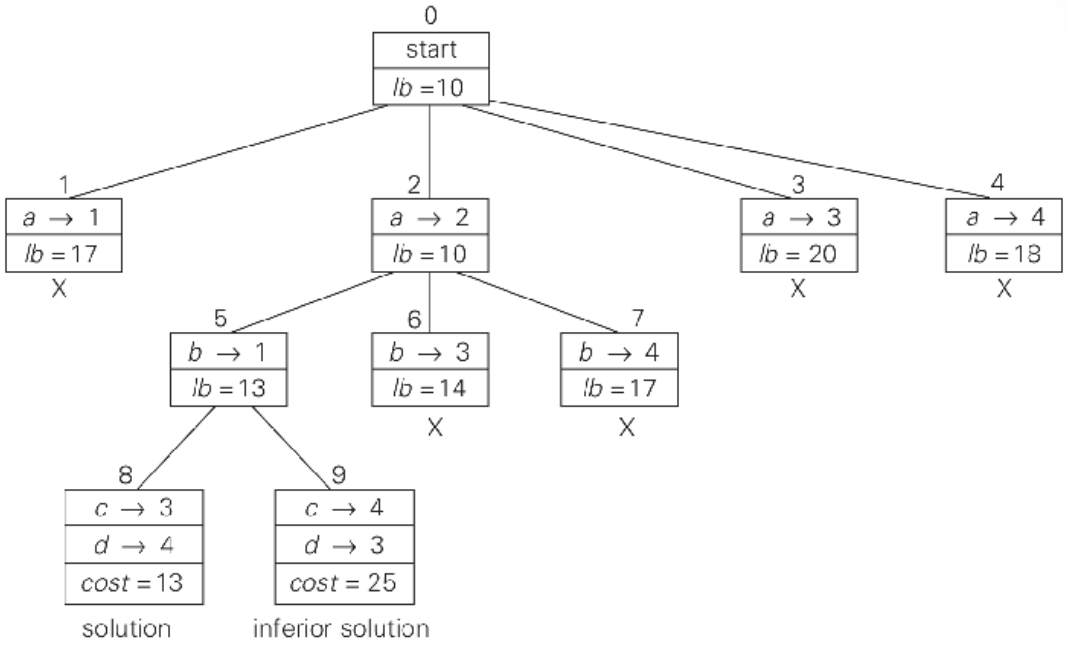


Branch and Bound

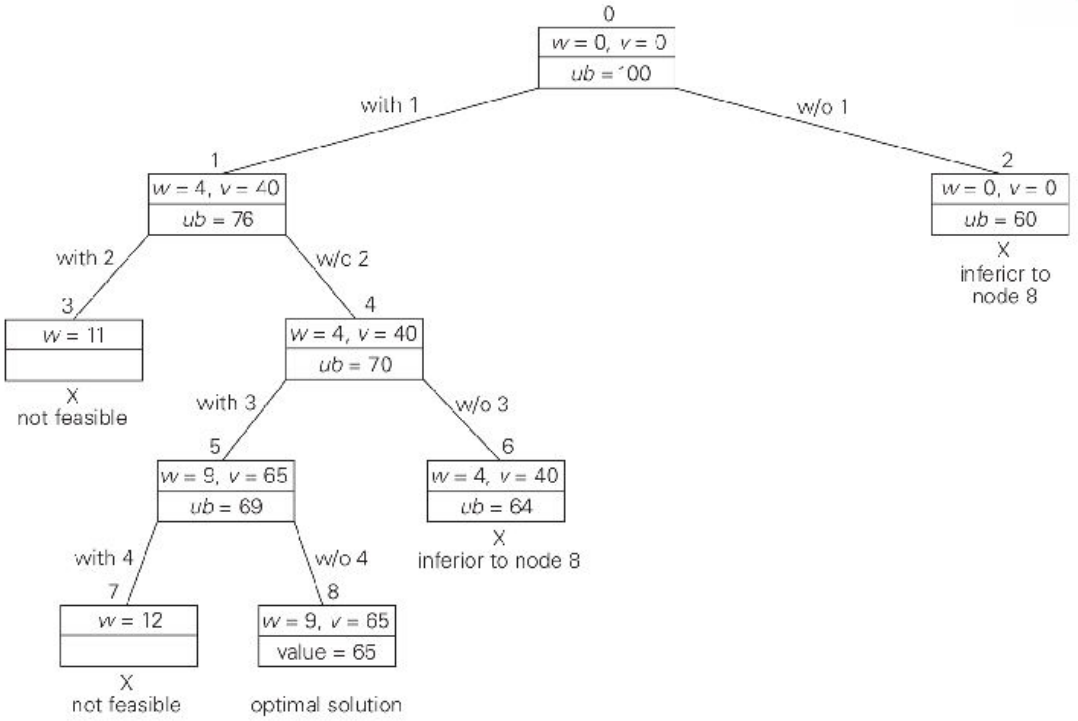
Example: First three levels of the state-space



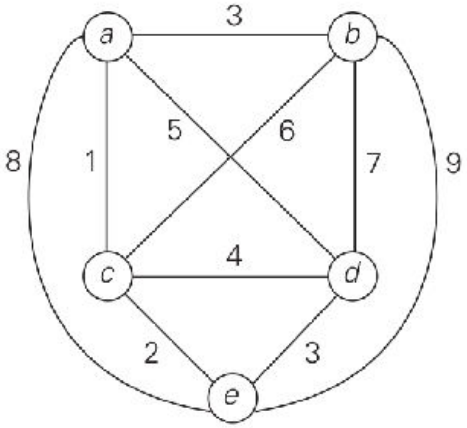
Branch and Bound
Example: Complete state-space tree

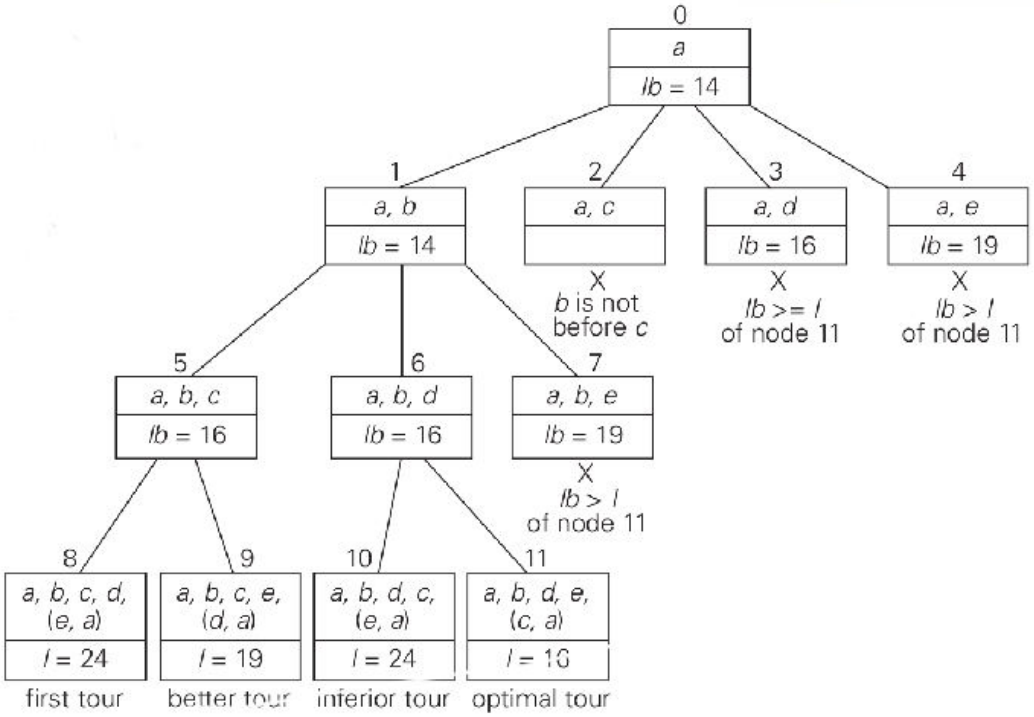


Branch and Bound
Example: Knapsack Problem



Branch and Bound
Example: Traveling Salesman Problem





Branch and Bound

Think About It

- What data structure would you use to keep track of live nodes in a best-first branch-and-bound algorithm?
- Solve the assignment problem by the best-first branch-and-bound algorithm with the bounding function based on matrix columns rather than rows